

TÀI LIỆU HỌC TẬP MÔN TRÍ TUỆ NHÂN TẠO

Giới thiệu Tài liệu học tập: Trí tuệ Nhân tạo - Lý thuyết và Ứng dụng

Chào mừng các bạn đến với môn học Trí tuệ Nhân tạo (AI). Tài liệu này được biên soạn nhằm cung cấp một hành trình khám phá toàn diện, đi từ những nguyên lý nền tảng nhất đến các kỹ thuật tiên tiến đang định hình thế giới của chúng ta.

Mục tiêu của môn học không chỉ dừng lại ở việc giới thiệu các thuật toán, mà còn giúp bạn xây dựng một tư duy có hệ thống về cách các "tác tử thông minh" (intelligent agents) có thể nhận thức, lập luận, học hỏi và ra quyết định trong các môi trường phức tạp. Chúng ta sẽ bắt đầu với việc định nghĩa AI và các khái niệm cốt lõi, trước khi đi sâu vào các phương pháp tìm kiếm kinh điển – xương sống của nhiều bài toán AI.

Tiếp theo, tài liệu sẽ đưa bạn đến với thế giới của sự bất định thông qua lý thuyết xác suất và các mô hình học máy, nền tảng cho các hệ thống có khả năng dự đoán và học hỏi từ dữ liệu. Cuối cùng, chúng ta sẽ khám phá các lĩnh vực hiện đại và đầy cảm hứng như học tăng cường, xử lý ngôn ngữ tự nhiên và thị giác máy tính, nơi AI đang tạo ra những bước đột phá ngoạn mục.

Với cách tiếp cận cân bằng giữa lý thuyết hàn lâm và các ví dụ ứng dụng trực quan, tài liệu này sẽ là một người bạn đồng hành tin cậy, giúp bạn xây dựng một nền tảng kiến thức vững chắc và khơi nguồn cảm hứng để tiếp tục khám phá và sáng tạo trong lĩnh vực đầy tiềm năng này.

Tóm tắt Nội dung các Chương

Dưới đây là tóm tắt các phần và chương chính có trong tài liệu:

Chương 1: Giới thiệu Tổng quan về Trí tuệ Nhân tạo

Chương mở đầu này đặt ra câu hỏi nền tảng "Trí tuệ nhân tạo là gì?" và giới thiệu các khái niệm cốt lõi sẽ được sử dụng trong suốt môn học.

- **Các định nghĩa về AI:** Trình bày bốn cách tiếp cận chính để định nghĩa AI, bao gồm: Suy nghĩ như con người (Thinking Humanly), Hành động như con người (Acting Humanly), Suy nghĩ hợp lý (Thinking Rationally), và Hành động hợp lý (Acting Rationally).
- **Mối quan hệ giữa AI, ML, và DL:** Sử dụng sơ đồ Venn để làm rõ mối quan hệ phân cấp, trong đó Học máy (Machine Learning) là một tập con của AI, và Học sâu (Deep Learning) là một tập con chuyên biệt của Học máy.
- **Nền tảng của AI:** Giới thiệu các lĩnh vực đã đóng góp vào sự hình thành của AI như triết học, toán học, kinh tế học, khoa học thần kinh, và khoa học máy tính.

- **Tác tử Thông minh (Intelligent Agents):** Đưa ra khái niệm tác tử làm chủ đề trung tâm để nghiên cứu AI. Một tác tử là một thực thể có khả năng cảm nhận môi trường thông qua các cảm biến và hành động lên môi trường đó thông qua các cơ cấu chấp hành.

Chương 2: Giải quyết Vấn đề bằng Tìm kiếm

Chương này đặt nền móng cho việc giải quyết bài toán trong AI bằng cách coi chúng như các bài toán tìm kiếm trong không gian trạng thái.

- **Phần 2.1-2.4:** Giới thiệu các **thuật toán tìm kiếm mù (uninformed search)** như Tìm kiếm theo chiều rộng (BFS), Tìm kiếm theo chiều sâu (DFS), và Tìm kiếm chi phí đồng nhất (UCS). Các phương pháp này chỉ dựa vào cấu trúc của bài toán mà không có thêm thông tin phụ.
- **Phần 2.5:** Trình bày các **thuật toán tìm kiếm có thông tin (informed/heuristic search)**. Trọng tâm là **Thuật toán Tham lam (Greedy Best-First Search)** và **Thuật toán A***, sử dụng các hàm heuristic để ước tính chi phí đến đích, giúp tìm kiếm hiệu quả hơn.
- **Phần 2.8:** Đi sâu vào **Bài toán Thỏa mãn Ràng buộc (Constraint Satisfaction Problems - CSP)**, một lớp bài toán đặc biệt nơi lời giải phải thỏa mãn một loạt các ràng buộc. Thuật toán **Quay lui (Backtracking)** và các cải tiến của nó được giới thiệu chi tiết.
- **Phần 2.9:** Khám phá **Tìm kiếm Đối kháng (Adversarial Search)**, nền tảng cho AI trong các trò chơi. Các thuật toán **Minimax** và tối ưu hóa **Cắt tỉa Alpha-Beta** được phân tích qua các ví dụ như cờ ca-rô.
- **Phần 2.10:** Giới thiệu **Tìm kiếm Cục bộ và Tối ưu hóa**, bao gồm các thuật toán tiến hóa như **Thuật toán Di truyền (Genetic Algorithm)**, mô phỏng quá trình chọn lọc tự nhiên để tìm kiếm lời giải tối ưu.

Chương 3: Tri thức, Lập luận và Lập kế hoạch

Chương này thực hiện một bước nhảy vọt từ việc "tìm kiếm" sang "hiểu biết", tập trung vào cách biểu diễn và suy luận từ tri thức.

- **Phần 3.1-3.2:** Giới thiệu khái niệm **Cơ sở Tri thức (Knowledge Base)** và ngôn ngữ biểu diễn tri thức đơn giản nhất, **Logic Mệnh đề (Propositional Logic)**. Phần này giải thích cú pháp, ngữ nghĩa và các phương pháp suy diễn như Kiểm tra mô hình và các quy tắc suy luận.
- **Phần 3.3:** Trình bày **Logic Vị từ Bậc nhất (First-Order Logic - FOL)**, một ngôn ngữ mạnh mẽ hơn cho phép mô tả các đối tượng và quan hệ giữa chúng. Các khái niệm như

Lượng từ, Hợp nhất (Unification) và các thuật toán suy diễn như Chuỗi tiến/lùi được giới thiệu.

- **Phần 3.5:** Tập trung vào **Lập luận với sự Bất định (Uncertainty)**. Phần này giới thiệu các khái niệm nền tảng của lý thuyết xác suất và đặc biệt là **Định lý Bayes**, công cụ cốt lõi cho phép các tác tử cập nhật "niềm tin" dựa trên bằng chứng.

Chương 4 & 5: Học máy (Machine Learning)

Hai chương này đi sâu vào các kỹ thuật cho phép máy tính học hỏi từ dữ liệu.

- **Tổng quan:** Giới thiệu các dạng học máy chính: **Học có giám sát (Supervised Learning)**, **Học không giám sát (Unsupervised Learning)** và **Học tăng cường (Reinforcement Learning)**.
- **Các mô hình cụ thể:** Phân tích các thuật toán kinh điển như **Hồi quy Tuyến tính**, **Cây Quyết định**, **Máy Vector Hỗ trợ (SVM)**, và các phương pháp gom cụm.

Chương 6: Học Tăng cường (Reinforcement Learning)

Chương này tập trung vào một lĩnh vực hấp dẫn của AI, nơi các tác tử học cách hành động trong một môi trường thông qua cơ chế "thử và sai" để tối đa hóa phần thưởng.

- **Nền tảng:** Giới thiệu các khái niệm cốt lõi như **Quy trình Quyết định Markov (MDP)**, hàm giá trị, hàm Q.
- **Các thuật toán chính:** Phân tích chi tiết **Q-Learning**, một thuật toán học không cần mô hình (model-free) kinh điển. Sau đó, giới thiệu sự đột phá của **Mạng Q Sâu (Deep Q-Network - DQN)**, kết hợp Q-Learning với mạng nơ-ron sâu để giải quyết các bài toán phức tạp.

Chương 7 & 8: Các Ứng dụng Nâng cao

Hai chương cuối cùng khám phá các lĩnh vực ứng dụng quan trọng và hiện đại của AI.

- **Xử lý Ngôn ngữ Tự nhiên (NLP):** Trình bày các phương pháp biểu diễn văn bản từ cổ điển như **Túi từ (Bag-of-Words)**, **TF-IDF** đến các kỹ thuật nhúng từ hiện đại như **Word2Vec**.
- **Thị giác Máy tính (Computer Vision):** Giới thiệu kiến trúc **Mạng Nơ-ron Tích chập (Convolutional Neural Network - CNN)**, nền tảng cho hầu hết các thành tựu trong nhận dạng và phân tích hình ảnh.

Mục lục

PHẦN 1: NỀN TẢNG TRÍ TUỆ NHÂN TẠO	9
CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN VỀ TRÍ TUỆ NHÂN TẠO	9
1.1. Trí tuệ nhân tạo là gì?.....	9
1.2. Lịch sử hình thành và phát triển của AI	16
ImageNet 2012: Cú hích quyết định đưa Học sâu ra ánh sáng	31
Sự trỗi dậy của Mô hình Ngôn ngữ Lớn và AI Tạo sinh: Khi máy móc biết “suy nghĩ” và “sáng tạo”	32
1.3. Các ứng dụng của Trí tuệ nhân tạo	35
Chẩn đoán Hình ảnh (Medical Image Diagnosis).....	37
Phát triển Thuốc (Drug Discovery).....	38
1.4. Tác tử thông minh (Intelligent Agents).....	44
Tác tử phản xạ dựa trên mô hình (Model-Based Reflex Agents)	54
Tác tử dựa trên mục tiêu (Goal-Based Agents).....	56
Tác tử dựa trên độ hữu dụng (Utility-Based Agents).....	58
1.4.5. Tác tử học hỏi (Learning Agents)	60
CHƯƠNG 2: GIẢI QUYẾT VẤN ĐỀ BẰNG TÌM KIẾM (TUẦN 2–7).....	63
2.1. Bài toán và Không gian Trạng thái (Problem and State Space).....	63
2.4. Tìm kiếm chi phí đồng nhất (Uniform-Cost Search – UCS)	76
2.5. Hàm Heuristic ($h(n)$) : “Kim chỉ nam” cho Tìm kiếm.....	79
2.6. Greedy Best-First Search	80
2.7. Thuật toán A^*	83
BUỔI HỌC 3: TÌM KIẾM CÓ THÔNG TIN (HEURISTIC SEARCH)	85
2.5. Hàm Heuristic ($h(n)$): Kim chỉ nam cho Tìm kiếm	87
2.5.2. Ví dụ về thiết kế hàm Heuristic	89
2.5.3. Các tiêu chí quan trọng của hàm Heuristic	92
2.6. Tìm kiếm Greedy Best-First: "Tham lam" đi về phía trước	93
2.7. Thuật toán A (A-star): Sự kết hợp hoàn hảo	95
2.8. Bài toán thỏa mãn ràng buộc (Constraint Satisfaction Problems - CSPs)	103
2.9. Tìm kiếm đối kháng (Adversarial Search).....	114
2.10. Thuật toán di truyền (Genetic Algorithms - GA).....	123
1. Quần thể (Population)	128
2. Nhiễm sắc thể (Chromosome)	129

3. Gen (Gene).....	129
4. Hàm Thích nghi (Fitness Function)	130
1. Quần thể (Population)	131
2. Nhiễm sắc thể (Chromosome)	131
3. Gen (Gene).....	132
4. Hàm thích nghi (Fitness Function)	133
Bước 1: Khởi tạo (Initialization).....	134
Bước 2: Đánh giá (Evaluation)	134
Bước 3: Vòng lặp Tiến hóa (The Evolutionary Loop).....	134
Bước 4: Thay thế và Điều kiện Dừng (Replacement & Termination)	136
Bước 1: Khởi tạo (Initialization) – Tạo ra “Thế hệ Adam & Eve”	137
Bước 2: Đánh giá (Evaluation) – “Ngày phán xét”	137
Bước 3: Chu trình Tiến hóa (Evolution Loop) – Trại tìm của GA.....	137
Bước 4: Tạo Quần thể Mới và Điều kiện Dừng.....	139
Tổng kết một Thế hệ	142
Mô hình hóa bài toán	143
Ánh xạ sang thuật toán di truyền	143
Chi tiết ví dụ: Giải bằng một thế hệ GA	143
Nhận xét và kết luận	145
CHƯƠNG 3: BIỂU DIỄN VÀ XỬ LÝ TRI THỨC.....	146
Kết luận: Bước nhảy từ “Tác tử Hành động” sang “Tác tử Hiểu biết”	148
Cấu trúc vòng lặp của tác tử dựa trên tri thức.....	151
Tổng quát hóa vòng lặp.....	152
Ý nghĩa của vòng lặp này.....	153
Mô tả tổng quan về Thế giới Wumpus.....	153
Khả năng cảm nhận của tác tử	153
Hành động của tác tử	154
Tri thức và suy luận trong Thế giới Wumpus	154
So sánh với tác tử tìm kiếm thông thường	155
Ý nghĩa giáo dục của ví dụ Wumpus	155
3.2. Logic Mệnh đề (Propositional Logic).....	155
Mệnh đề (Proposition):	155
Tính chất của logic mệnh đề:	156

Mục tiêu khi học logic mệnh đề trong AI:	156
3.2.1. Giới thiệu Logic Mệnh đề (Propositional Logic).....	156
3.1. Các Quy tắc Suy diễn Trực quan và Chaining.....	167
3.2. Hợp giải (Resolution) – Một Quy tắc Toàn năng.....	168
3.3. Thuật toán DPLL: Giải quyết Bài toán SAT hiệu quả	168
4. So sánh các Chiến lược Suy diễn.....	170
5. Kết luận: Lựa chọn Công cụ phù hợp	170
3.3.1. Hạn chế của Logic Mệnh đề (Why Propositional Logic Falls Short).....	170
3.3.2. Cú pháp của Logic Vị từ (First-Order Logic – FOL)	172
3.3.3. Suy diễn trong Logic Vị từ (FOL): Từ Quy tắc đến Lập luận Tổng quát.....	174
3.4.1 Luật sản xuất (Production Rules).....	177
3.4.2. Mạng ngữ nghĩa (Semantic Networks)	179
3.5. Xử lý Tri thức Bất định	181
3.5.2. Nhập môn Lý thuyết Xác suất	183
Suy luận Bayes dưới góc nhìn Quy tắc nhân	187
Ví dụ: Chẩn đoán Viêm màng não	187
Ý nghĩa trong Trí tuệ Nhân tạo	188
Ghi nhớ Quan trọng	189
CHƯƠNG 4: HỌC MÁY	190
4.1. Tổng quan về Học máy (Machine Learning).....	190
4.1.2. Ba loại hình Học máy chính.....	193
4.2.1. Hồi quy tuyến tính (Linear Regression).....	200
4.3. Học không giám sát (Unsupervised Learning)	204
4.4.1. Nguồn cảm hứng từ sinh học	210
4.4.2 Perceptron – nơ-ron đơn giản nhất.....	212
4.4.4. Huấn luyện Mạng Nơ-ron	215
PHẦN 4 · CÁC CHƯƠNG NÂNG CAO VÀ ĐỊNH HƯỚNG DỰ ÁN.....	217
CHƯƠNG 5 · HỌC SÂU (DEEP LEARNING)	217
5.0 Giới thiệu về học sâu	217
5.1. Mạng Nơ-ron Tích chập (Convolutional Neural Networks – CNNs) cho Xử lý ảnh.....	218
5.1.3 Các thành phần kiến trúc cốt lõi của CNN	221
5.1.4. Kiến trúc CNN tiêu biểu	225
5.1.5. Ứng dụng và Hướng dẫn Dự án.....	228

5.2. Mạng Nơ-ron Hồi quy (Recurrent Neural Networks – RNNs) cho Dữ liệu Chuỗi.....	230
5.2.2 Giải pháp của RNN – Vòng lặp và Trí nhớ.....	231
5.2.3 Kiến trúc RNN cơ bản	232
5.2.5. Giải pháp: Long Short-Term Memory (LSTM) và Gated Recurrent Unit (GRU)	235
5.2.6 Ứng dụng và Hướng dẫn Dự án.....	236
5.3.1 Vấn đề với RNN	237
5.3.2 Giải pháp của Transformer – “ <i>Attention Is All You Need</i> ”	238
5.3.4 Sự trỗi dậy của Mô hình Ngôn ngữ Lớn (Large Language Models – LLMs)	240
5.3.5 Ứng dụng và Hướng dự án.....	242
CHƯƠNG 6: XỬ LÝ NGÔN NGỮ TỰ NHIÊN (NATURAL LANGUAGE PROCESSING – NLP).....	245
6.0 Giới thiệu – “Nói chuyện với máy tính”	245
6.1 Vì sao NLP là một bài toán khó?	245
6.1 Biểu diễn văn bản: “Số hóa” ngôn ngữ.....	245
6.1.2 Phương pháp cổ điển 2: TF-IDF (Term Frequency – Inverse Document Frequency).....	249
6.1.3 Phương pháp hiện đại: Nhúng từ (Word Embeddings).....	250
6.2 Các bài toán NLP cơ bản và Hướng dự án.....	252
6.2.2 Nhận dạng Thực thể Có tên (<i>Named Entity Recognition – NER</i>).....	254
CHƯƠNG 7 HỌC TĂNG CƯỜNG (REINFORCEMENT LEARNING).....	257
7.1 Quá trình quyết định Markov (Markov Decision Processes – MDP).....	257
7.2 Thuật toán Q-Learning (Model-Free)	258
7.3 Chiến lược khám phá – khai thác.....	258
7.4 Mở rộng & Ứng dụng	258
7.5 Gợi ý thực hành.....	258
7.1.2 Thuộc tính Markov (The Markov Property)	259
7.1.3 Các thành phần cấu thành một MDP	260
7.1.4 Mục tiêu trong MDP – Tìm chính sách tối ưu π^*	261
7.2 Thuật toán Q-Learning — Học mà không cần mô hình	263
7.2.2 Hàm Q-value (Action-Value Function).....	264
7.2.3 Cốt lõi của Q-Learning	267
7.3.1 Vấn đề với Bảng Q (Tabular Q-Value Representation)	269
7.3.2 Giải pháp Deep Q-Network (DQN): Xấp xỉ hàm Q bằng mạng nơ-ron sâu	270
7.3.3. Hai cải tiến đột phá trong Deep Q-Networks (DQN)	271

7.3.4 Ứng dụng và Hướng dẫn Dự án với DQN	273
CHƯƠNG 8 AI CÓ TRÁCH NHIỆM (RESPONSIBLE AI).....	275
Giới thiệu Sức mạnh đi đôi với trách nhiệm.....	275
8.1 Các vấn đề đạo đức trong AI.....	275
8.1.2. Công bằng (Fairness): Một khái niệm đa diện.....	276
8.2. An toàn và An ninh trong AI	280
8.2.2.1 Tấn công đối kháng (Adversarial Attacks)	282
8.2.2.2 Đầu độc dữ liệu (Data Poisoning).....	282
8.2.2.3 So sánh và Kết nối	283
Kết luận và Định hướng dự án.....	283
Gợi ý dự án cuối kỳ.....	285
1. Game AI: Minimax + Cắt tia Alpha–Beta cho trò chơi bàn cờ nâng cao (Caro, Cờ Gánh...)	285
2. Hệ chuyên gia tư vấn (cấu hình PC, dinh dưỡng...).	286
3. Dự đoán với Học máy (Kaggle dataset).....	287
4. Chatbot đơn giản (Rule-Based hoặc ML nhẹ)	287
5. Ứng dụng Thuật toán Di truyền (GA) cho tối ưu hóa.....	288
Hướng dẫn chung.....	289

PHẦN 1: NỀN TẢNG TRÍ TUỆ NHÂN TẠO

CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN VỀ TRÍ TUỆ NHÂN TẠO

1.1. Trí tuệ nhân tạo là gì?

Trí tuệ nhân tạo (Artificial Intelligence - AI) là một lĩnh vực nghiên cứu đa ngành, phức tạp và không có một định nghĩa đơn lẻ nào có thể bao trùm toàn bộ phạm vi cũng như bản chất của nó. Thay vì cố gắng đưa ra một định nghĩa thống nhất, các học giả và nhà nghiên cứu đã đề xuất bốn hướng tiếp cận chính để hiểu về AI, giúp xây dựng cái nhìn đa chiều và sâu sắc hơn. Bốn hướng tiếp cận đó bao gồm:

- **Suy nghĩ như con người** (Thinking Humanly),
- **Hành động như con người** (Acting Humanly),
- **Suy nghĩ hợp lý** (Thinking Rationally),
- **Hành động hợp lý** (Acting Rationally).

Mỗi cách tiếp cận này mang theo một hệ thống mục tiêu, công cụ và phương pháp nghiên cứu khác nhau, từ đó hình thành nên sự đa dạng trong phát triển các hệ thống AI hiện đại.

Để hình dung rõ hơn về mối quan hệ giữa các khái niệm trong AI, hình minh họa dưới đây cho thấy sơ đồ Venn thể hiện sự giao thoa giữa ba lĩnh vực: Trí tuệ nhân tạo (AI), Học máy (Machine Learning), và Học sâu (Deep Learning).



Hình 1. Sơ đồ Venn thể hiện mối quan hệ giữa Trí tuệ nhân tạo (AI), Học máy (Machine Learning) và Học sâu (Deep Learning).

1.1.1. Suy nghĩ như con người (*Thinking Humanly*): Cách tiếp cận Mô hình hóa Nhận thức

Một trong những hướng tiếp cận nền tảng trong nghiên cứu trí tuệ nhân tạo là mô hình hóa nhận thức (cognitive modeling), tức là xây dựng các hệ thống máy tính mô phỏng quy trình tư duy của con người. Mục tiêu chính của phương pháp này là tạo ra các hệ thống AI có khả năng suy nghĩ và giải quyết vấn đề theo cách tương tự như cách con người làm trong thực tế.

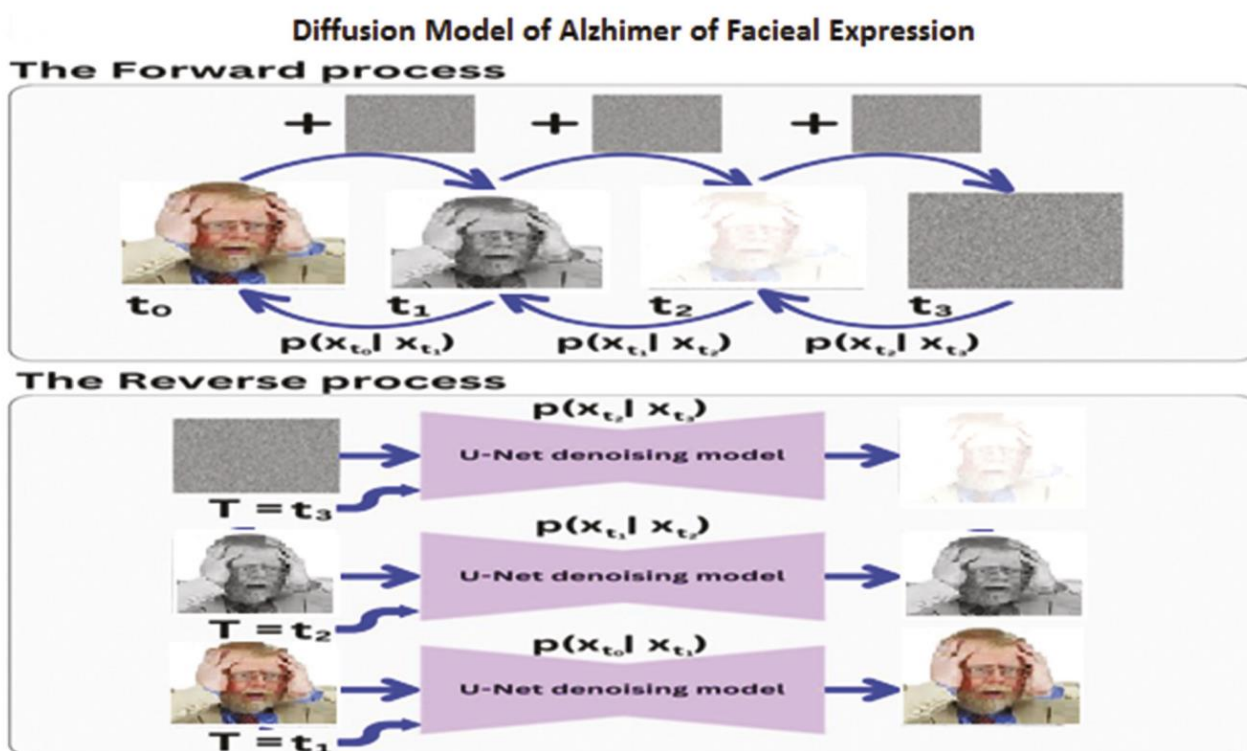
Để hiện thực hóa điều này, các nhà nghiên cứu thường sử dụng ba phương pháp thu thập và phân tích dữ liệu về quá trình nhận thức của con người:

- **Nội quan (Introspection):** Phương pháp này dựa trên việc tự phản ánh nội tâm để xác định các bước của quá trình tư duy khi giải quyết vấn đề, từ đó trừu tượng hóa thành mô hình máy tính.
- **Thí nghiệm tâm lý (Psychological Experiments):** Các nghiên cứu thực nghiệm được thiết kế nhằm theo dõi và ghi nhận hành vi tư duy của con người, giúp xây dựng các mô hình suy luận logic và quy trình xử lý thông tin.
- **Quét não bộ (Brain Imaging – fMRI):** Sử dụng các công nghệ hình ảnh hiện đại để xác định vùng não hoạt động khi con người thực hiện nhiệm vụ cụ thể, cung cấp dữ liệu sinh học hỗ trợ việc xây dựng mô hình AI sinh lý học.

Một minh họa tiêu biểu cho phương pháp tiếp cận này là hệ thống *General Problem Solver (GPS)* do Allen Newell và Herbert Simon phát triển vào thập niên 1950. GPS không chỉ được thiết kế để giải bài toán, mà còn thực hiện theo trình tự tương tự cách con người giải quyết vấn đề — bằng cách chia nhỏ mục tiêu, lập kế hoạch và đánh giá hiệu quả từng bước. Mục tiêu của họ là tạo ra một hệ thống thực sự “suy nghĩ như con người”.

Sự phát triển của hướng nghiên cứu này đã góp phần thúc đẩy sự giao thoa giữa trí tuệ nhân tạo, tâm lý học và khoa học thần kinh, từ đó định hình nên lĩnh vực khoa học nhận thức (cognitive science) hiện đại.

Một ví dụ đương đại cho thấy sự ứng dụng của mô hình hóa nhận thức là việc sử dụng mạng nơ-ron tích chập (CNN) trong phân tích biểu cảm khuôn mặt và chuyển động mắt để phát hiện bệnh Alzheimer. Hình minh họa dưới đây mô tả cách hệ thống AI sử dụng dữ liệu hình ảnh khuôn mặt để suy luận trạng thái nhận thức — một minh chứng rõ ràng cho AI “suy nghĩ” theo kiểu con người.



Hình 2: Phân tích biểu cảm khuôn mặt của bệnh nhân Alzheimer bằng CNN trong học máy.

Ví dụ nổi tiếng cho góc nhìn "suy nghĩ như con người" là **Hệ thống GPS (General Problem Solver)** được phát triển bởi Allen Newell và Herbert Simon vào những năm 1950. GPS là một chương trình máy tính được thiết kế để giải quyết các bài toán theo cách tiếp cận từng bước tương tự như cách mà con người sử dụng khi đối mặt với vấn đề phức tạp. Mô hình này bao gồm việc đặt ra các mục tiêu nhỏ hơn, xây dựng chiến lược giải quyết từng mục tiêu, và đánh giá hiệu quả của từng bước. Điều đáng chú ý là Newell và Simon không chỉ muốn GPS giải được bài toán, mà họ còn muốn nó giải theo cách gần gũi với phương pháp mà con người sử dụng.

Thông qua cách tiếp cận mô hình hóa nhận thức này, AI đã tạo ra một cầu nối quan trọng giữa khoa học máy tính và các lĩnh vực như tâm lý học và khoa học thần kinh, từ đó đóng góp vào sự hình thành và phát triển của lĩnh vực khoa học nhận thức (cognitive science). Điều này giúp AI không chỉ là một công cụ kỹ thuật, mà còn là một phương tiện mạnh mẽ để hiểu rõ hơn bản chất của trí tuệ con người.

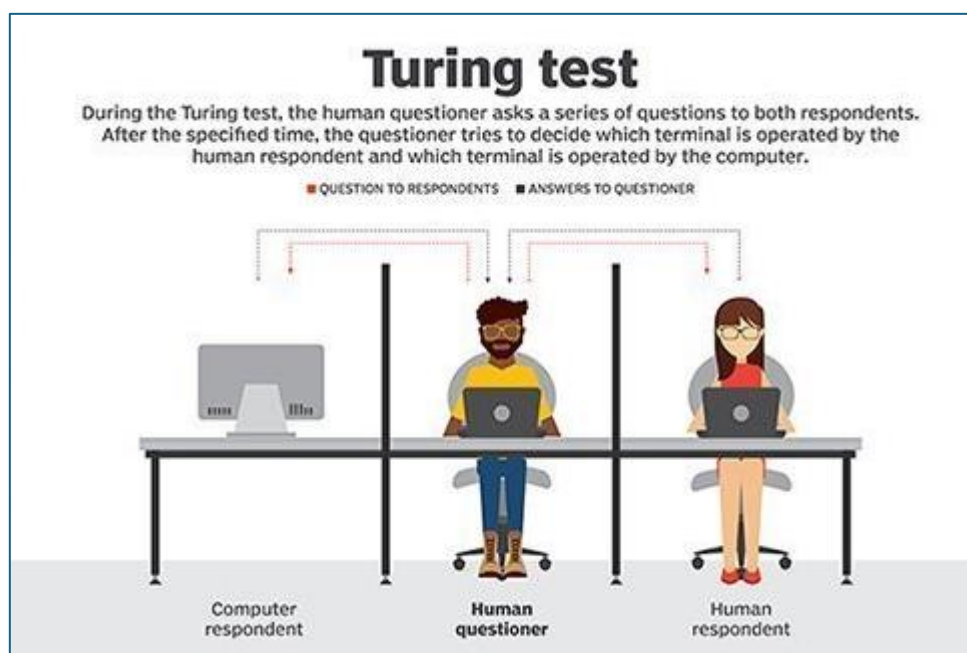
1.1.2. Hành động như con người (Acting Humanly): Phép thử Turing

Một cách tiếp cận khác để định nghĩa trí tuệ nhân tạo tập trung vào hành vi của máy tính thay vì mô phỏng các quá trình tư duy bên trong. Theo cách tiếp cận này, một hệ thống trí tuệ nhân tạo được xem là thông minh nếu nó có thể hành động giống như con người, nghĩa là có khả năng thực hiện các hành vi mà chúng ta thường coi là dấu hiệu của trí thông minh khi được con người thực hiện. Góc nhìn này được thể hiện rõ nhất qua **Phép thử Turing (Turing Test)**,

một trong những tiêu chuẩn đánh giá AI nổi tiếng nhất, được đề xuất bởi nhà toán học Alan Turing vào năm 1950.

Phép thử Turing

Thiết lập thử nghiệm: Phép thử Turing được thực hiện thông qua một trò chơi mà trong đó có một người giám khảo tương tác với hai đối tượng: một người và một máy tính. Các tương tác này diễn ra thông qua giao diện văn bản (chẳng hạn như màn hình và bàn phím) để người giám khảo không thể nhìn thấy hoặc nghe thấy trực tiếp các đối tượng tham gia. Nhiệm vụ của người giám khảo là đặt câu hỏi và nhận câu trả lời để từ đó xác định đối tượng nào là máy và đối tượng nào là người.



Hình 3: Sơ đồ minh họa cách hoạt động của Phép thử Turing, bao gồm người thẩm vấn (interrogator), máy tính và con người.

Tiêu chí thành công: Nếu sau khi tương tác một thời gian nhất định, người giám khảo không thể phân biệt được đâu là con người và đâu là máy, thì máy tính đó được xem là đã vượt qua Phép thử Turing. Theo tiêu chuẩn này, máy tính được coi là thông minh dựa trên khả năng thể hiện hành vi bên ngoài không thể phân biệt được với con người, bất kể cách thức nội bộ mà nó đạt được hành vi đó là như thế nào.

Các khả năng cần thiết để vượt qua Phép thử Turing: Để một cỗ máy có thể thành công trong việc thuyết phục giám khảo rằng nó là con người, cỗ máy đó cần có bốn khả năng thiết yếu sau:

- **Xử lý ngôn ngữ tự nhiên (Natural Language Processing):** Khả năng hiểu và tạo ra ngôn ngữ con người một cách mượt mà, giúp máy tính giao tiếp hiệu quả và tự nhiên như con người.

- **Biểu diễn tri thức (Knowledge Representation):** Khả năng lưu trữ thông tin về thế giới một cách có cấu trúc, cho phép máy tính sử dụng tri thức để trả lời các câu hỏi và đưa ra quyết định chính xác.
- **Lập luận tự động (Automated Reasoning):** Khả năng suy luận logic từ những tri thức đã có để giải quyết vấn đề hoặc trả lời các câu hỏi một cách hợp lý và nhất quán.
- **Học máy (Machine Learning):** Khả năng cải thiện hiệu suất và tự thích nghi thông qua kinh nghiệm, giúp máy tính cập nhật hành vi và tri thức theo thời gian mà không cần lập trình rõ ràng.

Ví dụ hiện đại: Chatbot

Các hệ thống trí tuệ nhân tạo hiện đại như ChatGPT của OpenAI hay Google Gemini là những minh chứng thực tế rõ ràng cho khả năng “hành động như con người”. Những chatbot này có thể tham gia vào các cuộc trò chuyện tự nhiên với con người, trả lời các câu hỏi, tạo ra văn bản mạch lạc, và thậm chí thể hiện sự sáng tạo trong việc tạo ra nội dung mới dựa trên các dữ liệu đã được huấn luyện. Mặc dù các chatbot này chưa hoàn toàn vượt qua Phép thử Turing trong mọi tình huống, nhưng chúng đã đạt được sự gần gũi đáng kể trong việc mô phỏng hành vi và khả năng tương tác tự nhiên của con người, từ đó thể hiện rõ nét giá trị thực tiễn của góc nhìn “hành động như con người” trong nghiên cứu và ứng dụng trí tuệ nhân tạo ngày nay.

1.1.3. Suy nghĩ hợp lý (Thinking Rationally): Cách tiếp cận "Quy luật của Tư duy"

Khác với hai góc nhìn trước đó tập trung vào việc tái hiện lại hành vi hoặc quá trình tư duy của con người, góc nhìn thứ ba này tiếp cận trí tuệ nhân tạo từ quan điểm của logic và sự hợp lý thuần túy. Mục tiêu chính của hướng nghiên cứu này là sử dụng logic hình thức (formal logic) làm nền tảng để mô tả, xây dựng và tự động hóa quá trình suy luận hợp lý của các hệ thống AI.

Nền tảng lý thuyết

Cách tiếp cận "suy nghĩ hợp lý" có nguồn gốc sâu xa từ các nguyên lý logic cổ điển được phát triển bởi Aristotle, người đã đưa ra khái niệm về các tam đoạn luận (syllogism). Một ví dụ nổi tiếng minh họa cho tam đoạn luận là:

- **Tiền đề 1:** Socrates là người.
- **Tiền đề 2:** Mọi người đều sẽ chết.
- **Kết luận:** Vậy, Socrates sẽ chết.

Các tam đoạn luận kiểu này là ví dụ cơ bản nhất của logic suy diễn (deductive logic), nơi mà kết luận được rút ra một cách chắc chắn từ các tiền đề đã được xác định rõ ràng. Dựa vào nền tảng này, các nhà nghiên cứu AI phát triển hệ thống trí tuệ nhân tạo có khả năng thực hiện các lập luận hợp lý và chính xác.

Ứng dụng logic trong AI

Logic hình thức đóng vai trò trung tâm trong việc xây dựng các hệ thống trí tuệ nhân tạo có khả năng suy luận tự động. Các nhà nghiên cứu sử dụng các ký hiệu logic để biểu diễn tri thức về thế giới, sau đó sử dụng các quy tắc suy diễn logic để đưa ra kết luận một cách tự động và hợp lý từ các thông tin đã cho.

Các hệ thống trí tuệ nhân tạo được xây dựng theo hướng này, chẳng hạn như các hệ thống chứng minh định lý tự động (automated theorem proving), có khả năng chứng minh các định lý toán học phức tạp bằng cách áp dụng các luật logic đã được lập trình sẵn một cách tự động. Đây là một minh chứng rõ ràng về khả năng của AI trong việc suy luận hợp lý và giải quyết các vấn đề dựa hoàn toàn trên logic.

Hạn chế của cách tiếp cận "suy nghĩ hợp lý"

Tuy nhiên, phương pháp dựa trên logic hình thức cũng tồn tại những hạn chế đáng kể. Thứ nhất, logic hình thức truyền thống gặp khó khăn trong việc biểu diễn và xử lý các loại tri thức không chắc chắn, mơ hồ, hoặc bất định mà chúng ta thường gặp trong thế giới thực. Chẳng hạn, trong các tình huống thực tế hàng ngày, việc đưa ra quyết định thường phải dựa vào thông tin không đầy đủ hoặc không chắc chắn. Logic hình thức thuần túy không thể dễ dàng xử lý các loại vấn đề này.

Ngoài ra, không phải tất cả các vấn đề trí tuệ nhân tạo đều có thể được giải quyết thông qua logic thuần túy. Ví dụ, các vấn đề liên quan đến nhận dạng mẫu (pattern recognition), xử lý hình ảnh, hoặc ngôn ngữ tự nhiên thường yêu cầu những kỹ thuật bổ sung ngoài logic hình thức để đạt được hiệu quả.

Tóm lại, mặc dù việc sử dụng logic hình thức cung cấp một nền tảng rõ ràng và vững chắc cho việc xây dựng các hệ thống AI có khả năng suy luận hợp lý, nhưng cách tiếp cận này vẫn cần kết hợp với các phương pháp khác để có thể giải quyết đầy đủ và linh hoạt hơn các vấn đề thực tế mà trí tuệ nhân tạo ngày nay đang đối diện.

1.1.4. Hành động hợp lý (Acting Rationally): Cách tiếp cận Tác tử Thông minh (Rational Agent)

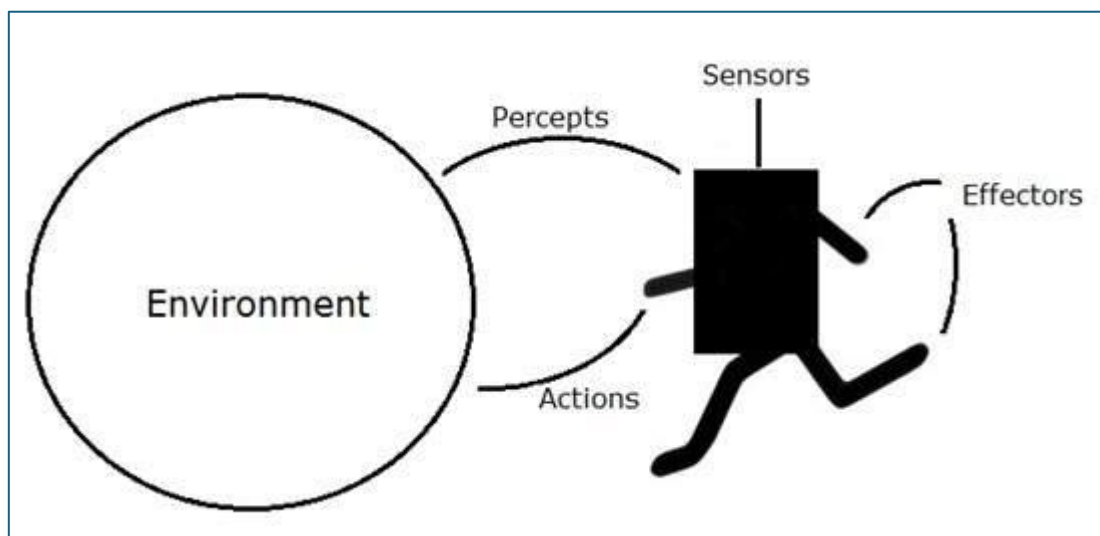
Hướng tiếp cận thứ tư – cũng là một trong những nền tảng trọng yếu của trí tuệ nhân tạo hiện đại – là hành động hợp lý (rational action). Thay vì chỉ mô phỏng tư duy hoặc hành vi của con người, quan điểm này xác định trí tuệ nhân tạo thông qua khả năng đưa ra các quyết định tối ưu nhằm tối đa hóa một thước đo hiệu suất đã được xác lập từ trước. Đây chính là nền tảng cho khái niệm tác tử thông minh (rational agent), một cấu trúc trung tâm trong nghiên cứu và triển khai AI ngày nay.

Định nghĩa về hành động hợp lý và tác tử thông minh

Một hành động được xem là hợp lý khi nó giúp tác tử đạt được kết quả tốt nhất hoặc gần tối ưu nhất theo các bằng chứng hiện có. Mức độ "tốt nhất" được đo lường thông qua một **thước**

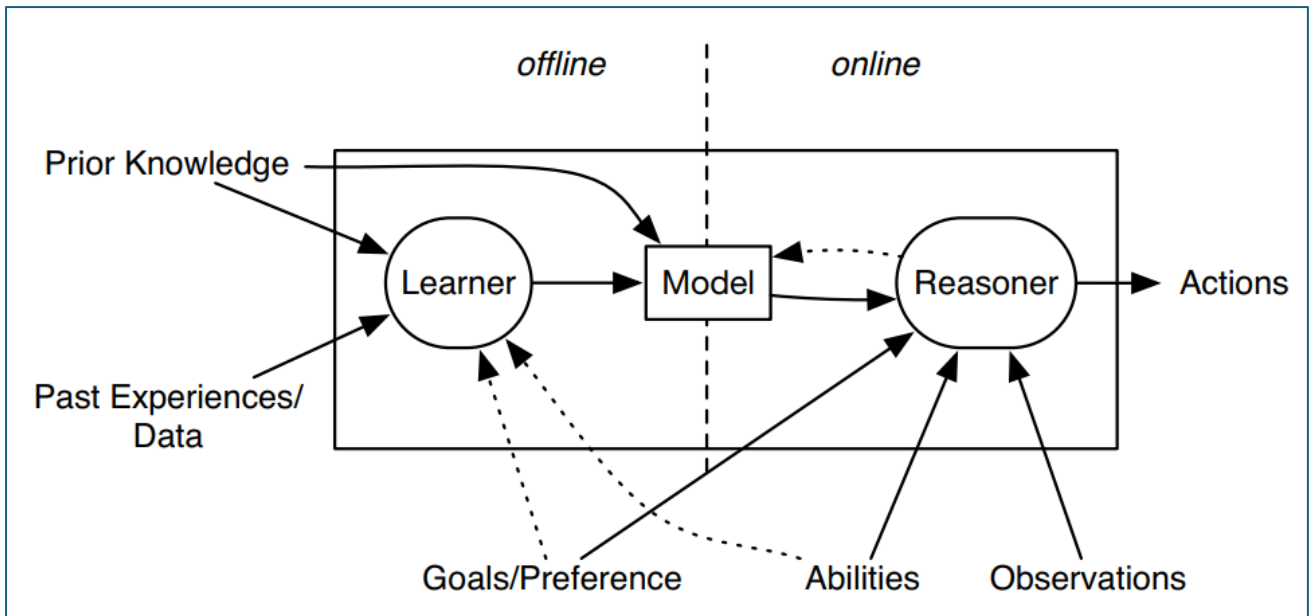
đo hiệu suất (performance measure) – tiêu chí cụ thể nhằm đánh giá thành công của tác tử trong việc đạt mục tiêu đã đặt ra.

Theo định nghĩa này, một **tác tử thông minh (rational agent)** là bất kỳ thực thể nào có khả năng cảm nhận môi trường thông qua **cảm biến (sensors)** và thực hiện hành động thông qua **cơ cấu chấp hành (actuators)** nhằm tối đa hóa thước đo hiệu suất đó.



Hình 4: Sơ đồ cơ bản mô tả một tác tử (agent) tương tác với môi trường (environment) thông qua cảm biến (sensors) và cơ cấu chấp hành (actuators).

Ngoài mô hình đơn giản, kiến trúc tác tử thông minh còn được phân tách thành hai giai đoạn chính: học tập **ngoại tuyến (offline learning)** – xây dựng mô hình hoặc nền tảng tri thức trước, và ra quyết định **trực tuyến (online reasoning)** – áp dụng kiến thức để hành động trong môi trường động. Minh họa chi tiết cho hai thành phần này giúp người học hiểu rõ hơn về cách tác tử hoạt động trong thực tế.



Hình 5: Sơ đồ phân tách giữa học ngoại tuyến và suy luận trực tuyến trong cấu trúc tác tử.

Ưu điểm của cách tiếp cận “hành động hợp lý”

Phương pháp “hành động hợp lý” được đánh giá là tổng quát và khoa học nhất, vì nó không giới hạn AI trong việc bắt chước tư duy hoặc hành vi của con người. Thay vào đó, nó cho phép tác tử sử dụng bất kỳ chiến lược nào – từ suy luận logic đến học máy, từ phản xạ tức thì đến phân tích xác suất – miễn là những chiến lược đó giúp đạt hiệu quả tối đa trong việc hoàn thành mục tiêu đề ra. Điều này đặc biệt hữu ích trong môi trường thực tế vốn luôn thay đổi và chứa nhiều thông tin không chắc chắn.

Ví dụ về tác tử thông minh trong thực tế

Một minh họa điển hình là **hệ thống xe tự lái (autonomous vehicles)** – nơi mà AI được triển khai để hành động một cách hợp lý nhằm đảm bảo an toàn, hiệu suất và tuân thủ quy định. Xe tự lái nhận diện môi trường bằng cảm biến (camera, radar, LiDAR), rồi đưa ra quyết định thông qua các cơ cấu điều khiển (vô-lăng, phanh, chân ga). Trong tình huống giao thông biến động, xe cần đánh giá nhanh và chọn phương án tối ưu như tránh chướng ngại hoặc đổi làn đường phù hợp – thể hiện rõ nguyên lý của một tác tử thông minh.

Qua ví dụ này, có thể thấy rằng mô hình tác tử thông minh cung cấp một khung lý thuyết linh hoạt, có khả năng thích nghi cao, rất phù hợp với yêu cầu của các hệ thống AI hiện đại hoạt động trong môi trường thực tế phức tạp.

1.2. Lịch sử hình thành và phát triển của AI

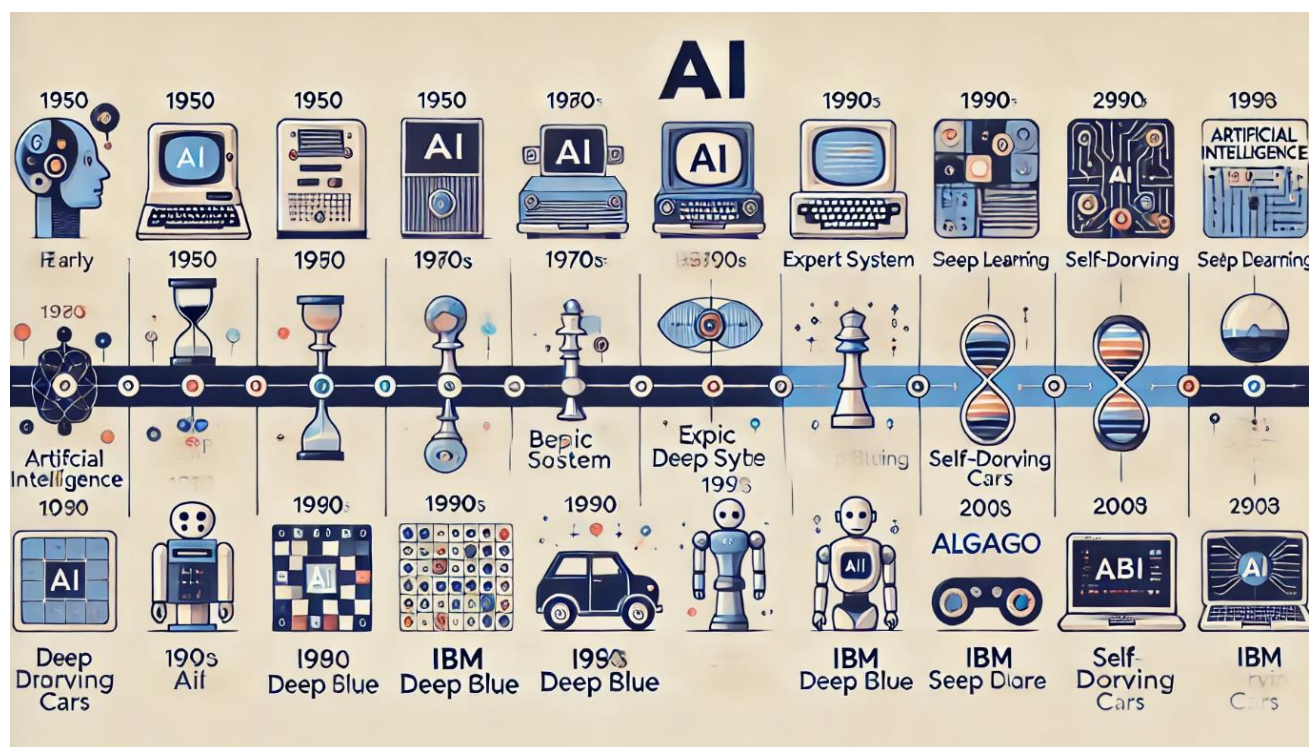
Trí tuệ nhân tạo (Artificial Intelligence – AI) không phải là một ngành khoa học mới nổi trong vài thập kỷ gần đây, mà thực chất đã có lịch sử hình thành và phát triển kéo dài từ giữa thế kỷ 20. Việc tìm hiểu lịch sử phát triển của AI không chỉ giúp chúng ta hiểu sâu hơn về các khái

niệm và kỹ thuật đã và đang được sử dụng, mà còn cung cấp bối cảnh cần thiết để đánh giá các xu hướng hiện tại và dự đoán tương lai của lĩnh vực này.

Quá trình phát triển của AI có thể được chia thành nhiều giai đoạn, mỗi giai đoạn gắn liền với các bước đột phá công nghệ cũng như các chu kỳ kỳ vọng và thất vọng (AI winters). Từ thời điểm Alan Turing đặt nền móng lý thuyết với câu hỏi “Máy móc có thể suy nghĩ không?” năm 1950, đến hội nghị Dartmouth năm 1956 – nơi cụm từ “Artificial Intelligence” lần đầu tiên được đặt tên, AI đã trải qua nhiều chu kỳ phát triển mạnh mẽ, sụt giảm và phục hồi.

Việc nghiên cứu tiến trình này giúp sinh viên không chỉ hiểu được vai trò của những cá nhân tiên phong như Turing, McCarthy, Minsky hay Simon, mà còn thấy được ảnh hưởng của các yếu tố như sức mạnh tính toán, dữ liệu, chính sách tài trợ và tiến bộ thuật toán trong sự phát triển của AI.

Hình 6 dưới đây minh họa trực quan các cột mốc chính trong lịch sử phát triển của AI, từ khởi nguồn lý thuyết đến các đột phá như máy học, học sâu và trí tuệ nhân tạo tổng quát.



Hình 6: Đồ họa dạng timeline tóm tắt các sự kiện và cột mốc quan trọng trong lịch sử phát triển của AI.

1.2.1. Giai đoạn thai nghén (1943-1955)

Giai đoạn đầu tiên của lịch sử AI được gọi là giai đoạn "thai nghén", diễn ra trong khoảng thời gian từ năm 1943 đến năm 1955. Đây là thời kỳ đặt nền móng lý thuyết và các ý tưởng cơ bản cho những nghiên cứu AI trong tương lai.

Mô hình nơ-ron nhân tạo của McCulloch & Pitts (1943)

Vào năm 1943, hai nhà khoa học Warren McCulloch và Walter Pitts đã giới thiệu mô hình nơ-ron nhân tạo đầu tiên. Trong bài báo kinh điển của mình, họ mô tả một cấu trúc toán học đơn giản nhưng đột phá để mô phỏng hoạt động của các tế bào thần kinh trong não bộ con người. Các nơ-ron nhân tạo này được thiết kế như những công tắc nhị phân (binary switches), có khả năng nhận đầu vào, thực hiện các phép tính logic đơn giản, và đưa ra một đầu ra xác định.

Mô hình nơ-ron nhân tạo McCulloch & Pitts đã cung cấp cơ sở lý thuyết đầu tiên cho việc xây dựng các mạng nơ-ron phức tạp hơn sau này. Đây là một bước tiến quan trọng, lần đầu tiên gợi mở khả năng mô phỏng trí tuệ con người bằng các cấu trúc máy tính.

Alan Turing và khả năng tư duy của máy móc (1950)

Một trong những cột mốc quan trọng và có ảnh hưởng sâu rộng nhất đến sự hình thành của lĩnh vực trí tuệ nhân tạo là bài viết kinh điển của Alan Turing vào năm 1950, mang tựa đề “*Computing Machinery and Intelligence*”. Trong tác phẩm này, Turing không chỉ đặt ra một câu hỏi mang tính bản chất – “*Máy móc có thể tư duy không?*” (*Can machines think?*) – mà còn đề xuất một khuôn khổ kiểm chứng khả năng tư duy đó thông qua một thử nghiệm cụ thể, được gọi là **Phép thử Turing (Turing Test)**.

Thử nghiệm này, như đã trình bày ở phần trước, đánh giá trí thông minh của một cỗ máy bằng cách quan sát khả năng tạo ra hành vi tương đương với con người trong một cuộc đối thoại thông qua văn bản. Nếu người thẩm vấn không thể phân biệt đâu là máy và đâu là người dựa trên câu trả lời nhận được, thì máy được xem là có trí tuệ ở mức độ con người.

Bài viết của Turing không chỉ có giá trị về mặt kỹ thuật mà còn là một bước ngoặt trong tư duy khoa học: lần đầu tiên, khả năng tư duy của máy móc được đặt trong khuôn khổ nghiên cứu nghiêm túc, có thể kiểm tra và đánh giá một cách khách quan. Đây là bước chuyển từ tưởng tượng khoa học viễn tưởng sang một lĩnh vực khoa học thực nghiệm.

Một ví dụ minh họa cho dạng đối thoại mà Turing hình dung trong thử nghiệm của mình có thể được trình bày như sau:

Người thẩm vấn: “Trong câu thơ đầu tiên trong bài sonnet của ông, ‘<i>Shall I compare thee to a summer’s day</i>,’ liệu dùng ‘<i>a spring day</i>’ có được không?” Nhân chứng: “Như vậy sẽ không đúng nhịp thơ.” Người thẩm vấn: “Thế còn ‘<i>a winter day</i>’ thì sao?” Nhân chứng: “Vâng, nhưng chẳng ai lại muốn được so sánh với một ngày mùa đông cả.” ... (đoạn tiếp hội thoại minh họa lối tư duy như con người).
--

Cuộc hội thoại trên thể hiện khả năng của hệ thống trong việc xử lý ngôn ngữ tự nhiên, lý luận theo ngữ cảnh và duy trì sự nhất quán – những yếu tố cốt lõi để đánh giá trí tuệ theo Turing.

Thời kỳ này, cùng với sự ra đời của mô hình nơ-ron nhân tạo của McCulloch & Pitts, đánh dấu giai đoạn thai nghén của trí tuệ nhân tạo như một ngành khoa học. Những ý tưởng này đã tạo nên nền móng lý thuyết vững chắc cho các nghiên cứu AI trong suốt những thập kỷ tiếp theo.

1.2.2. Sự ra đời của AI (1956)

Năm 1956 đánh dấu một bước ngoặt quan trọng trong lịch sử trí tuệ nhân tạo: đây là năm mà thuật ngữ "**Artificial Intelligence**" chính thức ra đời và lần đầu tiên được sử dụng rộng rãi trong cộng đồng khoa học. Sự kiện mang tính cột mốc này diễn ra tại hội thảo Dartmouth (Dartmouth Conference), tổ chức vào mùa hè năm 1956 tại Đại học Dartmouth, bang New Hampshire, Hoa Kỳ.

Hội thảo Dartmouth (1956)

Hội thảo Dartmouth được tổ chức bởi một nhóm các nhà nghiên cứu hàng đầu trong lĩnh vực khoa học máy tính và toán học thời bấy giờ, bao gồm John McCarthy, Marvin Minsky, Nathaniel Rochester và Claude Shannon. John McCarthy, khi đó là giáo sư trẻ tại Đại học Dartmouth, đã đề xuất thuật ngữ "**Artificial Intelligence**" trong đề xuất nghiên cứu cho hội thảo, nhằm mục đích tạo ra một nền tảng chung để tập trung nghiên cứu khả năng mô phỏng trí thông minh bằng máy tính.



Hình 7. Các nhà khoa học tại Hội thảo Dartmouth năm 1956 – Nơi khai sinh Trí tuệ nhân tạo.

Bức ảnh nhóm chụp các nhà khoa học tham dự Hội thảo Dartmouth mùa hè năm 1956 tại Đại học Dartmouth, Hanover, New Hampshire. Từ trái sang phải: **Hàng sau:** Oliver Selfridge, Nathaniel Rochester, Marvin Minsky, John McCarthy. **Hàng trước:** Ray Solomonoff, Claude Shannon. Nguồn: RoboticsBiz. robotsauthority.com.

Mục tiêu mà nhóm nghiên cứu Dartmouth đặt ra là rất tham vọng: họ hy vọng rằng trong thời gian hai tháng của hội thảo, họ sẽ có thể tạo ra những bước tiến đáng kể trong việc xây dựng các chương trình máy tính có khả năng tự động giải quyết vấn đề, sử dụng ngôn ngữ tự nhiên, và thể hiện các hành vi thông minh tương tự con người.

Ý nghĩa lịch sử của hội thảo Dartmouth

Mặc dù các mục tiêu tham vọng ban đầu của hội thảo chưa thể đạt được hoàn toàn trong thời gian ngắn ngủi đó, nhưng sự kiện này lại có ý nghĩa lịch sử sâu sắc. Nó được coi là "giấy khai sinh" chính thức của lĩnh vực trí tuệ nhân tạo như một ngành khoa học riêng biệt. Hội thảo Dartmouth đã tập hợp nhiều nhà khoa học lỗi lạc, đồng thời thúc đẩy sự hợp tác và chia sẻ các ý tưởng mới, đặt nền tảng cho rất nhiều nghiên cứu quan trọng sau này.

Kể từ hội thảo này, thuật ngữ "**Artificial Intelligence**" đã được chấp nhận rộng rãi và trở thành từ khóa chính thức cho một lĩnh vực nghiên cứu mới đầy hứa hẹn. Nó giúp cộng đồng khoa học thống nhất trong việc xây dựng các chương trình nghiên cứu dài hạn, thiết lập các mục tiêu rõ ràng, và huy động nguồn lực lớn từ các tổ chức nghiên cứu và tài trợ trên khắp thế giới.

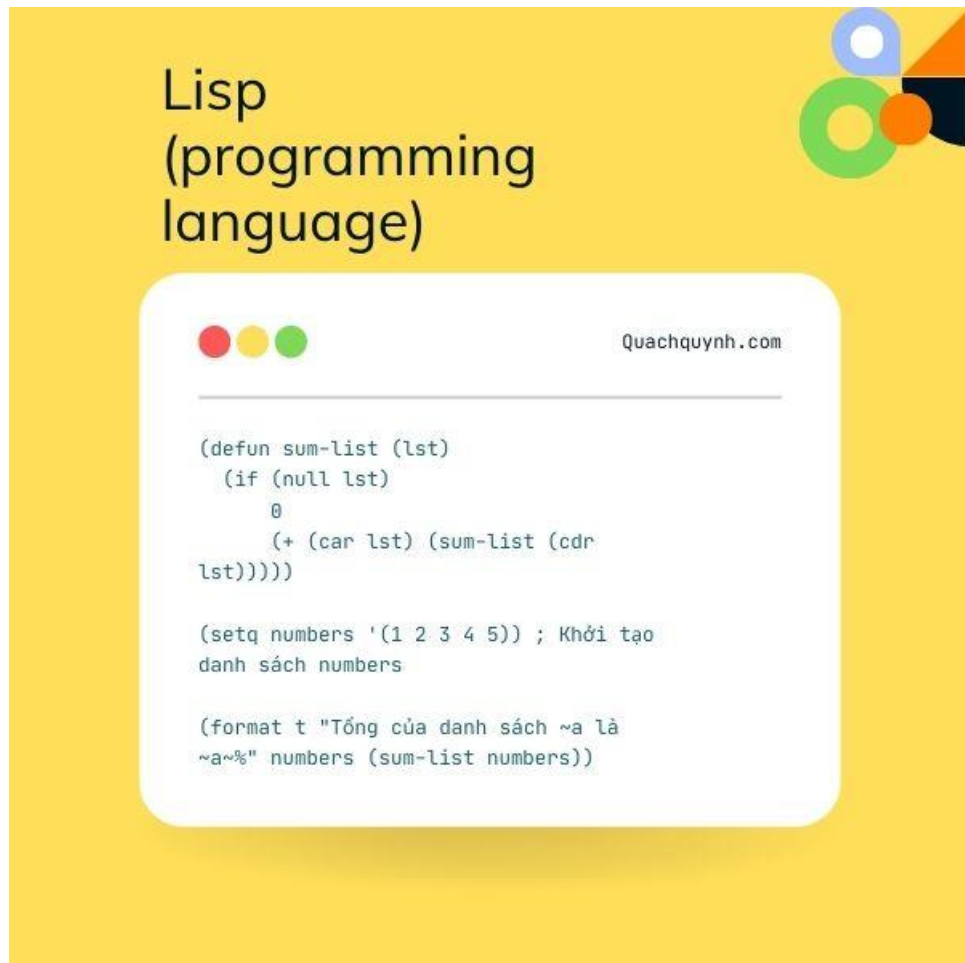
Như vậy, hội thảo Dartmouth không chỉ đặt tên cho lĩnh vực nghiên cứu này mà còn định hình bản sắc, mục tiêu, và lộ trình nghiên cứu cho trí tuệ nhân tạo trong các thập kỷ tiếp theo. Đây chính là điểm khởi đầu chính thức của AI như một ngành khoa học, ảnh hưởng sâu rộng đến toàn bộ quá trình phát triển sau này của lĩnh vực.

1.2.3. Kỷ nguyên của sự lạc quan (1952-1969)

Giai đoạn tiếp theo của lịch sử trí tuệ nhân tạo, kéo dài từ đầu những năm 1950 đến cuối những năm 1960, thường được gọi là "**kỷ nguyên của sự lạc quan**" (Era of Optimism). Trong thời kỳ này, các nhà nghiên cứu tràn đầy hy vọng vào khả năng phát triển nhanh chóng của AI, nhờ vào những bước tiến đột phá đầu tiên trong việc xây dựng các chương trình máy tính có khả năng giải quyết vấn đề và chơi các trò chơi trí tuệ đơn giản.

Ngôn ngữ lập trình Lisp và General Problem Solver (GPS)

Một trong những thành tựu quan trọng nhất của thời kỳ này là sự ra đời của **ngôn ngữ lập trình Lisp** (List Processing) vào năm 1958 bởi John McCarthy. Lisp là ngôn ngữ đầu tiên được thiết kế đặc biệt để xử lý các bài toán trí tuệ nhân tạo. Với khả năng linh hoạt trong việc biểu diễn và xử lý dữ liệu dạng danh sách (list), Lisp nhanh chóng trở thành công cụ chủ đạo trong nghiên cứu AI và được sử dụng rộng rãi để phát triển nhiều hệ thống trí tuệ nhân tạo trong các thập kỷ sau đó.

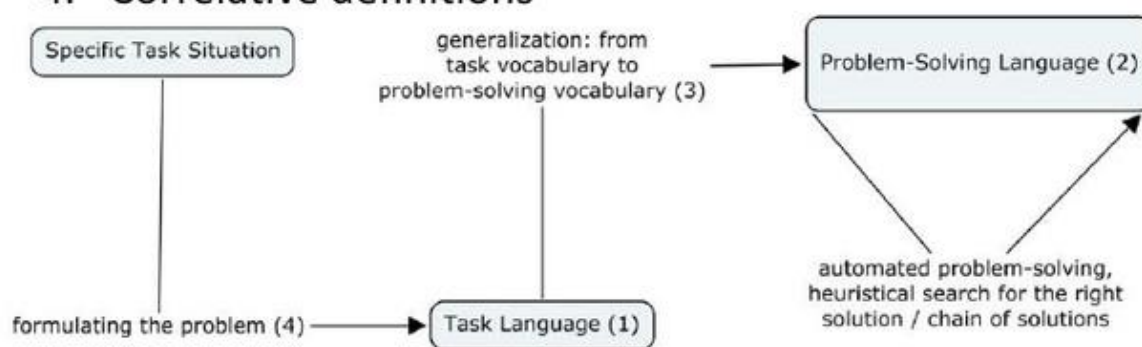


Hình 8. Lisp – Ngôn ngữ lập trình tiên phong cho Trí tuệ nhân tạo (John McCarthy, 1958).

Hình ảnh minh họa cú pháp và cấu trúc cơ bản của ngôn ngữ lập trình Lisp, được phát triển bởi John McCarthy vào năm 1958 tại MIT. Lisp là ngôn ngữ đầu tiên được thiết kế đặc biệt cho các ứng dụng trí tuệ nhân tạo, với khả năng xử lý dữ liệu dạng danh sách và hỗ trợ đệ quy mạnh mẽ. Nguồn: Quách Quỳnh.

General Problem Solver (GPS) – Generalized reasoning

1. Task environment vocabulary
 - proper nouns $\leftrightarrow$ common nouns
2. Problem-solving vocabulary
3. Conversion between 1 and 2
4. Correlative definitions



Hình 9. Sơ đồ hoạt động của chương trình General Problem Solver (GPS).

Sơ đồ minh họa cách thức hoạt động của chương trình General Problem Solver (GPS), được phát triển bởi Allen Newell và Herbert A. Simon vào năm 1957. GPS là một trong những chương trình AI đầu tiên mô phỏng quá trình giải quyết vấn đề của con người thông qua phân tích phương tiện-mục tiêu (means–ends analysis). **Nguồn:** [Timetoastmedia.timetoast.com](http://timetoastmedia.timetoast.com).

Song song với sự phát triển của Lisp, vào năm 1957, hai nhà khoa học Allen Newell và Herbert Simon đã giới thiệu **General Problem Solver (GPS)**, một chương trình mang tính đột phá có khả năng giải quyết các bài toán tổng quát bằng cách mô phỏng chiến lược tư duy của con người. GPS hoạt động dựa trên nguyên tắc chia nhỏ bài toán lớn thành các mục tiêu nhỏ hơn, sau đó lần lượt giải quyết các mục tiêu này một cách tuần tự. GPS không chỉ giải được các bài toán toán học đơn giản mà còn chứng minh khả năng mô phỏng hành vi giải quyết vấn đề theo cách gần gũi với con người, mở ra những hướng đi mới cho trí tuệ nhân tạo.

Những thành công ban đầu với các trò chơi trí tuệ

Giai đoạn này cũng chứng kiến những thành công đáng chú ý trong việc phát triển các chương trình máy tính chơi các trò chơi trí tuệ. Một trong những ví dụ nổi bật nhất là chương trình chơi cờ đam (checkers) do Arthur Samuel phát triển vào cuối những năm 1950. Samuel đã áp dụng các phương pháp học máy đơn giản, cho phép chương trình tự cải thiện khả năng chơi qua thời gian bằng cách học từ các trận đấu trước đó.



Hình 10. Arthur Samuel và chương trình cờ đam học máy trên IBM 701.

Bức ảnh chụp Arthur Samuel đang tương tác với chương trình chơi cờ đam trên máy tính IBM 701. Đây là một trong những chương trình đầu tiên áp dụng phương pháp học máy, cho phép máy tính cải thiện khả năng chơi cờ thông qua kinh nghiệm từ các ván đấu trước đó.

computer-timeline.com. Nguồn: Aitificial Blog

Chương trình chơi cờ đam của Samuel không chỉ đánh bại nhiều người chơi thông thường mà còn chứng minh rằng máy tính có thể học hỏi, tự cải thiện khả năng, và đạt được mức độ thông minh nhất định trong một lĩnh vực cụ thể. Thành công này đã tạo nên một làn sóng lạc quan mạnh mẽ trong cộng đồng nghiên cứu trí tuệ nhân tạo, giúp thu hút thêm nhiều nhà nghiên cứu tài năng và nguồn tài trợ lớn từ các tổ chức nghiên cứu trên khắp thế giới.

Nhìn chung, giai đoạn từ 1952 đến 1969 là thời kỳ đầy hứng khởi và lạc quan, khi trí tuệ nhân tạo bắt đầu cho thấy những tiềm năng to lớn. Sự phát triển của Lisp, GPS, cùng những thành tựu đầu tiên trong lĩnh vực trò chơi trí tuệ không chỉ củng cố niềm tin vào khả năng tạo ra các hệ thống trí tuệ thực sự mà còn đặt nền tảng vững chắc cho sự phát triển của AI trong các giai đoạn tiếp theo.

1.2.4. "Mùa đông AI" lần thứ nhất (1966-1973)

Sau gần một thập kỷ của những tiến bộ ban đầu đầy hứa hẹn, được gọi là “thời kỳ hoàng kim”, lĩnh vực trí tuệ nhân tạo đã bước vào một giai đoạn thử thách nghiêm trọng, thường được biết đến với tên gọi **“Mùa đông AI” (AI Winter) lần thứ nhất**, kéo dài từ khoảng năm 1966 đến năm 1973. Đây là thời kỳ mà sự lạc quan ban đầu va chạm với những rào cản lý thuyết và kỹ thuật không thể vượt qua, dẫn đến sự sụt giảm niềm tin, và quan trọng nhất là sự cắt giảm mạnh mẽ các nguồn tài trợ cho nghiên cứu. Giai đoạn này được định hình bởi ba vấn đề cốt lõi: sự phức tạp nội tại của các bài toán, những hạn chế về năng lực tính toán của phần cứng, và sự hoài nghi từ các cơ quan tài trợ.

Nguyên nhân dẫn đến khủng hoảng

1. Vấn đề “Bùng nổ Tổ hợp” (Combinatorial Explosion)

Một trong những trở ngại lý thuyết lớn nhất mà các nhà nghiên cứu AI thời kỳ đầu phải đối mặt là hiện tượng “bùng nổ tổ hợp”. Thuật ngữ này mô tả tình trạng mà số lượng các khả năng, trạng thái hoặc bước đi tiềm năng trong một bài toán tăng lên theo cấp số nhân khi quy mô của bài toán chỉ tăng một cách tuyến tính. Nhiều thuật toán AI, đặc biệt là các thuật toán tìm kiếm và lập kế hoạch, hoạt động bằng cách duyệt qua một không gian các khả năng để tìm ra lời giải. Đối với các “bài toán đồ chơi” (toy problems) với quy mô nhỏ, phương pháp này tỏ ra hiệu quả. Tuy nhiên, khi áp dụng vào các vấn đề thực tế, không gian tìm kiếm trở nên lớn đến mức không thể quản lý được.

Một ví dụ kinh điển để minh họa cho hiện tượng này là bài toán xây dựng **Hình vuông Latin (Latin Square)**. Một hình vuông Latin cấp n là một ma trận $n \times n$, trong đó mỗi hàng và mỗi cột đều chứa n ký hiệu khác nhau và mỗi ký hiệu chỉ xuất hiện đúng một lần.

Chẳng hạn, một hình vuông Latin cấp 3 có thể có dạng:

1	2	3
2	3	1
3	1	2

Về mặt lý thuyết, đây là một bài toán đơn giản. Tuy nhiên, số lượng các hình vuông Latin khả thi tăng với tốc độ chóng mặt khi n tăng lên. Hiện tượng này cho thấy rõ ràng một bài toán trông có vẻ đơn giản có thể nhanh chóng trở nên bất khả thi đối với các phương pháp tìm kiếm vét cạn.

n	The number of Latin squares of order n
1	1
2	2
3	12
4	576
5	161,280
6	812,851,200
7	61,479,419,904,000
8	108,776,032,459,082,956,800
9	5,524,751,496,156,892,842,531,225,600
10	9,982,437,658,213,039,871,725,064,756,920,320,000
11	776,966,836,171,770,144,107,444,346,734,230,682,311,065,600,000

Hình 11. Sơ đồ minh họa hiện tượng “bùng nổ tổ hợp”. Số lượng các hình vuông Latin (một bài toán tổ hợp) tăng theo cấp số nhân khi kích thước bài toán (cấp n) tăng, khiến các thuật

toán dựa trên tìm kiếm vét cạn trở nên không khả thi với năng lực tính toán hạn chế của thời kỳ đó. Nguồn: Wikipedia.

Vấn đề này không chỉ giới hạn trong các bài toán trừu tượng. Các dự án dịch máy (Machine Translation) đầy tham vọng cũng thất bại vì lý do tương tự. Các nhà nghiên cứu nhận ra rằng để dịch một câu một cách chính xác, máy tính không chỉ cần biết từ vựng mà còn phải xử lý sự mơ hồ về ngữ nghĩa và cấu trúc cú pháp phức tạp, vốn tạo ra một số lượng khổng lồ các diễn giải khả dĩ.

2. Hạn chế về Năng lực Tính toán và Bộ nhớ

Các ý tưởng và thuật toán AI, dù có tiềm năng, đã bị kìm hãm nghiêm trọng bởi giới hạn của công nghệ phần cứng thời bấy giờ. Máy tính trong những năm 1960 và 1970 có tốc độ xử lý chậm (đo bằng Kilo-Instructions Per Second - KIPS) và dung lượng bộ nhớ cực kỳ hạn chế (thường chỉ vài chục kilobyte). Các chương trình AI phức tạp đòi hỏi phải lưu trữ một lượng lớn trạng thái và thực hiện hàng triệu phép tính, một yêu cầu vượt xa khả năng của các hệ thống máy tính đương thời. Điều này tạo ra một khoảng cách lớn giữa những gì các nhà lý thuyết AI hình dung và những gì họ có thể thực sự triển khai.

3. Cắt giảm Tài trợ và các Báo cáo Mang tính Chỉ trích

Khi những lời hứa hẹn về một trí tuệ nhân tạo toàn năng không thành hiện thực, sự kiên nhẫn của các cơ quan tài trợ bắt đầu cạn kiệt. Hai sự kiện mang tính biểu tượng đã chính thức khởi đầu cho mùa đông AI:

- **Báo cáo ALPAC (1966) tại Hoa Kỳ:** Ủy ban Tư vấn Xử lý Ngôn ngữ Tự động (ALPAC) được chính phủ Hoa Kỳ thành lập để đánh giá tiến độ trong lĩnh vực dịch máy. Báo cáo năm 1966 của ủy ban này kết luận rằng, sau hơn một thập kỷ đầu tư hàng triệu đô la, các hệ thống dịch máy không nhanh hơn, chính xác hơn hay rẻ hơn so với người dịch. Kết quả là, chính phủ đã cắt giảm gần như toàn bộ ngân sách cho các dự án dịch máy, giáng một đòn mạnh vào một trong những lĩnh vực ứng dụng được kỳ vọng nhất của AI.
- **Báo cáo Lighthill (1973) tại Vương quốc Anh:** Tại Anh, giáo sư toán học ứng dụng James Lighthill đã được yêu cầu đánh giá tình hình nghiên cứu AI. Báo cáo của ông đã đưa ra những lời chỉ trích gay gắt, cho rằng các thành tựu của AI là quá khiêm tốn và không thể mở rộng quy mô từ phòng thí nghiệm ra thế giới thực. Lighthill chỉ ra rằng các thành công của AI chỉ giới hạn trong các “microworlds” (thế giới vi mô) được định nghĩa chặt chẽ và không thể giải quyết được các vấn đề phức tạp trong thực tế. Báo cáo này đã thuyết phục chính phủ Anh cắt giảm mạnh mẽ tài trợ cho nghiên cứu AI tại các trường đại học, ngoại trừ một vài trung tâm.

Tác động của những báo cáo này và sự thay đổi trong chính sách của các cơ quan tài trợ lớn như DARPA (Cơ quan Chỉ đạo các Dự án Nghiên cứu Quốc phòng Tiên tiến) ở Hoa Kỳ đã lan rộng ra cộng đồng quốc tế. Nguồn vốn cho nghiên cứu AI cạn kiệt, các dự án bị đình chỉ, và nhiều nhà khoa học đã phải từ bỏ hoặc “thay tên đổi họ” cho nghiên cứu của mình để tránh nhãn “AI” đang bị mất uy tín.

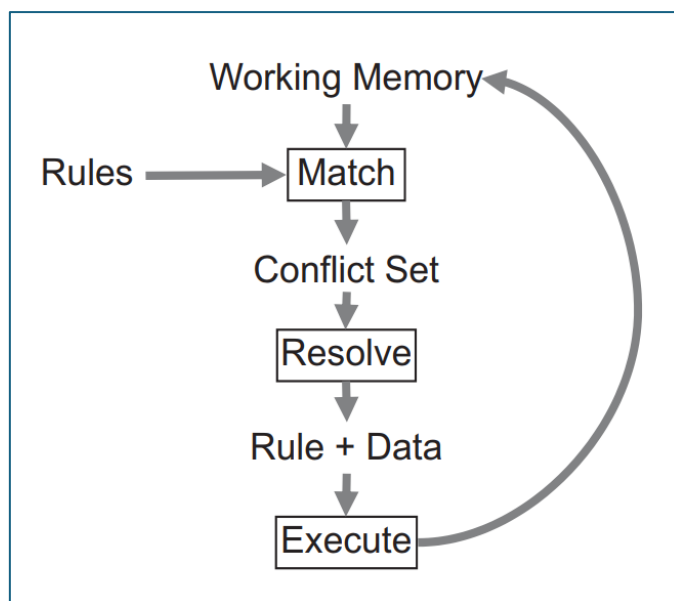
Hệ quả và Bài học

Mùa đông AI lần thứ nhất không chỉ là một giai đoạn trì trệ mà còn là một bài học đắt giá cho toàn ngành. Nó cho thấy sự nguy hiểm của việc thổi phồng quá mức (hype) và đưa ra những lời hứa hẹn thiếu cơ sở thực tiễn. Giai đoạn này buộc các nhà nghiên cứu phải có cái nhìn thực tế hơn về những thách thức phía trước.

Thay vì theo đuổi mục tiêu xây dựng một trí tuệ tổng quát, cộng đồng AI bắt đầu tập trung vào các vấn đề hẹp hơn, dễ quản lý hơn, dẫn đến sự ra đời của các lĩnh vực chuyên biệt như **hệ chuyên gia (expert systems)**, vốn sẽ trở thành động lực cho làn sóng AI tiếp theo vào những năm 1980. Mùa đông AI đã thanh lọc lĩnh vực này, loại bỏ những kỳ vọng viển vông và đặt nền móng cho một cách tiếp cận khoa học, bền vững và thực tế hơn, tạo tiền đề cho những đột phá trong tương lai.

1.2.5. Sự trỗi dậy của Hệ chuyên gia (1969-1986)

Sau thời kỳ "mùa đông AI" lần thứ nhất, trí tuệ nhân tạo đã tìm thấy một hướng đi mới để hồi sinh và phát triển mạnh mẽ trở lại trong giai đoạn từ năm 1969 đến năm 1986. Giai đoạn này được biết đến như là thời kỳ "**sự trỗi dậy của các hệ chuyên gia**" (the rise of expert systems). Đây là lúc ngành AI bắt đầu chuyển trọng tâm từ việc tìm kiếm các thuật toán tổng quát sang việc xây dựng các hệ thống dựa trên tri thức chuyên biệt (knowledge-based systems) với các ứng dụng cụ thể trong thực tế.



Hình 11. Chu trình khớp – giải quyết – thực thi của một IE dựa trên luật suy diễn tiến.

Chu trình trên kết thúc khi không có luật nào khớp hoặc một trong các luật thực thi một hành động gọi là Dừng (Halt). Hình 11 minh họa chu trình Khớp – Giải quyết – Thực thi của một IE dựa trên luật suy diễn tiến. Nó là suy diễn tiến, hay theo hướng dữ liệu, bởi vì các luật khớp với dữ liệu và thêm (hoặc xóa) dữ liệu vào WM, sau đó lại khớp các luật tiếp tục theo cách này. Ngược lại, người ta có thể có một phương pháp suy diễn lùi theo hướng mục tiêu, chọn các luật khớp với mục tiêu cần đạt được.

Chuyển hướng sang các hệ thống dựa trên tri thức

Khác với các phương pháp trước đây tập trung chủ yếu vào các thuật toán tổng quát, giai đoạn này chứng kiến sự chuyển hướng mạnh mẽ sang các hệ thống dựa trên tri thức (knowledge-based systems). Các hệ thống này được xây dựng dựa trên nền tảng các luật logic và tri thức chuyên môn từ các chuyên gia trong các lĩnh vực cụ thể. Việc thu thập và biểu diễn tri thức trở thành yếu tố trung tâm, thay vì chỉ dựa vào các thuật toán suy luận đơn thuần.

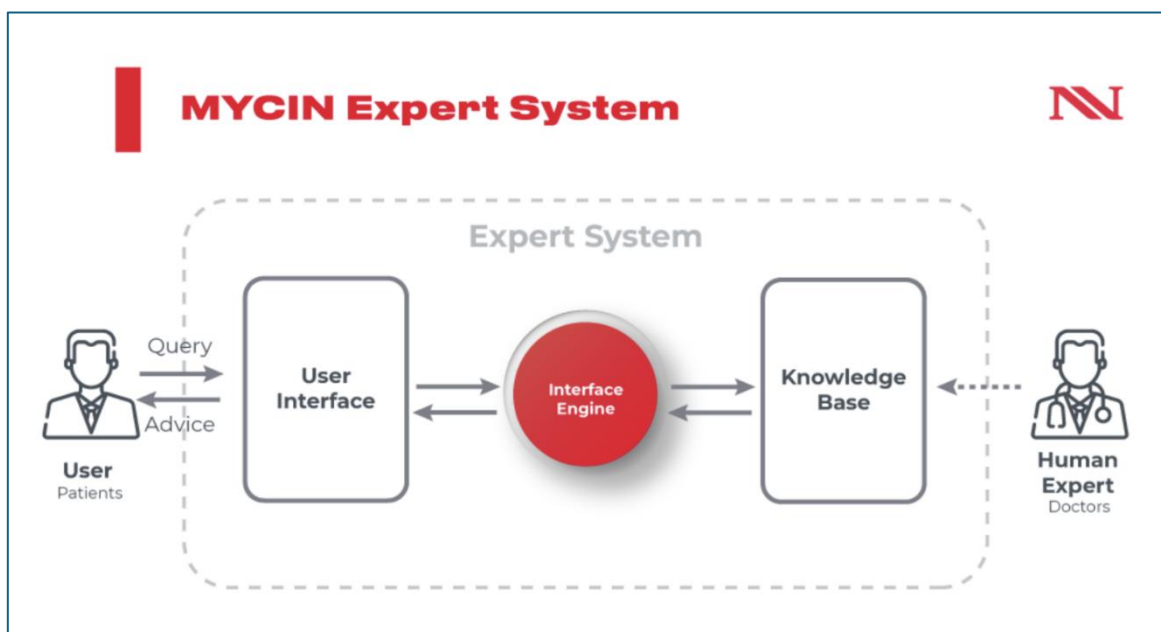
Các nhà nghiên cứu và các công ty công nghệ bắt đầu tập trung vào việc thu thập kiến thức từ các chuyên gia và đưa vào các hệ thống máy tính, tạo ra các hệ thống trí tuệ nhân tạo có thể đưa ra những quyết định chính xác và đáng tin cậy trong các lĩnh vực chuyên biệt như y tế, tài chính, và kỹ thuật.

Phương châm "Knowledge is power" (Tri thức là sức mạnh)

Trong giai đoạn này, phương châm "**Knowledge is power**" (tri thức là sức mạnh) trở nên nổi tiếng và được xem như triết lý phát triển chủ đạo của ngành AI. Ý tưởng này nhấn mạnh rằng sức mạnh thật sự của trí tuệ nhân tạo đến từ việc hệ thống có thể tiếp cận và sử dụng tri thức chuyên sâu trong các lĩnh vực cụ thể. Việc khai thác tri thức từ các chuyên gia và sử dụng nó để hỗ trợ quyết định là chìa khóa giúp AI trở nên hiệu quả và ứng dụng thực tế.

Thành công của hệ chuyên gia MYCIN

Một trong những ví dụ tiêu biểu nhất cho sự thành công của các hệ chuyên gia là hệ thống **MYCIN**, được phát triển tại Đại học Stanford vào đầu những năm 1970. MYCIN được thiết kế để chẩn đoán các bệnh nhiễm trùng máu và đề xuất các phương pháp điều trị phù hợp dựa trên kiến thức chuyên môn của các chuyên gia y khoa.



Hình 12. Kiến trúc hệ chuyên gia MYCIN.

Mô tả: Sơ đồ minh họa kiến trúc của hệ chuyên gia MYCIN, bao gồm các thành phần chính như hệ thống tư vấn, hệ thống giải thích, hệ thống thu thập tri thức và cơ sở dữ liệu động. MYCIN sử dụng tập hợp các quy tắc (rules) để chẩn đoán và đề xuất phương pháp điều trị cho các bệnh nhiễm trùng máu. Nguồn: SlideServe.

MYCIN sử dụng một bộ quy tắc (rules) rất lớn được xây dựng dựa trên tri thức từ các chuyên gia về bệnh truyền nhiễm. Hệ thống này có khả năng đưa ra chẩn đoán chính xác và gợi ý điều trị dựa trên thông tin nhập vào từ các triệu chứng và dữ liệu xét nghiệm. Đáng chú ý là trong các thử nghiệm thực tế, MYCIN đã cho thấy khả năng chẩn đoán bệnh nhiễm trùng máu với độ chính xác ngang bằng hoặc thậm chí vượt trội so với các bác sĩ chuyên khoa có kinh nghiệm.

Sự thành công của MYCIN và các hệ chuyên gia tương tự đã làm hồi sinh niềm tin vào tiềm năng của trí tuệ nhân tạo, giúp AI thoát ra khỏi thời kỳ mùa đông trước đó và thúc đẩy mạnh mẽ các nghiên cứu và ứng dụng trong nhiều lĩnh vực thực tiễn khác.

Tóm lại, giai đoạn từ 1969 đến 1986 là thời kỳ đánh dấu sự hồi phục mạnh mẽ của trí tuệ nhân tạo với trọng tâm là các hệ chuyên gia. Thành công của các hệ thống như MYCIN đã minh chứng rõ ràng cho sức mạnh và giá trị thực tế của việc tích hợp tri thức chuyên môn vào các hệ thống trí tuệ nhân tạo, mở ra các triển vọng mới đầy hứa hẹn cho ngành AI.

1.2.6. AI trở thành một ngành khoa học (1987-nay)

Giai đoạn từ cuối thập niên 1980 đến nay đánh dấu sự chuyên môn hóa mạnh mẽ của Trí tuệ nhân tạo (AI), chính thức khẳng định vị thế là một ngành khoa học thực thụ. Trong thời kỳ này, AI được xây dựng trên các nền tảng lý thuyết vững chắc của toán học, thống kê, và khoa học máy tính. Ba xu hướng phát triển chính định hình nên bộ mặt của AI hiện đại bao gồm: sự hồi sinh của mạng nơ-ron nhân tạo, sự tập trung vào nền tảng toán học và thống kê, và sự trỗi dậy của các phương pháp suy diễn dựa trên xác suất.

Sự trở lại của Mạng nơ-ron và thuật toán Lan truyền ngược

Một trong những cột mốc quan trọng nhất của giai đoạn này là sự phát triển vượt bậc của các mô hình Mạng nơ-ron nhân tạo (Artificial Neural Networks - ANN). Động lực chính cho bước tiến này là sự ra đời của thuật toán **lan truyền ngược (backpropagation)** vào cuối những năm 1980, được công bố chi tiết bởi các nhà nghiên cứu tiên phong như David Rumelhart, Geoffrey Hinton, và Ronald Williams.

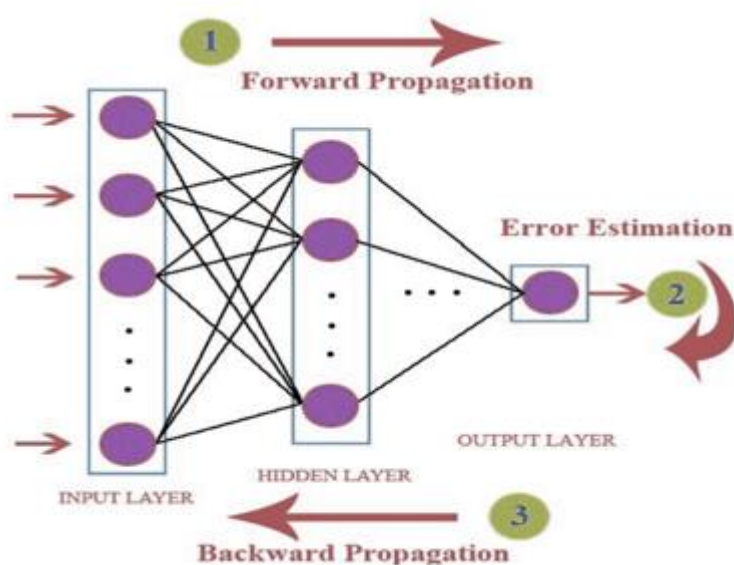
Trước đó, việc huấn luyện các mạng nơ-ron đa lớp (multi-layer neural networks) là một thách thức lớn do thiếu một phương pháp hiệu quả để cập nhật trọng số. Thuật toán lan truyền ngược đã giải quyết bài toán này bằng cách cung cấp một cơ chế hiệu quả để điều chỉnh trọng số của các nơ-ron nhằm tối thiểu hóa sai số của mô hình.

Quá trình huấn luyện một mạng nơ-ron bao gồm hai pha chính:

- **Lan truyền xuôi (Forward Propagation):** Là quá trình tính toán tín hiệu từ lớp đầu vào (input layer) qua các lớp ẩn (hidden layers) để đến lớp đầu ra (output layer). Tại mỗi nơ-ron, tín hiệu đầu vào từ lớp trước được tổng hợp và xử lý bởi một hàm kích hoạt (activation function) để tạo ra tín hiệu đầu ra cho lớp tiếp theo.

- **Lan truyền ngược (Backpropagation):** Sau khi có được kết quả dự đoán từ pha lan truyền xuôi, một hàm mất mát (loss function) sẽ được sử dụng để đo lường sự khác biệt giữa kết quả dự đoán và giá trị thực tế. Dựa trên giá trị mất mát này, thuật toán lan truyền ngược sẽ tính toán gradient (đạo hàm) của hàm mất mát theo từng trọng số trong mạng. Quá trình này diễn ra theo chiều ngược lại, từ lớp đầu ra về lớp đầu vào. Các gradient này sau đó được sử dụng trong một thuật toán tối ưu hóa, chẳng hạn như Gradient Descent, để cập nhật các trọng số, qua đó cải thiện độ chính xác của mô hình qua mỗi vòng lặp.

Sự thành công của lan truyền ngược đã mở đường cho sự phát triển của các kiến trúc mạng nơ-ron phức tạp, đặt nền móng cho cuộc cách mạng **Học sâu (Deep Learning)** sau này và trở thành công nghệ cốt lõi trong các lĩnh vực như thị giác máy tính, xử lý ngôn ngữ tự nhiên và nhận dạng tiếng nói.



Hình 13. Sơ đồ minh họa quá trình lan truyền ngược (Backpropagation).

Sơ đồ minh họa quá trình lan truyền ngược (Backpropagation). Gradient của hàm mất mát được tính toán tại lớp đầu ra và lan truyền ngược lại qua các lớp để cập nhật trọng số, giúp mô hình học hỏi từ dữ liệu.

Nền tảng Toán học và Thống kê

Song song với sự phát triển của mạng nơ-ron, lĩnh vực AI đã có một sự chuyển dịch mang tính nền tảng: tập trung vào việc xây dựng một cơ sở lý thuyết chặt chẽ dựa trên toán học và thống kê. Thay vì chỉ phát triển các thuật toán riêng lẻ, giới nghiên cứu đã tìm cách xây dựng các mô hình toán học tổng quát để lý giải cơ chế hoạt động và đảm bảo hiệu suất của các phương pháp AI.

Các lĩnh vực toán học như **đại số tuyến tính**, **giải tích**, và **lý thuyết tối ưu hóa** trở thành công cụ không thể thiếu. Đặc biệt, **lý thuyết xác suất và thống kê** đã cung cấp một khuôn khổ vững chắc để mô hình hóa sự không chắc chắn (uncertainty) và xử lý dữ liệu nhiễu trong thế giới thực. Việc áp dụng các nguyên lý toán học này không chỉ giúp các mô hình AI trở nên mạnh

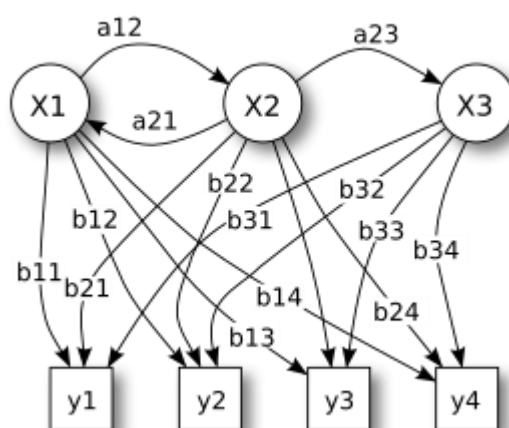
mẽ và đáng tin cậy hơn mà còn cho phép đánh giá và so sánh hiệu suất của chúng một cách khách quan.

Sự phát triển của các phương pháp Suy diễn xác suất

Một trong những thành tựu nổi bật của việc tích hợp lý thuyết xác suất vào AI là sự phát triển của các phương pháp **suy diễn xác suất (probabilistic reasoning)**. Các kỹ thuật này cho phép hệ thống AI đưa ra quyết định và dự đoán hợp lý khi đối mặt với thông tin không đầy đủ hoặc không chắc chắn.

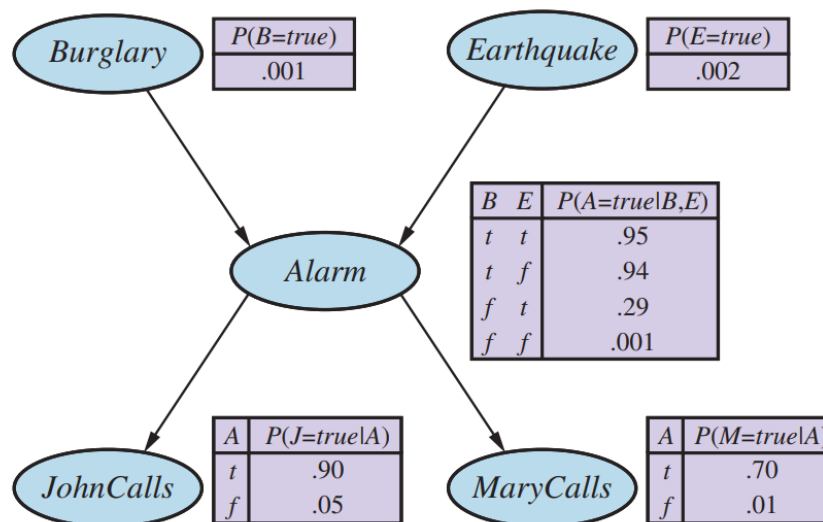
Hai mô hình tiêu biểu trong giai đoạn này là:

- **Mô hình Markov ẩn (Hidden Markov Models - HMMs):** HMM là một mô hình thống kê dùng để mô tả các hệ thống mà trạng thái bên trong (state) không thể quan sát trực tiếp, nhưng các kết quả đầu ra (observation) phụ thuộc vào trạng thái đó lại có thể quan sát được. HMM đặc biệt hiệu quả trong việc mô hình hóa các chuỗi dữ liệu theo thời gian, chẳng hạn như nhận dạng giọng nói (chuỗi âm thanh) hoặc xử lý ngôn ngữ tự nhiên (chuỗi từ).



Hình 14. Sơ đồ một Mô hình Markov ẩn (HMM). Trạng thái ẩn (X) quyết định đến các quan sát (y). Mô hình được định nghĩa bởi: xác suất chuyển trạng thái (a), xác suất phát sinh quan sát từ một trạng thái (b), và phân phối xác suất trạng thái ban đầu.

- **Mạng Bayesian (Bayesian Networks):** Mạng Bayesian là một mô hình đồ thị xác suất biểu diễn mối quan hệ phụ thuộc có điều kiện giữa một tập hợp các biến ngẫu nhiên. Mạng bao gồm một đồ thị có hướng không chu trình (DAG), trong đó mỗi nút đại diện cho một biến và các cạnh biểu diễn sự phụ thuộc xác suất. Mỗi nút được liên kết với một **Bảng xác suất có điều kiện (Conditional Probability Table - CPT)**, chứa xác suất của nút đó với mỗi tổ hợp giá trị của các nút cha. Mạng Bayesian là một công cụ mạnh mẽ để suy diễn trong các hệ thống phức tạp, chẳng hạn như chẩn đoán y khoa hay phát hiện gian lận.



Hình 15. Mạng Bayesian minh họa mối quan hệ nhân quả giữa Trộm, Động đất và phản ứng của hàng xóm.

Ví dụ về một Mạng Bayesian. Mạng mô tả mối quan hệ nhân quả giữa các sự kiện: Trộm (Burglary), Động đất (Earthquake), Chuông báo (Alarm), và các cuộc gọi từ hàng xóm (JohnCalls, MaryCalls). Các Bảng xác suất có điều kiện (CPT) định lượng hóa các mối quan hệ này.

Sự ra đời của các phương pháp suy diễn xác suất đã giúp AI giải quyết hiệu quả các bài toán thực tế phức tạp, nơi sự không chắc chắn là một yếu tố không thể tránh khỏi.

Tóm lại, giai đoạn từ 1987 đến nay đã chứng kiến sự trưởng thành của AI, chuyển từ một lĩnh vực dựa nhiều vào kinh nghiệm và thực nghiệm sang một ngành khoa học có nền tảng lý thuyết chặt chẽ. Những đột phá về mạng nơ-ron cùng với việc áp dụng sâu rộng các nguyên lý toán học và thống kê đã tạo ra những công cụ mạnh mẽ, đáng tin cậy, mở ra kỷ nguyên phát triển rực rỡ cho trí tuệ nhân tạo.

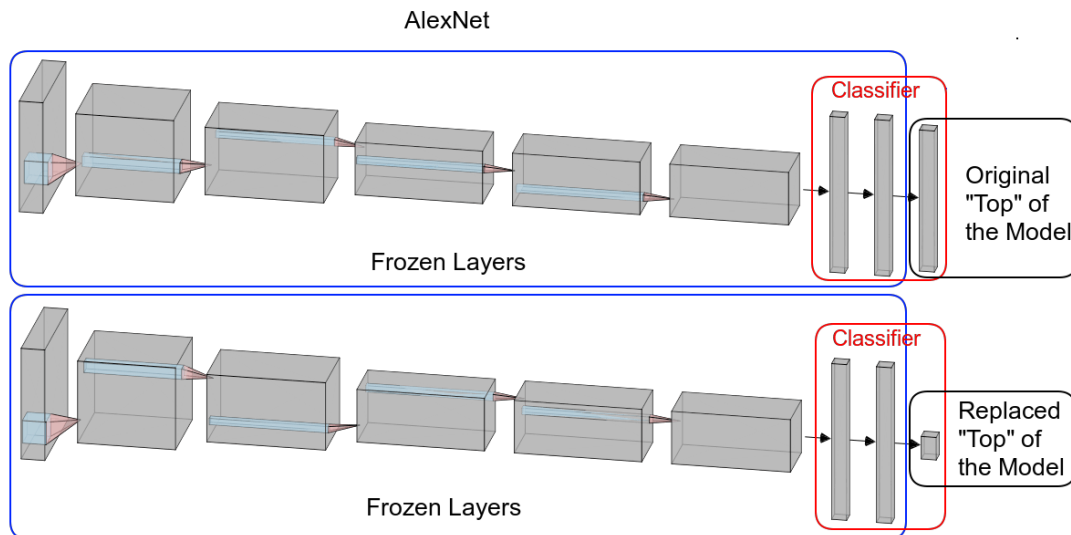
1.2.7. Kỷ nguyên của Dữ liệu lớn và Học sâu (2011-nay) AI Chuyển mình và Bùng nổ

Kể từ đầu thập kỷ thứ hai của thế kỷ 21, **Trí tuệ nhân tạo (AI)** đã bước vào một giai đoạn phát triển bùng nổ, được thúc đẩy bởi sự hội tụ của hai yếu tố then chốt: sự sẵn có của **dữ liệu lớn (Big Data)** và những đột phá trong các thuật toán **học sâu (Deep Learning)**. Đây là thời kỳ mà AI đã vượt ra khỏi giới hạn của các phòng thí nghiệm để trở thành một lực lượng công nghệ có tác động sâu rộng, định hình lại gần như mọi lĩnh vực của xã hội hiện đại.

ImageNet 2012: Cú hích quyết định đưa Học sâu ra ánh sáng

Cột mốc mang tính biểu tượng, đánh dấu sự khởi đầu của kỷ nguyên này chính là cuộc thi **ImageNet Large Scale Visual Recognition Challenge (ILSVRC) năm 2012**. Trong sự kiện này, một mạng nơ-ron tích chập sâu (Deep Convolutional Neural Network) mang tên **AlexNet**, được phát triển bởi Alex Krizhevsky, Ilya Sutskever và “bố già” của AI, Geoffrey Hinton, đã tạo nên một cơn địa chấn.

AlexNet đã giành chiến thắng áp đảo với tỷ lệ lỗi chỉ **15.3%**, vượt xa mức **26.2%** của đối thủ gần nhất vốn sử dụng các phương pháp thị giác máy tính truyền thống.



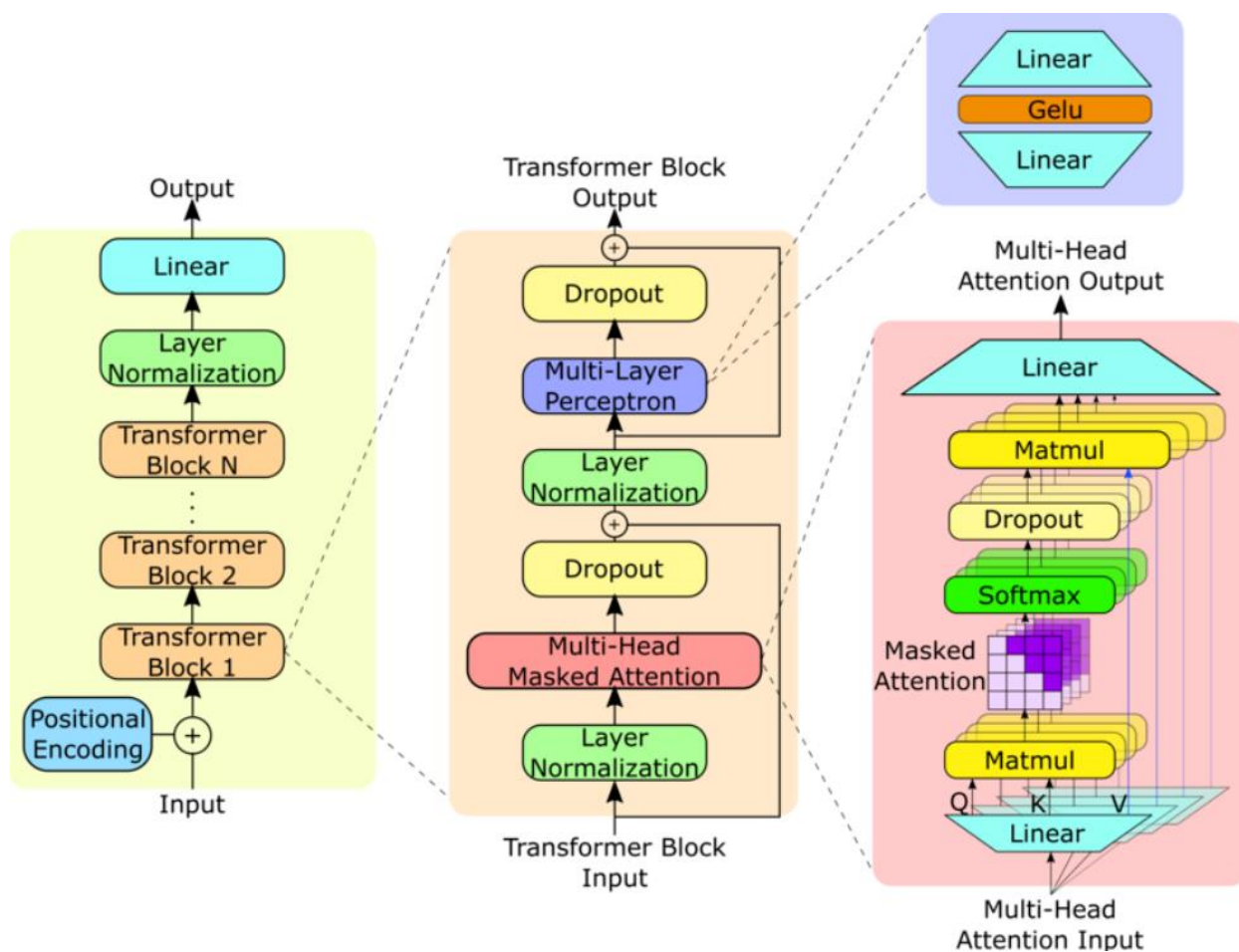
Hình 16. Kiến trúc đột phá của AlexNet.

Hình trên là kiến trúc đột phá của AlexNet. Với 5 lớp tích chập theo sau bởi 3 lớp kết nối đầy đủ, AlexNet đã chứng minh sức mạnh của việc “chồng” nhiều lớp nơ-ron để học các đặc trưng phức tạp từ dữ liệu hình ảnh, mở ra kỷ nguyên thống trị của học sâu.

Chiến thắng của AlexNet không chỉ đơn thuần là một giải thưởng. Nó đã trở thành một **bước ngoặt lịch sử**, khẳng định một cách đánh thép rằng các mô hình học sâu, khi được huấn luyện trên các tập dữ liệu cực lớn, có khả năng giải quyết các bài toán nhận dạng phức tạp với độ chính xác chưa từng có. Sự kiện này đã châm ngòi cho một làn sóng nghiên cứu và ứng dụng học sâu trên toàn cầu, đưa nó trở thành công nghệ lõi trong **thị giác máy tính (computer vision)**, **xử lý ngôn ngữ tự nhiên (natural language processing)**, và nhiều ứng dụng đột phá khác.

Sự trỗi dậy của Mô hình Ngôn ngữ Lớn và AI Tạo sinh: Khi máy móc biết “suy nghĩ” và “sáng tạo”

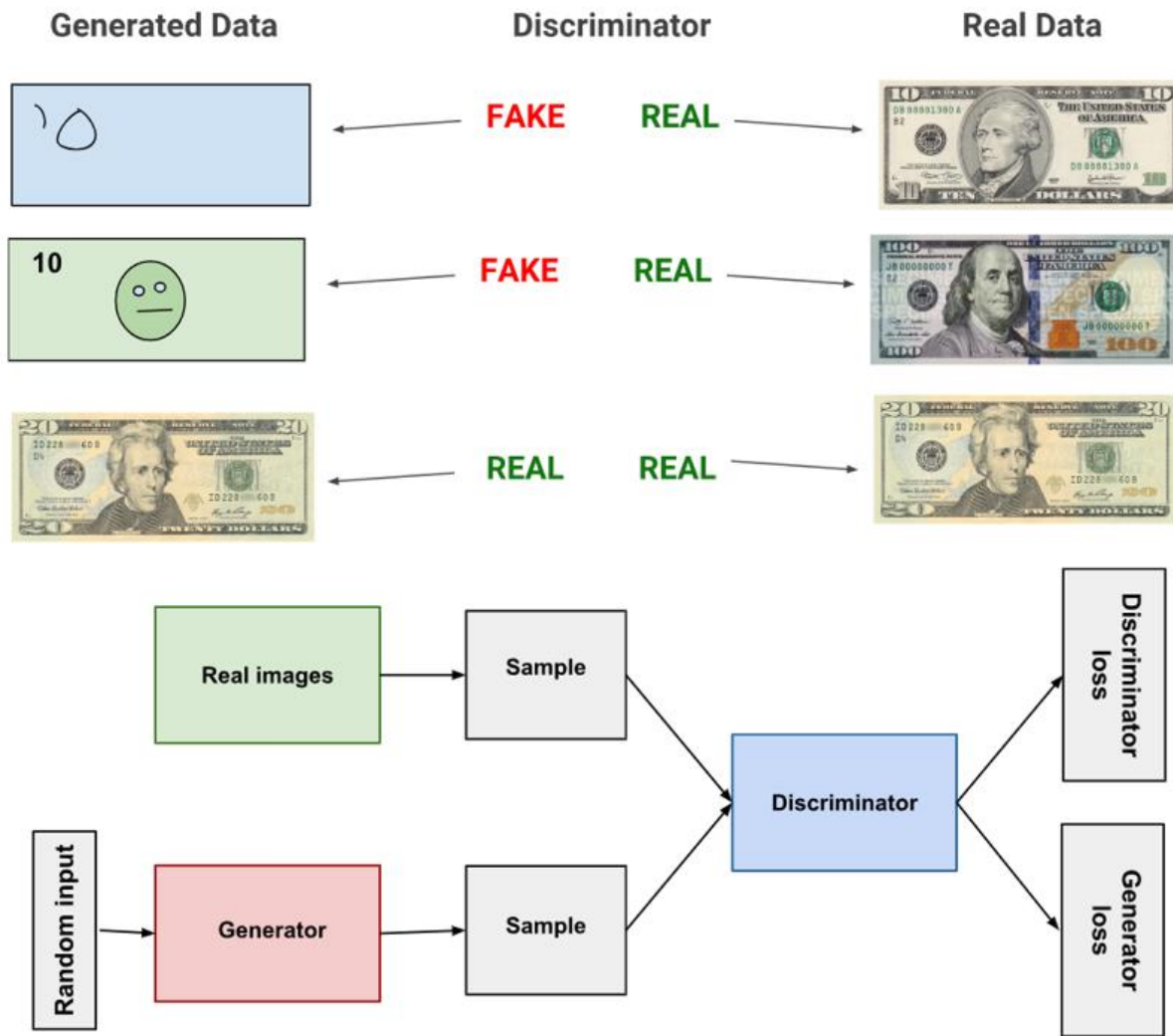
Tiếp nối thành công trong lĩnh vực hình ảnh, một cuộc cách mạng khác nhanh chóng diễn ra trong lĩnh vực ngôn ngữ với sự phát triển của các **Mô hình Ngôn ngữ Lớn (Large Language Models - LLMs)**. Các mô hình tiên phong như kiến trúc **Transformer** và các ứng dụng của nó như **GPT (Generative Pre-trained Transformer)** của OpenAI hay **BERT** của Google, cùng các hậu duệ nổi tiếng như **ChatGPT**, **GPT-4**, **Google Gemini**, đã chứng minh khả năng phi thường trong việc hiểu, xử lý và tạo ra văn bản tự nhiên.



Hình 17. Minh họa kiến trúc của một mô hình dựa trên Transformer như GPT-2.

Hình ảnh thể hiện cách mô hình xử lý chuỗi đầu vào thông qua các khối Transformer decoder và cơ chế tự chú ý (self-attention) để tạo ra văn bản một cách logic và mạch lạc.

Song song đó, **AI Tạo sinh (Generative AI)** cũng bùng nổ mạnh mẽ, đặc biệt với các kỹ thuật tiên tiến như **Mạng đối nghịch Tạo sinh (Generative Adversarial Networks - GANs)**. Những công nghệ này cho phép AI không chỉ phân tích thông tin mà còn có thể **sáng tạo** ra những nội dung hoàn toàn mới—from những bức ảnh chân thực, video tổng hợp, cho đến âm nhạc và các tác phẩm nghệ thuật.



Hình 18. Nguyên lý hoạt động của GAN với hai mạng đối đầu: Generator và Discriminator.

Nguyên lý hoạt động của GAN. Hệ thống bao gồm hai mạng nơ-ron “đối đầu” nhau: **Mạng Tạo (Generator)** có nhiệm vụ tạo ra dữ liệu giả, trong khi **Mạng Phân biệt (Discriminator)** cố gắng phân biệt đâu là dữ liệu thật, đâu là giả.

Quá trình “cạnh tranh” này giúp Mạng Tạo ngày càng học được cách tạo ra những sản phẩm tinh vi và chân thực hơn, như được minh họa qua các giai đoạn huấn luyện dưới đây.

Minh họa sự tiến bộ của Mạng Tạo trong GAN qua các giai đoạn. **(a) Giai đoạn đầu:** Ảnh tạo ra (trái) còn rất nhiều và Mạng Phân biệt (phải) dễ dàng nhận ra là giả. **(b) Giai đoạn giữa:** Ảnh tạo ra đã có cấu trúc rõ ràng hơn, gây khó khăn hơn cho Mạng Phân biệt. **(c) Giai đoạn cuối:** Ảnh tạo ra rất giống thật, khiến Mạng Phân biệt bắt đầu phân loại sai, cho thấy Mạng Tạo đã học rất tốt.

Giai đoạn từ năm 2011 đến nay đã chứng kiến sự chuyển mình ngoạn mục của trí tuệ nhân tạo. Được tiếp sức bởi “nhiên liệu” là dữ liệu lớn và “động cơ” là học sâu, AI đã phát triển từ một lĩnh vực học thuật trở thành một công nghệ nền tảng, thông minh hơn, sáng tạo hơn và có tầm ảnh hưởng sâu rộng đến nền kinh tế và cuộc sống con người trên toàn thế giới.

1.3. Các ứng dụng của Trí tuệ nhân tạo

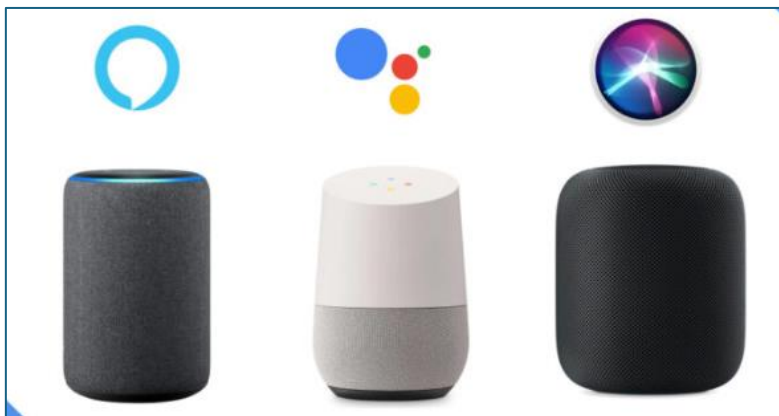
Trong suốt quá trình phát triển của mình, trí tuệ nhân tạo (AI) đã từng bước thâm nhập và thay đổi sâu sắc nhiều lĩnh vực của đời sống hàng ngày cũng như các ngành công nghiệp. Bằng cách liên kết lý thuyết với thực tế thông qua các ví dụ cụ thể, chúng ta có thể dễ dàng nhận thấy sức mạnh và tầm ảnh hưởng rộng lớn của AI trong cuộc sống hiện đại.

Trong đời sống hàng ngày

Trong cuộc sống hàng ngày, chúng ta đã tương tác thường xuyên với các hệ thống trí tuệ nhân tạo mà đôi khi chính chúng ta cũng không nhận ra. Những ứng dụng này giúp cuộc sống trở nên tiện lợi, hiệu quả và thú vị hơn.

Trợ lý ảo (Virtual Assistants)

Các trợ lý ảo như **Siri (Apple)**, **Google Assistant (Google)**, hay **Alexa (Amazon)** là những ví dụ điển hình về việc ứng dụng AI vào cuộc sống hàng ngày. Những trợ lý này sử dụng các công nghệ như xử lý ngôn ngữ tự nhiên (NLP), nhận dạng giọng nói, và học máy để hiểu, phản hồi, và thực hiện các yêu cầu từ người dùng. Chúng có thể thực hiện nhiều nhiệm vụ khác nhau như đặt lịch hẹn, gửi tin nhắn, chơi nhạc, điều khiển các thiết bị thông minh trong nhà, cung cấp thông tin thời tiết, hay trả lời các câu hỏi đơn giản. Khả năng tương tác tự nhiên và liên tục học hỏi từ dữ liệu giúp các trợ lý ảo ngày càng trở nên thông minh và hữu ích hơn trong cuộc sống hằng ngày.



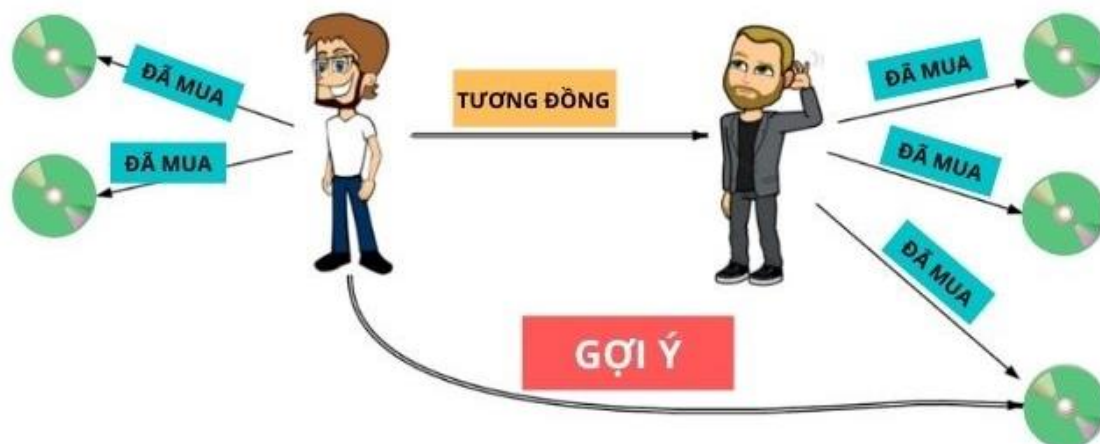
Hình 19. Voice Assistants & Smart Speakers.

Hệ thống gợi ý (Recommendation Systems)

Một ứng dụng phổ biến và được nhiều người dùng trải nghiệm hàng ngày là các hệ thống gợi ý sản phẩm và nội dung. Ví dụ, **Netflix** sử dụng thuật toán học máy để phân tích lịch sử xem phim của người dùng và đưa ra những gợi ý phim phù hợp nhất với sở thích cá nhân. Tương tự, ứng dụng nghe nhạc như **Spotify** sử dụng AI để xây dựng các danh sách phát cá nhân hóa dựa trên sở thích và thói quen nghe nhạc của người dùng.

Trong lĩnh vực thương mại điện tử, các nền tảng như **Tiki**, **Shopee** hay **Amazon** đều tận dụng AI để gợi ý các sản phẩm phù hợp với nhu cầu của khách hàng dựa trên lịch sử mua hàng, tìm

kiểm trước đó và các xu hướng mua sắm hiện tại. Nhờ những hệ thống gợi ý này, trải nghiệm mua sắm và giải trí của người dùng trở nên dễ dàng, cá nhân hóa và hiệu quả hơn.

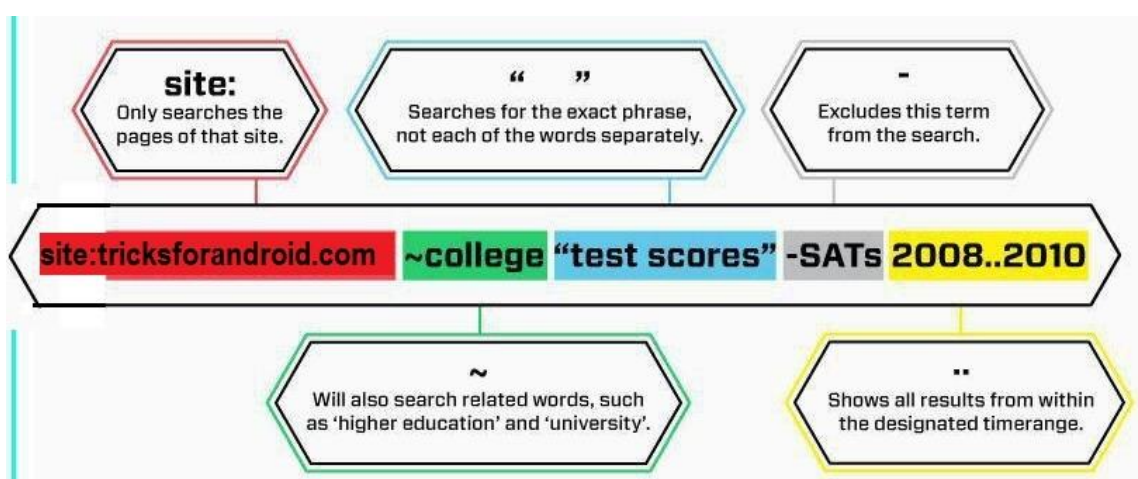


Hình 20. Hệ thống gợi ý (Recommendation Systems).

Sơ đồ mô tả quá trình hệ thống gợi ý hoạt động: thu thập dữ liệu người dùng, phân tích, và đưa ra gợi ý cá nhân hóa. Minh họa cách AI phân tích hành vi người dùng để đưa ra các đề xuất phù hợp, nâng cao trải nghiệm người dùng.

Tìm kiếm thông minh (Intelligent Search)

Một ứng dụng AI khác cực kỳ phổ biến mà chúng ta sử dụng hàng ngày là công cụ tìm kiếm như **Google Search**. Công cụ này không chỉ đơn thuần hiển thị các kết quả dựa trên từ khóa mà còn sử dụng các thuật toán học máy, xử lý ngôn ngữ tự nhiên, và mô hình ngôn ngữ lớn để hiểu ngữ cảnh, ý định tìm kiếm của người dùng một cách chính xác và sâu sắc hơn.



Hình 21. Ứng dụng Trí tuệ nhân tạo trong Công cụ Tìm kiếm (ví dụ Google Search).

Google Search cũng tích hợp AI để tự động sửa lỗi chính tả, cung cấp các kết quả tìm kiếm dự đoán nhanh, gợi ý các câu hỏi và thông tin liên quan. Việc sử dụng AI giúp cải thiện đáng kể chất lượng và độ chính xác của các kết quả tìm kiếm, tạo ra trải nghiệm người dùng trực quan và hiệu quả hơn rất nhiều so với các phương pháp truyền thống trước đây.

Như vậy, chỉ qua một số ví dụ tiêu biểu trong đời sống hàng ngày, chúng ta có thể thấy rõ ràng sức mạnh và khả năng lan tỏa rộng lớn của trí tuệ nhân tạo. Các ứng dụng này không chỉ làm cho cuộc sống của chúng ta dễ dàng và thuận tiện hơn mà còn cho thấy sự hiện diện không thể thiếu của AI trong xã hội hiện đại.

Trong y tế

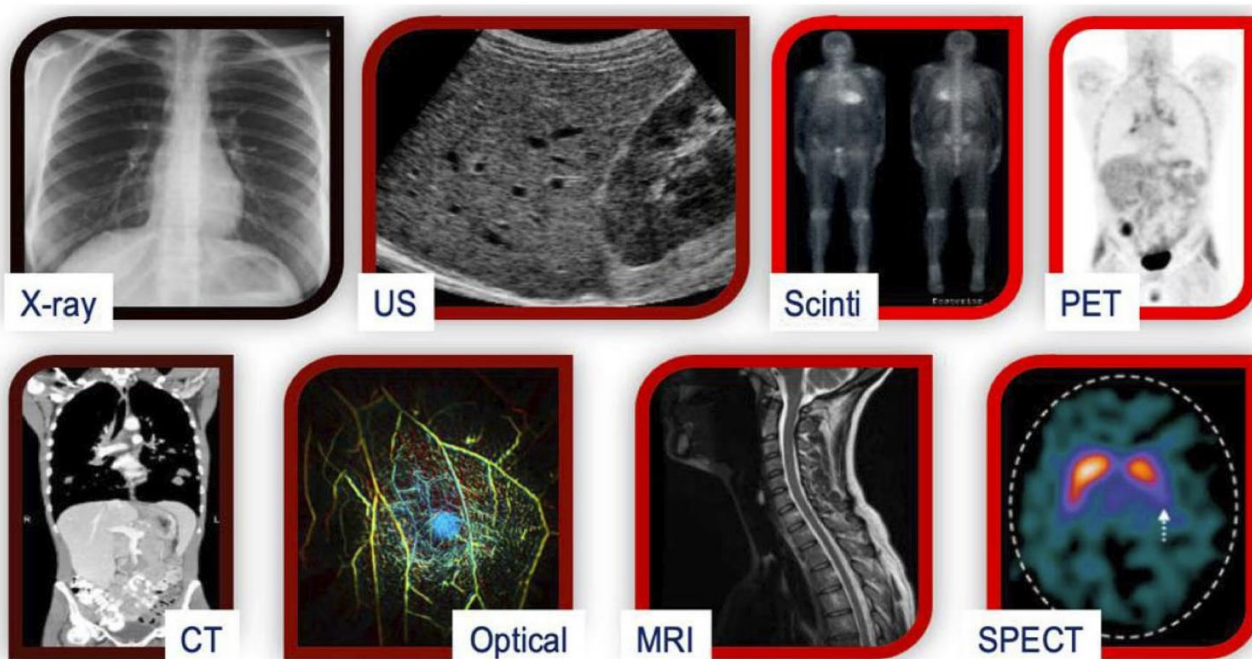
Trí tuệ nhân tạo (AI) ngày càng khẳng định vai trò then chốt trong việc chuyển đổi ngành y tế, cung cấp các công cụ mạnh mẽ hỗ trợ bác sĩ và chuyên gia sức khỏe trong chẩn đoán, điều trị và nghiên cứu y học. Sự hội tụ giữa AI và y học không chỉ giúp cải thiện đáng kể độ chính xác và tốc độ xử lý các vấn đề phức tạp mà còn mở ra những cơ hội đột phá cho các phương pháp trị liệu và khám phá khoa học trong tương lai.

Chẩn đoán Hình ảnh (Medical Image Diagnosis)

Một trong những ứng dụng thành công và phổ biến nhất của AI trong lĩnh vực y tế là chẩn đoán hình ảnh. Các hệ thống trí tuệ nhân tạo, đặc biệt là các

mạng nơ-ron tích chập sâu (Deep Convolutional Neural Networks - CNNs), được huấn luyện để phân tích và phát hiện các dấu hiệu bệnh lý từ các hình ảnh y khoa như X-quang, MRI (chụp cộng hưởng từ), CT scan, hay siêu âm. Các thuật toán này đã chứng tỏ hiệu quả cao trong việc phát hiện các bất thường như khối u ung thư, tổn thương nội tạng, hay các bệnh lý phổi.

Chẳng hạn, AI có khả năng phân tích nhanh và chính xác hình ảnh X-quang để sàng lọc sớm các dấu hiệu bất thường như viêm phổi, ung thư phổi, hoặc thậm chí là COVID-19. Tương tự, các hệ thống AI phân tích ảnh MRI giúp bác sĩ phát hiện sớm và đánh giá các khối u não với độ chính xác cao, qua đó giảm thiểu sai sót và tăng cơ hội điều trị thành công cho bệnh nhân.



Hình 22. Tổng quan về các kỹ thuật hình ảnh y tế.

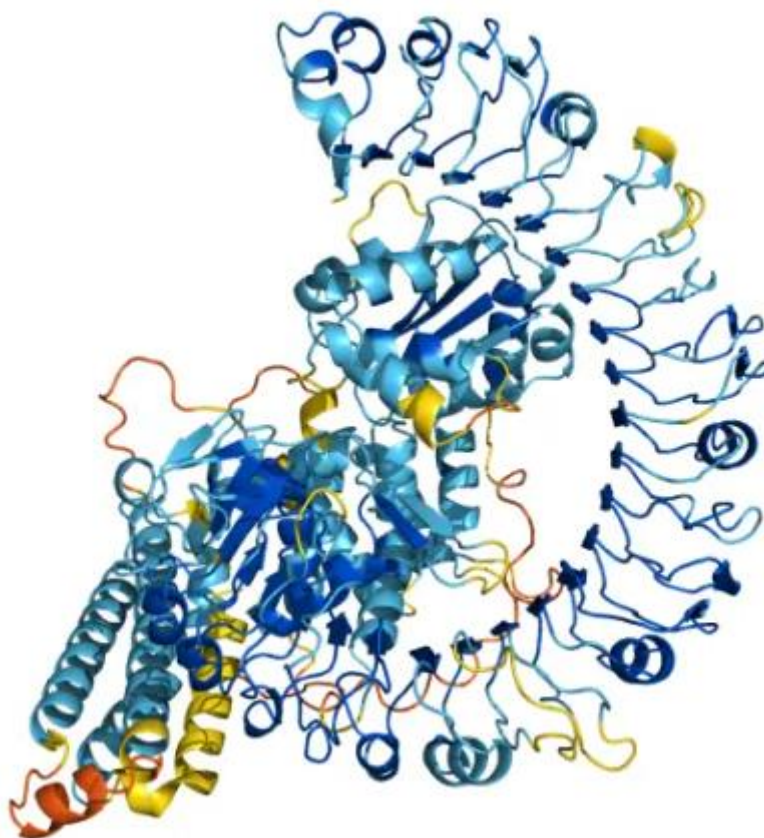
Hình trên minh họa tổng quan về các kỹ thuật hình ảnh y tế. Các kỹ thuật học sâu đã được áp dụng thành công trên các phương thức này cho các nhiệm vụ như phân vùng, phân loại, và tái tạo hình ảnh.

Phát triển Thuốc (Drug Discovery)

Một ứng dụng mang tính cách mạng khác của AI là vai trò của nó trong việc đẩy nhanh quá trình phát triển thuốc mới, đặc biệt là thông qua bài toán dự đoán cấu trúc protein. Hệ thống

AlphaFold của DeepMind là một ví dụ tiêu biểu, sử dụng các mạng nơ-ron học sâu để dự đoán cấu trúc không gian ba chiều của protein từ chuỗi axit amin của chúng với độ chính xác chưa từng có.

Trước đây, việc xác định cấu trúc protein là một quy trình đòi hỏi các kỹ thuật thực nghiệm vô cùng phức tạp, tốn kém và mất nhiều thời gian, chẳng hạn như nhiễu xạ tia X hay kính hiển vi điện tử đông lạnh (Cryo-EM). AlphaFold đã tạo ra một bước ngoặt khi có thể dự đoán cấu trúc protein với độ chính xác tương đương, thậm chí vượt trội so với các phương pháp thực nghiệm, nhưng với tốc độ nhanh hơn rất nhiều. Thành tựu này mở ra khả năng thiết kế và phát triển các liệu pháp điều trị mới một cách hiệu quả, giúp rút ngắn đáng kể thời gian và chi phí đưa một loại thuốc mới ra thị trường.



Hình 23. Cấu trúc 3D của protein kháng bệnh ở thực vật (Q8W3K0) được dự đoán bởi AlphaFold.

Hình trên minh họa cấu trúc 3D của protein kháng bệnh ở thực vật (Q8W3K0) được dự đoán bởi AlphaFold. Khả năng dự đoán chính xác cấu trúc protein là một bước đột phá, thúc đẩy mạnh mẽ quá trình khám phá và phát triển thuốc mới.

Tóm lại, trong lĩnh vực y tế, AI đang tạo ra những thay đổi sâu sắc và tích cực, từ việc nâng cao năng lực chẩn đoán đến việc cách mạng hóa quá trình nghiên cứu và phát triển thuốc. Những ứng dụng này không chỉ cho thấy tiềm năng to lớn của trí tuệ nhân tạo mà còn khẳng định vai trò của nó trong việc cải thiện chất lượng dịch vụ y tế và mang lại lợi ích thiết thực cho sức khỏe con người.

Trong giao thông vận tải

Trí tuệ nhân tạo đang có tác động mạnh mẽ và sâu rộng đến ngành giao thông vận tải, mang lại những tiến bộ quan trọng trong việc nâng cao an toàn, hiệu quả và tính tiện lợi khi di chuyển. Hai lĩnh vực nổi bật mà AI thể hiện rõ vai trò quan trọng của mình là phát triển xe tự lái và tối ưu hóa lộ trình di chuyển.

Xe tự lái (Autonomous Vehicles)

Một trong những ứng dụng nổi bật nhất của AI trong ngành giao thông vận tải là sự phát triển của các phương tiện tự lái, điển hình là hệ thống **Autopilot** của hãng xe điện **Tesla** và dự án xe tự hành của **Waymo**, công ty con thuộc Alphabet (Google). Những chiếc xe tự lái này tích hợp nhiều công nghệ AI tiên tiến như học sâu (deep learning), thị giác máy tính (computer vision), và các thuật toán xử lý dữ liệu cảm biến thời gian thực, giúp xe có khả năng nhận biết, dự đoán và phản ứng với môi trường xung quanh một cách hiệu quả và an toàn.



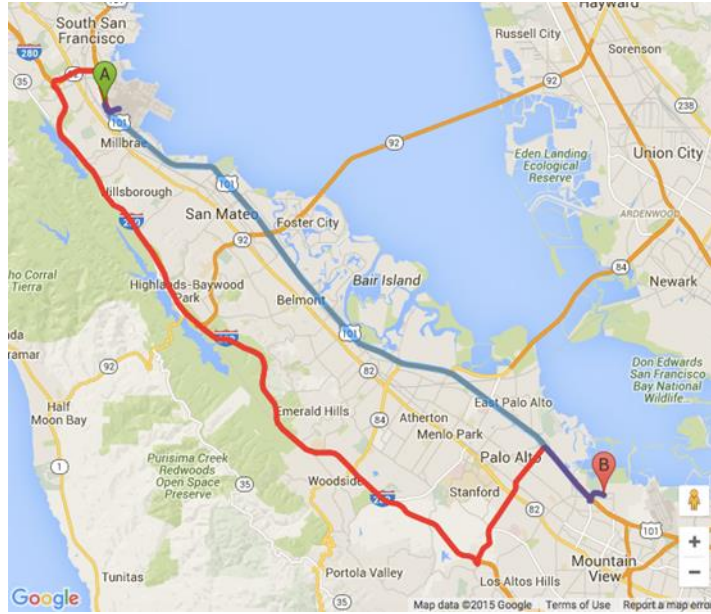
Hình 24. Xe tự lái của Tesla sánh ngang kỹ năng của Waymo dù ít phần cứng hơn hẳn.

Hình ảnh này minh họa một chiếc xe tự lái, đại diện cho các hệ thống như Autopilot của Tesla và xe tự hành của Waymo, sử dụng AI để nhận biết môi trường và điều hướng.

Hệ thống Autopilot của Tesla sử dụng camera, radar, cảm biến siêu âm, và các thuật toán AI để xử lý dữ liệu liên tục, giúp xe có khả năng tự lái, giữ làn đường, phanh tự động khi gặp chướng ngại vật, và tránh va chạm. Tương tự, xe tự hành của Waymo sử dụng các cảm biến LiDAR, radar, và hệ thống máy học mạnh mẽ để hiểu và dự đoán tình huống giao thông phức tạp, từ đó điều hướng xe một cách an toàn và đáng tin cậy trên đường phố đô thị đông đúc.

Tối ưu lộ trình (Route Optimization)

Một ứng dụng khác quan trọng và phổ biến không kém là việc sử dụng AI để tối ưu hóa lộ trình di chuyển, mà điển hình là ứng dụng bản đồ thông minh **Google Maps**. Google Maps không chỉ đơn thuần cung cấp thông tin về tuyến đường, mà còn sử dụng các thuật toán AI và dữ liệu thời gian thực để đề xuất lộ trình tối ưu nhất cho người dùng dựa trên các tiêu chí như thời gian di chuyển ngắn nhất, tránh kẹt xe, hoặc giảm thiểu tiêu hao nhiên liệu.



Hình 25. Dự đoán tương lai với API của Google Maps.

Hình ảnh này thể hiện giao diện của một ứng dụng bản đồ như Google Maps, minh họa cách AI được sử dụng để tối ưu hóa lộ trình và dự đoán tình trạng giao thông.

Công nghệ AI trong Google Maps còn có khả năng dự đoán tình trạng giao thông trong tương lai gần bằng cách phân tích dữ liệu lịch sử và dữ liệu thời gian thực từ hàng triệu người dùng. Điều này giúp người dùng tránh được những điểm ùn tắc, tiết kiệm thời gian di chuyển và cải thiện đáng kể trải nghiệm đi lại hàng ngày.

Như vậy, trong lĩnh vực giao thông vận tải, trí tuệ nhân tạo không chỉ giúp nâng cao an toàn và hiệu quả vận hành mà còn đem đến sự tiện lợi tối ưu cho người dùng. Các ví dụ như xe tự lái của Tesla và Waymo hay công cụ tối ưu hóa lộ trình Google Maps thể hiện rõ ràng những lợi ích thiết thực mà AI đang mang lại, đồng thời mở ra những triển vọng to lớn cho tương lai của ngành giao thông thông minh.

Trong tài chính & kinh doanh

Trí tuệ nhân tạo ngày càng đóng vai trò quan trọng trong lĩnh vực tài chính và kinh doanh, cung cấp các công cụ mạnh mẽ để nâng cao khả năng dự đoán, quản lý rủi ro, và tối ưu hóa quyết định kinh doanh. Hai lĩnh vực ứng dụng nổi bật của AI trong tài chính và kinh doanh là phát hiện gian lận giao dịch và giao dịch chứng khoán tự động.

Phát hiện giao dịch gian lận thẻ tín dụng (Credit Card Fraud Detection)

Một trong những ứng dụng thành công và rộng rãi nhất của AI trong lĩnh vực tài chính là việc phát hiện và ngăn chặn các giao dịch gian lận, đặc biệt trong thanh toán bằng thẻ tín dụng. Các ngân hàng và công ty tài chính sử dụng thuật toán trí tuệ nhân tạo và máy học để phân tích hàng tỷ giao dịch diễn ra mỗi ngày, giúp phát hiện các dấu hiệu bất thường hoặc gian lận trong thời gian thực.



Hình 26. Phát hiện giao dịch gian lận thẻ tín dụng (Credit Card Fraud Detection).

Hình ảnh này minh họa việc AI được sử dụng để phân tích và phát hiện các giao dịch gian lận trong lĩnh vực tài chính, giúp bảo vệ người dùng và ngân hàng.

Các hệ thống này thường áp dụng các kỹ thuật như mạng nơ-ron nhân tạo, học máy giám sát (supervised machine learning), và các mô hình xác suất để xác định và phân loại những giao dịch có nguy cơ cao. Bằng cách so sánh từng giao dịch với mô hình hành vi thông thường của người dùng, AI có thể nhanh chóng xác định những giao dịch khả nghi và cảnh báo kịp thời, từ đó giảm thiểu đáng kể thiệt hại do gian lận, bảo vệ người dùng và tổ chức tài chính.

Giao dịch chứng khoán tự động (Algorithmic Trading)

Một ứng dụng quan trọng khác của AI trong tài chính là **giao dịch chứng khoán tự động** (algorithmic trading), nơi các thuật toán trí tuệ nhân tạo được sử dụng để phân tích dữ liệu tài chính, dự đoán xu hướng thị trường, và tự động thực hiện các giao dịch dựa trên các tín hiệu xác định trước. Các thuật toán giao dịch AI sử dụng nhiều kỹ thuật như phân tích chuỗi thời gian (time-series analysis), mạng nơ-ron tái phát (recurrent neural networks - RNN), và học tăng cường (reinforcement learning) để liên tục cải thiện khả năng dự đoán và quyết định giao dịch.



Hình 27. Hình ảnh này thể hiện việc AI được sử dụng để phân tích dữ liệu thị trường và tự động thực hiện các giao dịch chứng khoán, tối ưu hóa lợi nhuận và hiệu quả.

Các công ty tài chính và các quỹ đầu tư sử dụng giao dịch tự động để thực hiện hàng ngàn giao dịch trong thời gian cực ngắn với độ chính xác cao. Điều này giúp tối đa hóa lợi nhuận, giảm thiểu sai sót do con người gây ra, và tận dụng hiệu quả các biến động nhỏ của thị trường mà người giao dịch thủ công khó có thể nhận ra. Việc ứng dụng AI vào giao dịch tự động đã góp phần làm thay đổi hoàn toàn cách thị trường tài chính hoạt động, tạo ra một cuộc cách mạng trong lĩnh vực đầu tư và quản lý tài sản.

Nhìn chung, việc ứng dụng trí tuệ nhân tạo trong lĩnh vực tài chính và kinh doanh đã giúp các tổ chức nâng cao hiệu quả hoạt động, quản lý rủi ro tốt hơn, và đưa ra những quyết định kinh doanh chính xác, kịp thời hơn bao giờ hết. Những ví dụ như phát hiện gian lận thẻ tín dụng và giao dịch chứng khoán tự động minh họa rõ ràng sức mạnh và giá trị thực tiễn của AI trong thế giới tài chính ngày nay.

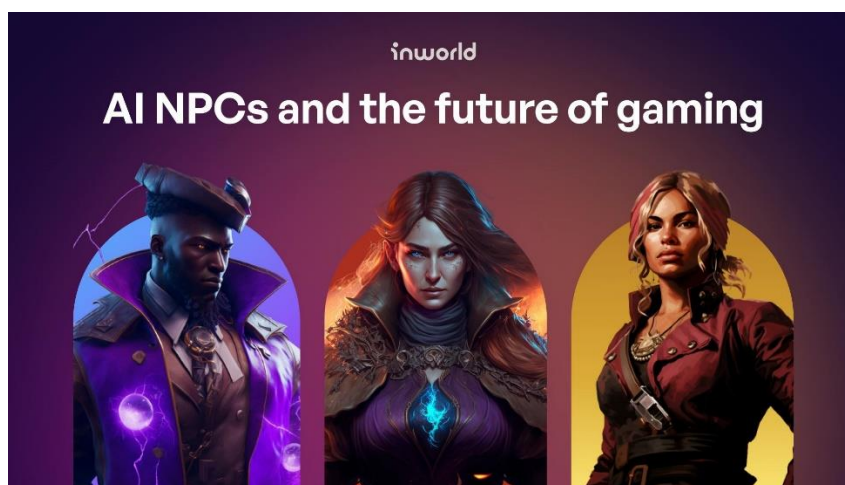
Trong giải trí

Trí tuệ nhân tạo cũng đang tạo ra những ảnh hưởng sâu sắc và rộng rãi trong ngành công nghiệp giải trí, không chỉ giúp nâng cao chất lượng sản phẩm mà còn mở ra những cơ hội sáng tạo hoàn toàn mới. Các ứng dụng nổi bật nhất của AI trong lĩnh vực giải trí có thể kể đến như điều khiển các nhân vật không phải người chơi (NPC) trong game và tạo hiệu ứng hình ảnh đặc biệt (VFX).

AI điều khiển các nhân vật không phải người chơi (NPC)

Một ứng dụng rõ nét của trí tuệ nhân tạo trong ngành công nghiệp trò chơi điện tử (video games) là điều khiển các nhân vật không phải người chơi (**Non-Player Characters - NPCs**).

Các NPC do AI điều khiển không chỉ đơn giản thực hiện những hành động lặp đi lặp lại mà ngày càng trở nên thông minh, linh hoạt và có khả năng tương tác phức tạp với người chơi.



Hình 28. AI trong điều khiển NPC trong trò chơi điện tử.

Hình ảnh trên minh họa một nhân vật trong trò chơi điện tử, đại diện cho việc AI được sử dụng để điều khiển các NPC trở nên thông minh và tương tác hơn, nâng cao trải nghiệm chơi game.

Các trò chơi hiện đại sử dụng nhiều kỹ thuật AI khác nhau như máy trạng thái hữu hạn (Finite State Machines - FSM), mạng thần kinh nhân tạo, và học tăng cường để tạo ra các NPC có khả năng thích nghi với hành vi của người chơi, đưa ra các quyết định thông minh, thậm chí học hỏi từ môi trường để cải thiện hiệu quả theo thời gian. Các NPC này có thể chiến đấu, lập chiến thuật, né tránh nguy hiểm, hoặc thậm chí hỗ trợ người chơi một cách tự nhiên và thông minh, làm tăng trải nghiệm chơi game một cách rõ rệt.

Tạo hiệu ứng hình ảnh (Visual Effects - VFX)

Một ứng dụng quan trọng khác của AI trong ngành giải trí là việc tạo ra các hiệu ứng hình ảnh đặc biệt (Visual Effects - VFX) trong điện ảnh, truyền hình và game. Các công cụ AI như mạng nơ-ron tạo sinh đối kháng (Generative Adversarial Networks - GANs) và các kỹ thuật học sâu khác giúp tạo ra hình ảnh chân thực, sống động, và ấn tượng hơn bao giờ hết.



Hình 29. Ứng dụng AI trong tạo hiệu ứng hình ảnh đặc biệt cho phim và truyền hình.

Hình ảnh trên thể hiện việc AI được ứng dụng để tạo ra các hiệu ứng hình ảnh đặc biệt trong phim ảnh và truyền hình, giúp sản xuất nội dung giải trí sống động và ấn tượng hơn.

AI hiện đang được sử dụng để tự động hóa và cải thiện nhiều khâu quan trọng trong quy trình sản xuất VFX như tạo hình nhân vật kỹ thuật số, phục dựng bối cảnh giả lập, xử lý đồ họa và hiệu ứng môi trường (ví dụ: khói, lửa, nước), hay thậm chí "làm trẻ hóa" hoặc tái tạo khuôn mặt diễn viên một cách chân thực. Các hiệu ứng này không chỉ làm tăng độ hấp dẫn thị giác của phim ảnh và game, mà còn giúp giảm đáng kể thời gian và chi phí sản xuất.

Nhìn chung, trong lĩnh vực giải trí, trí tuệ nhân tạo ngày càng đóng vai trò quan trọng, giúp nâng cao trải nghiệm người dùng, thúc đẩy sự sáng tạo, và làm thay đổi hoàn toàn cách thức sản xuất nội dung giải trí. Những ứng dụng điển hình như điều khiển NPC thông minh và tạo hiệu ứng hình ảnh VFX minh họa rõ nét khả năng vô hạn mà AI đang mang đến cho ngành công nghiệp giải trí hiện đại.

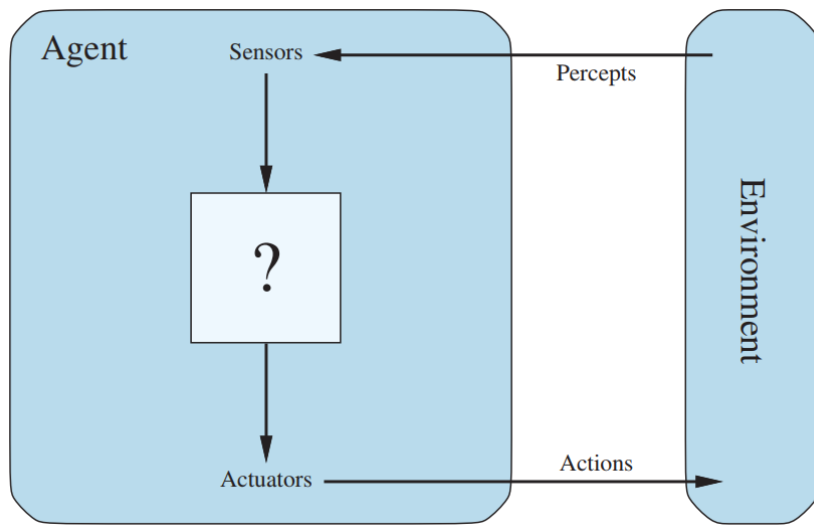
1.4. Tác tử thông minh (Intelligent Agents)

1.4.1. Giới thiệu khái niệm

Khái niệm **tác tử thông minh (Intelligent Agent)** là một trong những trụ cột lý thuyết quan trọng và hợp nhất nhất của ngành trí tuệ nhân tạo (AI) hiện đại. Nó cung cấp một lăng kính toàn diện để định nghĩa và phân tích các hệ thống thông minh, không phân biệt chúng là phần mềm, robot vật lý hay thậm chí là con người. Bằng cách mô hình hóa một hệ thống AI như một "tác tử" hoạt động trong một "môi trường" cụ thể, chúng ta có thể phân tích, so sánh và xây dựng chúng một cách có hệ thống. Cách tiếp cận này không chỉ làm rõ bản chất tương tác của AI mà còn đặt ra nền tảng vững chắc để thiết kế các hệ thống tự  (autonomous), linh hoạt và hiệu quả.

Một tác tử được định nghĩa là **bất cứ thứ gì có thể được xem là nhận biết môi trường của nó thông qua các cảm biến (sensors) và hành động lên môi trường đó thông qua các cơ cấu chấp hành (actuators).**

Mô hình tương tác nền tảng này được minh họa trong hình dưới đây:



Hình 30. Các tác nhân tương tác với môi trường thông qua cảm biến và bộ truyền động.

Mô hình tương tác cơ bản giữa một tác tử và môi trường. Tác tử sử dụng **cảm biến** để thu nhận thông tin và **cơ cấu chấp hành** để thực hiện hành động.

1.4.2. Các thành phần và thuật ngữ cốt lõi

Để hiểu sâu hơn về hoạt động của tác tử, chúng ta cần định nghĩa các thuật ngữ chính sau:

- **Cảm biến (Sensor):** Là bộ phận giúp tác tử thu thập thông tin về trạng thái của môi trường.
 - **Ví dụ:** Mắt, tai, và các cơ quan cảm giác khác của **con người**. Camera và cảm biến hồng ngoại của một **robot**. Các gói tin mạng (network packets) hay các dòng nhập liệu từ bàn phím/chuột đối với một **tác tử phần mềm**.
- **Cơ cấu chấp hành (Actuator):** Là bộ phận giúp tác tử thực hiện hành động để thay đổi trạng thái của môi trường.
 - **Ví dụ:** Tay, chân, dây thanh quản của **con người**. Các động cơ, bánh xe, cánh tay robot của một **robot**. Việc ghi tệp, gửi gói tin mạng, hiển thị thông tin ra màn hình của một **tác tử phần mềm**.
- **Môi trường (Environment):** Là không gian nơi tác tử tồn tại và hoạt động. Về lý thuyết, môi trường có thể là toàn bộ vũ trụ. Tuy nhiên, trong thực tế, nó được giới hạn trong phạm vi những yếu tố mà chúng ta quan tâm khi thiết kế tác tử—phần không gian ảnh hưởng đến những gì tác tử cảm nhận và bị ảnh hưởng bởi hành động của tác tử đó.
- **Tri giác (Percept):** Là nội dung cụ thể mà cảm biến của tác tử thu nhận được tại một thời điểm nhất định.
 - **Ví dụ:** Một hình ảnh, một âm thanh, một dòng văn bản.
- **Chuỗi tri giác (Percept Sequence):** Là lịch sử đầy đủ của mọi thứ mà tác tử đã từng tri giác. Hành động của một tác tử tại bất kỳ thời điểm nào đều được quyết định dựa trên tri thức có sẵn và toàn bộ chuỗi tri giác mà nó đã ghi nhận cho đến nay.

1.4.3. Hàm tác tử và Chương trình tác tử

Hành vi của một tác tử được mô tả một cách toán học bởi **hàm tác tử (agent function)**. Đây là một ánh xạ trừu tượng từ một chuỗi tri giác bất kỳ tới một hành động cụ thể.

$$f: P^* \rightarrow A$$

Trong đó P^* là tập hợp tất cả các chuỗi tri giác có thể có, và A là tập hợp các hành động có thể thực hiện. Về lý thuyết, hàm này định nghĩa đầy đủ hành vi của tác tử. Nếu chúng ta có thể lập một bảng mô tả hành động tương ứng với mọi chuỗi tri giác khả dĩ, chúng ta đã mô tả toàn bộ về tác tử đó. Tuy nhiên, bảng này thường có kích thước khổng lồ, thậm chí là vô hạn.

Trong khi hàm tác tử là một mô tả toán học trừu tượng, **chương trình tác tử (agent program)** là một triển khai cụ thể của hàm đó, được cài đặt và chạy trên một hệ thống vật lý (ví dụ: máy tính, robot).

Điều quan trọng là phải phân biệt rõ hai khái niệm này:

- **Hàm tác tử (Agent Function):** Là đặc tả (specification) trừu tượng về mặt lý thuyết.
- **Chương trình tác tử (Agent Program):** Là cách triển khai (implementation) cụ thể để hiện thực hóa đặc tả đó.

Nói cách khác, nhiệm vụ của các nhà nghiên cứu và kỹ sư AI là thiết kế và xây dựng **chương trình tác tử** hiệu quả nhất để hiện thực hóa **hàm tác tử** mong muốn, giúp tác tử hoạt động một cách thông minh và tối ưu trong môi trường của nó.

1.4.1. Tác tử và Môi trường (Agents and Environments)

Một **tác tử (agent)**, theo định nghĩa rộng nhất, là bất cứ thứ gì có khả năng nhận biết môi trường xung quanh (perceiving) thông qua các thiết bị cảm biến (sensors) và có thể hành động lên môi trường đó thông qua các cơ cấu chấp hành (actuators). Nói cách khác, tác tử là một hệ thống tương tác trực tiếp với môi trường bên ngoài, thu nhận thông tin từ môi trường, và phản ứng bằng cách thực hiện các hành động nhằm đạt được một mục tiêu nhất định.

Một cách khái quát, mọi hệ thống thông minh—từ sinh học đến máy móc—đều có thể được xem như một tác tử hoạt động trong một môi trường cụ thể. Ví dụ:

- **Tác tử người (Human Agent):** Con người là một tác tử điển hình với khả năng cảm nhận môi trường thông qua các giác quan như mắt (thị giác), tai (thính giác), mũi (khứu giác), và da (xúc giác). Các cơ cấu chấp hành của con người là các bộ phận cơ thể như tay, chân, miệng để thực hiện hành động tương tác với môi trường.
- **Tác tử robot (Robotic Agent):** Một robot là tác tử nhân tạo, có thể nhận biết môi trường bằng các thiết bị cảm biến như camera (thị giác máy), cảm biến hồng ngoại (nhiệt độ, khoảng cách), hoặc các cảm biến siêu âm. Các hành động của robot được thực hiện thông qua các cơ cấu chấp hành như động cơ, bánh xe, cánh tay robot hay hệ thống điều khiển chuyển động khác.

Khái niệm tác tử giúp chúng ta hiểu rõ hơn cách các hệ thống trí tuệ nhân tạo tương tác với môi trường thực tế để đạt được mục tiêu đề ra, đồng thời cung cấp một khung lý thuyết rõ ràng và hữu ích để nghiên cứu và phát triển các công nghệ AI trong thực tiễn.

1.4.2. Phân tích tác tử với mô hình PEAS

Để có thể thiết kế, mô tả và đánh giá một cách có hệ thống các tác tử thông minh, chúng ta sử dụng một khuôn khổ (framework) được gọi là **PEAS**. Đây là một công cụ phân tích cơ bản trong Trí tuệ nhân tạo, giúp định nghĩa rõ ràng vai trò và bối cảnh hoạt động của một tác tử. PEAS là viết tắt của bốn thành phần cốt lõi: **Performance measure** (Thước đo hiệu suất), **Environment** (Môi trường), **Actuators** (Cơ cấu chấp hành), và **Sensors** (Cảm biến).

Việc xác định rõ ràng các yếu tố PEAS là bước đầu tiên và quan trọng nhất trong quá trình xây dựng một tác tử. Nó không chỉ làm rõ mục tiêu cần đạt được mà còn định hình các yêu cầu về phần cứng và phần mềm, giúp các kỹ sư và nhà phát triển có một cái nhìn tổng quan và nhất quán về hệ thống cần xây dựng.

Các Thành Phần Của Mô Hình PEAS

Mỗi thành phần của PEAS đóng một vai trò riêng biệt trong việc định nghĩa tác tử:

1. Performance Measure (Thước đo hiệu suất)

Đây là một tiêu chí hoặc một tập hợp các tiêu chí khách quan để đánh giá mức độ thành công của tác tử. Thước đo hiệu suất phải xác định được “cái gì” tạo nên một hành động tốt hoặc một kết quả mong muốn, chứ không phải “làm thế nào” tác tử đạt được điều đó.

- **Đặc điểm:** Một thước đo hiệu suất tốt cần phải có thể đo lường được và không phụ thuộc vào trạng thái nội tại của tác tử. Ví dụ, thay vì đo “sự hiểu biết” của một xe tự lái, chúng ta đo các chỉ số cụ thể như **mức độ an toàn** (số lần va chạm), **tốc độ di chuyển** (thời gian đến đích), **sự thoải mái của hành khách**, và **mức tiêu thụ nhiên liệu**.
- **Ví dụ khác:**
 - **Robot hút bụi:** Mức độ sạch của sàn, lượng điện tiêu thụ, thời gian hoàn thành.
 - **Hệ thống gợi ý sản phẩm:** Tỷ lệ click vào sản phẩm, doanh thu từ các sản phẩm được gợi ý, mức độ đa dạng hóa gợi ý.

2. Environment (Môi trường)

Đây là bối cảnh mà tác tử tồn tại, nhận thức và hành động. Môi trường cung cấp thông tin cho tác tử thông qua cảm biến và bị tác động bởi tác tử thông qua cơ cấu chấp hành. Môi trường có thể là vật lý (nhà máy, đường phố) hoặc ảo (thế giới game, thị trường chứng khoán, internet).

Để phân tích sâu hơn, các môi trường thường được phân loại theo các thuộc tính sau:

- **Quan sát được (Observable):** Toàn bộ vs. Một phần.
- **Số lượng tác tử (Agents):** Một tác tử vs. Nhiều tác tử.
- **Tính xác định (Deterministic):** Xác định vs. Ngẫu nhiên (Stochastic).

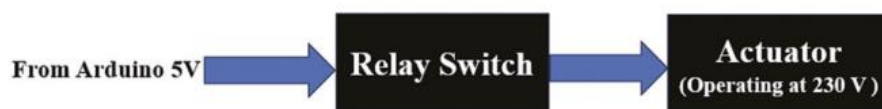
- **Tính liên tục (Episodic):** Theo từng hồi vs. Tuần tự (Sequential).
- **Tính thay đổi (Dynamic):** Tĩnh vs. Động.
- **Dữ liệu (Data):** Rời rạc vs. Liên tục.

3. Actuators (Cơ cấu chấp hành)

Đây là các bộ phận mà tác tử sử dụng để thực hiện hành động và tác động lên môi trường. Các hành động này được lựa chọn bởi “bộ não” của tác tử (chương trình tác tử) nhằm đạt được mục tiêu theo thước đo hiệu suất.

- **Ví dụ vật lý:** Động cơ, bánh xe, cánh tay robot, màn hình hiển thị, loa phát thanh.
- **Ví dụ phần mềm:** Gửi gói tin qua mạng, hiển thị một thông báo trên giao diện người dùng, thực hiện một lệnh giao dịch trên sàn chứng khoán.

Trong các hệ thống nhúng, cơ cấu chấp hành thường được điều khiển bởi một vi điều khiển như Arduino. Như minh họa trong **Hình 31**, các thiết bị yêu cầu điện áp cao như quạt, đèn nhiệt, hoặc máy bơm thường được kết nối thông qua một công tắc rơ-le (relay switch) để vi điều khiển có thể bật/tắt chúng một cách an toàn.



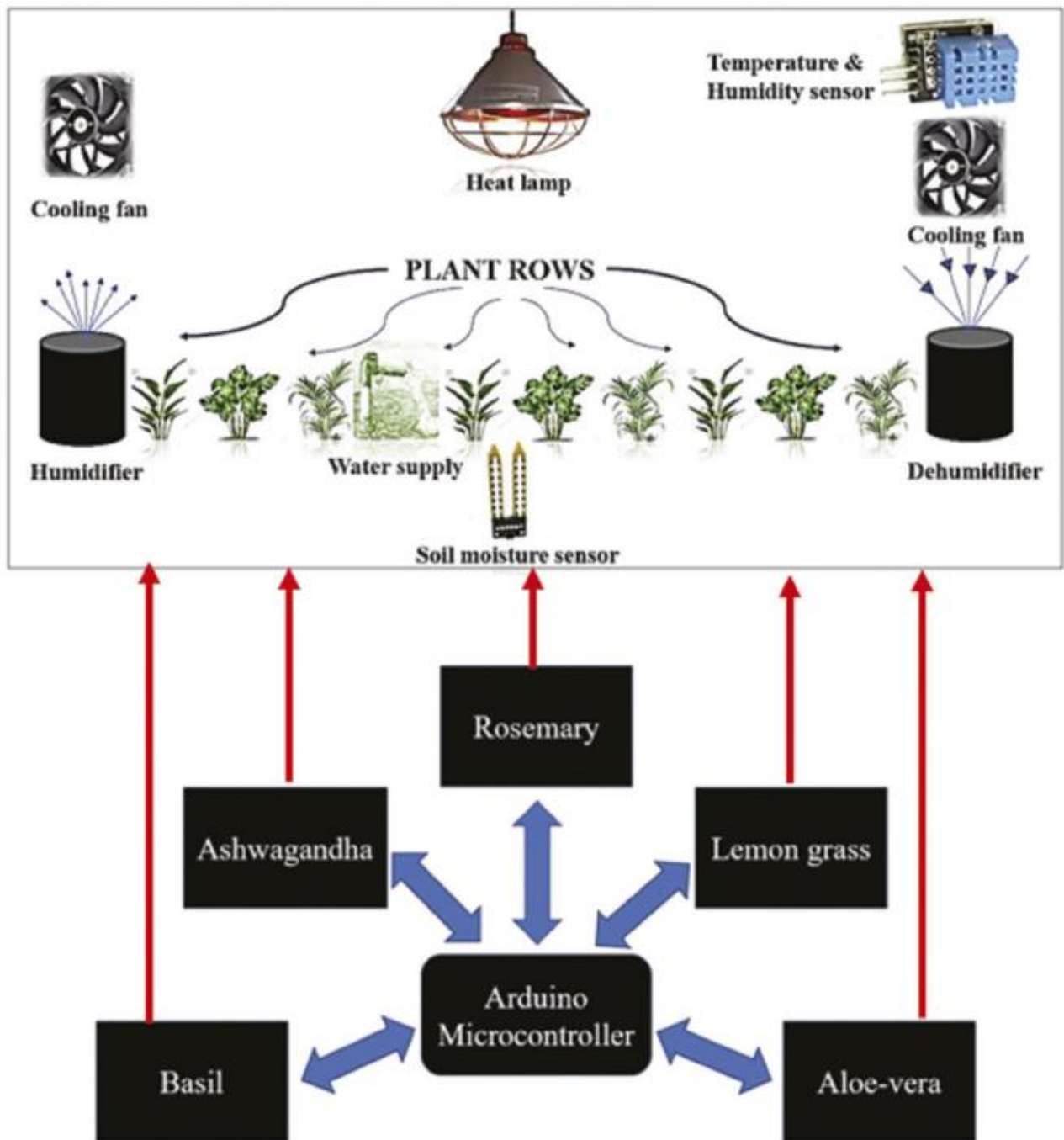
Hình 31. Sơ đồ kết nối cơ cấu chấp hành với vi điều khiển Arduino thông qua rơ-le.

4. Sensors (Cảm biến)

Đây là các bộ phận giúp tác tử thu thập thông tin và nhận thức về trạng thái hiện tại của môi trường. Dữ liệu từ cảm biến là đầu vào cho chương trình tác tử, giúp nó đưa ra quyết định hành động tiếp theo.

- **Ví dụ vật lý:** Camera, microphone, cảm biến GPS, cảm biến nhiệt độ, cảm biến hồng ngoại.
- **Ví dụ phần mềm:** Nhận tín hiệu từ bàn phím, đọc dữ liệu từ một API, phân tích một gói tin mạng.

Quá trình hoạt động của một tác tử thông minh có thể được hình dung như một chu trình khép kín: cảm biến thu thập dữ liệu từ môi trường, chương trình tác tử xử lý dữ liệu này và ra quyết định, sau đó cơ cấu chấp hành thực thi hành động, làm thay đổi môi trường, và chu trình lại tiếp tục. **Hình 32** dưới đây mô tả một ví dụ điển hình về một hệ thống tự động (tác tử) tương tác với môi trường của nó.



Hình 32. Sơ đồ luồng hoạt động của một vườn ươm cây thuốc và cây hương liệu tự động.

Bài Tập Ví Dụ: Phân Tích Hệ Thống Đèn Đường Thông Minh

Để vận dụng mô hình PEAS, chúng ta hãy cùng phân tích một ví dụ thực tế: **Hệ thống đèn đường thông minh**.

Thành phần PEAS	Mô tả chi tiết
Performance Measure (Thước đo hiệu suất)	Tối ưu hóa năng lượng: Giảm thiểu điện năng tiêu thụ. Đảm bảo an toàn: Cung cấp đủ ánh sáng cho người và phương tiện, giảm thiểu tai nạn giao thông. Tăng tuổi thọ thiết bị: Giảm thời gian hoạt động

Thành phần PEAS	Mô tả chi tiết
	không cần thiết. Giảm chi phí vận hành: Tự động hóa việc bảo trì và giám sát.
Environment (Môi trường)	Đường phố, khu đô thị, cao tốc. <i>Phân loại môi trường:</i> • Một phần quan sát được: Một cảm biến chỉ bao quát được một khu vực nhất định. • Nhiều tác tử: Có nhiều phương tiện, người đi bộ, và các đèn thông minh khác. • Ngẫu nhiên: Sự xuất hiện của người và xe là không thể đoán trước hoàn toàn. • Động: Môi trường thay đổi liên tục khi tác tử đang ra quyết định (xe đang di chuyển). • Tuần tự: Hành động hiện tại (bật/tắt đèn) sẽ ảnh hưởng đến các quyết định trong tương lai.
Actuators (Cơ cấu chấp hành)	• Bóng đèn LED: Có khả năng điều chỉnh cường độ sáng (dimming) hoặc bật/tắt. • Hệ thống giao tiếp: Gửi tín hiệu đến các đèn lân cận hoặc trung tâm điều khiển.
Sensors (Cảm biến)	• Cảm biến chuyển động (Motion/PIR sensors): Phát hiện sự di chuyển của người và xe. • Cảm biến ánh sáng (Light sensors/LDR): Đo cường độ ánh sáng tự nhiên để xác định trời tối hay sáng. • Camera: Phân tích hình ảnh để xác định mật độ và loại phương tiện. • Microphone: Phát hiện âm thanh bất thường (ví dụ: tiếng va chạm).

Thông qua việc phân tích PEAS, chúng ta không chỉ xác định rõ các thành phần cấu thành nên hệ thống đèn đường thông minh mà còn hiểu sâu hơn về bối cảnh và thách thức mà nó phải đối mặt. Đây là nền tảng để lựa chọn thuật toán điều khiển phù hợp và đánh giá hiệu quả của hệ thống một cách khoa học.

1.4.3. Phân Loại và Đặc Tính Của Các Môi Trường Tác Vụ

Trong lĩnh vực Trí tuệ nhân tạo, hiệu năng của một tác tử thông minh không chỉ được quyết định bởi kiến trúc và thuật toán nội tại của nó, mà còn phụ thuộc mật thiết vào bản chất của **môi trường tác vụ (task environment)** mà nó được triển khai. Việc phân tích và định danh chính xác các thuộc tính của môi trường là một bước đi nền tảng, có ý nghĩa quyết định trong việc lựa chọn mô hình thiết kế, xác định độ phức tạp của bài toán và định hướng chiến lược giải quyết. Một tác tử được tối ưu hóa cho một môi trường có đặc tính này có thể hoạt động kém hiệu quả, thậm chí thất bại hoàn toàn khi được đặt trong một môi trường có đặc tính khác. Do đó, việc khảo sát các chiều phân loại của môi trường là một yêu cầu mang tính tiên quyết.

Một trong những thuộc tính cơ bản nhất là **mức độ quan sát được (observability)**. Một môi trường được định nghĩa là **quan sát được toàn phần (fully observable)** nếu tại mọi thời điểm, hệ thống cảm biến của tác tử có khả năng thu nhận đầy đủ mọi dữ liệu cần thiết để tái tạo lại chính xác và hoàn chỉnh trạng thái của môi trường. Các trò chơi bàn cờ như cờ vua hay cờ vây là những ví dụ kinh điển, nơi tác tử có thể “nhìn thấy” toàn bộ trạng thái của cuộc chơi. Ngược lại, hầu hết các bài toán trong thế giới thực đều thuộc môi trường **quan sát được một phần (partially observable)**. Trong môi trường này, tác tử phải đối mặt với tình trạng thông tin không đầy đủ do nhiều cảm biến, dữ liệu bị che khuất, hoặc giới hạn vật lý của bộ phận cảm biến. Xe tự lái là một minh chứng rõ ràng: nó không thể biết được ý định của các tài xế khác hay quan sát một phương tiện bị che khuất hoàn toàn. Để hoạt động hiệu quả, tác tử trong môi

trường này buộc phải duy trì một **trạng thái nội tại (internal state)** hoặc một **trạng thái niềm tin (belief state)**, sử dụng lịch sử quan sát để suy luận và ước lượng về các thành phần ẩn của môi trường.

Tiếp theo, môi trường được phân biệt bởi **số lượng tác tử (number of agents)**. Trong môi trường **đơn tác tử (single-agent)**, chỉ có một thực thể duy nhất ra quyết định và hành động. Các bài toán như giải đố ô chữ hay robot hút bụi trong một căn phòng trống thuộc loại này. Tuy nhiên, các hệ thống phức tạp hơn thường vận hành trong môi trường **đa tác tử (multi-agent)**, nơi nhiều tác tử cùng tồn tại, tương tác và ảnh hưởng lẫn nhau. Sự tương tác này có thể mang tính **cạnh tranh (competitive)**, như trong các trò chơi đối kháng hoặc giao dịch trên thị trường tài chính, hoặc mang tính **hợp tác (cooperative)**, như một đội robot cộng tác trong nhà kho hoặc một phi đội máy bay không người lái. Thiết kế tác tử cho môi trường đa tác tử đòi hỏi các kỹ thuật phức tạp hơn, yêu cầu tác tử phải có khả năng mô hình hóa và dự đoán hành vi của các tác tử khác, một lĩnh vực nghiên cứu cốt lõi của lý thuyết trò chơi (game theory).

Tính tất định (determinism) của môi trường cũng là một yếu tố quan trọng. Một môi trường được xem là **tất định (deterministic)** nếu trạng thái kế tiếp của nó được xác định một cách duy nhất bởi trạng thái hiện tại và hành động được tác tử lựa chọn. Trong khi đó, môi trường **ngẫu nhiên (stochastic)** lại chứa đựng yếu tố bất định, khiến cho cùng một hành động trong cùng một trạng thái có thể dẫn đến các kết quả khác nhau. Nếu một tác tử điều khiển một trò chơi mô phỏng không có yếu tố ngẫu nhiên, nó hoạt động trong môi trường tất định. Nhưng khi tác tử đó điều khiển một robot vật lý, nơi hành động có thể bị ảnh hưởng bởi ma sát, nhiễu động cơ, hay các yếu tố ngoại cảnh không lường trước, môi trường đó trở thành ngẫu nhiên. Điều này buộc tác tử phải có khả năng xử lý sự không chắc chắn, thường thông qua các mô hình xác suất và kỹ thuật học tăng cường.

Một khía cạnh khác là **bản chất của thời gian và chuỗi hành động**. Trong môi trường **theo từng hồi (episodic)**, trải nghiệm của tác tử được phân chia thành các đoạn riêng biệt, độc lập gọi là “hồi”. Ở mỗi hồi, tác tử nhận thức và hành động, và quyết định trong hồi này không ảnh hưởng đến các hồi tiếp theo. Ví dụ điển hình là một hệ thống phân loại hình ảnh, nơi việc phân loại một ảnh không liên quan đến ảnh kế tiếp. Ngược lại, môi trường **tuần tự (sequential)** có tính phụ thuộc thời gian mạnh mẽ; một quyết định ở hiện tại sẽ có hệ quả kéo dài, ảnh hưởng đến các trạng thái và lựa chọn trong tương lai. Lái xe và chơi cờ là các môi trường tuần tự điển hình, đòi hỏi tác tử phải có khả năng “nhìn xa trông rộng” và lập kế hoạch chiến lược dài hạn.

Tính động (dynamics) của môi trường cũng ảnh hưởng lớn đến thiết kế tác tử. Môi trường được gọi là **tĩnh (static)** nếu nó không tự thay đổi trong khi tác tử đang trong quá trình xử lý thông tin và ra quyết định. Ngược lại, môi trường **động (dynamic)** có thể thay đổi trạng thái một cách độc lập với tác tử. Giao thông trên đường phố là một môi trường động vì các phương tiện khác vẫn di chuyển ngay cả khi xe tự lái đang tính toán lộ trình. Tác tử trong môi trường động phải đối mặt với áp lực thời gian, cân cân bằng giữa việc tìm ra hành động tối ưu và việc phải phản ứng đủ nhanh trước khi môi trường thay đổi theo hướng bất lợi. Một trường hợp trung gian là môi trường **bán động (semidynamic)**, nơi bản thân môi trường không đổi nhưng thước đo hiệu suất của tác tử lại phụ thuộc vào thời gian, ví dụ như chơi cờ vua tính giờ.

Cuối cùng, môi trường được phân loại dựa trên **kiểu dữ liệu (data type)** là **rời rạc (discrete)** hay **liên tục (continuous)**. Thuộc tính này áp dụng cho không gian trạng thái, hành động, và

cách biểu diễn thời gian. Các trò chơi bàn cờ có không gian trạng thái và hành động rời rạc. Trong khi đó, các bài toán điều khiển robot, mô phỏng vật lý, hay lái xe tự động lại hoạt động trong không gian liên tục, nơi các biến như vị trí, vận tốc, và góc lái nhận các giá trị thực. Việc xử lý không gian liên tục thường đòi hỏi các phương pháp toán học và thuật toán phức tạp hơn so với không gian rời rạc.

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

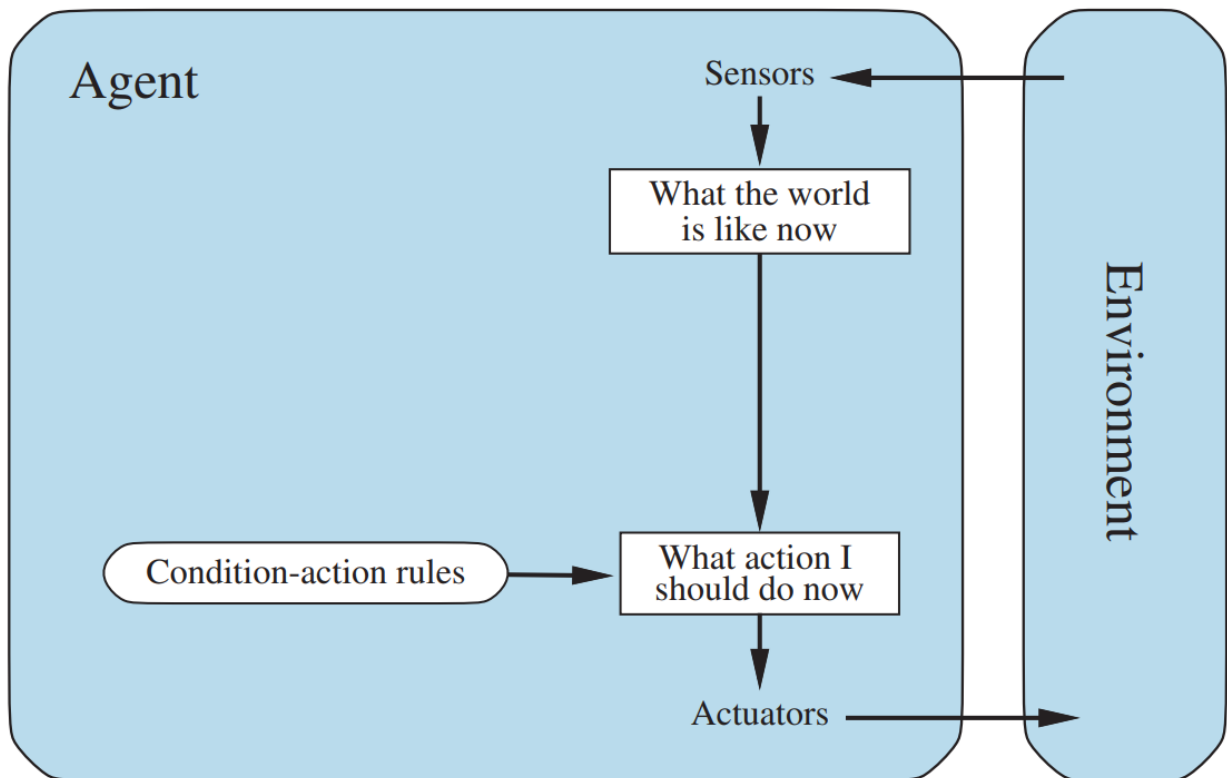
Hình 33. Ví dụ về môi trường tác vụ và các đặc điểm của chúng.

Việc hệ thống hóa các đặc điểm này, như được tổng hợp trong các bảng phân loại môi trường tác vụ, cung cấp một bộ khung khái niệm vững chắc, cho phép các kỹ sư AI phân tích một cách có hệ thống bất kỳ bài toán nào và lựa chọn kiến trúc tác tử phù hợp nhất, từ các tác tử phản xạ đơn giản cho đến các hệ thống dựa trên mục tiêu, hữu dụng và học tập phức tạp.

1.4.4. Cấu trúc các chương trình tác tử

Trong lĩnh vực Trí tuệ nhân tạo, các chương trình tác tử có thể được phân loại theo nhiều cấu trúc khác nhau, từ đơn giản đến phức tạp, tùy thuộc vào yêu cầu của nhiệm vụ. Cấu trúc cơ bản nhất trong số này là **tác tử phản xạ đơn giản (simple reflex agent)**. Các tác tử này hoạt động dựa trên một nguyên tắc rất trực tiếp: chúng chọn hành động chỉ dựa vào **tri giác hiện tại (current percept)**, hoàn toàn bỏ qua lịch sử các tri giác đã thu nhận trước đó.

Cơ chế cốt lõi của một tác tử phản xạ đơn giản là một tập hợp các **luật điều kiện - hành động (condition-action rules)**, thường được biểu diễn dưới dạng *IF-THEN*. Khi cảm biến của tác tử thu nhận thông tin về trạng thái thế giới, nó tạo ra một tri giác. Tri giác này sau đó được đối chiếu với các luật đã định sẵn. Ngay khi một luật có phần điều kiện khớp với trạng thái hiện tại được tìm thấy, hành động tương ứng sẽ được kích hoạt và thực thi thông qua các bộ truyền động. Quá trình này, được minh họa trong sơ đồ kiến trúc ở **Hình 43**, thể hiện một luồng xử lý trực tiếp từ cảm nhận đến hành động mà không cần thông qua bất kỳ bộ nhớ hay trạng thái nội bộ nào.



Hình 34. Sơ đồ nguyên lý của một tác nhân phản xạ đơn giản.

Một ví dụ kinh điển là hệ thống phanh khẩn cấp tự động (AEB) trên xe hơi. Hệ thống này sử dụng camera và radar làm cảm biến để liên tục theo dõi khoảng cách đến vật cản phía trước. Tri giác ở đây là dữ liệu về khoảng cách và tốc độ tương đối. Hệ thống tuân theo một luật đơn giản: *IF* khoảng cách quá gần và có nguy cơ va chạm, *THEN* kích hoạt phanh. Quyết định này được đưa ra ngay lập tức mà không cần biết chiếc xe đã đi đâu hay điều kiện giao thông trước đó như thế nào.

Mặc dù có kiến trúc gọn nhẹ và khả năng phản ứng cực kỳ nhanh chóng, các tác tử phản xạ đơn giản lại bộc lộ những hạn chế đáng kể. Chính sự phụ thuộc hoàn toàn vào tri giác hiện tại khiến chúng hoạt động kém hiệu quả trong các môi trường **quan sát được một phần (partially observable)**, nơi cần có thông tin về quá khứ để đưa ra quyết định tối ưu. Chẳng hạn, một tác tử không có bộ nhớ có thể dễ dàng bị mắc kẹt trong các vòng lặp vô tận. Hơn nữa, khi môi trường trở nên phức tạp, việc xây dựng và quản lý một tập hợp luật *IF-THEN* khổng lồ sẽ trở nên bất khả thi. Do những nhược điểm này, để giải quyết các bài toán phức tạp hơn, cần đến những kiến trúc tác tử tiên tiến hơn có khả năng duy trì một **trạng thái nội bộ (internal state)** để theo dõi sự thay đổi của thế giới.

Ví dụ cụ thể: Hệ thống phanh khẩn cấp tự động trên ô tô

Một ví dụ điển hình về tác tử phản xạ đơn giản là hệ thống phanh khẩn cấp tự động (**automatic emergency braking system**) trong xe hơi. Hệ thống này sử dụng các cảm biến như radar hoặc camera để liên tục theo dõi phía trước của xe. Khi phát hiện có vật cản bất ngờ ngay trước mặt xe (điều kiện), hệ thống sẽ lập tức kích hoạt phanh tự động (hành động), bất kể trạng thái trước đó là gì.

Cấu trúc IF-THEN rõ ràng của hệ thống này có thể được trình bày như sau:

- **Điều kiện (IF):** Cảm biến phát hiện vật cản ở khoảng cách nguy hiểm ngay phía trước.
- **Hành động (THEN):** Kích hoạt hệ thống phanh khẩn cấp để tránh va chạm.

Mặc dù đơn giản, tác tử phản xạ đơn giản như vậy vẫn rất hiệu quả trong nhiều trường hợp, đặc biệt là trong các tình huống cần phản ứng nhanh và tức thời. Tuy nhiên, việc không có khả năng ghi nhớ và học hỏi cũng đồng nghĩa với việc các tác tử này không thể xử lý hiệu quả các tình huống phức tạp hoặc môi trường thay đổi liên tục và khó dự đoán. Các cấu trúc tác tử tiên tiến hơn sẽ bổ sung thêm những khả năng đó.

Tác tử phản xạ dựa trên mô hình (Model-Based Reflex Agents)

Để khắc phục hạn chế cốt lõi của tác tử phản xạ đơn giản—sự thất bại trong các môi trường quan sát được một phần (partially observable environments)—chúng ta cần một cấu trúc tác tử có khả năng duy trì và cập nhật nhận thức về thế giới. Đây chính là mục tiêu của **tác tử phản xạ dựa trên mô hình (model-based reflex agent)**. Thay vì chỉ phản ứng với tri giác hiện tại, loại tác tử này xây dựng và duy trì một **trạng thái nội tại (internal state)**, là tập hợp các niềm tin của nó về trạng thái thực của môi trường. Trạng thái nội tại này cho phép tác tử “ghi nhớ” các khía cạnh của thế giới mà nó không thể cảm nhận trực tiếp tại một thời điểm nhất định.

Để duy trì một trạng thái nội tại chính xác, tác tử cần có kiến thức về cách thế giới vận hành. Kiến thức này được mã hóa trong một **mô hình (model)**. Một mô hình trả lời hai câu hỏi quan trọng:

- **Thế giới tự tiến hóa như thế nào? (How the world evolves)** - Mô hình mô tả các quy luật vật lý hoặc logic của môi trường, giúp dự đoán trạng thái tiếp theo sẽ ra sao ngay cả khi không có sự can thiệp của tác tử.
- **Hành động của tôi ảnh hưởng đến thế giới ra sao? (What my actions do)** - Mô hình cũng dự báo kết quả từ các hành động của chính tác tử.

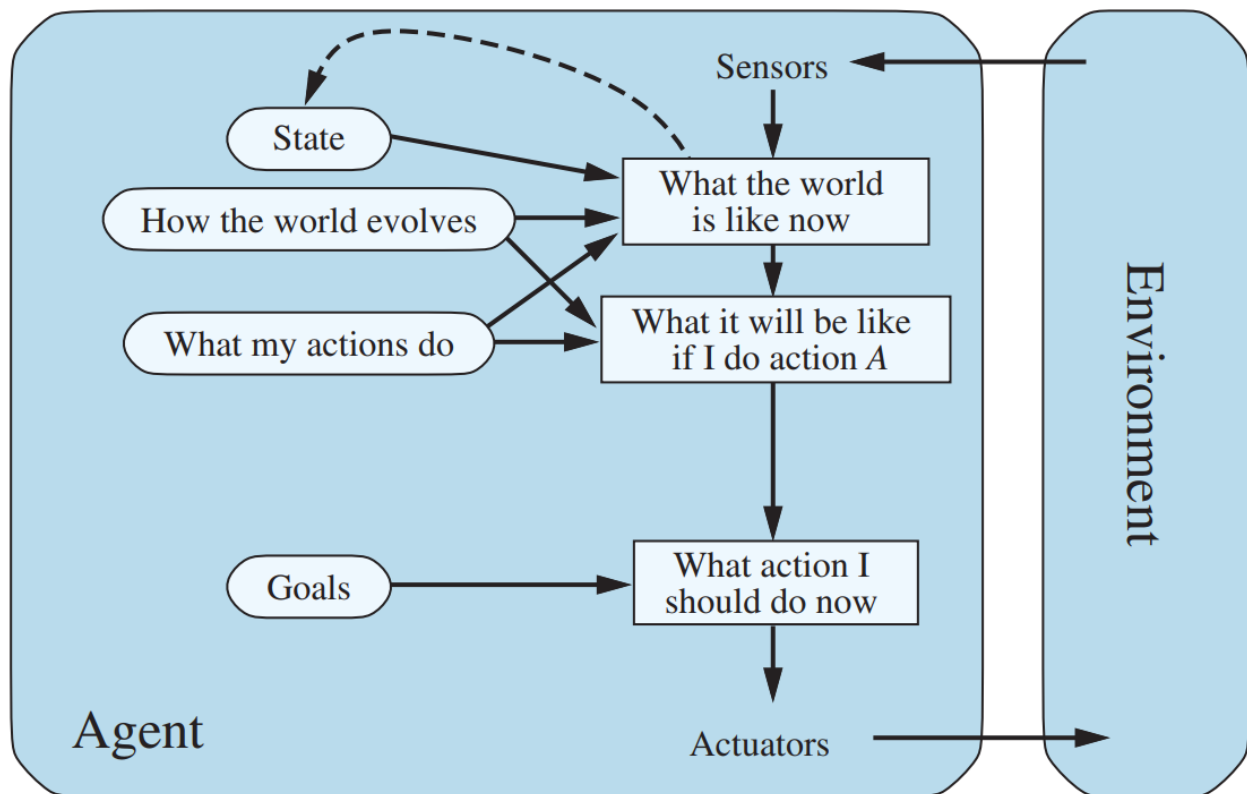
Bằng cách kết hợp trạng thái nội tại trước đó với tri giác hiện tại, tác tử sử dụng mô hình của mình để suy luận ra một bức tranh cập nhật và đầy đủ hơn về trạng thái hiện tại của thế giới.

Cấu trúc và quy trình hoạt động

Kiến trúc của một tác tử phản xạ dựa trên mô hình, như minh họa trong **Hình 35**, là một sự mở rộng của tác tử phản xạ đơn giản. Quy trình ra quyết định của nó phức tạp hơn và diễn ra như sau:

- **Cảm biến (Sensors)** thu nhận tri giác từ môi trường.
- Hàm **UPDATE-STATE** được gọi để cập nhật trạng thái nội tại. Hàm này nhận đầu vào là trạng thái cũ, hành động vừa thực thi, tri giác mới nhất, và mô hình thế giới. Dựa trên mô hình, nó tính toán ra trạng thái nội tại mới.
- Sau khi có trạng thái nội tại cập nhật (mô tả “thế giới hiện tại như thế nào”), tác tử sẽ sử dụng tập **luật điều kiện-hành động (rules)** để tìm ra hành động phù hợp, tương tự như tác tử phản xạ đơn giản.

- Hành động được chọn sẽ được gửi đến **bộ truyền động (actuators)** để thực thi.



Hình 35. Cấu trúc tác tử phản xạ dựa trên mô hình.
(Nguồn: Dựa theo Russell & Norvig, 2021).

Hình trên là sơ đồ cấu trúc của một tác tử phản xạ dựa trên mô hình. Nó sử dụng một trạng thái nội tại (state) và một mô hình (model) về thế giới để theo dõi các khía cạnh không quan sát được của môi trường, từ đó đưa ra quyết định hành động.

Giải thuật giả mã cho tác tử này có thể được mô tả như sau:

```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current belief of the state of the world
               model, a description of how the next state depends on current state and action
               rules, a set of condition-action rules
               action, the most recent action, initially none

  state ← UPDATE-STATE(state, action, percept, model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Ví dụ cụ thể: Xe tự lái khi theo đuôi xe khác

Hãy xem xét một chiếc xe tự lái (tác tử) đang theo sau một phương tiện khác.

- Tri giác (Percept):** Dữ liệu từ camera và LiDAR về khoảng cách, hình dạng và tốc độ của xe phía trước.

- **Trạng thái nội tại (Internal state):** Vị trí, vận tốc, hướng di chuyển của xe phía trước và của chính nó.
- **Mô hình (Model):** Mô hình bao gồm kiến thức rằng “các phương tiện có xu hướng tiếp tục di chuyển theo hướng và vận tốc hiện tại” và “nếu tôi đạp phanh, vận tốc của xe tôi sẽ giảm theo một gia tốc nhất định”.

Kịch bản: Xe phía trước bị một chiếc xe tải lớn che khuất trong vài giây.

- **Tác tử phản xạ đơn giản:** Khi không còn tri giác về xe phía trước, nó sẽ cho rằng xe đó đã biến mất và có thể đưa ra quyết định sai lầm (ví dụ: tăng tốc).
- **Tác tử phản xạ dựa trên mô hình:** Mặc dù tri giác từ camera bị gián đoạn, tác tử vẫn duy trì trạng thái nội tại. Nó sử dụng mô hình chuyển động để **dự đoán** vị trí tiếp theo của chiếc xe bị che khuất. Trạng thái nội tại của nó được cập nhật thành “xe phía trước có lẽ đang ở vị trí X với vận tốc Y, hiện đang bị che khuất”. Dựa trên trạng thái được cập nhật này, xe tự lái vẫn quyết định duy trì khoảng cách an toàn, sẵn sàng phanh khi chiếc xe kia xuất hiện trở lại.

Như vậy, tác tử phản xạ dựa trên mô hình thể hiện một bước tiến quan trọng, cho phép tác tử hành động một cách thông minh và an toàn trong những môi trường phức tạp và không chắc chắn, nơi thông tin không phải lúc nào cũng đầy đủ.

Tác tử dựa trên mục tiêu (Goal-Based Agents)

Mặc dù tác tử phản xạ dựa trên mô hình có khả năng duy trì một bức tranh nội tại về thế giới, nó vẫn thiếu một yếu tố quan trọng: một định hướng rõ ràng. Nó biết thế giới hiện tại như thế nào và hành động của nó sẽ gây ra điều gì, nhưng nó không có cơ sở để lựa chọn giữa các trạng thái tương lai khác nhau. Ví dụ, tại một ngã tư, việc đi thẳng, rẽ trái hay rẽ phải đều là những hành động hợp lệ theo mô hình, nhưng đâu là hành động “tốt nhất”? Để trả lời câu hỏi này, chúng ta cần một **tác tử dựa trên mục tiêu (goal-based agent)**.

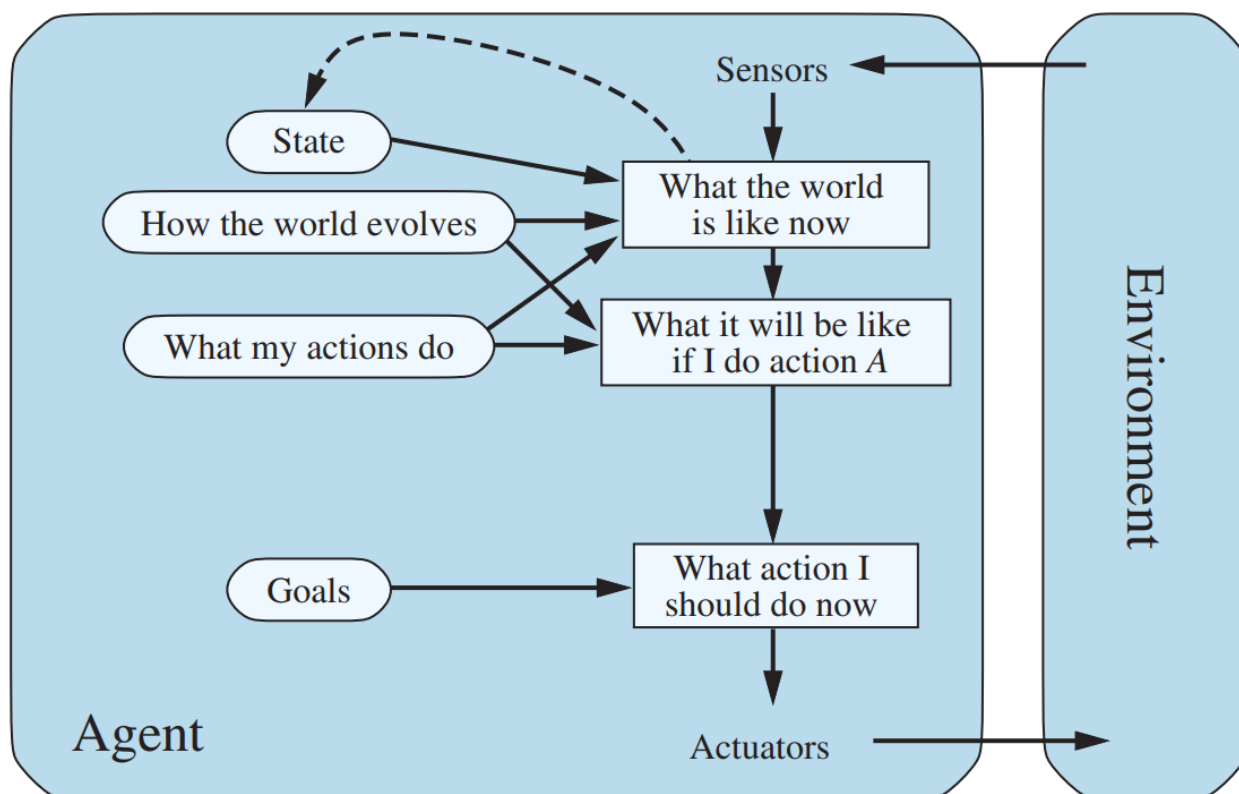
Loại tác tử này là một bước tiến về khả năng tư duy, vì ngoài trạng thái nội tại và mô hình thế giới, nó còn được cung cấp thông tin về **mục tiêu (goal)**. Mục tiêu là một tập hợp các tình huống hoặc trạng thái thế giới mà tác tử mong muốn đạt được. Việc có một mục tiêu rõ ràng cho phép tác tử không chỉ chọn hành động dựa trên trạng thái hiện tại, mà còn có thể đưa ra các quyết định hướng tới tương lai, nhằm đạt được các trạng thái mong muốn đó. Thay vì trả lời câu hỏi “Tôi nên làm gì bây giờ?”, nó trả lời câu hỏi “Hành động nào sẽ đưa tôi đến gần mục tiêu của mình?”.

Cơ chế hoạt động: Tìm kiếm và Lập kế hoạch

Sự khác biệt cơ bản trong hoạt động của tác tử dựa trên mục tiêu nằm ở cách nó chọn hành động. Thay vì chỉ sử dụng các luật điều kiện-hành động một cách trực tiếp, nó phải dự đoán kết quả của các chuỗi hành động khác nhau để tìm ra chuỗi nào sẽ dẫn đến mục tiêu. Quá trình này được gọi là **tìm kiếm và lập kế hoạch (search and planning)**. Đây là một lĩnh vực cốt lõi trong Trí tuệ nhân tạo, nơi các thuật toán được thiết kế để phân tích các con đường khả thi và tìm ra một lời giải.

Cấu trúc của tác tử này, như minh họa trong **Hình 36**, là một sự mở rộng của tác tử dựa trên mô hình. Sau khi cập nhật trạng thái nội tại, tác tử sẽ sử dụng thông tin về mục tiêu để xem xét

các chuỗi hành động tiềm năng. Ví dụ, nó có thể tự hỏi: “Điều gì sẽ xảy ra nếu tôi rẽ trái rồi đi thẳng?” hoặc “Điều gì sẽ xảy ra nếu tôi đi thẳng?”. Bằng cách sử dụng mô hình thế giới để dự báo kết quả của những chuỗi hành động này, nó có thể tìm ra một kế hoạch dẫn đến mục tiêu và thực thi bước đầu tiên của kế hoạch đó.



Hình 36. Sơ đồ cấu trúc của một tác tử dựa trên mô hình và mục tiêu.

Hình trên là sơ đồ cấu trúc của một tác tử dựa trên mô hình và mục tiêu. Nó mở rộng tác tử dựa trên mô hình bằng cách bổ sung một “mục tiêu” (goal), cho phép tác tử sử dụng các quy trình tìm kiếm và lập kế hoạch để chọn hành động dẫn đến các trạng thái mong muốn.

Ví dụ cụ thể: Robot tìm đường trong mê cung

Hãy xem xét một robot có nhiệm vụ tìm đường thoát khỏi một mê cung phức tạp.

- **Trạng thái nội tại & Mô hình:** Robot duy trì vị trí và hướng hiện tại của nó. Mô hình của nó bao gồm bản đồ các bức tường đã biết và quy tắc di chuyển (ví dụ: hành động “tiến_lên” sẽ thay đổi tọa độ (x, y) theo hướng hiện tại, trừ khi có tường cản).
- **Mục tiêu (Goal):** Một trạng thái cụ thể, ví dụ: $vị_trí = (cửa_ra_x, cửa_ra_y)$.
- **Tìm kiếm và Lập kế hoạch:** Khi được đặt vào mê cung, robot không di chuyển ngẫu nhiên. Thay vào đó, nó kích hoạt một thuật toán tìm kiếm, chẳng hạn như A^* (A-star). Thuật toán này sẽ “khám phá” các con đường trên bản đồ nội tại của robot, tính toán chi phí (quãng đường đã đi) và ước lượng chi phí còn lại để đến đích. Nó sẽ tìm ra đường đi ngắn nhất từ vị trí hiện tại đến cửa ra.

- **Hành động:** Robot sẽ thực hiện hành động đầu tiên trong chuỗi hành động tối ưu mà thuật toán A* đã tìm thấy (ví dụ: “rẽ_phải”). Sau khi di chuyển, nó sẽ lặp lại quá trình lập kế hoạch từ vị trí mới.

Ưu điểm và Tính linh hoạt

Ưu điểm lớn nhất của tác tử dựa trên mục tiêu là **tính linh hoạt (flexibility)**. Các hành vi của nó có thể dễ dàng được thay đổi bằng cách thay đổi mục tiêu. Nếu chúng ta muốn robot trong mê cung đi đến một căn phòng chứa pin sạc thay vì cửa ra, chúng ta chỉ cần cung cấp cho nó một mục tiêu mới: $vị_trí = (trạm_sạc_x, trạm_sạc_y)$. Toàn bộ bộ máy lập kế hoạch sẽ tự động tìm ra một lộ trình mới để đạt được mục tiêu đó mà không cần phải lập trình lại bất kỳ quy tắc hành động nào.

Tuy nhiên, quá trình ra quyết định của tác tử dựa trên mục tiêu thường chậm hơn và đòi hỏi tính toán nhiều hơn so với các tác tử phản xạ, vì quá trình tìm kiếm và lập kế hoạch có thể rất phức tạp. Dù vậy, khả năng đưa ra các quyết định có tính toán và hướng tới tương lai xa giúp chúng giải quyết được một lớp bài toán rộng lớn hơn nhiều.

Tác tử dựa trên độ hữu dụng (Utility-Based Agents)

Các tác tử dựa trên mục tiêu, mặc dù có khả năng lập kế hoạch dài hạn, vẫn gặp phải một hạn chế quan trọng: mục tiêu thường là một khái niệm nhị phân (đạt được hoặc không đạt được). Điều này không đủ để xử lý các tình huống phức tạp trong thực tế, nơi có nhiều con đường để đạt được cùng một mục tiêu, nhưng một số con đường lại “tốt hơn” những con đường khác (nhanh hơn, an toàn hơn, hoặc ít tốn kém hơn). Hơn nữa, tác tử có thể phải đối mặt với nhiều mục tiêu mâu thuẫn nhau (ví dụ: muốn đến nơi nhanh nhất nhưng với chi phí thấp nhất). Để giải quyết sự tinh vi này, chúng ta cần đến cấu trúc tác tử tổng quát và mạnh mẽ nhất: **tác tử dựa trên độ hữu dụng (utility-based agent)**.

Thay vì chỉ phân biệt giữa trạng thái mục tiêu và không phải mục tiêu, tác tử dựa trên độ hữu dụng sử dụng một **hàm hữu dụng (utility function)** để lượng hóa mức độ “đáng mong muốn” hay “hạnh phúc” của một trạng thái. Hàm này ánh xạ một trạng thái (hoặc một chuỗi các trạng thái) tới một số thực, cung cấp một thước đo chi tiết về mức độ ưu tiên. Điều này cho phép tác tử không chỉ tìm cách đạt được mục tiêu mà còn tối ưu hóa chất lượng của quá trình đó.

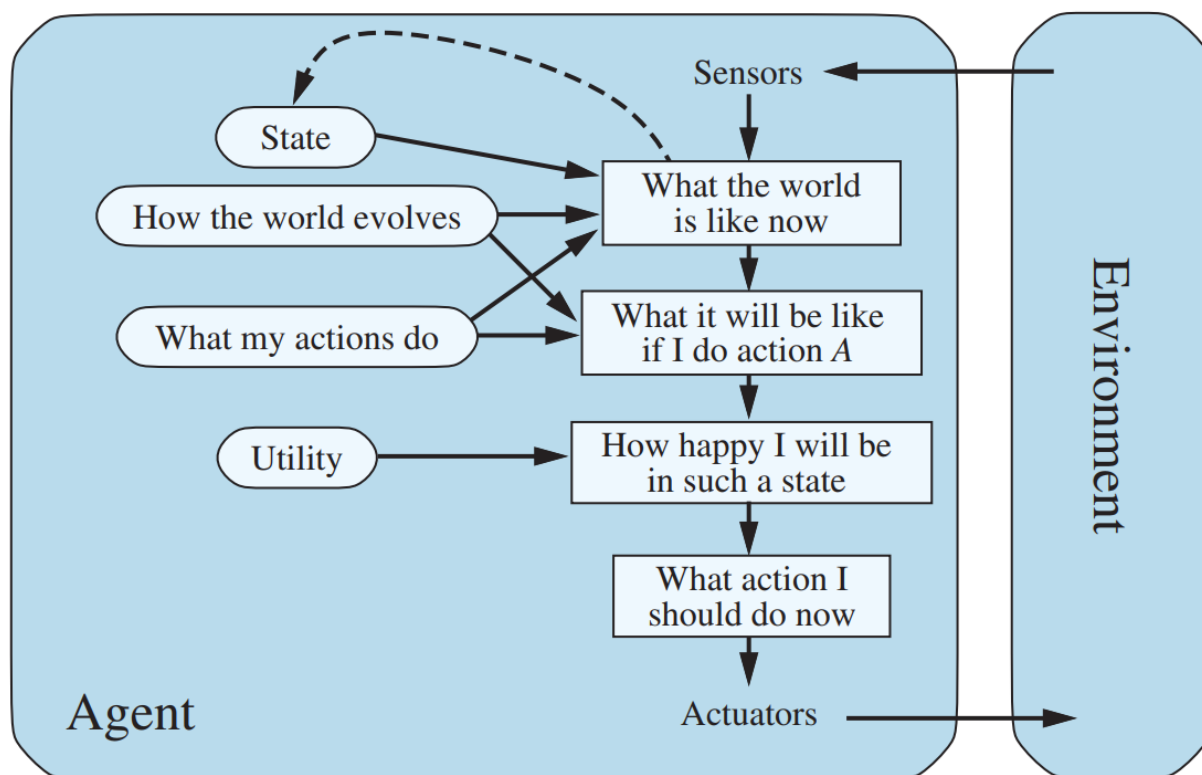
Cơ chế hoạt động: Tối đa hóa Độ hữu dụng Kỳ vọng

Trong một thế giới không chắc chắn, hành động của tác tử không phải lúc nào cũng dẫn đến một kết quả duy nhất đã định. Do đó, tác tử không thể chỉ chọn hành động dẫn đến trạng thái có độ hữu dụng cao nhất, vì nó không chắc chắn sẽ đạt được trạng thái đó. Thay vào đó, một tác tử hữu dụng hợp lý sẽ ra quyết định dựa trên nguyên tắc **tối đa hóa độ hữu dụng kỳ vọng (Maximum Expected Utility - MEU)**.

Độ hữu dụng kỳ vọng của một hành động được tính bằng tổng trọng số của độ hữu dụng của tất cả các kết quả có thể xảy ra, trong đó trọng số chính là xác suất xảy ra của mỗi kết quả đó. Công thức có thể biểu diễn như sau:

$$E[U(action)] = \sum_{result} P(result | action) \times U(result)$$

Bằng cách tính toán giá trị này cho mọi hành động khả thi, tác tử có thể chọn ra hành động mang lại cho nó sự thỏa mãn trung bình cao nhất trong dài hạn, có tính đến mọi rủi ro và cơ hội. Kiến trúc của tác tử này, như minh họa trong **Hình 37**, cho thấy thành phần “mục tiêu” đã được thay thế bằng một “hàm hữu dụng” tổng quát hơn để đánh giá các trạng thái.



Hình 37. Sơ đồ cấu trúc của một tác tử dựa trên mô hình và độ hữu dụng.

Hình trên là sơ đồ cấu trúc của một tác tử dựa trên mô hình và độ hữu dụng. Tác tử này sử dụng một mô hình thế giới để dự đoán các kết quả có thể xảy ra, sau đó áp dụng một hàm hữu dụng để đánh giá mức độ đáng mong muốn của những kết quả đó, nhằm chọn ra hành động tối đa hóa độ hữu dụng kỳ vọng.

Ví dụ cụ thể: Hệ thống gợi ý tuyến đường thông minh

Hãy xem xét một ứng dụng điều hướng GPS như Google Maps khi gợi ý tuyến đường cho người dùng.

- **Mục tiêu đơn giản:** Tìm đường đi ngắn nhất từ A đến B.
- **Bài toán thực tế phức tạp hơn:** Người dùng không chỉ quan tâm đến khoảng cách. Họ còn quan tâm đến thời gian di chuyển (có thể kẹt xe), chi phí (phí cầu đường), và thậm chí cả sự an toàn (tránh các khu vực nguy hiểm).

Một tác tử dựa trên độ hữu dụng sẽ giải quyết bài toán này như sau:

- **Xác định các trạng thái kết quả:** Các tuyến đường khác nhau (Tuyến 1, Tuyến 2, Tuyến 3).
- **Xây dựng hàm hữu dụng:** Hàm này là một công thức kết hợp nhiều yếu tố, ví dụ:

$$Utility(tuyến_đường) = w_1 \times (thời_gian_ước_tính) + w_2 \times (chi_phí_xăng_dầu +$$

$phí_cầu_đường) + w_3 \times (mức_độ_an_toàn_tuyến_đường)$ Trong đó, w_1, w_2, w_3 là các trọng số thể hiện mức độ ưu tiên của người dùng (ví dụ, một người đang vội sẽ có w_1 rất cao).

- **Xử lý sự không chắc chắn:** Thời gian di chuyển không phải là một con số cố định. Tác tử sử dụng mô hình giao thông (dữ liệu lịch sử và thời gian thực) để tính toán **xác suất** của các khoảng thời gian di chuyển khác nhau cho mỗi tuyến đường.
- **Tối đa hóa độ hữu dụng kỳ vọng:** Hệ thống sẽ tính toán độ hữu dụng kỳ vọng cho mỗi tuyến đường bằng cách nhân độ hữu dụng của mỗi kịch bản (ví dụ: đến nơi trong 20 phút, 25 phút, 30 phút) với xác suất tương ứng của kịch bản đó. Tuyến đường có độ hữu dụng kỳ vọng cao nhất sẽ được gợi ý cho người dùng.

Bằng cách này, tác tử có thể đưa ra một quyết định cân bằng tinh tế giữa các mục tiêu mâu thuẫn, cung cấp một giải pháp thực sự tối ưu và được cá nhân hóa. Cấu trúc tác tử dựa trên độ hữu dụng cung cấp một khuôn khổ hoàn chỉnh và tổng quát cho việc xây dựng các tác tử thông minh và hợp lý.

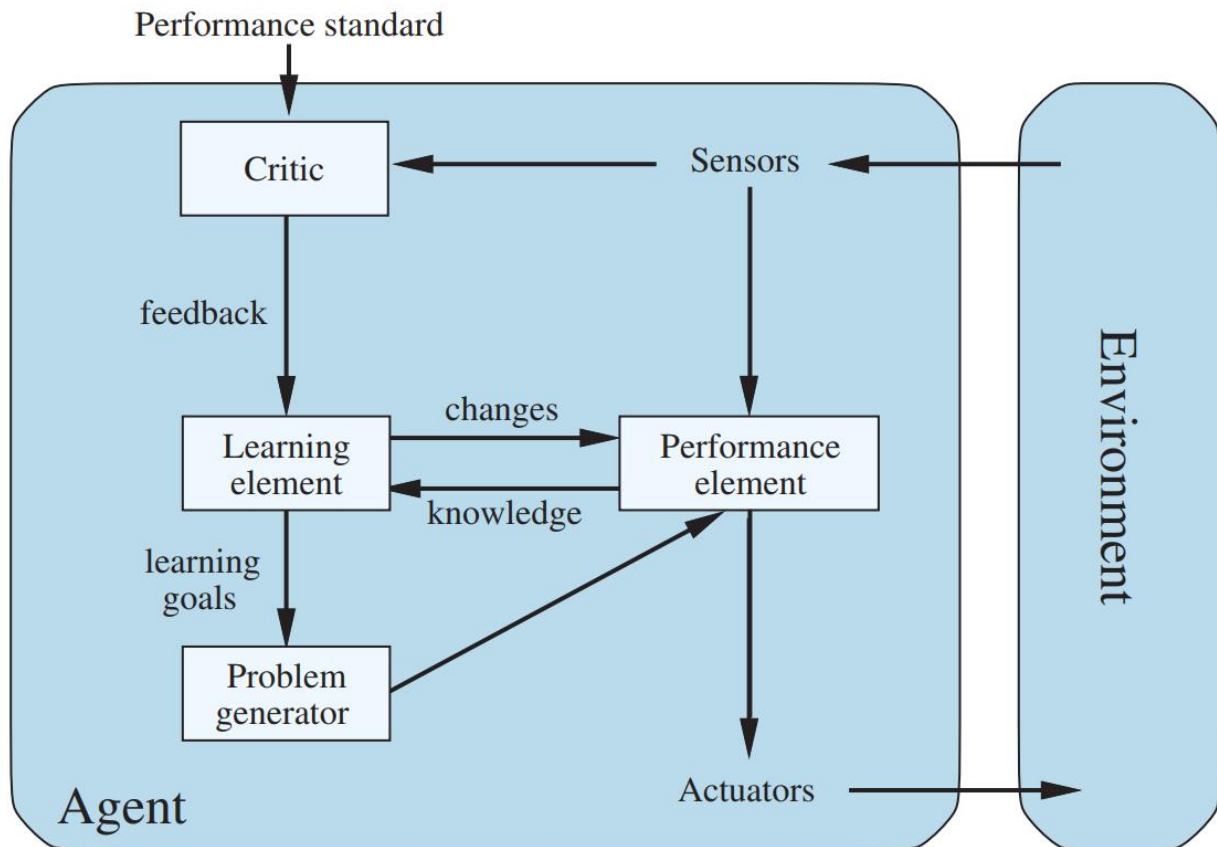
1.4.5. Tác tử học hỏi (Learning Agents)

Các cấu trúc tác tử mà chúng ta đã thảo luận—từ phản xạ đơn giản đến dựa trên độ hữu dụng—đều được thiết kế với một bộ kiến thức ban đầu cố định. Trí thông minh của chúng được lập trình sẵn và không thay đổi theo thời gian. Tuy nhiên, một trong những dấu hiệu của trí thông minh thực sự là khả năng thích ứng và tự hoàn thiện. Để đạt được sự tự chủ (autonomy) ở mức độ cao nhất, một tác tử phải có khả năng hoạt động hiệu quả trong các môi trường chưa xác định và học hỏi từ kinh nghiệm. Đây chính là vai trò của **tác tử học hỏi (learning agent)**.

Về bản chất, một tác tử học hỏi không phải là một kiến trúc hoàn toàn mới, mà là một cấu trúc meta được xây dựng bao quanh một tác tử hiện có. Cấu trúc này được chia thành hai phần chính:

- **Phần tử thực thi (Performance Element):** Đây chính là một trong các loại tác tử mà chúng ta đã biết (ví dụ: một tác tử dựa trên độ hữu dụng), chịu trách nhiệm lựa chọn và thực hiện các hành động trong môi trường dựa trên kiến thức hiện tại của nó.
- **Phần tử học hỏi (Learning Element):** Đây là thành phần chịu trách nhiệm cải tiến Phần tử thực thi bằng cách điều chỉnh kiến thức của nó dựa trên kinh nghiệm.

Quá trình học hỏi này được hiện thực hóa thông qua một vòng lặp tương tác giữa bốn thành phần riêng biệt, như được minh họa trong **Hình 38**.



Hình 38. Sơ đồ kiến trúc của một tác tử học hỏi tổng quát.

Hình trên là sơ đồ kiến trúc của một tác tử học hỏi tổng quát. Nó bao gồm một Phần tử thực thi để tương tác với môi trường và một vòng lặp học hỏi gồm Nhà phê bình, Phần tử học hỏi và Bộ phát sinh vấn đề để liên tục cải thiện hiệu suất.

Vòng lặp học hỏi

Quá trình tự cải thiện của tác tử diễn ra liên tục qua các bước sau:

- **Nhà phê bình (Critic):** Sau khi Phần tử thực thi thực hiện một hành động, Nhà phê bình sẽ quan sát thế giới và so sánh kết quả với một thước đo hiệu suất đã định trước. Nó cung cấp một **tín hiệu phản hồi (feedback)** cho Phần tử học hỏi, cho biết hành động đó tốt hay xấu và ở mức độ nào.
- **Phần tử học hỏi (Learning Element):** Đây là trái tim của quá trình học. Nó nhận phản hồi từ Nhà phê bình và sử dụng thông tin đó để sửa đổi các thành phần kiến thức bên trong Phần tử thực thi. Ví dụ, nó có thể cập nhật mô hình thế giới, điều chỉnh hàm hữu dụng, hoặc thay đổi các quy tắc hành động để lần sau Phần tử thực thi sẽ đưa ra quyết định tốt hơn trong tình huống tương tự. Toàn bộ lĩnh vực Học máy (Machine Learning) chính là nghiên cứu cách thiết kế các Phần tử học hỏi hiệu quả.
- **Bộ phát sinh vấn đề (Problem Generator):** Thành phần này chịu trách nhiệm khuyến khích sự **khám phá (exploration)**. Thay vì luôn chọn hành động được cho là tốt nhất, nó đề xuất các hành động mới hoặc thử nghiệm để tác tử thu thập kinh nghiệm về những khía cạnh chưa biết của môi trường. Điều này rất quan trọng để tránh việc tác tử bị mắc kẹt trong một chiến lược tốt nhưng không phải là tối ưu.

Ví dụ cụ thể: Hệ thống lọc thư rác (Spam Filter)

Hãy phân tích một hệ thống lọc thư rác thông minh qua lăng kính của tác tử học hỏi:

- **Môi trường:** Luồng email đến và các hành động của người dùng (đọc, xóa, đánh dấu là spam).
- **Phân tử thực thi:** Một bộ phân loại email hiện có. Nó sử dụng các quy tắc và trọng số đã học để tự động chuyển email vào thư mục “Hộp thư đến” hoặc “Thư rác”.
- **Nhà phê bình:** Khi người dùng di chuyển một email từ “Hộp thư đến” sang “Thư rác”, Nhà phê bình ghi nhận đây là một lỗi của Phân tử thực thi. Nó tạo ra một tín hiệu phản hồi tiêu cực cho Phân tử học hỏi. Ngược lại, khi người dùng chuyển một email ra khỏi mục “Thư rác”, đó là một tín hiệu phản hồi tích cực.
- **Phân tử học hỏi:** Nhận tín hiệu lỗi, nó sẽ phân tích email bị phân loại sai đó. Sử dụng một thuật toán học máy (ví dụ: cập nhật trọng số của mạng nơ-ron), nó điều chỉnh các quy tắc của bộ phân loại—chẳng hạn, tăng trọng số cho một số từ khóa hoặc giảm độ tin cậy của một địa chỉ gửi nhất định—để giảm khả năng mắc phải lỗi tương tự trong tương lai.
- **Bộ phát sinh vấn đề:** Trong trường hợp này, nó có thể là một cơ chế thỉnh thoảng yêu cầu người dùng xác nhận về một email “ngghi ngờ” có đặc điểm mới lạ, nhằm chủ động thu thập thêm dữ liệu huấn luyện và cải thiện mô hình.

Tóm lại, tác tử học hỏi đại diện cho đỉnh cao của sự tự chủ trong Trí tuệ nhân tạo, cho phép các hệ thống không chỉ hành động hợp lý dựa trên những gì chúng biết, mà còn liên tục mở rộng và hoàn thiện kiến thức đó thông qua tương tác với thế giới thực.

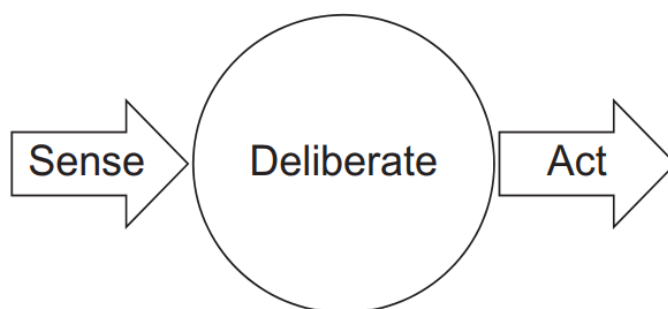
CHƯƠNG 2: GIẢI QUYẾT VẤN ĐỀ BẰNG TÌM KIẾM

Trong buổi học này, sinh viên sẽ được tiếp cận với việc mô hình hóa bài toán dưới dạng không gian trạng thái – một kỹ thuật quan trọng trong trí tuệ nhân tạo để biểu diễn các bài toán cần giải quyết. Từ đó, các thuật toán tìm kiếm mù cơ bản như Tìm kiếm theo chiều rộng (Breadth-First Search - BFS) và Tìm kiếm theo chiều sâu (Depth-First Search - DFS) sẽ được giới thiệu và triển khai thực tế. Song song với phần lý thuyết, sinh viên sẽ thực hành sử dụng các cấu trúc dữ liệu List và Tuple trong ngôn ngữ lập trình Python để cài đặt và kiểm thử các thuật toán tìm kiếm này. Mục tiêu chính của buổi học là giúp sinh viên chuyển đổi một bài toán thực tế thành mô hình không gian trạng thái, hiểu rõ nguyên lý hoạt động và biết cách triển khai hai thuật toán tìm kiếm mù BFS và DFS, đồng thời vận dụng thành thạo các cấu trúc dữ liệu list và tuple trong Python để biểu diễn và thao tác với các trạng thái trong quá trình tìm kiếm.

2.1. Bài toán và Không gian Trạng thái (Problem and State Space)

Một tác tử giải quyết vấn đề (problem-solving agent) hoạt động một cách có phương pháp thông qua một chu trình gồm ba giai đoạn tuần tự, đây là một quy trình **tìm kiếm ngoại tuyến (offline search)** vì toàn bộ quá trình tính toán để tìm ra giải pháp được hoàn tất trước khi thực hiện bất kỳ hành động nào trong thế giới thực. Chu trình này, được minh họa trong **Hình 39**, bao gồm:

- **Định hình (Formulate):** Giai đoạn đầu tiên, trong đó tác tử phân tích nhiệm vụ và môi trường để xây dựng một mô hình trừu tượng của bài toán.
- **Tìm kiếm (Search):** Dựa trên mô hình đã định hình, tác tử sẽ “tưởng tượng” ra các chuỗi hành động khác nhau để tìm ra một lộ trình (một giải pháp) từ trạng thái hiện tại đến trạng thái mục tiêu.
- **Thực thi (Execute):** Sau khi tìm thấy một giải pháp, tác tử sẽ thực hiện chuỗi hành động đó trong môi trường thực tế.



Hình 39. Chu trình ba giai đoạn hoạt động của một tác tử giải quyết vấn đề: Định hình bài toán, Tìm kiếm giải pháp, và Thực thi giải pháp.

2.1.1. Tư duy giải quyết vấn đề của tác tử (Problem-Solving Thought of a Goal-Based Agent)

Bước **Định hình** là bước quan trọng nhất, vì nó chuyển một vấn đề thực tế phức tạp thành một cấu trúc hình thức mà máy tính có thể xử lý. Cấu trúc này được gọi là **bài toán không gian trạng thái**. Việc trừu tượng hóa này đòi hỏi chúng ta phải bỏ qua những chi tiết không liên quan và chỉ tập trung vào những yếu tố cốt lõi.

Một bài toán được định nghĩa chính thức bởi năm thành phần sau:

- **Không gian Trạng thái (State Space):** Một tập hợp tất cả các trạng thái khả dĩ mà môi trường có thể tồn tại. Mỗi **trạng thái (state)** là một bản mô tả cô đọng về thế giới tại một thời điểm.
- **Trạng thái Ban đầu (Initial State):** Trạng thái xuất phát của tác tử.
- **Hành động (Actions):** Một mô tả về các hành động mà tác tử có thể thực hiện. *Actions(s)* là một hàm trả về tập các hành động có thể thực hiện được trong trạng thái *s*.
- **Mô hình Chuyển tiếp (Transition Model):** Một hàm *Result(s, a)* trả về trạng thái kế tiếp khi thực hiện hành động *a* trong trạng thái *s*. Mô hình này cùng với không gian trạng thái và các hành động định nghĩa nên một đồ thị, trong đó các đỉnh là trạng thái và các cạnh là hành động.
- **Kiểm tra Mục tiêu (Goal Test):** Một điều kiện xác định xem một trạng thái cho trước có phải là trạng thái mục tiêu hay không.
- **Chi phí Đường đi (Path Cost):** Một hàm gán một chi phí số cho một đường đi. Thông thường, chi phí của một đường đi là tổng chi phí của các hành động trên đường đi đó.

Ví dụ: Robot trong mê cung

Hãy áp dụng định nghĩa trên cho bài toán robot tìm đường ra khỏi mê cung:

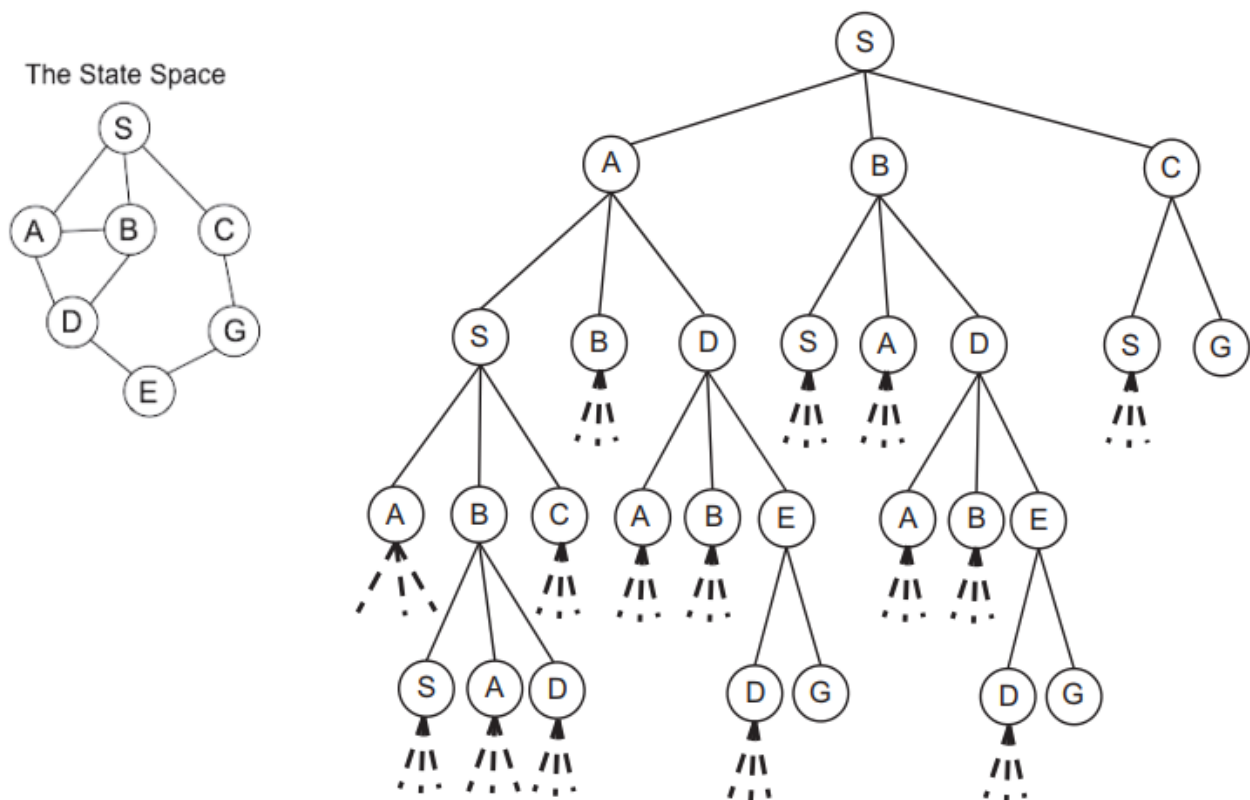
- **Trạng thái:** Tọa độ (x, y) của robot trong mê cung.
- **Trạng thái Ban đầu:** Tọa độ ô xuất phát, ví dụ $(1, 1)$.
- **Hành động:** Tập $\{Lên, Xuống, Trái, Phải\}$.
- **Mô hình Chuyển tiếp:** *Result((x, y), Lên)* sẽ trả về $(x, y+1)$ nếu ô đó không phải là tường.
- **Kiểm tra Mục tiêu:** *GoalTest((x, y))* trả về *true* nếu (x, y) là tọa độ của ô lối ra.
- **Chi phí Đường đi:** Mỗi bước di chuyển có chi phí là 1.

Từ Không gian Trạng thái đến Cây Tìm kiếm

Không gian trạng thái là một biểu diễn đồ thị của toàn bộ bài toán. Tuy nhiên, một thuật toán tìm kiếm không khám phá toàn bộ không gian này cùng một lúc. Thay vào đó, nó xây dựng một **cây tìm kiếm (search tree)** một cách dần dần, bắt đầu từ trạng thái ban đầu và mở rộng các nút bằng cách áp dụng các hành động khả thi. Gốc của cây là trạng thái ban đầu, và các nhánh của cây được hình thành bởi các chuỗi hành động.

Điều quan trọng cần phân biệt là một trạng thái trong không gian trạng thái có thể xuất hiện nhiều lần trong cây tìm kiếm, tương ứng với các đường đi khác nhau để đến trạng thái đó.

Hình 40 minh họa rõ sự chuyển đổi này.



Hình 40. Không gian trạng thái và cây tìm kiếm trong quá trình giải bài toán.

Hình trên là một không gian trạng thái (bên trái) là một đồ thị của tất cả các trạng thái. Một thuật toán tìm kiếm sẽ khám phá không gian này bằng cách xây dựng một cây tìm kiếm (bên phải) từ trạng thái bắt đầu (S) để tìm đường đến trạng thái mục tiêu (G).

Bằng cách định hình vấn đề theo cấu trúc này, chúng ta có thể áp dụng các thuật toán tìm kiếm tổng quát để tìm ra một **giải pháp (solution)**—một chuỗi hành động dẫn từ trạng thái ban-đầu đến trạng thái mục-tiêu. Quá trình này là nền tảng cho vô số ứng dụng AI, từ việc lập kế hoạch cho robot, giải các câu đố, đến việc tìm đường đi tối ưu trong các hệ thống logistics.

2.1.2. Định nghĩa một bài toán có cấu trúc

Để triển khai các thuật toán tìm kiếm một cách chặt chẽ, trước tiên ta phải **mô tả bài toán** dưới dạng “có cấu trúc”, bao gồm năm thành phần chính sau:

- **Trạng thái ban đầu (Initial State):** Trạng thái xuất phát s_0 của tác tử, mô tả hoàn chỉnh tình huống ban đầu trong thế giới bài toán. Ví dụ trong mê cung, s_0 là tọa độ ô khởi đầu; trong bài toán 8-puzzle, s_0 là cấu hình ban đầu của các ô.
- **Hành động (Actions):** Tập hợp các hành động khả dĩ A . Với mỗi trạng thái s , hàm $ACTIONS(s)$ trả về danh sách hành động hợp lệ tại s .
 - Trong mê cung, $ACTIONS(s)$ có thể trả về $\{\text{Lên, Xuống, Trái, Phải}\}$ nếu không gặp tường.

- Trong 8-puzzle, $ACTIONS(s)$ là các lần dịch chuyển miếng đệm (blank) sang vị trí liền kề.
- **Hàm chuyển tiếp (Transition Model):** Hàm $RESULT(s, a)$ mô tả trạng thái kế tiếp thu được khi thực hiện hành động a ở trạng thái s .
 - Ví dụ, nếu $s = (x, y)$ trong mê cung và $a = \text{Lên}$, thì $RESULT(s, a) = (x - 1, y)$ (nếu ô đó không bị chặn).
 - Trong 8-puzzle, $RESULT(s, a)$ là cấu hình mới khi hoán vị ô trống với ô liền kề theo hành động.
- **Kiểm tra đích (Goal Test):** Một hàm boolean $GOAL_{TEST}(s)$ trả về **true** nếu s thuộc tập trạng thái đích.
 - Trong mê cung: $GOAL_{TEST}(s)$ kiểm tra xem s có phải là ô thoát không.
 - Trong 8-puzzle: $GOAL_{TEST}(s)$ kiểm tra cấu hình s có trùng với cấu hình mục tiêu hay không.
- **Chi phí đường đi (Path Cost):** Hàm chi phí $g(n)$ gán giá trị cho mỗi đường đi từ trạng thái khởi đầu tới nút n . Thông thường, chi phí là tổng chi phí của từng hành động trên đường đi đó:

$$g(n) = \sum_{i=1}^k c(s_{i-1}, a_i)$$

trong đó $c(s, a)$ là chi phí thực hiện hành động a ở trạng thái s .

- Ví dụ, trong mê cung đơn giản, $c(s, a) = 1$ cho mỗi bước di chuyển, nên $g(n)$ chính là số bước đã đi.
- Trong trường hợp các bước di chuyển có trọng số khác nhau (ví dụ ô đường khó đi tốn nhiều chi phí hơn), $c(s, a)$ sẽ phản ánh điều đó.

Ví dụ minh họa: Bài toán mê cung

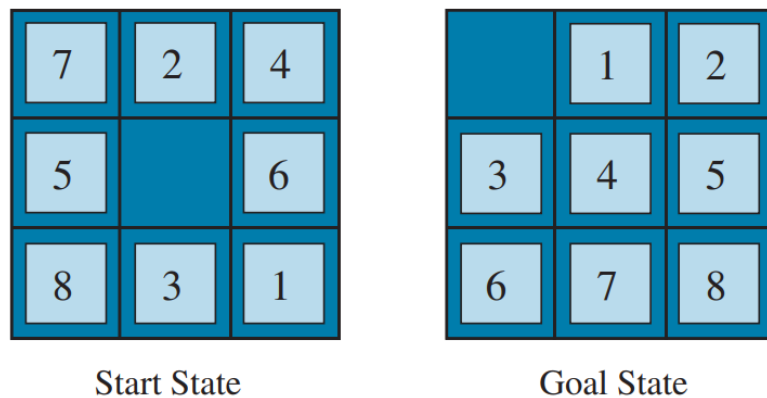
- **Initial State:** $s_0 = (0,0)$ (ô góc trên bên trái)
- **Actions:** $ACTIONS(s) = \{\text{Lên}, \text{Xuống}, \text{Trái}, \text{Phải}\}$ khi ô đích nằm trong lưới và không bị chặn.
- **Result:** $RESULT((x, y), \text{Phải}) = (x, y + 1)$ nếu ô $(x, y + 1)$ thông thoáng.
- **Goal Test:** $GOAL_{TEST}(s) = (s == (m - 1, n - 1))$ với mê cung kích thước $m \times n$.
- **Path Cost:** $g(n) = \text{số bước di chuyển từ } (0,0) \text{ đến trạng thái } n$.

Qua định nghĩa chính thức này, mọi bài toán giải bằng tìm kiếm đều trở thành một **tuple** $\langle s_0, ACTIONS, RESULT, GOAL_{TEST}, g \rangle$, từ đó thuật toán tìm kiếm có thể vận hành một cách tổng quát và có thể tái sử dụng cho nhiều dạng bài toán khác nhau.

2.1.3. Ví dụ kinh điển: Bài toán 8-puzzle

Bài toán **8-puzzle** là một ví dụ tiêu biểu để minh họa năm thành phần cấu trúc bài toán đã định nghĩa ở trên. Hình minh họa cho một instance điển hình được thể hiện trong Figure 3.3 của Russell & Norvig.

Thành phần	Minh họa trong 8-puzzle
States	Mỗi trạng thái là một cách sắp xếp duy nhất của 8 ô đánh số và 1 ô trống trên lưới 3×3 . Tổng số hoán vị là $9! = 362\,880$, nhưng chỉ một nửa (181\,440) là với parity phù hợp và có thể đạt được từ một goal cho trước .
Initial State	Bất kỳ cấu hình nào được chọn làm xuất phát; ví dụ instance trong Figure 3.3 có ô trống và các số được xáo trộn so với trạng thái đích .
Actions	Tập {Left, Right, Up, Down}. Tại các vị trí cạnh hoặc góc, một số hành động sẽ không áp dụng được .
Transition Model	Hàm $RESULT(s, a)$ trả về trạng thái mới bằng cách hoán đổi ô trống với ô số kề bên theo hướng a . Ví dụ, $RESULT(\text{start}, \text{Left})$ sẽ đổi chỗ ô trống với ô “5” bên trái nó .
Goal Test	Kiểm tra xem các ô có được xếp thứ tự từ 1 đến 8, ô trống ở góc dưới-phải hay không (đúng cấu hình đích như ở Figure 3.3) .
Path Cost	Mỗi lần di chuyển (hành động) tốn một đơn vị chi phí, do đó chi phí đường đi $g(n)$ là số bước di chuyển từ trạng thái ban đầu đến trạng thái n .

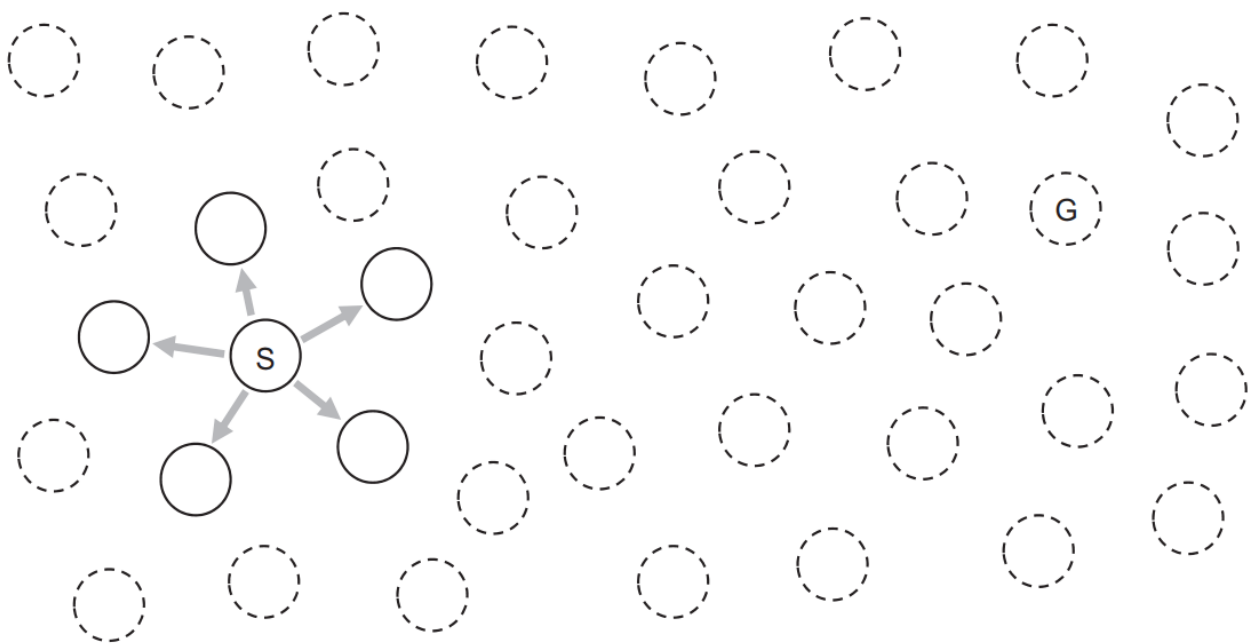


Hình 41: Một cấu hình điển hình của bài toán 8-puzzle. (Nguồn: Russell & Norvig – AIMA)

Hình ảnh trên hiển thị một cấu hình điển hình của bài toán 8-puzzle. Đây là hình ảnh trực tiếp được đề cập trong văn bản của bạn và minh họa rõ ràng trạng thái ban đầu và trạng thái mục tiêu của bài toán.

2.1.4. Không gian trạng thái (State Space)

Không gian trạng thái của một bài toán tìm kiếm có thể được hình dung dưới dạng **một đồ thị (graph)**, trong đó mỗi **nút (node)** tương ứng với một trạng thái khả dĩ của thế giới, và mỗi **cạnh (edge)** biểu diễn một hành động chuyển từ trạng thái này sang trạng thái khác . Trên thực tế, đồ thị này thường là **ngầm định (implicit)**, được sinh ra qua hàm sinh hành động (MoveGen) chứ không lưu trữ trực tiếp hết mọi cạnh và nút.



Hình 42. Không gian trạng thái là tập hợp các trạng thái có thể xảy ra.

Hình ảnh trên minh họa khái niệm không gian trạng thái như là một tập hợp các trạng thái khả thi, nơi mỗi trạng thái có thể được xem như một nút trong một đồ thị ngầm định. Đây là hình ảnh lý tưởng để trực quan hóa "không gian trạng thái" như một đồ thị.

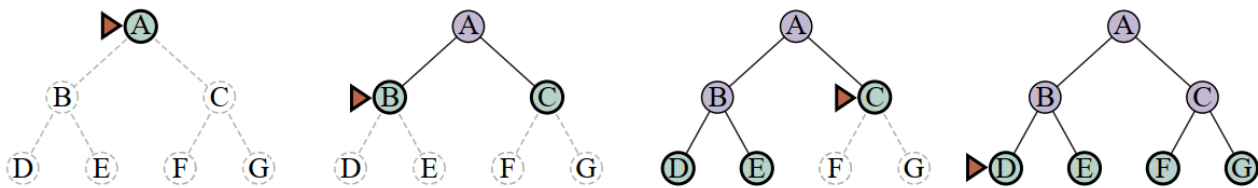
Thành phần	Định nghĩa
Node (Nút)	Một trạng thái $s \in S$, mô tả hoàn chỉnh tình huống hiện tại (ví dụ: cấu hình 8-puzzle, vị trí robot trong mê cung).
Edge (Cạnh)	Một hành động $a \in ACTIONS(s)$ nối hai trạng thái s và $s' = RESULT(s, a)$.
Path (Đường đi)	Một dãy các cạnh ($s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_n$), tương ứng với chuỗi hành động đưa tác tử từ trạng thái ban đầu s_0 đến trạng thái đích.
Optimal Path	Đường đi có chi phí nhỏ nhất (tức tổng $g(n)$ tối ưu) trong mọi đường đi dẫn tới trạng thái mục tiêu.

Giải pháp (solution) của vấn đề tìm kiếm chính là một **đường đi** (path) nối nút ban đầu và một nút đích. Khi bài toán yêu cầu, ta còn quan tâm đến **giải pháp tối ưu (optimal solution)**—đường đi có chi phí $g(n)$ thấp nhất trên đồ thị.

2.2. Tìm kiếm theo chiều rộng (Breadth-First Search – BFS)

2.2.1. Nguyên lý cốt lõi

Breadth-First Search (BFS) là một chiến lược tìm kiếm “mù” (uninformed) dựa trên nguyên tắc mở rộng theo **tầng lớp độ sâu** trước rồi mới đi sâu hơn.



Hình 43. Tìm kiếm theo chiều rộng trên cây nhị phân đơn giản.

Hình ảnh trên minh họa trực quan thuật toán tìm kiếm theo chiều rộng (BFS) khi nó khám phá một cây nhị phân đơn giản, thể hiện cách nó mở rộng các nút theo từng cấp độ từ nút gốc. Điều này phù hợp hoàn hảo với phần "Nguyên lý cốt lõi" và phần "Trực quan hóa như sóng lan tỏa" của bạn.

- **Mở rộng các nút nông nhất trước** BFS sử dụng hàng đợi FIFO để đảm bảo rằng mỗi nút mới sinh (luôn có độ sâu lớn hơn hoặc bằng độ sâu của nút cha) được đưa vào cuối hàng đợi, trong khi các nút có độ sâu nhỏ hơn luôn được lấy ra và mở rộng trước.
- **Đảm bảo tìm ra đường đi ngắn nhất về số lượng hành động** Vì BFS chỉ sinh và kiểm tra các nút ở độ sâu d sau khi đã hoàn thành việc mở rộng toàn bộ các nút ở độ sâu $d-1$, nên ngay khi gặp trạng thái đích trong quá trình sinh nút, ta đã chắc chắn thu được một **giải pháp tối ưu** về số bước đi.
- **Trực quan hóa như sóng lan tỏa** Có thể hình dung BFS giống như các vòng sóng lan dần trên mặt nước: bắt đầu từ tâm (trạng thái khởi đầu), thuật toán khám phá một "vòng" các nút ở cùng độ sâu, rồi mới tới vòng kế tiếp (độ sâu $+1$), đảm bảo không bỏ sót bất kỳ trạng thái nào ở cùng mức trước khi tiến sâu hơn.

2.2.2. Cách thức hoạt động

Breadth-First Search (BFS) vận hành dựa trên cấu trúc hàng đợi FIFO (First-In, First-Out) để quản lý **frontier** (các nút biên), đồng thời duy trì một tập **explored** để tránh lặp vô hạn trên đồ thị.

- **Khởi tạo**
 - Tạo một nút gốc *node* từ **trạng thái ban đầu**.
 - Nếu *node* đã thỏa mãn **kiểm tra đích**, thuật toán trả về giải pháp ngay lập tức.
 - Khởi tạo hàng đợi *frontier* chứa *node*.
 - Khởi tạo tập $explored = \{ state(node) \}$.
- **Vòng lặp tìm kiếm**

```
while frontier không rỗng do
  node ← POP(frontier)           # Lấy nút đầu tiên ra — nút nông nhất
  for mỗi child in EXPAND(problem, node) do
    s ← child.STATE
    if problem.IS-GOAL(s) then return child
```

```

    if s  $\notin$  explored then
        explored  $\leftarrow$  explored  $\cup$  { s }
        PUSH(child, frontier)      # Thêm con vào cuối hàng đợi
    return failure                 # Nếu hết frontier mà không tìm thấy

```

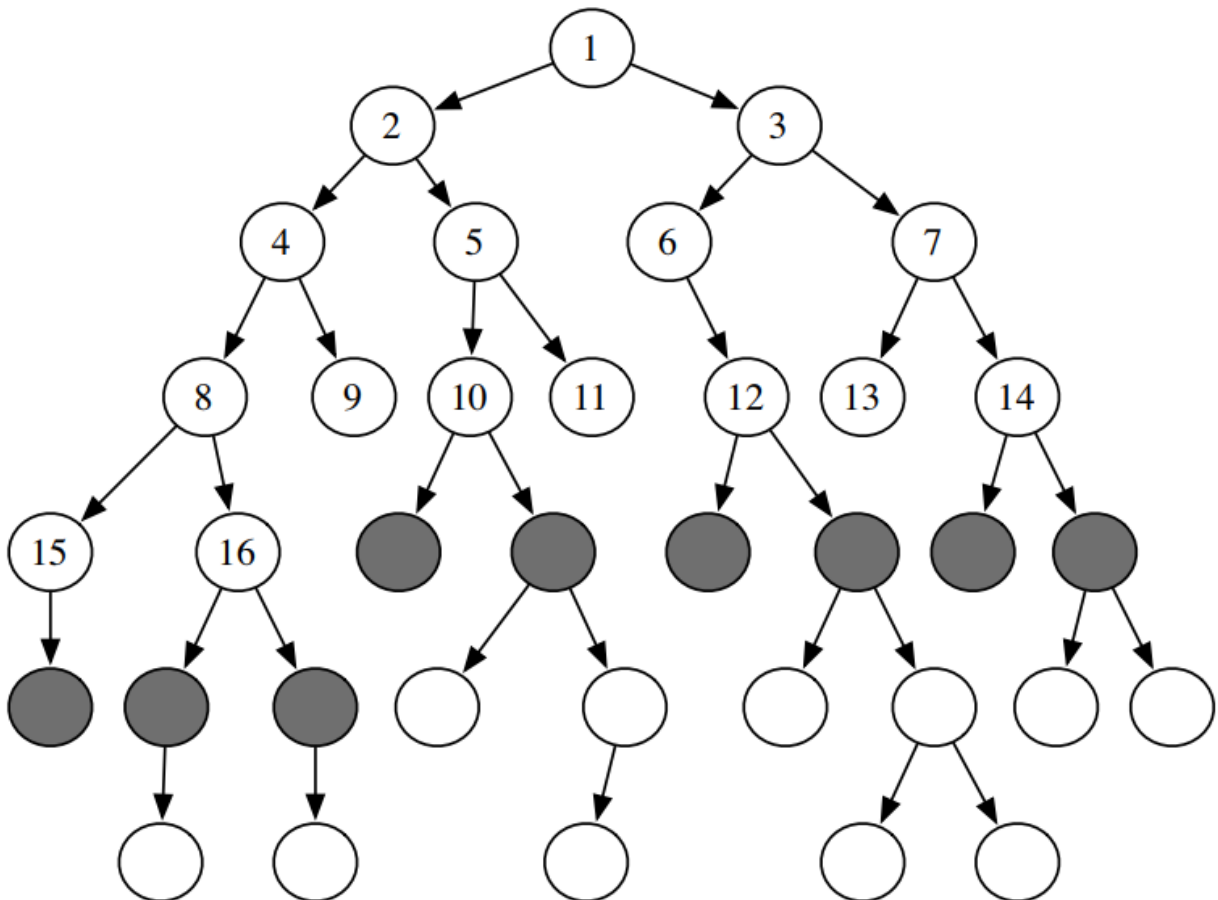
Pseudocode theo Russell & Norvig.

- **Ghi nhớ để tránh lặp**

- Mỗi khi sinh một nút mới, kiểm tra xem trạng thái của nó đã nằm trong *explored* hay chưa.
- Việc này đảm bảo **khả năng hoàn tất** (completeness) ngay cả trên đồ thị vô hạn, và ngăn ngừa thuật toán bị mắc kẹt trong các chu trình.

- **Đặc điểm lưu ý**

- BFS luôn **tìm ra** một **giải pháp tối ưu** về số bước (khi mỗi hành động có chi phí bằng nhau), vì nó khám phá tuần tự theo độ sâu tăng dần.
- Mức độ sử dụng bộ nhớ (kích thước của *frontier*) có thể rất lớn, do tất cả các nút ở cùng độ sâu phải được lưu trước khi khám phá độ sâu kế tiếp.



Hình 44. Thứ tự mở rộng các đường đi trong tìm kiếm theo chiều rộng.

Hình ảnh trên mô tả chi tiết hơn về thứ tự các đường dẫn được mở rộng trong tìm kiếm theo chiều rộng (BFS), cho thấy các nút ở cuối các đường dẫn được mở rộng và các nút ở cuối đường biên. Nó minh họa rõ ràng cơ chế hoạt động của BFS, đặc biệt là việc mở rộng theo từng lớp độ sâu và vai trò của hàng đợi FIFO.

2.2.3. Phân Tích và Đánh Giá Thuật Toán Tìm Kiếm Theo Chiều Rộng (BFS)

Để đánh giá hiệu quả và tính phù hợp của một thuật toán tìm kiếm, chúng ta cần xem xét nó qua bốn tiêu chí cơ bản: tính đầy đủ, tính tối ưu, độ phức tạp thời gian và độ phức tạp không gian. Thuật toán Tìm Kiếm Theo Chiều Rộng (BFS) được phân tích chi tiết như sau:

Tính Đầy Đủ (Completeness)

BFS có tính đầy đủ. Điều này có nghĩa là nếu một lời giải tồn tại, BFS chắc chắn sẽ tìm thấy nó. Cơ chế hoạt động của BFS là duyệt qua tất cả các nút ở độ sâu d trước khi chuyển sang các nút ở độ sâu $d+1$. Do đó, nó không thể bị mắc kẹt trong một nhánh vô hạn của cây tìm kiếm (một rủi ro tiềm tàng với các thuật toán như Tìm Kiếm Theo Chiều Sâu - DFS). Miễn là hệ số rẽ nhánh b (branching factor) là hữu hạn, BFS sẽ khám phá không gian trạng thái một cách có hệ thống theo từng lớp, đảm bảo rằng mọi nút có thể truy cập được cuối cùng sẽ được ghé thăm.

Tính Tối Ưu (Optimality)

BFS có tính tối ưu, nhưng chỉ với một điều kiện quan trọng: chi phí của mỗi hành động (step cost) phải là đồng nhất. Vì BFS luôn tìm thấy nút đích ở độ sâu nông nhất (d), và nếu mỗi bước đi có chi phí như nhau (ví dụ, bằng 1), thì lời giải nông nhất cũng chính là lời giải có chi phí thấp nhất.

Tuy nhiên, nếu chi phí các hành động không bằng nhau, BFS không còn đảm bảo tính tối ưu. Nó có thể tìm thấy một đường đi ngắn về số bước nhưng lại có tổng chi phí cao hơn một đường đi dài hơn. Trong trường hợp này, các thuật toán như **Tìm Kiếm Chi Phí Đồng Nhất (Uniform-Cost Search)** sẽ là lựa chọn phù hợp hơn.

Độ Phức Tạp Thời Gian (Time Complexity)

Độ phức tạp thời gian của BFS được xác định bởi số lượng nút mà nó phải tạo ra và duyệt qua. Trong trường hợp xấu nhất, BFS phải duyệt qua tất cả các nút cho đến độ sâu d của lời giải nông nhất. Giả sử cây tìm kiếm có hệ số rẽ nhánh là b , số nút ở mỗi cấp được tính như sau:

- Cấp 0 (gốc): 1 nút
- Cấp 1: b nút
- Cấp 2: b^2 nút
- ...
- Cấp d : b^d nút

Tổng số nút được tạo ra sẽ là: $1 + b + b^2 + \dots + b^d$. Theo quy tắc của chuỗi hình học, tổng này xấp xỉ bằng số nút ở cấp cuối cùng, do đó độ phức tạp thời gian của BFS là $O(b^d)$.

Độ Phức Tạp Không Gian (Space Complexity)

Độ phức tạp không gian là một trong những hạn chế lớn nhất của BFS. Thuật toán này yêu cầu lưu trữ tất cả các nút biên (frontier) trong bộ nhớ. Trong trường hợp xấu nhất, toàn bộ các nút ở cấp độ d sẽ nằm trong hàng đợi (queue) cùng một lúc. Do đó, độ phức tạp không gian cũng là $O(b^d)$.

Yêu cầu về bộ nhớ này tăng theo cấp số nhân với độ sâu của lời giải. Đối với các bài toán thực tế có không gian trạng thái lớn, BFS có thể nhanh chóng cạn kiệt bộ nhớ trước khi tìm thấy lời giải.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Hình 45. Đánh giá các thuật toán tìm kiếm theo tiêu chí.

Bảng trên so sánh các thuật toán tìm kiếm mù (uninformed search) dựa trên bốn tiêu chí hiệu suất. Với b là hệ số rẽ nhánh, d là độ sâu của lời giải nông nhất, và m là độ sâu tối đa của cây tìm kiếm. (Nguồn: *Stuart Russell & Peter Norvig, "Artificial Intelligence: A Modern Approach"*)

Tóm lại, BFS là một thuật toán mạnh mẽ, đảm bảo tìm ra lời giải nông nhất. Tuy nhiên, **nhược điểm chí mạng** của nó là yêu cầu bộ nhớ và thời gian tăng theo hàm mũ, khiến nó không khả thi cho các bài toán phức tạp với không gian tìm kiếm rộng lớn hoặc lời giải nằm ở độ sâu. Chính vì hạn chế về khả năng mở rộng này mà trong nhiều ứng dụng, các thuật toán tìm kiếm có thông tin (heuristic search) như A* thường được ưa chuộng hơn.

2.3. Tìm kiếm theo chiều sâu (Depth-First Search – DFS)

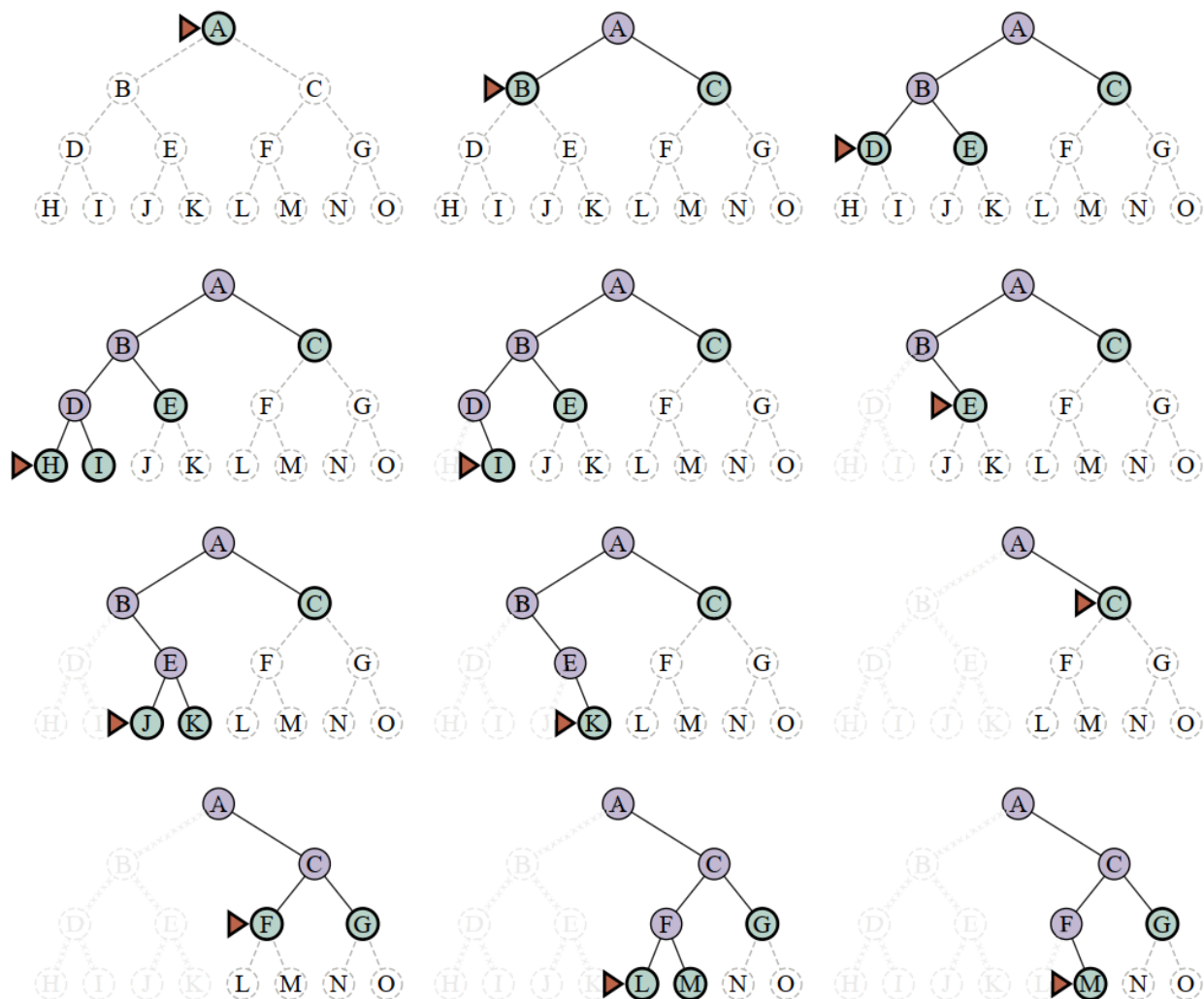
Tìm kiếm theo chiều sâu (DFS) là một trong những chiến lược tìm kiếm “mù” (uninformed) cơ bản, có cách tiếp cận trái ngược hoàn toàn với Tìm kiếm theo chiều rộng (BFS). Thay vì khám phá các nút theo từng lớp độ sâu, DFS ưu tiên đi sâu vào một nhánh của cây tìm kiếm cho đến khi không thể đi tiếp, sau đó mới quay lui để khám phá các lựa chọn thay thế.

2.3.1. Nguyên lý cốt lõi

Nguyên lý hoạt động của DFS được định nghĩa bởi việc luôn luôn mở rộng nút sâu nhất trong tập các nút biên (frontier). Chiến lược này được hiện thực hóa một cách tự nhiên thông qua việc sử dụng cấu trúc dữ liệu **ngăn xếp (stack)**, hoạt động theo cơ chế **LIFO (Last-In, First-Out)**.

- **Cơ chế Ưu tiên Chiều sâu:** Khi một nút được mở rộng, các nút con (successors) của nó sẽ được thêm vào đỉnh của ngăn xếp. Ở bước tiếp theo, thuật toán sẽ lấy ra nút nằm ngay trên cùng của ngăn xếp để tiếp tục mở rộng. Điều này đảm bảo rằng thuật toán luôn ưu tiên khám phá con cháu của nút vừa được mở rộng, tạo ra một hành trình “lao xuống” theo một đường đi duy nhất cho đến khi đạt đến điểm cuối cùng của nhánh đó.
- **Quá trình Quay lui (Backtracking):** Khi thuật toán đi đến một nút không có nút con nào (nút lá) hoặc một nút mà tất cả các đường đi từ nó đã được khám phá mà vẫn chưa tìm thấy trạng thái đích, nó sẽ phải quay lui. Quá trình này được thực hiện bằng cách loại bỏ nút hiện tại khỏi ngăn xếp. Nút tiếp theo được lấy ra sẽ là nút anh em (sibling) của nút vừa rồi, hoặc nếu không còn anh em, đó sẽ là nút cha (hoặc tổ tiên) có các nhánh khác chưa được khám phá. Đây chính là cơ chế **quay lui** đặc trưng, cho phép DFS thoát khỏi các “ngõ cụt” và thử một con đường khác một cách có hệ thống.

Quá trình này được minh họa rõ nét trong Hình 46, thể hiện các bước của DFS trên một cây tìm kiếm. Thuật toán bắt đầu tại A, sau đó đi sâu xuống nhánh bên trái qua B và D. Khi đến D, nó tiếp tục đi sâu tới H, I, và cứ thế tiếp tục cho đến khi tìm thấy nút đích M hoặc đã duyệt hết mọi khả năng.



Hình 46. Quá trình tìm kiếm theo chiều sâu (DFS) từ trạng thái A đến M trên cây nhị phân.

Hình 46. Minh họa 12 bước trong quá trình tìm kiếm theo chiều sâu (DFS). Quá trình bắt đầu từ trạng thái A với mục tiêu là M. Tập các nút biên (frontier) được tô màu xanh lá cây, với một hình tam giác đánh dấu nút sẽ được mở rộng tiếp theo. Các nút đã được duyệt qua có màu tím nhạt. Hình ảnh này cho thấy rõ chiến lược “đi sâu hết mức” của DFS: thay vì mở rộng B rồi đến C như BFS, DFS mở rộng B rồi đi sâu ngay xuống D, sau đó là H, rồi quay lui để khám phá các nhánh khác.

2.3.2. Cách thức hoạt động

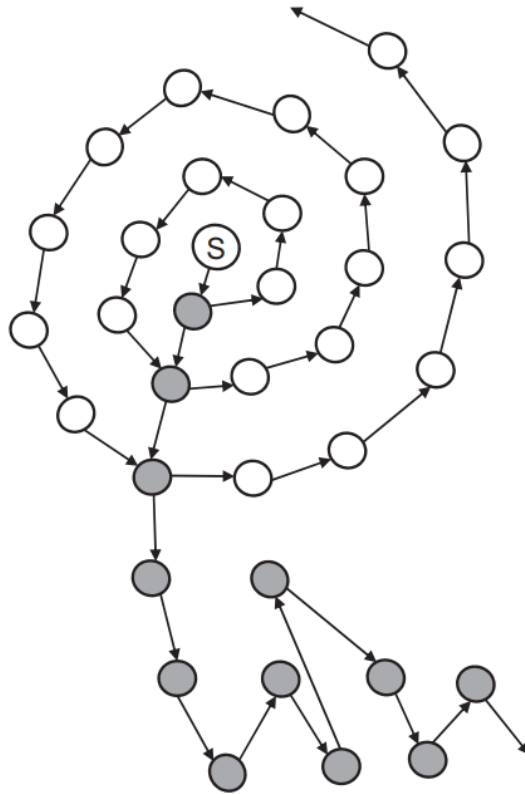
Depth-First Search (DFS) vận hành dựa trên nguyên tắc “đi sâu trước, rồi quay lui” nhờ sử dụng **ngăn xếp LIFO** (Last-In, First-Out) để quản lý **frontier** (các nút biên), kết hợp với một tập **explored/visited** để tránh lặp vô hạn trong đồ thị.

- **Khởi tạo**

- Tạo nút gốc (root) từ **trạng thái ban đầu**.
- Nếu nút gốc thỏa mãn **Goal Test**, trả về giải pháp ngay.
- Đưa nút gốc vào **stack**.
- Khởi tạo $visited = \emptyset$.

- **Vòng lặp tìm kiếm**

```
while stack không rỗng do
  node ← POP(stack)           # Lấy nút trên cùng (sâu nhất)
  if node.STATE ∉ visited then
    visited ← visited ∪ { node.STATE } # Đánh dấu đã thăm
    if GoalTest(node.STATE)           # Kiểm tra đích
      return SOLUTION(node)           # Trả về chuỗi hành động
    for each a in ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, a)
      PUSH(child, stack)              # Đưa con lên đỉnh ngăn xếp
  return failure                # Nếu exhaust frontier mà không tìm thấy
```



Hình 47. Lộ trình tìm kiếm của DFS và BFS trong không gian trạng thái.

Hình ảnh trên so sánh quỹ đạo của cả DFS và BFS trong không gian trạng thái, giúp người đọc hình dung sự khác biệt trong cách hai thuật toán này khám phá các nút. Mặc dù nó hiển thị cả hai, bạn có thể tập trung giải thích phần DFS.

• Chi tiết hoạt động

- Mỗi lần lặp, DFS chọn **nút mới sinh gần đây nhất** (sâu nhất) để mở rộng.
- Khi một nhánh khám phá đến tận cùng (không còn successor), thuật toán **quay lui** (backtrack) lên nút cha để tiếp tục các nhánh khác.
- Tập *visited* đảm bảo thuật toán không quay lại khám phá các trạng thái đã thăm, tránh vòng lặp và làm cho DFS an toàn trên đồ thị hữu hạn .

Minh họa tiến trình DFS trên một cây nhị phân Quá trình tìm kiếm được thể hiện qua 12 bước (từ trên xuống, trái sang phải), trong đó frontier được tô màu xanh lá, mũi tên tam giác đánh dấu nút sắp được mở rộng, các nút đã mở rộng màu tím nhạt, và các cạnh tiềm năng được vẽ đứt mờ .

2.3.3. Đánh giá thuật toán

Breadth-First Search đã được xem xét trước, còn Depth-First Search (DFS) có những đặc điểm sau, đánh giá theo bốn tiêu chí:

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Hình 48. Đánh giá các thuật toán tìm kiếm. b là hệ số phân nhánh.

Tương tự như phần BFS, hình ảnh trên cung cấp một bảng tổng hợp đánh giá các thuật toán tìm kiếm, nơi bạn có thể làm nổi bật các đặc điểm của DFS liên quan đến tính đầy đủ, tính tối ưu, độ phức tạp thời gian và không gian. Điều này giúp so sánh trực tiếp với BFS.

- **Tính đầy đủ (Completeness):** DFS **không** đảm bảo tìm ra giải pháp nếu tồn tại, vì có thể bị kẹt trong một nhánh vô hạn.

Lưu ý: Nếu không gian trạng thái là **hữu hạn** và **không có chu trình**, DFS sẽ hoàn tất và do đó trở nên đầy đủ.

- **Tính tối ưu (Optimality):** DFS **không** đảm bảo tìm ra lời giải tối ưu, bởi thuật toán đi sâu mà không so sánh chi phí giữa các nhánh.

Độ phức tạp thời gian (Time Complexity): $O(b^m)$

với b là hệ số rẽ nhánh (branching factor) và m là độ sâu tối đa của không gian trạng thái (có thể lớn hơn độ sâu lời giải d).

Độ phức tạp không gian (Space Complexity): $O(b \times m)$

DFS chỉ cần lưu đường dẫn từ gốc tới nút hiện tại và các nhánh phụ, nên tiêu thụ bộ nhớ tuyến tính theo m , đây là ưu điểm lớn nhất của phương pháp này.

Nhìn chung, DFS phù hợp với các không gian trạng thái sâu nhưng hẹp, nơi yêu cầu tiết kiệm bộ nhớ, còn trong các không gian phức tạp hoặc có chu trình vô hạn thì cần cân nhắc sử dụng các biến thể như Iterative Deepening Search.

2.4. Tìm kiếm chi phí đồng nhất (Uniform-Cost Search – UCS)

2.4.1. Nguyên lý cốt lõi

Uniform-Cost Search (UCS) là sự tổng quát hóa của phương pháp tìm kiếm theo chiều rộng (BFS), trong đó:

Mở rộng nút có chi phí đường đi thấp nhất Thay vì lần lượt khám phá theo độ sâu tăng dần như BFS, UCS luôn chọn để mở rộng nút n sao cho $g(n)$ = chi phí từ trạng thái ban đầu đến n là nhỏ nhất trong frontier.

- **Lan tỏa theo sóng chi phí thay vì sóng độ sâu** BFS “lan tỏa” theo lớp độ sâu (depth), còn UCS “lan tỏa” theo lớp chi phí (path-cost). Mỗi bước thuật toán đảm bảo rằng tất

cả các đường đi có chi phí nhỏ hơn hoặc bằng một ngưỡng C nhất định đã được khám phá trước khi bước sang chi phí $> C$.

- **Giảm về BFS khi chi phí đều bằng nhau** Nếu mọi hành động đều có chi phí bằng nhau (ví dụ $c(s, a) = 1$), thì sóng chi phí cũng chính là sóng độ sâu, và UCS hoạt động y hệt BFS.

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
  node  $\leftarrow$  NODE(problem.INITIAL)
  if problem.IS-GOAL(node.STATE) then return node
  frontier  $\leftarrow$  a FIFO queue, with node as an element
  reached  $\leftarrow$  {problem.INITIAL}
  while not IS-EMPTY(frontier) do
    node  $\leftarrow$  POP(frontier)
    for each child in EXPAND(problem, node) do
      s  $\leftarrow$  child.STATE
      if problem.IS-GOAL(s) then return child
      if s is not in reached then
        add s to reached
        add child to frontier
  return failure

function UNIFORM-COST-SEARCH(problem) returns a solution node, or failure
  return BEST-FIRST-SEARCH(problem, PATH-COST)
```

Hình 49. Thuật toán tìm kiếm theo chiều rộng và thuật toán tìm kiếm chi phí đồng nhất.

Hình ảnh trên cung cấp mã giả của cả thuật toán BFS và UCS, làm nổi bật sự giống nhau và khác nhau trong cấu trúc của chúng. Đặc biệt, nó cho thấy UCS sử dụng hàng đợi ưu tiên và mở rộng nút có chi phí đường đi thấp nhất.

Pseudocode cho Breadth-First Search và Uniform-Cost Search
UCS được thực hiện như một trường hợp của Best-First Search, với hàm đánh giá $f(n) = g(n)$ (Path-Cost).

Best-First-Search(*problem*, PATH-COST) là thuật toán chung, trong đó *frontier* là hàng đợi ưu tiên sắp xếp theo $g(n)$.

2.4.2. Cách thức hoạt động

Uniform-Cost Search (UCS) triển khai tìm kiếm dựa trên **đường đi có chi phí thấp nhất** bằng cách sử dụng **hàng đợi ưu tiên (priority queue)**, sắp xếp *frontier* theo giá trị $g(n)$.

- **Khởi tạo**
 - Tạo nút gốc *node* từ **trạng thái ban đầu** với chi phí $g(\text{node}) = 0$.
 - Đưa *node* vào **priority queue** (*frontier*), sắp xếp theo $g(n)$.

- Khởi tạo $reached = \{\}, \text{state}(node)\}$ để theo dõi các trạng thái đã thêm vào frontier.

- **Vòng lặp tìm kiếm**

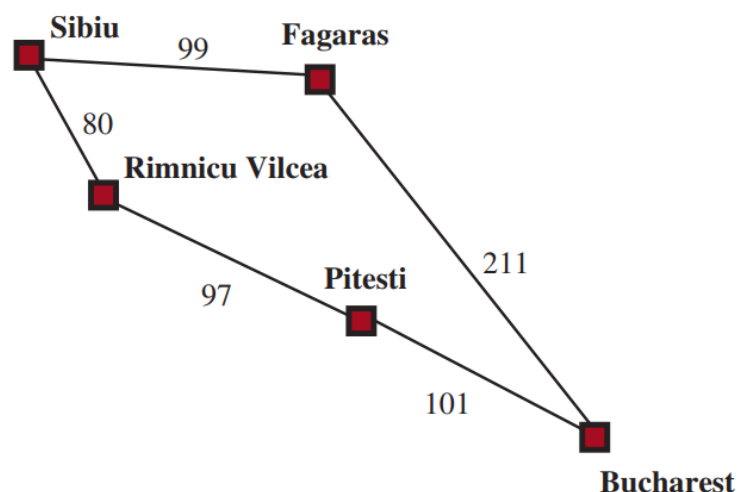
```

while frontier không rỗng do
  node ← POP-MIN(frontier) # Lấy nút n có chi phí g(n) thấp nhất
  if problem.IS-GOAL(node.STATE) then
    return SOLUTION(node) # Trả về chuỗi hành động dẫn đến node
  for each action a in ACTIONS(node.STATE) do
    child ← CHILD-NODE(problem, node, a)
    new_cost ← node.g + COST(node.STATE,a,child.STATE)
    if child.STATE ∉ reached OR new_cost < child.g then
      child.g ← new_cost          # Cập nhật chi phí
      child.parent ← node
      if child.STATE ∉ reached then
        add child.STATE to reached
      update frontier with child (priority = child.g)
  return failure                # Nếu hết frontier mà không tìm thấy

```

Pseudocode minh họa cách gọi BEST-FIRST-SEARCH với $PATH-COST = g(n)$

- **Ví dụ minh họa: Không gian trạng thái Romania** Hình 3.10 trong Russell & Norvig cho thấy phần không gian trạng thái trên bản đồ Romania, với chi phí đường đi khác nhau giữa các thành phố. UCS sẽ lần lượt mở rộng các nút có tổng chi phí nhỏ nhất (ví dụ từ Sibiu → Rimnicu Vilcea (80) trước Fagaras (99)), rồi cập nhật các đường đi mới và lựa chọn tiếp theo dựa trên chi phí đã cộng dồn.



Hình 50: Bản đồ Romania minh họa không gian trạng thái cho thuật toán UCS.
(Nguồn: Russell & Norvig – AIMA)

Hình ảnh trên hiển thị một phần của bản đồ Romania, được chọn đặc biệt để minh họa quá trình tìm kiếm chi phí đồng nhất (UCS). Nó giúp trực quan hóa cách thuật toán mở rộng các nút dựa trên tổng chi phí đường đi tích lũy.

Qua cơ chế frontier ưu tiên theo chi phí $g(n)$, UCS đảm bảo tìm ra lời giải có chi phí thấp nhất (cost-optimal), đồng thời giảm về BFS khi mọi hành động đều có chi phí bằng nhau.

2.4.3. Đánh giá thuật toán

Uniform-Cost Search (UCS) được đánh giá theo bốn tiêu chí chính như sau:

- **Tính đầy đủ (Completeness):** UCS luôn tìm được lời giải nếu tồn tại, **miễn là** chi phí của mỗi bước hành động đều lớn hơn một hằng số dương, tức $c(s, a) \geq \epsilon > 0$ courses.grainger.illinois.edu.
- **Tính tối ưu (Optimality):** UCS đảm bảo tìm ra lời giải có **chi phí thấp nhất** (cost-optimal), vì mỗi lần chọn luôn là nút có $g(n)$ nhỏ nhất courses.grainger.illinois.edu.
- **Độ phức tạp thời gian (Time Complexity):** Giả sử C^* là chi phí của lời giải tối ưu và mỗi bước chi phí ít nhất là ϵ , thì UCS sẽ mở rộng mọi nút có $g(n) \leq C^*$. Trong trường hợp xấu nhất, số nút đó lên đến $O\left(b^{1+\lceil \frac{C^*}{\epsilon} \rceil}\right)$ stackoverflow.com.
- **Độ phức tạp không gian (Space Complexity):** Tương tự độ phức tạp thời gian, UCS phải lưu toàn bộ frontier theo chi phí, dẫn tới $O\left(b^{1+\lceil \frac{C^*}{\epsilon} \rceil}\right)$ stackoverflow.com.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Hình 51. Đánh giá các thuật toán tìm kiếm. b là hệ số phân nhánh.

Hình ảnh trên cung cấp một bảng tổng hợp đánh giá các thuật toán tìm kiếm, bao gồm BFS, DFS và UCS, theo các tiêu chí như tính đầy đủ, tính tối ưu, độ phức tạp thời gian và không gian. Đây là một hình ảnh trực tiếp và hoàn hảo để trình bày phần này.

* Với chi phí mỗi hành động bằng nhau (ví dụ $c = 1$), BFS trở thành UCS và tìm lời giải ngắn nhất về số bước.

† DFS chỉ đầy đủ nếu không gian trạng thái hữu hạn và không tồn tại chu trình.

2.5. Hàm Heuristic ($h(n)$) : “Kim chỉ nam” cho Tìm kiếm

2.5.1. Khái niệm

Hàm Heuristic, ký hiệu $h(n)$, là một hàm ước tính chi phí từ trạng thái tại nút n đến trạng thái đích gần nhất. Đây chỉ là một **ước tính** (không bao giờ vượt quá chi phí thực tế) và thể hiện tri thức chuyên biệt cho từng bài toán (problem-specific knowledge).

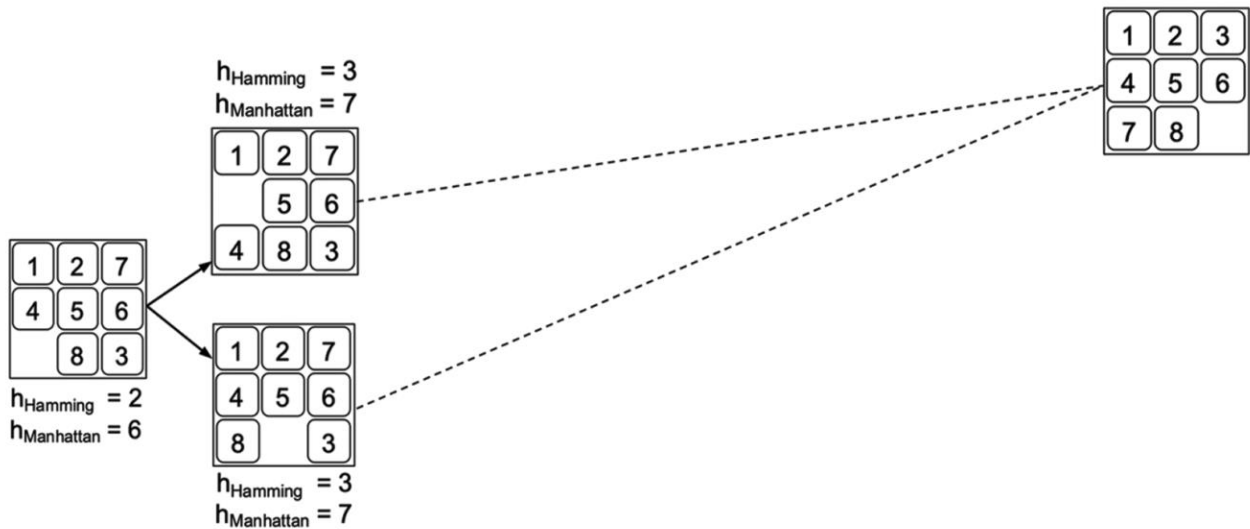
2.5.2. Ví dụ

- **Bài toán tìm đường:**

$h(n)$ = khoảng cách đường chim bay (Euclidean distance) từ thành phố n đến thành phố đích, bởi quãng đường thực tế không bao giờ ngắn hơn đường chim bay.

- **Bài toán 8-puzzle:**

$h(n)$ có thể là số ô không đúng vị trí (misplaced tiles) hoặc tổng khoảng cách Manhattan của từng viên so với vị trí đích.



Hình 52. Hai hàm heuristic cho bài toán 8 ô.

Hình ảnh trên hiển thị hai hàm heuristic phổ biến cho bài toán 8-puzzle: khoảng cách Hamming (số ô sai vị trí) và khoảng cách Manhattan. Nó trực quan hóa cách các hàm này ước tính chi phí đến đích.

2.6. Greedy Best-First Search

2.6.1. Ý tưởng

Mỗi bước, thuật toán chọn nút trên frontier (OPEN) có giá trị heuristic nhỏ nhất, tức “tham lam” hướng càng gần đích càng tốt, nhưng **không** xét chi phí đường đi đã đi.

2.6.2. Mã giả (pseudo-code)

```
def greedy_best_first(start, goal, h):  
    OPEN = PriorityQueue(order_by=h) # Ưu tiên h(n)  
    CLOSED = set()  
    OPEN.push(start)  
    while OPEN:  
        n = OPEN.pop()  
        if n == goal:  
            return reconstruct_path(n)  
        CLOSED.add(n)  
        for m in neighbors(n):  
            if m not in OPEN and m not in CLOSED:
```

```
parent[m] = n
OPEN.push(m)
return None
```

2.6.3. Đặc điểm

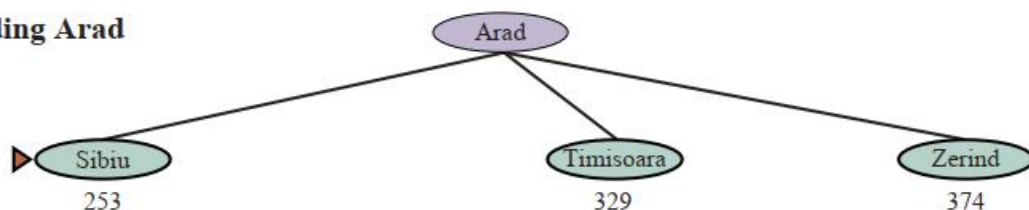
- **Không đảm bảo tìm đường ngắn nhất** vì bỏ qua chi phí $g(n)$
- **Hoàn thiện** (complete) trong không gian trạng thái hữu hạn, nhưng có thể **vòng lặp** trong không gian vô hạn
- **Độ phức tạp:**
 - Worst-case $O(|V|)$, giữ toàn bộ đỉnh chưa thăm trong OPEN.
 - Với heuristic tốt, có thể giảm về $O(b^m)$ trên một số bài toán

2.6.4. Ví dụ minh họa

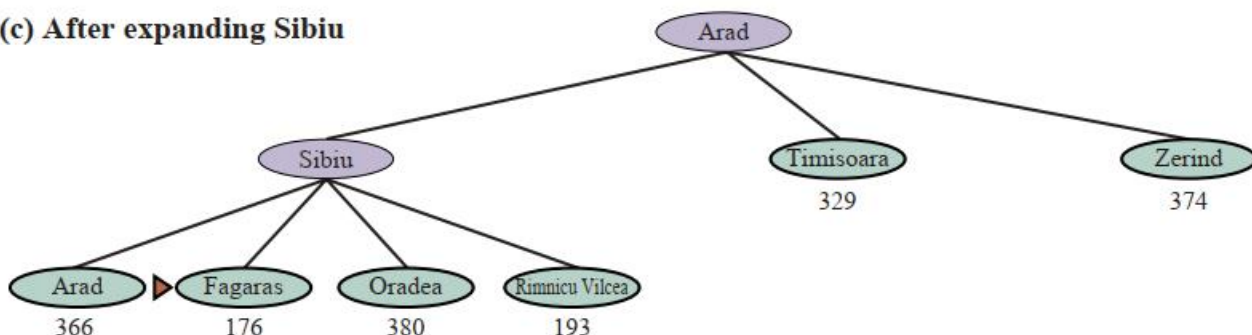
(a) The initial state



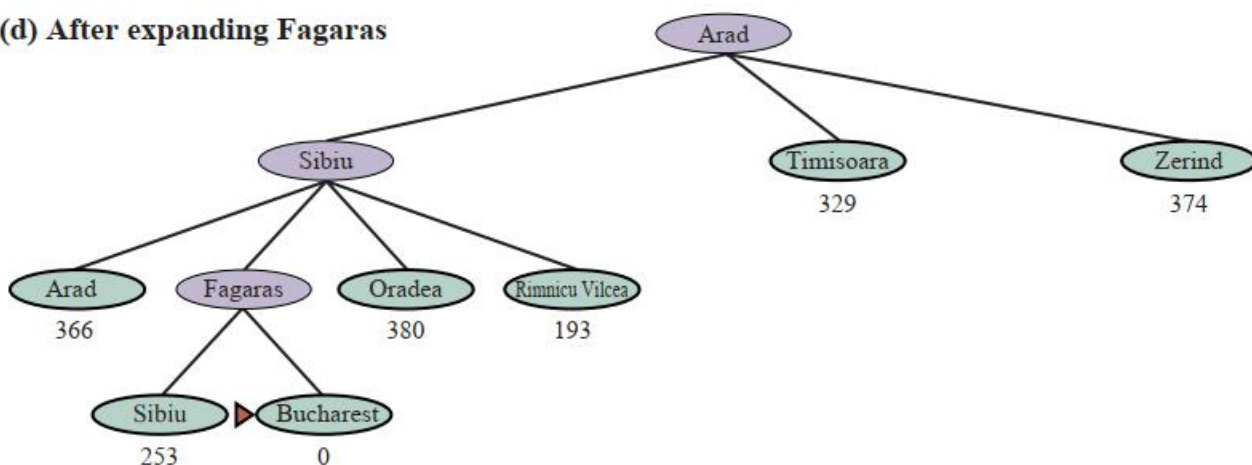
(b) After expanding Arad



(c) After expanding Sibiu



(d) After expanding Fagaras



Hình 53. Các giai đoạn trong quá trình tìm kiếm tham lam ưu tiên-nút dạng cây đến Bucharest với hàm heuristic khoảng cách đường thẳng hSLD.

Hình ảnh trên minh họa từng bước quá trình tìm kiếm tham lam tốt nhất (Greedy Best-First Search) trên bản đồ Romania, sử dụng heuristic khoảng cách đường chim bay (hSLD) để ước tính đường đi đến Bucharest. Nó trực quan hóa cách thuật toán "tham lam" lựa chọn nút có heuristic tốt nhất.

các bước mở rộng cho Greedy Best-First tìm đường từ Arad đến Bucharest, lấy thẳng-đường-bay làm h .

2.7. Thuật toán A*

2.7.1. Nguyên tắc

Đánh giá mỗi nút n bằng $f(n) = g(n) + h(n)$, trong đó:

- $g(n)$: chi phí từ start đến n .
- $h(n)$: ước tính chi phí từ n đến goal.

2.7.2. Mã giả

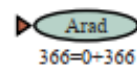
```
def A_star(start, goal, h):
    OPEN = PriorityQueue(order_by=f) # Ưu tiên f(n)=g(n)+h(n)
    CLOSED = set()
    g[start] = 0
    OPEN.push(start, f=start_cost + h(start))
    while OPEN:
        n = OPEN.pop()
        if n == goal:
            return reconstruct_path(n)
        CLOSED.add(n)
        for m, cost in neighbors(n):
            tentative_g = g[n] + cost
            if m in CLOSED and tentative_g >= g.get(m, ∞):
                continue
            if m not in OPEN or tentative_g < g.get(m, ∞):
                parent[m] = n
                g[m] = tentative_g
                f = tentative_g + h(m)
                OPEN.push(m, f)
    return None
```

2.7.3. Tính chất

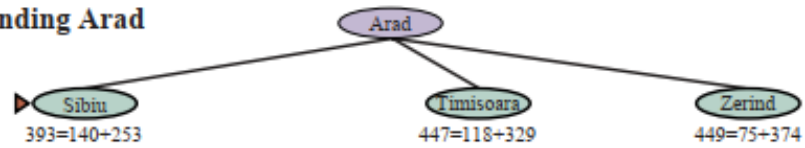
- **Admissible**: nếu h không vượt ước chi phí thực, A* luôn trả về lời giải ngắn nhất
- **Consistent** (monotonic): với mọi cạnh (n, n') , $h(n) \leq c(n, n') + h(n') \Rightarrow$ không cần cập nhật lại frontier
- **Optimally efficient**: không có thuật toán nào dùng cùng h mà mở ít nút hơn A* trước khi tìm lời giải ngắn nhất

2.7.4. Ví dụ minh họa

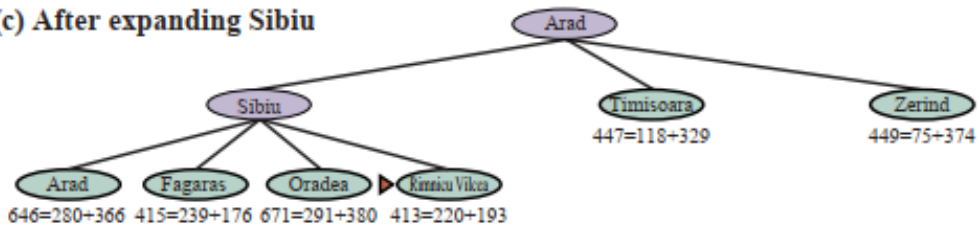
(a) The initial state



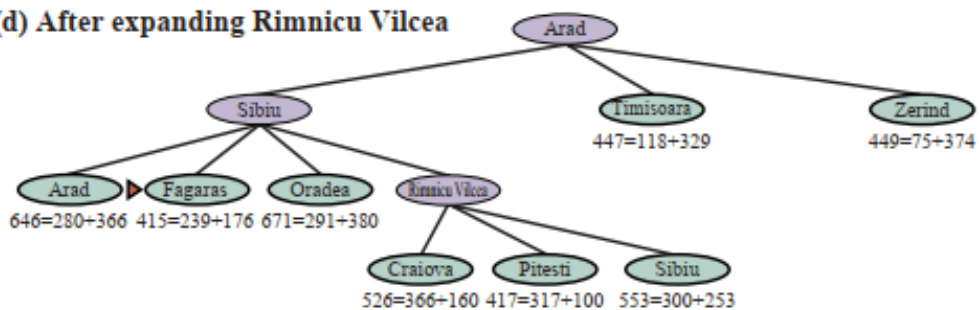
(b) After expanding Arad



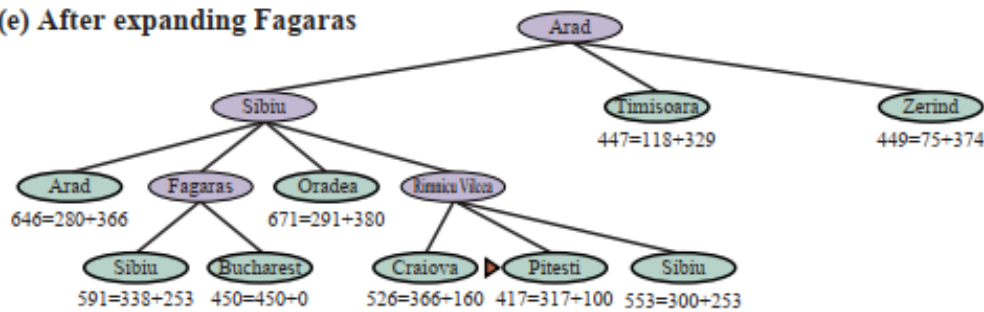
(c) After expanding Sibiu



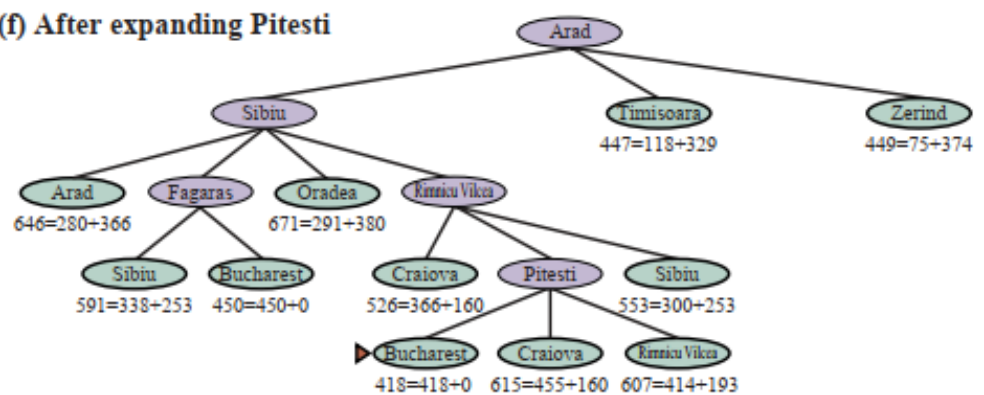
(d) After expanding Rimnicu Vilcea



(e) After expanding Fagaras



(f) After expanding Pitesti



Hình 54: Quá trình mở rộng nút của thuật toán A trên bản đồ Romania. (Nguồn: Russell & Norvig – AIMA)

tiến trình A* trên bản đồ Romania với h =straight-line distance, cho thấy Bucharest được phát hiện tại bước mà f nhỏ nhất là 418.

Hình 54 trình bày các giai đoạn của thuật toán tìm kiếm A* để tìm đường tối ưu đến Bucharest. Nó cho thấy cách A* cân bằng giữa chi phí đã đi ($g(n)$) và ước tính chi phí còn lại ($h(n)$) để tìm kiếm hiệu quả hơn, đảm bảo tính tối ưu.

Thảo luận cuối buổi:

- So sánh Greedy Best-First và A* về độ tối ưu và hiệu năng.
- Khi nào nên sử dụng heuristic “nhạy nhưng không đảm bảo” (inhomogeneous) so với heuristic đảm bảo (admissible)?
- Thực nghiệm trên đồ thị 4×4 : so sánh số nút mở rộng của mỗi thuật toán.

BUỔI HỌC 3: TÌM KIẾM CÓ THÔNG TIN (HEURISTIC SEARCH)

Mở đầu: Tại sao Tìm kiếm mù là chưa đủ?

Trong các buổi học trước, chúng ta đã tìm hiểu các thuật toán tìm kiếm truyền thống như **Tìm kiếm theo chiều rộng (Breadth-First Search – BFS)** và **Tìm kiếm chi phí đều (Uniform-Cost Search – UCS)**. Cả hai đều thuộc nhóm **thuật toán tìm kiếm không có thông tin (uninformed search)**. Chúng không sử dụng bất kỳ thông tin nào về mục tiêu ngoài việc xác định trạng thái nào là trạng thái đích.

- **BFS** đảm bảo tìm được lời giải tối ưu nhất về **số bước đi**, nếu mỗi bước có chi phí bằng nhau.
- **UCS** đảm bảo tìm được lời giải tối ưu nhất về **tổng chi phí đường đi**, bằng cách mở rộng nút có chi phí đường đi từ gốc là nhỏ nhất.

Tuy nhiên, cả hai đều có điểm yếu quan trọng: **không hiệu quả về mặt tính toán**.

Vấn đề: Tìm kiếm mù là “mù quáng”

Các thuật toán trên **khám phá không gian trạng thái một cách mù quáng**, không hề phân biệt được hướng đi nào có vẻ khả quan hơn. Chúng có thể tiêu tốn thời gian và bộ nhớ rất lớn vì phải mở rộng nhiều trạng thái **không cần thiết**, ngay cả khi một số hướng đi trông “rõ ràng là không hợp lý”.

Ví

dụ:

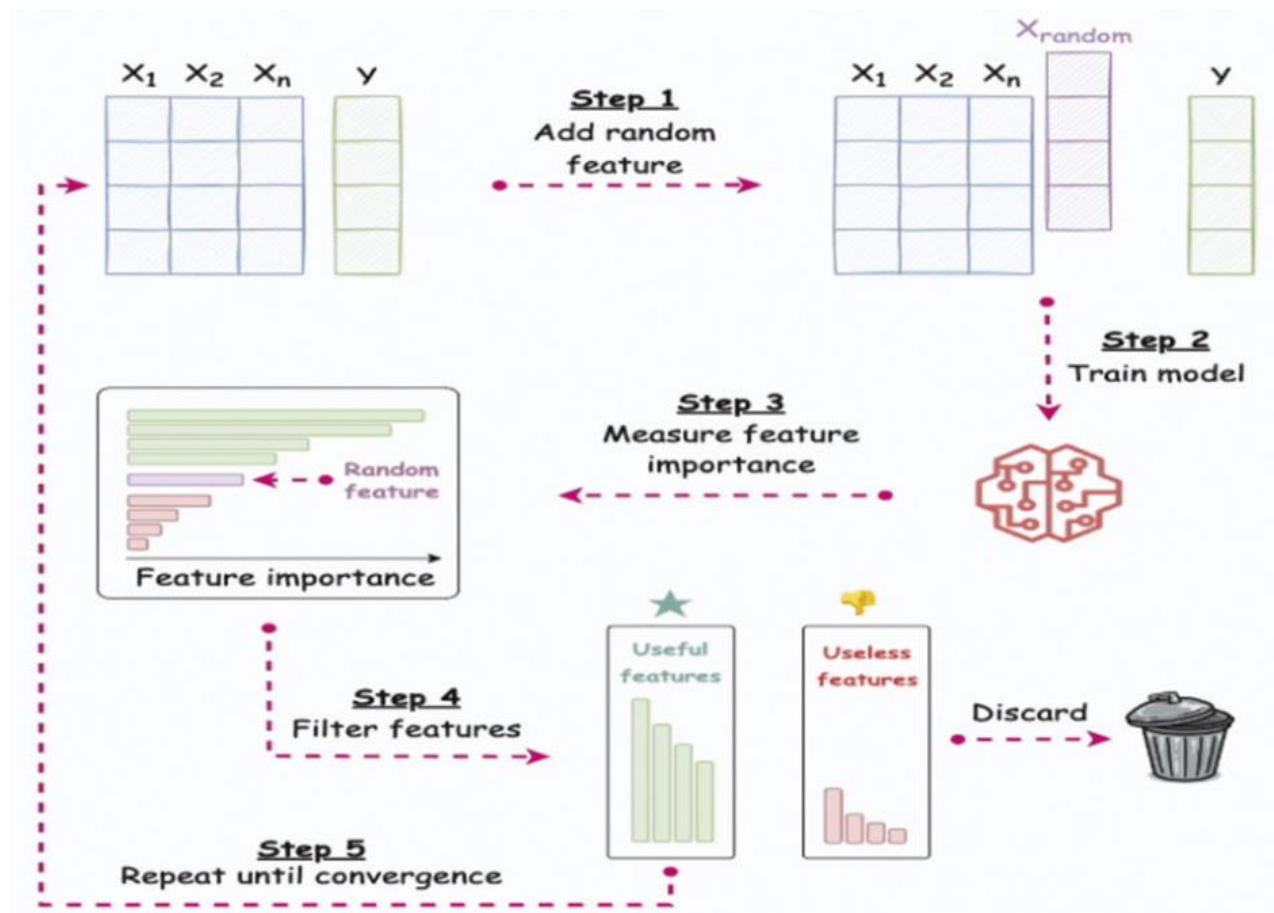
Trong bài toán tìm đường từ **Buôn Ma Thuật đến TP. Hồ Chí Minh**, một người có kinh nghiệm sẽ **không chọn** đường đi vòng ra **Nha Trang**, vì điều đó xa hơn nhiều so với đi thẳng qua Đồng Nai hoặc Đắk Nông. Nhưng một thuật toán tìm kiếm mù có thể **xét tất cả khả năng** đó một cách máy móc, dẫn đến lãng phí tài nguyên.

Giải pháp: Cần một “kim chỉ nam”

Để cải thiện hiệu quả tìm kiếm, ta cần một cách để **ước lượng trước mức độ “hứa hẹn”** của một trạng thái. Nếu ta biết trạng thái nào có vẻ gần đích hơn, ta có thể ưu tiên mở rộng các hướng đi “tốt hơn”, và bỏ qua các hướng kém khả thi.

Đây là lý do các thuật toán tìm kiếm **có thông tin (informed search)** ra đời – mà trọng tâm là việc sử dụng **hàm Heuristic**. Hàm này đóng vai trò như một “**kim chỉ nam**” giúp thuật toán định hướng:

- Trạng thái nào có **khả năng cao** dẫn đến lời giải?
- Nên **ưu tiên mở rộng** những trạng thái nào?



Hình 55. Phương pháp Probe – Kỹ thuật chọn đặc trưng đáng tin cậy trong học máy.

Hình trên mô tả phương pháp Probe – một kỹ thuật chọn đặc trưng đáng tin cậy trong học máy. Trong bối cảnh trí tuệ nhân tạo, việc lựa chọn trạng thái để mở rộng tương đương với việc chọn đặc trưng phù hợp để rút gọn không gian tìm kiếm. Cũng giống như chọn đặc trưng giúp tối ưu hóa mô hình học máy, sử dụng heuristic giúp hướng thuật toán tìm kiếm đến các hướng đi tiềm năng nhất, giảm thiểu mở rộng không cần thiết.

- Hình trên mô tả **phương pháp chọn đặc trưng đáng tin cậy (Probe Method)**, một kỹ thuật trong học máy để tối ưu hóa lựa chọn thông tin.
- Trong AI, việc lựa chọn **chiến lược mở rộng nút (state expansion)** tốt tương đương với việc chọn lựa thông tin (feature) hiệu quả trong không gian trạng thái.
- Hình này giúp sinh viên hình dung vai trò của **heuristic** (hàm đánh giá) hoặc **thông tin định hướng tìm kiếm** là quan trọng như thế nào trong việc rút gọn không gian tìm kiếm và tăng hiệu quả giải bài toán.

Trong các phần tiếp theo, chúng ta sẽ tìm hiểu kỹ hơn về **hàm heuristic**, và hai thuật toán kinh điển sử dụng nó: **Greedy Best-First Search** và **A***.

2.5. Hàm Heuristic ($h(n)$): Kim chỉ nam cho Tìm kiếm

2.5.1. Khái niệm

Trong các thuật toán tìm kiếm có thông tin (informed search), **hàm heuristic**, ký hiệu là $h(n)$, đóng vai trò trung tâm trong việc định hướng chiến lược mở rộng các trạng thái trong không gian tìm kiếm. Đây là một hàm dùng để **ước tính chi phí còn lại từ trạng thái tại nút n đến trạng thái đích gần nhất**.

Khác với hàm chi phí thực tế $g(n)$, biểu diễn chi phí đã đi từ trạng thái bắt đầu đến n , thì $h(n)$ không phản ánh chi phí thật mà chỉ là **một sự ước lượng mang tính định hướng**.

Cụ thể:

- $h(n)$ là một **ước tính** – điều này có nghĩa là nó không yêu cầu phải chính xác tuyệt đối, mà chỉ cần cung cấp một gợi ý về khoảng cách tương đối từ trạng thái hiện tại đến đích.
- Một **hàm heuristic tốt** sẽ giúp thuật toán tránh mở rộng những trạng thái ít hứa hẹn, từ đó **tiết kiệm đáng kể thời gian và bộ nhớ**.
- Hàm này thường dựa trên **tri thức đặc thù của bài toán** (problem-specific knowledge), ví dụ như vị trí hình học trong bài toán định tuyến, hay cấu trúc trạng thái trong bài toán xếp hình.

Lưu ý quan trọng: $h(n)$ **không phải là chi phí thực tế**, do đó nó **không được dùng độc lập để đảm bảo lời giải tối ưu**, trừ khi nó tuân theo các tính chất đặc biệt như *admissible* (không vượt quá chi phí thực) hoặc *consistent* (đơn điệu).

Việc thiết kế một hàm heuristic hiệu quả là **một nghệ thuật** và thường đòi hỏi sự hiểu biết sâu sắc về bản chất của bài toán đang được giải quyết.

2.5.2. Ví dụ về thiết kế hàm Heuristic

Để hiểu rõ vai trò và cách xây dựng một hàm heuristic hiệu quả, ta cùng xét hai ví dụ tiêu biểu sau:

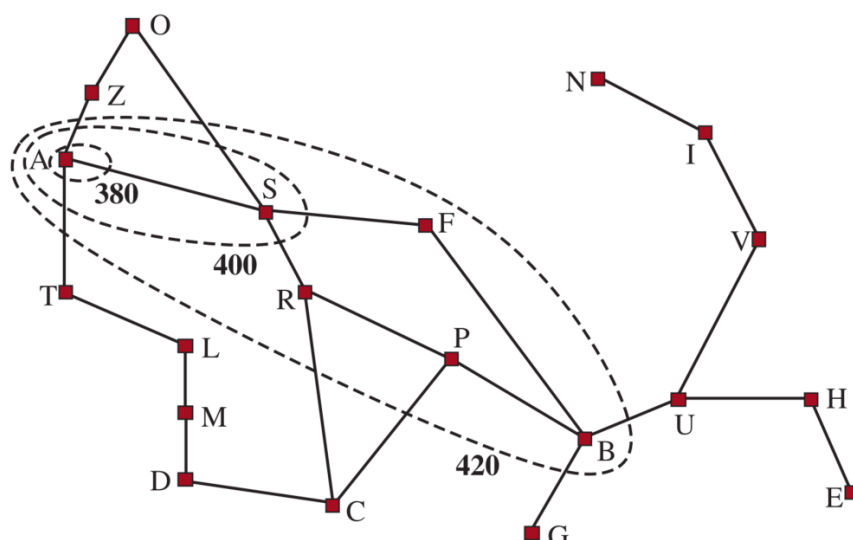
(1) Bài toán tìm đường (Route Finding Problem)

Trong bài toán này, mỗi trạng thái là một thành phố, và mục tiêu là tìm đường đi ngắn nhất từ một thành phố xuất phát đến một thành phố đích.

- **Heuristic đề xuất:**
 $h(n) = \text{khoảng cách đường chim bay (Euclidean distance)}$ từ thành phố hiện tại n đến thành phố đích.
- **Giải thích:** Vì đường chim bay là đường thẳng nối hai điểm trong không gian hai chiều, nên nó **luôn nhỏ hơn hoặc bằng quãng đường đi thực tế** theo đường bộ.

- **Tính chất:** Đây là một **heuristic admissible**, vì nó **không bao giờ vượt quá chi phí thực** từ n đến đích – do đó đảm bảo tính tối ưu khi dùng trong thuật toán A^* .

Ví dụ minh họa: Trong bài toán tìm đường qua các thành phố Romania (Arad $\rightarrow$ Bucharest), hàm heuristic thường dùng là khoảng cách đường chim bay từ mỗi thành phố đến Bucharest.



Hình 56. Bản đồ Romania với các thành phố và khoảng cách giữa các đoạn đường. (Nguồn: Stuart J. Russell, Peter Norvig - *Artificial Intelligence: A Modern Approach*, Global Edition-Pearson (2021))

Hình trên mô tả bài toán kinh điển về tìm đường đi tại Romania. Sơ đồ này biểu diễn một mạng lưới các thành phố được kết nối bởi các tuyến đường bộ. Quan trọng hơn, nó còn chỉ rõ khoảng cách đường chim bay (tức khoảng cách Euclidean) từ mỗi thành phố đến Bucharest, một giá trị thường được sử dụng làm heuristic chấp nhận được (admissible heuristic) trong thuật toán tìm kiếm A^* .

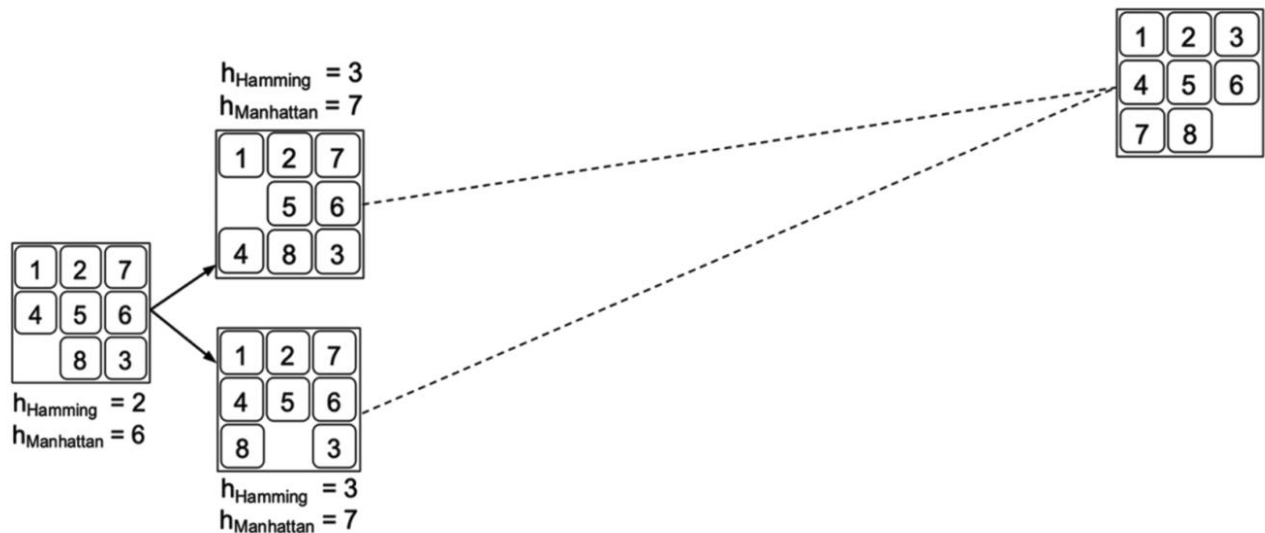
(2) Bài toán 8-puzzle

Trong bài toán này, trạng thái là một bảng 3×3 chứa 8 ô số (từ 1 đến 8) và một ô trống. Mục tiêu là đưa các ô về đúng vị trí mong muốn thông qua các phép hoán đổi với ô trống.

- **Heuristic 1 – Số ô sai vị trí (Misplaced Tiles):**
 - $h_1(n)$ = số lượng ô (không kể ô trống) **không nằm đúng vị trí so với trạng thái đích**.
 - **Ưu điểm:** Tính toán nhanh và đơn giản.
 - **Nhược điểm:** Thông tin hạn chế – không phản ánh mức độ sai lệch thực tế của từng ô.
- **Heuristic 2 – Tổng khoảng cách Manhattan:**
 - $h_2(n)$ = tổng khoảng cách Manhattan của tất cả các ô đến vị trí đích của chúng.

- Mỗi ô có khoảng cách được tính bằng **tổng số bước ngang và dọc cần di chuyển** để đến đúng vị trí.
- **Ưu điểm:** Phản ánh chính xác hơn chi phí cần thiết để đạt trạng thái đích.
- **Nhược điểm:** Tốn thời gian tính toán hơn so với $h_1(n)$.

Nhận xét: Cả hai heuristic đều **admissible**, nhưng $h_2(n)$ thường được ưa chuộng hơn vì cho kết quả sát hơn với chi phí thực, từ đó giúp **giảm số lượng trạng thái cần mở rộng**.



Hình 57. So sánh hai hàm heuristic Manhattan và số ô sai vị trí trong bài toán 8 ô. (Nguồn: Stuart J. Russell, Peter Norvig - *Artificial Intelligence: A Modern Approach, Global Edition*- Pearson (2021))

Hình trên minh họa sự so sánh giữa hai hàm heuristic phổ biến được sử dụng trong bài toán 8 ô: số ô sai vị trí (Misplaced Tile) và khoảng cách Manhattan (Manhattan Distance). Cả hai đều nhằm ước lượng chi phí còn lại để đạt đến trạng thái đích, tuy nhiên độ chính xác của mỗi hàm là khác nhau. Trong khi heuristic số ô sai vị trí chỉ đếm tổng số ô chưa nằm đúng vị trí, thì khoảng cách Manhattan tính tổng số bước cần thiết để di chuyển từng ô về đúng vị trí của nó theo chiều ngang và dọc. Hình ảnh cho thấy rõ ràng heuristic Manhattan đưa ra đánh giá chi phí gần với thực tế hơn, từ đó giúp thuật toán tìm kiếm hoạt động hiệu quả hơn và dẫn đến lời giải nhanh hơn. Đây cũng là lý do vì sao Manhattan thường được ưu tiên sử dụng trong các thuật toán tìm kiếm có sử dụng heuristic như A*.

2.5.2. Ví dụ về thiết kế hàm Heuristic

Để làm rõ cách thiết kế và áp dụng hàm heuristic trong các bài toán cụ thể, chúng ta xét hai ví dụ kinh điển trong lĩnh vực trí tuệ nhân tạo: bài toán tìm đường và bài toán xếp hình 8-puzzle.

(a) Bài toán tìm đường

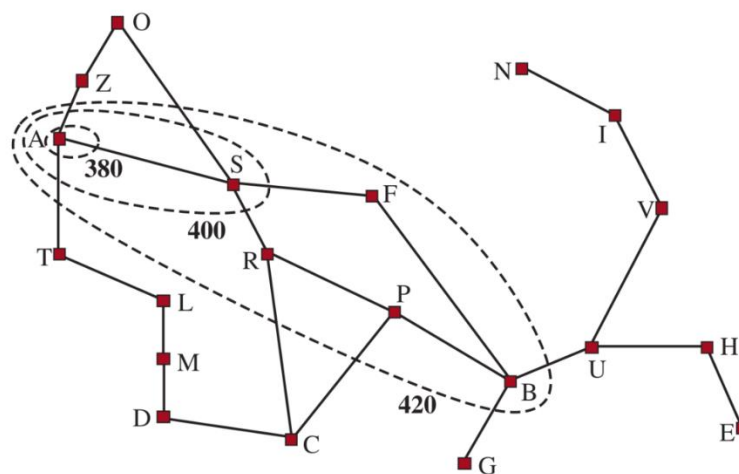
Trong bài toán tìm đường, mỗi trạng thái tương ứng với một vị trí địa lý (thường là các thành phố), và mục tiêu là tìm một lộ trình tối ưu từ điểm xuất phát đến đích qua một mạng lưới các đoạn đường có trọng số.

- **Hàm heuristic đề xuất:**

$h(n)$ = Khoảng cách đường chim bay (Euclidean distance) từ thành phố n đến đích

- **Cơ sở logic:** Đường chim bay đại diện cho đường thẳng nối hai điểm trong mặt phẳng hai chiều, do đó luôn nhỏ hơn hoặc bằng chiều dài của bất kỳ lộ trình thực tế nào tuân theo đường bộ có giới hạn. Heuristic này mang lại một ước tính hợp lý về khoảng cách còn lại đến mục tiêu mà không đòi hỏi kiến thức chi tiết về toàn bộ bản đồ.
- **Tính chất:** Đây là một hàm heuristic **admissible** vì nó không bao giờ đánh giá quá cao chi phí thực tế còn lại. Nhờ đó, khi được sử dụng trong thuật toán A^* , nó đảm bảo tính tối ưu của lời giải.

Ví dụ minh họa: Trong bài toán tìm đường giữa các thành phố ở Romania, hàm heuristic $h(n)$ được tính bằng khoảng cách thẳng từ thành phố hiện tại n đến Bucharest – thành phố đích. Cách tiếp cận này giúp thuật toán A^* nhanh chóng loại bỏ những hướng đi không hợp lý.



Hình 58. Bản đồ Romania minh họa bài toán tìm đường cổ điển. (Nguồn: Stuart J. Russell, Peter Norvig - *Artificial Intelligence: A Modern Approach*, Global Edition-Pearson (2021))

Hình 58 minh họa bài toán tìm đường cổ điển trên bản đồ Romania – một ví dụ kinh điển trong lĩnh vực trí tuệ nhân tạo. Bản đồ thể hiện các thành phố được kết nối với nhau thông qua các đoạn đường có độ dài cụ thể, mô phỏng một không gian trạng thái mà trong đó mỗi nút là một thành phố và mỗi cạnh là một hành động di chuyển. Đặc biệt, hình này còn thể hiện khoảng cách đường thẳng (khoảng cách Euclid) từ mỗi thành phố đến Bucharest – thành phố đích. Khoảng cách này thường được sử dụng như một hàm heuristic hợp lệ trong các thuật toán tìm kiếm có thông tin, đặc biệt là A^* , nhằm ước lượng chi phí còn lại đến mục tiêu một cách hiệu quả. Qua đó, hình ảnh không chỉ giúp minh họa cấu trúc không gian trạng thái mà còn làm rõ vai trò của heuristic trong việc tối ưu hóa quá trình tìm kiếm.

(b) Bài toán 8-puzzle

Bài toán 8-puzzle là một trò chơi giải đố trên bảng kích thước 3×3 , bao gồm tám ô số (từ 1 đến 8) và một ô trống. Mục tiêu là đưa các ô về đúng vị trí trong cấu hình đích, thông qua các phép di chuyển hợp lệ của ô trống.

Hai hàm heuristic phổ biến cho bài toán này bao gồm:

- **Heuristic 1: Số ô sai vị trí (Misplaced Tiles)**

$h_1(n)$ = Số ô (ngoại trừ ô trống) không nằm đúng vị trí so với trạng thái đích

- **Ưu điểm:** Dễ tính toán, chi phí tính toán thấp.
- **Nhược điểm:** Mức độ phân biệt giữa các trạng thái còn hạn chế, do không phản ánh khoảng cách thực tế mà mỗi ô cần di chuyển.

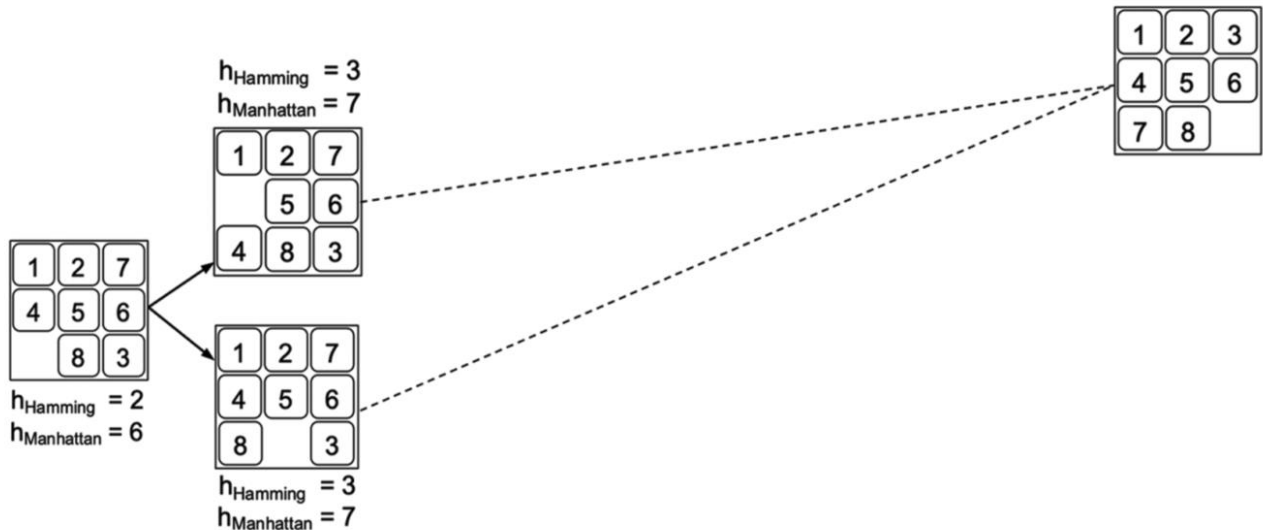
- **Heuristic 2: Tổng khoảng cách Manhattan (Manhattan Distance)**

$$h_2(n) = \sum_{i=1}^8 (|x_i - x_i^*| + |y_i - y_i^*|)$$

Trong đó, (x_i, y_i) là vị trí hiện tại của ô số i , và (x_i^*, y_i^*) là vị trí mong muốn trong trạng thái đích.

- **Ưu điểm:** Cung cấp một ước lượng sát hơn với chi phí thực tế cần thiết để đạt mục tiêu, nhờ đó cải thiện hiệu quả tìm kiếm.
- **Nhược điểm:** Chi phí tính toán cao hơn so với $h_1(n)$, nhưng đổi lại là độ chính xác cao hơn.

Đánh giá: Cả hai hàm $h_1(n)$ và $h_2(n)$ đều là **heuristic admissible**, tuy nhiên $h_2(n)$ thường được ưu tiên hơn trong thực tế do khả năng hướng dẫn tìm kiếm tốt hơn.



Hình 59. So sánh hai hàm heuristic Manhattan và số ô sai vị trí trong bài toán 8 ô. (Nguồn: Stuart J. Russell, Peter Norvig - *Artificial Intelligence: A Modern Approach, Global Edition*-Pearson (2021))

Hình 59 minh họa sự so sánh giữa hai hàm heuristic phổ biến trong bài toán 8 ô: số ô sai vị trí (Misplaced Tile) và khoảng cách Manhattan (Manhattan Distance). Cả hai đều được sử dụng để ước lượng chi phí còn lại từ trạng thái hiện tại đến trạng thái mục tiêu trong các thuật toán tìm kiếm có thông tin như A*. Trong khi heuristic số ô sai vị trí chỉ đơn giản đếm số lượng ô chưa đúng vị trí, thì khoảng cách Manhattan tính tổng số bước di chuyển cần thiết để đưa mỗi ô về đúng vị trí đích, theo trục ngang và dọc. Hình ảnh cho thấy rằng khoảng cách Manhattan thường cung cấp ước lượng chính xác hơn, từ đó dẫn đến quá trình tìm kiếm hiệu quả hơn cả về thời gian lẫn số bước mở rộng. Đây cũng là lý do vì sao heuristic Manhattan thường được ưu tiên trong các ứng dụng thực tế của bài toán 8 ô.

2.5.3. Các tiêu chí quan trọng của hàm Heuristic

Trong việc thiết kế và đánh giá các hàm heuristic, hai tiêu chí quan trọng hàng đầu là **tính chấp nhận được (admissibility)** và **tính nhất quán (consistency)**. Đây là các thuộc tính then chốt đảm bảo hiệu quả và độ tin cậy của các thuật toán tìm kiếm có thông tin, đặc biệt là A*.

(a) Tính chấp nhận được (Admissibility)

Một hàm heuristic được gọi là **admissible** nếu nó không bao giờ đánh giá quá cao chi phí thực tế còn lại từ một trạng thái đến trạng thái đích. Xem thêm Hình 58 minh họa tính chất admissible của hàm heuristic Euclidean distance trong bài toán tìm đường Romania.

- **Định nghĩa chính thức:**

$$h(n) \leq h^*(n), \quad \forall n$$

Trong đó, $h(n)$ là giá trị ước lượng của heuristic tại nút n , và $h^*(n)$ là chi phí thực tế nhỏ nhất để đi từ n đến đích.

- **Hệ quả:** Với một hàm heuristic admissible, thì:

$$h(\text{goal}) = 0$$

vì chi phí để đi từ trạng thái đích đến chính nó là bằng 0.

- **Ý nghĩa trong thuật toán A*:** Nếu heuristic được sử dụng là admissible, A* **luôn đảm bảo tìm được lời giải tối ưu**, vì nó không bao giờ đánh giá sai theo hướng lạc quan (overestimate).

Ví dụ: Hàm khoảng cách đường chim bay trong bài toán tìm đường là một heuristic admissible, vì nó không bao giờ vượt quá chi phí thực đi theo đường bộ.
(Nguồn: Stuart J. Russell, Peter Norvig - *Artificial Intelligence: A Modern Approach*, Global Edition-Pearson (2021))

(b) Tính nhất quán (Consistency)

Tính nhất quán – còn gọi là tính đơn điệu – yêu cầu rằng **ước lượng chi phí tại một nút không lớn hơn chi phí để chuyển sang nút kế tiếp cộng với ước lượng tại nút kế tiếp đó**.

- **Định nghĩa chính thức:** Với mọi cặp trạng thái liên tiếp n và n' , có:

$$h(n) \leq c(n, a, n') + h(n')$$

Trong đó, $c(n, a, n')$ là chi phí thực để đi từ n đến n' qua hành động a .

- **Hệ quả:** Mọi hàm heuristic **consistent thì luôn admissible**, nhưng điều ngược lại **không đúng**. Vì vậy, consistency là một điều kiện **mạnh hơn** admissibility.
- **Lợi ích khi heuristic consistent:**
 - A* không cần phải cập nhật lại nút đã mở rộng (do không có trường hợp “lùi bước” tối ưu).
 - Cấu trúc của hàm $f(n) = g(n) + h(n)$ duy trì tính tăng dần khi duyệt các nút, giúp thuật toán hoạt động hiệu quả hơn và đơn giản hóa việc triển khai.

Ví dụ: Trong bài toán 8-puzzle, hàm heuristic tổng khoảng cách Manhattan là một hàm consistent, vì mỗi bước di chuyển của một ô làm thay đổi chi phí không quá một đơn vị và không vi phạm bất đẳng thức tam giác.

2.6. Tìm kiếm Greedy Best-First: "Tham lam" đi về phía trước

2.6.1. Nguyên lý

Thuật toán **Greedy Best-First Search** (tạm dịch: Tìm kiếm tốt nhất theo kiểu tham lam) là một chiến lược tìm kiếm có thông tin sử dụng trực tiếp giá trị của hàm heuristic để điều hướng việc mở rộng không gian trạng thái. Đây là một phương pháp **đơn giản nhưng mạnh mẽ**, đặt mục tiêu tiến nhanh nhất về phía trạng thái đích dựa trên nhận định hiện tại. Xem lại Hình 53 để minh họa cho Greedy Best-First Search sử dụng hSLD.

- **Chiến lược chính:** Tại mỗi bước, thuật toán lựa chọn và mở rộng nút n có **giá trị heuristic thấp nhất**, tức nút được xem là “gần đích” nhất theo ước lượng.

- **Hàm đánh giá:**

$$f(n) = h(n)$$

Thuật toán **bỏ qua hoàn toàn chi phí đã đi từ trạng thái gốc đến n** (tức không xét đến $g(n)$), và chỉ dựa vào giá trị dự đoán còn lại đến đích.

- **Giải thích tên gọi "Greedy":** Từ “Greedy” phản ánh đặc điểm cốt lõi của thuật toán – nó luôn chọn phương án **có vẻ tối ưu nhất ngay tại thời điểm hiện tại**, bất kể hậu quả lâu dài hay chi phí đã phát sinh trước đó. Do đó, thuật toán có thể nhanh chóng hướng về đích, nhưng không đảm bảo lời giải tìm được là tối ưu.
- **Ưu điểm:**
 - Dễ cài đặt và vận hành nhanh với một heuristic hợp lý.
 - Thường tìm ra lời giải với tốc độ cao trên các bài toán có không gian tìm kiếm lớn.
- **Hạn chế:**
 - **Không đảm bảo tìm ra lời giải tối ưu**, vì không xét chi phí đã đi.
 - Dễ rơi vào bẫy cục bộ (local minima) hoặc đi lạc nếu heuristic kém chính xác.
 - Có thể mở rộng nhiều trạng thái không cần thiết nếu heuristic không đủ phân biệt.

Greedy Best-First Search là phương pháp tìm kiếm "hướng đích" một cách trực tiếp dựa vào thông tin từ heuristic. Tuy nhiên, để đảm bảo lời giải tối ưu và hiệu quả ổn định, cần có sự đánh đổi giữa tốc độ và độ tin cậy của lời giải.

2.6.2. Hạn chế và Đánh giá

Mặc dù Greedy Best-First Search có khả năng đưa ra lời giải nhanh trong nhiều tình huống thực tế nhờ tận dụng thông tin từ hàm heuristic, thuật toán này cũng tồn tại một số hạn chế rõ rệt về mặt lý thuyết và thực tiễn. Việc đánh giá chính xác các đặc tính của nó là rất quan trọng để quyết định khi nào nên hoặc không nên áp dụng chiến lược "tham lam" này. Hình Figure 3.17 (đã giới thiệu trong mục 2.6.1) không chỉ minh họa nguyên lý hoạt động của Greedy Best-First Search, mà còn gián tiếp phản ánh các rủi ro điển hình của chiến lược này, như chọn đường tưởng "gần đích" nhưng thực tế lại không hiệu quả.

- **Tính đầy đủ (Completeness):** Greedy Best-First Search **không đảm bảo tính đầy đủ**, đặc biệt trong các không gian tìm kiếm vô hạn hoặc có chu trình. Trong những trường hợp như vậy, thuật toán có thể **đi vào vòng lặp vô tận** hoặc **rơi vào đường cụt** mà không tìm được lời giải, nếu không có cơ chế phát hiện trạng thái đã duyệt.

- **Tính tối ưu (Optimality):** Thuật toán **không đảm bảo tìm ra lời giải tối ưu**. Do chỉ xét đến giá trị heuristic $h(n)$ mà bỏ qua chi phí đã phát sinh $g(n)$, thuật toán có thể chọn một hướng tưởng chừng như "gần đích", nhưng thực tế lại dẫn đến con đường **rất dài hoặc tốn kém**.

Ví dụ minh họa: Trong bài toán tìm đường, một nút có giá trị $h(n)$ nhỏ (tức gần đích về khoảng cách thẳng) có thể dẫn đến một tuyến đường vòng, phải đi qua các địa hình phức tạp như núi hoặc vùng cấm, khiến tổng chi phí thực tế cao hơn nhiều so với các lựa chọn khác.

- **Độ phức tạp thời gian và không gian:** Trong trường hợp xấu nhất, Greedy Best-First Search có độ phức tạp **thời gian và không gian** là:

$$O(b^m)$$

với:

- b : hệ số phân nhánh (branching factor)
- m : độ sâu tối đa của lời giải

Mặc dù vậy, nếu hàm heuristic đủ chính xác và phân biệt tốt giữa các trạng thái, số lượng trạng thái cần mở rộng có thể **giảm đáng kể**, từ đó cải thiện hiệu năng thực tế của thuật toán.

Greedy Best-First Search là một phương pháp tìm kiếm nhanh và trực quan, nhưng tiềm ẩn nhiều rủi ro về độ đầy đủ và tối ưu. Việc sử dụng thuật toán này nên đi kèm với đánh giá cẩn trọng về chất lượng của heuristic và cấu trúc không gian trạng thái cụ thể.

2.7. Thuật toán A (A-star): Sự kết hợp hoàn hảo

Trong lĩnh vực tìm kiếm có thông tin (informed search), **A* (A-star)** được xem là một trong những thuật toán nền tảng, mạnh mẽ và được ứng dụng rộng rãi nhất. Sức mạnh của A* đến từ cách nó kết hợp một cách thông minh hai chiến lược tìm kiếm: một bên là sự cẩn trọng, tối ưu của **Tìm kiếm chi phí đồng nhất (Uniform-Cost Search - UCS)** và một bên là khả năng định hướng, tốc độ của **Tìm kiếm Tham lam Tốt nhất Đầu tiên (Greedy Best-First Search)**.

2.7.1. Nguyên lý cốt lõi

Trọng tâm của thuật toán A* là việc sử dụng một hàm đánh giá toàn diện, ký hiệu là $f(n)$, để xác định thứ tự ưu tiên mở rộng các nút. Hàm này ước tính tổng chi phí của một đường đi từ trạng thái bắt đầu đến trạng thái đích nếu đường đi đó đi qua nút n .

Công thức của hàm đánh giá được định nghĩa như sau:

$$f(n) = g(n) + h(n)$$

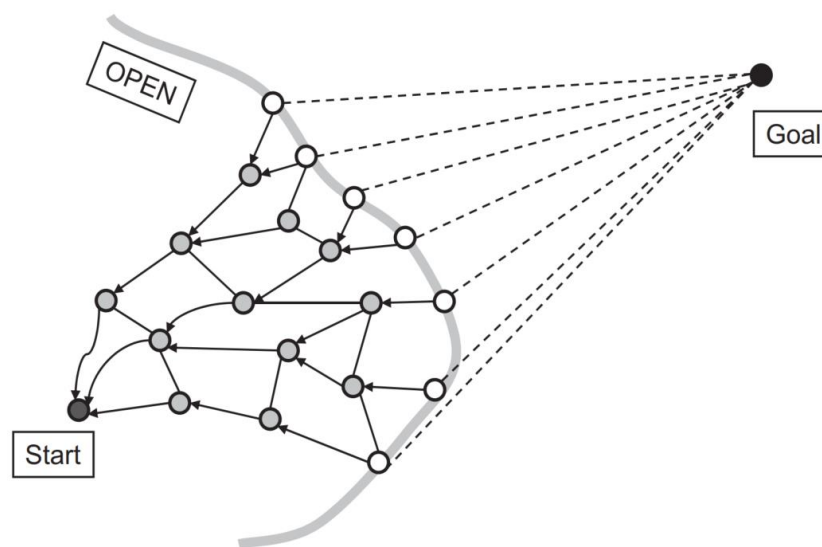
Trong đó, mỗi thành phần đóng một vai trò riêng biệt và quan trọng:

- **$g(n)$ - Chi phí thực tế đã đi:** Đây là chi phí chính xác của đường đi đã được tìm thấy từ nút khởi đầu đến nút hiện tại n . Thành phần này kế thừa trực tiếp từ thuật toán UCS. Bằng cách tối thiểu hóa $g(n)$, A* đảm bảo rằng nó luôn ưu tiên những đường đi

có chi phí thấp, tránh sa đà vào các nhánh quá “đắt đỏ”. Nếu ta bỏ qua thành phần heuristic ($h(n) = 0$), A* sẽ hoạt động giống hệt như UCS.

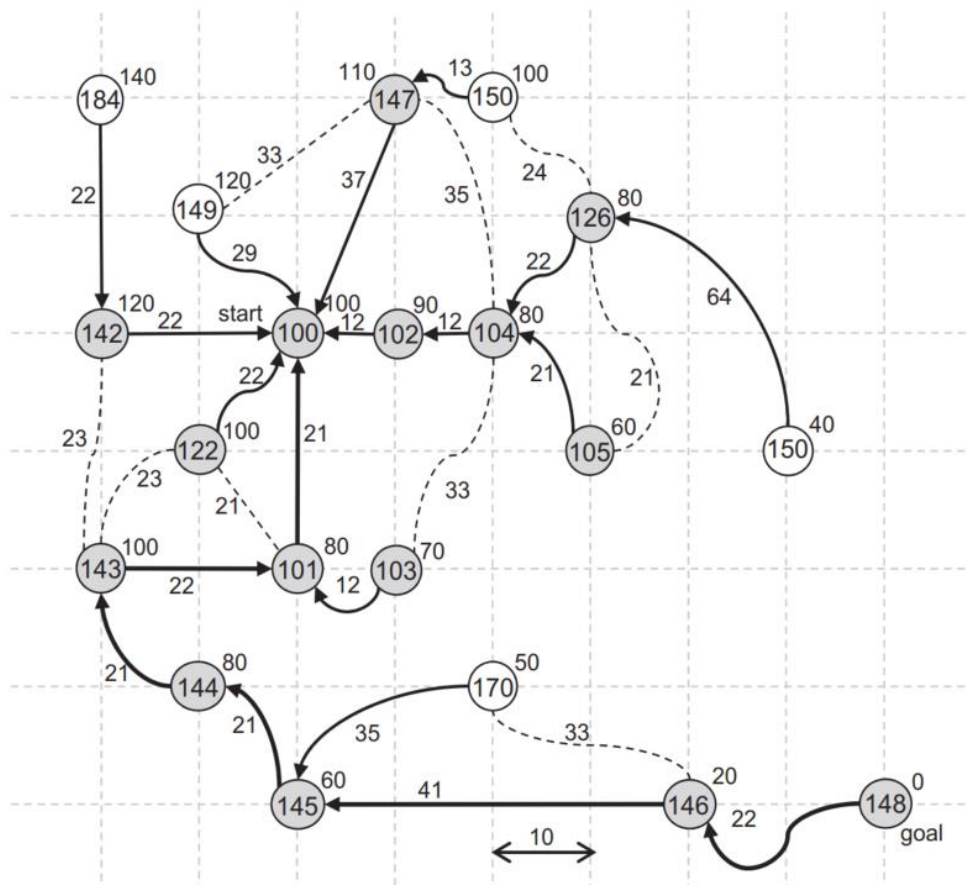
- ** $h(n)$ - Chi phí ước tính đến đích (Heuristic):** Đây là chi phí *ước tính* từ nút n đến trạng thái đích. Giá trị này được cung cấp bởi một **hàm heuristic**, là “bộ não” của thuật toán, giúp định hướng tìm kiếm. Thành phần này được kế thừa từ Greedy Best-First Search. Bằng cách tối thiểu hóa $h(n)$, A* ưu tiên các nút có vẻ gần với đích nhất, giúp quá trình tìm kiếm đi đúng hướng và nhanh hơn đáng kể so với các thuật toán “mù”. Nếu ta bỏ qua chi phí thực tế ($g(n) = 0$), A* sẽ hoạt động như một thuật toán tham lam.

Sự kết hợp của $g(n)$ và $h(n)$ tạo ra một cơ chế cân bằng hoàn hảo. A* không chỉ “tham lam” chạy về phía trước mà còn tính đến “cái giá phải trả” cho quãng đường đã đi. Nó luôn mở rộng nút có giá trị $f(n)$ thấp nhất, tức là nút nằm trên con đường hứa hẹn nhất về tổng chi phí.



Hình 60. Minh họa cách thuật toán A hoạt động trên không gian trạng thái với hàm $f(n) = g(n) + h(n)$.

Hình 60 mô tả hàm đánh giá của A trong không gian trạng thái.** Đối với mỗi nút n , thuật toán tính toán một chi phí ước tính $f(n)$. Chi phí này là tổng của $g(n)$ – chi phí đã biết từ điểm xuất phát đến n (đường liền nét) – và $h(n)$, chi phí heuristic ước tính từ n đến đích (đường đứt nét). A* sẽ luôn ưu tiên mở rộng nút có tổng chi phí ước tính $f(n)$ nhỏ nhất.



Hình 61. Quá trình thuật toán A tìm đến nút đích trên đồ thị – minh họa giá trị $f(n)$ và heuristic Manhattan.

Hình 61. Ví dụ về quá trình A tìm đường đi.
Sơ đồ này minh họa các bước A* thực hiện để tìm đường đi tối ưu. Các nút được tô bóng là những nút đã được duyệt và đưa vào danh sách CLOSED. Giá trị bên trong mỗi nút là tổng chi phí ước tính $f(n)$, trong khi giá trị bên ngoài là chi phí heuristic $h(n)$ (ở đây sử dụng khoảng cách Manhattan). Thuật toán đã duyệt qua 14 nút để tìm thấy đường đi tối ưu với tổng chi phí là 148.

Thuật toán A* là một chiến lược tìm kiếm toàn diện và hiệu quả, nổi bật nhờ khả năng cân bằng giữa tính chính xác và tốc độ. Tuy nhiên, hiệu suất và đặc biệt là **tính tối ưu** của A* phụ thuộc hoàn toàn vào chất lượng của hàm heuristic $h(n)$. Việc thiết kế một hàm heuristic tốt – cụ thể là phải **chấp nhận được (admissible)** và lý tưởng nhất là **nhất quán (consistent)** – là chìa khóa để khai thác tối đa sức mạnh và đảm bảo A* luôn tìm ra lời giải tốt nhất có thể.

2.7.2. Cách Thức Hoạt Động Chi Tiết Của Thuật Toán A*

Thuật toán A* vận hành thông qua một quy trình tìm kiếm có hệ thống, luôn ưu tiên các hướng đi hứa hẹn nhất. Để quản lý quá trình này, A* sử dụng hai cấu trúc dữ liệu chính: một danh sách mở (OPEN) và một danh sách đóng (CLOSED).

- **Danh sách OPEN (Frontier):** Đây là một **hàng đợi ưu tiên (priority queue)**, chứa tất cả các nút đã được tạo ra nhưng chưa được duyệt. Các nút được sắp xếp dựa trên giá trị $f(n)$, trong đó nút có $f(n)$ nhỏ nhất sẽ luôn có độ ưu tiên cao nhất.

- **Danh sách CLOSED (Explored Set):** Đây là một tập hợp chứa tất cả các nút đã được lấy ra khỏi OPEN và đã được mở rộng. Mục đích của danh sách này là để ngăn thuật toán duyệt lại những trạng thái đã xét, tránh các vòng lặp vô tận trong đồ thị có chu trình.

Quá trình tìm kiếm của A* là một vòng lặp liên tục cho đến khi tìm thấy đích:

- **Khởi tạo:** Bắt đầu với danh sách OPEN chỉ chứa nút khởi đầu. Danh sách CLOSED rỗng.
- **Lặp lại các bước sau:** a. Kiểm tra nếu danh sách OPEN rỗng. Nếu đúng, quá trình tìm kiếm thất bại và không có lời giải nào tồn tại. b. Lấy nút n có giá trị $f(n)$ nhỏ nhất từ danh sách OPEN. c. Nếu n là trạng thái đích, thuật toán kết thúc thành công. Đường đi được truy vết ngược từ n về nút khởi đầu. d. Chuyển nút n vào danh sách CLOSED. e. **Mở rộng nút n :** Tạo ra tất cả các nút kề (successors) của n . Đối với mỗi nút kề n' , thực hiện: i. Tính chi phí $g(n')$ mới thông qua đường đi qua n . ii. Nếu n' đã có trong danh sách OPEN hoặc CLOSED với một đường đi tốt hơn (chi phí g thấp hơn), bỏ qua nút n' này. iii. Ngược lại, tính $f(n') = g(n') + h(n')$ và thêm n' vào danh sách OPEN (hoặc cập nhật vị trí của nó nếu nó đã có trong OPEN với chi phí g cao hơn).

Việc luôn chọn nút có $f(n)$ nhỏ nhất là chìa khóa giúp A* cân bằng giữa việc đi theo con đường rẻ nhất đã biết ($g(n)$) và việc hướng về phía mục tiêu một cách thông minh ($h(n)$).

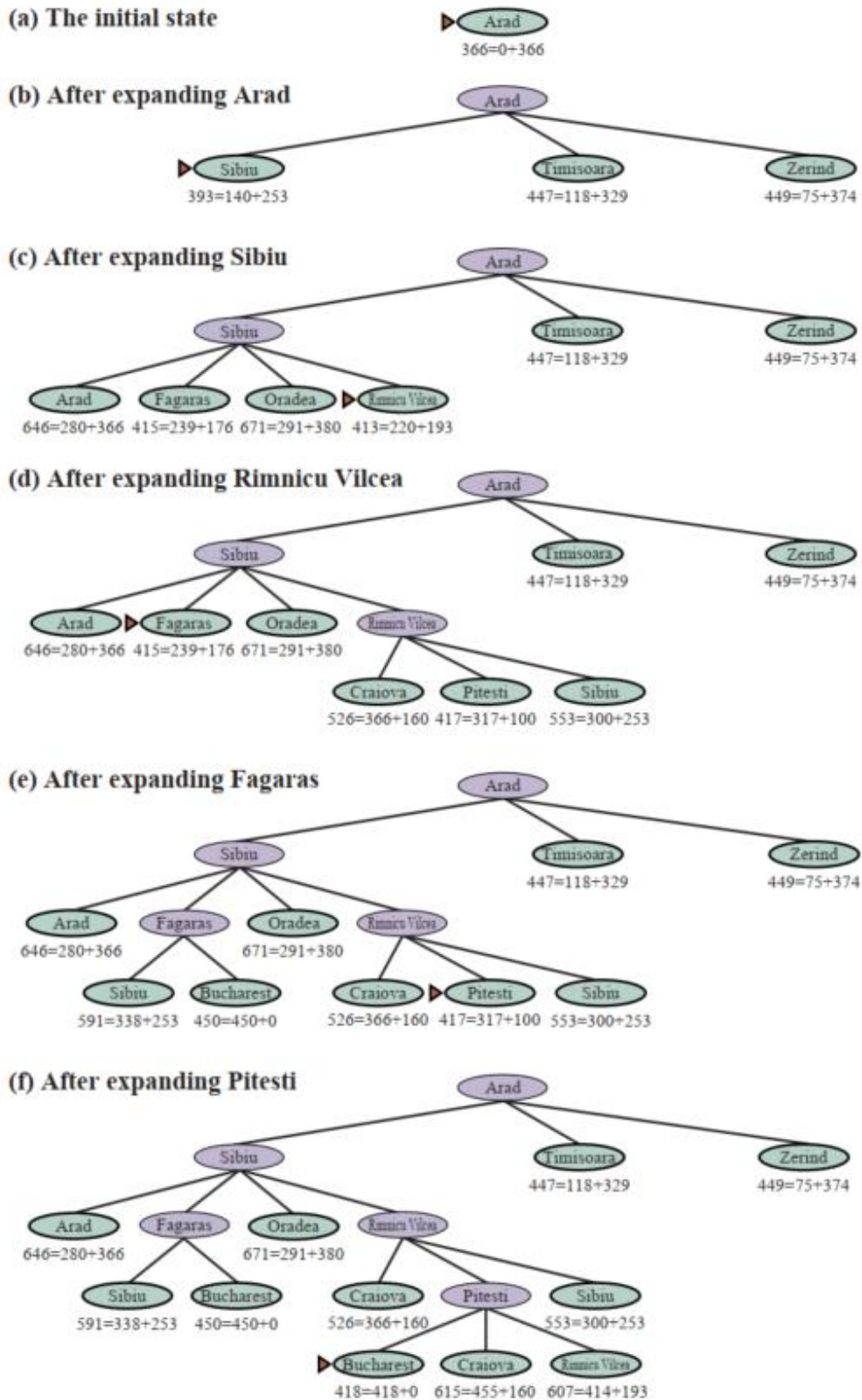
Ví dụ Minh Họa: Tìm đường từ Arad đến Bucharest

Hãy xem cách A* hoạt động trên bài toán tìm đường kinh điển ở Romania. Ta sử dụng **khoảng cách đường chim bay (Straight-Line Distance - h_{SLD})** làm hàm heuristic.

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Hình 62. Giá trị h_{SLD} – Khoảng cách đường thẳng đến Bucharest.

Bảng giá trị trên cung cấp thông tin heuristic cho thuật toán, vốn không có sẵn trong bản đồ đường bộ. Ví dụ, $h_{SLD}(\text{textArad}) = 366$.



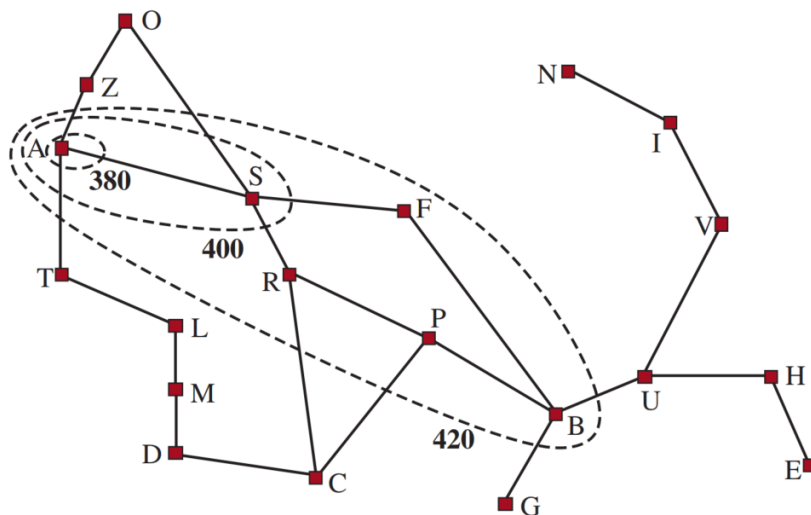
Hình 63. Các bước trong quá trình tìm kiếm Arad đến Bucharest bằng A*.

Hình 63. Các giai đoạn trong một lượt tìm kiếm A từ Arad đến Bucharest. Các nút được gán nhãn với giá trị $f = g + h$.

- **Bắt đầu tại Arad:** $f(\text{textArad}) = g(\text{textArad}) + h(\text{textArad}) = 0 + 366 = 366$
- **Mở rộng Arad:** Các nút kề được thêm vào OPEN:
 - Sibiu: $f(\text{textSibiu}) = (g = 140) + (h = 253) = 393$
 - Zerind: $f(\text{textZerind}) = (g = 75) + (h = 374) = 449$
 - Timisoara: $f(\text{textTimisoara}) = (g = 118) + (h = 329) = 447$
- **Chọn nút tiếp theo:** A* chọn nút có f nhỏ nhất trong OPEN, đó là **Sibiu** (393).
- **Mở rộng Sibiu:** Quá trình tiếp tục, A* sẽ mở rộng các nút từ Sibiu, và cứ thế, luôn ưu tiên nút có tổng chi phí ước tính thấp nhất cho đến khi đến được Bucharest.

Trực Quan Hóa Quá Trình Tìm Kiếm

Một cách hữu ích để hình dung quá trình tìm kiếm của A* là vẽ các **đường đồng mức (contours)** trong không gian trạng thái. Mỗi đường đồng mức tương ứng với một giá trị f nhất định.



Hình 64. Bản đồ Romania với các đường đồng mức $f = 380, f = 400$ và $f = 420$, bắt đầu từ Arad.

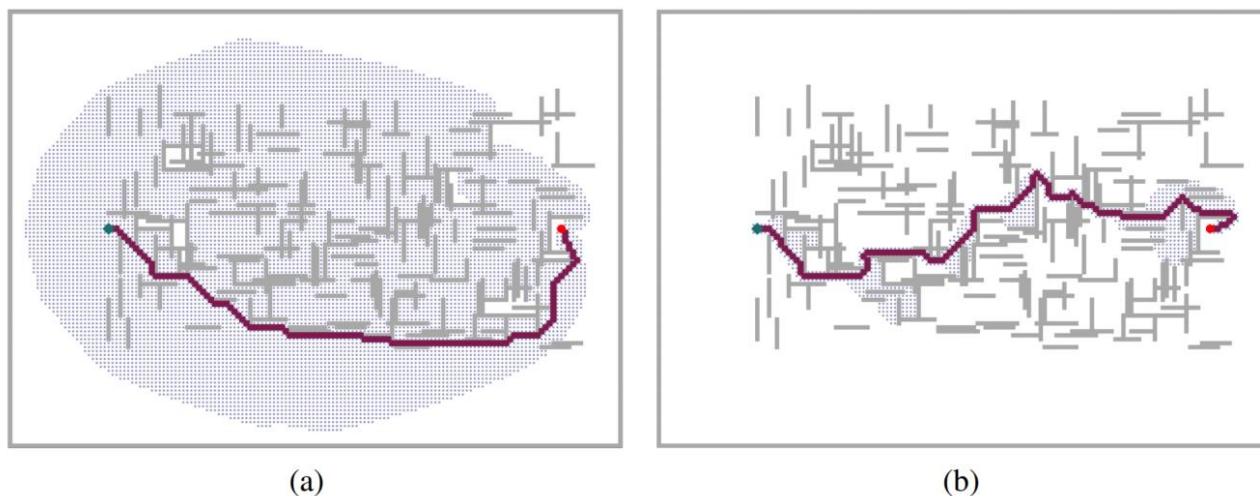
Hình 64. Bản đồ Romania với các đường đồng mức $f = 380, f = 400, f = 420$ từ Arad. A* mở rộng các nút theo các dải đồng mức có giá trị f tăng dần. Nhờ có heuristic tốt, các dải này bị “kéo dài” và tập trung về phía đích (Bucharest), thay vì lan tỏa đều ra mọi hướng như trong Uniform-Cost Search.

Biến Thể và Sự Đánh Đổi: A có trọng số (Weighted A*)

Trong nhiều ứng dụng thực tế, việc tìm ra lời giải nhanh chóng còn quan trọng hơn tìm ra lời giải tối ưu tuyệt đối. A có trọng số * là một biến thể đáp ứng nhu cầu này bằng cách tăng tầm ảnh hưởng của heuristic:

$$f(n) = g(n) + W \times h(n) \text{ với } W > 1$$

Việc sử dụng trọng số W khiến thuật toán trở nên “tham lam” hơn, tập trung mạnh mẽ hơn vào hướng của heuristic.



Hình 65. So sánh giữa thuật toán A^* và A^* có trọng số trên cùng một lưới tìm kiếm.

Hình 65 so sánh giữa (a) A^* chuẩn và (b) A^* có trọng số ($W=2$). Đường màu tím là đường đi, các chấm nhỏ là các trạng thái đã được duyệt. A^* có trọng số duyệt ít trạng thái hơn đáng kể (nhanh hơn) nhưng tìm thấy một đường đi dài hơn 5%. Đây là sự đánh đổi kinh điển giữa **tốc độ** và **tính tối ưu**.

2.7.3. Các đặc tính vàng của A^*

Thuật toán A^* được xem là một trong những chiến lược tìm kiếm mạnh nhất trong lĩnh vực trí tuệ nhân tạo vì nó đạt được những thuộc tính lý tưởng mà nhiều thuật toán khác không thể đảm bảo đồng thời. Dưới đây là các đặc tính chính mà A^* có thể đạt được khi được kết hợp với một hàm heuristic phù hợp.

- **Tính đầy đủ (Completeness):** A^* là thuật toán **đầy đủ**, nghĩa là nếu tồn tại một lời giải, thuật toán **chắc chắn sẽ tìm thấy** nó, với điều kiện không gian trạng thái là hữu hạn và chi phí chuyển trạng thái luôn lớn hơn một hằng số dương (tránh vòng lặp vô hạn với chi phí bằng 0).
- **Tính tối ưu (Optimality):** A^* đảm bảo **tìm ra lời giải tối ưu** nếu hàm heuristic $h(n)$ thỏa mãn tính **admissible**, tức không bao giờ đánh giá quá cao chi phí thực tế còn lại. Nếu heuristic còn thỏa tính **consistent**, thuật toán có thể được triển khai một cách hiệu quả hơn mà không cần phải kiểm tra lại các nút đã mở rộng.

Chứng minh sơ lược tính tối ưu của A^*

Giả sử có một trạng thái đích G_2 là lời giải không tối ưu, và G_1 là trạng thái đích thuộc đường đi tối ưu (với tổng chi phí nhỏ hơn). Ta xét hai giá trị đánh giá tương ứng:

- $f(G_2) = g(G_2) + h(G_2)$
- $f(G_1) = g(G_1) + h(G_1)$

Do $h(G_2) = 0$ (vì là đích), ta có:

$$f(G_2) = g(G_2)$$

Mặt khác, tồn tại ít nhất một nút n trên đường đi từ trạng thái bắt đầu đến G_1 , chưa được mở rộng, và do heuristic là admissible nên:

$$f(n) = g(n) + h(n) \leq g(G_1) = f(G_1) < f(G_2)$$

Điều này có nghĩa là A^* sẽ ưu tiên chọn nút n (nằm trên đường tối ưu) để mở rộng trước khi chọn G_2 , do $f(n) < f(G_2)$. Như vậy, *A sẽ không bao giờ mở rộng một lời giải không tối ưu khi vẫn còn tồn tại một đường đi tốt hơn** trong frontier. Các hình như **Figure 3.18** và **Figure 3.20 (đã trình bày ở mục trước)** trực quan cho thấy cách A duy trì ưu tiên mở rộng các nút tốt hơn về giá trị $f(n)$, từ đó đảm bảo tính tối ưu và đầy đủ. Đặc biệt, **Figure 6.9** thể hiện rõ chiến lược tránh mở rộng lời giải không tối ưu nếu còn đường tốt hơn trong OPEN list.

Với một hàm heuristic được thiết kế hợp lý, thuật toán A^* vừa **hoàn chỉnh**, vừa **tối ưu**, và là một trong số ít thuật toán thỏa mãn cả hai tiêu chí này trong cùng một khung tìm kiếm. Đây chính là lý do A^* được mệnh danh là “thuật toán vàng” trong tìm kiếm có thông tin.

2.7.4. Hiệu quả của A^*

Mặc dù thuật toán A^* về mặt lý thuyết đảm bảo tìm ra lời giải tối ưu, **hiệu suất thực tế** của nó — tức số lượng trạng thái cần mở rộng và chi phí tính toán — **phụ thuộc chặt chẽ vào chất lượng của hàm heuristic** được sử dụng. Một heuristic tốt sẽ giúp A^* tập trung tìm kiếm vào các vùng “hứa hẹn” trong không gian trạng thái, từ đó **rút ngắn thời gian và giảm lượng bộ nhớ tiêu thụ**.

- **Chất lượng của heuristic:** Một hàm heuristic được xem là **tốt** khi giá trị ước tính của nó càng gần với chi phí thực tế còn lại từ nút n đến đích, tức:

$$h(n) \approx h^*(n)$$

trong đó:

- $h(n)$: giá trị ước lượng từ heuristic.
- $h^*(n)$: chi phí thực sự từ n đến đích.

Tuy nhiên, để đảm bảo tính tối ưu của thuật toán, heuristic vẫn phải tuân thủ ràng buộc:

$$h(n) \leq h^*(n)$$

Vì phạm điều kiện này (tức $h(n) > h^*(n)$) có thể khiến A^* đánh giá sai lệch đường đi tốt nhất, dẫn đến **lời giải không tối ưu**.

- **Cân bằng giữa tính chính xác và chi phí tính toán:** Heuristic càng sát với thực tế thì càng giúp giảm số lượng trạng thái cần mở rộng, tuy nhiên:
 - Heuristic càng phức tạp, chi phí tính toán càng cao.

- Có những bài toán mà việc tính một heuristic chính xác đòi hỏi gần như phải giải toàn bộ bài toán con, làm mất đi lợi thế của việc sử dụng heuristic.

Vì vậy, trong thực tiễn, một heuristic tốt cần đảm bảo **sự cân bằng giữa tính đơn giản (efficiency) và mức độ gần đúng với chi phí thực (accuracy)**.

- **Hiệu quả tối ưu (Optimal Efficiency):** Khi sử dụng một hàm heuristic admissible và consistent, A^* được chứng minh là **thuật toán tối ưu về mặt hiệu quả** trong số tất cả các thuật toán tìm kiếm đồ thị sử dụng cùng một heuristic. Nói cách khác, **không có thuật toán nào mở rộng ít nút hơn A^*** và vẫn đảm bảo tìm được lời giải tối ưu.

Hiệu quả của A^* không chỉ đến từ cấu trúc thuật toán mà còn phụ thuộc vào **chất lượng thiết kế của hàm heuristic**. Việc đầu tư xây dựng một heuristic tốt sẽ trực tiếp dẫn đến **giảm chi phí tính toán** mà vẫn đảm bảo **độ chính xác của lời giải**. Hiệu quả của A với một heuristic tốt được minh họa rõ trong Figure 3.20, nơi không gian tìm kiếm được rút gọn nhờ các đường đồng mức $f(n)$. Bên cạnh đó, Figure 3.21 cho thấy sự khác biệt giữa A^* và A^* có trọng số, nhấn mạnh tầm quan trọng của việc duy trì tính admissible để tối ưu hóa hiệu suất mà không đánh mất tính chính xác.

2.8. Bài toán thỏa mãn ràng buộc (Constraint Satisfaction Problems - CSPs)

2.8.1. Mở đầu: Một cách nhìn khác về bài toán

Trong nhiều bài toán trong thực tiễn và lý thuyết, thay vì mô hình hóa không gian trạng thái theo kiểu truyền thống (với các hành động và trạng thái kế tiếp), một cách tiếp cận tự nhiên và hiệu quả hơn là biểu diễn vấn đề dưới dạng **tập hợp các biến cần gán giá trị**, đi kèm với **tập hợp các ràng buộc** mà các giá trị đó phải thỏa mãn. Đây chính là cốt lõi của mô hình **Constraint Satisfaction Problems (CSPs)** – Bài toán thỏa mãn ràng buộc.

- **Định nghĩa tổng quát:** Một bài toán CSP được đặc trưng bởi ba thành phần:
 - Tập hợp các biến:** $X = \{X_1, X_2, \dots, X_n\}$
 - Miền giá trị (domain) cho mỗi biến:** D_i là tập hợp giá trị khả dĩ của X_i
 - Tập ràng buộc (constraints):** Tập hợp các điều kiện logic quy định **tập hợp giá trị hợp lệ** giữa một hoặc nhiều biến.

Mục tiêu của CSP là tìm một phép gán giá trị cho tất cả các biến sao cho **mọi ràng buộc đều được thỏa mãn**.

Ví dụ minh họa: Bài toán xếp lịch học

Một ví dụ điển hình cho CSP là **bài toán xếp lịch học** tại một trường đại học. Cấu trúc bài toán có thể mô hình hóa như sau:

- **Biến:** Các tiết học cần được tổ chức (ví dụ: Giải tích 1, Cơ sở dữ liệu, Trí tuệ nhân tạo, v.v.)

- **Miền giá trị:** Tập các kết hợp hợp lệ của thời gian, phòng học, và giảng viên cho từng tiết học.
- **Ràng buộc:**
 - Một giáo viên không thể dạy hai lớp cùng lúc.
 - Một phòng học không thể được sử dụng bởi hai lớp tại cùng một thời điểm.
 - Một môn học không thể được xếp trùng với môn học khác thuộc cùng chương trình học của một lớp sinh viên.

Ý nghĩa của CSP trong trí tuệ nhân tạo

CSP là một trong những mô hình cốt lõi trong AI, đặc biệt hiệu quả cho các bài toán **mà cấu trúc ràng buộc là trung tâm**. Khác với tìm kiếm truyền thống, tìm kiếm trong CSP thường **không cần mô hình hóa rõ ràng các hành động chuyển trạng thái**, mà tập trung vào **khám phá không gian gán giá trị** sao cho hợp lệ.

Ví dụ ứng dụng khác của CSP:

- Giải Sudoku (các ô là biến, giá trị là số 1–9, ràng buộc theo hàng, cột, khối).
- Tô màu bản đồ (các vùng là biến, giá trị là màu, ràng buộc là không trùng màu giữa các vùng kề nhau).
- Phân công lịch làm việc cho nhân viên.

Kết luận: CSP cung cấp một khung mô hình hóa đơn giản nhưng mạnh mẽ để giải quyết nhiều loại bài toán có cấu trúc ràng buộc rõ ràng. Khả năng tách rời giữa **nội dung bài toán (các biến và ràng buộc)** và **thuật toán giải quyết** là một trong những ưu điểm lớn nhất của mô hình này.

2.8.2. Định nghĩa chính thức một CSP

Để hiểu rõ cấu trúc và cách thức vận hành của các bài toán thỏa mãn ràng buộc, ta cần một định nghĩa hình thức hóa đầy đủ. Một **Constraint Satisfaction Problem (CSP)** được đặc trưng bởi một bộ ba (X, D, C) , trong đó:

- **(1) Tập biến – Variables (X)**

$$X = \{X_1, X_2, \dots, X_n\}$$

Đây là **danh sách các biến cần gán giá trị**, đại diện cho các thành phần trong bài toán. Mỗi biến X_i phải nhận một giá trị từ tập giá trị cho phép tương ứng.

- **(2) Tập miền giá trị – Domains (D)**

$$D = \{D_1, D_2, \dots, D_n\}$$

Với mỗi biến X_i , miền giá trị D_i là tập các giá trị mà X_i có thể nhận. Các giá trị này thường là rời rạc và hữu hạn, ví dụ như tập màu sắc, số nguyên, hoặc các ca học cụ thể.

Ví dụ:

- Với bài toán tô màu bản đồ: $D_i = \{\text{Đỏ, Xanh, Vàng}\}$
- Với bài toán Sudoku: $D_i = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

- **(3) Tập ràng buộc – Constraints (C)**

$$C = \{C_1, C_2, \dots, C_k\}$$

Mỗi ràng buộc C_j xác định một **tập con các biến** cùng với **một điều kiện phải được thỏa mãn** giữa các giá trị được gán cho các biến này. Ràng buộc có thể là:

- **Nhị phân (binary):** Áp dụng cho hai biến, như $X_i \neq X_j$
- **Đại số:** Ví dụ $X_1 + X_2 = X_3$
- **Tổng quát (n-ary):** Áp dụng cho một tập biến bất kỳ với điều kiện phức tạp.

Ví dụ:

Trong bài toán xếp lịch:

- C_1 : Giáo viên không thể dạy hai lớp cùng lúc.
- C_2 : Một phòng học chỉ có thể chứa một lớp trong một thời điểm.
- C_3 : Một sinh viên không được xếp học hai môn trùng lịch.

Phép gán (Assignment)

Một **phép gán (assignment)** là một ánh xạ từ tập biến sang các giá trị trong miền tương ứng. Một phép gán:

- **Đầy đủ (complete)** nếu tất cả biến đã được gán.
- **Hợp lệ (consistent)** nếu nó thỏa mãn tất cả các ràng buộc liên quan.

Mục tiêu của bài toán CSP là tìm một **phép gán đầy đủ và hợp lệ**.

Cấu trúc hình thức (X, D, C) của một CSP giúp trừu tượng hóa bài toán thành một khuôn mẫu rõ ràng, tách biệt giữa thông tin bài toán và thuật toán giải quyết. Điều này tạo điều kiện thuận lợi cho việc phát triển các giải pháp tổng quát và tối ưu.

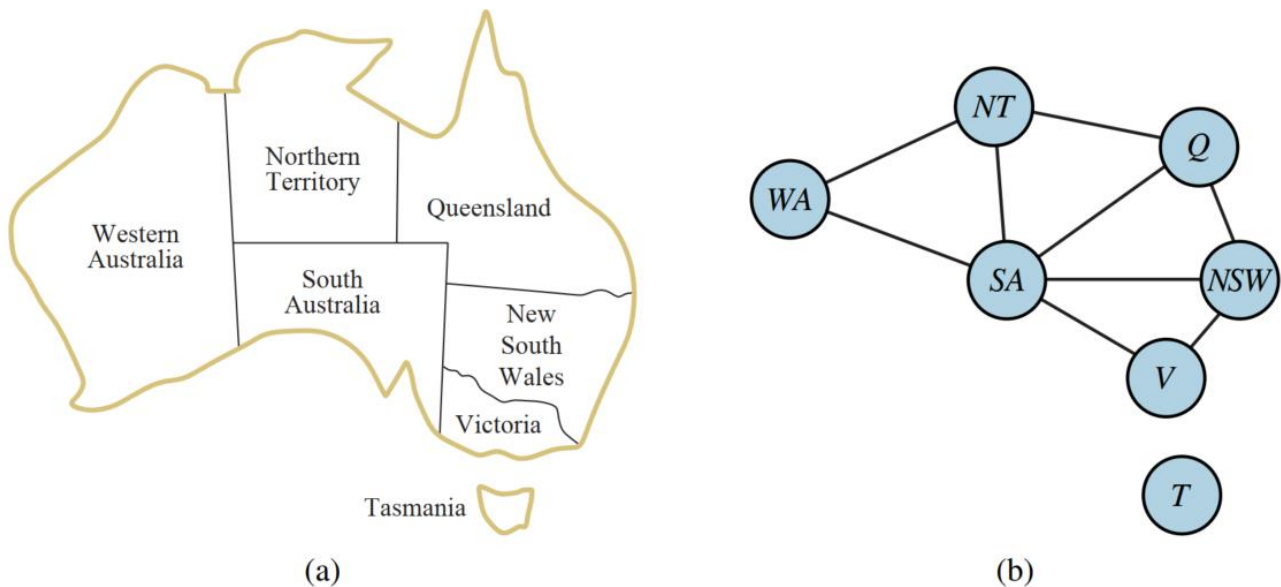
2.8.3. Các Ví Dụ Kinh Điển Về Bài Toán Thỏa Mãn Ràng Buộc (CSP)

Để hiểu rõ hơn về cấu trúc và ứng dụng của mô hình Bài toán Thỏa mãn Ràng buộc (Constraint Satisfaction Problem - CSP), chúng ta sẽ phân tích một trong những ví dụ kinh điển và trực quan nhất: **Bài toán Tô màu Bản đồ**. Ví dụ này không chỉ thể hiện rõ cách một vấn đề có thể được mô hình hóa thành bộ ba thành phần cốt lõi (X, D, C) mà còn là nền tảng để giới thiệu nhiều kỹ thuật giải quyết CSP hiệu quả.

Bài toán Tô màu Bản đồ (Map-Coloring Problem)

Đây là một bài toán nổi tiếng trong lý thuyết đồ thị và trí tuệ nhân tạo. Yêu cầu của bài toán rất đơn giản: cho một bản đồ gồm nhiều vùng lãnh thổ, hãy tô màu cho mỗi vùng sao cho không có hai vùng nào có chung đường biên giới được tô cùng một màu.

Hãy cùng phân tích bài toán tô màu bản đồ nước Úc với ba màu có sẵn (Đỏ, Xanh lá, Xanh lam) theo mô hình CSP.



Hình 66. (a) Bản đồ các bang và vùng lãnh thổ của Australia; (b) Biểu diễn bài toán tô màu bản đồ dưới dạng đồ thị ràng buộc.

Hình 66. (a) Bản đồ các bang và vùng lãnh thổ của Úc. Việc tô màu bản đồ này có thể được xem như một bài toán thỏa mãn ràng buộc. (b) Biểu diễn bài toán dưới dạng một **đồ thị ràng buộc (constraint graph)**, trong đó mỗi đỉnh là một vùng và mỗi cạnh nối hai vùng có chung biên giới.

1. Định nghĩa các Biến (Variables - X)

Trong bài toán này, mỗi vùng lãnh thổ trên bản đồ tương ứng với một biến. Đối với bản đồ nước Úc, tập hợp các biến là:

$$X = \{WA, NT, Q, NSW, V, SA, T\}$$

trong đó WA là Western Australia, NT là Northern Territory, v.v.

2. Xác định Miền giá trị (Domains - D)

Miền giá trị là tập hợp các giá trị mà mỗi biến có thể nhận. Ở đây, đó là tập hợp các màu có thể sử dụng. Giả sử chúng ta có ba màu, miền giá trị cho mỗi biến sẽ là:

$$D_i = \{\text{đỏ, xanh lá, xanh lam}\}$$

với i là một trong các biến thuộc X .

3. Thiết lập các Ràng buộc (Constraints - C)

Các ràng buộc định nghĩa những cặp giá trị hợp lệ cho các biến có liên quan. Nguyên tắc của bài toán là “hai vùng kề nhau không được có cùng màu”. Đây là loại **ràng buộc nhị phân (binary constraint)**. Dựa trên bản đồ, ta có các ràng buộc sau:

$$C = \text{textSAneqtextWA}, \text{textSAneqtextNT}, \text{textSAneqtextQ}, \text{textSAneqtextNSW}, \text{textSAneqtextV}, \\ \text{textWAneqtextNT}, \text{textNTneqtextQ}, \text{textQneqtextNSW}, \text{textNSWneqtextV}$$

Mỗi ràng buộc, ví dụ textSAneqtextWA , là một cách viết tắt cho một tập hợp các cặp giá trị được phép, chẳng hạn như: $(\text{textđỏ}, \text{xanhlá})$, $(\text{textđỏ}, \text{xanhlam})$, $(\text{textxanhlá}, \text{đỏ})$, ...

Biểu diễn bằng Đồ thị Ràng buộc và Ý nghĩa

Như trong Hình 66(b), bài toán có thể được trực quan hóa bằng một đồ thị ràng buộc. Trong đó, mỗi đỉnh là một biến (vùng lãnh thổ) và một cạnh được vẽ giữa hai đỉnh bất kỳ nếu chúng tham gia vào cùng một ràng buộc (có chung biên giới).

Việc hình thức hóa một bài toán dưới dạng CSP mang lại nhiều lợi ích to lớn:

- **Tính tự nhiên:** Mô hình CSP cung cấp một cách biểu diễn rất tự nhiên và rõ ràng cho nhiều vấn đề trong thực tế.
- **Hiệu quả giải quyết:** Các thuật toán giải CSP tổng quát đã được phát triển và tối ưu hóa trong nhiều năm, cho phép giải quyết vấn đề một cách hiệu quả.
- **Khả năng cắt tỉa không gian tìm kiếm:** Đây là ưu điểm vượt trội nhất. Khi một biến được gán một giá trị, các ràng buộc liên quan sẽ ngay lập tức giới hạn miền giá trị của các biến khác. Ví dụ, một khi ta gán $\text{textSA} = \text{xanhlam}$, ta có thể suy luận ngay rằng 5 vùng lân cận của nó (WA, NT, Q, NSW, V) không thể có màu xanh lam. Điều này giúp loại bỏ một phần khổng lồ của không gian tìm kiếm. Thay vì phải xét $3^5 = 243$ tổ hợp màu cho 5 vùng này, giờ ta chỉ cần xét $2^5 = 32$ tổ hợp, giảm tới 87% số trường hợp.

Do tính đơn giản và rõ ràng, bài toán tô màu bản đồ là một ví dụ lý tưởng để giới thiệu và minh họa các kỹ thuật giải CSP cốt lõi như **quay lui (backtracking)** và các thuật toán **lan truyền ràng buộc (constraint propagation)**.

(b) Bài toán N-Hậu (N-Queens Problem)

Bài toán N-Hậu là một bài toán tổ hợp kinh điển, yêu cầu đặt N quân hậu lên một bàn cờ vua kích thước $N \times N$ sao cho không có hai quân hậu nào có thể tấn công lẫn nhau. Theo luật cờ vua, điều này có nghĩa là không có hai quân hậu nào được nằm trên cùng một **hàng**, cùng một **cột**, hoặc cùng một **đường chéo**.

So với bài toán Tô màu bản đồ, N-Hậu thể hiện một lớp các ràng buộc phức tạp hơn (cụ thể là ràng buộc hình học), và là một bài toán tiêu chuẩn (benchmark) quan trọng để đánh giá hiệu năng của các thuật toán giải CSP.

1. Mô hình hóa bài toán N-Hậu dưới dạng CSP

Một cách tiếp cận ban đầu có thể là định nghĩa N^2 biến, mỗi biến tương ứng với một ô trên bàn cờ. Tuy nhiên, cách làm này không hiệu quả. Một cách biểu diễn thông minh và cô đọng hơn là tận dụng ràng buộc rằng mỗi hàng chỉ có thể chứa đúng một quân hậu.

- **Biến (Variables - X):** Ta định nghĩa N biến, mỗi biến Q_i đại diện cho quân hậu ở **hàng thứ i** .

$$X = \{Q_1, Q_2, \dots, Q_N\}$$

- **Miền giá trị (Domains - D):** Giá trị của mỗi biến Q_i sẽ là **vị trí cột** mà quân hậu ở hàng i được đặt. Như vậy, miền giá trị cho tất cả các biến là như nhau:

$$D_i = \{1, 2, \dots, N\}$$

Cách biểu diễn này tự nó đã thỏa mãn ràng buộc không có hai quân hậu nào trên cùng một hàng.

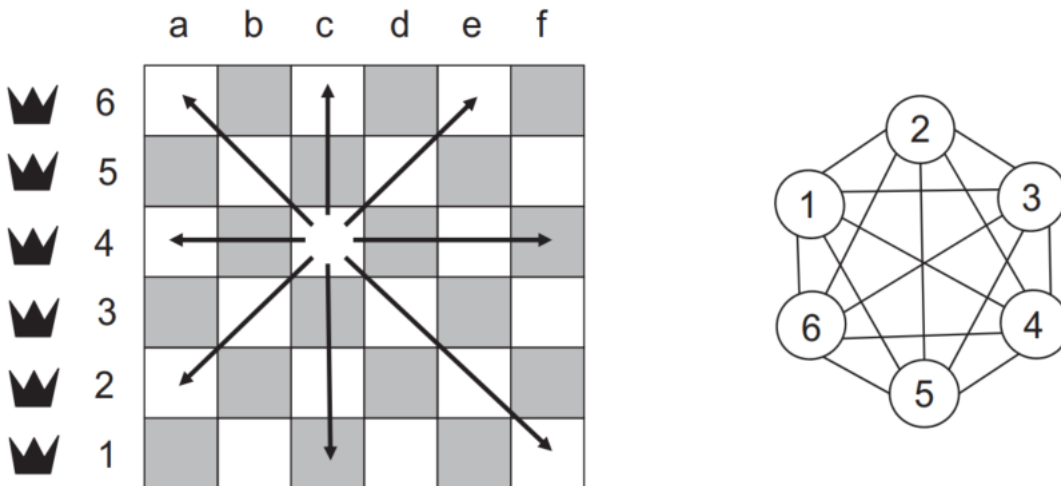
- **Ràng buộc (Constraints - C):** Với cách định nghĩa trên, chúng ta chỉ cần biểu diễn các ràng buộc còn lại cho mọi cặp biến Q_i và Q_j (với $i \neq j$):

1. **Ràng buộc cột:** Hai quân hậu không được ở trên cùng một cột.

$$Q_i \neq Q_j$$

2. **Ràng buộc đường chéo:** Hai quân hậu không được ở trên cùng một đường chéo. Điều này xảy ra khi khoảng cách theo hàng bằng khoảng cách theo cột.

$$|i - j| \neq |Q_i - Q_j|$$

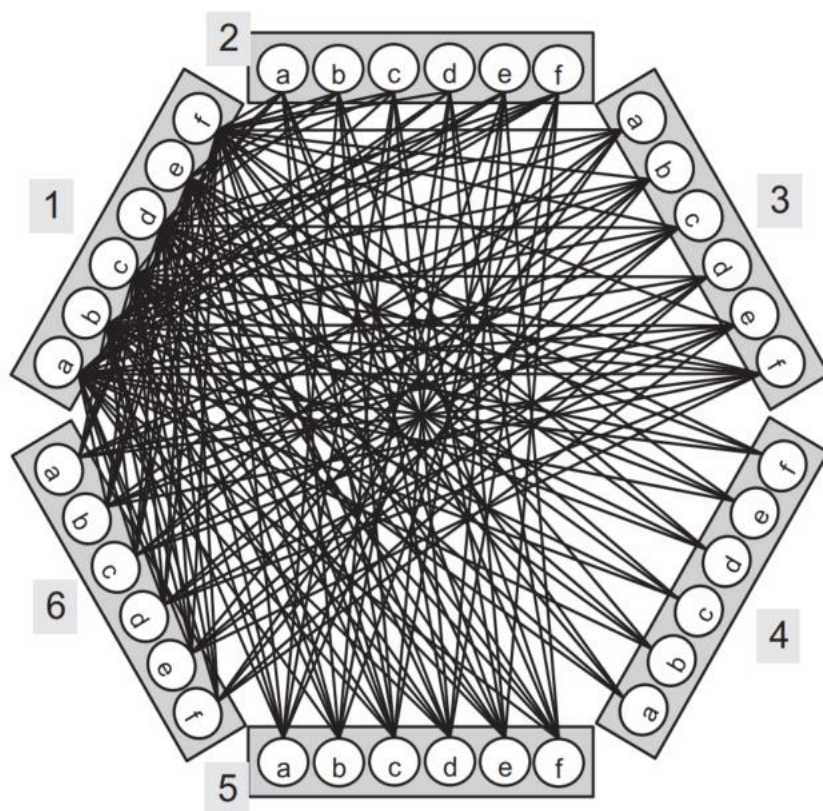


Hình 67. Biểu diễn bài toán 6 hậu trên bàn cờ 6×6 và đồ thị ràng buộc tương ứng.

Hình 67. Bên trái là bài toán 6-Hậu trên bàn cờ 6×6 , minh họa các ô bị một quân hậu tại c4 tấn công. Bên phải là đồ thị ràng buộc tương ứng. Vì mỗi quân hậu ràng buộc tất cả các quân hậu khác, đồ thị này là một **đồ thị đầy đủ (complete graph)**, cho thấy mức độ tương tác phức tạp của bài toán.

2. Trục quan hóa Ràng buộc và Lời giải

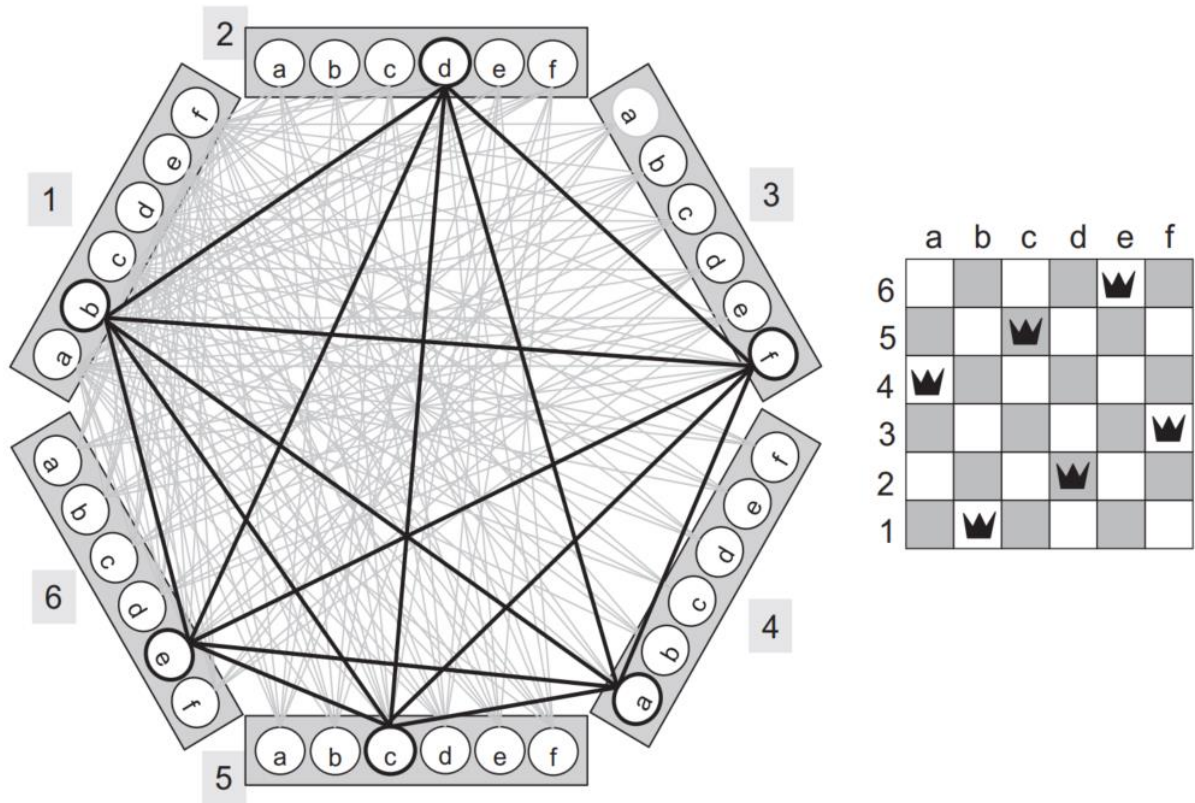
Đồ thị ràng buộc đầy đủ của bài toán N-Hậu cho thấy mọi quyết định gán giá trị cho một biến đều có ảnh hưởng đến tất cả các biến khác. Điều này có thể được minh họa thêm bằng biểu đồ ghép nối (matching diagram).



Hình 68. Biểu đồ ghép nối minh họa các quan hệ ràng buộc trong bài toán 6 hậu.

Hình 68. Biểu đồ ghép nối cho bài toán 6-Hậu. Biểu đồ này chỉ vẽ một phần các cạnh ràng buộc nhưng đã cho thấy một “đám sương mù” dày đặc các kết nối. Việc tìm lời giải chính là tìm ra một tập hợp các giá trị (một giá trị cho mỗi biến) sao cho tất cả các cặp giá trị được kết nối với nhau bởi các cạnh hợp lệ.

Một lời giải cho bài toán là một bộ giá trị gán cho các biến $(Q_1, Q_2, \dots, Q_6)$ thỏa mãn tất cả ràng buộc.



Hình 69. Lời giải $\langle b, d, f, a, c, e \rangle$ cho bài toán 6 hậu thể hiện trên biểu đồ ghép nối và bàn cờ.

Hình 69. Một lời giải cho bài toán 6-Hậu: $\langle b, d, f, a, c, e \rangle$, tương ứng với việc đặt hậu ở các ô (1,b), (2,d), (3,f), (4,a), (5,c), (6,e). Lời giải được thể hiện rõ ràng trên cả biểu đồ ghép nối (bên trái) và trên bàn cờ (bên phải).

Ý nghĩa và Tổng kết

Bài toán N-Hậu, với các ràng buộc phi tuyến tính và đồ thị ràng buộc dày đặc, là một ví dụ điển hình cho thấy sự cần thiết của các kỹ thuật giải CSP thông minh. Các thuật toán tìm kiếm quay lui (backtracking) đơn thuần sẽ đối mặt với sự bùng nổ tổ hợp. Do đó, bài toán này thường được dùng để kiểm tra và so sánh hiệu quả của các kỹ thuật **lan truyền ràng buộc (constraint propagation)** và các **heuristic** để lựa chọn biến và giá trị, nhằm cắt tỉa không gian tìm kiếm một cách hiệu quả.

Cùng với bài toán Tô màu bản đồ, N-Hậu cho thấy sự linh hoạt và mạnh mẽ của khung mẫu CSP trong việc mô hình hóa các vấn đề đa dạng, từ các ràng buộc logic đơn giản đến các ràng buộc hình học phức tạp. Chúng là nền tảng vững chắc để nghiên cứu và phát triển các thuật toán giải quyết vấn đề tổng quát trong Trí tuệ Nhân tạo.

2.8.4. Giải quyết CSP bằng thuật toán Quay lui (Backtracking)

Thuật toán **quay lui (backtracking)** là phương pháp nền tảng và phổ biến nhất để giải quyết các bài toán thỏa mãn ràng buộc (CSP). Về bản chất, đây là một dạng của thuật toán **tìm kiếm theo chiều sâu (Depth-First Search - DFS)**, nhưng được tối ưu hóa bằng cách kiểm tra các ràng buộc ở mỗi bước để sớm loại bỏ các nhánh không có lời giải. Mặc dù có vẻ đơn giản, thuật toán này là cơ sở cho nhiều kỹ thuật giải CSP tiên tiến và mạnh mẽ trong lĩnh vực trí tuệ nhân tạo.

Nguyên lý Hoạt động Cốt lõi

Ý tưởng trung tâm của thuật toán quay lui là xây dựng lời giải từng bước một. Quá trình này diễn ra như sau:

- **Khởi tạo:** Bắt đầu với một phép gán rỗng (chưa có biến nào được gán giá trị).
- **Gán giá trị Tuần tự:** Chọn một biến chưa được gán và thử gán cho nó một giá trị từ miền giá trị của nó.
- **Kiểm tra Ràng buộc:** Sau mỗi lần gán, thuật toán sẽ kiểm tra xem phép gán tạm thời này có vi phạm bất kỳ ràng buộc nào với các biến đã được gán trước đó không.
 - **Nếu Hợp lệ (Consistent):** Phép gán được chấp nhận. Thuật toán tiếp tục đệ quy để gán giá trị cho biến tiếp theo.
 - **Nếu Vi phạm (Inconsistent):** Phép gán hiện tại bị hủy bỏ. Thuật toán sẽ “quay lui” (backtrack) và thử một giá trị khác cho biến hiện tại. Nếu tất cả giá trị trong miền đã được thử mà không thành công, thuật toán sẽ quay lui lên biến trước đó và thử một giá trị mới cho biến đó.
- **Kết thúc:** Quá trình kết thúc khi:
 - Tìm thấy một lời giải hoàn chỉnh (tất cả các biến đã được gán giá trị và mọi ràng buộc đều được thỏa mãn).
 - Đã khám phá hết tất cả các khả năng mà không tìm thấy lời giải nào.

Toàn bộ không gian tìm kiếm này có thể được hình dung như một **cây tìm kiếm**, trong đó mỗi cấp độ của cây tương ứng với một biến và mỗi nhánh đại diện cho một giá trị được gán. Quay lui chính là hành động cắt tỉa những nhánh cây không thể dẫn đến lời giải hợp lệ.

Ví dụ Minh họa: Bài toán Tô màu Bản đồ

Bài toán tô màu bản đồ là một ví dụ kinh điển của CSP. Yêu cầu là tô màu các vùng trên bản đồ sao cho không có hai vùng nào có chung đường biên giới lại có cùng màu.

Hãy xem xét một bản đồ nhỏ gồm 5 vùng: **A, B, C, D, E**.

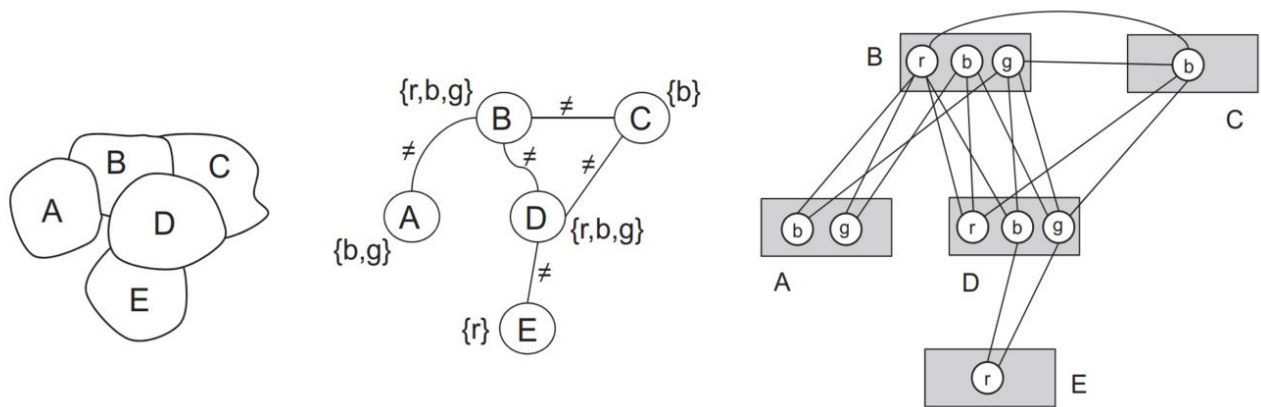
- **Biến (X):** Mỗi vùng là một biến: $\{A, B, C, D, E\}$.
- **Miền giá trị (D):** Tập hợp các màu có thể được sử dụng cho mỗi vùng.
 - $DA = \{\text{xanh dương, xanh lá}\}$
 - $DB = \{\text{đỏ, xanh dương, xanh lá}\}$
 - $DC = \{\text{xanh dương}\}$

- $DD = \{\text{đỏ, xanh dương, xanh lá}\}$
- $DE = \{\text{đỏ}\}$
- **Ràng buộc (C):** Các cặp vùng kề nhau không được có cùng màu.
 - $A \neq B$
 - $B \neq C$
 - $B \neq D$
 - $C \neq D$
 - $D \neq E$

Thực quan hóa bài toán

Để hiểu rõ hơn, chúng ta có thể sử dụng hai loại biểu đồ:

- **Đồ thị Ràng buộc (Constraint Graph):** Mỗi biến là một đỉnh, và một cạnh nối giữa hai đỉnh nếu chúng có ràng buộc với nhau.
- **Biểu đồ Đối sánh (Matching Diagram):** Biểu diễn rõ ràng các cặp giá trị hợp lệ giữa các biến có ràng buộc.

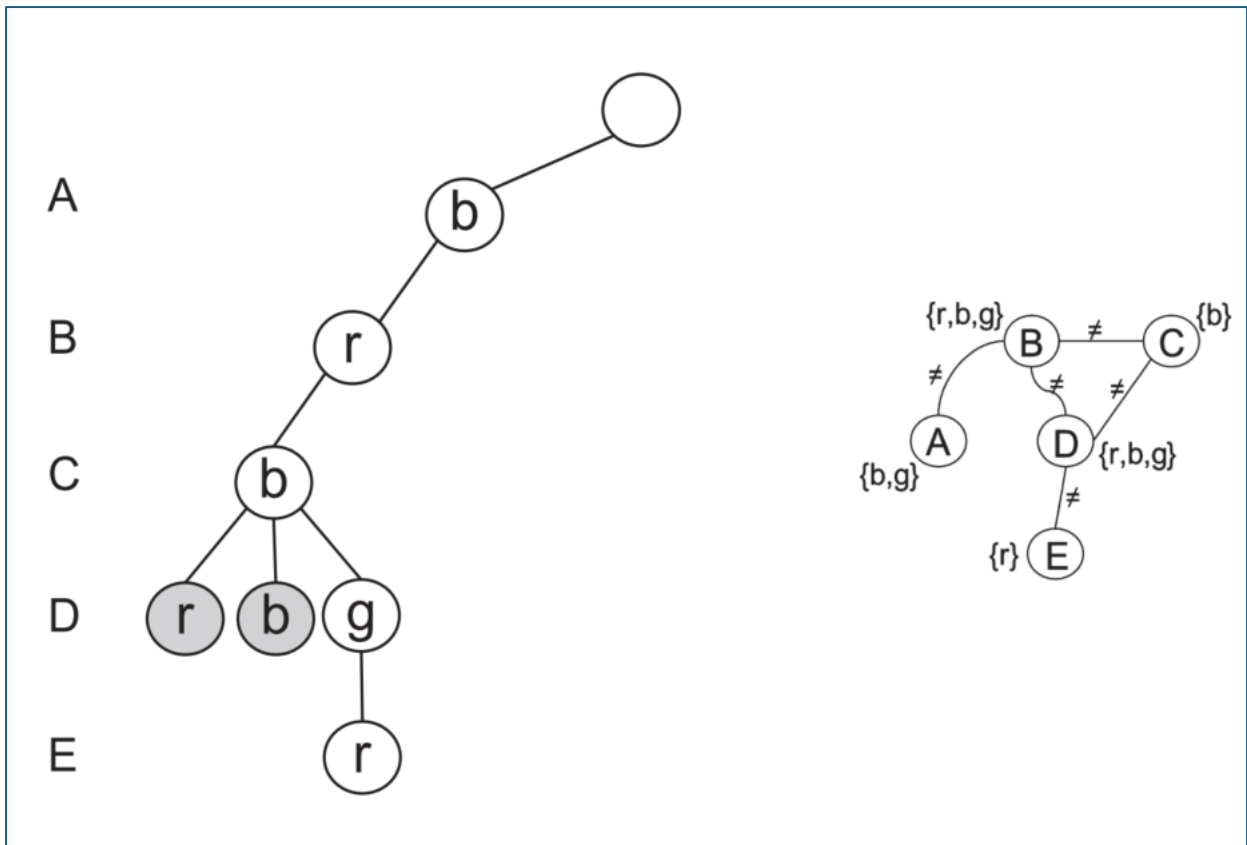


Hình 70. Bài toán tô màu bản đồ với đồ thị ràng buộc và biểu đồ matching.

Hình 70: (Trái) Bản đồ với 5 vùng. (Giữa) Đồ thị ràng buộc cho thấy các vùng kề nhau. (Phải) Biểu đồ đối sánh chỉ ra các cặp màu hợp lệ (ví dụ: một cạnh nối giữa b của A và r của B có nghĩa là $(A=b, B=r)$ là một phép gán hợp lệ).

Quá trình Tìm kiếm bằng Quay lui

Hãy theo dõi thuật toán quay lui tìm lời giải cho bài toán trên, với thứ tự gán biến là A, B, C, D, E.



Hình 71. Thuật toán quay lui thực hiện tìm kiếm theo chiều sâu cho bài toán tô màu bản đồ và tìm ra lời giải $\langle b, r, b, g, r \rangle$.

Hình 71: Cây tìm kiếm của thuật toán quay lui. Các nhánh bị gạch chéo là những lựa chọn thất bại dẫn đến quay lui.

- **Gán cho A:** $A = \text{xanh dương } (b)$. Hợp lệ.
- **Gán cho B:** $B = \text{đỏ } (r)$. Hợp lệ (vì $r \neq b$).
- **Gán cho C:** $C = \text{xanh dương } (b)$. Hợp lệ (vì $b \neq r$).
- **Gán cho D:**
 - Thử $D = \text{đỏ } (r)$. **Thất bại.** Vi phạm ràng buộc $D \neq B$ (vì $B=r$). **Quay lui.**
 - Thử $D = \text{xanh dương } (b)$. **Thất bại.** Vi phạm ràng buộc $D \neq C$ (vì $C=b$). **Quay lui.**
 - Thử $D = \text{xanh lá } (g)$. Hợp lệ (vì $g \neq r$ và $g \neq b$).
- **Gán cho E:** $E = \text{đỏ } (r)$. Hợp lệ (vì $r \neq g$).

Lời giải được tìm thấy: $\{A=b, B=r, C=b, D=g, E=r\}$ hay $\{A=\text{xanh dương}, B=\text{đỏ}, C=\text{xanh dương}, D=\text{xanh lá}, E=\text{đỏ}\}$.

Các Cải tiến để Tăng hiệu quả cho Thuật toán Quay lui

Thuật toán quay lui cơ bản có thể rất chậm khi không gian tìm kiếm lớn. Do đó, các kỹ thuật heuristic và suy luận được áp dụng để tăng tốc quá trình tìm kiếm một cách đáng kể.

1. Heuristic chọn biến (Variable Ordering)

- Nguyên tắc “Thất bại trước” (Fail-First): “Nên gán giá trị cho biến nào tiếp theo?”
- Kỹ thuật: **Minimum Remaining Values (MRV)**: Ưu tiên chọn biến có ít giá trị hợp lệ còn lại nhất trong miền của nó.
 - Lý do: Việc này giúp nhanh chóng phát hiện ra các xung đột. Nếu một biến chỉ còn một giá trị duy nhất và giá trị đó gây ra mâu thuẫn, chúng ta có thể cắt tỉa nhánh tìm kiếm đó ngay lập tức thay vì đi sâu vào nó một cách vô ích.

2. Heuristic chọn giá trị (Value Ordering)

- Nguyên tắc “Thành công trước” (Succeed-First): “Nên thử giá trị nào trước cho biến đã chọn?”
- Kỹ thuật: **Least Constraining Value (LCV)**: Ưu tiên chọn giá trị ít gây ra ràng buộc nhất cho các biến lân cận chưa được gán. Tức là, chọn giá trị mà loại bỏ ít lựa chọn nhất từ các biến “hàng xóm”.
 - Lý do: Kỹ thuật này giữ lại sự linh hoạt tối đa cho các phép gán trong tương lai, làm tăng khả năng tìm thấy lời giải mà không cần phải quay lui.

3. Suy diễn (Inference) để Lan truyền Ràng buộc

- Kỹ thuật: **Forward Checking (Kiểm tra phía trước)**: Đây là một trong những dạng suy luận đơn giản và hiệu quả nhất.
 - Cách hoạt động: Mỗi khi một biến X được gán một giá trị v , thuật toán sẽ kiểm tra tất cả các biến Y chưa được gán mà có ràng buộc với X . Nó sẽ loại bỏ khỏi miền DY của Y bất kỳ giá trị nào không tương thích với v .
 - Lợi ích: Nếu bất kỳ miền nào của các biến lân cận trở nên rỗng sau khi gán giá trị cho X , thuật toán biết ngay rằng phép gán $X=v$ sẽ dẫn đến thất bại. Nó sẽ quay lui ngay lập tức mà không cần đi sâu hơn. Điều này giúp phát hiện các “ngõ cụt” sớm hơn rất nhiều so với quay lui đơn thuần.

Thuật toán quay lui là một công cụ mạnh mẽ và linh hoạt để giải quyết các bài toán CSP. Mặc dù phiên bản cơ bản có thể không hiệu quả, nhưng khi được kết hợp với các **heuristic chọn biến (MRV)**, **heuristic chọn giá trị (LCV)**, và các kỹ thuật **suy luận (như Forward Checking)**, nó trở thành một chiến lược cực kỳ hiệu quả, có khả năng giải quyết các bài toán phức tạp trong thực tế.

2.9. Tìm kiếm đối kháng (Adversarial Search)

2.9.1. Giới thiệu Trò chơi đối kháng

Trong các chương trước, chúng ta đã khám phá các thuật toán tìm kiếm trong môi trường **đơn tác tử (single-agent)**. Trong những môi trường này, tác tử (agent) giống như một người giải câu đố đơn độc; thế giới xung quanh là thụ động và có thể dự đoán được (ví dụ: tìm đường đi ngắn nhất trên bản đồ, giải trò chơi 8-puzzle). Mục tiêu duy nhất là tìm ra một chuỗi hành động để đạt được trạng thái mong muốn.

Tuy nhiên, thế giới thực tế hiếm khi đơn giản như vậy. Trong nhiều tình huống như thi đấu thể thao, đàm phán kinh doanh, hay chiến lược quân sự, các tác tử phải hoạt động trong **môi trường cạnh tranh (competitive environment)**. Ở đây, môi trường không còn thụ động nữa mà bao gồm các tác tử khác với những mục tiêu xung đột trực tiếp với chúng ta. Thành công không

chỉ phụ thuộc vào hành động của chính mình mà còn phụ thuộc vào việc dự đoán và đối phó với hành động của đối thủ. Lĩnh vực nghiên cứu sự tương tác chiến lược này được gọi là **tìm kiếm đối kháng (adversarial search)**, thường được mô hình hóa qua các **trò chơi (games)**.

Đặc điểm của Trò chơi Đối kháng

Các trò chơi đối kháng mà chúng ta xem xét trong AI thường có những đặc điểm chung sau:

- **Nhiều tác tử cạnh tranh:** Có ít nhất hai tác tử (người chơi), và mục tiêu của họ đối lập nhau. Hành động của một người chơi nhằm tối đa hóa lợi ích của mình, thường đồng nghĩa với việc tối thiểu hóa lợi ích của đối thủ.
- **Hành động luân phiên (Turn-taking):** Các người chơi lần lượt thực hiện nước đi của mình.
- **Thông tin hoàn hảo (Perfect Information):** Toàn bộ trạng thái của trò chơi đều được công khai cho tất cả người chơi. Không có thông tin nào bị che giấu. Ví dụ, trong cờ vua, cả hai người chơi đều thấy rõ vị trí của tất cả các quân cờ. (Điều này trái ngược với các trò chơi có thông tin không hoàn hảo như bài Poker).
- **Xác định (Deterministic):** Kết quả của một nước đi là hoàn toàn xác định, không có yếu tố ngẫu nhiên như gieo xúc xắc.
- **Tổng bằng không (Zero-sum):** Đây là một đặc điểm phổ biến, có nghĩa là tổng lợi ích của tất cả người chơi là một hằng số (thường là 0). Lợi ích mà một người chơi nhận được chính là thiệt hại của người chơi kia. Trong cờ vua hoặc tic-tac-toe, một bên thắng (+1) thì bên kia thua (-1), tổng lợi ích là 0.

Ví dụ kinh điển:

- **Tic-tac-toe (Cờ ca-rô 3x3):** Một trò chơi đơn giản nhưng thể hiện đầy đủ các đặc điểm trên.
- **Cờ vua (Chess), Cờ vây (Go):** Những trò chơi cực kỳ phức tạp, là “sân thử” kinh điển cho các thuật toán AI đối kháng.

Cây Trò chơi (Game Tree): Biểu diễn Toàn bộ Khả năng

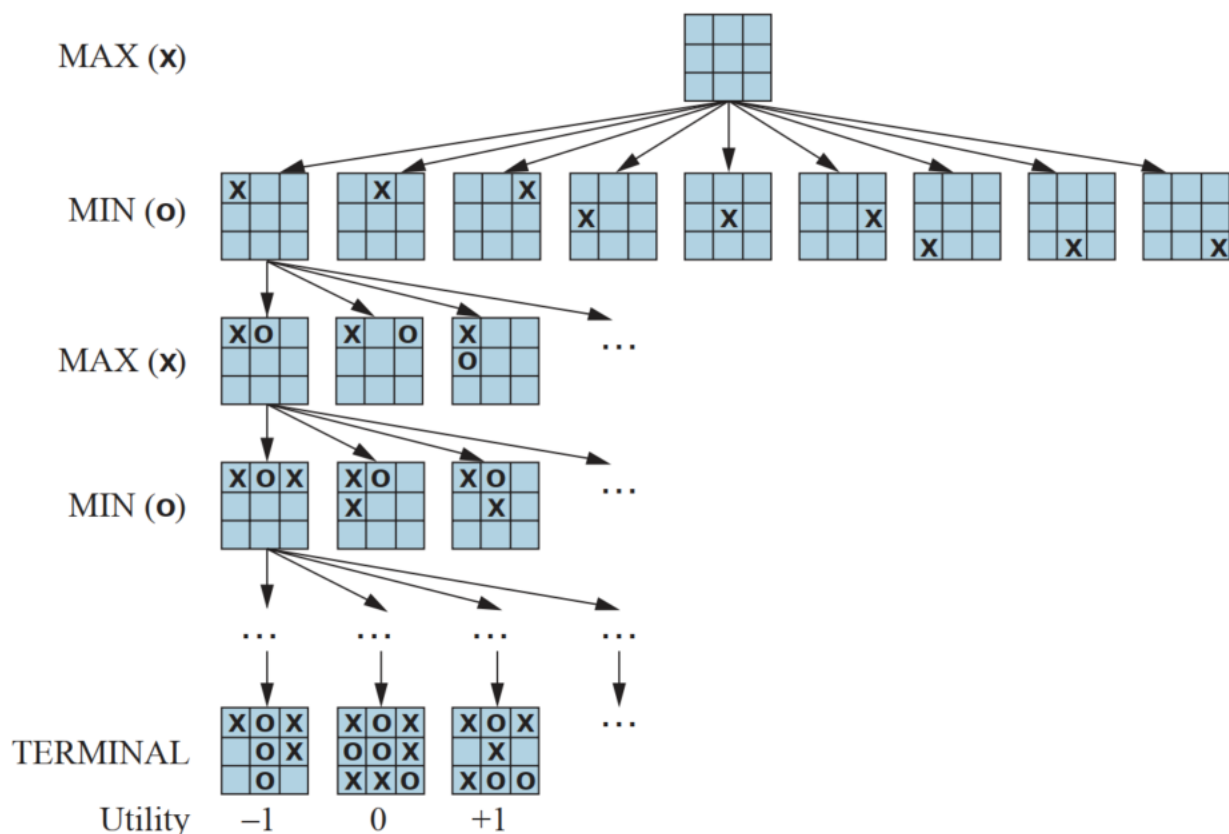
Để một tác tử AI có thể “suy nghĩ” một cách chiến lược, nó cần một cách để biểu diễn tất cả các diễn biến có thể xảy ra của trận đấu. Cấu trúc dữ liệu dùng cho mục đích này là **cây trò chơi (game tree)**.

- **Nút (Node):** Đại diện cho một trạng thái (state) của bàn cờ.
- **Nút Gốc (Root):** Trạng thái ban đầu của trò chơi.
- **Cạnh (Edge):** Đại diện cho một nước đi (move) hợp lệ.
- **Nút Lá (Leaf/Terminal Node):** Đại diện cho một trạng thái kết thúc trò chơi (thắng, thua, hoặc hòa). Mỗi nút lá có một giá trị số gọi là **lợi ích (utility)** hoặc **điểm số (payoff)**.

Để thuận tiện, chúng ta quy ước hai người chơi là **MAX** và **MIN**:

- **MAX:** Người chơi mà AI đang điều khiển, luôn cố gắng thực hiện nước đi để dẫn đến trạng thái kết thúc có điểm số *cao nhất*.

- **MIN**: Đối thủ, luôn cố gắng thực hiện nước đi để dẫn đến trạng thái kết thúc có điểm số *thấp nhất* (theo góc nhìn của MAX).



Hình 72. Một phần cây trò chơi của trò chơi cờ ca-rô (tic-tac-toe).

Hình 72: Một phần của cây trò chơi Tic-tac-toe.

- Nút gốc là bàn cờ trống, và **MAX (đi X)** có 9 lựa chọn ban đầu.
- Hình trên minh họa một nhánh: MAX đi vào ô trung tâm.
- Đến lượt **MIN (đi O)**, MIN có 8 lựa chọn còn lại.
- Các lượt đi cứ tiếp diễn, luân phiên giữa MAX và MIN, tạo thành các tầng của cây.
- Quá trình kết thúc ở các nút lá, nơi trò chơi có kết quả. Các giá trị $+1$, -1 , 0 được gán cho các trạng thái này, thể hiện lợi ích từ góc nhìn của MAX:
 - $+1$: MAX thắng.
 - -1 : MIN thắng (MAX thua).
 - 0 : Hòa.

Thách thức về sự Bùng nổ Tổ hợp

Mục tiêu của AI là chọn một nước đi từ trạng thái hiện tại để cuối cùng dẫn đến một nút lá có giá trị lợi ích tối đa, giả định rằng đối thủ MIN cũng chơi một cách tối ưu để ngăn chặn điều đó.

Về lý thuyết, nếu có thể xây dựng toàn bộ cây trò chơi, AI chỉ cần chọn đường đi tốt nhất. Tuy nhiên, thực tế điều này là bất khả thi với hầu hết các trò chơi thú vị:

- **Tic-tac-toe:** Cây trò chơi có dưới 362,880 nút lá, một con số khá nhỏ cho máy tính hiện đại.
- **Cờ vua (Chess):** Số lượng nút trong cây trò chơi ước tính lớn hơn 10^{40} . Đây là một con số khổng lồ, vượt xa khả năng tính toán của bất kỳ siêu máy tính nào trên thế giới.

Do đó, thách thức cốt lõi của tìm kiếm đối kháng không phải là duyệt toàn bộ cây, mà là **làm thế nào để ra quyết định tối ưu mà chỉ cần khám phá một phần nhỏ của cây trò chơi**. Đây chính là tiền đề cho thuật toán **Minimax** và các cải tiến của nó như cắt tỉa Alpha-Beta.

2.9.2. Thuật toán Minimax: Chơi tối ưu khi đối thủ cũng chơi tối ưu

Khi bước vào một môi trường đối kháng, một tác tử thông minh không thể chỉ chọn hành động mang lại lợi ích trước mắt. Nó phải dự đoán các nước đi đáp trả của đối thủ và lên kế hoạch cho phù hợp. **Thuật toán Minimax** là một thuật toán đệ quy kinh điển, cung cấp một phương pháp toán học để lựa chọn nước đi tối ưu. Thuật toán này hoạt động dựa trên một giả định cốt lõi: **đối thủ của bạn cũng là một người chơi hoàn hảo và sẽ luôn chọn nước đi tốt nhất cho họ (tức là tệ nhất cho bạn)**.

Nguyên lý Cốt lõi: Tối đa hóa Lợi ích trong Kịch bản Tệ nhất

Thuật toán hoạt động bằng cách xem xét toàn bộ cây trò chơi từ trạng thái hiện tại. Nó gán cho hai người chơi hai vai trò đối lập:

- **MAX:** Là tác tử của chúng ta. Mục tiêu của MAX là thực hiện một chuỗi các nước đi để đi đến trạng thái kết thúc có **lợi ích (utility) cao nhất**.
- **MIN:** Là đối thủ. Mục tiêu của MIN là thực hiện các nước đi để dẫn đến trạng thái kết thúc có **lợi ích thấp nhất** (theo góc nhìn của MAX).

Do đó, chiến lược của MAX là chọn ra nước đi dẫn đến trạng thái có giá trị **tối đa**, sau khi MIN đã chơi để chọn ra trạng thái có giá trị **tối thiểu**. Nói cách khác, MAX tìm cách **tối đa hóa giá trị tối thiểu** mà đối thủ có thể ép mình phải nhận. Đây chính là nguồn gốc của cái tên **Minimax**.

Quy trình Thuật toán Minimax

Thuật toán Minimax thực hiện một cuộc duyệt sâu trên cây trò chơi. Quá trình này bao gồm ba bước chính:

- **Xây dựng Cây trò chơi:** Từ trạng thái hiện tại, tạo ra một cây biểu diễn tất cả các chuỗi nước đi có thể xảy ra cho đến một độ sâu nhất định hoặc đến khi gặp các trạng thái kết thúc (thắng, thua, hòa).
- **Tính toán Hàm Lợi ích (Utility Function):** Tại các nút lá (trạng thái kết thúc) của cây, gán một giá trị số để biểu thị kết quả của ván chơi từ góc nhìn của MAX.
 - Thắng: Một giá trị dương lớn (ví dụ: +1 hoặc $+\infty$).
 - Thua: Một giá trị âm lớn (ví dụ: -1 hoặc $-\infty$).
 - Hòa: 0.
- **Lan truyền Giá trị Ngược (Backpropagation):** Tính toán giá trị Minimax cho các nút bên trong cây, đi từ các nút lá ngược lên nút gốc.

- Tại một nút thuộc lượt đi của **MAX**, giá trị của nó là **GIÁ TRỊ LỚN NHẤT (max)** trong số các giá trị của các nút con.
- Tại một nút thuộc lượt đi của **MIN**, giá trị của nó là **GIÁ TRỊ NHỎ NHẤT (min)** trong số các giá trị của các nút con.

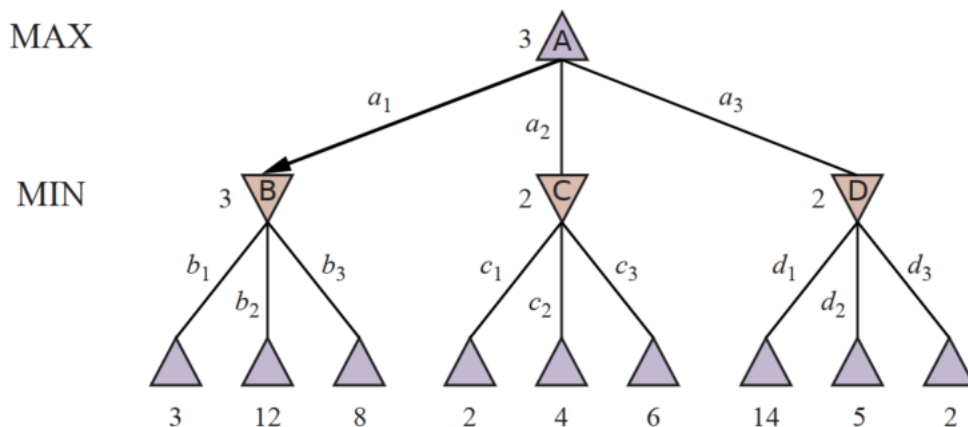
Giá trị Minimax của một trạng thái s có thể được định nghĩa chính thức bằng công thức đệ quy sau:

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{nếu } s \text{ là trạng thái kết thúc} \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{nếu } s \text{ là lượt của MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{nếu } s \text{ là lượt của MIN} \end{cases}$$

Nước đi mà MAX nên chọn chính là hành động a dẫn đến trạng thái có giá trị Minimax cao nhất.

Minh họa chi tiết với Cây trò chơi

Hãy cùng phân tích một cây trò chơi đơn giản (2 lớp, tức là một lượt đi của MAX và một lượt đi của MIN) để thấy rõ cách Minimax hoạt động.



Hình 73. Cây trò chơi hai lớp với các giá trị minimax.

Hình 73: Cây trò chơi hai lớp. Nút gốc A là nút MAX. Các nút B, C, D là các nút MIN. Các nút lá có giá trị lợi ích từ 2 đến 14.

Quá trình tính toán diễn ra từ dưới lên trên:

- **Tầng Lá (Terminal Nodes):** Các giá trị lợi ích đã được cho sẵn (3, 12, 8, 2, ...).
- **Tầng của MIN (Lượt đi của đối thủ):** MIN sẽ chọn nước đi dẫn đến giá trị thấp nhất cho MAX.
 - Tại nút **B**, MIN có thể đi đến các trạng thái có giá trị {3, 12, 8}. MIN sẽ chọn nước đi dẫn đến giá trị 3. Vì vậy, giá trị Minimax của B là **$\min(3, 12, 8) = 3$** .

- Tại nút **C**, MIN sẽ chọn nước đi dẫn đến giá trị 2. Giá trị Minimax của C là $\min(2, 4, 6) = 2$.
- Tại nút **D**, MIN sẽ chọn nước đi dẫn đến giá trị 2. Giá trị Minimax của D là $\min(14, 5, 2) = 2$.
- **Tầng của MAX (Nút gốc - Lượt đi của ta):** MAX bây giờ sẽ chọn nước đi dẫn đến giá trị cao nhất trong số các lựa chọn mà MIN đã “chốt” ở trên.
 - Tại nút gốc **A**, MAX có thể chọn đi đến các trạng thái B, C, hoặc D, với các giá trị Minimax tương ứng là $\{3, 2, 2\}$. MAX sẽ chọn nước đi dẫn đến giá trị cao nhất. Do đó, giá trị Minimax của A là $\max(3, 2, 2) = 3$.

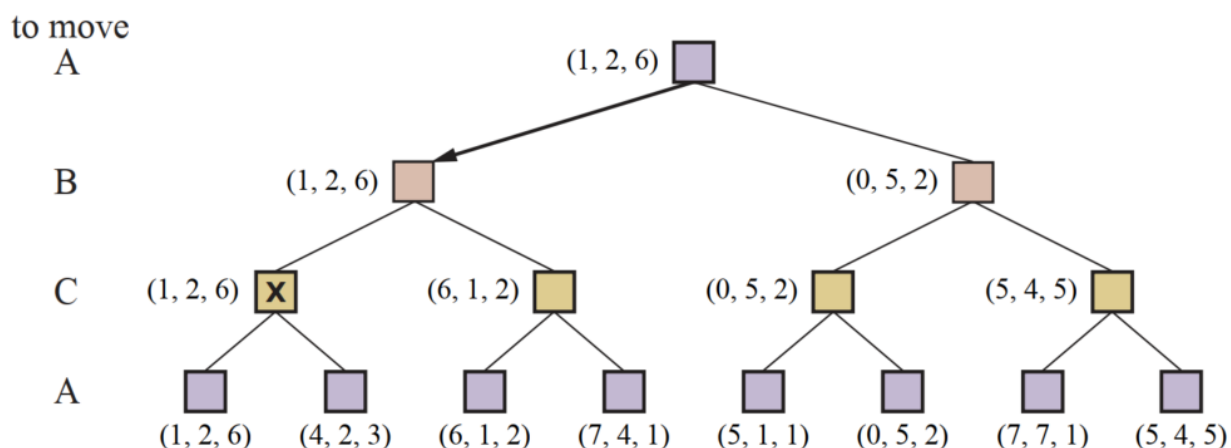
Quyết định Minimax: Nước đi tối ưu cho MAX tại trạng thái A là đi nước **a1**, vì nó dẫn đến trạng thái B có giá trị Minimax cao nhất (là 3). Bằng cách chọn a1, MAX đảm bảo rằng kết quả tệ nhất mà mình có thể nhận được là 3, bất kể MIN có chơi thông minh đến đâu.

Thuộc tính của Thuật toán Minimax

- **Tính đầy đủ (Completeness):** Có. Nếu cây trò chơi là hữu hạn, Minimax chắc chắn sẽ tìm ra một nước đi.
- **Tính tối ưu (Optimality):** Có, với điều kiện đối thủ cũng chơi tối ưu. Nếu đối thủ chơi kém, MAX có thể đạt được kết quả tốt hơn, nhưng nước đi Minimax vẫn là lựa chọn an toàn nhất.
- **Độ phức tạp thời gian (Time Complexity):** $O(b^m)$, trong đó b là **hệ số nhánh** (số nước đi trung bình ở mỗi trạng thái) và m là **độ sâu tối đa** của cây. Đây là điểm yếu lớn nhất của Minimax, vì nó phải duyệt toàn bộ cây trò chơi.
- **Độ phức tạp không gian (Space Complexity):** $O(bm)$ nếu sử dụng thuật toán duyệt theo chiều sâu để tạo cây.

Mở rộng cho Trò chơi Nhiều người

Thuật toán Minimax có thể được mở rộng cho các trò chơi có nhiều hơn hai người chơi. Thay vì một giá trị lợi ích duy nhất tại mỗi nút, chúng ta sử dụng một **vector lợi ích**, ví dụ $\langle v_A, v_B, v_C \rangle$ cho trò chơi ba người A, B, C. Tại mỗi nút, người chơi có lượt sẽ chọn nước đi để **tối đa hóa thành phần lợi ích của chính mình** trong vector đó.



Hình 74. Ba lớp đầu tiên của cây trò chơi với ba người chơi.

Hình 74: Trong cây trò chơi 3 người này, tại nút X, đến lượt người chơi C. C có hai lựa chọn dẫn đến các vector lợi ích $\langle 1, 2, 6 \rangle$ và $\langle 4, 2, 3 \rangle$. Vì lợi ích của C (v_C) ở lựa chọn đầu tiên (6) lớn hơn lựa chọn thứ hai (3), C sẽ chọn nước đi đầu tiên. Do đó, toàn bộ vector $\langle 1, 2, 6 \rangle$ được lan truyền ngược lên làm giá trị cho nút X.

Thuật toán Minimax đặt nền móng lý thuyết vững chắc cho việc ra quyết định chiến lược trong AI. Bằng cách giả định một đối thủ hoàn hảo, nó buộc tác tử phải tìm kiếm một chiến lược “an toàn”, đảm bảo kết quả tốt nhất có thể trong trường hợp xấu nhất. Tuy nhiên, do độ phức tạp tính toán khổng lồ ($O(b^m)$), Minimax trong dạng thuần túy của nó không thực tế cho các trò chơi phức tạp như cờ vua. Vấn đề này chính là động lực để phát triển các thuật toán hiệu quả hơn như **Cắt tỉa Alpha-Beta**, một cải tiến trực tiếp của Minimax.

2.9.3. Cắt tỉa Alpha-Beta (α - β Pruning): Tối ưu hóa cho Minimax

Vấn đề của Minimax và Nhu cầu Tối ưu hóa

Thuật toán Minimax cung cấp một chiến lược hoàn hảo về mặt lý thuyết để tìm ra nước đi tối ưu. Tuy nhiên, điểm yếu chí mạng của nó là độ phức tạp thời gian $O(b^m)$ (với b là hệ số nhánh và m là độ sâu tìm kiếm). Điều này có nghĩa là số lượng trạng thái cần xem xét tăng theo cấp số nhân với độ sâu. Đối với các trò chơi phức tạp như cờ vua ($b \approx 35$) hay cờ vây ($b \approx 250$), việc duyệt toàn bộ cây trò chơi dù chỉ ở độ sâu vài nước đi cũng là bất khả thi.

Cắt tỉa Alpha-Beta ra đời như một sự cải tiến vượt bậc cho Minimax. Nó cho phép chúng ta đạt được quyết định y hệt như Minimax nhưng bằng cách bỏ qua (cắt tỉa) các phần lớn của cây trò chơi mà ta biết chắc chắn rằng chúng sẽ không ảnh hưởng đến kết quả cuối cùng.

Ý tưởng Cốt lõi: “Đừng lãng phí thời gian vào những nước đi tệ”

Hãy tưởng tượng bạn (MAX) đang cân nhắc các nước đi.

- Bạn phân tích nước đi **A** và thấy rằng, dù đối thủ (MIN) có chơi tốt đến đâu, bạn vẫn đảm bảo được điểm số ít nhất là **10**.
- Sau đó, bạn phân tích nước đi **B**. Ngay ở lựa chọn đầu tiên của đối thủ trong nhánh này, họ có thể ép bạn vào một tình thế chỉ còn tối đa **3** điểm.

Tại thời điểm này, bạn có cần phân tích thêm các lựa chọn khác của đối thủ trong nhánh **B** không? **Hoàn toàn không**. Vì bạn đã biết chắc chắn nước đi **A** (đảm bảo 10 điểm) tốt hơn hẳn nước đi **B** (bị ép xuống tối đa 3 điểm), bạn có thể ngừng phân tích nhánh **B** ngay lập tức. Đây chính là bản chất của cắt tỉa.

Để thực hiện điều này một cách có hệ thống, thuật toán duy trì hai giá trị:

- **α (Alpha):** Giá trị **tốt nhất** (cao nhất) mà người chơi **MAX** có thể đảm bảo nhận được cho đến thời điểm hiện tại trên đường đi từ gốc đến nút đang xét. Đây là “mức sàn” của MAX.
- **β (Beta):** Giá trị **tốt nhất** (thấp nhất) mà người chơi **MIN** có thể đảm bảo nhận được cho đến thời điểm hiện tại (từ góc nhìn của MAX). Đây là “mức trần” của MAX.

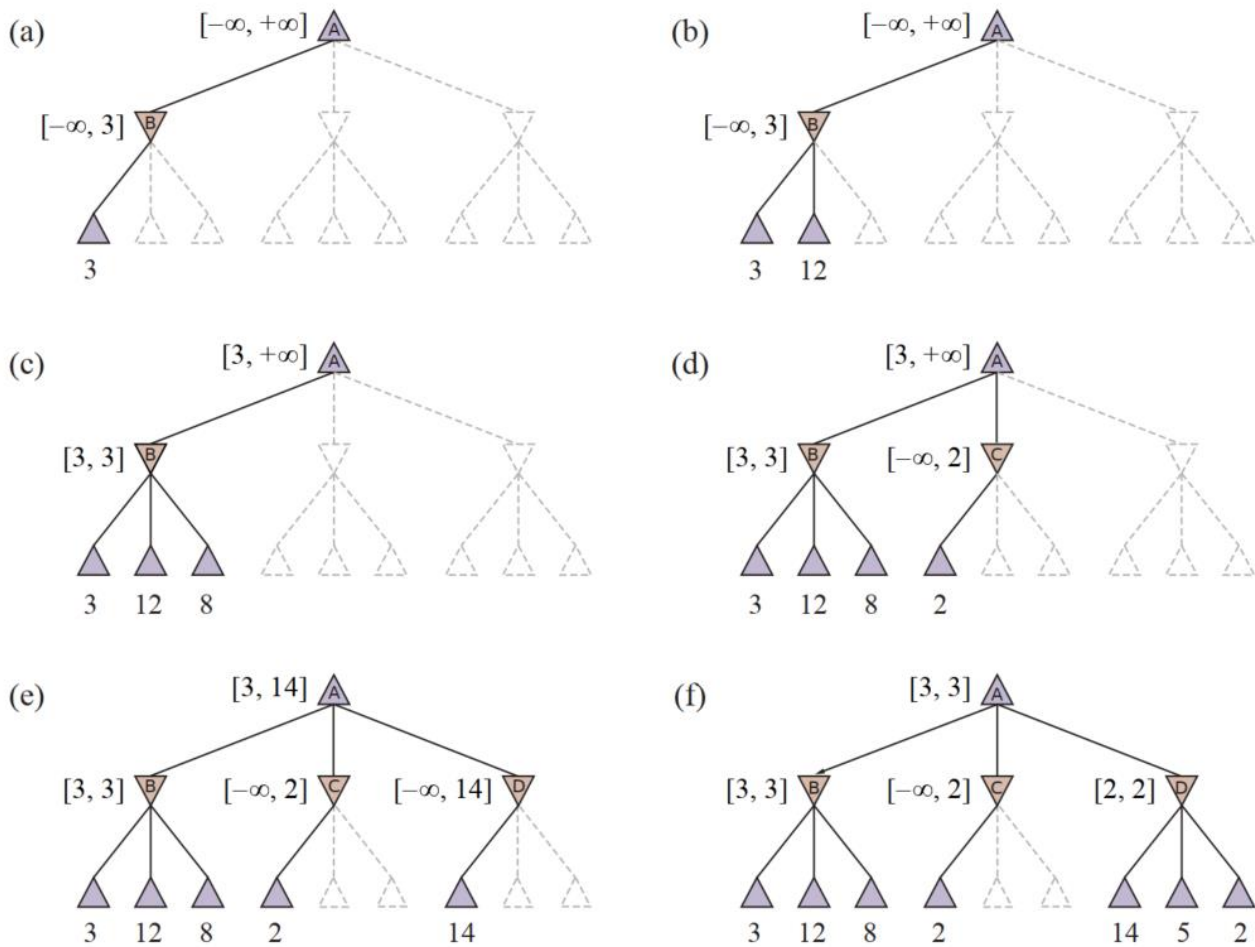
Điều kiện Cắt tỉa

Quá trình tìm kiếm (duyệt cây theo chiều sâu) sẽ truyền các giá trị α và β xuống các nút con. Điều kiện cắt tỉa tổng quát là: **khi $\alpha \geq \beta$** .

- **Cắt tỉa tại nút MIN (Cắt tỉa Beta):** Một nút MIN đang được xét. Nếu giá trị tạm thời v của nó trở nên **nhỏ hơn hoặc bằng α** ($v \leq \alpha$), ta có thể dừng việc duyệt các nhánh con còn lại của nút MIN này.
 - **Lý do:** Nút MIN này đã tìm được một nước đi để hạ điểm của MAX xuống v . Vì $v \leq \alpha$, có nghĩa là nước đi này còn tệ hơn cả lựa chọn mà MAX đã đảm bảo được ở một nhánh khác (α). Do đó, MAX sẽ không bao giờ đi vào nhánh này, và ta không cần xem MIN còn có thể làm gì tệ hơn nữa.
- **Cắt tỉa tại nút MAX (Cắt tỉa Alpha):** Một nút MAX đang được xét. Nếu giá trị tạm thời v của nó trở nên **lớn hơn hoặc bằng β** ($v \geq \beta$), ta có thể dừng việc duyệt các nhánh con còn lại của nút MAX này.
 - **Lý do:** Nút MAX này đã tìm thấy một nước đi mang lại điểm số v . Vì $v \geq \beta$, có nghĩa là nước đi này tốt hơn cả mức mà MIN có thể chấp nhận ở một nhánh khác (β). Đối thủ MIN (ở tầng trên) sẽ không bao giờ để cho MAX đi vào nhánh này.

Minh họa Chi tiết từng bước với Cắt tỉa Alpha-Beta

Chúng ta sẽ sử dụng lại cây trò chơi từ ví dụ Minimax và theo dõi quá trình duyệt với α (khởi tạo là $-\infty$) và β (khởi tạo là $+\infty$).



Hình 75. Các bước tính toán quyết định tối ưu với tĩa alpha-beta.

- (a) $A \rightarrow B \rightarrow \text{Lá}(3)$:
 - Tại A (MAX), $\alpha = -\infty$, $\beta = +\infty$.
 - Đi tới B (MIN), B kế thừa $\alpha = -\infty$, $\beta = +\infty$.
 - Xét lá đầu tiên của B, giá trị là **3**. Giá trị tạm thời của B là 3. MIN cập nhật β của mình: $\beta = \min(+\infty, 3) = 3$. Bây giờ tại B, giá trị của nó sẽ là *tối đa* 3.
- (b) $A \rightarrow B \rightarrow \text{Lá}(12)$:
 - Xét lá thứ hai của B, giá trị là **12**. MIN sẽ không chọn nước đi này vì nó tệ hơn nước đi có giá trị 3 đã tìm thấy. Giá trị của B vẫn là *tối đa* 3.
- (c) $A \rightarrow B$ (Hoàn tất):
 - Xét lá cuối cùng của B, giá trị là **8**. MIN cũng sẽ không chọn nước đi này.
 - B duyệt xong, giá trị chính xác của B là **3**. Giá trị này được trả về cho A.
 - A (MAX) cập nhật α của mình: $\alpha = \max(-\infty, 3) = 3$. Bây giờ tại A, MAX biết mình có thể đảm bảo được kết quả *ít nhất* là 3.
- (d) $A \rightarrow C \rightarrow \text{Lá}(2) \dots \text{CẮT TĨA!}$
 - A đi tới C (MIN). C kế thừa $\alpha = 3$, $\beta = +\infty$.
 - Xét lá đầu tiên của C, giá trị là **2**. Giá trị tạm thời của C là 2.
 - **Kiểm tra điều kiện cắt tĩa**: Giá trị tạm thời của C (là 2) $\leq \alpha$ của A (là 3). Điều kiện $v \leq \alpha$ được thỏa mãn!

- **Hành động: CẮT TỈA.** Chúng ta không cần xem xét các nhánh con còn lại của C (nhánh có giá trị 4 và 6). Lý do: MAX sẽ không bao giờ chọn đi đến C (để bị MIN ép xuống tối đa 2 điểm) vì đã có lựa chọn B tốt hơn (đảm bảo 3 điểm).
- **(e) $A \rightarrow D \rightarrow \text{Lá}(14)$:**
 - A đi tới D (MIN). D kế thừa $\alpha=3, \beta=+\infty$.
 - Xét lá đầu tiên của D, giá trị là **14**. Giá trị tạm thời của D là 14. MIN cập nhật β của mình: $\beta = \min(+\infty, 14) = 14$. Tại D, MIN biết có thể ép MAX xuống tối đa 14.
- **(f) $A \rightarrow D$ (Hoàn tất):**
 - Xét lá thứ hai của D, giá trị là **5**. Giá trị tạm thời của D là $\min(14, 5) = 5$. MIN cập nhật β của mình: $\beta = \min(14, 5) = 5$.
 - Xét lá cuối cùng của D, giá trị là **2**. Giá trị chính xác của D là $\min(5, 2) = 2$.
 - D duyệt xong, trả về giá trị **2** cho A.
 - A (MAX) xem xét giá trị này: $\max(3, 2) = 3$. α của A không đổi.

Kết quả cuối cùng: A đã duyệt xong tất cả các nhánh. Quyết định tối ưu là đi đến **B**, với giá trị Minimax là **3**. Thuật toán đã cho ra kết quả chính xác mà **không cần phải xem xét 2 nút lá**.

Hiệu suất và Tầm quan trọng của Thứ tự Nước đi

- **Đảm bảo kết quả:** Cắt tỉa Alpha-Beta luôn cho ra kết quả giống hệt Minimax.
- **Hiệu quả vượt trội:** Trong trường hợp xấu nhất (thứ tự duyệt các nước đi tệ), Alpha-Beta không cắt tỉa được gì và có độ phức tạp là $O(b^m)$. Tuy nhiên, trong trường hợp tốt nhất, độ phức tạp giảm xuống chỉ còn **** $O\left(b^{\frac{m}{2}}\right)$. Điều này có nghĩa là, với cùng một lượng thời gian, AI có thể nhìn sâu gấp đôi**** vào cây trò chơi. 🚀
- **Thứ tự nước đi (Move Ordering):** Hiệu quả của Alpha-Beta phụ thuộc rất nhiều vào thứ tự duyệt các nhánh con. Để đạt hiệu quả cắt tỉa tối đa, thuật toán nên ưu tiên duyệt các nước đi “tốt nhất” trước. Trong thực tế, các chương trình AI thường sử dụng các hàm đánh giá heuristic để sắp xếp các nước đi tiềm năng trước khi bắt đầu duyệt sâu.

Cắt tỉa Alpha-Beta không chỉ là một “mẹo” tối ưu hóa; nó là một thành phần thiết yếu biến Minimax từ một thuật toán lý thuyết trở thành một công cụ mạnh mẽ và khả thi trong thực tế. Bằng cách loại bỏ các tính toán vô ích một cách thông minh, nó cho phép các tác tử AI đưa ra các quyết định chiến lược sâu sắc và chính xác hơn trong các trò chơi đối kháng phức tạp.

2.10. Thuật toán di truyền (Genetic Algorithms - GA)

2.10.1. Giới thiệu: Lấy cảm hứng từ sự tiến hóa

Trong số các phương pháp tìm kiếm và tối ưu hóa trong trí tuệ nhân tạo, **thuật toán di truyền (Genetic Algorithms – GA)** là một nhánh quan trọng thuộc nhóm **tìm kiếm ngẫu nhiên có định hướng (stochastic search)**. Khác với các thuật toán tìm kiếm truyền thống vốn dựa vào việc phân tích cấu trúc rõ ràng của không gian trạng thái, GA được thiết kế dựa trên **nguyên lý chọn lọc tự nhiên và tiến hóa sinh học**.

Nguồn gốc sinh học: Chọn lọc tự nhiên và di truyền học

GA bắt nguồn từ quan sát trong sinh học tiến hóa rằng:

“Trong tự nhiên, các cá thể có khả năng thích nghi tốt hơn với môi trường sẽ có xác suất cao hơn để sinh tồn và truyền lại đặc điểm di truyền của mình cho thế hệ sau.”

Dựa trên nguyên tắc này, GA mô phỏng quá trình tiến hóa bằng cách:

- **Biểu diễn lời giải tiềm năng dưới dạng “cá thể” (individuals)** – thường là các chuỗi bit hoặc chuỗi ký hiệu.
- **Áp dụng một hàm đánh giá (fitness function)** để đo mức độ thích nghi (chất lượng) của từng cá thể.
- **Tiến hành quá trình tiến hóa nhân tạo qua các thế hệ**, bao gồm các bước:
 - d. **Lựa chọn (Selection)**: Chọn cá thể có độ thích nghi cao để tham gia sinh sản.
 - e. **Lai ghép (Crossover)**: Kết hợp thông tin di truyền từ hai cá thể cha mẹ để tạo ra cá thể con.
 - f. **Đột biến (Mutation)**: Áp dụng thay đổi ngẫu nhiên nhỏ lên cá thể con để duy trì đa dạng di truyền và khám phá thêm không gian tìm kiếm.

Đặc điểm kỹ thuật và vai trò trong AI

GA không cố gắng xây dựng một lời giải theo hướng quy nạp từng bước như các thuật toán A* hay Backtracking. Thay vào đó, nó **duy trì một tập hợp dân số (population)** gồm nhiều lời giải tiềm năng, và cải thiện chúng qua nhiều thế hệ dựa vào các thao tác mô phỏng sinh học. Đây là một ví dụ điển hình của **thuật toán tìm kiếm dựa trên quần thể (population-based search)**.

Một số điểm nổi bật:

- **Tìm kiếm song song**: GA có khả năng **tìm kiếm đồng thời trên nhiều khu vực của không gian lời giải**, nhờ vào việc làm việc trên một tập hợp cá thể thay vì một trạng thái duy nhất.
- **Không yêu cầu cấu trúc hàm mục tiêu khả vi**: GA có thể áp dụng cho các bài toán tối ưu hóa không tuyến tính, phi khả vi, không liên tục, hoặc không có mô hình rõ ràng.
- **Không đảm bảo tối ưu toàn cục**, nhưng thường tìm được lời giải “gần tối ưu” trong thời gian hợp lý, đặc biệt hữu ích với các bài toán phức tạp, có nhiều đỉnh cực bộ hoặc không gian tìm kiếm rộng lớn.

Ví dụ ứng dụng thực tiễn

Thuật toán di truyền đã được áp dụng hiệu quả trong nhiều lĩnh vực, đặc biệt là những bài toán tối ưu hóa phức tạp:

- **Tối ưu hóa kỹ thuật:** Thiết kế cấu trúc chịu lực trong xây dựng, tối ưu hóa các thông số kỹ thuật của hệ thống cơ điện, thiết kế mạch tích hợp.
- **AI trong trò chơi:** Học chiến lược chơi tối ưu trong các trò chơi như chiến thuật thời gian thực, hoặc tự động phát triển luật chơi.
- **Logistics và vận tải:** Lập lịch sản xuất, tối ưu hóa tuyến đường giao hàng (Vehicle Routing Problem), phân phối tài nguyên hiệu quả.
- **Học máy:** Tối ưu hóa cấu trúc mạng nơ-ron, lựa chọn tập đặc trưng (feature selection), thiết kế thuật toán học phù hợp.

Thuật toán di truyền là một trong những đại diện tiêu biểu của phương pháp tìm kiếm ngẫu nhiên định hướng trong trí tuệ nhân tạo. Nhờ lấy cảm hứng từ tiến hóa tự nhiên, GA cung cấp một chiến lược tìm kiếm mạnh mẽ, mềm dẻo và linh hoạt – đặc biệt thích hợp với các bài toán **tối ưu hóa phức tạp, phi tuyến tính**, nơi mà các phương pháp cổ điển tỏ ra không hiệu quả.

2.10.2. Cấu trúc và cơ chế hoạt động của thuật toán di truyền

Giới thiệu: Lấy cảm hứng từ Tiến hóa Tự nhiên

Thuật toán Di truyền (Genetic Algorithm - GA) là một phương pháp tìm kiếm và tối ưu hóa mạnh mẽ, được xây dựng dựa trên việc mô phỏng các nguyên lý của quá trình tiến hóa và chọn lọc tự nhiên. Thay vì phân tích bài toán bằng các công cụ toán học truyền thống, GA vận hành trên một **quần thể (population)** các lời giải tiềm năng. Qua mỗi **thế hệ (generation)**, các lời giải “tốt nhất” sẽ được ưu tiên để “sinh sản” và tạo ra thế hệ con, mang theo những đặc tính di truyền ưu việt. Quá trình này, bao gồm **chọn lọc, lai ghép và đột biến**, giúp quần thể dần dần hội tụ về lời giải tối ưu hoặc gần tối ưu cho bài toán.

Các Thành phần Cấu trúc của một Thuật toán Di truyền

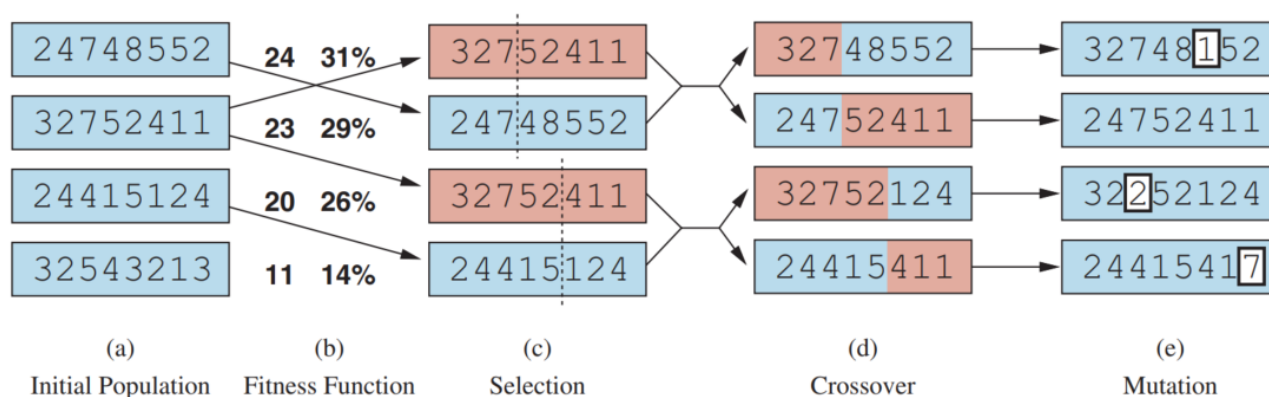
Để áp dụng GA, chúng ta cần định nghĩa ba thành phần cốt lõi:

- **Biểu diễn Cá thể (Chromosome - Nhiễm sắc thể):** Đây là cách chúng ta mã hóa một lời giải tiềm năng thành một cấu trúc dữ liệu mà thuật toán có thể làm việc được, tương tự như chuỗi DNA trong sinh học. Dạng mã hóa phổ biến nhất là chuỗi nhị phân, nhưng cũng có thể là chuỗi số nguyên, số thực, hoặc thậm chí là một cấu trúc cây (trong Lập trình Di truyền).
 - **Ví dụ (Bài toán 8 quân hậu):** Một lời giải có thể được mã hóa thành một chuỗi 8 chữ số $[r1, r2, \dots, r8]$, trong đó ri là vị trí hàng của quân hậu ở cột thứ i .
- **Hàm Thích nghi (Fitness Function):** Đây là thước đo định lượng, đánh giá một cá thể (lời giải) “tốt” đến mức nào. Hàm này đóng vai trò như “môi trường” trong tự nhiên, quyết định khả năng sinh tồn và sinh sản của cá thể. Việc thiết kế hàm thích nghi là bước quan trọng nhất và đặc thù cho từng bài toán.
 - **Ví dụ (Bài toán 8 quân hậu):** Một hàm thích nghi tốt là đếm **số cặp quân hậu không tấn công nhau**. Một bàn cờ hoàn hảo (lời giải tối ưu) sẽ có $8 * 7 / 2 = 28$ cặp không tấn công nhau. Giá trị hàm thích nghi càng cao, lời giải càng tốt.

- **Quần thể (Population):** Là một tập hợp gồm nhiều cá thể. GA làm việc đồng thời trên cả quần thể thay vì một lời giải duy nhất, giúp nó khám phá không gian tìm kiếm một cách đa dạng và tránh bị mắc kẹt ở các điểm tối ưu cục bộ.

Chu trình Tiến hóa của Thuật toán Di truyền

Một thuật toán di truyền điển hình vận hành theo một vòng lặp, mô phỏng các thế hệ nối tiếp nhau.



Hình 76. Thuật toán di truyền áp dụng cho bài toán 8 quân hậu.

(a) Khởi tạo Quần thể ban đầu: Thuật toán bắt đầu bằng việc tạo ra một quần thể gồm N cá thể một cách ngẫu nhiên. Trong Hình 76(a), chúng ta có 4 cá thể, mỗi cá thể là một chuỗi 8 chữ số đại diện cho một cách xếp 8 quân hậu.

Vòng lặp tiến hóa:

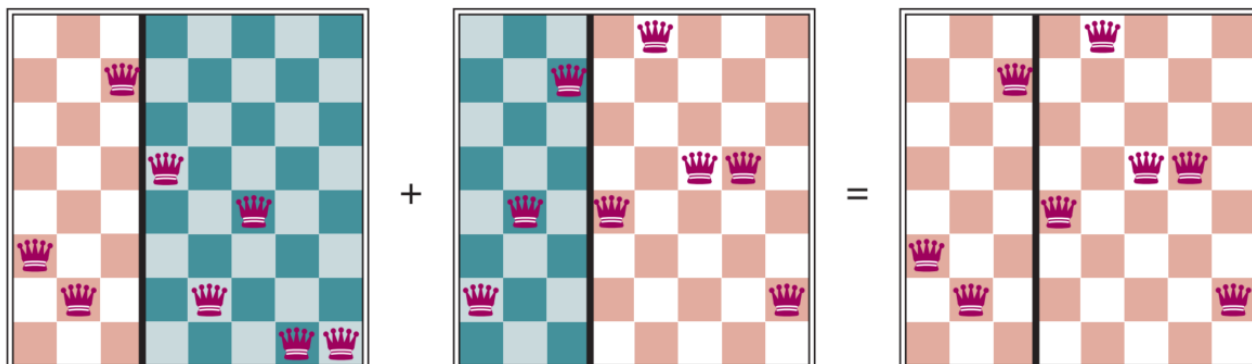
(b) Bước 1: Đánh giá (Evaluation) Mỗi cá thể trong quần thể được đánh giá bằng hàm thích nghi. Trong ví dụ, các giá trị fitness tương ứng là 24, 23, 20, và 11 (số cặp hậu không tấn công nhau). Các giá trị này sau đó được chuẩn hóa thành xác suất lựa chọn, cho thấy cá thể có fitness cao hơn sẽ có cơ hội được chọn làm cha mẹ cao hơn.

**** (c) Bước 2: Chọn lọc (Selection)**** Đây là giai đoạn “chọn lọc tự nhiên”. Các cá thể được chọn từ quần thể hiện tại để trở thành “cha mẹ” cho thế hệ tiếp theo. Các phương pháp phổ biến bao gồm:

- **Chọn lọc Bánh xe Roulette (Roulette Wheel Selection):** Mỗi cá thể chiếm một phần trên “bánh xe” tương ứng với tỷ lệ fitness của nó. Quay bánh xe N lần để chọn ra N cha mẹ.
- **Chọn lọc Giải đấu (Tournament Selection):** Chọn ngẫu nhiên một nhóm nhỏ cá thể, và cá thể có fitness cao nhất trong nhóm đó sẽ được chọn. Trong Hình 76(c), hai cặp cha mẹ được chọn. Lưu ý rằng cá thể (24) được chọn hai lần, trong khi cá thể (11) không được chọn lần nào.

**** (d) Bước 3: Lai ghép (Crossover/Recombination)**** Đây là quá trình “sinh sản”. Hai cá thể cha mẹ được chọn sẽ trao đổi thông tin di truyền để tạo ra một hoặc hai cá thể con. Một điểm cắt (crossover point) được chọn ngẫu nhiên.

- **Ví dụ:** Hai cha mẹ là 32752411 và 24748552. Điểm cắt được chọn sau vị trí thứ 3.
 - Con 1 = 327 (từ cha 1) + 48552 (từ cha 2) = 32748552
 - Con 2 = 247 (từ cha 2) + 52411 (từ cha 1) = 24752411 Quá trình này được minh họa trên bàn cờ trong hình dưới đây.



Hình 77. Trạng thái 8 quân hậu của hai cá thể cha và một cá thể con.

Hình 77: Minh họa trực quan phép lai ghép trên bàn cờ 8 quân hậu.

(e) Bước 4: Đột biến (Mutation) Sau khi lai ghép, mỗi gen trong cá thể con có một xác suất rất nhỏ để bị thay đổi một cách ngẫu nhiên. Đột biến đóng vai trò cực kỳ quan trọng:

- Duy trì sự đa dạng di truyền trong quần thể.
- Giúp thuật toán thoát khỏi các điểm tối ưu cục bộ. Trong Hình 76(e), một vài chữ số trong các cá thể con đã bị thay đổi ngẫu nhiên.

Bước 5: Tạo Thế hệ Mới và Thuyết Tinh hoa (Elitism) Các cá thể con vừa được tạo ra sẽ hình thành nên quần thể cho thế hệ tiếp theo. Một chiến lược rất phổ biến và hiệu quả là **Thuyết Tinh hoa (Elitism)**: giữ lại một hoặc một vài cá thể tốt nhất từ thế hệ cũ và chuyển thẳng sang thế hệ mới mà không qua lai ghép hay đột biến. Điều này đảm bảo rằng lời giải tốt nhất mà thuật toán từng tìm thấy sẽ không bao giờ bị mất đi.

Điều kiện Dừng: Vòng lặp tiến hóa sẽ tiếp tục cho đến khi một trong các điều kiện sau được thỏa mãn:

- Đạt đến số thế hệ tối đa.
- Fitness của cá thể tốt nhất không được cải thiện trong nhiều thế hệ liên tiếp.
- Tìm thấy một lời giải đạt yêu cầu.

Tổng hợp quy trình qua Giả mã

Toàn bộ các bước trên được tóm tắt trong giả mã của thuật toán di truyền.

```

function GENETIC-ALGORITHM(population, fitness) returns an individual
  repeat
    weights ← WEIGHTED-BY(population, fitness)
    population2 ← empty list
    for i = 1 to SIZE(population) do
      parent1, parent2 ← WEIGHTED-RANDOM-CHOICES(population, weights, 2)
      child ← REPRODUCE(parent1, parent2)
      if (small random probability) then child ← MUTATE(child)
      add child to population2
    population ← population2
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to fitness

function REPRODUCE(parent1, parent2) returns an individual
  n ← LENGTH(parent1)
  c ← random number from 1 to n
  return APPEND(SUBSTRING(parent1, 1, c), SUBSTRING(parent2, c + 1, n))

```

Hình 78. Mô tả thuật toán di truyền.

Kết luận: Sức mạnh của sự Tiến hóa Mô phỏng

Thuật toán Di truyền là một công cụ tối ưu hóa linh hoạt và mạnh mẽ, đặc biệt hiệu quả cho các bài toán có không gian tìm kiếm lớn, phức tạp, phi tuyến tính, hoặc không có mô hình toán học rõ ràng. Bằng cách mô phỏng các nguyên lý cốt lõi của tự nhiên—chọn lọc, tái tổ hợp và biến dị—GA có khả năng khám phá và tìm ra các lời giải chất lượng cao mà các phương pháp truyền thống có thể gặp khó khăn.

2.10.3. Các khái niệm cốt lõi trong thuật toán di truyền (dựa trên sinh học)

Thuật toán Di truyền (GA) là một bản giao hưởng của các nguyên lý sinh học được áp dụng vào khoa học máy tính. Để thực sự làm chủ được GA, việc hiểu rõ ngôn ngữ và các khái niệm nền tảng của nó—vốn được mượn trực tiếp từ di truyền học—là điều kiện tiên quyết. Các khái niệm này không chỉ là thuật ngữ, chúng là những bánh răng cốt lõi vận hành cỗ máy tiến hóa.

1. Quần thể (Population)

- **Định nghĩa chi tiết**

Quần thể không chỉ là một tập hợp các lời giải, mà nó là một **vốn gen (gene pool)** của toàn bộ quá trình tìm kiếm. Nó đại diện cho sự đa dạng của các chiến lược giải quyết vấn đề đang được xem xét tại một thời điểm (thế hệ). Mỗi cá thể trong quần thể là một “giả thuyết” về lời giải tốt nhất.

- **Chức năng và Vai trò**

Quần thể là sân khấu nơi quá trình tiến hóa diễn ra. Sự tương tác, cạnh tranh và sinh sản giữa các cá thể trong quần thể sẽ thúc đẩy thuật toán hướng về những vùng có lời giải tốt hơn trong không gian tìm kiếm. Một quần thể khỏe mạnh là một quần thể có sự cân bằng giữa các lời giải tốt và sự đa dạng để khám phá những hướng đi mới.

- **Quy mô quần thể (Population Size) và Sự cân bằng**

Quy mô quần thể là một siêu tham số quan trọng, quyết định sự cân bằng giữa **khám phá (exploration)** và **khai thác (exploitation)**:

- **Quần thể quá lớn:**
 - **Ưu điểm:** Độ đa dạng di truyền cao, giúp thuật toán **khám phá** được nhiều vùng khác nhau của không gian tìm kiếm, giảm nguy cơ hội tụ sớm vào một đáp án “tốt nhưng chưa phải tốt nhất” (tối ưu cục bộ).
 - **Nhược điểm:** Tốn kém chi phí tính toán khổng lồ vì phải đánh giá rất nhiều cá thể trong mỗi thế hệ, làm quá trình hội tụ chậm lại.
- **Quần thể quá nhỏ:**
 - **Ưu điểm:** Tốc độ xử lý mỗi thế hệ nhanh.
 - **Nhược điểm:** Đa dạng di truyền thấp, dễ dẫn đến hiện tượng “giao phối cận huyết”, làm mất đi các đặc tính tốt và khiến quần thể nhanh chóng bị mắc kẹt tại một điểm tối ưu cục bộ.

2. Nhiễm sắc thể (Chromosome)

- **Định nghĩa chi tiết**

Nhiễm sắc thể là cấu trúc dữ liệu mã hóa cho một lời giải ứng viên duy nhất. Nó được ví như **bản thiết kế di truyền (genotype)** của một cá thể. Thuật toán không trực tiếp làm việc với lời giải thực tế (phenotype) mà làm việc trên phiên bản mã hóa này.

- **Các dạng biểu diễn phổ biến**

Việc lựa chọn cách biểu diễn nhiễm sắc thể phụ thuộc hoàn toàn vào bản chất của bài toán:

- **Chuỗi nhị phân (Binary String):** Cổ điển và đơn giản nhất.
 - *Ví dụ (Bài toán cái túi - Knapsack):* Cho 5 món đồ, nhiễm sắc thể *10110* có thể biểu diễn việc “chọn món 1, 3, 4 và bỏ qua món 2, 5”.
- **Chuỗi số nguyên/số thực (Integer/Real-valued String):** Dùng cho các bài toán tối ưu hóa tham số.
 - *Ví dụ (Tối ưu siêu tham số mạng nơ-ron):* Nhiễm sắc thể *[0.001, 256, 0.95]* có thể biểu diễn (tốc độ học, kích thước lô, động lượng).
- **Chuỗi hoán vị (Permutation String):** Dùng cho các bài toán yêu cầu sắp xếp thứ tự.
 - *Ví dụ (Bài toán người du lịch - TSP):* Cho 5 thành phố A, B, C, D, E, nhiễm sắc thể *[C, A, E, B, D]* biểu diễn một lộ trình cụ thể.

3. Gen (Gene)

- **Định nghĩa chi tiết**

Gen là đơn vị thông tin cơ bản nhất, một thành phần cấu tạo nên nhiễm sắc thể. Nó đại diện cho một đặc tính hoặc một tham số đơn lẻ của lời giải.

- **Ví dụ cụ thể**

- Trong nhiễm sắc thể *10110* của bài toán cái túi, mỗi bit (*1* hoặc *0*) là một gen, quyết định việc “chọn” hay “không chọn” một món đồ.

- Trong nhiễm sắc thể $[C, A, E, B, D]$ của bài toán TSP, mỗi ký tự $(C, A, \dots)$ là một gen, xác định một thành phố trong lộ trình.
- **Ý nghĩa trong các toán tử di truyền**
Các toán tử di truyền cốt lõi như lai ghép và đột biến đều tác động trực tiếp lên các gen.
 - **Đột biến (Mutation):** Thay đổi giá trị của một gen (ví dụ: lật bit 0 thành 1).
 - **Lai ghép (Crossover):** Trao đổi một hoặc nhiều gen giữa các nhiễm sắc thể cha mẹ. Do đó, cách định nghĩa gen sẽ quyết định cách thuật toán tạo ra sự đa dạng và khám phá các lời giải mới.

4. Hàm Thích nghi (Fitness Function)

- **Định nghĩa chi tiết**
Hàm thích nghi là cầu nối duy nhất giữa bài toán thực tế và thuật toán di truyền. Nó đóng vai trò như “**áp lực của môi trường**”, định lượng mức độ phù hợp của một lời giải (cá thể) đối với mục tiêu của bài toán. Đây chính là hàm mục tiêu (objective function) mà GA đang cố gắng tối ưu hóa.
- **Vai trò tối quan trọng**
 - **Kim chỉ nam cho sự tiến hóa:** Bằng cách gán một điểm số “thích nghi” cho mỗi cá thể, hàm này trực tiếp quyết định cá thể nào có nhiều khả năng sống sót, sinh sản và truyền lại gen cho thế hệ sau.
 - **Định lượng hóa mục tiêu:** Nó biến một khái niệm trừu tượng như “lời giải tốt” thành một con số cụ thể, cho phép thuật toán so sánh và xếp hạng các lời giải.
- **Thiết kế và Thách thức**
Thiết kế một hàm thích nghi tốt là nghệ thuật và là yếu tố quyết định thành công của GA.
 - **Tính chính xác:** Hàm phải phản ánh đúng mục tiêu bài toán. Một hàm thích nghi sai lệch sẽ dẫn dắt quá trình tiến hóa đi sai hướng.
 - **Tính hiệu quả:** Hàm này sẽ được gọi rất nhiều lần (số cá thể * số thế hệ), do đó nó cần được tính toán một cách nhanh chóng.
 - **Ví dụ (Bài toán TSP):**
 - *Mục tiêu:* Tìm lộ trình có tổng quãng đường ngắn nhất.
 - *Hàm thích nghi:* Vì GA thường là bài toán *tối đa hóa*, chúng ta không thể dùng trực tiếp tổng quãng đường (cần *tối thiểu hóa*). Thay vào đó, ta có thể định nghĩa $Fitness = 1 / Tổng_quãng_đường$. Như vậy, lộ trình càng ngắn, tổng quãng đường càng nhỏ, và giá trị fitness sẽ càng cao.

Bốn khái niệm **Quần thể**, **Nhiễm sắc thể**, **Gen**, và **Hàm thích nghi** tạo thành một vòng lặp tương tác khép kín. **Quần thể** cung cấp các **Nhiễm sắc thể** (lời giải). **Hàm thích nghi** đánh giá chúng. Dựa trên điểm số đó, các toán tử di truyền sẽ thao tác trên các **Gen** để tạo ra thế hệ mới. Vòng lặp này cứ tiếp diễn, mô phỏng quá trình tiến hóa để từng bước tinh lọc và hội tụ đến lời giải tối ưu. Hiểu rõ sự tương tác này chính là chìa khóa để triển khai GA một cách hiệu quả.

2.10.3. Các khái niệm cốt lõi trong thuật toán di truyền (dựa trên sinh học)

Thuật toán Di truyền (GA) không chỉ đơn thuần là một phương pháp tính toán, mà nó là một triết lý giải quyết vấn đề được vay mượn từ cơ chế tiến hóa hàng triệu năm của tự nhiên. Để hiểu và áp dụng GA một cách hiệu quả, chúng ta cần nắm vững “ngôn ngữ” của nó, vốn có nguồn gốc sâu sắc từ di truyền học.

1. Quần thể (Population)

Định nghĩa

Quần thể là một tập hợp các **cá thể** (individuals), trong đó mỗi cá thể là một lời giải ứng viên cho bài toán. Thay vì xem xét từng lời giải một cách riêng lẻ, GA làm việc trên toàn bộ “vốn gen” (gene pool) của quần thể, cho phép khám phá đồng thời nhiều khu vực trong không gian tìm kiếm. Đây là điểm khác biệt cơ bản so với các thuật toán tìm kiếm cục bộ truyền thống.

Vai trò

Quần thể là **đơn vị của sự tiến hóa**. Mọi hoạt động chính của GA—từ đánh giá, chọn lọc, lai ghép đến đột biến—đều diễn ra trên quy mô quần thể. Nó giống như một hệ sinh thái thu nhỏ, nơi các lời giải “cạnh tranh” với nhau để tồn tại và sinh sản, từ đó thúc đẩy sự cải thiện chung của cả quần thể qua từng thế hệ.

Kích thước Quần thể (Population Size)

Đây là một siêu tham số mang tính chiến lược, quyết định sự cân bằng giữa **khám phá (exploration)** và **khai thác (exploitation)**:

- **Kích thước quá nhỏ:**
 - **Rủi ro:** Dễ mất đi sự đa dạng di truyền. Quần thể nhanh chóng bị chiếm lĩnh bởi một vài cá thể vượt trội, dẫn đến **hội tụ sớm** (premature convergence). Thuật toán có thể bị “mắc kẹt” tại một đỉnh đồi thấp (tối ưu cục bộ) mà không đủ sức mạnh để khám phá những đỉnh cao hơn.
 - **Tương tự trong sinh học:** Giống như một quần thể động vật nhỏ bị suy thoái do giao phối cận huyết.
- **Kích thước quá lớn:**
 - **Rủi ro:** Chi phí tính toán cho việc đánh giá mỗi thế hệ trở nên khổng lồ, làm **chậm tốc độ hội tụ**. Mặc dù khả năng khám phá tốt hơn, nhưng thuật toán có thể mất quá nhiều thời gian để tìm ra lời giải tốt.
 - **Tương tự trong sinh học:** Một quần thể quá lớn và dàn trải có thể làm loãng các gen tốt, khiến quá trình chọn lọc tự nhiên diễn ra chậm hơn.

2. Nhiễm sắc thể (Chromosome)

Định nghĩa

Một **nhiễm sắc thể** là cấu trúc dữ liệu dùng để **mã hóa** một cá thể, tức là một lời giải tiềm năng. Nó được ví như **kiểu gen (genotype)**—bản thiết kế di truyền—của lời giải. Thuật toán không trực tiếp thao tác trên lời giải thực tế (kiểu hình - phenotype), mà trên phiên bản mã hóa này.

Mã hóa (Encoding)

Việc lựa chọn phương pháp mã hóa là một trong những quyết định thiết kế quan trọng nhất, phụ thuộc vào bản chất bài toán:

- **Mã hóa Nhị phân:** Dạng cổ điển và phổ biến nhất, đặc biệt phù hợp với các bài toán lựa chọn hoặc các bài toán có tham số có thể được rời rạc hóa.
 - *Ví dụ:* Bài toán cái túi (chọn hoặc không chọn vật phẩm).
- **Mã hóa Số nguyên/Số thực:** Phù hợp cho các bài toán tối ưu hóa tham số.
 - *Ví dụ:* Tìm các trọng số tối ưu cho một mạng nơ-ron hoặc tìm cực trị của hàm số toán học.
- **Mã hóa Hoán vị:** Chuyên dụng cho các bài toán yêu cầu tìm một thứ tự tối ưu.
 - *Ví dụ:* Bài toán người du lịch (tìm thứ tự các thành phố), bài toán lập lịch (sắp xếp các công việc).

Yêu cầu của việc Mã hóa

Một cơ chế mã hóa tốt cần đảm bảo:

- **Tính Toàn vẹn:** Có khả năng biểu diễn mọi lời giải khả dĩ trong không gian tìm kiếm.
- **Tính Hiệu quả:** Dễ dàng thực hiện các phép toán lai ghép và đột biến mà không tạo ra các lời giải “bất hợp pháp” (infeasible solutions).
- **Tính Minh bạch:** Dễ dàng giải mã từ genotype (chuỗi mã hóa) trở lại phenotype (lời giải thực tế) để đánh giá.

3. Gen (Gene)

Định nghĩa

Gen là đơn vị thông tin cơ bản cấu thành nên một nhiễm sắc thể. Mỗi gen mã hóa cho một đặc tính hoặc một phần nhỏ của lời giải tổng thể.

Ví dụ minh họa

Xét bài toán tìm giá trị lớn nhất của hàm $f(x) = x^2$ với $x \in [0,31]$, mã hóa x bằng chuỗi nhị phân 5-bit:

- **Nhiễm sắc thể:** 10110 (biểu diễn số nguyên 22).
- **Các Gen:** 1, 0, 1, 1, 0. Mỗi gen (bit) tại một vị trí cụ thể đóng góp vào giá trị tổng thể của lời giải.

Chức năng

Gen là đối tượng tác động trực tiếp của các toán tử di truyền:

- **Đột biến (Mutation):** Thao tác thay đổi giá trị của một hoặc một vài gen (ví dụ: lật bit 1 thành 0).
- **Lai ghép (Crossover):** Thao tác trao đổi các đoạn gen giữa các nhiễm sắc thể cha mẹ.

4. Hàm thích nghi (Fitness Function)

Định nghĩa

Hàm thích nghi là một hàm số dùng để **định lượng chất lượng** của một lời giải. Nó đóng vai trò như “áp lực chọn lọc của môi trường tự nhiên”, quyết định cá thể nào “khỏe mạnh” và xứng đáng được tồn tại, sinh sản.

Vai trò

Đây là thành phần **quan trọng nhất**, là kim chỉ nam định hướng toàn bộ quá trình tiến hóa:

- **Cầu nối giữa bài toán và thuật toán:** Nó dịch mục tiêu của bài toán (ví dụ: “giảm thiểu chi phí”, “tối đa hóa lợi nhuận”) thành một giá trị số mà thuật toán có thể hiểu và tối ưu hóa.
- **Cơ sở để Chọn lọc:** Các cá thể có giá trị fitness cao hơn sẽ có xác suất được chọn làm cha mẹ cao hơn, đảm bảo rằng các đặc tính tốt được lưu giữ và nhân rộng.
- **Thước đo sự Hội tụ:** Bằng cách theo dõi giá trị fitness của cá thể tốt nhất qua các thế hệ, chúng ta có thể biết được khi nào thuật toán đã hội tụ và nên dừng lại.

Yêu cầu và Thiết kế

- **Chuyển đổi bài toán Tối thiểu hóa:** Vì GA về bản chất là một quá trình tối đa hóa (tìm fitness cao nhất), các bài toán tối thiểu hóa (ví dụ: tìm đường đi ngắn nhất) cần được chuyển đổi. Một cách phổ biến là $Fitness = 1 / Cost$ hoặc $Fitness = C_{max} - Cost$.
- **Xử lý Ràng buộc:** Đối với các bài toán có ràng buộc, hàm fitness thường kết hợp một **thành phần phạt (penalty)**. Các lời giải vi phạm ràng buộc sẽ bị trừ điểm fitness, làm giảm khả năng sinh sản của chúng.

Bốn khái niệm **Quần thể**, **Nhiễm sắc thể**, **Gen**, và **Hàm thích nghi** tạo nên một hệ sinh thái nhân tạo hoàn chỉnh. **Quần thể** là môi trường sống, chứa các **Nhiễm sắc thể** (lời giải). **Hàm thích nghi** đóng vai trò là quy luật sinh tồn, đánh giá các nhiễm sắc thể này. Dựa trên sự đánh giá đó, các toán tử di truyền tác động lên các **Gen** để tạo ra thế hệ tiếp theo, ưu việt hơn. Việc nắm vững các khái niệm này là nền tảng để xây dựng và tinh chỉnh các thuật toán di truyền mạnh mẽ và hiệu quả.

2.10.4. Chu trình của một thuật toán di truyền: Các bước tiến hóa qua thế hệ

Thuật toán Di truyền (GA) vận hành theo một quy trình lặp lại, có cấu trúc chặt chẽ, mô phỏng quá trình tiến hóa của sự sống. Mỗi vòng lặp được gọi là một **thế hệ** (generation), nơi một quần thể các lời giải được sàng lọc và cải tiến. Chu trình này là một vũ điệu tinh tế giữa việc kế thừa những đặc tính tốt và khám phá những khả năng mới, cho đến khi tìm thấy lời giải tối ưu hoặc đạt đến giới hạn cho phép.

Hãy cùng đi sâu vào từng bước của hành trình tiến hóa này.

Bước 1: Khởi tạo (Initialization)

- **Mục tiêu**

Tạo ra “thế hệ Adam và Eve”—một quần thể ban đầu gồm N cá thể. Đây là điểm xuất phát của toàn bộ quá trình tìm kiếm, giống như việc gieo những hạt giống đầu tiên vào một mảnh đất màu mỡ.

- **Phương pháp**

- **Khởi tạo Ngẫu nhiên (Random Initialization):** Đây là phương pháp phổ biến nhất. Các gen của mỗi nhiễm sắc thể được gán giá trị một cách ngẫu nhiên trong phạm vi cho phép. Cách này đảm bảo tính đa dạng tối đa lúc ban đầu, giống như việc “quăng một mẻ lưới rộng” để không bỏ sót bất kỳ khu vực tiềm năng nào trong không gian lời giải.
- **Khởi tạo có Hướng (Heuristic Initialization):** Trong một số trường hợp, nếu chúng ta có sẵn kiến thức về bài toán (ví dụ: một lời giải tốt từ một thuật toán khác), ta có thể “gieo mầm” cho quần thể ban đầu bằng những lời giải này. Việc này có thể giúp thuật toán hội tụ nhanh hơn, nhưng cần cẩn thận để không làm giảm sự đa dạng cần thiết.

- **Yêu cầu quan trọng**

Quần thể ban đầu **phải đa dạng**. Một quần thể khởi đầu mà các cá thể quá giống nhau sẽ hạn chế khả năng khám phá của thuật toán và làm tăng nguy cơ bị mắc kẹt sớm tại một điểm tối ưu cục bộ.

Bước 2: Đánh giá (Evaluation)

- **Mục tiêu**

“Ngày phán xét” cho từng cá thể. Sử dụng **hàm thích nghi (fitness function)** đã định nghĩa, chúng ta tính toán và gán một điểm số cho mỗi cá thể trong quần thể. Điểm số này định lượng mức độ “tốt” hay “phù hợp” của lời giải mà cá thể đó đại diện.

- **Ý nghĩa**

Đây là bước **duy nhất** mà bài toán thực tế “giao tiếp” với thuật toán. Hàm thích nghi là kim chỉ nam, là ngọn hải đăng định hướng cho toàn bộ quá trình tiến hóa. Các cá thể có điểm số cao sẽ là những ứng cử viên sáng giá cho việc duy trì nòi giống ở thế hệ tiếp theo.

Bước 3: Vòng lặp Tiến hóa (The Evolutionary Loop)

Đây là trái tim của thuật toán, nơi các thế hệ được sinh ra và chọn lọc. Vòng lặp này bao gồm ba giai đoạn chính:

a. Chọn lọc (Selection)

- **Mục tiêu**

Áp dụng nguyên tắc “sự sống còn của kẻ thích nghi nhất” (survival of the fittest). Từ quần thể hiện tại, chúng ta chọn ra một nhóm cá thể để làm “cha mẹ” (parents) cho thế hệ sau.

- **Các phương pháp phổ biến**

- **Chọn lọc Bánh xe Roulette (Roulette Wheel):** Tưởng tượng một bánh xe roulette, mỗi cá thể chiếm một miếng bánh có diện tích tỉ lệ thuận với điểm fitness của nó. Quay bánh xe N lần, cá thể nào được chọn sẽ trở thành cha mẹ. Cách này đảm bảo cá thể khỏe hơn có cơ hội được chọn cao hơn, nhưng cá thể yếu hơn vẫn có một cơ hội nhỏ.
- **Chọn lọc Giải đấu (Tournament Selection):** Tổ chức các “giải đấu” nhỏ. Chọn ngẫu nhiên k cá thể từ quần thể, và cá thể có fitness cao nhất trong nhóm nhỏ này sẽ chiến thắng và được chọn làm cha mẹ. Phương pháp này rất hiệu quả và dễ điều chỉnh “áp lực chọn lọc” bằng cách thay đổi kích thước giải đấu k .
- **Chọn lọc theo Thứ hạng (Rank-based Selection):** Sắp xếp các cá thể theo thứ hạng fitness. Xác suất được chọn sẽ dựa trên thứ hạng chứ không phải giá trị fitness tuyệt đối. Cách này giúp giảm bớt sự thống trị của các cá thể “siêu sao” và tránh hội tụ sớm.

b. Lai ghép (Crossover)

- **Mục tiêu**

Khai thác (exploitation). Tạo ra các cá thể con mới bằng cách kết hợp vật liệu di truyền từ hai cá thể cha mẹ đã được chọn. Đây là cơ chế chính để tạo ra các lời giải mới, hy vọng rằng con cái sẽ kế thừa những đặc tính tốt từ cả cha và mẹ.

- **Các phương pháp phổ biến**

- **Lai ghép một điểm (One-point Crossover):** Chọn một điểm cắt ngẫu nhiên trên nhiễm sắc thể. Cá thể con được tạo ra bằng cách lấy phần đầu từ cha 1 và phần đuôi từ cha 2.
- **Lai ghép hai điểm (Two-point Crossover):** Chọn hai điểm cắt. Đoạn gen ở giữa sẽ được hoán đổi giữa hai cha mẹ.
- **Lai ghép đồng nhất (Uniform Crossover):** Với mỗi vị trí gen, tung một đồng xu để quyết định xem gen của con sẽ được thừa hưởng từ cha 1 hay cha 2.

- **Tỷ lệ Lai ghép (Crossover Rate)**

Đây là xác suất mà một cặp cha mẹ sẽ thực hiện lai ghép (thường từ 60% đến 90%). Nếu không lai ghép, con cái sẽ là bản sao chính xác của cha mẹ.

c. Đột biến (Mutation)

- **Mục tiêu**

Khám phá (exploration). Đột biến là một sự thay đổi nhỏ và ngẫu nhiên trên một cá thể con. Nó đóng vai trò như một “tia lửa sáng tạo bất ngờ”, giúp đưa những thông tin di truyền mới vào quần thể.

- **Vai trò tối quan trọng**

- Ngăn ngừa hội tụ sớm bằng cách duy trì sự đa dạng di truyền.
- Giúp thuật toán thoát ra khỏi các điểm tối ưu cục bộ. Nếu không có đột biến, một khi một gen nào đó biến mất khỏi quần thể, nó sẽ không bao giờ xuất hiện trở lại.

- **Cách thực hiện**

- **Với chuỗi nhị phân:** Lật một bit ngẫu nhiên ($0 \leftrightarrow 1$).
- **Với chuỗi số thực:** Thêm một nhiễu nhỏ (ví dụ, từ phân phối Gaussian) vào một gen.
- **Với chuỗi hoán vị:** Hoán đổi vị trí của hai gen ngẫu nhiên.

- **Tỷ lệ Đột biến (Mutation Rate)**

Xác suất này thường rất nhỏ (ví dụ, 0.1% đến 1%). Nếu quá cao, quá trình tìm kiếm sẽ trở nên hỗn loạn như tìm kiếm ngẫu nhiên. Nếu quá thấp, quần thể có thể mất đi sự đa dạng cần thiết.

Bước 4: Thay thế và Điều kiện Dừng (Replacement & Termination)

- **Tạo Thế hệ Mới**

Sau khi các cá thể con được tạo ra, chúng ta cần quyết định quần thể cho thế hệ tiếp theo sẽ trông như thế nào.

- **Thay thế theo Thế hệ (Generational Replacement):** Toàn bộ quần thể cũ bị thay thế bởi quần thể con mới.
- **Thuyết Tinh hoa (Elitism):** Một chiến lược cực kỳ quan trọng và phổ biến. Chúng ta giữ lại một hoặc một vài cá thể có fitness cao nhất từ thế hệ cũ và chuyển thẳng sang thế hệ mới. Điều này **đảm bảo rằng lời giải tốt nhất từng được tìm thấy sẽ không bao giờ bị mất đi**.

- **Điều kiện Dừng**

Vòng lặp tiến hóa sẽ tiếp diễn cho đến khi một trong các tiêu chí sau được thỏa mãn:

1. **Đạt số thế hệ tối đa:** Giới hạn thời gian tính toán cho phép.
2. **Hội tụ:** Giá trị fitness của cá thể tốt nhất không cải thiện đáng kể sau một số lượng lớn các thế hệ.
3. **Đạt mục tiêu:** Tìm thấy một lời giải có giá trị fitness vượt qua một ngưỡng mong muốn.

Khi thuật toán dừng lại, cá thể có fitness cao nhất trong quần thể cuối cùng được xem là lời giải tốt nhất mà GA tìm được cho bài toán.

2.10.4. Chu trình của một thuật toán di truyền: Các bước tiến hóa từ tự nhiên sang thuật toán

Thuật toán Di truyền (GA) là một bản mô phỏng tinh vi của thuyết “chọn lọc tự nhiên” do Charles Darwin khởi xướng. Nó không giải quyết bài toán bằng logic toán học cứng nhắc, mà bằng cách cho một “quần thể” các lời giải tự “tiến hóa” qua các thế hệ. Mỗi thế hệ là một chu kỳ chọn lọc, lai tạo và biến đổi, nơi những lời giải tốt hơn dần chiếm ưu thế và những lời giải kém hơn bị đào thải. Quá trình này lặp lại cho đến khi quần thể hội tụ về một lời giải xuất sắc.

Dưới đây là mô tả chi tiết từng giai đoạn trong hành trình tiến hóa này:

Bước 1: Khởi tạo (Initialization) – Tạo ra “Thế hệ Adam & Eve”

- **Mục tiêu**

Tạo ra điểm xuất phát cho quá trình tiến hóa: một quần thể ban đầu gồm N cá thể, mỗi cá thể là một lời giải ứng viên được mã hóa. Đây là bước gieo những hạt giống di truyền đầu tiên, quyết định “vốn gen” ban đầu của thuật toán.

- **Phương pháp Khởi tạo**

- **Ngẫu nhiên (Random):** Đây là cách tiếp cận phổ biến nhất, tương tự như sự đa dạng sinh học ban sơ. Các gen của mỗi cá thể được tạo ra ngẫu nhiên trong miền giá trị hợp lệ. Phương pháp này đảm bảo một sự đa dạng tối đa, cho phép thuật toán “quăng lưới” ra khắp không gian tìm kiếm.
- **Có định hướng (Seeding/Heuristic):** Nếu có kiến thức chuyên môn hoặc lời giải tốt từ các thuật toán khác, ta có thể “gieo mầm” chúng vào quần thể ban đầu. Việc này giúp quá trình tiến hóa có một khởi đầu thuận lợi và có thể hội tụ nhanh hơn, nhưng cần cân trọng để không làm giảm sự đa dạng cần thiết.

- **Yêu cầu cốt lõi**

Sự đa dạng là chìa khóa. Một quần thể ban đầu mà các cá thể quá giống nhau (thiếu đa dạng di truyền) sẽ khiến thuật toán dễ dàng bị mắc kẹt tại một đỉnh đồi thấp (tối ưu cục bộ) và không bao giờ khám phá ra được những đỉnh cao hơn.

Bước 2: Đánh giá (Evaluation) – “Ngày phán xét”

- **Mục tiêu**

Định lượng chất lượng của từng cá thể trong quần thể thông qua **hàm thích nghi (fitness function)**. Mỗi cá thể sẽ nhận được một “điểm số thích nghi”, phản ánh mức độ phù hợp của nó với mục tiêu bài toán.

- **Chức năng**

Đây là “áp lực chọn lọc của môi trường”. Giá trị fitness là yếu tố duy nhất quyết định sự sống còn và khả năng sinh sản của một cá thể. Các cá thể có fitness cao được xem là “khỏe mạnh”, có khả năng thích nghi tốt và sẽ có nhiều cơ hội để truyền lại bộ gen ưu việt của mình.

- **Yêu cầu đối với Hàm Fitness**

- **Phản ánh đúng mục tiêu:** Phải đo lường chính xác mức độ “tốt” của một lời giải. Một hàm fitness sai lệch sẽ dẫn dắt quá trình tiến hóa đi sai hướng.
- **Hiệu quả tính toán:** Hàm này được gọi rất nhiều lần, do đó cần được tính toán nhanh chóng.
- **Phân biệt rõ ràng:** Có khả năng phân biệt rạch ròi giữa các lời giải tốt, trung bình và kém.

Bước 3: Chu trình Tiến hóa (Evolution Loop) – Trái tim của GA

Đây là giai đoạn lặp đi lặp lại, nơi các thế hệ mới được tạo ra thông qua sự kết hợp của ba lực lượng tiến hóa chính:

(a) Chọn lọc (Selection) – Sự sống còn của kẻ thích nghi nhất

- **Mục tiêu**

Chọn ra những cá thể “ưu tú” từ quần thể hiện tại để làm “cha mẹ” cho thế hệ tiếp theo. Đây là cơ chế **khai thác (exploitation)**, tập trung vào việc nhân rộng những gen tốt đã được tìm thấy.

- **Phương pháp chọn lọc phổ biến**

- **Bánh xe Roulette (Roulette Wheel):** Mỗi cá thể được chia một phần trên “bánh xe” tỉ lệ với fitness của nó. Quay bánh xe N lần để chọn ra N cha mẹ. Phương pháp này cho cả những cá thể yếu hơn một cơ hội nhỏ để tồn tại, giúp duy trì đa dạng.
- **Giải đấu (Tournament):** Tổ chức các “trận đấu” nhỏ bằng cách chọn ngẫu nhiên k cá thể và chỉ người chiến thắng (fitness cao nhất) được chọn. Đây là một phương pháp hiệu quả và dễ dàng điều chỉnh “áp lực chọn lọc”.
- **Theo Thứ hạng (Rank):** Sắp xếp các cá thể theo fitness và chọn dựa trên thứ hạng. Cách này ngăn chặn sự thống trị tuyệt đối của các cá thể “siêu sao”, tránh hội tụ sớm.

(b) Lai ghép (Crossover) – Tái tổ hợp để tạo sự đột phá

- **Mục tiêu**

Tạo ra các cá thể con (offspring) bằng cách kết hợp vật liệu di truyền từ hai cha mẹ. Đây là nguồn sức mạnh chính của GA, cho phép tạo ra những lời giải hoàn toàn mới mà vẫn kế thừa những đặc tính tốt từ thế hệ trước.

- **Phương pháp lai ghép**

- **Một điểm (One-point):** Chọn một điểm cắt ngẫu nhiên, con cái sẽ thừa hưởng phần đầu từ cha 1 và phần đuôi từ cha 2.
- **Hai điểm (Two-point):** Chọn hai điểm cắt, hoán đổi đoạn gen nằm giữa hai cha mẹ.
- **Đồng nhất (Uniform):** Với mỗi gen, tung đồng xu để quyết định xem con sẽ thừa hưởng gen từ cha 1 hay cha 2.

- **Tỷ lệ Lai ghép (Crossover Rate)**

Là xác suất một cặp cha mẹ sẽ tiến hành lai ghép (thường 0.6 – 0.9). Nếu không, chúng sẽ chỉ tạo ra bản sao của chính mình.

(c) Đột biến (Mutation) – Tia lửa của sự sáng tạo

- **Mục tiêu**

Khám phá (exploration). Đột biến là một sự thay đổi nhỏ và ngẫu nhiên trên một vài gen của cá thể con. Nó đóng vai trò như một “lưới an toàn”, đảm bảo rằng không một thông tin di truyền nào bị mất vĩnh viễn khỏi quần thể.

- **Phương pháp đột biến**

- **Đảo bit (Bit-flip):** Lật một bit từ 0 sang 1 hoặc ngược lại (cho chuỗi nhị phân).
- **Thêm nhiễu (Gaussian Noise):** Thêm một giá trị ngẫu nhiên nhỏ (cho số thực).
- **Hoán vị (Swap):** Đổi vị trí hai gen (cho bài toán hoán vị).

- **Tỷ lệ Đột biến (Mutation Rate)**

Phải rất nhỏ (thường 0.001 – 0.01). Nếu quá cao, GA sẽ trở thành một cuộc tìm kiếm hỗn loạn. Nếu quá thấp, quần thể có nguy cơ mất đi sự đa dạng và hội tụ sớm.

Bước 4: Tạo Quần thể Mới và Điều kiện Dừng

- **Thay thế (Replacement)**

Sau khi thế hệ con được tạo ra, chúng sẽ thay thế quần thể cũ để bắt đầu một chu kỳ mới.

- **Thay thế Toàn bộ (Generational):** Tất cả cá thể cũ bị loại bỏ.
- **Thay thế Ổn định (Steady-state):** Chỉ một vài cá thể yếu nhất bị thay thế bởi những cá thể con tốt nhất.
- **Thuyết Tinh hoa (Elitism):** Một chiến lược cực kỳ quan trọng. Một hoặc vài cá thể có fitness cao nhất từ thế hệ cũ được **đảm bảo** sống sót và chuyển thẳng sang thế hệ mới. Điều này ngăn chặn việc vô tình làm mất đi lời giải tốt nhất đã tìm thấy.

- **Điều kiện Dừng**

Quá trình tiến hóa sẽ kết thúc khi một trong các tiêu chí sau được thỏa mãn:

- **Đạt số thế hệ tối đa:** Giới hạn về tài nguyên tính toán.
- **Sự hội tụ:** Chất lượng của lời giải tốt nhất không còn cải thiện đáng kể sau nhiều thế hệ.
- **Đạt mục tiêu:** Tìm thấy một cá thể có fitness vượt qua ngưỡng yêu cầu.
- **Mất đa dạng:** Khi tất cả các cá thể trong quần thể trở nên quá giống nhau.

Khi thuật toán dừng lại, cá thể có fitness cao nhất trong quần thể cuối cùng chính là lời giải tốt nhất mà quá trình tiến hóa đã tìm ra.

2.10.5. Ví dụ minh họa: Giải bài toán Ba lô (0/1 Knapsack) bằng thuật toán di truyền

Để làm sáng tỏ cơ chế hoạt động của Thuật toán Di truyền (GA), không có gì tốt hơn là áp dụng nó vào một bài toán kinh điển. Chúng ta sẽ sử dụng **Bài toán Ba lô 0/1 (0/1 Knapsack Problem)**—một bài toán tối ưu hóa tổ hợp nổi tiếng, thường được dùng làm “thuốc thử” cho hiệu quả của các thuật toán metaheuristic như GA.

Mô tả Bài toán

Tưởng tượng bạn là một nhà thám hiểm chuẩn bị cho một chuyến đi. Bạn có một chiếc ba lô với sức chứa giới hạn và một loạt các vật phẩm, mỗi vật phẩm có giá trị và trọng lượng riêng. Bạn phải quyết định nên mang theo những vật phẩm nào.

- **Đầu vào:**

- Một ba lô có sức chứa tối đa **W**.
- **N** đồ vật. Mỗi đồ vật *i* có:
 - Giá trị (value) **vi**
 - Khối lượng (weight) **wi**

- **Mục tiêu:** Chọn một tập hợp các đồ vật để bỏ vào ba lô sao cho:

4. Tổng khối lượng của chúng không vượt quá sức chứa **W**.
5. Tổng giá trị của chúng là **lớn nhất có thể**.

Đây là một bài toán **NP-hard**, có nghĩa là không tồn tại thuật toán hiệu quả (chạy trong thời gian đa thức) để tìm ra lời giải tối ưu cho mọi trường hợp. Đây chính là lý do tại sao các phương pháp tìm kiếm gần đúng như GA lại trở nên hữu ích.

Bước 1: Thiết lập Thuật toán Di truyền

a. Biểu diễn Lời giải: Mã hóa Chuỗi Nhị phân (Binary Encoding)

Cách biểu diễn trực quan và hiệu quả nhất cho bài toán này là sử dụng một chuỗi nhị phân.

- **Nhiễm sắc thể (Chromosome):** Mỗi lời giải (cá thể) được mã hóa bằng một chuỗi nhị phân có độ dài N .
 - Bit thứ $i = 1$ có nghĩa là “chọn đồ vật thứ i ”.
 - Bit thứ $i = 0$ có nghĩa là “không chọn đồ vật thứ i ”.
- **Ví dụ:** Với $N = 5$ đồ vật, nhiễm sắc thể 10110 đại diện cho việc chọn các đồ vật 1, 3, và 4, bỏ qua đồ vật 2 và 5.

b. Hàm Thích nghi (Fitness Function) – “Đo lường” chất lượng

Hàm thích nghi phải vừa khuyến khích việc tối đa hóa giá trị, vừa trừng phạt những lời giải vi phạm ràng buộc.

- **Tính tổng giá trị (V) và tổng khối lượng (Wtotal)** của các vật phẩm được chọn bởi một nhiễm sắc thể.

Hàm Fitness được định nghĩa như sau: $\text{fitness} = \sum_{i=1}^N v_i \cdot x_i$

trong đó $x_i \in \{0,1\}$ là giá trị bit tại vị trí i

Hàm này hoạt động như một người gác cổng nghiêm khắc:

- Nếu ba lô không bị quá tải, fitness của lời giải chính là tổng giá trị của nó.
- Nếu ba lô bị quá tải (vi phạm ràng buộc), lời giải đó là “bất hợp pháp” và bị gán fitness bằng 0. Điều này khiến nó gần như không có cơ hội được chọn để sinh sản, và sẽ bị đào thải khỏi quần thể.

Bước 2: Minh họa một Thế hệ GA

Hãy cùng đi qua từng bước của một thế hệ với ví dụ cụ thể sau:

- Số đồ vật $N = 4$.
- Sức chứa ba lô $W = 9$.
- Bảng đồ vật:

Đồ vật	Giá trị (v)	Khối lượng (w)
1	6	4
2	4	3
3	5	5

Đồ vật	Giá trị (v)	Khối lượng (w)
4	7	6

a. Khởi tạo Quần thể (Population Size = 4)

Giả sử chúng ta khởi tạo ngẫu nhiên được 4 cá thể sau:

Cá thể	Nhiễm sắc thể	Diễn giải
A	1010	Chọn đồ vật 1, 3
B	0110	Chọn đồ vật 2, 3
C	1100	Chọn đồ vật 1, 2
D	1001	Chọn đồ vật 1, 4

b. Đánh giá (Tính Fitness)

Ta tính toán giá trị và khối lượng cho từng cá thể:

Cá thể	Tổng Giá trị (V)	Tổng Khối lượng (Wtotal)	Fitness
A	$6 + 5 = 11$	$4 + 5 = 9 (\leq 9)$	11
B	$4 + 5 = 9$	$3 + 5 = 8 (\leq 9)$	9
C	$6 + 4 = 10$	$4 + 3 = 7 (\leq 9)$	10
D	$6 + 7 = 13$	$4 + 6 = 10 (> 9)$	0

Lưu ý quan trọng: Cá thể D, mặc dù có tổng giá trị tiềm năng cao nhất (13), nhưng lại vi phạm ràng buộc khối lượng ($10 > 9$). Do đó, fitness của nó bị phạt xuống còn 0.

c. Chọn lọc (Selection)

Sử dụng phương pháp **Bánh xe Roulette**. Tổng fitness của quần thể = $11 + 9 + 10 + 0 = 30$.

- Xác suất chọn A: $11/30 \approx 36.7\%$
- Xác suất chọn B: $9/30 = 30.0\%$
- Xác suất chọn C: $10/30 \approx 33.3\%$
- Xác suất chọn D: $0/30 = 0\%$ (Sẽ không được chọn)

Giả sử sau 4 lần “quay bánh xe”, chúng ta chọn được 2 cặp cha mẹ: **(A, C)** và **(A, B)**.

d. Lai ghép (Crossover)

Thực hiện lai ghép một điểm (one-point crossover) cho từng cặp.

- **Cặp 1: (A, C)** với điểm cắt sau vị trí 2:
 - Cha A: 10 | 10
 - Cha C: 11 | 00
 - **Con 1:** 10 + 00 = 1000
 - **Con 2:** 11 + 10 = 1110
- **Cặp 2: (A, B)** với điểm cắt sau vị trí 3:

- Cha A: $101 \mid 0$
- Cha B: $011 \mid 0$
- **Con 3:** $101 + 0 = 1010$
- **Con 4:** $011 + 0 = 0110$

e. Đột biến (Mutation)

Giả sử Con 2 (1110) bị đột biến ngẫu nhiên tại bit thứ 4.

- Trước đột biến: 1110
- Sau đột biến: 1111 (bit thứ 4 lật từ 0 thành 1)

f. Thế hệ Mới và Đánh giá lại

Quần thể mới bây giờ bao gồm 4 cá thể con vừa được tạo ra.

Cá thể con	Nhiễm sắc thể	Tổng Giá trị (V)	Tổng Khối lượng (W_{total})	Fitness
Con 1	1000	6	$4 (\leq 9)$	6
Con 2 (đột biến)	1111	$6+4+5+7 = \mathbf{22}$	$4+3+5+6 = 18 (> 9)$	0
Con 3	1010	$6 + 5 = \mathbf{11}$	$4 + 5 = 9 (\leq 9)$	11
Con 4	0110	$4 + 5 = \mathbf{9}$	$3 + 5 = 8 (\leq 9)$	9

Tổng kết một Thế hệ

Qua một thế hệ duy nhất, chúng ta thấy:

- Cá thể D (vi phạm ràng buộc) đã bị **loại bỏ**.
- Các đặc tính tốt từ A, B, C đã được **kế thừa và tái tổ hợp** để tạo ra thế hệ mới.
- Một đột biến đã tạo ra một lời giải hoàn toàn mới (1111), mặc dù trong trường hợp này nó không hợp lệ, nhưng nó cho thấy khả năng **khám phá** của thuật toán.

Khi quá trình này được lặp lại qua hàng trăm hoặc hàng nghìn thế hệ, các tổ hợp gen tốt sẽ dần dần chiếm ưu thế và hội tụ về một lời giải có tổng giá trị cao mà vẫn thỏa mãn sức chứa của ba lô.

2.10.5. Ví dụ minh họa chi tiết: Giải bài toán Ba lô (0/1 Knapsack) bằng thuật toán di truyền

Để giúp sinh viên hiểu rõ cơ chế vận hành của thuật toán di truyền trong thực tế, chúng ta cùng triển khai một ví dụ điển hình: **bài toán ba lô nhị phân (0/1 Knapsack Problem)**. Đây là một bài toán tối ưu tổ hợp cổ điển trong đó người giải cần lựa chọn một tập hợp con các đồ vật sao cho tổng giá trị là lớn nhất, nhưng không vượt quá sức chứa của ba lô.

Mô hình hóa bài toán

Mô tả bài toán:

Cho một tập hợp gồm N đồ vật, mỗi đồ vật i có:

- **Giá trị (value)** v_i
- **Khối lượng (weight)** w_i

Ba lô có sức chứa tối đa C .

Mục tiêu: Tìm một tập con các đồ vật sao cho:

- Tổng khối lượng $\sum w_i \leq C$
- Tổng giá trị $\sum v_i$ là lớn nhất có thể.

Ánh xạ sang thuật toán di truyền

1. Biểu diễn cá thể (Nhiễm sắc thể):

- Mỗi cá thể được biểu diễn bằng một **chuỗi bit độ dài N** .
- Bit thứ i :
 - 1: chọn đồ vật thứ i
 - 0: không chọn đồ vật thứ i

Ví dụ: “1010” nghĩa là chọn đồ vật 1 và 3.

2. Hàm thích nghi (Fitness Function):

Fitness của một cá thể được tính như sau:

- Nếu tổng khối lượng không vượt quá C :
$$\text{fitness} = \sum_{i=1}^N v_i \cdot x_i$$
- Nếu vượt quá:
$$\text{fitness} = 0$$

Chi tiết ví dụ: Giải bằng một thế hệ GA

Dữ liệu cho trước:

- Số lượng đồ vật: $N = 4$
- Sức chứa ba lô: $C = 10$

Đồ vật	Giá trị v_i	Khối lượng w_i
1	6	4
2	4	3

Đồ vật	Giá trị v_i	Khối lượng w_i
3	5	5
4	7	6

Bước 1: Khởi tạo quần thể ban đầu (Population size = 4)

Cá thể	Nhiễm sắc thể	Diễn giải
A	1010	Chọn đồ 1 và 3
B	0110	Chọn đồ 2 và 3
C	1100	Chọn đồ 1 và 2
D	1001	Chọn đồ 1 và 4

Bước 2: Đánh giá fitness

Tính tổng giá trị và tổng khối lượng cho mỗi cá thể:

Cá thể	Nhiễm sắc thể	Giá trị	Khối lượng	Fitness
A	1010	$6 + 5 = 11$	$4 + 5 = 9$	11
B	0110	$4 + 5 = 9$	$3 + 5 = 8$	9
C	1100	$6 + 4 = 10$	$4 + 3 = 7$	10
D	1001	$6 + 7 = 13$	$4 + 6 = 10$	13

Bước 3: Chọn lọc (Selection – Roulette Wheel)

Tính xác suất chọn của từng cá thể:

Tổng fitness = $11 + 9 + 10 + 13 = 43$

Cá thể	Fitness	Xác suất chọn
A	11	$11 / 43 \approx 25.6\%$
B	9	$9 / 43 \approx 20.9\%$
C	10	$10 / 43 \approx 23.3\%$
D	13	$13 / 43 \approx 30.2\%$

Giả sử chọn ngẫu nhiên: **A và D** làm cha mẹ.

Bước 4: Lai ghép (Crossover – One-point tại vị trí thứ 2)

- Cha mẹ:

- A: 1 0 1 0
- D: 1 0 0 1
- Lai ghép:
 - Trước điểm cắt: giữ nguyên từ cha.
 - Sau điểm cắt: lấy từ mẹ.

Con	Kết quả lai ghép	Diễn giải
1	1 0 0 1	Chọn đồ 1 và 4
2	1 0 1 0	Chọn đồ 1 và 3

Bước 5: Đột biến (Mutation – xác suất 10%)

Áp dụng đột biến trên Con 2 (giả sử bit thứ 3 – từ 1 thành 0):

Trước đột biến	Sau đột biến	Diễn giải
1 0 1 0	1 0 0 0	Chỉ chọn đồ 1

Bước 6: Đánh giá thế hệ con

Cá thể	Nhiễm sắc thể	Giá trị	Khối lượng	Fitness
Con 1	1001	$6 + 7 = 13$	$4 + 6 = 10$	13
Con 2	1000	6	4	6

Nhận xét và kết luận

- Sau một vòng lặp GA, cá thể tốt nhất (fitness = 13) được bảo toàn hoặc tái tạo.
- Các thao tác như chọn lọc, lai ghép và đột biến đều đóng vai trò quan trọng:
 - **Chọn lọc:** Ưu tiên lời giải tốt.
 - **Lai ghép:** Tái tổ hợp để tạo biến thể mới.
 - **Đột biến:** Bảo toàn tính đa dạng và tránh bẫy cực trị cục bộ.

Qua ví dụ trên, chúng ta thấy rõ cách thuật toán di truyền mô phỏng tiến hóa để tìm kiếm lời giải tốt cho bài toán tối ưu hóa. Cấu trúc đơn giản, nhưng khả năng kết hợp và biến đổi thông minh khiến GA trở thành một công cụ mạnh mẽ trong việc giải các bài toán không gian lớn, phi tuyến và không khả vi.

CHƯƠNG 3: BIỂU DIỄN VÀ XỬ LÝ TRI THỨC

3.1. Giới thiệu về Tri thức và Lập luận

3.1.1. Vượt qua giới hạn của Tìm kiếm

Trong các chương trước, chúng ta đã khám phá sức mạnh của các thuật toán tìm kiếm (tìm kiếm mù, heuristic, đối kháng, di truyền). Các phương pháp này giải quyết bài toán bằng cách duyệt qua một không gian các **trạng thái (states)**. Tuy nhiên, đối với các thuật toán này, mỗi trạng thái chỉ là một **“hộp đen” (black box)**—một thực thể dữ liệu không có cấu trúc nội tại và không có ý nghĩa rõ ràng.

Hạn chế của Mô hình “Trạng thái Hộp đen”

Hãy xét lại bài toán 8-puzzle. Một thuật toán tìm kiếm như A^* chỉ biết rằng:

- Trạng thái A là một cấu hình các ô số.
- Trạng thái A khác với trạng thái B .
- Có một hành động hợp lệ (ví dụ: “di chuyển ô trống lên trên”) để chuyển từ A sang B .

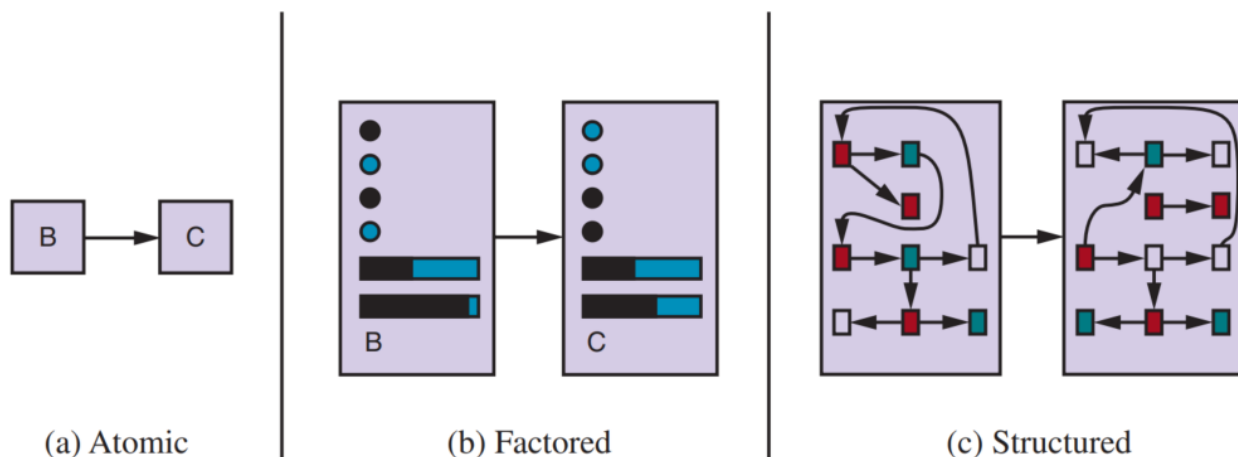
Thuật toán không hề “hiểu” rằng:

- Đây là một bàn cờ.
- Các ô được đánh số có ý nghĩa toán học.
- Mục tiêu là sắp xếp chúng theo một thứ tự cụ thể.

Tác tử chỉ đơn thuần thực thi các quy tắc một cách máy móc để tìm đường đi từ điểm đầu đến điểm cuối trong một không gian trừu tượng. Sự thiếu hiểu biết này là một rào cản lớn để xây dựng các hệ thống thực sự thông minh.

Các Cấp độ Biểu diễn Trạng thái: Một Nấc thang của Sự Tinh vi

Để một tác tử có thể “hiểu” thế giới, nó cần một cách biểu diễn thông tin phong phú hơn là một “hộp đen”. Chúng ta có thể phân loại các cách biểu diễn này theo một trục tăng dần về độ phức tạp và sức mạnh biểu đạt.



Hình 79. Ba cách biểu diễn trạng thái và chuyển trạng thái.

Hình 79: Ba cách biểu diễn trạng thái và chuyển trạng thái: (a) Nguyên tử (Atomic), (b) Phân rã (Factored), và (c) Có cấu trúc (Structured).

- **Cấp độ 1: Biểu diễn Nguyên tử (Atomic Representation)**

- **Mô tả:** Mỗi trạng thái là một thực thể đơn nhất, không thể chia cắt, không có cấu trúc bên trong. Nó chỉ có thể được so sánh bằng phép toán “bằng” hoặc “không bằng”.
- **Ví dụ:** Trong bài toán tìm đường đi, mỗi trạng thái chỉ là tên của một thành phố (“Hà Nội”, “Đà Nẵng”). Chúng ta chỉ biết mình đang ở đâu, không biết gì thêm.
- **Ứng dụng:** Đây là nền tảng cho các thuật toán tìm kiếm trạng thái (Chương 2), mô hình Markov ẩn, và quy hoạch động cơ bản.

- **Cấp độ 2: Biểu diễn Phân rã (Factored Representation)**

- **Mô tả:** Mỗi trạng thái được chia nhỏ thành một tập hợp các thuộc tính hoặc biến (variables/attributes) có giá trị cụ thể. Nó giống như một vector các đặc trưng.
- **Ví dụ:** Thay vì chỉ biết “đang ở Hà Nội”, trạng thái bây giờ có thể là một tập hợp các giá trị: $\{Vị\ tr\acute{i}: Hà\ N\grave{o}i, L\grave{u}ợng\ x\ăng: 15\ lít, Th\ời\ ti\ế\ t: N\ặng, Ng\ân\ s\á\ch: 2.000.000\ VNĐ\}$. Hai trạng thái khác nhau giờ đây có thể có những điểm chung (cùng vị trí) và khác biệt (lượng xăng khác nhau).
- **Ứng dụng:** Nền tảng cho các bài toán thỏa mãn ràng buộc (CSP), logic mệnh đề, lập kế hoạch, và mạng Bayes.

- **Cấp độ 3: Biểu diễn Có cấu trúc (Structured Representation)**

- **Mô tả:** Đây là cấp độ biểu diễn cao nhất. Trạng thái không chỉ là một tập các thuộc tính, mà là một tập hợp các **đối tượng (objects)** và các **mối quan hệ (relationships)** phức tạp giữa chúng.
- **Ví dụ:** Thay vì có một biến $XeT\grave{a}iB\grave{i}B\grave{o}Ch\grave{ậ}nĐ\grave{u}ờng = true$, một biểu diễn có cấu trúc sẽ mô tả:
 - Có một đối tượng X thuộc lớp $XeT\grave{a}i$.
 - Có một đối tượng Y thuộc lớp $B\grave{o}$.
 - Tồn tại một mối quan hệ $đang_ch\grave{ậ}n(Y, X)$.

- **Ứng dụng:** Nền tảng cho logic vị từ bậc nhất, cơ sở dữ liệu quan hệ, và đặc biệt là xử lý ngôn ngữ tự nhiên (vì ngôn ngữ của con người vốn dĩ mô tả các đối tượng và mối quan hệ giữa chúng).

Sức mạnh Biểu đạt (Expressiveness) và Sự Đánh đổi

Trục phân loại trên cũng chính là trục đo **sức mạnh biểu đạt**. Một ngôn ngữ biểu diễn có sức mạnh biểu đạt cao hơn có thể mô tả thông tin phức tạp một cách ngắn gọn và chính xác hơn rất nhiều.

- **Ví dụ:** Để mô tả luật chơi cờ vua:
 - Dùng **logic vị từ** (có cấu trúc): cần vài trang giấy.
 - Dùng **logic mệnh đề** (phân rã): cần hàng nghìn trang.
 - Dùng **mô hình trạng thái hữu hạn** (nguyên tử): cần khoảng 10^{38} trang!

Tuy nhiên, **sức mạnh biểu đạt càng cao, quá trình suy luận và học hỏi trên đó càng phức tạp và tốn kém về mặt tính toán.**

Sự cần thiết của Lập luận Logic (Reasoning)

Để một tác tử thực sự “thông minh”, nó không chỉ cần một cách biểu diễn tri thức phong phú, mà còn cần một “động cơ” để xử lý tri thức đó. Động cơ này chính là **lập luận (reasoning)**.

- **Biểu diễn tri thức (Knowledge Representation - KR):** Là quá trình mô hình hóa thông tin về thế giới dưới dạng có cấu trúc mà máy tính có thể xử lý.
- **Lập luận (Reasoning):** Là quá trình **suy ra tri thức mới** từ những tri thức đã biết một cách logic.

Trong khi tìm kiếm chỉ *tìm ra một đường đi* qua các trạng thái đã biết, lập luận có thể *tạo ra sự thật mới*.

- **Ví dụ kinh điển:**
 6. **Tri thức có sẵn 1:** Mọi người đều sẽ chết. ($\forall x, \text{Người}(x) \Rightarrow \text{SẽChết}(x)$)
 7. **Tri thức có sẵn 2:** Socrates là người. ($\text{Người}(\text{Socrates})$)
 8. **Lập luận suy ra tri thức mới:** Socrates sẽ chết. ($\text{SẽChết}(\text{Socrates})$)

Một tác tử có khả năng lập luận sẽ không chỉ biết *Trạng thái A $\rightarrow$ Trạng thái B*, mà còn có thể hiểu *tại sao*, dự đoán các hệ quả, và thích nghi với các tình huống chưa từng gặp phải.

Kết luận: Bước nhảy từ “Tác tử Hành động” sang “Tác tử Hiểu biết”

Các thuật toán tìm kiếm mạnh mẽ nhưng chúng tạo ra các “tác tử hành động” (acting agents) chỉ biết làm theo chỉ dẫn. Để xây dựng các “tác tử hiểu biết” (knowing agents) có khả năng giải thích, lập kế hoạch linh hoạt, và tương tác một cách thông minh với thế giới, chúng ta phải

vượt qua giới hạn của trạng thái “hộp đen”. Chương này sẽ đi sâu vào việc trang bị cho tác tử hai năng lực cốt lõi của trí tuệ: **biểu diễn tri thức** và **suy diễn logic**.

3.1.2. Cơ sở tri thức (Knowledge Base – KB)

Sau khi nhận thấy sự cần thiết phải vượt qua giới hạn của tìm kiếm, chúng ta tiến đến khái niệm cốt lõi cho phép một tác tử “hiểu” được thế giới: **Cơ sở Tri thức (Knowledge Base – KB)**. Đây không chỉ là một kho lưu trữ dữ liệu, mà là trung tâm của quá trình tư duy và lập luận của một tác tử thông minh.

Định nghĩa: KB là gì?

Cơ sở tri thức (KB) là một tập hợp các **câu (sentences)** được biểu diễn bằng một **ngôn ngữ hình thức (formal language)**.

- **Câu (Sentence):** Mỗi câu là một khẳng định (assertion) về một khía cạnh nào đó của thế giới. Nó có thể là một sự thật hiển nhiên, một quy luật, hoặc một mối quan hệ. Những câu được cung cấp ban đầu mà không cần chứng minh được gọi là **tiên đề (axioms)**.
- **Ngôn ngữ Hình thức:** Việc sử dụng một ngôn ngữ có cú pháp và ngữ nghĩa chặt chẽ (như Logic Mệnh đề, Logic Vị từ) là bắt buộc để loại bỏ sự mơ hồ của ngôn ngữ tự nhiên, cho phép máy tính có thể suy luận một cách chính xác.

Về bản chất, KB đóng vai trò như “**bộ não**” hay “**trí nhớ dài hạn**” của tác tử. Nó lưu giữ tất cả những gì tác tử “biết” và là nền tảng để từ đó suy ra những điều mới.

Tương tác với Cơ sở Tri thức: Hai Hành động Cốt lõi

Để KB trở nên hữu dụng, tác tử phải có khả năng tương tác với nó. Có hai hành động cơ bản, được đặt tên theo quy ước là *TELL* và *ASK*.

- **TELL: Học hỏi và Cập nhật Tri thức**
 - **Chức năng:** *TELL(kb, sentence)*
 - **Ý nghĩa:** Thêm một câu mới vào KB. Đây chính là quá trình **học hỏi** của tác tử. Khi nó nhận được một thông tin mới từ môi trường (qua cảm biến), nó sẽ “kể” cho KB của mình biết, qua đó làm giàu thêm vốn hiểu biết.
 - **Ví dụ:** Ban đầu, KB của một robot chỉ biết “Năng lượng của tôi thấp”. Sau khi quan sát, nó *TELL* thêm một câu mới: “Có một trạm sạc ở phòng A”.
- **ASK: Tư duy và Truy vấn Tri thức**
 - **Chức năng:** *ASK(kb, query)*
 - **Ý nghĩa:** Truy vấn KB để kiểm tra xem một câu (query) có phải là hệ quả logic từ những gì KB đã biết hay không. Đây là quá trình **tư duy** hoặc **suy ngẫm** của tác tử để ra quyết định. Nó không chỉ là một thao tác tìm kiếm đơn thuần trong cơ sở dữ liệu.
 - **Ví dụ:** Robot *ASK* KB của mình: “Tôi có nên đi đến phòng A không?”. Để trả lời, KB phải sử dụng suy luận.

Động cơ Suy luận (Inference Engine): Từ “Biết” đến “Suy ra”

Quá trình trả lời câu hỏi của *ASK* được thực hiện bởi một **động cơ suy luận (inference engine)**. **Suy luận (Inference)** là quá trình rút ra các câu mới (kết luận) từ những câu đã có trong KB (tiền đề).

Một câu α được gọi là **hệ quả logic (logical consequence)** của KB, ký hiệu là $KB \models \alpha$, nếu và chỉ nếu trong mọi thế giới khả dĩ mà các câu trong KB đều đúng, thì câu α cũng phải đúng.

Điều này đảm bảo rằng suy luận của tác tử là **sound** (chặt chẽ), nó không “bịa” ra thông tin mới mà chỉ rút ra những sự thật chắc chắn được ẩn chứa bên trong những gì nó đã biết.

- **Ví dụ:**

- KB chứa 2 câu:
 1. (Tiền đề) *Nếu trời mưa thì đường sẽ trơn.*
 2. (Quan sát) *Trời đang mưa.*
- Khi *ASK*(KB, “*Đường có trơn không?*”), động cơ suy luận sẽ áp dụng quy tắc Modus Ponens để suy ra câu trả lời là “Đúng”, vì “Đường trơn” là một hệ quả logic ($KB \models \text{Đường trơn}$).

Kiến trúc của một Tác tử Dựa trên Tri thức (Knowledge-Based Agent)

Một tác tử dựa trên tri thức được xây dựng xoay quanh KB và động cơ suy luận.

```
function KB-AGENT(percept) returns an action  
persistent: KB, a knowledge base  
              t, a counter, initially 0, indicating time  
  
  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))  
  action  $\leftarrow$  ASK(KB, MAKE-ACTION-QUERY(t))  
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))  
  t  $\leftarrow$  t + 1  
  return action
```

Hình 80. Tác nhân dựa trên tri thức tổng quát.

Chu trình hoạt động của nó như sau:

- **Cảm nhận (Perceive):** Tác tử nhận thông tin từ môi trường qua cảm biến (*percept*).
- **Cập nhật Trạng thái Thế giới:** Nó tạo một câu biểu diễn tri thức từ thông tin cảm nhận được (*MAKE-PERCEPT-SENTENCE*).
- **Học hỏi (Learn):** Nó **TELL** câu mới này cho KB, cập nhật hiểu biết của mình về thế giới.
- **Tư duy (Reason):** Nó **ASK** KB xem nên thực hiện hành động nào tiếp theo (*MAKE-ACTION-QUERY*). Động cơ suy luận sẽ tìm ra hành động tốt nhất dựa trên tri thức hiện có và mục tiêu của tác tử.

- **Hành động (Act):** Tác tử thực hiện hành động đã được chọn và **TELL** cho KB biết rằng nó đã thực hiện hành động đó (**MAKE-ACTION-SENTENCE**).

Các Cấp độ Trừu tượng và Phương pháp Xây dựng

- **Cấp độ Tri thức vs. Cấp độ Thực thi (Knowledge vs. Implementation Level)** Một ưu điểm lớn của cách tiếp cận này là nó cho phép chúng ta phân tích tác tử ở **cấp độ tri thức**—tức là mô tả *cái gì* tác tử biết và mục tiêu của nó là gì—mà không cần quan tâm đến **cấp độ thực thi**—tức là tri thức đó được lưu trữ và xử lý *như thế nào* (dưới dạng danh sách liên kết, mảng, hay mạng nơ-ron). Điều này giúp việc thiết kế và phân tích trở nên rõ ràng và độc lập với công nghệ.
- **Phương pháp Khai báo vs. Phương pháp Thủ tục (Declarative vs. Procedural)** Xây dựng một KB là một phương pháp **khai báo (declarative)**. Chúng ta chỉ cần “khai báo” cho tác tử những sự thật về thế giới. Điều này linh hoạt và dễ mở rộng hơn nhiều so với phương pháp **thủ tục (procedural)**, nơi hành vi của tác tử được mã hóa cứng thành các dòng lệnh *if-then-else* phức tạp.

Cơ sở tri thức (KB), được vận hành bởi một **động cơ suy luận**, chính là thành phần nâng một tác tử từ cấp độ “phản xạ” lên cấp độ “tư duy”. Bằng cách cho phép tác tử lưu trữ, cập nhật và suy luận từ những gì nó biết, KB tạo ra nền tảng cho các hành vi thông minh, có mục đích và có khả năng thích ứng, mở đường cho việc xây dựng các hệ thống trí tuệ nhân tạo thực sự mạnh mẽ.

3.1.3. Vòng lặp của Tác tử dựa trên Tri thức (Knowledge-Based Agent Loop)

Một trong những mục tiêu cốt lõi của trí tuệ nhân tạo là xây dựng được các **tác tử thông minh** – các hệ thống có khả năng cảm nhận môi trường, suy luận, ra quyết định, và thích nghi theo thời gian. Khi tích hợp **cơ sở tri thức (Knowledge Base – KB)** vào kiến trúc tác tử, ta thu được mô hình gọi là **tác tử dựa trên tri thức (knowledge-based agent)**.

Mô hình này mở rộng năng lực của tác tử từ việc chỉ phản ứng theo quy tắc có sẵn sang khả năng **tư duy logic, học hỏi và giải thích hành vi của chính nó**, nhờ vào khả năng tương tác linh hoạt với cơ sở tri thức.

Cấu trúc vòng lặp của tác tử dựa trên tri thức

Hoạt động của tác tử tri thức diễn ra liên tục theo một **chu trình vòng lặp** gồm các bước sau:

Bước 1: Cảm nhận môi trường (Perceive)

- Tác tử thu thập dữ liệu từ **các cảm biến** (sensors) về trạng thái hiện tại của môi trường.
- Thông tin này được biểu diễn dưới dạng các **mệnh đề hoặc sự kiện**, sẵn sàng để xử lý trong ngôn ngữ hình thức.

Ví dụ: Tác tử robot nhận được tín hiệu “Có chướng ngại vật phía trước”.

Bước 2: TELL – Cập nhật tri thức cảm nhận vào KB

- Sử dụng lệnh:

TELL(KB,percept)

- Hành động này ghi nhận thông tin mới vừa cảm nhận vào cơ sở tri thức.

Ví dụ:

TELL(KB,ObstacleAhead)

Điều này giúp KB luôn **phản ánh đúng trạng thái hiện tại của thế giới**, duy trì độ cập nhật và độ chính xác trong suy luận.

Bước 3: ASK – Truy vấn KB để ra quyết định

- Dựa trên tri thức hiện tại, tác tử đặt câu hỏi:

ASK(KB,What should I do now?)

- Câu hỏi này được thể hiện bằng một **công thức logic**, và hệ thống suy diễn (inference engine) sẽ trả lời dựa trên các mệnh đề trong KB.

Ví dụ: Với các tri thức trong KB như:

- “Nếu có chướng ngại vật thì rẽ phải”
 - “ObstacleAhead”
- thì suy diễn sẽ dẫn tới hành động:

Action ← TurnRight

Bước 4: Thực hiện hành động (Act)

- Tác tử thực hiện hành động được đề xuất, tương tác trở lại với môi trường thông qua **cơ cấu chấp hành (actuators)**.

Ví dụ: Thực hiện lệnh: “TurnRight”.

Bước 5: TELL – Ghi lại hành động đã thực hiện

- Sau khi hành động, tác tử **ghi nhận hành động đó vào KB**:

TELL(KB,ActionTaken)

- Việc này cho phép tác tử:
 - Ghi nhớ các hành động đã thực hiện
 - Phân tích hậu quả của hành động
 - Sử dụng làm dữ liệu huấn luyện hoặc kiểm tra giả thuyết trong tương lai

Ví dụ:

TELL(KB,Did(TurnRight))

Tổng quát hóa vòng lặp

Giai đoạn	Hành động	Mục đích chính
1. Cảm nhận	Nhận thông tin từ cảm biến	Biết trạng thái hiện tại
2. TELL	Cập nhật tri thức vào KB	Cập nhật hiểu biết về môi trường

Giai đoạn	Hành động	Mục đích chính
3. ASK	Suy luận hành động nên thực hiện	Ra quyết định logic
4. Hành động	Thực hiện hành động được chọn	Tác động lên môi trường
5. TELL	Ghi lại hành động đã thực hiện vào KB	Duy trì nhật ký hành vi và học tập

Ý nghĩa của vòng lặp này

- Cho phép **tác tử trở nên chủ động**, có khả năng phản ứng không chỉ dựa trên quy tắc cứng, mà còn **suy luận linh hoạt theo tri thức hiện có**.
- Giúp **tái sử dụng và tích lũy tri thức**, nâng cao khả năng thích nghi trong các tình huống phức tạp và chưa gặp phải.
- Là nền tảng để xây dựng các hệ thống AI có **khả năng tự giải thích (explainable AI)** – có thể truy xuất nguồn gốc lý do tại sao chọn một hành động nào đó.

Vòng lặp của tác tử dựa trên tri thức là **cơ chế vận hành trí tuệ** cốt lõi, kết nối cảm nhận – lưu trữ – suy luận – hành động một cách logic và chặt chẽ. Đây là tiền đề để phát triển các tác tử AI thực sự thông minh, học hỏi từ kinh nghiệm và tương tác hiệu quả trong thế giới không hoàn hảo.

3.1.4. Ví dụ xuyên suốt: Thế giới Wumpus (Wumpus World)

Để minh họa cụ thể cách biểu diễn tri thức và suy luận logic vận hành trong môi trường trí tuệ nhân tạo, chúng ta sử dụng một ví dụ kinh điển trong sách giáo khoa AI: **Thế giới Wumpus (Wumpus World)**. Đây là một môi trường giả lập đơn giản nhưng đủ phức tạp để thể hiện rõ giá trị của tri thức và lập luận logic trong hành vi của tác tử.

Mô tả tổng quan về Thế giới Wumpus

Thế giới Wumpus được mô hình hóa như một **lưới ô vuông hai chiều**, thông thường là 4×4 , trong đó mỗi ô có thể chứa:

- **Hố sâu (Pit):** Nếu tác tử rơi vào ô này, nó chết ngay lập tức.
- **Quái vật Wumpus:** Một sinh vật nguy hiểm. Nếu tác tử bước vào ô có Wumpus khi nó còn sống, tác tử cũng chết.
- **Vàng (Gold):** Mục tiêu chính. Tác tử chiến thắng nếu nhặt được vàng và thoát ra an toàn.
- **Không gian trống:** An toàn, không có rủi ro hay phần thưởng.

Tác tử bắt đầu ở ô $[1,1]$ (góc dưới trái), quay mặt về hướng Đông, và **không biết vị trí chính xác của hố, Wumpus hay vàng**.

Khả năng cảm nhận của tác tử

Tác tử không nhìn thấy toàn bộ lưới, mà chỉ nhận được **cảm biến tại ô hiện tại**:

Cảm biến	Ý nghĩa
Stench	Có mùi hôi: Một trong các ô kề có Wumpus
Breeze	Cảm thấy gió: Một trong các ô kề có Hố
Glitter	Lấp lánh: Có vàng trong ô hiện tại
Bump	Va vào tường: bước đi vượt khỏi giới hạn
Scream	Tiếng hét: Wumpus đã bị tiêu diệt

Hành động của tác tử

Tác tử có thể thực hiện các hành động sau:

- **MOVE FORWARD:** Tiến một bước theo hướng đang đối mặt.
- **TURN LEFT/RIGHT:** Quay 90 độ.
- **GRAB:** Nhặt vàng nếu có ở ô hiện tại.
- **SHOOT:** Bắn mũi tên về phía trước (chỉ có 1 mũi tên).
- **CLIMB:** Thoát khỏi lưới khi đứng tại ô [1,1].

Tri thức và suy luận trong Thế giới Wumpus

Điều khiến Wumpus World trở thành ví dụ kinh điển là: **không thể giải bài toán này chỉ bằng tìm kiếm**, mà cần khả năng **suy luận logic** để đưa ra quyết định an toàn. Hãy xét một ví dụ:

Tình huống:

- Tác tử bắt đầu tại ô [1,1].
- Tại ô [1,1], không có Breeze (không gió) và không có Stench (không mùi).
- Tác tử đang xem xét khả năng đi đến các ô kề [1,2] và [2,1].

Suy luận logic:

- Theo định nghĩa: **nếu một ô có Breeze thì ít nhất một ô kề chứa Hố.**
- Ngược lại: **nếu không có Breeze, thì tất cả các ô kề đều không chứa Hố.**

Vì vậy, từ việc không cảm nhận được gió tại [1,1], tác tử **có thể suy ra một cách hợp lý và chắc chắn rằng:**

$$\neg \text{Pit}(1,2) \quad \text{và} \quad \neg \text{Pit}(2,1)$$

Tác tử do đó **biết được rằng các ô [1,2] và [2,1] là an toàn**, và có thể ưu tiên khám phá hai ô này mà không sợ chết vì rơi xuống hố.

So sánh với tác tử tìm kiếm thông thường

Một thuật toán tìm kiếm như BFS hay A^* sẽ cần biểu diễn rõ ràng toàn bộ không gian trạng thái và thường **không thể đưa ra kết luận từ các điều kiện phủ định** (như “không có Breeze” $\Rightarrow$ “các ô kề không có Hố”), trừ khi được lập trình cụ thể.

Ngược lại, một **tác tử dựa trên tri thức** với hệ thống suy diễn logic có thể **tự động rút ra các hệ quả từ các điều kiện quan sát được**, mà không cần liệt kê trước toàn bộ tình huống.

Ý nghĩa giáo dục của ví dụ Wumpus

- Đây là môi trường đơn giản nhưng thể hiện rõ:
 - Tầm quan trọng của **tri thức âm tính (negative knowledge)**.
 - Cách **một hệ thống biểu diễn logic** có thể khai thác điều kiện cảm biến để rút ra kết luận mới.
 - Khả năng **ra quyết định an toàn dựa trên suy luận**, chứ không chỉ dựa vào trải nghiệm hoặc thử-sai.

Thế giới Wumpus là một mô hình kinh điển minh họa sự cần thiết và hiệu quả của việc trang bị cho tác tử khả năng **biểu diễn tri thức và suy luận logic**. Qua ví dụ này, sinh viên có thể thấy rõ sự khác biệt giữa tác tử “tìm kiếm” và tác tử “hiểu biết”, mở đầu cho các chương sâu hơn về Logic Mệnh đề và Logic Vị từ.

3.2. Logic Mệnh đề (Propositional Logic)

3.2.1. Giới thiệu

Logic mệnh đề (Propositional Logic), còn gọi là **Logic vị ngữ bậc 0**, là **hình thức logic hình thức đơn giản nhất** được sử dụng trong trí tuệ nhân tạo để biểu diễn tri thức và thực hiện suy luận. Đây là điểm khởi đầu tự nhiên cho việc nghiên cứu các hệ thống suy luận, vì nó dễ tiếp cận, cấu trúc rõ ràng và có tính nền tảng vững chắc về mặt toán học.

Mệnh đề (Proposition):

Một **mệnh đề** là **một câu phát biểu** có thể mang **giá trị chân lý xác định**, tức là **hoặc đúng (True), hoặc sai (False)** – không có trạng thái thứ ba.

Ví dụ:

- “Trời đang mưa” – là một mệnh đề, vì nó có thể đúng hoặc sai.
- “Hôm nay là thứ Hai” – cũng là một mệnh đề.
- “Ai là người giỏi nhất?” – **không** phải mệnh đề, vì không có giá trị chân lý xác định.

Trong logic mệnh đề, các mệnh đề cơ bản thường được biểu diễn bằng **các ký hiệu chữ cái**: $P, Q, R, A, B, \dots$

Tính chất của logic mệnh đề:

- **Ngôn ngữ không có cấu trúc nội tại của mệnh đề:** Logic mệnh đề xem mỗi mệnh đề là **nguyên tử**, không phân tích sâu hơn nội dung của nó.
 - Ví dụ: “Trời mưa ở Hà Nội” được biểu diễn là P , không xét đến chủ thể “Trời”, hành động “mưa” hay vị trí “Hà Nội”.
- Do đó, logic mệnh đề **không thể biểu diễn các cấu trúc phức tạp như:**
 - Đối tượng, thuộc tính, quan hệ giữa các đối tượng.
 - Đại lượng tổng quát (ví dụ: “mọi người đều phải chết” – cần logic vị từ).

Tuy vậy, chính sự đơn giản đó lại làm cho logic mệnh đề:

- **Dễ cài đặt và tự động hóa.**
- **Hiệu quả cho các hệ thống suy luận nhỏ, rời rạc.**
- **Là bước đệm cần thiết để học các hệ thống logic phức tạp hơn.**

Mục tiêu khi học logic mệnh đề trong AI:

- Biết cách **biểu diễn tri thức thực tế** bằng các mệnh đề và công thức logic.
- Nắm được các **toán tử logic cơ bản** và **cách kết hợp mệnh đề** để mô tả các tình huống phức tạp hơn.
- Thực hiện được các **phép suy diễn logic**: chứng minh, kiểm tra mâu thuẫn, suy luận theo luật.

Logic mệnh đề là công cụ đầu tiên để **biến tri thức ngôn ngữ tự nhiên thành ngôn ngữ hình thức**, tạo cơ sở cho suy luận máy tính. Mặc dù đơn giản, nó cung cấp nền tảng cho các hệ thống biểu diễn tri thức trong AI và sẽ được mở rộng trong các chương tiếp theo với Logic Vị từ (First-Order Logic).

3.2.1. Giới thiệu Logic Mệnh đề (Propositional Logic)

Logic Mệnh đề (LMĐ), hay Lô-gíc Vị từ, là ngôn ngữ hình thức cơ bản nhất trong lĩnh vực biểu diễn tri thức. Nó cung cấp một bộ khung xương vững chắc để mô tả thế giới thông qua các khẳng định chỉ có thể **đúng (True)** hoặc **sai (False)**, không tồn tại trạng thái trung gian như “có thể” hay “một phần đúng”. Đây là bước đầu tiên để máy tính có thể “lập luận” một cách chính xác.

1. Tại sao cần một Ngôn ngữ Hình thức như Logic Mệnh đề?

Ngôn ngữ tự nhiên của con người (như tiếng Việt, tiếng Anh) rất phong phú nhưng cũng đầy mơ hồ. Xét câu: *“Tôi thấy một người đàn ông trên ngọn đồi bằng ống nhòm.”* Câu này có thể hiểu theo nhiều cách: ai đang cầm ống nhòm, tôi hay người đàn ông? Máy tính cần sự chính xác tuyệt đối. LMĐ ra đời để giải quyết vấn đề này.

- **Nền tảng cho Lập luận Hình thức:** LMĐ cung cấp các quy tắc chặt chẽ, không mơ hồ để biểu diễn các sự thật và suy ra các sự thật mới một cách chắc chắn. Nó là bảng chữ cái và ngữ pháp cho tư duy logic của máy tính.
- **Khả năng Tự động hóa Suy diễn:** Với cú pháp và ngữ nghĩa rõ ràng, chúng ta có thể xây dựng các thuật toán suy diễn (như kiểm tra mô hình, giải giải) để tự động kiểm tra tính đúng đắn của một lập luận hoặc tìm ra các hệ quả logic.
- **Ứng dụng Thực tiễn:** LMĐ là nền tảng để mô hình hóa các hệ thống logic. Ví dụ kinh điển là “thế giới Wumpus”, nơi các quy tắc như “một ô có gió khi và chỉ khi một trong các ô kề bên có hố” được biểu diễn chính xác bằng một mệnh đề tương đương: $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$.

2. Cú pháp (Syntax): Các quy tắc xây dựng câu

Cú pháp định nghĩa các câu hợp lệ trong LMĐ. Nó giống như ngữ pháp của một ngôn ngữ, quy định cách các từ được kết hợp với nhau.

- **Mệnh đề Nguyên tử (Atomic Proposition):**
 - Đây là những đơn vị cơ bản nhất, không thể chia nhỏ hơn. Chúng là các khẳng định đơn giản về thế giới.
 - Chúng ta sử dụng các ký hiệu như $P, Q, R, \dots$ hoặc các tên có ý nghĩa như $Wumpus_Alive, Pit_1_2$ để biểu diễn chúng.
 - Ví dụ: $W_{1,3}$ là một mệnh đề nguyên tử có thể mang ý nghĩa “Có Wumpus ở ô (1,3)”.
- **Mệnh đề Phức hợp (Complex Proposition):**
 - Được xây dựng bằng cách kết hợp các mệnh đề nguyên tử (hoặc mệnh đề phức hợp khác) bằng các **toán tử logic (logical connectives)**.
- **Ngữ pháp Chính tắc (BNF Grammar):** Cấu trúc của một câu hợp lệ trong LMĐ có thể được định nghĩa bằng ngữ pháp sau:
 - $Sentence \rightarrow AtomicSentence \mid ComplexSentence$
 - $AtomicSentence \rightarrow True \mid False \mid P \mid Q \mid \dots$
 - $ComplexSentence \rightarrow \neg Sentence \mid (Sentence \wedge Sentence) \mid (Sentence \vee Sentence) \mid (Sentence \Rightarrow Sentence) \mid (Sentence \Leftrightarrow Sentence)$
- **Thứ tự ưu tiên của Toán tử:** Để giảm việc sử dụng dấu ngoặc, LMĐ quy định thứ tự ưu tiên từ cao đến thấp: $\neg$ (**phủ định**), $\wedge$ (**hội**), $\vee$ (**tuyển**), $\Rightarrow$ (**kéo theo**), $\Leftrightarrow$ (**tương đương**).

3. Ngữ nghĩa (Semantics): Gán ý nghĩa cho câu

Ngữ nghĩa định nghĩa “ý nghĩa” của các câu, tức là quy tắc để xác định một câu là True hay False.

- **Mô hình (Model):**
 - Một mô hình là một sự gán giá trị chân trị (True hoặc False) cho **tất cả** các ký hiệu mệnh đề trong bài toán.

- Nó đại diện cho một “**thế giới khả dĩ**” (**possible world**). Nếu có 3 ký hiệu mệnh đề (P, Q, R), sẽ có $2^3 = 8$ mô hình (thế giới) khác nhau.
- **Bảng Chân trị (Truth Tables) của các Toán tử Logic**: Ngữ nghĩa của các toán tử logic được định nghĩa hoàn hảo thông qua bảng chân trị. **Hình 81**: Bảng chân trị của năm phép nối logic.
 - **Phủ định ($\neg$ NOT)**: Lật ngược giá trị chân trị. $\neg \text{True}$ là *False*.
 - **Hội ($\wedge$ AND)**: Chỉ đúng khi **cả hai** mệnh đề thành phần đều đúng.
 - **Tuyển ($\vee$ OR)**: Chỉ sai khi **cả hai** mệnh đề thành phần đều sai.
 - **Kéo theo ($\Rightarrow$ IMPLIES)**: Giống như một “lời hứa”. $P \Rightarrow Q$ chỉ sai trong trường hợp duy nhất: lời hứa bị phá vỡ (P đúng nhưng Q lại sai). Trong mọi trường hợp khác, nó đều đúng.
 - **Tương đương ($\Leftrightarrow$ IFF - If and only if)**: Chỉ đúng khi **cả hai** mệnh đề thành phần có cùng giá trị chân trị (cùng True hoặc cùng False).
- **Đánh giá một Câu phức trong một Mô hình cụ thể**: Chúng ta có thể tính giá trị chân trị của bất kỳ câu phức nào bằng cách đệ quy từ trong ra ngoài.
 - **Ví dụ**: Đánh giá câu $S = \neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1})$ trong mô hình $m_1 = \{P_{1,2} = \text{false}, P_{2,2} = \text{false}, P_{3,1} = \text{true}\}$.
 1. **Đánh giá về trong ngoặc**: $(P_{2,2} \vee P_{3,1}) \rightarrow (\text{false} \vee \text{true}) \rightarrow \text{true}$
 2. **Đánh giá phép phủ định**: $\neg P_{1,2} \rightarrow \neg \text{false} \rightarrow \text{true}$
 3. **Đánh giá phép hội**: $\text{true} \wedge \text{true} \rightarrow \text{true}$
 - **Kết luận**: Câu S là **True** trong mô hình m_1 .

Logic Mệnh đề, với **cú pháp** chặt chẽ để xây dựng câu và **ngữ nghĩa** rõ ràng để xác định ý nghĩa, là viên gạch nền tảng cho việc biểu diễn tri thức. Nó cho phép chúng ta biến những khẳng định về thế giới thành các cấu trúc mà máy tính có thể phân tích và suy luận một cách tự động. Mặc dù có những hạn chế về sức mạnh biểu đạt (ví dụ: không thể mô tả các đối tượng hay quan hệ một cách tổng quát), nó là bước khởi đầu thiết yếu để xây dựng các hệ thống AI có khả năng lập luận, làm cơ sở cho các ngôn ngữ logic mạnh mẽ hơn như Logic Vị từ Bậc nhất.

3.2.2. *Cú pháp (Syntax) của Logic Mệnh đề*

Cú pháp trong Logic Mệnh đề (LMĐ) định nghĩa các quy tắc để xây dựng nên những câu hợp lệ. Nó giống như ngữ pháp của một ngôn ngữ, quy định “bảng chữ cái” và cách kết hợp các “từ” để tạo thành các câu có cấu trúc đúng đắn. Việc tuân thủ cú pháp chặt chẽ đảm bảo rằng mọi câu logic đều không mơ hồ và có thể được xử lý bởi máy tính.

1. *Các Thành phần Cơ bản (The Alphabet)*

Bảng chữ cái của LMĐ bao gồm hai loại ký hiệu cơ bản: mệnh đề nguyên tử và các toán tử logic.

a. Mệnh đề Nguyên tử (Atomic Sentences)

Đây là những đơn vị cơ sở, không thể chia nhỏ hơn, đại diện cho những khẳng định đơn giản về thế giới. Chúng được coi là các “danh từ” hoặc các “sự thật cơ bản” của ngôn ngữ logic. Có hai loại mệnh đề nguyên tử:

- **Hằng logic (Logical Constants):**
 - *True* (hoặc T, 1): Đại diện cho một mệnh đề luôn luôn đúng.
 - *False* (hoặc F, 0): Đại diện cho một mệnh đề luôn luôn sai.
- **Ký hiệu Mệnh đề (Propositional Symbols):**
 - Là các ký hiệu đại diện cho các mệnh đề mà giá trị chân trị của chúng (đúng hay sai) phụ thuộc vào “thế giới” (mô hình) mà chúng ta đang xét.
 - Theo quy ước, chúng thường bắt đầu bằng chữ cái in hoa: $P, Q, R, \dots$
 - Trong các ứng dụng thực tế, chúng ta thường dùng các tên có ý nghĩa để tăng tính rõ ràng. Ví dụ trong thế giới Wumpus:
 - $W1,3$ có thể biểu diễn cho khẳng định: “Có Wumpus ở ô (1,3)”.
 - $B1,1$ có thể biểu diễn cho khẳng định: “Tác tử cảm nhận có gió ở ô (1,1)”.

b. Các Toán tử Logic (Logical Connectives)

Đây là các “liên từ” dùng để kết hợp các mệnh đề đơn giản thành các mệnh đề phức hợp, thể hiện các mối quan hệ logic. Năm toán tử chính bao gồm:

- **$\neg$ (Phủ định - Negation):** *NOT*. Đảo ngược giá trị chân trị. Ví dụ: $\neg W1,3$ (Không có Wumpus ở ô (1,3)).
- **$\wedge$ (Hội - Conjunction):** *AND*. Đúng khi tất cả các thành phần đều đúng. Ví dụ: $W1,3 \wedge P3,1$ (Có Wumpus ở (1,3) VÀ có hố ở (3,1)).
- **$\vee$ (Tuyển - Disjunction):** *OR*. Đúng khi có ít nhất một thành phần đúng. Ví dụ: $(W1,3 \wedge P3,1) \vee W2,2$.
- **$\Rightarrow$ (Kéo theo - Implication):** *IMPLIES*. Thể hiện mối quan hệ “nếu... thì...”. Ví dụ: $(W1,3 \wedge P3,1) \Rightarrow \neg W2,2$ (NẾU có Wumpus ở (1,3) và hố ở (3,1) THÌ không có Wumpus ở (2,2)).
- **$\Leftrightarrow$ (Tương đương - Biconditional):** *IF AND ONLY IF*. Đúng khi hai thành phần có cùng giá trị chân trị. Ví dụ: $W1,3 \Leftrightarrow \neg W2,2$.

2. Ngữ pháp Hình thức (Formal Grammar): Quy tắc Kết hợp

Để đảm bảo tính chặt chẽ, cú pháp của LMD được định nghĩa bằng một ngữ pháp đệ quy, thường được biểu diễn dưới dạng **Backus-Naur (BNF)**:

Sentence	$\rightarrow$		AtomicSentence		ComplexSentence
AtomicSentence	$\rightarrow$	True False	P Q R ...		
ComplexSentence	$\rightarrow$	(	Sentence	)	
		$\neg$	Sentence		
			$\wedge$	Sentence	
			$\vee$	Sentence	

	Sentence	$\Rightarrow$	Sentence
	Sentence $\Leftrightarrow$ Sentence		

Ngữ pháp này có thể được diễn giải như sau:

- Một câu hợp lệ có thể là một mệnh đề nguyên tử hoặc một mệnh đề phức hợp.
- Một mệnh đề phức hợp được tạo thành bằng cách áp dụng các toán tử logic vào các câu đã hợp lệ.
- Dấu ngoặc đơn $()$ được sử dụng để nhóm các biểu thức và làm rõ thứ tự thực hiện phép toán.

3. Thứ tự Ưu tiên và Sự Rõ ràng

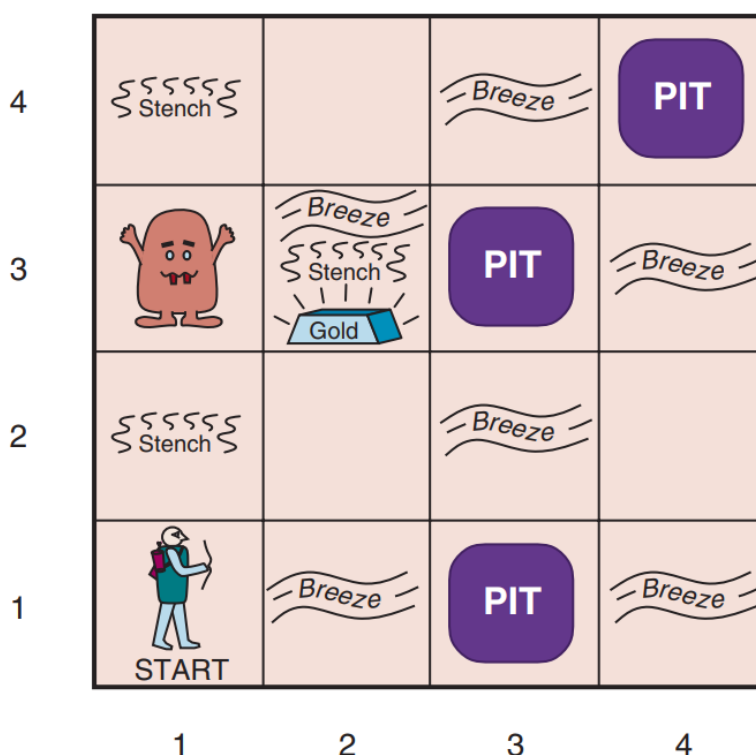
Để tránh việc phải sử dụng quá nhiều dấu ngoặc, LMĐ quy định thứ tự ưu tiên cho các toán tử, từ cao nhất đến thấp nhất: **Thứ tự ưu tiên:** $\neg$, $\wedge$, $\vee$, $\Rightarrow$, $\Leftrightarrow$

Điều này có nghĩa là toán tử $\neg$ (phủ định) sẽ được áp dụng trước tiên, sau đó đến $\wedge$ (hội), và cứ thế tiếp diễn.

- **Ví dụ:** Câu $\neg A \wedge B \Rightarrow C \vee D$ sẽ được hiểu là $(\neg A) \wedge B \Rightarrow (C \vee D)$.

4. Áp dụng Cú pháp vào Thế giới Wumpus

Hãy xem cách các quy tắc cú pháp này được dùng để xây dựng cơ sở tri thức cho tác tử trong thế giới Wumpus.



Hình 81. Môi trường Wumpus điển hình.

Hình 81: Môi trường Wumpus điển hình. Tác tử ở ô $[1,1]$ và đang nhìn về phía Đông.

- **Định nghĩa các mệnh đề nguyên tử:**
 - $P_{i,j}$: Có hố (Pit) ở ô (i,j) .
 - $W_{i,j}$: Có Wumpus ở ô (i,j) .
 - $B_{i,j}$: Cảm nhận có gió (Breeze) ở ô (i,j) .
 - $S_{i,j}$: Cảm nhận có mùi hôi (Stench) ở ô (i,j) .
- **Xây dựng các câu phức hợp:**
 - Một quy tắc đơn giản: “Không có hố ở ô $(1,1)$ ”.
 - Cú pháp: $\neg P_{1,1}$
 - Một quy tắc phức tạp hơn về cảm biến: “Một ô có gió khi và chỉ khi có ít nhất một ô kề bên nó có hố”.
 - Cú pháp cho ô $(1,1)$: $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
 - *Phân tích cú pháp*: Câu này được tạo ra bằng cách kết nối mệnh đề nguyên tử $B_{1,1}$ với mệnh đề phức hợp $(P_{1,2} \vee P_{2,1})$ bằng toán tử $\Leftrightarrow$. Mệnh đề phức hợp bên trong lại được tạo từ hai mệnh đề nguyên tử $P_{1,2}$ và $P_{2,1}$ bằng toán tử $\vee$.

Cú pháp của Logic Mệnh đề cung cấp một bộ khung sườn không mơ hồ để biểu diễn các sự thật và quy luật. Nó bao gồm:

- **Bảng chữ cái**: Các ký hiệu mệnh đề ($P, Q, \dots$) và các hằng ($True, False$).
- **Các toán tử**: $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$.
- **Ngữ pháp**: Các quy tắc (thường là BNF) để kết hợp các ký hiệu và toán tử thành các câu hợp lệ.
- **Độ ưu tiên**: Các quy tắc để diễn giải các câu phức hợp mà không cần quá nhiều dấu ngoặc.

Nắm vững cú pháp là bước đầu tiên và thiết yếu để có thể xây dựng các cơ sở tri thức logic, làm nền tảng cho các hệ thống AI có khả năng suy diễn tự động.

3.2.3. Ngữ nghĩa (Semantics) của Logic Mệnh đề

1. Khái niệm mô hình và giá trị chân lý

Ngữ nghĩa của Logic mệnh đề quy định **cách gán giá trị Đúng (True) hoặc Sai (False) cho mọi câu** trên cơ sở một **mô hình** (model) – tức một phép gán giá trị cho từng ký hiệu mệnh đề. Từ giá trị của các mệnh đề nguyên tử, giá trị của câu phức được xác định **đệ quy** theo năm toán tử logic.

Ví dụ, với mô hình

$$m_1 = \{P_{1,2} = \text{false}, P_{2,2} = \text{false}, P_{3,1} = \text{true}\}$$

câu $\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1})$ được đánh giá là

$$\text{true} \wedge (\text{false} \vee \text{true}) = \text{true}.$$

2. Bảng chân trị của năm toán tử

Ngữ nghĩa chi tiết của từng toán tử được biểu diễn đầy đủ trong **Bảng 7.8** dưới đây (trích nguyên bản):

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

3. Quy tắc diễn giải từ bảng chân trị

Từ bảng trên ta rút ra **năm quy tắc chân trị** (đúng trong *mọi* mô hình):

- $\neg P$ đúng **iff** P sai.
- $P \wedge Q$ đúng **iff** P và Q đều đúng.
- $P \vee Q$ đúng **iff** ít nhất một trong hai mệnh đề đúng.
- $P \Rightarrow Q$ sai **chỉ khi** P đúng *nhưng* Q sai; ngoài trường hợp đó luôn đúng.
- $P \Leftrightarrow Q$ đúng **iff** P và Q cùng giá trị (cả hai đúng hoặc cả hai sai).

Những quy tắc này giúp **tự động hoá quá trình đánh giá**: chỉ cần thay thế (substitute) giá trị của các mệnh đề con theo mô hình và áp dụng quy tắc tương ứng.

4. Vai trò bảng chân trị trong AI

- **Kiểm chứng mô hình**: Bảng chân trị cho phép *model checking* – liệt kê mọi mô hình để xác nhận $KB \models \alpha$.
- **Nền tảng suy luận**: Các thuật toán như *PL-TRUE?* đánh giá chân trị của câu bất kỳ dựa trên bảng này, làm cơ sở cho kiểm tra thoả mãn (SAT), *resolution*, v.v.
- **Giải thích trực quan**: Sinh viên có thể thực hành viết và kiểm tra quy tắc (ví dụ “*Nếu trời mưa thì đường ướt*”) bằng cách điền bảng chân trị tương ứng.

Mô hình xác định giá trị cho từng ký hiệu; **bảng chân trị** của năm toán tử cung cấp **bộ quy tắc hoàn chỉnh** để diễn giải mọi câu mệnh đề. Nhờ vậy, Logic mệnh đề trở thành **chuẩn ngữ nghĩa hình thức** đầu tiên cho các hệ thống biểu diễn tri thức và suy luận tự động trong Trí tuệ nhân tạo.

3.2.4. Suy diễn (Inference) trong Logic Mệnh đề

Nếu **cú pháp** là ngữ pháp để viết nên các câu và **ngữ nghĩa** là quy tắc để hiểu ý nghĩa của chúng, thì **suy diễn (inference)** chính là quá trình “tư duy”—hành động rút ra những kết luận mới từ những điều đã biết. Đây là động cơ làm cho một Cơ sở Tri thức (KB) trở nên sống động và hữu ích, cho phép tác tử đưa ra quyết định thông minh thay vì hành động ngẫu nhiên.

1. Mục tiêu của Suy diễn: Hệ quả Logic (Logical Entailment)

Nhiệm vụ trung tâm của suy diễn là kiểm tra mối quan hệ **hệ quả logic (logical entailment)**. Chúng ta muốn biết liệu một câu truy vấn α có phải là một hệ quả tất yếu từ Cơ sở Tri thức KB hay không.

- **Ký hiệu:** $KB \models \alpha$
- **Phát biểu:** “ KB kéo theo α ” (KB entails α).
- **Định nghĩa chính thức:** $KB \models \alpha$ khi và chỉ khi trong **mọi mô hình (thế giới khả dĩ)** mà tất cả các câu trong KB đều đúng, thì câu α cũng **phải đúng**.
- **Diễn giải trực quan:** Nếu chúng ta chấp nhận mọi điều trong KB là sự thật, thì chúng ta có bị buộc phải chấp nhận α cũng là sự thật hay không? Nếu câu trả lời là “có” một cách tuyệt đối, không có ngoại lệ, thì α là một hệ quả logic.

Ví dụ kinh điển:

- $KB = \{ \text{“Trời đang mưa”, “Nếu trời mưa thì đường trơn”} \}$
- $\alpha = \text{“Đường trơn”}$
- Ta có $KB \models \alpha$, vì không thể tồn tại một thế giới nào mà hai câu trong KB đúng nhưng câu α lại sai.

Quá trình kiểm tra $KB \models \alpha$ được thực hiện bởi một **thủ tục suy diễn (inference procedure)**.

2. Phương pháp 1: Kiểm tra Mô hình (Model Checking)

Đây là phương pháp suy diễn trực tiếp nhất, dựa thẳng vào định nghĩa của hệ quả logic. Ý tưởng của nó rất đơn giản nhưng mạnh mẽ: duyệt qua tất cả các thế giới khả dĩ để kiểm tra.

Ý tưởng cốt lõi

- Tạo ra tất cả các mô hình (model) có thể có cho các ký hiệu mệnh đề trong bài toán.
- Loại bỏ những mô hình nào mâu thuẫn với các câu trong KB .
- Trong số các mô hình còn lại (những mô hình mà KB đúng), kiểm tra xem câu truy vấn α có đúng trong **tất cả** chúng hay không.
- Nếu có, kết luận $KB \models \alpha$. Nếu không, kết luận phủ định.

Minh họa chi tiết với Hành trình của Tác tử Wumpus

Hãy cùng theo dõi hành trình của tác tử và cách nó sử dụng suy diễn để đưa ra quyết định, dựa trên chuỗi hình ảnh 82, 83 và 84.

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

A

 = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

(a)

(b)

Hình 82. Bước đi đầu tiên của tác nhân trong thế giới Wumpus.

Hình 82: Bước đi đầu tiên của tác nhân trong thế giới Wumpus. (a) Tình huống ban đầu: Tác tử ở ô [1,1] và không cảm nhận được gì (*[None, None, None, None, None]*). Dựa vào luật chơi, tác tử có thể suy ra các ô kề bên [1,2] và [2,1] là an toàn. (b) Sau khi di chuyển đến ô [2,1]: Tác tử cảm nhận được một luồng gió (*Breeze*). Đây là lúc quá trình suy diễn phức tạp hơn bắt đầu.

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

A

 = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

1,4	2,4 P?	3,4	4,4
1,3 W!	2,3 A S G B	3,3 P?	4,3
1,2 S V OK	2,2 V OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

(a)

(b)

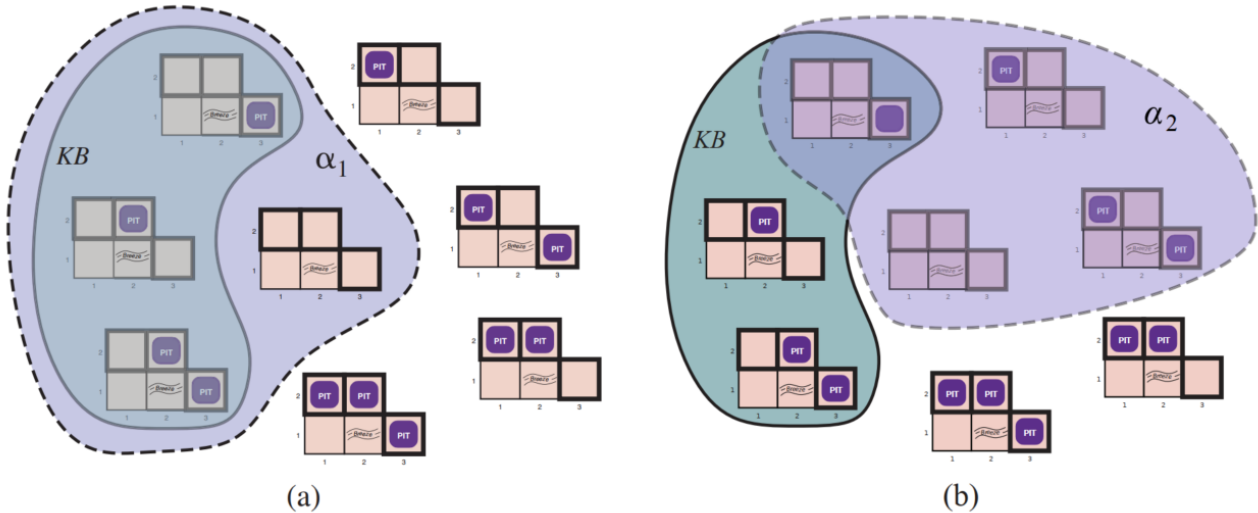
Hình 83. Các giai đoạn tiếp theo trong hành trình của tác nhân Wumpus.

Hình 83: Các giai đoạn tiếp theo trong hành trình của tác nhân Wumpus. Hình ảnh này mô tả các bước đi sau đó, nhưng chúng ta sẽ tập trung vào thời điểm quan trọng ở Hình 82(b) để phân tích quá trình suy diễn.

Tình huống cần suy diễn: Sau khi đến [2,1] và cảm nhận thấy gió, *KB* của tác tử được cập nhật. Bây giờ nó cần quyết định xem các ô kề bên chưa khám phá là [1,2], [2,2], và [3,1] có an toàn hay không.

- **Cơ sở tri thức (KB)** bao gồm:
 - Các luật của trò chơi (ví dụ: $B_{i,j} \Leftrightarrow (P_{i,j+1} \vee P_{i,j-1} \vee P_{i+1,j} \vee P_{i-1,j})$).
 - Các quan sát: $\neg P1,1$, $\neg B1,1$ (từ bước đi trước), và quan sát mới $B2,1$.
- **Câu hỏi (Query):** “Ô [1,2] có an toàn không?” ($\alpha1 = \neg P1,2$), “Ô [2,2] có an toàn không?” ($\alpha2 = \neg P2,2$).

Bây giờ, hãy áp dụng Kiểm tra Mô hình cho 3 ô tình nghi này. Có 3 ký hiệu mệnh đề ($P1,2$, $P2,2$, $P3,1$), vậy có $2^3 = 8$ mô hình khả dĩ.



Hình 84. Các mô hình khả dĩ về sự có mặt của hố trong thế giới Wumpus.

Hình 84: Kiểm tra mô hình cho các hố.

- **Liệt kê 8 mô hình:** Hình trên biểu diễn 8 thế giới khả dĩ, mỗi thế giới là một cách sắp xếp hố khác nhau trong 3 ô [1,2], [2,2], [3,1].
- **Lọc theo KB:** Dựa trên các luật chơi và quan sát ($\neg B1,1$ và $B2,1$), chúng ta loại bỏ các mô hình không phù hợp. Ví dụ, bất kỳ mô hình nào có hố ở [1,2] đều bị loại vì tác tử không cảm thấy gió ở [1,1]. Tương tự, vì có gió ở [2,1], ít nhất một trong các ô kề [1,1], [2,2], [3,1] phải có hố. Sau khi lọc, chỉ còn lại **3 mô hình** hợp lệ (được bao quanh bởi đường liền nét).
- **Kiểm tra truy vấn $\alpha1 = \neg P1,2$ (“Không có hố ở [1,2]”):**
 - Nhìn vào 3 mô hình hợp lệ, ta thấy trong **cả 3 mô hình**, ô [1,2] đều không có hố.
 - **Kết luận:** $KB \models \neg P1,2$. Tác tử có thể chắc chắn rằng ô [1,2] an toàn.
- **Kiểm tra truy vấn $\alpha2 = \neg P2,2$ (“Không có hố ở [2,2]”):**
 - Nhìn vào 3 mô hình hợp lệ, ta thấy có 2 mô hình mà [2,2] không có hố, nhưng có 1 mô hình mà [2,2] **có** hố.
 - Vì $\alpha2$ không đúng trong *tất cả* các mô hình mà KB đúng, ta **không thể** kết luận $KB \models \neg P2,2$. Tác tử không thể chắc chắn về sự an toàn của ô [2,2].
- **Độ phức tạp:** Phương pháp này có độ phức tạp là $O(2^n)$ với n là số ký hiệu mệnh đề, do đó nó chỉ khả thi cho các KB có quy mô nhỏ.

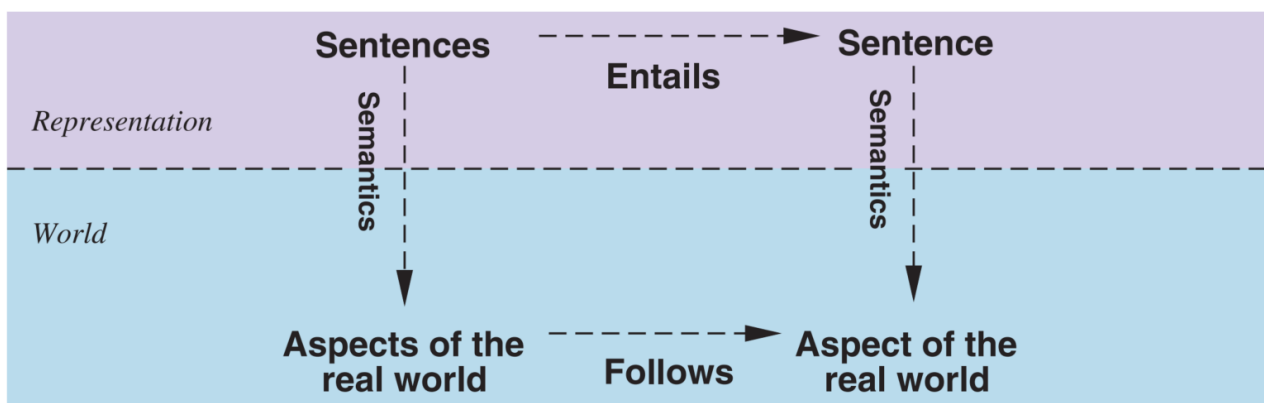
3. Đánh giá các Thủ tục Suy diễn: Đúng đắn và Đầy đủ

Làm thế nào để biết một thủ tục suy diễn là tốt? Chúng ta cần hai thuộc tính quan trọng:

- **Suy diễn (Derivation):** Nếu một thủ tục suy diễn i có thể rút ra câu α từ KB , ta viết: $KB \vdash i \alpha$. Đây là một quá trình thao tác trên **cú pháp**.
- **Tính Đúng đắn (Soundness):** Một thủ tục suy diễn i được gọi là đúng đắn nếu nó chỉ suy ra những câu là hệ quả logic. > **Tức là:** Nếu $KB \vdash i \alpha$ thì $KB \models \alpha$. **Ý nghĩa:** “Thuật toán không bịa chuyện”. Mọi kết luận nó đưa ra đều được đảm bảo là đúng nếu tri thức ban đầu đúng.
- **Tính Đầy đủ (Completeness):** Một thủ tục suy diễn i được gọi là đầy đủ nếu nó có thể suy ra mọi câu là hệ quả logic. > **Tức là:** Nếu $KB \models \alpha$ thì $KB \vdash i \alpha$. **Ý nghĩa:** “Thuật toán có thể tìm thấy mọi sự thật”. Nếu một điều gì đó là sự thật ẩn chứa trong KB , một thuật toán đầy đủ sẽ có khả năng tìm ra nó.

Kiểm tra Mô hình là một thủ tục vừa **đúng đắn** vừa **đầy đủ** cho Logic Mệnh đề.

4. Mối quan hệ giữa Cú pháp, Ngữ nghĩa và Thực tế



Hình 85. Suy luận logic như quá trình chuyển đổi cấu hình vật lý.

Hình 85: Suy luận logic như quá trình chuyển đổi cấu hình vật lý.

Điều kỳ diệu của logic nằm ở sự tương ứng này:

- **Suy diễn là quá trình vật lý:** Bên trong máy tính, suy diễn chỉ là các thao tác trên **cú pháp**—thay đổi các bit trong bộ nhớ.
- **Hệ quả logic là quan hệ ngữ nghĩa:** Nó phản ánh mối quan hệ chân lý trong thế giới trừu tượng của các mô hình.
- **Sự tương ứng:** Một thủ tục suy diễn **đúng đắn** đảm bảo rằng các thao tác cú pháp bên trong máy tính (bên phải hình) phản ánh chính xác các mối quan hệ ngữ nghĩa của thế giới (bên trái hình).

Cuối cùng, **sự tiếp đất (grounding)** là quá trình kết nối các ký hiệu logic với thế giới thực. Điều này được thực hiện thông qua **cảm biến** của tác tử. Khi cảm biến mũi phát hiện mùi hôi,

chương trình sẽ tạo ra câu $S1,2$ trong KB. Do đó, câu $S1,2$ trong KB là đúng trong thế giới thực vì nó được tạo ra bởi chính quá trình cảm nhận thế giới thực đó.

3. Phương pháp 2 – Suy diễn dựa trên Quy tắc (Rule-Based Inference)

Trong khi **Kiểm tra Mô hình (Model Checking)** tiếp cận bài toán suy diễn bằng cách duyệt qua toàn bộ các “thế giới khả dĩ” (khía cạnh ngữ nghĩa), thì **suy diễn dựa trên quy tắc** lại có một hướng đi khác. Phương pháp này thao tác trực tiếp trên **cú pháp** của các câu logic, áp dụng các quy tắc biến đổi để sinh ra các câu mới từ những câu đã có. Mỗi quy tắc này đều được chứng minh là **đúng đắn (sound)**, đảm bảo rằng mọi kết luận rút ra đều là hệ quả logic.

3.1. Các Quy tắc Suy diễn Trực quan và Chaining

Đây là nhóm các quy tắc đơn giản, mô phỏng cách con người thường tư duy. Chúng là nền tảng cho các thuật toán suy diễn tiến và lùi (forward/backward chaining).

a. Modus Ponens (MP) – Quy tắc Vàng của Lập luận Tiến

Quy tắc:

$$\frac{A \Rightarrow B, A}{\therefore B}$$

- **Diễn giải:** Nếu bạn biết rằng “Nếu α đúng, thì β đúng” ($\alpha \Rightarrow \beta$), và bạn cũng biết rằng “ α thực sự đúng” (α), thì bạn có thể kết luận một cách chắc chắn rằng “ β cũng đúng” (β).
- **Ví dụ:**
 - **KB chứa:**
 1. Nếu trời mưa thì đường ướt. ($Mưa \Rightarrow Uớt$)
 2. Trời đang mưa. ($Mưa$)
 - **Kết luận suy ra được:** Đường ướt. ($Uớt$)
- **Tầm quan trọng:** Modus Ponens là quy tắc suy diễn phổ biến và trực quan nhất, là xương sống của các hệ thống suy diễn tiến (forward chaining), nơi hệ thống liên tục áp dụng các luật để sinh ra các sự thật mới.

b. And-Introduction – Quy tắc Tổng hợp Sự thật

Quy tắc:

$$\frac{A, B}{\therefore A \wedge B}$$

- **Diễn giải:** Nếu bạn biết α là đúng và β cũng là đúng, bạn có thể kết luận rằng mệnh đề hội “ α và β ” ($\alpha \wedge \beta$) là đúng.

- **Ứng dụng:** Đây là một bước xây dựng Cơ sở tri thức (KB) điển hình. Khi một tác tử nhận được hai mâu thuẫn tin riêng biệt từ cảm biến, nó có thể gộp chúng lại thành một sự thật tổng hợp duy nhất.

3.2. Hợp giải (Resolution) – Một Quy tắc Toàn năng

Trong khi các quy tắc như Modus Ponens rất trực quan, chúng lại không đầy đủ (complete) cho toàn bộ Logic Mệnh đề. **Hợp giải (Resolution)** là một quy tắc suy diễn duy nhất nhưng lại mạnh mẽ, vừa đúng đắn vừa đầy đủ.

- **Quy tắc Hợp giải (cho mệnh đề con):** $(l_1 \vee \dots \vee l_k), (m_1 \vee \dots \vee m_n) \vdash l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n$ Trong đó l_i và m_j là các literal (ký hiệu mệnh đề hoặc phủ định của nó) và $l_i = \neg m_j$.
- **Diễn giải:** Nếu bạn có hai mệnh đề tuyển (OR), và một literal trong mệnh đề này là phủ định của một literal trong mệnh đề kia, bạn có thể tạo ra một mệnh đề tuyển mới bằng cách kết hợp phần còn lại của cả hai.

Suy diễn bằng Phản chứng với Hợp giải

Để áp dụng Hợp giải, thuật toán không chứng minh $KB \models \alpha$ một cách trực tiếp. Thay vào đó, nó sử dụng **chứng minh bằng phản chứng (proof by refutation)**:

- **Giả sử α sai:** Tức là ta thêm $\neg \alpha$ vào KB .
- **Mục tiêu:** Chứng minh rằng tập hợp câu mới $(KB \wedge \neg \alpha)$ dẫn đến một **mâu thuẫn**.
- **Quy trình:** a. **Chuyển về Dạng Chuẩn Hội (CNF):** Tất cả các câu trong $(KB \wedge \neg \alpha)$ phải được chuyển đổi thành dạng CNF (một chuỗi các mệnh đề AND với nhau, trong đó mỗi mệnh đề con là một chuỗi các literal OR với nhau). b. **Áp dụng Hợp giải lặp lại:** Liên tục áp dụng quy tắc hợp giải cho các cặp mệnh đề con trong tập hợp để sinh ra các mệnh đề con mới. c. **Kết luận:** * Nếu quá trình này sinh ra **mệnh đề rỗng (empty clause - $\perp$)**, điều này chứng tỏ có mâu thuẫn. Giả định ban đầu ($\neg \alpha$ đúng) là sai. Do đó, α phải đúng, và ta kết luận $KB \models \alpha$. * Nếu không thể sinh ra mệnh đề con mới nào nữa mà vẫn chưa có mệnh đề rỗng, ta kết luận KB không kéo theo α .

3.3. Thuật toán DPLL: Giải quyết Bài toán SAT hiệu quả

DPLL (Davis-Putnam-Logemann-Loveland) là một thuật toán nền tảng để giải quyết **Bài toán Thỏa mãn Lô-gic (SAT)**, một bài toán có quan hệ mật thiết với suy diễn. Về bản chất, DPLL là một thuật toán tìm kiếm theo chiều sâu, có hồi quy để tìm một mô hình (một cách gán trị True/False) làm cho một câu CNF trở nên đúng.

```

function DPLL-SATISFIABLE?(s) returns true or false
  inputs: s, a sentence in propositional logic

  clauses  $\leftarrow$  the set of clauses in the CNF representation of s
  symbols  $\leftarrow$  a list of the proposition symbols in s
  return DPLL(clauses, symbols, { })

function DPLL(clauses, symbols, model) returns true or false

  if every clause in clauses is true in model then return true
  if some clause in clauses is false in model then return false
  P, value  $\leftarrow$  FIND-PURE-SYMBOL(symbols, clauses, model)
  if P is non-null then return DPLL(clauses, symbols – P, model  $\cup$  {P=value})
  P, value  $\leftarrow$  FIND-UNIT-CLAUSE(clauses, model)
  if P is non-null then return DPLL(clauses, symbols – P, model  $\cup$  {P=value})
  P  $\leftarrow$  FIRST(symbols); rest  $\leftarrow$  REST(symbols)
  return DPLL(clauses, rest, model  $\cup$  {P=true}) or
    DPLL(clauses, rest, model  $\cup$  {P=false})

```

Hình 86. Thuật toán DPLL kiểm tra tính thỏa mãn trong logic mệnh đề.

DPLL vượt trội hơn kiểm tra mô hình thông thường nhờ 3 cải tiến chính:

- **Kết thúc sớm (Early Termination):**
 - Một câu CNF là **sai** ngay khi có một mệnh đề con bị sai (tất cả các literal trong nó đều sai).
 - Một câu CNF là **đúng** nếu trong mỗi mệnh đề con, có ít nhất một literal là đúng.
 - DPLL có thể đưa ra kết luận này ngay cả khi mô hình chưa được gán giá trị đầy đủ, giúp cắt tĩa các nhánh tìm kiếm không cần thiết.
- **Heuristic Ký hiệu Thuần túy (Pure Symbol Heuristic):**
 - Một ký hiệu được gọi là “thuần túy” nếu nó chỉ xuất hiện dưới một dạng duy nhất (hoặc luôn là *P*, hoặc luôn là $\neg P$) trong tất cả các mệnh đề con chưa được thỏa mãn.
 - DPLL có thể gán giá trị cho ký hiệu này để làm cho literal của nó đúng mà không gây rủi ro làm sai bất kỳ mệnh đề nào.
- **Heuristic Mệnh đề Đơn vị (Unit Clause Heuristic):**
 - Một mệnh đề đơn vị là một mệnh đề chỉ còn một literal chưa được gán giá trị. Để mệnh đề này đúng, literal đó bắt buộc phải nhận một giá trị cụ thể.
 - DPLL ưu tiên gán giá trị cho các literal này. Việc này có thể tạo ra một chuỗi các phép gán bắt buộc gọi là **lan truyền đơn vị (unit propagation)**, giúp giảm đáng kể không gian tìm kiếm.

Với những cải tiến này và các kỹ thuật tối ưu hiện đại (như học mệnh đề xung đột, khởi động lại ngẫu nhiên), các bộ giải SAT dựa trên DPLL có thể giải quyết các bài toán với hàng triệu biến số.

4. So sánh các Chiến lược Suy diễn

Tiêu chí	Kiểm tra Mô hình (Model Checking)	Suy diễn dựa trên Quy tắc
Độ đúng đắn (Soundness)	Luôn đúng đắn về mặt bản chất (dựa trên định nghĩa).	Phụ thuộc vào tính đúng đắn của từng quy tắc (MP, Hợp giải đều đúng đắn).
Độ đầy đủ (Completeness)	Luôn đầy đủ (vì xét hết mọi mô hình).	Phụ thuộc vào bộ quy tắc. Hợp giải là đầy đủ, MP thì không.
Hiệu quả (Efficiency)	Kém hiệu quả, độ phức tạp $O(2^n)$, bùng nổ với số lượng ký hiệu lớn.	Hiệu quả hơn nhiều. Có thể là tuyến tính cho các KB dạng Horn. Tệ nhất là NP-complete.
Khả năng mở rộng	Hạn chế, chỉ phù hợp cho KB nhỏ.	Linh hoạt, là nền tảng cho các hệ chuyên gia và các hệ thống lập luận lớn.

5. Kết luận: Lựa chọn Công cụ phù hợp

Suy diễn trong Logic Mệnh đề cung cấp hai con đường chính:

- Tiếp cận Ngữ nghĩa (Model Checking):** Đảm bảo chính xác tuyệt đối và dễ hiểu về mặt lý thuyết, nhưng chi phí tính toán rất cao.
- Tiếp cận Cú pháp (Rule-Based):** Hiệu quả và linh hoạt hơn nhiều trong thực tế. Các thuật toán như Chaining (dựa trên MP) và các bộ giải SAT (dựa trên DPLL) là những công cụ mạnh mẽ, cho phép xây dựng các tác tử AI có khả năng lập luận trên các cơ sở tri thức quy mô lớn.

Việc nắm vững cả hai phương pháp giúp người thiết kế hệ thống AI có thể lựa chọn kỹ thuật phù hợp nhất tùy theo đặc điểm của bài toán, quy mô của KB và yêu cầu về thời gian đáp ứng.

3.3.1. Hạn chế của Logic Mệnh đề (Why Propositional Logic Falls Short)

1. Tính biểu đạt (expressiveness) bị giới hạn

Logic Mệnh đề (LMĐ) chỉ cho phép thao tác trên **các câu khẳng định đơn lẻ**—mỗi ký hiệu mệnh đề tương ứng với *toàn bộ* một sự thật cụ thể. LMĐ **không “thấy”** cấu trúc bên trong của sự thật ấy: nó không biết “vua”, “hó”, “ô (1,2)” là gì, cũng không diễn tả được quan hệ *liên kết, lớn hơn, cha của...* Vì vậy, LMĐ **thiếu cả đối tượng (object) lẫn quan hệ (relation)** trong mô hình thế giới.

Hệ quả: các quy luật tổng quát trong tự nhiên ngôn ngữ (“Mọi vua đều là người”, “Ô nào kẻ hồ thì có gió”...) **không thể viết một lần** trong LMĐ.

2. “Bùng nổ câu” khi cố gắng khái quát

Để mô tả một quy tắc chung, ta buộc phải **liệt kê thủ công** mọi trường hợp riêng lẻ. Trong thế giới Wumpus, câu “*một ô có gió khi và chỉ khi một ô kề có hố*” phải được chép **cho từng ô**:

scssCopyEdit $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

$B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$

...

Tác giả AIMA ghi chú rõ: “Điều này phải được phát biểu cho **mỗi ô**; ở đây chúng ta mới nêu vài ô liên quan”.

Nếu lưới là $n \times n$, ta sẽ cần n^2 luật—quy mô KB tăng **bậc hai** theo kích thước môi trường, gây tốn bộ nhớ và thời gian suy diễn.

3. Không thể khái quát hoá bằng biến và lượng từ

Khó khăn trên bắt nguồn từ chỗ LMĐ **không có biến** ($x, y, \dots$), **không có lượng từ** ($\forall, \exists$). Trong chương 8, Russell & Norvig chỉ ra rằng First-Order Logic (FOL) khắc phục việc này nhờ *ontological commitment* mới: thế giới **gồm đối tượng + quan hệ**; nhờ đó ta viết gọn:

$$\forall s (Pit(s) \Rightarrow \forall s' [Adjacent(s, s') \Rightarrow Breeze(s')])$$

– một câu đủ cho **mọi** ô.

4. Khả năng suy luận bị ràng buộc

Chỉ với các ký hiệu Boolean, LMĐ không thể:

- **Đếm** hoặc so sánh đại lượng (“*nhiều hơn ba kẻ thù*”).
- **Biểu diễn sở hữu ngữ cảnh** ($Father(John) = Henry\ II$).
- **Thay đổi theo thời gian** – phải tạo thêm biến mệnh đề mới cho mỗi thời điểm.

Điều này khiến tác nhân dựa trên LMĐ khó mở rộng sang các tác vụ nhận thức phức tạp.

5. Tóm tắt so sánh

Khía cạnh	Logic Mệnh đề	First-Order Logic
Yếu tố mô tả	Chỉ sự kiện đúng/sai	Đối tượng, quan hệ, hàm
Khả năng khái quát	Không (phải liệt kê)	Có ($\forall, \exists$)
Kích thước KB cho quy luật “Breeze–Pit”	n^2 câu	1 câu
Ví dụ	$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$	$\forall s \forall s' (Pit(s) \wedge Adj(s, s') \Rightarrow Breeze(s'))$
Ứng dụng	Hệ luận đơn giản, mô hình nhỏ	Hệ chuyên gia, NLP, hệ tri thức phức tạp

Logic Mệnh đề là **bước khởi đầu cần thiết** để hiểu lập luận hình thức, nhưng chính sự thiếu vắng **đối tượng, quan hệ và lượng từ** khiến nó **không đủ sức** mô hình hoá những quy luật tổng quát trong AI hiện đại. Sự chuyển dịch sang Logic Vị từ là tất yếu khi bài toán yêu cầu *biểu diễn tri thức phong phú và suy luận mở rộng* về thế giới.

3.3.2. Cú pháp của Logic Vị từ (First-Order Logic – FOL)

1. Bảng chữ cái (Alphabet) của FOL

Loại ký hiệu	Vai trò	Ví dụ	Ghi chú
Hằng (Constant Symbols)	Đặt tên đối tượng cụ thể	<i>John, Richard, Wumpus</i>	Chữ cái đầu in hoa; mỗi hằng gắn với đúng một thực thể trong miền
Biến (Variables)	Đóng vai trò đại diện cho mọi đối tượng	x, y, s'	Viết thường; giá trị do lượng từ gán
Hàm (Function Symbols)	Nhận n đối tượng, trả về một đối tượng	<i>FatherOf(John), S(n)</i>	Mỗi hàm có arity cố định (số đối số)
Vị từ (Predicate Symbols)	Kiểm tra thuộc tính/quan hệ	<i>King(x), Adjacent(s,s')</i>	Cũng có arity; = là vị từ đặc biệt đẳng thức
Kết nối mệnh đề	$\neg \neg \wedge \vee \Rightarrow \Leftrightarrow$	Ghép câu phức	Độ ưu tiên giống PL: $\neg > \wedge > \vee > \Rightarrow > \Leftrightarrow$
Lượng từ	$\forall$ (phổ quát) & $\exists$ (tồn tại)	$\forall x \text{ King}(x) \dots, \exists y \text{ Crown}(y) \dots$	Bao quát phạm vi tới hết công thức bên phải

Ví dụ nhận diện

- *Đối tượng*: “Richard the Lionheart”, số 0.
- *Quan hệ*: *Brother(Richard, John)* (nhị nguyên).
- *Hàm*: *LeftLeg(John)* (nhất nguyên).

(Nguồn: Stuart J. Russell & Peter Norvig – AIMA 2021)

2. Ngữ pháp hình thức (BNF)

bnfCopyEditSentence $\rightarrow$ AtomicSentence | ComplexSentence

AtomicSentence $\rightarrow$ Predicate | Predicate(Term,...) | Term = Term

ComplexSentence $\rightarrow$ (Sentence)

| $\neg$ Sentence

| Sentence $\wedge$ Sentence

| Sentence $\vee$ Sentence

| Sentence $\Rightarrow$ Sentence

| Sentence $\Leftrightarrow$ Sentence

| Quantifier Variable,... Sentence

Term $\rightarrow$ Function(Term,...) | Constant | Variable

Quantifier $\rightarrow \forall \mid \exists$

(Trích Figure 8.3 – AIMA, 2021)

Lưu ý: Mỗi **lượng từ** chi phối mọi biến liên kề; nên viết $\forall x (P(x) \Rightarrow Q(x))$ thay vì $\forall x P(x) \Rightarrow Q(x)$ để tránh hiểu nhầm.

3. Lượng từ và phạm vi

- **Lượng từ phổ quát ($\forall$)** – “với mọi”:

$$\forall x [King(x) \Rightarrow Person(x)]$$

“Mọi vua đều là người”

- **Lượng từ tồn tại ($\exists$)** – “tồn tại ít nhất một”:

$$\exists x [Crown(x) \wedge OnHead(x, John)]$$

“Có một vương miện nằm trên đầu John”

- **Các quy tắc De Morgan cho lượng từ**

$$\neg \exists x P \equiv \forall x \neg P, \quad \neg \forall x P \equiv \exists x \neg P$$

(và tương tự cho $\wedge, \vee$)

4. Vai trò của hàm và đẳng thức

Hàm cho phép “đặt tên động” cho đối tượng mới: $S(S(0))$ đại diện số 2 trong Peano; đẳng thức = giúp ràng buộc hai term là **cùng một** đối tượng, ví dụ

$$Father(John) = Henry$$

xác lập cha của John là Henry.

5. Mô hình minh họa

Hình 8.2 mô tả mô hình gồm **5 đối tượng**, **2** quan hệ nhị nguyên (*brother*, *on-head*), **3** thuộc tính nhất nguyên (*person*, *king*, *crown*) và **1** hàm *left-leg*.

6. Ví dụ diễn đạt tri thức

Miền	Phát biểu tự nhiên	Công thức FOL
Wumpus	“Ô kè hổ sẽ có gió.”	$\forall s [Pit(s) \Rightarrow \forall s' (Adjacent(s', s) \Rightarrow Breeze(s'))]$
Gia phả	“Mọi người đều có mẹ.”	$\forall p \exists m \text{ MotherOf}(p) = m$
Số tự nhiên	“0 là số tự nhiên và kế tiếp của số tự nhiên cũng là số tự nhiên.”	$NatNum(0) \wedge \forall n [NatNum(n) \Rightarrow NatNum(S(n))]$

Logic Vị từ **mở rộng** Logic Mệnh đề bằng cách:

- **Đưa vào cấu trúc thế giới:** hằng, biến, hàm, vị từ $\rightarrow$ mô hình chứa **đối tượng & quan hệ**.

- **Cho phép khái quát** nhờ lượng từ $\forall, \exists$ – viết **một** luật áp dụng cho vô vàn trường hợp.
- **Giữ nguyên** hệ toán tử của PL nên kế thừa mọi kỹ thuật suy luận, đồng thời mở đường cho các thủ tục mạnh hơn (unification, resolution bậc nhất).

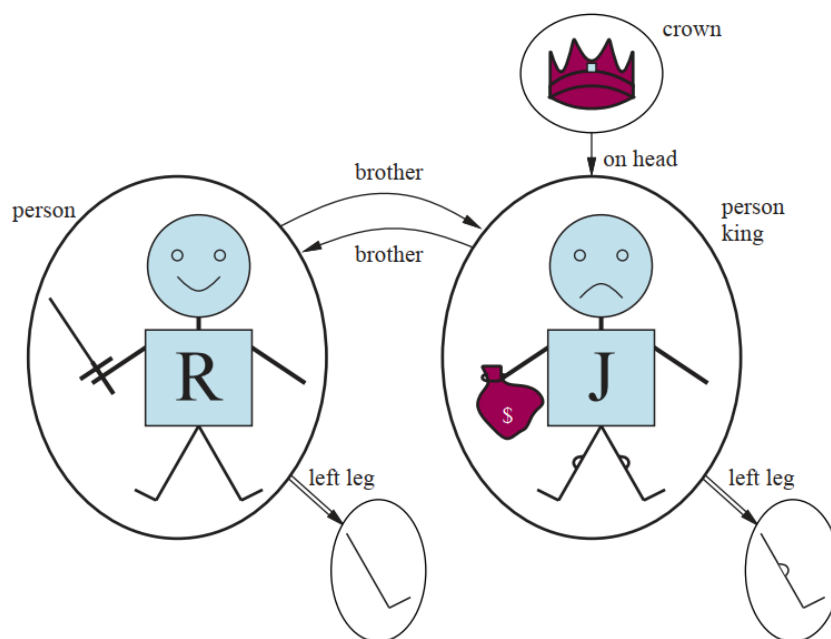
Nhờ năng lực biểu đạt cao và cú pháp chặt chẽ, FOL trở thành **ngôn ngữ “chuẩn”** cho biểu diễn tri thức trong trí tuệ nhân tạo – từ mô hình Wumpus giản đơn tới hệ chuyên gia và cơ sở tri thức quy mô lớn hiện nay.

3.3.3. Suy diễn trong Logic Vị từ (FOL): Từ Quy tắc đến Lập luận Tổng quát

1. Thách thức của Suy diễn trong FOL: Vượt qua giới hạn của Logic Mệnh đề

Trong Logic Mệnh đề, mỗi ký hiệu (P, Q) là một “hộp đen”, không có mối liên hệ nội tại. *VuaJohn_ThamLam* và *VuaRichard_ThamLam* là hai mệnh đề hoàn toàn riêng biệt. Điều này khiến chúng ta không thể tạo ra các quy tắc tổng quát áp dụng cho tất cả các vị vua.

Logic Vị từ (FOL) giải quyết vấn đề này bằng cách cho phép chúng ta nói về **đối tượng (objects)** và các **quan hệ (relations)** giữa chúng.



Hình 87. Một mô hình trong logic vị từ với các đối tượng và quan hệ.

Hình 87: Một mô hình trong FOL. Mô hình này không chỉ có các sự thật đúng/sai, mà nó chứa các đối tượng cụ thể: *Vua Richard*, *Vua John*, *trượng miện*, *chân trái*. Nó cũng định nghĩa các quan hệ giữa chúng: *AnhEm(Richard, John)* và các thuộc tính: *Vua(John)*, *Người(Richard)*.

Thế giới của FOL phong phú hơn nhiều, và do đó, quá trình suy diễn cũng đòi hỏi những công cụ mạnh mẽ hơn. Chúng ta không thể chỉ dùng Modus Ponens đơn giản nữa, mà cần một cơ chế có thể làm việc với các **biến** và **lượng từ**. Mục tiêu vẫn là kiểm tra hệ quả logic ($KB \models \alpha$), nhưng quá trình thực hiện sẽ được “nâng cấp”.

2. “Nâng cấp” Modus Ponens: Generalized Modus Ponens (GMP)

a. Tại sao cần “Nâng cấp”?

Quy tắc Modus Ponens cổ điển ($\alpha \Rightarrow \beta, \alpha \vdash \beta$) không thể hoạt động với các biến.

- **Quy tắc trong KB:** $\forall x \text{ Vua}(x) \wedge \text{ThamLam}(x) \Rightarrow \text{ÁcMa}(x)$ (Mọi vua tham lam đều là ác ma).
- **Sự kiện trong KB:** $\text{Vua}(\text{John}), \text{ThamLam}(\text{John})$.
- **Vấn đề:** $\text{Vua}(\text{John})$ không khớp chính xác với $\text{Vua}(x)$. Chúng ta cần một cách để “gắn” biến x với hằng số John .

b. Định nghĩa Generalized Modus Ponens (GMP)

GMP là phiên bản mở rộng của Modus Ponens để làm việc với các câu chứa biến.

Cho các mệnh đề nguyên tử $p_i, p_{i'}$, và q . Nếu tồn tại một **phép thế (substitution)** θ sao cho $\text{SUBST}(\theta, p_{i'}) = \text{SUBST}(\theta, p_i)$ cho mọi i , thì từ các tiền đề:

$$(p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q), p_1, p_2, \dots, p_n$$

ta có thể suy ra kết luận: $\text{SUBST}(\theta, q)$.

c. Diễn giải và Ví dụ

- **Quy tắc chung:** $\forall x \text{ Vua}(x) \wedge \text{ThamLam}(x) \Rightarrow \text{ÁcMa}(x)$
- **Các sự kiện:** $\text{Vua}(\text{John}), \text{ThamLam}(\text{John})$
- **Tìm phép thế:** Thuật toán tìm thấy một phép thế $\theta = \{x/\text{John}\}$.
- **Áp dụng phép thế:** Khi thay x bằng John , vế trái của quy tắc $\text{Vua}(\text{John}) \wedge \text{ThamLam}(\text{John})$ khớp hoàn toàn với các sự kiện đã biết.
- **Rút ra kết luận:** Áp dụng phép thế θ vào vế phải của quy tắc ($\text{ÁcMa}(x)$), ta suy ra được sự thật mới: $\text{ÁcMa}(\text{John})$.

GMP là một quy tắc suy diễn được “nâng cấp” (lifted), hoạt động ở cấp độ tổng quát của các biến thay vì các mệnh đề cụ thể.

3. Hợp nhất (Unification): “Keo dán” cho các Biểu thức Logic

Hợp nhất là trái tim của mọi thuật toán suy diễn trong FOL. Nó là quá trình tìm kiếm một phép thế các biến để làm cho hai biểu thức logic trở nên giống hệt nhau.

- **Mục tiêu:** Tìm một phép thế θ sao cho $\text{UNIFY}(p, q) = \theta$ với $\text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$.
- **Bộ Hợp nhất Tổng quát nhất (Most General Unifier - MGU):** Thuật toán luôn cố gắng tìm MGU, tức là phép thế ít ràng buộc nhất. Ví dụ, để hợp nhất $\text{Biết}(\text{John}, x)$ và $\text{Biết}(y, z)$, MGU là $\{y/\text{John}, x/z\}$, chứ không phải một phép thế cụ thể hơn như $\{y/\text{John}, x/\text{Jane}, z/\text{Jane}\}$.
- **Thuật toán UNIFY:** Hoạt động bằng cách duyệt đệ quy qua cấu trúc của hai biểu thức và tìm các phép thế cho biến.

- **Kiểm tra Xảy ra (Occurs Check):** Một bước cực kỳ quan trọng để tránh các vòng lặp vô hạn. Thuật toán sẽ thất bại nếu nó cố gắng hợp nhất một biến v với một biểu thức t mà v lại xuất hiện bên trong t .
 - **Ví dụ:** $UNIFY(Biết(x, x), Biết(y, Cha(y)))$ sẽ thất bại. Nếu ta thế $\{y/x\}$, ta sẽ phải hợp nhất x với $Cha(x)$, dẫn đến một cấu trúc vô hạn $x = Cha(Cha(Cha(...)))$.

4. Các Chiến lược Suy diễn sử dụng GMP và Unification

GMP và Unification là “động cơ”, nhưng chúng ta cần một “chiến lược lái” để điều khiển quá trình suy diễn.

Thuật toán	Ý tưởng Vận hành	Vai trò của GMP & Unify	Đặc điểm
Chuỗi tiến (Forward Chaining)	Data-driven: Bắt đầu từ các sự kiện đã biết, liên tục áp dụng các quy tắc để sinh ra các sự thật mới cho đến khi tìm thấy câu truy vấn.	Mỗi bước áp dụng quy tắc chính là GMP . Unification được dùng để khớp các sự kiện với vế trái của quy tắc.	Hiệu quả cho các KB dạng mệnh đề xác định (definite-clause). Phản ứng tốt khi có dữ kiện mới.
Chuỗi lùi (Backward Chaining)	Goal-driven: Bắt đầu từ câu truy vấn (mục tiêu), tìm các quy tắc có thể chứng minh nó. Các tiền đề của quy tắc đó trở thành các mục tiêu con cần chứng minh.	Mỗi bước lùi từ mục tiêu về tiền đề là một phép GMP ngược . Unification dùng để khớp mục tiêu với vế phải của quy tắc và truyền các phép thế biến.	Tiết kiệm không gian bộ nhớ hơn. Rất phù hợp để trả lời các câu hỏi cụ thể. Là nền tảng của ngôn ngữ lập trình Prolog.
Hợp giải (Resolution)	Proof by Refutation: Để chứng minh $KB \models \alpha$, ta chứng minh $KB \wedge \neg \alpha$ dẫn đến mâu thuẫn (mệnh đề rỗng).	Quy tắc hợp giải trong FOL là một phiên bản tổng quát, sử dụng Unification để tìm các literal đối ngẫu và kết hợp hai mệnh đề con.	Là một hệ thống đầy đủ cho toàn bộ FOL. Tuy nhiên, không gian tìm kiếm có thể bùng nổ nếu không có các chiến lược điều khiển thông minh.

5. Đặc tính của Suy diễn trong FOL

- **Ưu điểm:**
 - **Tổng quát hóa:** Cho phép lập luận về vô số đối tượng chỉ bằng một vài quy tắc.
 - **Hiệu quả:** Tiết kiệm hơn rất nhiều so với việc “mệnh đề hóa” (propositionalization) toàn bộ thế giới—tức là tạo một mệnh đề cho mọi sự thật của mọi đối tượng.
- **Thách thức: Bán-Quyết định (Semi-decidability):**
 - Đây là một thuộc tính quan trọng của FOL.

- Nếu $KB \models \alpha$: Một thuật toán suy diễn đầy đủ sẽ tìm ra chứng minh trong một thời gian hữu hạn.
- Nếu KB không kéo theo α : Thuật toán có thể sẽ chạy vô hạn mà không bao giờ kết thúc, vì nó không thể duyệt hết một không gian các hệ quả vô hạn để chắc chắn rằng α không có trong đó.

6. Kết luận: Xương sống của Lập luận Tự động

Cặp đôi **Generalized Modus Ponens (GMP)** và **Hợp nhất (Unification)** chính là “xương sống” cơ bản của hầu hết các hệ thống suy luận trong Logic Vị từ. Chúng cho phép máy tính vượt qua giới hạn của việc xử lý các sự thật riêng lẻ để tiến đến khả năng lập luận trên các quy tắc và khái niệm tổng quát. Được dẫn dắt bởi các chiến lược như chuỗi tiến, chuỗi lùi hay hợp giải, chúng biến một cơ sở tri thức hình thức thành một cỗ máy suy luận mạnh mẽ, là nền tảng cho các hệ chuyên gia, hệ hỏi-đáp và nhiều ứng dụng AI tiên tiến khác.

3.4.1 Luật sản xuất (Production Rules)

Khái niệm tổng quát

Luật sản xuất là một hình thức *biểu diễn tri thức thủ tục* theo mẫu **IF <điều kiện> THEN <hành động/kết luận>**. Toàn bộ tập luật kết hợp với *cơ sở dữ liệu sự kiện* và *bộ suy diễn* (inference engine) tạo thành **hệ sản xuất (production system)** – kiến trúc hạt nhân của nhiều hệ chuyên gia đời đầu, như **R1** cho chẩn đoán thiết bị điện tử hay **PROSPECTOR** cho địa chất học.

1. Cấu trúc hình thức của một luật

Thành phần	Ý nghĩa học thuật	Ví dụ ngắn
Tiền đề (condition)	Tập các mệnh đề có thể nối bằng <i>AND / OR</i> ; được so khớp (matched) với các sự kiện hiện có trong <i>working memory</i> .	<i>Bệnh nhân có sốt</i> $\wedge$ <i>Bệnh nhân có ho</i>
Hậu đề (action/conclusion)	Phát sinh một hành động (thủ tục) hoặc khẳng định mới chèn vào cơ sở tri thức.	$\rightarrow$ <i>Gợi ý xét nghiệm cúm</i>

Ví dụ đầy đủ (trích outline bài giảng) :

IF Bệnh nhân có sốt **AND** Bệnh nhân có ho **THEN** Gợi ý xét nghiệm cúm

2. Mô hình hoạt động của hệ sản xuất

- **Working Memory** – tập sự kiện hiện thời
- **Rule Base** – tập luật sản xuất (KB)
- **Inference Engine** – cơ chế so khớp & thi hành
- **Conflict Resolution** – chiến lược lựa chọn luật khi nhiều luật đồng thời thỏa mãn (ưu tiên mới-cũ, độ chuyên biệt, v.v.).

3. Chiến lược suy diễn

Phương pháp	Cơ chế chính	Đặc điểm – khi dùng
Forward Chaining (suy diễn tiến)	Lắp: so khớp tiền đề → bắn luật → thêm kết luận, cho tới khi đạt mục tiêu hoặc bão hòa.	Phù hợp tác vụ <i>phát hiện</i> (data-driven); được mô tả chi tiết trong §9.3 của AIMA, trang 304–311
Backward Chaining (suy diễn lùi)	Khởi phát từ giả thuyết/đích, tìm chuỗi luật sao cho kết luận của luật khớp giả thuyết; đệ quy lên tiền đề.	Hữu hiệu cho hệ tư vấn/giải đáp; giải thuật minh họa tại H. 9.6, AIMA

Ghi chú sơ phạm: Forward chaining thường được triển khai bằng **thuật toán Rete** để tăng tốc so khớp mẫu theo lô trên hàng nghìn luật; thuật toán này xuất hiện cùng mục lục trang 310 của AIMA, nhấn mạnh vai trò tối ưu hóa hiệu năng của hệ luật lớn.

4. Ưu điểm & Giới hạn

- **Ưu điểm**
 - Dễ diễn đạt tri thức chuyên gia thành từng *luật độc lập*, hỗ trợ mở rộng dần.
 - Thân thiện với lời giải thích (explanation facility): hệ thống có thể liệt kê chuỗi luật đã kích hoạt.
- **Giới hạn**
 - Quy mô lớn → *bùng nổ luật cạnh tranh*; cần chiến lược phân giải xung đột tốt.
 - Thiếu khả năng diễn đạt độ không chắc chắn; phải bổ sung cơ chế *certainty factor* hoặc chuyển sang logic xác suất (x. chương 12 AIMA).

5. Ví dụ minh họa

Bài toán tư vấn y khoa sơ bộ

- *Working Memory* ban đầu:
 - *Có_sốt(Bệnh_nhân_A)*
 - *Có_ho(Bệnh_nhân_A)*
- *Rule Base* chứa luật đã cho ở trên.
- *Forward Chaining* trích xuất đề xuất:
cssCopyEditLuật_1 khớp → Thêm sự kiện Đề nghị_xét_nghiệm_cúm(Bệnh_nhân_A)
- Hệ thống báo cáo cho bác sĩ:
“Dựa trên dấu hiệu sốt và ho, nên xét nghiệm cúm cho bệnh nhân A.”

6. Vị trí của luật sản xuất trong **hệ chuyên gia**

Một hệ chuyên gia cổ điển = *Rule Base* + *Working Memory* + *Inference Engine* + *Explanation & Knowledge-Acquisition Facilities*. Sơ đồ khối chi tiết, cũng như ví dụ nổi tiếng **MYCIN**

(chẩn đoán nhiễm trùng huyết, 1975), được trình bày trong các mục “Knowledge Representation” và “Expert Systems” của AIMA (Ch. 10, Ch. 14) .

Luật sản xuất cung cấp một khuôn khổ *đơn giản nhưng mạnh* để mã hóa tri thức thủ tục dưới dạng điều kiện–hành động. Hiểu rõ cú pháp, ngữ nghĩa và cơ chế suy diễn của chúng là nền tảng để sinh viên tiếp cận các hệ chuyên gia cũng như nhận thấy những thách thức khi mở rộng sang các phương pháp biểu diễn tri thức phong phú hơn.

3.4.2. Mạng ngữ nghĩa (Semantic Networks)

1. Khái niệm tổng quát

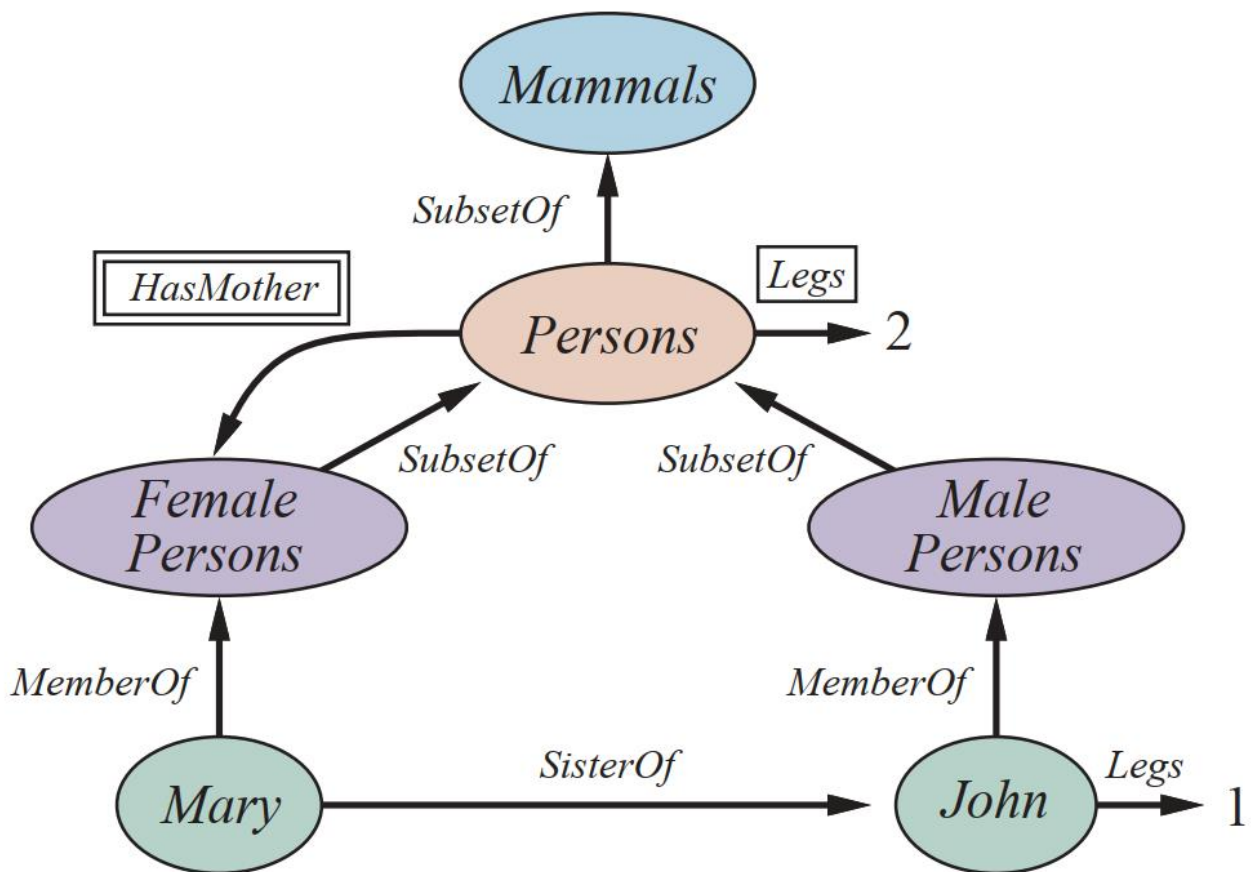
Mạng ngữ nghĩa (semantic network) là một lược đồ đồ thị dùng để **biểu diễn đồng thời các đối tượng cá biệt (individual objects), các hạng (categories) và quan hệ giữa chúng**. Tên đối tượng/hạng thường được đặt trong *oval* hoặc *hình hộp*; các cung nhãn (labeled links) gắn hai nút lại với nhau biểu thị một khẳng định lô-gic tương ứng—chẳng hạn, liên kết **MemberOf** giữa *Mary* và *FemalePersons* tương ứng với mệnh đề $Mary \in FemalePersons$.

2. Cấu trúc hình thức

- **Nút (nodes)**
 - *Object nodes*: biểu diễn thực thể đơn (John, Mary...).
 - *Category nodes*: biểu diễn tập hợp/khái niệm (Persons, Birds...).
- **Cung (edges/links)**
 - *Quan hệ nhị nguyên* được đặt tên rõ ràng (**MemberOf, SisterOf, SubsetOf,...**).
 - Để tránh sai ngữ nghĩa, các cung “kép hộp” được dùng khi muốn gán thuộc tính cho **mọi phần tử** của một hạng thay vì cho bản thân hạng đó.

Quan hệ trọng tâm trong giáo trình

- **is-a / SubsetOf**: liên kết phân cấp “là một”.
- **has-part**: liên kết phần-tổng (mereological). Quan hệ part–whole được Russel & Norvig thảo luận như một biến thể quan trọng của cấu trúc kế thừa.



Hình 88. Mạng ngữ nghĩa đơn giản với các đối tượng và quan hệ phân cấp.

Hình 88 minh họa một **mạng ngữ nghĩa (semantic network)**, trong đó:

- **Nút** đại diện cho các đối tượng (John, Mary) và các lớp khái niệm (Person, Female).
- **Cung** được gán nhãn (MemberOf, SisterOf) thể hiện mối quan hệ logic giữa chúng.

Điểm mạnh của mạng là cơ chế **suy luận kế thừa (inheritance)**: vì Mary là thành viên của lớp Person, cô ấy sẽ tự động kế thừa thuộc tính HasMother của lớp này.

3. Cơ chế kế thừa (Inheritance)

Khi cần truy vấn thuộc tính của một đối tượng, thuật toán sẽ:

- đi theo liên kết **MemberOf** lên hạng trực tiếp;
- tiếp tục leo qua các cung **SubsetOf** cho tới khi gặp thuộc tính cần tìm;
- giá trị đầu tiên gặp được là câu trả lời.

Ví dụ, Mary thừa hưởng thuộc tính có 2 chân nhờ thuộc hạng Persons. Cơ chế này **rất hiệu quả so với chứng minh định lý FOL đầy đủ** và là sức hấp dẫn chính của mạng ngữ nghĩa.

Ngoài kế thừa “cứng”, mạng ngữ nghĩa còn hỗ trợ **giá trị mặc định**: Một người mặc nhiên có hai chân trừ khi tồn tại ngoại lệ cụ thể—ngoại lệ sẽ ghi đè (override) khi phát hiện.

4. Ví dụ minh họa: “Chim hoàng yến”

Giả sử ta có các nút và cung sau:

Canary —MemberOf→ CanaryBirds

CanaryBirds —SubsetOf→ Birds

Birds —Property→ CanFly = TRUE

Theo cơ chế kế thừa, **Canary** sẽ thừa hưởng thuộc tính *CanFly = TRUE* từ hạng *Birds*, thỏa mãn nhận xét “Chim hoàng yến biết bay, có lông vũ”.

(Nguồn: Stuart J. Russell, Peter Norvig – *Artificial Intelligence: A Modern Approach*, 2021)

5. Ưu điểm nổi bật

Khía cạnh	Lợi ích	Trích dẫn
<i>Tính trực quan</i>	Dạng đồ thị dễ hình dung, hữu ích cho xây dựng tri thức tương tác.	
<i>Suy luận kế thừa</i>	Truy xuất thuộc tính theo liên kết MemberOf/SubsetOf nhanh và đơn giản.	
<i>Hỗ trợ giá trị mặc định & phi đơn điệu</i>	Cho phép “giả định” và rút lại thông tin khi có dữ liệu mâu thuẫn mới.	

6. Hạn chế và mở rộng

- **Giới hạn nhị nguyên:** liên kết chỉ mô tả *quan hệ hai ngôi*; để biểu diễn mệnh đề *Fly(Shankar, NY, Delhi, Yesterday)* phải “tái vật hoá” (reify) sự kiện thành một nút *FlyEvent* cùng nhiều cung nhẵn.
- **Kế thừa đa hạng (multiple inheritance)** dễ dẫn tới xung đột giá trị; cần cơ chế ưu tiên hoặc logic mặc định để xử lý.
- **Biểu thức lô-gic phức tạp** (phủ định, tồn tại, n-ary...) khó/nặng khi biểu diễn thuần mạng; do đó, **Description Logic** được phát triển để bổ sung cú pháp hình thức và đảm bảo khả năng suy diễn đa thức.

Mạng ngữ nghĩa cung cấp **một hình thức biểu diễn tri thức bán-hình thức, dựa trên đồ thị, hỗ trợ kế thừa và suy luận mặc định hiệu quả**. Dù không đạt sức biểu đạt của logic vị từ bậc nhất, nó **thích hợp cho các bài toán phân cấp khái niệm, hệ chuyên gia và các hệ thống cần suy luận nhanh trên đồ thị phân cấp**.

3.5. Xử lý Tri thức Bất định

3.5.1. Vấn đề với Logic Cổ điển

Logic mệnh đề và logic vị từ—dù rất mạnh trong các miền rạch ròi—đều **cam kết nhị trị**: mỗi mệnh đề chỉ có thể *Đúng* hoặc *Sai*. Trong các bài toán thế giới thực, ba hiện tượng sau khiến giả định ấy trở nên không thực tế:

Nguồn gốc của bất định	Minh hoạ	Hậu quả đối với suy luận lô-gic
Thiếu thông tin (ignorance)	Bác sĩ chỉ biết bệnh nhân <i>có thể</i> sâu răng	Không quyết được giá trị chân lý cho mệnh đề <i>Cavity(patient)</i>
Thế giới ngẫu nhiên (stochasticity)	Hành động “bắn” chỉ trúng đích 70 %	Suy luận kết quả hành động không chắc chắn
Cảm biến/khái niệm mơ hồ (vagueness)	“Vienna là <i>thành phố lớn?</i> ” tùy tiêu chí	Biên giới Đúng/Sai không sắc nét

“Vagueness is a real issue in any application of logic... fuzzy logic takes vagueness as a given and develops a theory around it.”

a) Ba trạng thái tri thức trong logic

Một tác nhân lô-gic chỉ cho phép: *tin đúng*, *tin sai*, hoặc *không biết*. Hàng thứ nhất của Hình 8.1 trích lại cho thấy hạn chế này:

Ngôn ngữ	Cam kết hiện tồn	Cam kết tri thức
Propositional	facts	true/false/unknown
First-order	facts, objects, relations	true/false/unknown
Probability	theory facts	degree of belief $\in [0,1]$
Fuzzy logic	facts with degree ...	interval value

b) Logic cổ điển trước bất định: “không nói được gì”

Trong thế giới Wumpus, từ *breeze(2,1)* ta **không biết** ô [2,2] có hồ hay không; ta chỉ liệt kê các mô hình còn khả dĩ. Để trả lời “xác suất có hồ bằng bao nhiêu?”, ta buộc phải rời bỏ logic nhị trị và dùng xác suất.

c) Xác suất như phần mở rộng của lô-gic

Khắc phục hạn chế trên, chương 12 của AIMA khẳng định:

- “Uncertainty arises because of both laziness and ignorance ... Probabilities express the agent’s inability to reach a definite decision regarding the truth of a sentence.”

Quan điểm này tương đồng với lập luận xác suất là “*calculus of belief*” trong tài liệu tổng hợp và với nhận định của Jaynes rằng “**phải mở rộng giá trị Đúng/Sai rời rạc thành độ khả tín liên tục**”.

d) Hệ quả: cần các khuôn khổ suy diễn mới

- **Xác suất & Lý thuyết quyết định**—kết hợp độ tin với độ ưa thích để chọn hành động tối ưu.
- **Logic mờ**—gán *độ đúng* cho mệnh đề, phục vụ các khái niệm mơ hồ.
- **Suy luận mặc định & phi đơn điệu**—cho phép “rút lại” kết luận khi có chứng cứ mới.

Như vậy, **logic cổ điển không bị “sai”**; nó chỉ **thiếu độ phân giải** cần thiết khi sự kiện không thể gán ngay giá trị Đúng/Sai. Các phương pháp xử lý bất định—đặc biệt là xác suất—cung cấp lớp biểu diễn giàu hơn, giúp hệ thống AI lập luận và ra quyết định hiệu quả trong môi trường phức tạp, ngẫu nhiên, và cảm biến không hoàn hảo.

3.5.2. Nhập môn Lý thuyết Xác suất

1. Vai trò của xác suất trong lập luận dưới bất định

Trong thế giới thực, nhiều mệnh đề không thể khẳng định dứt khoát *Đúng/ Sai*. Xác suất cung cấp một khuôn khổ số học để gán “mức độ tin cậy” (degree of belief) cho các mệnh đề ấy, nhờ đó suy luận của hệ thống trí tuệ nhân tạo trở nên **liên tục** và **mềm dẻo** hơn so với lô-gic kinh điển. Quan điểm này được coi là sự mở rộng tự nhiên của tư duy lô-gic sang “độ khả tín” (plausibility) của E. T. Jaynes (2003) .

2. Một số khái niệm nền tảng

Khái niệm	Ký hiệu	Diễn giải ngắn gọn
Không gian mẫu	Ω	Tập tất cả kết quả có thể của phép thử.
Biến ngẫu nhiên	X, Y	Hàm ánh xạ $\Omega \rightarrow$ tập giá trị T .
Phân bố xác suất	$p(x)$ hay $P(X=x)$	Hàm gán cho mỗi biến ngẫu nhiên một độ đo, thỏa $\sum p=1$ (rời rạc) hoặc $\int p \, dx=1$ (liên tục) .

3. Xác suất có điều kiện $P(A | B)$

Xác suất có điều kiện đo “mức tin” vào A khi **biết** B đã xảy ra. Dựa trên công thức Bayes tổng quát:

$$p(x | y) = p(y | x) p(x) / p(y) \quad (6.23)$$

Trong đó:

- **prior** $p(x)$: tri thức trước quan sát,
- **likelihood** $p(y | x)$: mô hình sinh dữ liệu,
- **evidence** $p(y)$: hệ số chuẩn hoá.

4. Quy tắc cộng (Sum Rule)

Quy tắc cộng còn gọi là **marginalisation property**:

$$\begin{aligned} p(x) &= \sum_{y \in Y} p(x, y) && \text{(discrete)} \\ &= \int_Y p(x, y) \, dy && \text{(continuous)} \end{aligned} \quad (6.20)$$

Nghĩa là “cộng (gộp)” (hoặc tích phân) các giá trị của biến cần khử để thu được phân bố biên

mong muốn. Thao tác này cho phép ta tính xác suất của một biến khi không (hoặc chưa) qua n sát biến kia.

5. Quy tắc nhân (Product Rule)

Quy tắc nhân liên hệ phân bố chung và phân bố có điều kiện:

$$p(x, y) = p(y | x) p(x) \quad (6.22)$$

:contentReference[oaicite:11]{index=11}CopyEdit

Từ đó suy ra ngay: $p(x, y) = p(x | y)p(y)$. Đây là nền tảng của hầu hết các thuật toán nội suy Bayes cũng như dẫn tới định lý Bayes (6.23).

6. Ví dụ Minh họa: Thao tác trên Phân bố Xác suất Đồng thời

Ví dụ 6.1 — Tính toán từ Bảng Phân bố Song biến

Để hiểu rõ cách các quy tắc xác suất cơ bản được áp dụng trong thực tế, chúng ta xét **Ví dụ 6.1** trích từ sách *Mathematics for Machine Learning*. Ví dụ này cung cấp một **bảng phân bố xác suất đồng thời (joint probability distribution)**, $p(x, y)$, cho hai biến ngẫu nhiên rời rạc X và Y . Đây là một ma trận, trong đó mỗi ô thể hiện xác suất để X và Y đồng thời nhận một cặp giá trị cụ thể.

Mục tiêu của bài tập này là từ bảng phân bố đồng thời đã cho, sinh viên phải thực hành tính toán hai loại phân bố quan trọng khác, qua đó củng cố việc áp dụng trực tiếp các quy tắc nền tảng của lý thuyết xác suất.

Các nhiệm vụ chính:

- **Tính Phân bố Biên (Marginal Distribution):**
 - **Mục tiêu:** Tìm phân bố xác suất của từng biến riêng lẻ, $p(x)$ và $p(y)$, bằng cách loại bỏ ảnh hưởng của biến còn lại.
 - **Phương pháp:** Áp dụng **quy tắc cộng (sum rule)**.
 - Để tính $p(x)$ cho một giá trị cụ thể của X , ta lấy tổng xác suất của tất cả các ô trong cùng một cột: $p(x) = \sum_y p(x, y)$.
 - Tương tự, để tính $p(y)$ cho một giá trị cụ thể của Y , ta lấy tổng xác suất của tất cả các ô trong cùng một hàng: $p(y) = \sum_x p(x, y)$.
- **Tính Phân bố có Điều kiện (Conditional Distribution):**
 - **Mục tiêu:** Tìm phân bố xác suất của một biến khi đã biết giá trị của biến còn lại, ví dụ $p(y|x)$ (xác suất của Y khi biết X).
 - **Phương pháp:** Áp dụng **quy tắc nhân (product rule)**, được biến đổi thành công thức tính xác suất có điều kiện:

$$p(y | x) = \frac{p(x, y)}{p(x)}$$

Để thực hiện phép tính này, sinh viên phải sử dụng kết quả của phân bố biên $p(x)$ đã tính ở bước trên.

Ví dụ này là một bài tập thực hành cốt lõi, giúp sinh viên chuyển từ lý thuyết sang ứng dụng. Bằng cách làm việc với một bảng dữ liệu cụ thể, người học có thể thấy rõ mối quan hệ giữa phân bố đồng thời, phân bố biên và phân bố có điều kiện, đồng thời hiểu sâu sắc vai trò của quy tắc cộng và quy tắc nhân trong việc suy diễn xác suất.

Ba công cụ trên—xác suất có điều kiện, quy tắc cộng, quy tắc nhân—cấu thành “ngữ pháp” của suy luận xác suất. Mọi kỹ thuật nâng cao (chuỗi Markov, mạng Bayes, học sâu xác suất, v. v.) đều quy chiếu về ba hằng đẳng thức cơ sở này. Vì vậy, nắm vững chúng là bước khởi đầu không thể thiếu trước khi đi sâu vào các mô hình xử lý bất định phức tạp hơn.

3.5.3. Định lý Bayes (Bayes' Rule)

1. Vai trò then chốt: Cầu nối giữa Nguyên nhân và Hậu quả

Trong thế giới thực, chúng ta thường dễ dàng suy luận theo chiều xuôi, từ **nguyên nhân** → **hậu quả**. Ví dụ, một bác sĩ có thể dễ dàng ước tính xác suất một bệnh nhân có triệu chứng *sốt* nếu biết người đó bị *cúm*. Tuy nhiên, bài toán mà các hệ thống thông minh thường phải đối mặt lại đi theo chiều ngược lại, từ **hậu quả** → **nguyên nhân**: “Nếu bệnh nhân bị sốt, xác suất họ bị cúm là bao nhiêu?”.

Định lý Bayes cung cấp một công thức toán học chặt chẽ để xây dựng chiếc “cầu” cho suy luận ngược này. Nó cho phép chúng ta cập nhật “niềm tin” của mình về một giả thuyết (nguyên nhân) dựa trên những bằng chứng (hậu quả) mà chúng ta quan sát được. Như Russell & Norvig đã nhấn mạnh: “Phương trình đơn giản này nằm bên dưới hầu hết hệ thống AI hiện đại về suy diễn xác suất”.

2. Công thức Chính tắc và Giải phẫu các Thành phần

Công thức Bayes cho phép tính xác suất của một giả thuyết h sau khi đã quan sát được bằng chứng e .

$$P(h | e) = \frac{P(e | h)P(h)}{P(e)}$$

Để hiểu sâu sắc công thức này, hãy “giải phẫu” từng thành phần của nó:

Ký hiệu	Tên gọi	Ý nghĩa và Vai trò
$P(h e)$	Xác suất Hậu nghiệm (Posterior Probability)	Đây là kết quả cuối cùng mà chúng ta muốn tìm: niềm tin được cập nhật vào giả thuyết h sau khi đã có trong tay bằng chứng e . Nó trả lời câu hỏi: "Bây giờ tôi nên tin vào h đến mức nào?".
$P(e h)$	Khả năng hợp lý (Likelihood)	Đây là xác suất quan sát được bằng chứng e , nếu giả sử giả thuyết h là đúng. Nó trả lời câu hỏi: "Giả thuyết h giải thích bằng chứng e tốt đến đâu?". Đây là thông tin suy luận theo chiều xuôi (nguyên nhân → hậu quả).

Ký hiệu	Tên gọi	Ý nghĩa và Vai trò
$P(h)$	Xác suất Tiên nghiệm (Prior Probability)	Đây là niềm tin ban đầu của chúng ta về giả thuyết h trước khi có bất kỳ bằng chứng nào. Nó đại diện cho kiến thức nền hoặc các thống kê có sẵn.
$P(e)$	Xác suất của Bằng chứng (Evidence)	Đây là tổng xác suất để quan sát được bằng chứng e , bất kể giả thuyết h đúng hay sai. Vai trò chính của nó là một hằng số chuẩn hóa (normalization constant) , đảm bảo rằng giá trị Posterior cuối cùng là một xác suất hợp lệ (nằm trong khoảng $[0, 1]$). Nó được tính bằng quy tắc tổng xác suất: $P(e) = \sum_h P(e h)P(h)$.

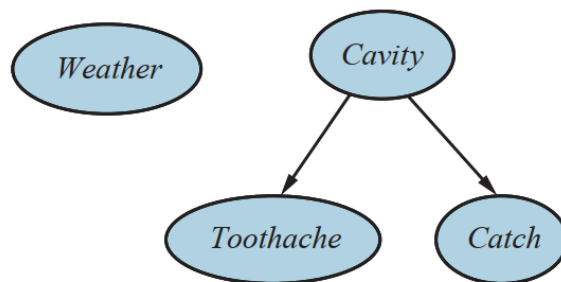
Như vậy, công thức có thể được diễn giải là: **Niềm tin cập nhật (Posterior) = [(Mức độ giải thích của giả thuyết) $\times$ (Niềm tin ban đầu)] / (Hằng số chuẩn hóa)**

Sách *Mathematics for Machine Learning* nhấn mạnh rằng giá trị **Posterior** tỷ lệ thuận với tích của **Likelihood** và **Prior**. Điều này có nghĩa là một giả thuyết sẽ được tin tưởng hơn nếu nó vừa có khả năng giải thích tốt bằng chứng, vừa có một niềm tin ban đầu hợp lý.

3. Từ Định lý Bayes đến Mạng Bayes

Định lý Bayes hoạt động hoàn hảo khi chúng ta chỉ có một giả thuyết và một bằng chứng. Tuy nhiên, các bài toán thực tế thường liên quan đến hàng chục hoặc hàng trăm biến số tương tác với nhau. Việc tính toán trực tiếp trở nên bất khả thi. Đây là lúc **Mạng Bayes (Bayesian Network)** phát huy vai trò.

Mạng Bayes là một đồ thị có hướng, biểu diễn một cách trực quan và hiệu quả các mối quan hệ phụ thuộc xác suất giữa một tập hợp các biến.



Hình 89. Mạng Bayes đơn giản với các quan hệ độc lập có điều kiện.

Hình ảnh này minh họa một mạng Bayes đơn giản, trong đó:

- **Các nút (Nodes):** Đại diện cho các biến ngẫu nhiên (*Weather*, *Cavity*, *Toothache*, *Catch*).
- **Các cung (Arcs):** Các mũi tên thể hiện mối quan hệ nhân quả trực tiếp. Ví dụ, mũi tên từ *Cavity* (Sâu răng) đến *Toothache* (Đau răng) và *Catch* (Que dò bị mắc) cho thấy *Cavity* là nguyên nhân trực tiếp gây ra hai hiện tượng này.
- **Sự độc lập (Independence):**

- *Weather* không có liên kết với các nút khác, cho thấy nó độc lập với các vấn đề nha khoa.
- Không có cung trực tiếp giữa *Toothache* và *Catch*, điều này thể hiện một khái niệm quan trọng: *Toothache* và *Catch* là **độc lập có điều kiện (conditionally independent)** khi đã biết *Cavity*. Tức là, nếu bạn đã chắc chắn có sâu răng, việc bạn bị đau răng không cung cấp thêm thông tin về việc que dò của nha sĩ có bị mắc hay không.

Mạng Bayes sử dụng cấu trúc đồ thị này để đơn giản hóa việc biểu diễn phân bố xác suất đồng thời. Thay vì một bảng khổng lồ, chúng ta chỉ cần các **Bảng Xác suất có Điều kiện (Conditional Probability Table - CPT)** tại mỗi nút, lưu trữ các giá trị $P(\text{Nút} \mid \text{Các nút cha})$. Những giá trị này chính là các thành phần **Likelihood** cần thiết để áp dụng Định lý Bayes trong một hệ thống phức tạp, cho phép suy luận hiệu quả ngay cả khi có nhiều biến.

Suy luận Bayes dưới góc nhìn Quy tắc nhân

Về bản chất, Định lý Bayes không phải là một công thức “thần kỳ” mà là một hệ quả trực tiếp và thanh lịch của **quy tắc nhân (product rule)** trong xác suất. Quy tắc nhân cho biết xác suất để hai sự kiện A và B cùng xảy ra có thể được viết theo hai cách:

- $P(A \wedge B) = P(A \mid B)P(B)$
- $P(A \wedge B) = P(B \mid A)P(A)$

Vì cả hai vế phải đều bằng $P(A \wedge B)$, chúng phải bằng nhau:

$$P(B \mid A)P(A) = P(A \mid B)P(B)$$

Bằng một thao tác đại số đơn giản là chia cả hai vế cho $P(A)$, chúng ta thu được công thức quen thuộc của Định lý Bayes:

$$P(B \mid A) = \frac{P(A \mid B)P(B)}{P(A)}$$

Cách nhìn này cho thấy Định lý Bayes không phải là một tiên đề, mà là một công cụ được suy ra một cách logic để “lật ngược” xác suất có điều kiện, cho phép chúng ta đi từ $P(A \mid B)$ đến $P(B \mid A)$.

Ví dụ: Chẩn đoán Viêm màng não

Để thấy sức mạnh và đôi khi là sự phản trực giác của suy luận Bayes, hãy xem xét ví dụ kinh điển sau trong chẩn đoán y khoa:

- **Bối cảnh:** Một bác sĩ đang xem xét khả năng một bệnh nhân bị viêm màng não (*meningitis*). Triệu chứng chính mà bác sĩ quan tâm là bệnh nhân bị cứng cổ (*stiff neck*).
- **Giả thuyết (h):** Bệnh nhân bị viêm màng não. Ký hiệu là m .
- **Bằng chứng (e):** Bệnh nhân bị cứng cổ. Ký hiệu là s .

Dữ liệu Y khoa đã biết (Các thành phần của công thức):

- **Prior $P(m)$:** Xác suất tiên nghiệm một bệnh nhân bất kỳ bị viêm màng não là rất thấp, giả sử $1/50,000 = 0.00002$.
- **Likelihood $P(s|m)$:** Xác suất bệnh nhân bị cứng cổ *nếu đã biết* họ bị viêm màng não là khá cao, giả sử **0.7** (70%).
- **Evidence $P(s)$:** Xác suất một bệnh nhân bất kỳ bị cứng cổ (vì nhiều lý do khác nhau như cúm, chấn thương...) là **1% = 0.01**.

Áp dụng Định lý Bayes để cập nhật niềm tin:

Bác sĩ muốn tính $P(m|s)$ – xác suất bệnh nhân thực sự bị viêm màng não khi đã biết họ bị cứng cổ.

$$P(m | s) = \frac{P(s | m)P(m)}{P(s)} = \frac{0.7 \times 0.00002}{0.01} = 0.0014$$

Diễn giải kết quả:

Mặc dù triệu chứng cứng cổ có vẻ là một dấu hiệu mạnh (likelihood = 70%), xác suất cuối cùng để bệnh nhân thực sự bị viêm màng não chỉ là **0.14%**. Kết quả có vẻ phản trực giác này cho thấy tầm quan trọng của **xác suất tiên nghiệm (prior)**. Vì bản thân căn bệnh viêm màng não cực kỳ hiếm, nên dù có một triệu chứng liên quan mạnh, khả năng mắc bệnh vẫn còn rất thấp. Đây là một lỗi tư duy phổ biến mà suy luận Bayes giúp chúng ta tránh được.

Ý nghĩa trong Trí tuệ Nhân tạo

Định lý Bayes không chỉ là một công thức toán học; nó là một triết lý nền tảng cho nhiều lĩnh vực trong AI hiện đại.

- **Cơ sở Thống nhất giữa Học và Suy diễn:** Hầu hết các mô hình học máy xác suất đều là các ứng dụng của Định lý Bayes ở quy mô lớn.
 - **Bộ phân loại Naïve Bayes:** Ước tính xác suất một đối tượng thuộc về một lớp dựa trên các đặc trưng của nó.
 - **Hồi quy Bayes (Bayesian Regression):** Tìm một phân bố xác suất cho các tham số của mô hình thay vì một giá trị duy nhất.
 - **Mạng Bayes:** Mô hình hóa một mạng lưới phức tạp các quan hệ nhân quả và thực hiện suy diễn trên đó. Về cơ bản, “học” một mô hình chính là quá trình tìm ra các tham số (ví dụ: các giá trị likelihood) từ dữ liệu, và “suy diễn” là quá trình sử dụng các tham số đó để đưa ra dự đoán về dữ liệu mới.
- **Cập nhật Tin cậy Động (Dynamic Belief Updating):** Công thức Bayes cho phép một hệ thống cập nhật niềm tin của mình một cách linh hoạt. Posterior của bước này sẽ trở thành Prior của bước tiếp theo. Vòng lặp *Prior* → *Quan sát* → *Posterior* (*Prior mới*) này là nền tảng cho các hệ thống theo dõi và lọc trạng thái, ví dụ như **bộ lọc Kalman (Kalman Filter)** được sử dụng trong robot, GPS, và hàng không vũ trụ để ước tính vị trí và vận tốc từ các cảm biến nhiễu.

- **Ra Quyết định dưới sự Bất định:** Trong thế giới thực, các tác tử hiếm khi có thông tin đầy đủ. Suy luận Bayes cung cấp một phân bố xác suất (posterior) trên các trạng thái có thể có của thế giới. Khi kết hợp phân bố này với một **hàm tiện ích (utility function)**—đo lường mức độ mong muốn của mỗi kết quả—tác tử có thể áp dụng **Lý thuyết Quyết định (Decision Theory)** để chọn ra hành động có **lợi ích kỳ vọng (expected utility)** cao nhất.

Ghi nhớ Quan trọng

Để áp dụng và hiểu đúng Định lý Bayes, cần ghi nhớ các điểm sau:

- **Vòng lặp Học-Cập nhật:** Định lý Bayes biến thông tin kinh nghiệm (likelihood) và niềm tin có sẵn (prior) thành một niềm tin đã được tinh chỉnh (posterior). Quá trình này tạo ra một vòng lặp học hỏi và cập nhật không ngừng, cho phép các hệ thống AI thích nghi với thông tin mới.
- **Phân biệt Likelihood và Probability:** Mặc dù $P(e|h)$ là một xác suất, nhưng trong ngữ cảnh Bayes, khi chúng ta cố định bằng chứng e và xem xét các giả thuyết h khác nhau, nó được gọi là hàm **likelihood** của h . Likelihood đo lường mức độ một giả thuyết “hợp lý” với dữ liệu, trong khi probability đo lường tần suất một sự kiện xảy ra.
- **Điều kiện Áp dụng:** Để áp dụng đúng công thức, xác suất của bằng chứng $P(e)$ phải lớn hơn 0. Quan trọng hơn, xác suất tiên nghiệm $P(h)$ không nên bằng 0 hoặc 1, trừ khi chúng ta hoàn toàn chắc chắn loại trừ hoặc khẳng định một khả năng. Gán prior bằng 0 đồng nghĩa với việc “đóng cửa” khả năng học hỏi về giả thuyết đó, bất kể có bao nhiêu bằng chứng được đưa ra.

CHƯƠNG 4: HỌC MÁY

4.1. Tổng quan về Học máy (Machine Learning)

4.1.1. Định nghĩa và bản chất

Học máy (Machine Learning – ML) là một nhánh của Trí tuệ Nhân tạo (AI) tập trung vào **xây dựng các hệ thống có khả năng tự động học hỏi và cải thiện hiệu năng thông qua kinh nghiệm** thay vì dựa vào các tập luật được lập trình sẵn. Arthur Samuel (1959) cung cấp định nghĩa kinh điển: “*Học máy là lĩnh vực nghiên cứu nhằm trao cho máy tính khả năng học mà không cần lập trình rõ ràng.*”

Nói cách khác, thay vì mô tả tường tận mọi quy tắc, **nhà phát triển cấp dữ liệu** và thuật toán sẽ rút trích mô hình/khung quy tắc trực tiếp từ dữ liệu đó. Cốt lõi của tiến trình học là **tối ưu hóa** một hàm mục tiêu (objective/utility) phản ánh mức độ “phù hợp” của mô hình đối với dữ liệu huấn luyện, đồng thời bảo đảm khả năng khái quát sang dữ liệu chưa từng gặp.

4.1.2. Ba thành phần cốt lõi của một hệ thống học máy

Phân tích toán học hiện đại cho thấy mọi hệ thống ML đều quy về ba khối chức năng chính:

Thành phần	Vai trò chính	Khái niệm then chốt
Dữ liệu (Data)	Biểu diễn thế giới thực dưới dạng vector đặc trưng số ; tiên quyết cho mọi thuật toán	Chuẩn hoá, tiền xử lý, trích chọn đặc trưng
Mô hình (Model)	Giả thuyết (function/distribution) mô phỏng quá trình sinh dữ liệu; càng “đúng” khi mô tả sâu sắc hiện tượng	Hàm dự đoán $f: \mathbf{x} \mapsto y$, mô hình xác suất, kiến trúc mạng...
Học (Learning)	Thuật toán tìm tập tham số tối ưu của mô hình dựa trên dữ liệu và một hàm mất mát/hiệu năng	Tối ưu hoá gradient, regularization, đánh giá chéo (cross-validation)

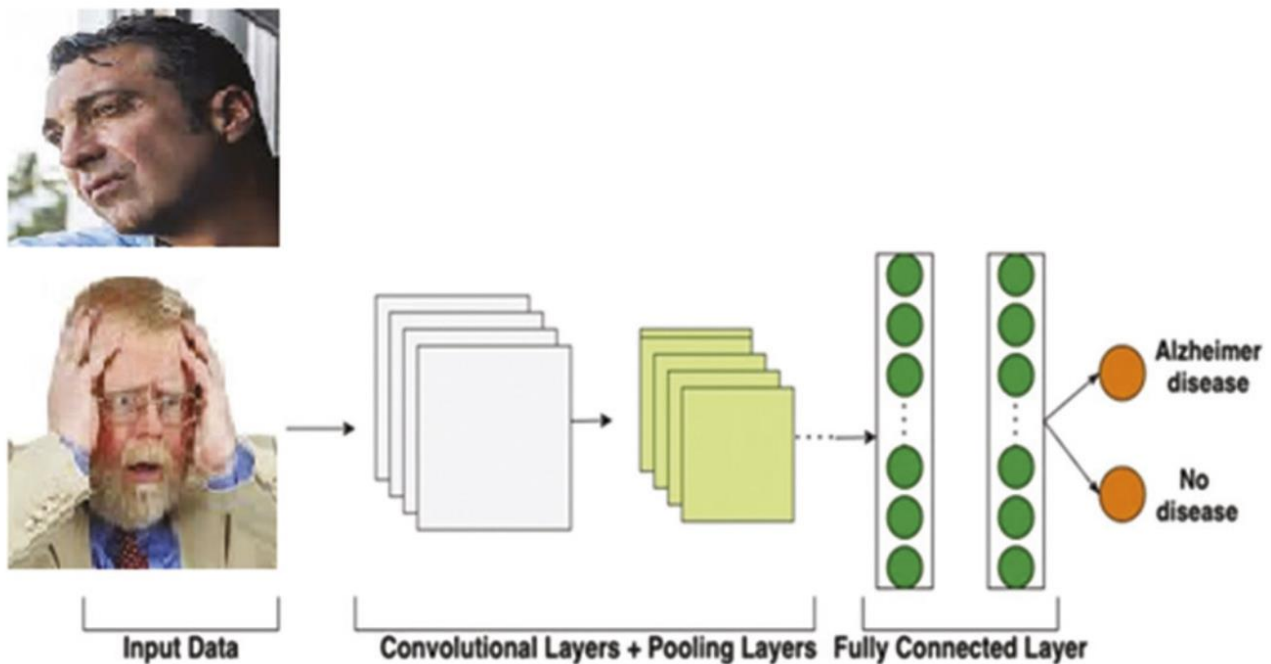
Ba thành phần nêu trên liên hệ chặt chẽ với “*bốn trụ cột*” của ML (Hồi quy, Giảm chiều, Ước lượng mật độ, Phân loại) được trình bày trong hình 1.1 của *Mathematics for Machine Learning*.

4.1.3. Ba loại hình học máy chủ đạo

Loại học	Đặc trưng dữ liệu	Mục tiêu & Nhiệm vụ tiêu biểu	Ví dụ ngắn
Học có giám sát (Supervised)	Tập dữ liệu được gán nhãn ($\mathbf{x}, y$)	Suy diễn hàm ánh xạ $f: \mathbf{x} \rightarrow y$. Nhiệm vụ: <i>Hồi quy</i> (giá trị liên tục) và <i>Phân loại</i> (nhãn rời rạc)	Lọc thư rác, dự đoán giá nhà
Học không giám sát (Unsupervised)	Không nhãn – chỉ có $\mathbf{x}$	Khám phá cấu trúc ẩn: <i>Phân cụm</i> , <i>Giảm chiều</i> , <i>Ước lượng mật độ</i>	K-Means nhóm hành vi khách hàng

Loại học	Đặc trưng dữ liệu	Mục tiêu & Nhiệm vụ tiêu biểu	Ví dụ ngắn
Học tăng cường (Reinforcement)	Dữ liệu phát sinh trong quá trình tương tác tác tử–môi trường	Tìm chính sách ($\pi(a s)$) tối đa hoá phần thưởng tích lũy theo thời gian	

Lưu ý sự phạm: Các dạng học trên tương ứng với ba “khâu vị” thuật toán được trình bày sâu hơn trong các tuần 12 → 14: *Tuần 12 – Học giám sát, Tuần 13 – Học không giám sát, Tuần 14 – Học tăng cường & mạng nơ-ron.*



Hình 90. Biểu cảm khuôn mặt và ánh mắt của bệnh nhân Alzheimer.

Trong học máy không giám sát, các biểu hiện trên khuôn mặt và ánh mắt – dù không được gán nhãn – có thể được phân tích để phát hiện các nhóm hành vi hoặc trạng thái tiềm ẩn. Ví dụ, các hình ảnh biểu cảm khuôn mặt từ bệnh nhân Alzheimer có thể được sử dụng trong các bài toán phân cụm để nhóm bệnh nhân theo các đặc điểm hành vi hoặc tình trạng sức khỏe. Bằng cách sử dụng các phương pháp học không giám sát, các đặc trưng này có thể được trích xuất và phân loại mà không cần sự can thiệp trực tiếp của nhãn dữ liệu. Hình ảnh minh họa Fig. 1.2 cung cấp một ví dụ thực tế về cách sử dụng mạng nơ-ron tích chập (CNN) để trích xuất các đặc trưng biểu cảm, phục vụ cho việc phân cụm bệnh nhân và phát hiện các trạng thái bệnh lý.

Để thực hiện điều này, mỗi hình ảnh xám (grayscale) sẽ được chuyển đổi thành một vector pixel có kích thước $N \times N$. Kết quả là ma trận dữ liệu chứa 100 cột, mỗi cột chứa một hình ảnh. Quá trình tiếp theo là ước tính hình ảnh trung bình của biểu cảm khuôn mặt từ tập dữ liệu, với việc loại trừ một hình ảnh để kiểm tra.

4.1.4. Ví dụ minh họa: Dữ liệu nhân sự dạng bảng



Hình 91. Cơ sở dữ liệu ảnh khuôn mặt.

Hệ thống nhận diện khuôn mặt đã sử dụng Mạng Nơ-ron Tích chập (CNN) trong nhiều vấn đề nhận dạng khuôn mặt. Với sự đơn giản và tốc độ, thuật toán CNN vượt trội so với các hệ thống nhận diện khuôn mặt khác. Trong hình ảnh Fig. 1.3, cơ sở dữ liệu khuôn mặt được hiển thị, với 100 bức ảnh từ 10 người, mỗi người có 10 bức ảnh khác nhau.

Mỗi hình ảnh xám (grayscale) sẽ được chuyển đổi thành một vector có kích thước $N \times N$ pixel. Kết quả là ma trận dữ liệu để áp dụng CNN có 100 cột, mỗi cột chứa một hình ảnh. Quá trình tiếp theo là ước tính hình ảnh trung bình của các biểu cảm khuôn mặt, loại trừ một bức ảnh để kiểm tra.

Tương tự như vậy, trong Bảng 8.1 của Mathematics for Machine Learning, dữ liệu nhân sự thô được chuyển thành ma trận số, đây là bước đầu tiên trong bất kỳ dự án học máy nào. Dữ liệu nhân sự được cấu trúc thành các trường như tên, giới tính, học vị, mã bưu điện, tuổi và mức lương hàng năm, từ đó có thể chuyển đổi thành dạng số học để áp dụng vào các mô hình học máy.

Bảng 1. trong Mathematics for Machine Learning minh họa cách chuyển dữ liệu thô thành ma trận số – bước đầu tiên của mọi dự án ML.

Name	Gender	Degree	Postcode	Age	Annual Salary
Aditya	M	MSc	W21BG	36	89 563
Bob	M	PhD	EC1A1BA	47	123 543
Chloé	F	BEcon	SW1A1BH	26	23 989

Name	Gender	Degree	Postcode	Age	Annual Salary
...	...	...	...	...	...

4.1.5. Tóm tắt chương

- Học máy trao cho máy tính khả năng **suy diễn mô hình** từ dữ liệu thay vì lập trình tĩnh.
- Mọi hệ thống ML gồm **Dữ liệu** → **Mô hình** → **Thuật toán học**, vận hành xoay quanh bài toán tối ưu hoá.
- Ba **loại hình học** – có giám sát, không giám sát, tăng cường – bao phủ hầu hết ứng dụng hiện đại, từ phân loại thư rác đến điều khiển tác tử trò chơi.

Kiến thức này đặt nền cho các chương tiếp theo, nơi chúng ta đi sâu vào từng loại thuật toán và thực nghiệm với **NumPy, Pandas, Scikit-learn** theo mục tiêu đã đề ra.

4.1.2. Ba loại hình Học máy chính

4.1.2.1. Học có giám sát (Supervised Learning)

Khái niệm: Trong học có giám sát, thuật toán được cung cấp **các cặp dữ liệu đã gán nhãn** (x, y) và phải học một hàm ánh xạ $f: x \rightarrow y$. Nói cách khác, tác tử quan sát đầu vào–đầu ra và học một hàm dự đoán thích hợp cho dữ liệu chưa từng gặp.

Mô hình hóa toán học: Cho tập huấn luyện gồm N cặp $(x_1, y_1), \dots, (x_N, y_N)$ sinh ra từ hàm ẩn $y = f(x)$. Nhiệm vụ là tìm một giả thuyết h (trong không gian giả thuyết H) sao cho $h \approx f$ trên toàn bộ dữ liệu. Khung nhìn hiện đại gọi đây là **bài toán tối ưu hóa hàm mất mát** trên dữ liệu gán nhãn, hay *Empirical Risk Minimization*.

Quy trình tổng quát

- **Tiền xử lý & trích chọn đặc trưng:** biến đổi dữ liệu thô (ảnh, email, cảm biến, ...) thành vector số.
- **Chọn không gian mô hình H** (đường thẳng, đa thức, mạng nơ-ron, ...).
- **Huấn luyện:** tìm bộ tham số θ^* để hàm mất mát trên tập huấn luyện cực tiểu.
- **Đánh giá & hiệu chỉnh:** kiểm tra trên tập kiểm định, điều chỉnh **quá khớp/thiếu khớp** (over-/under-fitting).



Hình 92. Thuật toán Hồi quy Logistic.

Trong học có giám sát, hồi quy Logistic là một thuật toán phân loại tuyến tính đơn giản nhưng mạnh mẽ, thường được sử dụng để phân biệt giữa hai lớp dữ liệu. Ví dụ điển hình là bài toán phân loại thư rác trong email, nơi chúng ta cần phân loại các email thành hai nhóm: thư rác (spam) và không phải thư rác (ham).

Hàm kích hoạt sigmoid trong hồi quy Logistic đảm bảo rằng các xác suất dự đoán luôn nằm trong khoảng $[0, 1]$, giúp đưa ra quyết định phân loại rõ ràng. Quá trình huấn luyện mô hình hồi quy Logistic bao gồm ba bước chính:

Tiền xử lý dữ liệu: Biến dữ liệu thô (như nội dung email) thành các đặc trưng số, ví dụ như sử dụng kỹ thuật TF-IDF (Term Frequency-Inverse Document Frequency) để biến văn bản thành các vector số học.

Huấn luyện mô hình: Áp dụng thuật toán hồi quy Logistic để tìm ra các tham số tối ưu cho mô hình, sao cho hàm mất mát trên tập huấn luyện được tối thiểu hóa.

Đánh giá và hiệu chỉnh mô hình: Kiểm tra mô hình trên tập kiểm định và điều chỉnh tham số để cải thiện độ chính xác, tránh tình trạng quá khớp (overfitting) hoặc thiếu khớp (underfitting).

Hồi quy Logistic là một trong những thuật toán cơ bản trong học có giám sát, đặc biệt hữu ích trong các bài toán phân loại với hai nhãn, chẳng hạn như dự đoán xem một email có phải là thư rác hay không.

4.1.2.2. Phân loại bài toán trong học có giám sát

Loại bài toán	Đầu ra y	Mục tiêu	Minh họa ngắn
Hồi quy (<i>Regression</i>)	Giá trị liên tục	Xấp xỉ hàm số $y = f(x)$.	Dự đoán giá nhà từ diện tích, vị trí.
Phân loại (<i>Classification</i>)	Nhãn rời rạc	Gán mỗi x vào một lớp $c \in \{1, \dots, K\}$.	Lọc email <i>spam</i> / <i>not spam</i> .

Ví dụ 4.1 Hồi quy tuyến tính: Giả sử ta có N quan sát $x_n \in \mathbb{R}^D$ cùng nhãn vô hướng y_n . Chọn lớp hàm **tuyến tính–phụ thuộc tham số**

$$f(x, \theta) = \theta^\top x,$$

để ước lượng θ^* sao cho $f(x_n, \theta^*) \approx y_n$ với mọi n .

4.1.2.3. Ví dụ thực tiễn

Bộ lọc thư rác

- **Dữ liệu:** Văn bản email x cùng nhãn $y \in \{\text{spam}, \text{ham}\}$.
- **Mục tiêu:** Học hàm f gán nhãn cho email mới.
- **Quy trình:**
 - Chuyển email thành vector (TF-IDF, BOW).
 - Huấn luyện mô hình (Logistic Regression, SVM).
 - Đánh giá bằng **độ chính xác, precision/recall**.

Ví dụ này minh họa rõ tính chất “hàm ánh xạ $f: x \rightarrow y$ ” đã nêu.

4.1.2.4. Kết luận

Học có giám sát là **trụ cột đầu tiên** của học máy: từ các cặp dữ liệu gán nhãn, thuật toán suy diễn mô hình dự đoán. Hai tiểu loại – **hồi quy** và **phân loại** – bao phủ phần lớn ứng dụng AI hiện đại, từ dự báo kinh tế đến nhận dạng hình ảnh. Nội dung chi tiết về các thuật toán (Hồi quy tuyến tính, K-NN, SVM, v.v.) sẽ được trình bày trong Mục 4.2.

4.1.2.2. Học không giám sát (*Unsupervised Learning*)

1. Khái niệm, mục tiêu và bối cảnh

Học không giám sát là quá trình **khai thác các mẫu, cấu trúc ẩn từ một tập dữ liệu không gán nhãn** chỉ có x . Các thuật toán loại này “học hoàn toàn từ các đầu vào chưa gán nhãn” và thường sinh ra **mô hình sinh (generative model)** chứ không đơn thuần dự đoán nhãn. Hai định hướng chính của học không giám sát bao gồm:

- **Học biểu diễn** – xây dựng các đặc trưng mới giúp những tác vụ sau (như phân loại) trở nên dễ dàng hơn.
- **Mô hình sinh** – ước lượng phân bố xác suất $P(x)$ hoặc $P(x, z)$ (trong đó z là biến tiềm ẩn) để có thể sinh ra dữ liệu mới giống dữ liệu gốc.

2. Bài toán Phân cụm (Clustering) – tâm điểm của học không giám sát

Phân cụm nhằm **chia N quan sát vào K cụm sao cho các điểm cùng cụm “gần” nhau nhất**. Tiêu chí gần thường dùng **tổng bình phương khoảng cách trong cụm (Within-Cluster Sum of Squares – WSS)**

$$WSS_k = \sum_{i=1}^{n_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

trong đó $\boldsymbol{\mu}_k$ là centroid của cụm thứ k .

3. Thuật toán K-Means

K-Means là thuật toán phân cụm phổ dụng nhờ tính đơn giản và khả năng mở rộng. Ý tưởng: lặp lại hai bước *Gán cụm* và *Cập nhật tâm* cho tới khi ổn định. Thuật toán gốc (trích nguyên bản):

```

1: procedure k-means(Xs, Es, k)
2: Inputs
3: Xs set of features,  $X = \{X_1, \dots, X_n\}$ 
4: Es set of training examples
5: k number of clusters
6: Output
7: cluster: function from examples to clusters
8: prediction: function from feature and cluster to a value
9: Local
10: integer cc[c], cc_new[c] # cluster counts
11: real fs[j,c], fs_new[j,c] # sums of feature values
12: Boolean stable
13: Initialize fs and cc randomly based on data
14: define prediction(j,c) = fs[j,c]/cc[c]
15: define cluster(e) = arg min_c  $\sum_j (\text{prediction}(j,c) - X_j(e))^2$ 
16: repeat
17:   fs_new and cc_new  $\leftarrow 0$ 
18:   for each example e  $\in$  Es do
19:     c  $\leftarrow$  cluster(e)
20:     cc_new[c]  $\leftarrow$  cc_new[c] + 1
21:     for each feature  $X_j \in$  Xs do
22:       fs_new[j,c]  $\leftarrow$  fs_new[j,c] +  $X_j(e)$ 
23:   stable  $\leftarrow$  (fs_new = fs) and (cc_new = cc)
24:   fs  $\leftarrow$  fs_new ; cc  $\leftarrow$  cc_new

```

25: until stable 26: return cluster, prediction
--

Độ phức tạp: $O(T \times K \times N \times d)$ với T số vòng lặp và d số chiều.

Ưu điểm / Hạn chế chính

- Nhanh, dễ triển khai, kết quả trực quan .
- Nhạy với khởi tạo centroid và giả thiết các cụm hình cầu, phương sai bằng nhau .
- Cần xác định K trước; thường dùng phương pháp **Elbow** hoặc **BIC** để chọn K tối ưu .

4. Ví dụ thực hành: Phân cụm khách hàng trung tâm mua sắm

Bộ dữ liệu *Mall_Customers.csv* (thu nhập hàng năm & điểm chi tiêu) không có nhãn đích. Triển khai *K-Means* (**n_clusters = 5**) bằng *Scikit-learn* giúp chia khách hàng thành các phân khúc tiêu dùng đặc trưng – từ đó ngân sách marketing được tối ưu .

5. Mở rộng: Các kỹ thuật khác trong học không giám sát

- **EM cho Soft-Clustering** – gán điểm cho nhiều cụm bằng phân bố xác suất .
- **Autoencoder & PCA** – giảm chiều và học biểu diễn nén .
- **Density Estimation** – ước lượng $P(x)$ khi dữ liệu đầy đủ .

6. Kết luận

Học không giám sát – tiêu biểu là **phân cụm** – mang lại khả năng “**khám phá tri thức**” trong những miền dữ liệu chưa gắn nhãn, từ phân đoạn khách hàng, phân tích y sinh cho đến phát hiện bất thường. Nắm vững K-Means và tiêu chí đánh giá (WSS, Silhouette, BIC) là bước khởi đầu thiết yếu trước khi tiếp cận các phương pháp nâng cao như EM, tự mã hóa biến đổi (VAE) hoặc mô hình hỗn hợp Gaussian.

4.1.2.3. Học tăng cường (Reinforcement Learning – RL)

1. Khái niệm và mô hình tác tử–môi trường

Học tăng cường nghiên cứu **cách một tác tử (agent) học thông qua tương tác tuần tự với một môi trường (environment)**. Ở mỗi bước, tác tử quan sát trạng thái s , chọn hành động a , nhận phần thưởng tức thì r và chuyển sang trạng thái mới s' . Mục tiêu dài hạn là **tối đa hóa tổng phần thưởng tích lũy** trên toàn bộ chuỗi tương tác .

2. Khuôn khổ toán học: Markov Decision Process (MDP)

Một bài toán RL thường được mô hình hoá thành MDP gồm bộ 5 thành phần (S, A, P, R, γ) :

- S – tập trạng thái khả dĩ.
- A – tập hành động khả dĩ.
- $P(s'|s, a)$ – hàm xác suất chuyển trạng thái.

- $R(s, a, s')$ – hàm thưởng tức thì.
- $\gamma \in [0,1)$ – hệ số chiết khấu, làm giảm giá trị của thưởng xa trong tương lai.

Phần thưởng tích lũy (hay **lợi tức** G_t) tại thời điểm t được định nghĩa

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

3. Chính sách và mục tiêu tối ưu

Chính sách $\pi(a|s)$ là chiến lược hành động của tác tử tại mỗi trạng thái. **Chính sách tối ưu** π^* thỏa mãn $\pi^* = \arg \max_{\pi} [G_t | \pi]$, nghĩa là đem lại giá trị kỳ vọng cao nhất. Hai đường hướng chính để đạt π^* là **học hàm hành động–giá trị** $Q(s, a)$ (ví dụ Q-Learning) hoặc **tìm kiếm trực tiếp trong không gian chính sách** (policy search) .

4. Thuật toán cơ bản: Q-Learning (không mô hình)

Pseudocode gốc của Q-Learning trong *AIMA* minh họa cơ chế cập nhật giá trị :

```
function Q-LEARNING-AGENT(percept) returns an action
  inputs: percept = (s', r)
  persistent: Q[s,a]  ▷ bảng giá trị
              Nsa[s,a] ▷ số lần (s,a)
              (s, a)  ▷ trạng thái – hành động trước
  if s ≠ null then
    Nsa[s,a] ← Nsa[s,a] + 1
    Q[s,a] ← Q[s,a] +  $\alpha$ (Nsa[s,a]) · (r +  $\gamma \cdot \max_{a'} Q[s',a'] - Q[s,a]$ )
    (s,a) ← (s',  $\arg\max_{a'} f(Q[s',a'], Nsa[s',a'])$ )
  return a
```

Thuật toán dần hội tụ tới Q^* khi (α, γ) thỏa điều kiện giảm đủ chậm và không gian trạng thái hữu hạn.

```

1: controller Q-learning(S, A, γ, alpha_fun)
2:   Inputs
3:     S is a set of states
4:     A is a set of actions
5:      $\gamma$  the discount
6:     alpha_fun is a function to compute step size from counts
7:   Local
8:     real array  $Q[S, A]$ 
9:     integer array  $N[S, A]$ 
10:    states  $s, s'$ 
11:    action  $a$ 
12:    initialize  $Q[S, A]$  arbitrarily
13:    initialize  $N[S, A]$  to 0
14:    observe current state  $s$ 
15:    repeat
16:      select an action  $a$ 
17:      do( $a$ )
18:       $N[s, a] := N[s, a] + 1$ 
19:       $\alpha := \text{alpha\_fun}(N[s, a])$ 
20:      observe reward  $r$  and state  $s'$ 
21:       $Q[s, a] := Q[s, a] + \alpha * (r + \gamma * \max_{a'} Q[s', a'] - Q[s, a])$ 
22:       $s := s'$ 
23:    until termination()

```

Hình 93. Bộ điều khiển *Q-learning*.

Hình 93 minh họa một bộ điều khiển *Q-learning*, nơi tác tử vừa thực hiện hành động vừa học từ các phản hồi nhận được. Lệnh "*do(a)*" ở dòng 17 chỉ định hành động *aaa* mà bộ điều khiển gửi đến cơ thể tác tử. Phần thưởng và trạng thái kết quả là các quan sát (percepts) mà bộ điều khiển nhận được từ môi trường.

Thuật toán trong Hình 93 cũng duy trì một mảng $N[s, a]$, dùng để đếm số lần hành động *a* được thực hiện tại trạng thái *s*. Hàm **alpha fun** tính toán giá trị α từ số lần hành động được thực hiện. Ví dụ, hàm $\alpha(c)=10/(9+c)$ thường hoạt động hiệu quả (xem bài tập 13.6, trang 607). Khi giá trị α được cố định, mảng *N* không cần phải duy trì (mặc dù nó cũng được sử dụng cho một số chiến lược khám phá).

Bộ điều khiển *Q-learning* học (một xấp xỉ của) hàm Q^* tối ưu miễn là tác tử khám phá đủ và không có giới hạn về số lần thử hành động trong bất kỳ trạng thái nào (tức là nó không chỉ thực hiện một tập hợp hành động cố định trong mỗi trạng thái).

5. Ví dụ minh họa: Đào tạo tác tử chơi cờ vua

Trong cờ vua, tác tử RL nhận **thưởng +1 khi thắng, 0 khi hoà, -1 khi thua**. Mọi thế cờ trung gian không mang thưởng; do đó phản hồi (reward) **không đến tức thì** mà chỉ xuất hiện cuối ván – khiến thuật toán phải dựa vào cơ chế **bootstrapping** để lan truyền giá trị ngược về các nước đi trước. Phương pháp này được hiện thực hoá trong các hệ thống hiện đại như **AlphaZero**, nơi mạng nơ-ron sâu ước lượng $V(s)$ và $\pi(a|s)$ đồng thời, kết hợp MCTS để tối ưu hoá chính sách.

6. Tóm tắt

- RL xem *học* là **quyết định nối tiếp dưới bất định**, tác tử tự cải thiện bằng trải nghiệm.
- MDP cung cấp ngôn ngữ hình thức cho môi trường, trong đó mục tiêu là tìm π^* .
- Thuật toán đại diện: **Q-Learning** (học giá trị) và **Policy Search** (tối ưu trực tiếp).
- Bài toán thực tế – từ **cờ vua** đến **điều khiển robot** – minh chứng sức mạnh của RL trong việc tối đa hoá phần thưởng dài hạn dưới phản hồi chậm trễ.

4.2.1. Hồi quy tuyến tính (Linear Regression)

(a) Bài toán và giả định

Hồi quy tuyến tính là bài toán **dự đoán một biến mục tiêu liên tục** y từ một (hoặc nhiều) biến đầu vào x . Giả định cốt lõi là **quan hệ giữa đầu vào và đầu ra xấp xỉ tuyến tính**: nếu vẽ dữ liệu trên mặt phẳng toạ độ, các điểm quan sát phân bố dọc quanh một đường thẳng (hoặc siêu phẳng trong không gian nhiều chiều).

(b) Mô hình (Hypothesis)

Với m đặc trưng $X_1, \dots, X_m$, mô hình tuyến tính viết dưới dạng *affine*

$$\hat{y}(e) = w_0 + w_1 X_1(e) + \dots + w_m X_m(e) = \sum_{i=0}^m w_i X_i(e), \quad X_0 \equiv 1$$

trong đó $w = (w_0, \dots, w_m)$ là **vector tham số** cần học.

(c) Hàm mất mát (Loss Function)

Để đo độ “khớp” giữa dự đoán và quan sát, ta dùng **Sai số bình phương trung bình** (Mean Squared Error, MSE)

$$\mathcal{L}_{\text{MSE}} = \frac{1}{|E_s|} \sum_{e \in E_s} (\hat{y}(e) - y(e))^2.$$

MSE tương đương với *squared loss* ở mỗi ví dụ, trong đó $\text{loss}(p, a) = (p - a)^2$. Việc **tối thiểu hoá MSE** là mục tiêu huấn luyện mô hình.

(d) Huấn luyện bằng Gradient Descent

Do hàm mất mát khả vi, ta cập nhật trọng số lặp lại theo **thuật toán descent dốc (gradient descent)**

$$w_i := w_i - \eta \frac{\partial}{\partial w_i} \mathcal{L}_{\text{MSE}} = w_i - \eta \frac{1}{|E_s|} \sum_{e \in E_s} (\hat{y}(e) - y(e)) X_i(e),$$

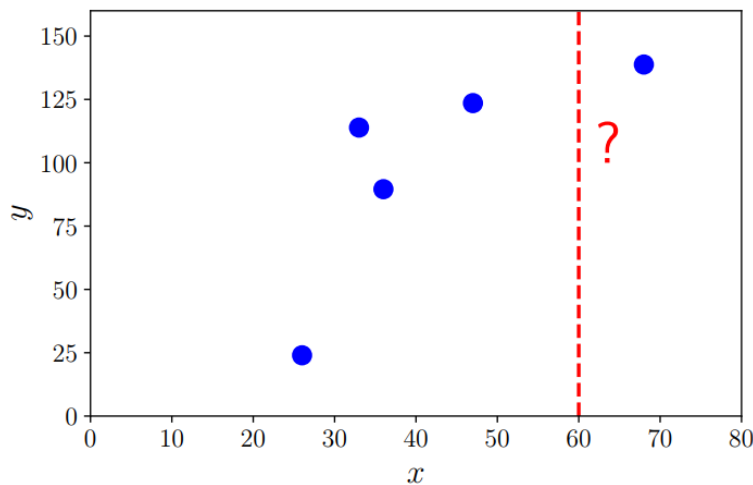
trong đó η là **tốc độ học (learning rate)**. Ở quy mô dữ liệu lớn, ta thường lấy *mini-batch* ngẫu nhiên $\rightarrow$ **Stochastic Gradient Descent (SGD)** giúp hội tụ nhanh mà vẫn xấp xỉ nghiệm tối ưu.

(e) Trực quan hoá

Một ví dụ trực quan về hồi quy tuyến tính với dữ liệu và đường hồi quy cực đại khả năng (Maximum Likelihood Estimate) được minh hoạ trong **Hình 94**. Hình cho thấy:

- **Tập huấn luyện** (các điểm chéo).
- **Những đường thẳng ứng viên.**
- **Đường tối ưu** giảm thiểu sai số.

Qua hình, sinh viên dễ quan sát ảnh hưởng của thay đổi tham số w tới đường khớp dữ liệu, cũng như ý nghĩa hình học của MSE (tổng diện tích bình phương khoảng cách thẳng đứng).



Hình 94. Dữ liệu giả lập cho hồi quy tuyến tính.

Hình 94 minh họa một bộ dữ liệu giả lập cho hồi quy tuyến tính. Dữ liệu huấn luyện được thể hiện dưới dạng các cặp (x_n, y_n) với $n=1, \dots, 10$, trong đó x là độ tuổi và y là mức lương hàng năm. Đặc biệt, hình này cũng làm nổi bật một điểm dữ liệu $x=60$ (độ tuổi 60), nơi một dự đoán được yêu cầu, mặc dù điểm này không có mặt trong dữ liệu huấn luyện.

Hình minh họa cách mà mô hình hồi quy tuyến tính sử dụng để tìm đường thẳng tốt nhất (được xác định bởi các tham số slope và intercept) nhằm dự đoán giá trị y (lương) dựa trên các giá trị x (tuổi), đồng thời thể hiện sự tương quan giữa các biến độc lập và phụ thuộc trong bài toán hồi quy tuyến tính.

Tóm lược kiến thức

- Hồi quy tuyến tính giả định quan hệ $x \leftrightarrow y$ tuyến tính.
- Mô hình: $\hat{y} = w^T x$.
- Mất mát: $MSE \rightarrow$ tối thiểu hoá qua gradient descent/SGD.
- Kết quả: tìm được **đường/siêu phẳng tối ưu** để dự đoán giá trị liên tục.

4.2.2. Phân loại: **K-Hàng xóm gần nhất (K-Nearest Neighbors – K-NN)**

1. Bài toán & bối cảnh

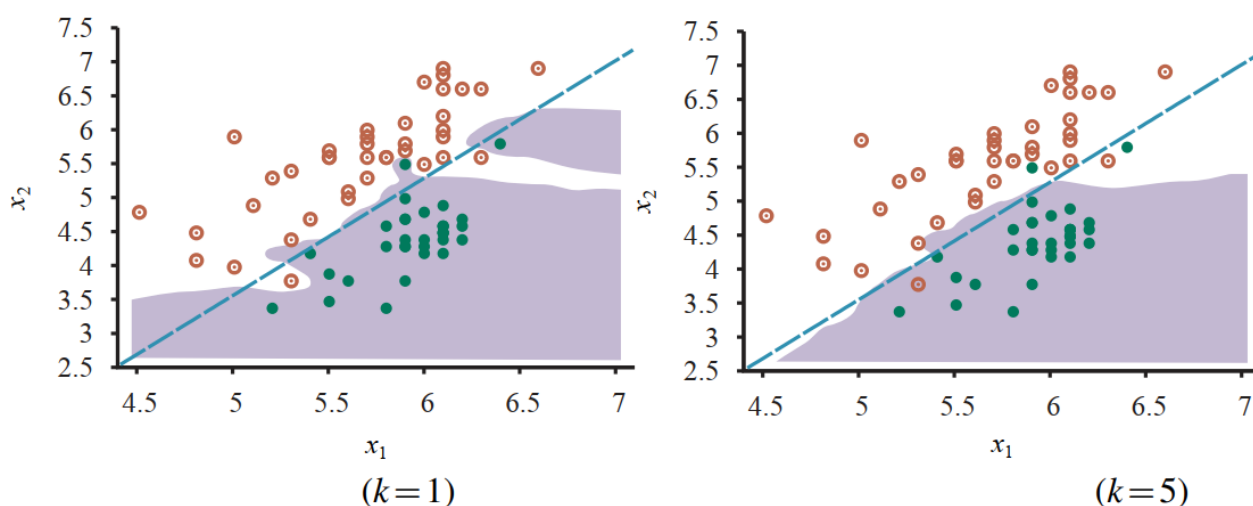
Cho một điểm dữ liệu truy vấn $\mathbf{x}_q$ chưa biết nhãn, nhiệm vụ của K-NN là **gán $\mathbf{x}_q$ vào một lớp trong số các lớp mà tập huấn luyện đã biết**. Phương pháp thực hiện điều này bằng cách *so sánh trực tiếp* $\mathbf{x}_q$ với toàn bộ các quan sát đã gán nhãn và “mượn” nhãn của k **mẫu gần nhất** trong không gian đặc trưng.

2. Đặc điểm “học lười” (*Lazy Learning*)

K-NN **không xây dựng mô hình khái quát trong giai đoạn huấn luyện**; nó chỉ *lưu toàn bộ dữ liệu* rồi trì hoãn việc tính toán cho tới lúc cần dự đoán. Bởi vậy, K-NN còn được xếp vào nhóm *instance-based learning* (Fix & Hodges, 1989) .

3. Quy trình thuật toán K-NN

Bước	Mô tả
(1)	Chuẩn hoá & mã hoá dữ liệu để các thước đo khoảng cách có ý nghĩa.
(2)	Tính khoảng cách giữa $\mathbf{x}_q$ và tất cả điểm huấn luyện (thường dùng <i>Euclid</i> , <i>Manhattan</i> hoặc tổng quát hơn là chuẩn L_p) .
(3)	Chọn tập k lân cận gần nhất $NN(k, \mathbf{x}_q)$.
(4)	Bỏ phiếu đa số (classification) hoặc lấy trung bình/median (regression) để suy ra nhãn $\hat{y}$.



Hình 95. Biên quyết định của mô hình K-Nearest Neighbors.

Hình 95 minh họa trực quan tác động của việc lựa chọn tham số k trong thuật toán K-NN đến khả năng khái quát hoá của mô hình phân loại:

- **(a)** Với $k=1$, mô hình K-NN tạo nên các đường biên phân lớp rất sắc nét, ôm sát từng điểm dữ liệu huấn luyện – điều này dẫn đến hiện tượng **quá khớp (overfitting)**, mô hình phản ứng mạnh với nhiễu và các điểm ngoại lệ.

- **(b)** Với $k=5$, đường biên quyết định trở nên mượt hơn, mô hình ổn định hơn và hiện tượng quá khớp được loại bỏ – từ đó cải thiện độ tổng quát khi áp dụng cho dữ liệu mới.

Hình ảnh này minh họa bản chất "**học theo ví dụ**" (**instance-based learning**) của K-NN – một mô hình phi tham số (non-parametric) không xây dựng biểu diễn khái quát, mà dựa hoàn toàn vào dữ liệu huấn luyện để đưa ra quyết định. Phương pháp đơn giản này lưu toàn bộ dữ liệu và khi cần, so sánh đầu vào với các ví dụ đã biết để suy ra đầu ra.

4. Khoảng cách & Chuẩn hoá

Khoảng cách **Minkowski**

$$d_p(\mathbf{x}, \mathbf{z}) = \left(\sum_i |x_i - z_i|^p \right)^{\frac{1}{p}}$$

phủ kín các trường hợp **Euclid** $p = 2$, **Manhattan** $p = 1$ và **Hamming** cho dữ liệu nhị phân. Lựa chọn chuẩn và *scale* đặc trưng (z-score, min-max) có ảnh hưởng trực tiếp tới độ chính xác của K-NN.

5. Chọn giá trị k

- k quá nhỏ $\rightarrow$ mô hình “nhảy” với nhiều, *overfitting* (hình 19.19a, $k = 1$).
- k vừa đủ lớn $\rightarrow$ đường biên quyết định mượt hơn, giảm nhiễu (hình 19.19b, $k = 5$).
- k quá lớn $\rightarrow$ *underfitting* do pha loãng ảnh hưởng của hàng xóm thật sự liên quan.

Như các phương pháp khác, **cross-validation** là kỹ thuật chuẩn để xác định k tối ưu.

Hình 95 Biên quyết định của K-NN với $k = 1$ (trái) và $k = 5$ (phải) minh họa hiện tượng quá khớp và cải thiện khi tăng k .

6. Độ phức tạp tính toán

- **Huấn luyện:** $O(1)$ – chỉ lưu dữ liệu.
- **Dự đoán:** $O(N \cdot d)$ cho tìm kiếm tuyến tính, với N số mẫu và d số chiều. Có thể giảm xuống $O(\log N)$ dùng **k-d tree** (phù hợp khi $d \lesssim 20$) hoặc **Locality-Sensitive Hashing** cho dữ liệu rất cao chiều.

7. Ưu & nhược điểm

Ưu điểm	Nhược điểm
Dễ hiểu, không giả định phân bố dữ liệu	Tính toán nặng khi dự đoán; cần cấu trúc dữ liệu tối ưu
Hỗ trợ cả phân loại & hồi quy	Nhạy với nhiều và thước đo khoảng cách
Thêm dữ liệu mới không cần huấn luyện lại	Khó xử lý dữ liệu thưa (sparse) , cao chiều

8. Ví dụ minh hoạ – Phân loại thư điện tử

Giả sử tập huấn luyện gồm các email đã gắn nhãn *spam* / *ham*. Khi một email mới đến:

- Biểu diễn email dưới dạng vector TF-IDF.
- Tính **khoảng cách Euclid** giữa email mới và mọi email huấn luyện.
- Lấy $k = 5$ hàng xóm gần nhất; **3 thuộc lớp *spam*, 2 thuộc lớp *ham*** $\rightarrow$ hệ thống gán nhãn *spam*.

Quá trình này cụ thể hoá đúng quy trình 4 bước đã nêu ở Mục 3, với ưu tiên chọn k lẻ để tránh hoà phiếu.

9. Tóm tắt mục

- K-NN là **thuật toán “học lười”**—không mô hình, dựa trên **tương tự lân cận**.
- Quy trình: *tính khoảng cách* $\rightarrow$ *chọn k* $\rightarrow$ *bỏ phiếu đa số*.
- Lựa chọn k và **chuẩn hoá đặc trưng** quyết định tới *bias–variance* của mô hình.
- Công cụ tối ưu tìm hàng xóm (k-d tree, LSH) giúp mở rộng K-NN cho dữ liệu lớn.

Kiến thức này đặt nền cho các thuật toán phân loại nâng cao (Logistic Regression, SVM, Cây quyết định) sẽ trình bày ở các mục kế tiếp.

4.3. Học không giám sát (Unsupervised Learning)

4.3.1 Giới thiệu

Định nghĩa và mục tiêu: Học không giám sát là hướng tiếp cận mà *thuật toán chỉ quan sát các ví dụ đầu vào x chưa gắn nhãn* và cố gắng **khám phá các mẫu (pattern) hoặc cấu trúc ẩn** trong dữ liệu. Nói cách khác, thay vì dự đoán một giá trị “đúng”, mô hình tìm cách “hiểu” dữ liệu bằng chính dữ liệu đó — một hình thức *khám phá tri thức* bên trong tập quan sát.

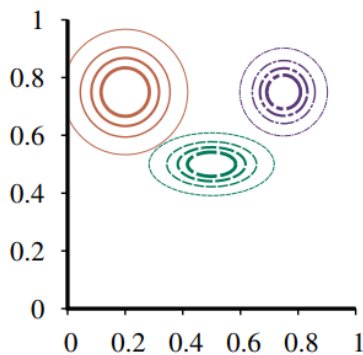
So với học có giám sát (cần cặp (x, y)), học không giám sát **giải quyết bài toán “tự đặt câu hỏi”**: “*Trong không gian dữ liệu này, những nhóm đối tượng nào tương đồng? Trật tự tiềm ẩn nào giải thích cho sự phân bố quan sát được?*” Việc không phụ thuộc nhãn giúp:

- tận dụng kho dữ liệu khổng lồ nhưng chưa gắn nhãn;
- hình thành **biểu diễn (representation)** mới hỗ trợ tác vụ khác;
- xây dựng **mô hình sinh (generative model)** để mô phỏng hoặc tổng hợp mẫu mới.

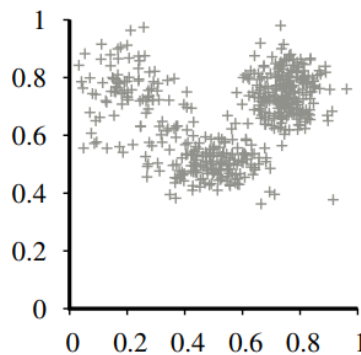
Các lớp bài toán chính

Nhóm bài toán	Mục tiêu	Ví dụ điển hình	Ghi chú
Phân cụm (Clustering)	Gom các điểm dữ liệu “gần” nhau thành cụm,	K-means, Gaussian Mixture Model	Nền tảng cho phân khúc thị

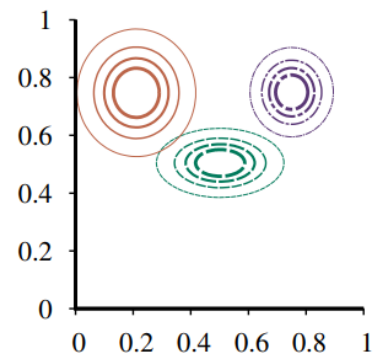
Nhóm bài toán	Mục tiêu	Ví dụ điển hình	Ghi chú
	tối đa hoá tính đồng nhất nội cụm, đồng thời khác biệt giữa các cụm.	(GMM) tìm cụm khách hàng mua sắm tương tự.	trường, phát hiện cộng đồng.
Giảm chiều (Dimensionality Reduction)	Tìm biểu diễn $\tilde{x} \in \mathbb{R}^m$, $m \ll d$ nhưng vẫn giữ thông tin quan trọng.	Principal Component Analysis (PCA) loại bỏ nhiễu và trực quan hoá dữ liệu ảnh độ nét cao.	Không gán nhãn; chỉ “nén” dữ liệu.
Ước lượng mật độ (Density Estimation)	Xấp xỉ hàm phân bố $p(x)$ của dữ liệu, phục vụ sinh mẫu mới hoặc phát hiện bất thường.	GMM – huấn luyện bằng thuật toán EM để khớp phân bố hỗn hợp Gaussian.	
Phát hiện bất thường (Anomaly Detection)	Nhận diện điểm hiếm, không phù hợp với mô hình dữ liệu “bình thường”.	Giám sát an ninh mạng: bất thường lưu lượng mạng.	Xây trên mô hình mật độ/giảm chiều.
Khai thác luật kết hợp (Association Rule Mining)	Khám phá mối quan hệ “nếu-thì” giữa các thuộc tính.	Luật “mua bánh $\rightarrow$ mua sữa” trong giỏ hàng siêu thị.	Phục vụ gợi ý sản phẩm.



(a)



(b)



(c)

Hình 96. Gaussian Mixture Model với ba thành phần và dữ liệu được sinh ra từ mô hình. (Nguồn: Stuart J. Russell, Peter Norvig – *Artificial Intelligence: A Modern Approach*, 2021).

(a): Mô hình GMM gồm ba phân phối Gaussian có trọng số lần lượt là 0.2, 0.3 và 0.5.

(b): Dữ liệu gồm 500 điểm được lấy mẫu từ mô hình trong (a).

(c): Kết quả phục hồi lại mô hình bằng thuật toán EM.

Hình ảnh này minh họa trực quan bài toán ước lượng mật độ và phân cụm không giám sát. GMM giả định dữ liệu được sinh từ nhiều phân phối con và mô hình hóa phân bố tổng thể thông qua tổ hợp tuyến tính có trọng số của các thành phần. Quy trình học bao gồm việc ước lượng các tham số (trọng số, trung tâm, hiệp phương sai) của từng thành phần Gaussian từ dữ

liệu chưa gán nhãn. Đây là cơ sở lý thuyết cho nhiều ứng dụng thực tiễn như phân đoạn khách hàng, nhận diện mẫu hiếm hoặc sinh mẫu mới.

Quy trình điển hình

- **Thu thập & chuẩn hoá dữ liệu:** loại bỏ trùng lặp, chuẩn hoá thang đo.
- **Lựa chọn đặc trưng (feature):** có thể áp dụng PCA trước khi phân cụm để giảm nhiễu.
- **Chọn mô hình & siêu tham số:** ví dụ chọn số cụm K cho K-means hoặc GMM (có thể dùng tiêu chí BIC/AIC).
- **Đánh giá mô hình:** vì thiếu “đáp án”, sử dụng chỉ số nội bộ (Silhouette, Dunn) hoặc kiểm chứng tính hữu ích qua tác vụ kế tiếp.

Thách thức & hướng khắc phục

- **Xác định số cấu trúc ẩn** (số cụm, bậc giảm chiều) – thường phải dùng **tiêu chí thống kê** hoặc quy chiếu miền tri thức.
- **Độ nhạy với nhiễu và tỉ lệ kích thước:** K-means dùng khoảng cách Euclid không phù hợp khi dữ liệu có thang đo khác nhau; nên **chuẩn hoá** hoặc chọn thước đo khác (cosine, Mahalanobis).
- **Diễn giải kết quả:** cần ánh xạ cụm hoặc thành phần principal back to domain knowledge để kết quả có ý nghĩa.

Học không giám sát đóng vai trò “**kính hiển vi**” của học máy: thay vì dự đoán, nó giúp ta *nhìn* vào dữ liệu để phát hiện mô hình ẩn và tạo nền tảng cho các bước phân tích hoặc học sâu hơn. Các kỹ thuật then chốt như phân cụm, PCA hay GMM **không chỉ hữu dụng độc lập** mà còn là **khối xây dựng** cho pipeline hiện đại về học sâu tự giám sát, mô hình sinh, và phát hiện bất thường trong hệ thống thông minh ngày nay.

4.3.2 Phân cụm: Thuật toán K-Means

1. Mục tiêu và bối cảnh

Thuật toán **K-Means** hướng tới việc **phân chia N điểm dữ liệu thành K cụm xác định trước** sao cho tổng sai số bình phương trong cụm (*within-cluster sum of squares – WCSS*) được tối thiểu hóa. Mỗi cụm được đại diện bởi **tâm cụm (centroid)** — trung bình hình học của tất cả điểm thuộc cụm đó.

Trong bối cảnh AI hiện đại, K-Means được xem là “**chuẩn mực cơ bản**” của **phân cụm cứng** (hard clustering) vì:

- độ phức tạp $\mathcal{O}(NKI)$ (với I là số vòng lặp) phù hợp dữ liệu lớn;
- dễ mở rộng cho ứng dụng thực tế (customer segmentation, ảnh, IoV, v.v.).

2. Thuật toán gốc (pseudocode)

```
1: procedure k-means( $X_s, E_s, k$ )
2:   Inputs
3:      $X_s$  set of features,  $X = \{X_1, \dots, X_n\}$ 
4:      $E_s$  set of training examples
5:      $k$  number of clusters
6:   Output
7:     cluster: function from examples to clusters
8:     prediction: function from feature and cluster to a value for that feature
9:   Local
10:    integer  $cc[c], cc\_new[c] \leftarrow$  cluster counts
11:    real  $fs[j,c], fs\_new[j,c] \leftarrow$  feature sums
12:    Boolean stable
13:    Initialize  $fs$  and  $cc$  randomly based on data
14:    define  $prediction(j,c)=fs[j,c]/cc[c]$ 
15:    define  $cluster(e)=\operatorname{argmin}_c \sum_j (prediction(j,c)-X_j(e))^2$ 
16:    repeat
17:      reset  $fs\_new, cc\_new \leftarrow 0$ 
18:      for each example  $e \in E_s$  do
19:         $c \leftarrow cluster(e); cc\_new[c]++$ 
20:         $fs\_new[j,c] += X_j(e) \quad \forall j$ 
21:         $stable \leftarrow (fs\_new = fs) \wedge (cc\_new = cc)$ 
22:         $fs \leftarrow fs\_new; cc \leftarrow cc\_new$ 
23:    until stable
24:    return cluster, prediction
```

Giải thích ngắn gọn

- **Khởi tạo:** gán ngẫu nhiên điểm $\rightarrow$ cụm.
- **Bước Cập nhật tâm (M-step):** tính trung bình các đặc trưng trong từng cụm, dòng 14.
- **Bước Gán cụm (E-step):** gán lại mỗi điểm vào cụm gần nhất theo khoảng cách Euclid, dòng 15.
- **Điều kiện hội tụ:** khi không còn điểm nào đổi cụm (dòng 21) $\rightarrow$ thuật toán dừng và đảm bảo hội tụ tới **cực tiểu địa phương** của WCSS.

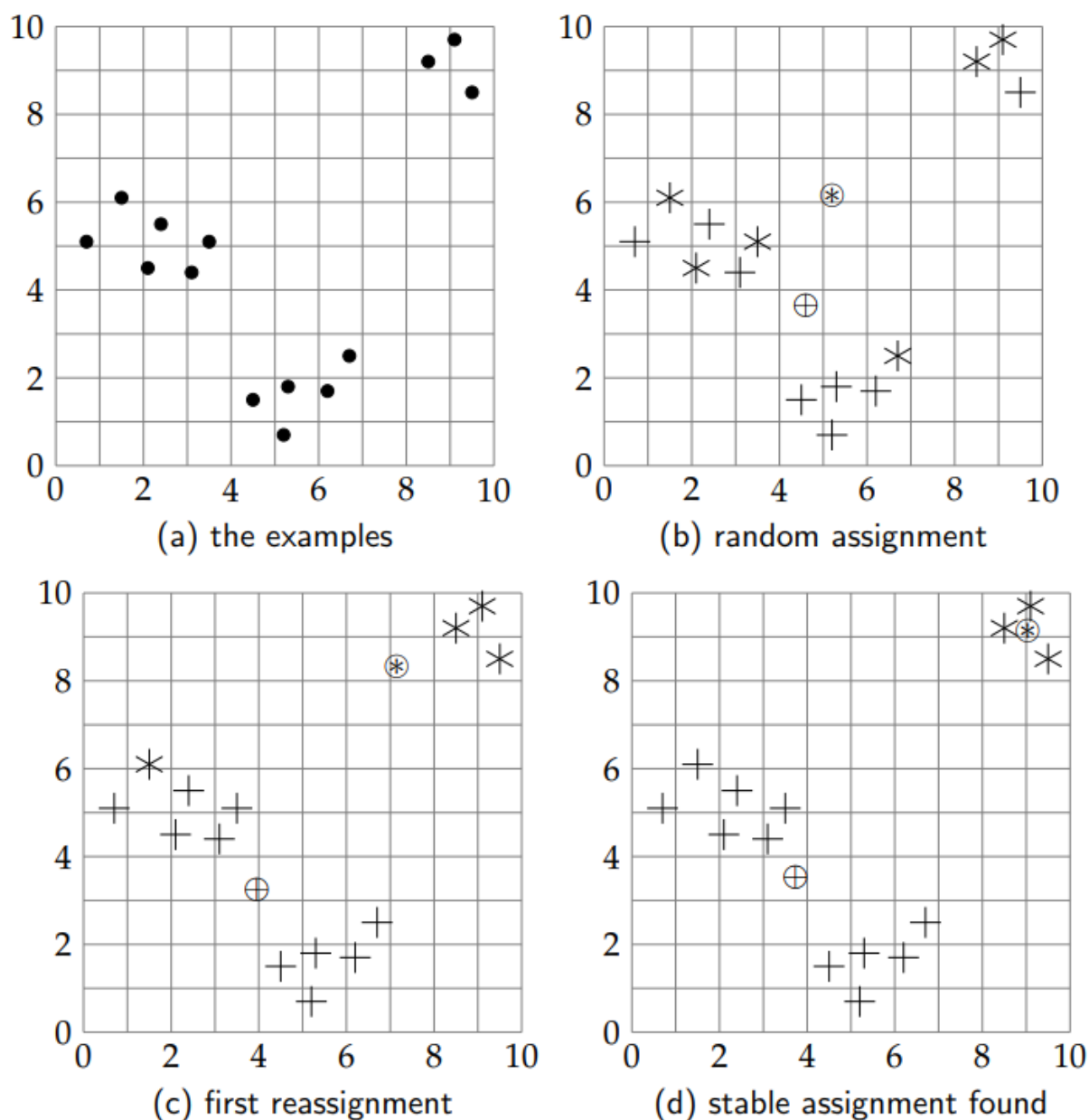
3. Trực quan hoá quá trình lặp

Ví dụ 10.9:

Giả sử một tác tử đã quan sát được tập dữ liệu gồm các cặp tọa độ (X,Y) như sau:

$\langle 0.7, 5.1 \rangle, \langle 1.5, 6.1 \rangle, \langle 2.1, 4.5 \rangle, \langle 2.4, 5.5 \rangle, \langle 3.1, 4.4 \rangle, \langle 3.5, 5.1 \rangle, \langle 4.5, 1.5 \rangle,$
 $\langle 5.2, 0.7 \rangle, \langle 5.3, 1.8 \rangle, \langle 6.2, 1.7 \rangle, \langle 6.7, 2.5 \rangle, \langle 8.5, 9.2 \rangle, \langle 9.1, 9.7 \rangle, \langle 9.5, 8.5 \rangle$
 $\rangle$.

Tập dữ liệu này gồm 14 điểm trong không gian 2 chiều, phân bố rải rác thành ba vùng chính: một cụm ở góc trái–trên, một cụm ở dưới cùng và một cụm ở góc phải–trên. Đây là cơ sở để áp dụng thuật toán K-Means nhằm phân nhóm các điểm thành các cụm tương đồng.



Hình 97. Quá trình lặp của K-Means với $K = 2$.

Hình 97 (trích từ *Search Methods in AI*) trình bày trực quan từng bước hoạt động của thuật toán K-Means khi phân cụm một tập dữ liệu 2D nhỏ thành hai cụm:

- **(a)** Hiện thị 14 điểm dữ liệu được quan sát dưới dạng tọa độ (X, Y).
- **(b)** Các điểm được gán ngẫu nhiên vào hai cụm ký hiệu bằng “+” và “*”; trung tâm cụm ban đầu được biểu diễn bằng ký hiệu $\oplus$ và $\otimes$.

- **(c)** Giai đoạn gán lại cụm dựa vào khoảng cách đến trung tâm gần nhất, sau đó tính lại trung tâm mới.
- **(d)** Quá trình gán cụm và tính lại trung tâm tiếp tục đến khi không còn thay đổi — trạng thái ổn định (converged).

Hình này minh họa rõ ràng nguyên lý “gán cụm–cập nhật tâm–kiểm tra hội tụ” của K-Means. Qua hình, người học dễ hình dung ảnh hưởng của việc khởi tạo cụm ngẫu nhiên và thấy được cách mà WCSS giảm dần theo mỗi vòng lặp. Việc này rất quan trọng để trực giác hóa hiện tượng hội tụ về cực tiểu địa phương, đặc biệt khi triển khai với dữ liệu thực.

4. Đánh giá & lựa chọn K

- **Elbow method:** vẽ WCSS theo K , chọn “khuyết” nơi WCSS giảm chậm dần.
- **Chỉ số Silhouette:** đo chênh lệch khoảng cách nội cụm và ngoài cụm, giá trị gần 1 càng tốt.
- **Thông tin mô hình (BIC/AIC):** cân bằng độ khớp và độ phức tạp.

5. Thách thức thực tiễn

Vấn đề	Nguyên nhân	Khắc phục
Chọn K	Không biết số cấu trúc ẩn	Elbow, Silhouette, hoặc mở rộng K-Means++ lặp nhiều lần.
Nhảy khởi tạo	Tâm ban đầu lệch → hội tụ xấu	Dùng K-Means++ hoặc chạy nhiều khởi tạo, lấy WCSS nhỏ nhất.
Outlier / dữ liệu tỷ lệ khác nhau	Khoảng cách Euclid bị chi phối	Chuẩn hoá (z-score) hoặc dùng khoảng cách Mahalanobis.
Cao chiều	“Lời nguyền chiều” làm WCSS kém ý nghĩa	Giảm chiều (PCA) trước khi phân cụm.

6. Triển khai Python với **Scikit-learn**

```
# Huấn luyện mô hình K-Means 5 cụm
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
kmeans.fit(X)      # X: ma trận đặc trưng không nhãn
labels = kmeans.labels_
centers = kmeans.cluster_centers_
```

Sau đó, dùng *matplotlib* để vẽ điểm và **centroids** màu đỏ hình sao như hướng dẫn ở dòng 95-103 của cùng ví dụ.

7. Tóm lược

K-Means là **thuật toán phân cụm kinh điển**: dễ hiểu, hiệu quả, và là nền tảng cho nhiều biến thể (K-Medoids, Mini-Batch, Spectral K-Means). Hiểu rõ quy trình **khởi tạo** → **gán** → **cập**

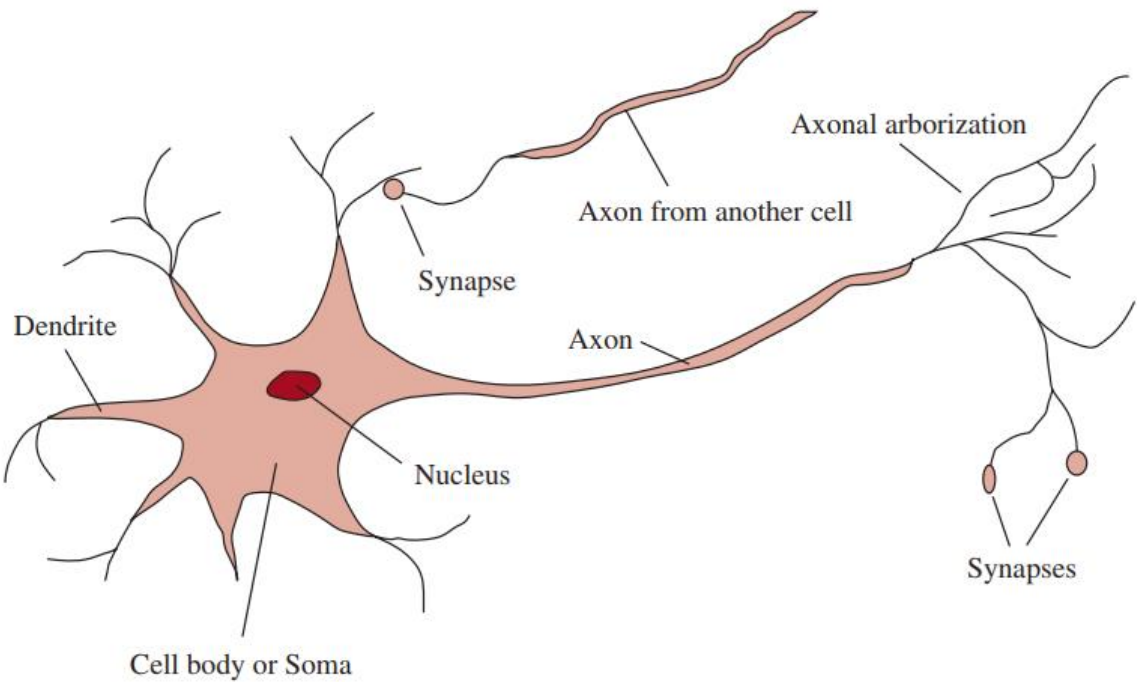
nhật → **kiểm tra** cùng các kỹ thuật đánh giá sẽ giúp sinh viên **vận dụng linh hoạt** thuật toán vào phân tích dữ liệu không nhãn trong các dự án AI thực tế.

4.4.1. Nguồn cảm hứng từ sinh học

Một mạng nơ-ron nhân tạo (Artificial Neural Network – ANN) ra đời từ khát vọng mô hình hoá cấu trúc và cơ chế học tập của **nơ-ron sinh học**—đơn vị xử lý cơ bản của hệ thần kinh động vật.

1. Cấu trúc nơ-ron sinh học

Nơ-ron gồm ba thành phần chính (xem Hình 4-1):



Hình 98. Cấu trúc một nơ-ron sinh học. Nguồn: Stuart J. Russell & Peter Norvig – Artificial Intelligence: A Modern Approach, 2021.

Hình này mô tả ba thành phần chính của một tế bào thần kinh (nơ-ron): **thân tế bào (soma)** chứa nhân tế bào, **các nhánh ngắn dendrite** dùng để tiếp nhận tín hiệu từ nơ-ron khác, và **sợi dài axon** có nhiệm vụ truyền tín hiệu điện hoá đến các nơ-ron kế tiếp thông qua các khớp thần kinh (synapse).

Độ dài của axon thường gấp hàng trăm lần đường kính thân tế bào và mỗi nơ-ron có thể kết nối với 10.000 đến 100.000 nơ-ron khác. Hoạt động truyền tín hiệu và điều chỉnh độ mạnh của các synapse chính là nền tảng cho khả năng học hỏi và thích nghi của hệ thần kinh.

Thành phần	Vai trò sinh lý
Soma (thân tế bào)	Chứa nhân, duy trì hoạt động trao đổi chất.
Dendrite	Các nhánh ngắn tiếp nhận xung điện-hoá từ nơ-ron khác.

Thành phần	Vai trò sinh lý
Axon	Sợi dài truyền tín hiệu đi; mỗi nơ-ron thường kết nối 10 – 100 000 nơ-ron khác tại khớp thần kinh (synapse) .

Nhờ phản ứng điện-hoá học tại synapse, xung thần kinh lan truyền và **độ mạnh** của mỗi synapse thay đổi theo kinh nghiệm—cơ chế nền tảng cho quá trình *học* của bộ não.

2. Tín hiệu – “trọng số” tự nhiên

Trong não động vật, **trọng số kết nối** quyết định lượng tín hiệu dendrite nhận được từ axon của nơ-ron khác; các trọng số này **dịch chuyển** suốt đời sống khi cá thể thích nghi với môi trường. Chính ý tưởng “điều chỉnh trọng số để học” được chuyển hoá trực tiếp vào các ANN.

3. Từ nơ-ron thật đến nơ-ron nhân tạo

- **Mô hình McCulloch–Pitts (1943)** trừu tượng hoá nơ-ron thành một **hàm bậc thang** của tổng có trọng số các đầu vào.
- **Perceptron** (Rosenblatt, 1958) kế thừa mô hình đó, trở thành khối xây dựng của mạng nơ-ron hiện đại.
- Trong ANN, mỗi *đơn vị* (unit) tính **tổng có trọng số** của các đầu vào rồi áp dụng **hàm kích hoạt phi tuyến** để tạo đầu ra.

Dù “**lỏng lẻo**” so với thần kinh học—ví dụ, nơ-ron nhân tạo thường cập nhật đồng bộ theo các vòng lặp rời rạc thay vì phát xung không đồng bộ—song phép ẩn dụ này mang lại hai giá trị then chốt:

- **Tính song song phân tán:** giống như bộ não, ANN gồm nhiều đơn vị đơn giản hoạt động đồng thời.
- **Khả năng tự điều chỉnh:** học tập chính là tối ưu các trọng số, tương ứng với quá trình củng cố hoặc suy yếu synapse sinh học.

4. Ý nghĩa cho kỹ thuật AI

- **Hiệu quả tính toán:** não người chậm hơn máy tính về tốc độ xung nhịp, nhưng bù lại bằng số kết nối khổng lồ và cấu trúc song song sâu. ANN khai thác ưu điểm này bằng cách mở rộng **độ sâu** và **độ rộng** của mạng.
- **Khả năng biểu diễn giàu:** các đường truyền dài qua nhiều lớp cho phép mô hình hoá hàm phi tuyến phức tạp—điều mà các mô hình nông (shallow) không thể thực hiện hiệu quả.

5. Tóm tắt

“Các mạng nơ-ron do học sâu huấn luyện được gọi là *neural networks* cho dù sự tương đồng với cấu trúc não là **bề mặt**”.

Mặc dù ANN chỉ phỏng theo sinh học ở mức ý niệm, việc **lấy cảm hứng từ cơ chế synapse – trọng số** đã cung cấp cho trí tuệ nhân tạo một khung tính toán linh hoạt, đủ sức giải quyết các bài toán nhận dạng thị giác, âm thanh và ra quyết định phức tạp hiện nay.

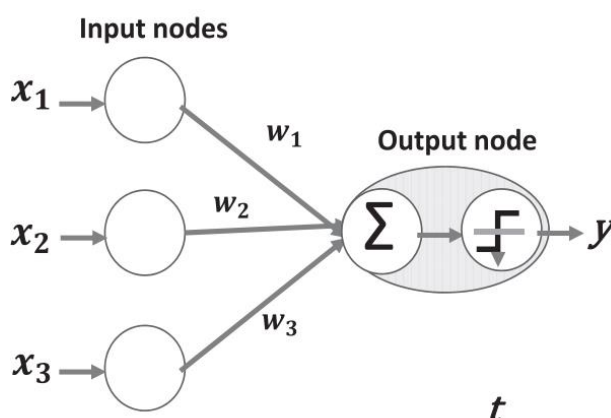
4.4.2 Perceptron – nơ-ron đơn giản nhất

4.4.2.1 Mô hình toán học

Perceptron gồm **n** đầu vào $x_0 = 1, x_1, \dots, x_n$ (trong đó x_0 là hằng 1 để hấp thụ ngưỡng), mỗi đầu vào gắn với một trọng số w_i . Nơ-ron tính **tổng có trọng số**

$$z = \sum_{i=0}^n w_i x_i$$

và so sánh với ngưỡng (ẩn trong w_0). Nếu $z > 0$ nơ-ron “phát xung” (đầu ra $y = 1$), ngược lại $y = 0$.



Hình 99. Sơ đồ một perceptron.

Hình 99 minh họa cấu trúc cơ bản của một **Perceptron** – viên gạch nền tảng cho mạng nơ-ron nhân tạo (ANN). Trong hình, perceptron được thể hiện như một nút đầu ra kết nối với ba nút đầu vào thông qua các cung có hướng mang trọng số (weights) w_1, w_2, w_3 .

Nội dung chính của hình:

- **Ba đầu vào x_1, x_2, x_3 :** tượng trưng cho các yếu tố ảnh hưởng đến quyết định, ví dụ như:
 - x_1 : tầm quan trọng của lớp học
 - x_2 : lời khuyên của người bạn siêng năng
 - x_3 : lời khuyên của người bạn lười biếng
- **Nút đầu ra:** thực hiện phép tính tổng có trọng số $z = w_1x_1 + w_2x_2 + w_3x_3$, sau đó so sánh với một ngưỡng t . Nếu $z > t$ thì đầu ra $y = 1$ (chẳng hạn, quyết định đi học); nếu không thì $y = -1$ (quyết định nghỉ học).
- **Diễn giải mô hình học:** Hình mô phỏng quá trình ra quyết định như một bài toán phân loại nhị phân, nơi Perceptron phân chia không gian đầu vào bằng một siêu phẳng tuyến tính dựa trên trọng số w và ngưỡng t .

4.4.2.2 Hàm kích hoạt

Bản gốc của Rosenblatt dùng **hàm bước**

$$\text{Step}(z) = \begin{cases} 1 & \text{nếu } z \geq 0 \\ 0 & \text{nếu } z < 0 \end{cases}$$

Hàm này cho ranh giới quyết định dạng đường thẳng/phẳng và không khả vi tại $z = 0$. Những hạn chế này về sau dẫn tới việc thay thế bằng hàm logistic, ReLU,... (mục 4.5).

4.4.2.3 Thuật toán học Perceptron

Giả sử tập huấn luyện gồm các cặp $(\mathbf{x}^{(k)}, y^{(k)})$ với $y^{(k)} \in \{0,1\}$. Quy tắc cập nhật lặp từng mẫu (stochastic) là

$$w_i \leftarrow w_i + \alpha (y^{(k)} - h_w(\mathbf{x}^{(k)})) x_i^{(k)},$$

trong đó $\alpha > 0$ là tốc độ học. Thuật toán **hội tụ** đến một bộ trọng số phân tách hoàn hảo nếu dữ liệu tuyến tính phân tách, nhưng sẽ dao động khi dữ liệu **không** phân tách tuyến tính (Hình 19.16).

4.4.2.4 Khả năng biểu diễn và các ví dụ cơ bản

Perceptron mô hình hoá **hàm phân loại tuyến tính**. Với hai đầu vào nhị phân, nó học được các hàm **AND** và **OR** – vì tồn tại đường thẳng tách hai lớp dương/âm tương ứng. Ví dụ, hàm OR được học bằng cách điều chỉnh trọng số để ba điểm (1,0), (0,1), (1,1) rơi về cùng một phía của đường biên.

Bảng 2. Bảng chân trị hàm AND và OR.

x_1	x_2	AND	OR
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

4.4.2.5 Giới hạn: bài toán XOR và “mùa đông AI” thứ nhất

Perceptron **thất bại** với các quan hệ **phi tuyến tính** như XOR; không tồn tại một đường thẳng duy nhất tách hai lớp của hàm này. Năm 1969, Minsky & Papert công bố *Perceptrons*, chứng minh giới hạn này và chỉ ra thiếu các thuật toán huấn luyện đa lớp, khiến tài trợ cho mạng nơ-ron suy giảm – sự kiện thường được xem là “mùa đông AI” lần thứ nhất. Sự hồi sinh chỉ đến khi thuật toán **Back-propagation** cho mạng đa lớp xuất hiện giữa thập niên 1980 (trình bày ở 4.4.4).

Tóm tắt: Perceptron là viên gạch nền của mạng nơ-ron. Nó minh hoạ cách kết hợp tuyến tính đầu vào, dùng hàm kích hoạt rời rạc và học bằng quy tắc điều chỉnh trọng số đơn giản. Mặc dù

mạnh cho bài toán *linearly separable*, hạn chế với XOR đã thúc đẩy sự ra đời của **mạng đa lớp** – mở đường cho Học sâu hiện đại.

4.4.3. Mạng Nơ-ron Đa Lớp (Multi-layer Perceptron – MLP)

1. Kiến trúc tổng quát

Một MLP là mạng nơ-ron có cấu trúc *xếp lớp* theo chiều truyền tiến (feed-forward):

Tầng	Chức năng chính	Ghi chú
Lớp đầu vào (Input layer)	Nhận vector đặc trưng $\mathbf{x} \in \mathbb{R}^D$.	Không thực hiện phép biến đổi; chỉ phân phối dữ liệu.
Lớp ẩn (Hidden layers)	Mỗi nơ-ron tính $\mathbf{a} = \mathbf{g}(\mathbf{w}\mathbf{T}\mathbf{x} + \mathbf{b})$, trong đó $\mathbf{g}$ là hàm kích hoạt phi tuyến.	Có thể có một hoặc nhiều lớp; càng nhiều/tham số càng tăng năng lực biểu diễn.
Lớp đầu ra (Output layer)	Biến đổi biểu diễn cuối cùng thành dự đoán $\hat{\mathbf{y}}$.	Chọn hàm kích hoạt phù hợp (sigmoid, softmax, v.v.).

Việc bổ sung ít nhất một **lớp ẩn** chính là chìa khóa giúp mạng học được mối quan hệ **phi tuyến** mà Perceptron đơn lớp không thể nắm bắt.

2. Toán học của lan truyền tiến: Giả sử mạng có L lớp (không kể lớp đầu vào). Với đầu vào $\mathbf{x} = \mathbf{f}^0$, lan truyền tiến được định nghĩa đệ quy

$$\mathbf{f}^l = \mathbf{g}^l(\mathbf{W}^l \mathbf{f}^{l-1} + \mathbf{b}^l), \quad l = 1, \dots, L-1$$
$$\hat{\mathbf{y}} = \mathbf{W}^L \mathbf{f}^{L-1} + \mathbf{b}^L$$

trong đó $\mathbf{W}^l$ và $\mathbf{b}^l$ là trọng số & độ lệch, còn $\mathbf{g}^l$ là hàm kích hoạt của lớp l .

3. Hàm kích hoạt & vai trò của phi tuyến: Nếu $\mathbf{g}$ tuyến tính, toàn bộ mạng chỉ tương đương một phép biến đổi tuyến tính đơn; do đó **phi tuyến** là điều kiện cần để tăng sức mạnh biểu diễn. Các hàm kích hoạt phổ biến (σ , tanh, ReLU, softplus) được minh họa trong Figure 22.2 của AIMA.

4. Định lý Xấp xỉ Phổ quát

“Một mạng chỉ cần **hai lớp tính toán** (một lớp ẩn phi tuyến và một lớp ra tuyến tính) đã có thể xấp xỉ **bất kỳ hàm liên tục** trên miền đóng tới độ chính xác tùy ý.”

Hệ quả: chỉ một lớp ẩn *đủ rộng* đã giúp MLP là “máy xấp xỉ vạn năng”, tuy nhiên số nơ-ron có thể phải **rất lớn**, và thực hành thường ưu tiên nhiều lớp hẹp sâu hơn (deep learning).

5. Ví dụ: giải bài toán XOR Perceptron đơn lớp thất bại với XOR vì đường phân cách đòi hỏi phi tuyến. Thêm một lớp ẩn gồm hai nơ-ron (mỗi nơ-ron học một nửa mặt phẳng) cho phép MLP biểu diễn biên quyết định hình chữ “V”, từ đó phân loại chính xác bốn điểm XOR.

6. Ghi chú triển khai

- Kích thước mạng:** Số lớp & nơ-ron là *siêu tham số* phải chọn/cân chỉnh; bảng tham số khuyến nghị (activation, solver, learning-rate schedule, v.v.) có thể tham khảo trong CRC Press (2024).

- **Huấn luyện:** Phần 4.4.4 sẽ trình bày chi tiết thuật toán **Back-propagation** và tối ưu hóa trọng số bằng (Stochastic) Gradient Descent.

Tóm lại, MLP là bước nhảy vọt khắc phục giới hạn tuyến tính của Perceptron: cấu trúc *nhiều lớp + phi tuyến* mang lại năng lực xấp xỉ hàm bất kỳ, đặt nền móng cho các kiến trúc sâu hơn trong học sâu (deep learning).

4.4.4. Huấn luyện Mạng Nơ-ron

1. Giới thiệu

Thuật toán **Lan truyền ngược (Backpropagation)** là phương pháp then chốt giúp “dạy” mạng nơ-ron đa lớp. Ý tưởng cốt lõi: sau khi tính được sai số (loss) ở đầu ra, ta **truyền ngược** thông tin sai số này qua từng lớp để thu được **đạo hàm của hàm mất mát đối với mọi trọng số**, từ đó cập nhật chúng bằng **gradient descent**.

2. Cơ sở toán học — quy tắc dây chuyền

Trong mô hình K lớp, hàm suy diễn có dạng

$$y = f_K(f_{K-1}(\dots f_1(x) \dots)).$$

Đạo hàm của hàm mất mát L theo trọng số tầng j thu được bằng cách lặp lại quy tắc dây chuyền (chain rule):

$$\frac{\partial L}{\partial \theta_j} = \frac{\partial L}{\partial f_K} \frac{\partial f_K}{\partial f_{K-1}} \dots \frac{\partial f_{j+1}}{\partial f_j} \frac{\partial f_j}{\partial \theta_j}.$$

Nhờ cấu trúc *đồ thị tính toán* (computation graph), các đạo hàm trung gian được tái sử dụng nên chi phí tính toán tỷ lệ thuận với số phép tính ở lượt lan truyền xuôi. Chế độ “reverse mode” của tự động vi phân (automatic differentiation) chính là backpropagation; gradient lan truyền **ngược chiều dòng dữ liệu** và rẻ hơn “forward mode” khi số đầu vào lớn.

3. Thuật toán Backpropagation (giả mã)

Bước	Mô tả
1. Lan truyền xuôi (Forward pass)	Tính đầu ra của từng lớp và lưu giá trị kích hoạt.
2. Tính loss	So sánh đầu ra mô hình với nhãn thật, ví dụ dùng <i>Cross-Entropy</i> hoặc <i>MSE</i> .
3. Lan truyền ngược (Backward pass)	a) Khởi tạo $\delta_K = \frac{\partial L}{\partial f_K}$. b) Với $j = K - 1 \rightarrow 1$: $\delta_j = (\theta_{j+1}^\top \delta_{j+1}) \odot \sigma'(z_j)$.
4. Cập nhật trọng số	$\theta_j \leftarrow \theta_j - \eta \frac{\partial L}{\partial \theta_j}$ với tốc độ học η .

4. Cập nhật bằng Gradient Descent

Gradient descent di chuyển tham số theo **hướng ngược gradient** để giảm loss:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} L(\theta^{(t)}).$$

Khi địa hình hàm lỗi “hẹp và dốc”, **Momentum** thêm thành phần $\alpha \Delta \theta^{(t-1)}$ giúp quỹ đạo mượt hơn và hội tụ nhanh hơn. Trong thực tế, **Stochastic Gradient Descent (SGD)** và các biến thể thích nghi (AdaGrad, Adam) điều chỉnh bước nhảy theo đặc tính địa phương của hàm lỗi.

5. Ví dụ minh họa

Xét MLP 2 lớp ẩn kích hoạt ReLU, bài toán phân loại nhị phân:

- **Forward:** tính $a^{(1)} = \text{ReLU}(W^{(1)}x + b^{(1)})$, $a^{(2)} = \text{ReLU}(W^{(2)}a^{(1)} + b^{(2)})$, $\hat{y} = \sigma(W^{(3)}a^{(2)} + b^{(3)})$.
- **Loss:** $L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$.
- **Backward:** triển khai quy tắc chain-rule để lấy $\frac{\partial L}{\partial W^{(i)}}$.
- **Update:** $W^{(i)} \leftarrow W^{(i)} - \eta \frac{\partial L}{\partial W^{(i)}}$.

Mã Python ngắn gọn cho MLP nhỏ có thể được phát triển như trong ví dụ perceptron ở mục trước, mở rộng thêm hai lớp ẩn và hàm ReLU.

6. Lược sử & thách thức

Backpropagation được công bố rộng rãi bởi **Rumelhart-Hinton-Williams (1986)**, khởi động “sự trở lại” của mạng nơ-ron hiện đại. Dù mạnh mẽ, thuật toán chịu các vấn đề:

- **Vanishing/Exploding Gradients** – gradient quá nhỏ/lớn khi lan truyền qua nhiều lớp.
- **Kẹt cục bộ & quá khớp** – cần regularization và kỹ thuật tối ưu tiên tiến.

Các hướng giải quyết gồm chuẩn hóa BatchNorm, hàm kích hoạt ReLU/tanh cải tiến, khởi tạo trọng số Xavier/He và kiến trúc sâu có phần dư (Residual).

Tóm tắt: Lan truyền ngược chuyển vấn đề “học” của mạng nơ-ron về **bài toán tối ưu hàm lỗi khả vi**, cho phép áp dụng toàn bộ kho tàng thuật toán tối ưu. Kiến thức toán học nền tảng (chuỗi hàm, đạo hàm ma trận) và các cải tiến trong gradient descent là chìa khóa để huấn luyện mạng sâu hiệu quả trong thực tiễn.

PHẦN 4 · CÁC CHƯƠNG NÂNG CAO VÀ ĐỊNH HƯỚNG DỰ ÁN

CHƯƠNG 5 · HỌC SÂU (DEEP LEARNING)

5.0 Giới thiệu về học sâu

5.0.1 Ôn lại Mạng Nơ-ron đa lớp (MLP)

Trong quá trình vượt qua giới hạn của Perceptron, **MLP** ra đời với kiến trúc gồm lớp đầu vào, một hoặc nhiều lớp ẩn và lớp đầu ra. Nhờ các lớp ẩn phi tuyến, MLP có khả năng xấp xỉ các hàm phức tạp theo Định lý Xấp xỉ Phổ quát (Universal Approximation Theorem).

Hạn chế chính:

- Khi xử lý dữ liệu có cấu trúc cao như **ảnh** hoặc **văn bản**, MLP phải “trải phẳng” dữ liệu thành vector, dẫn tới **mất thông tin không gian** và **bùng nổ tham số** – một ảnh 100×100 pixel đã sinh ra hàng triệu trọng số ở lớp ẩn đầu tiên.
- Việc học các đặc trưng (features) phải dựa nhiều vào thiết kế thủ công (feature engineering), vừa tốn công vừa dễ bỏ sót mối quan hệ ẩn.

Những hạn chế đó thôi thúc sự ra đời của **Học sâu**.

5.0.2 Định nghĩa Học sâu (Deep Learning)

Học sâu là một nhánh của Học máy sử dụng các mạng nơ-ron với **nhiều lớp ẩn** (deep neural networks) để **tự động** học các biểu diễn ở nhiều cấp độ trừu tượng khác nhau. Khả năng xếp chồng các tầng phi tuyến giúp mạng học “đặc trưng của đặc trưng”: lớp dưới phát hiện mẫu cục bộ, lớp trên kết hợp chúng thành mẫu phức tạp hơn, v.v. — tạo nên **hệ thống phân cấp đặc trưng** do dữ liệu dẫn dắt thay vì do con người thiết kế.

5.0.3 Vì sao cần Học sâu?

Thách thức của MLP / ML truyền thống	Học sâu giải quyết
Nhiều tham số, khó huấn luyện khi dữ liệu đầu vào lớn	Khai thác <i>kết nối cục bộ</i> , <i>chia sẻ tham số</i> (CNN); <i>t tuần tự hoá</i> (RNN) để giảm số lượng tham số và tận dụng cấu trúc dữ liệu
*Mất cấu trúc * của ảnh, chuỗi, đồ thị	Kiến trúc chuyên biệt (CNN, RNN/LSTM, Transformer, GNN) giữ nguyên quan hệ không gian hay thời gian
Phụ thuộc feature engineering	Tự học biểu diễn nhiều tầng, giảm gánh nặng thủ công
Hiệu năng thấp trên bài toán phức tạp	Thể hiện bước nhảy vọt trong nhận dạng ảnh (ImageNet), xử lý ngôn ngữ, thị giác máy, v.v.

5.0.4 Các dòng kiến trúc chính

- **Convolutional Neural Networks (CNNs)** – khai thác tính lưới và tính cục bộ của **ảnh** (mục 5.1).

- **Recurrent / Long Short-Term Memory (RNN/LSTM)** – xử lý **chuỗi** thời gian hoặc ngôn ngữ.
- **Transformer & Self-Attention** – biểu diễn **trình tự** song song, nền tảng của các Mô hình Ngôn ngữ Lớn.
- **Autoencoder, GAN, Diffusion** – học biểu diễn nén và sinh dữ liệu mới.

Các chủ đề này lần lượt được trình bày trong các mục tiếp theo của Chương 5, dựa trên khung chương mục “Simple Feedforward → CNN → RNN → Ứng dụng” trong AIMA 4/e.

5.1. Mạng Nơ-ron Tích chập (Convolutional Neural Networks – CNNs) cho Xử lý ảnh

5.1.1. Vấn đề với MLP khi xử lý ảnh

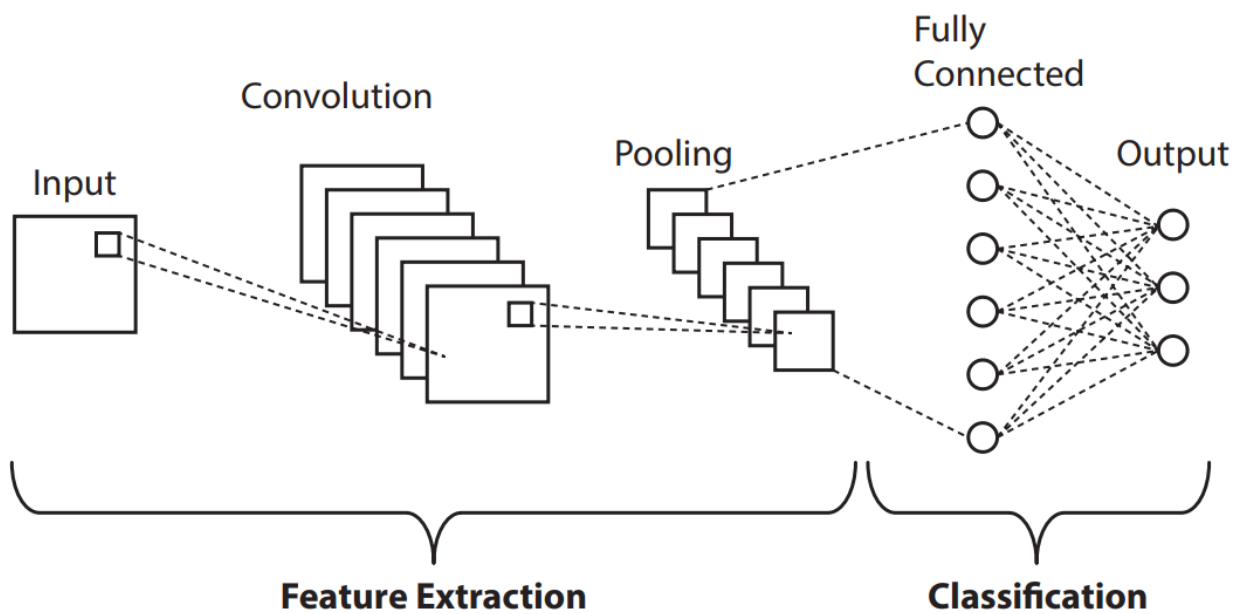
Mạng nơ-ron đa lớp (MLP) là mô hình kết nối đầy đủ (fully-connected). Khi áp dụng trực tiếp cho dữ liệu ảnh, MLP bộc lộ hai hạn chế cơ bản:

Hạn chế	Phân tích chi tiết
Bùng nổ tham số (Parameter Explosion)	Với ảnh đơn kênh 100×100 px, lớp đầu vào đã có 10 000 đặc trưng. Nếu lớp ẩn đầu tiên gồm N nơ-ron và kết nối đầy đủ, số trọng số là $10\,000 \times N$ (chưa kể N bias). Chỉ cần $N = 256$ đã sinh ra 2,56 triệu trọng số; ảnh RGB 224×224 px thậm chí vượt quá 100 triệu. Số tham số khổng lồ làm tăng chi phí tính toán , đòi hỏi bộ nhớ lớn và dễ overfitting khi tập huấn luyện không tương xứng.
Mất thông tin không gian (Spatial Information Loss)	Để đưa vào MLP, ảnh phải được “trải phẳng” thành vector. Khi thứ tự pixel bị hoán vị ngẫu nhiên, mạng vẫn nhận cùng một tập giá trị, nên kết quả hầu như không đổi. Điều này chứng tỏ MLP không nhận biết tính lân cận của pixel – thông tin then chốt đối với hình học cảnh vật và cấu trúc đối tượng.

Hệ quả trực tiếp

- *Khả năng khái quát kém*: mạng phải học riêng rẽ từng mẫu vị trí pixel, thay vì khai thác tính lặp lại của các họa tiết (edges, corners).
- *Hiệu suất huấn luyện thấp*: tốc độ hội tụ chậm do không tận dụng được các ràng buộc hình học vốn có của ảnh.
- *Không phù hợp với ảnh kích thước lớn hoặc nhiều kênh*: mỗi lần tăng độ phân giải sẽ làm số trọng số tăng bình phương, nhanh chóng vượt quá khả năng lưu trữ và tính toán của phần cứng thông thường.

Những rào cản này đặt ra nhu cầu về một kiến trúc tận dụng *kết nối cục bộ* và *chia sẻ tham số* để bảo toàn cấu trúc không gian trong dữ liệu ảnh—tiền đề dẫn tới sự ra đời của **CNN**, sẽ được trình bày chi tiết ở mục 5.1.2.



Hình 100. Mạng nơ-ron tích chập (CNN).

Hình ảnh mô tả kiến trúc của một **Mạng Nơ-ron Tích chập (Convolutional Neural Network - CNN)**, được thiết kế chuyên biệt để xử lý dữ liệu dạng lưới như hình ảnh.

Kiến trúc này bao gồm các lớp chính:

- **Lớp Tích chập (Convolutional Layer):** Sử dụng các bộ lọc (filters) để quét qua ảnh, tự động phát hiện và trích xuất các đặc trưng như cạnh, góc, và hoa văn.
- **Lớp Gộp (Pooling Layer):** Giảm kích thước không gian của dữ liệu, giúp làm nổi bật các đặc trưng quan trọng nhất và giảm chi phí tính toán.

Cấu trúc phân cấp này cho phép CNN học các biểu diễn từ đơn giản đến phức tạp, giúp nhận dạng đối tượng một cách hiệu quả và mạnh mẽ.

5.1.2 Giải pháp của CNN – Mô phỏng Hệ Thống Thị Giác Sinh Học

(a) Ý tưởng sinh học: trường tiếp nhận cục bộ

Các nghiên cứu về **tế bào “simple” và “complex”** trong vỏ não thị giác cho thấy mỗi tế bào chỉ “nhìn” một vùng nhỏ của võng mạc (receptive field) nhưng những vùng này chồng-lên-nhau dọc theo các tầng để dần xây dựng nên nhận thức toàn cảnh. **Convolutional Neural Network (CNN)** kế thừa trực tiếp nguyên lý đó bằng hai cơ chế then chốt:

- **Kết nối cục bộ (local receptive fields)** – mỗi nơ-ron ở lớp tích chập chỉ nhận tín hiệu từ một cửa sổ $k \times k$ nhỏ của ảnh đầu vào, tôn trọng tính lân cận của pixel và cắt giảm mạnh số trọng số cần học.
- **Chia sẻ tham số (parameter sharing)** – cùng một bộ trọng số (kernel) được “tái sử dụng” trên mọi vị trí của ảnh, biến mỗi nơ-ron thành một **bộ dò đặc trưng (feature detector)** có khả năng phát hiện mép, góc... bất kể chúng xuất hiện ở đâu trong ảnh.

Nhờ hai cơ chế này, số tham số của lớp conv phụ thuộc vào kích thước kernel, **không phụ thuộc vào độ phân giải ảnh**, đảo ngược hoàn toàn tình thế “ n^2 trọng số” của MLP truyền thống.

(b) Phép tích chập và bản đồ đặc trưng

Với ảnh đầu vào X kích thước $H \times W \times C$ và kernel $K \in \mathbb{R}^{k \times k \times C}$:

$$Y_{i,j} = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} \sum_{c=1}^C K_{u,v,c} X_{i+u, j+v, c}$$

Mỗi ma trận Y sinh ra gọi là *bản đồ đặc trưng* (feature map). Một lớp conv thường áp dụng D kernel, trả về D feature-map xếp chồng theo chiều kênh, tạo thành biểu diễn bậc cao hơn của ảnh.

- **Stride** xác định bước trượt; *padding* giúp bảo toàn kích thước không gian.
- Sau conv, ta áp dụng hàm kích hoạt phi tuyến (thường là ReLU) để tăng khả năng biểu diễn.

(c) Lớp gộp (Pooling) – Tính bất biến cục bộ

CNN kế tiếp một **lớp gộp** nhằm:

- giảm kích thước không gian $\rightarrow$ bền vững với dịch chuyển nhỏ;
- giới hạn số tham số ở các tầng cao.

Max-Pooling 2×2 là cấu hình phổ dụng: lấy giá trị lớn nhất trong mỗi ô kề cận.

(d) Kiến trúc chuẩn “CONV $\rightarrow$ RELU $\rightarrow$ POOL”

Nhiều khối [CONV–RELU–POOL] được xếp chồng. Phần **đặc trưng bậc cao** cuối cùng được “làm phẳng” và ghép vào các lớp **kết nối đầy đủ** (fully-connected) để thực hiện phân loại tương tự MLP. Sơ đồ tổng quát:

$$\text{INPUT} \rightarrow [\text{CONV} \rightarrow \text{ReLU} \rightarrow \text{POOL}]^{\times N} \rightarrow \text{FLATTEN} \rightarrow \text{FC} \rightarrow \text{SOFTMAX}$$

(e) Ví dụ minh họa kiến trúc CNN

Hình 5.1 – Mô hình CNN ba tầng Convolution-Pooling dùng cho nhận dạng chữ chỉ ngôn ngữ ký hiệu (trích *Algorithms in Advanced Artificial Intelligence*, Fig. 48.5)

(Nguồn: R. N. V. Jagan Mohan et al. - *Algorithms in Advanced Artificial Intelligence* (2024))

Hình 5.1 phác họa rõ quy trình: ảnh $64 \times 64 \times 3 \rightarrow$ ba lớp **Conv (3 $\times$ 3, 32-64-128 filters)** $\rightarrow$ **Max-Pooling 2 $\times$ 2** $\rightarrow$ **Flatten** $\rightarrow$ **FC + SoftMax**, cho phép hệ thống đạt độ chính xác cao trong nhận dạng ký hiệu.

(f) Lợi ích vượt trội so với MLP

Khía cạnh	MLP	CNN
Số tham số	$O(n^2)$ với n pixel $\Rightarrow$ “nổ” tham số	$O(k^2 D)$, độc lập H, W
Khai thác cấu trúc	Mất quan hệ không gian	Bảo toàn lân cận & bất biến dịch chuyển
Khả năng học đặc trưng	Phụ thuộc thiết kế tay	Học <i>đa cấp độ</i> trực tiếp từ dữ liệu
Thành công thực nghiệm	Giới hạn ở ảnh nhỏ	Vượt trội trên ImageNet – Top-5 accuracy $\uparrow$ 98 %

(g) Tóm lược

CNN khắc phục triệt để hai nhược điểm của MLP bằng cách **mô phỏng cơ chế nhận thức thị giác**: *kết nối cục bộ* giảm tham số và *chia sẻ trọng số* bảo đảm bất biến không gian. Nhờ đó, CNN trở thành nền tảng cho mọi tiến bộ hiện đại trong thị giác máy tính – từ LeNet đến ResNet, từ chẩn đoán y ảnh đến xe tự lái.

5.1.3 Các thành phần kiến trúc cốt lõi của CNN

(a) Lớp Tích chập (Convolutional Layer)

Lớp tích chập là khối xây dựng trung tâm của CNN, thực hiện phép biến đổi tuyến tính có tham số trên dữ liệu ảnh (hoặc ma trận đặc trưng) để trích xuất đặc trưng cục bộ.

- **Nhân/Bộ lọc (Kernel/Filter).** Một kernel $K \in \mathbb{R}^{h \times w}$ thường rất nhỏ ($3 \times 3, 5 \times 5, \dots$) so với ảnh đầu vào. Trong quá trình **quét (stride)**, cùng một kernel được “trượt” qua toàn bộ ảnh — mỗi lần trượt thực hiện tích chập:

$$(X * K)(i, j) = \sum_{u=0}^{h-1} \sum_{v=0}^{w-1} K_{u,v} X_{i+u, j+v}.$$

Cơ chế này **chia sẻ tham số** (parameter sharing): mọi vị trí đều dùng chung K , do đó số trọng số cần học giảm từ hàng triệu (nếu dùng MLP) xuống chỉ vài chục hoặc vài trăm.

- **Bản đồ đặc trưng (Feature Map).** Kết quả của mỗi kernel là một bản đồ đặc trưng F đánh dấu cường độ xuất hiện của khuôn mẫu (mép, góc, hoa văn, ...) mà kernel học được trên khắp ảnh. Khi dùng c kernel, ta có c feature maps xếp chồng theo chiều sâu, tạo thành một “khối tích chập” ($H' \times W' \times c$).
- **Kết nối cục bộ (Local Receptive Fields).** Mỗi nơ-ron trong F chỉ “nhìn” một vùng nhỏ (receptive field) của ảnh gốc, giúp mô hình tập trung vào mối quan hệ láng giềng và giữ nguyên tính dịch chuyển (translation invariance) của đặc trưng.

(b) Ý nghĩa hai đột phá thiết kế

Ý tưởng	Ảnh hưởng	Lợi ích chính
Chia sẻ tham số	Mọi vị trí dùng cùng kernel	- Giảm mạnh số lượng tham số $\rightarrow$ ít nguy cơ overfitting. - Khai thác tính đồng nhất của cấu trúc ảnh (đặc trưng có thể xuất hiện ở bất kỳ đâu).
Kết nối cục bộ	Mỗi nơ-ron quan sát vùng nhỏ	- Bảo toàn quan hệ không gian lân cận. - Cho phép mạng học đặc trưng phức tạp dần qua nhiều tầng, vì receptive field mở rộng khi chồng lớp.

(c) Các siêu tham số quan trọng

- **Kích thước kernel** (h, w). Quy định độ chi tiết đặc trưng. Kernel càng lớn $\rightarrow$ receptive field rộng nhưng chi phí cao.
- **Stride** s . Số bước di chuyển kernel mỗi lần quét. Tăng s giảm kích thước đầu ra nhưng có thể làm mất chi tiết.
- **Padding** p . Bổ sung viền 0 để giữ kích thước hoặc kiểm soát receptive field ở biên.
- **Số kernel** c . Tương đương số kênh đặc trưng; quyết định “độ sâu” thông tin trích xuất.

Công thức kích thước đầu ra (giả thiết ảnh đầu vào $H \times W$) :

$$H' = \left\lfloor \frac{H - h + 2p}{s} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W - w + 2p}{s} \right\rfloor + 1.$$

(d) Ví dụ tính toán nhanh

Một ảnh màu $32 \times 32 \times 3$ qua lớp conv với 16 kernel 3×3 , $s = 1$, $p = 1$ cho:

$$H' = W' = 32, \quad \text{đầu ra: } 32 \times 32 \times 16.$$

Số tham số phải học: $16 \times (3 \times 3 \times 3 + 1) = 448$ (thêm 1 bias/kernel) — thấp hơn hàng triệu trọng số nếu ảnh được “trải phẳng” và đưa vào MLP.

(e) Vai trò trong pipeline CNN

Sau mỗi khối **[CONV $\rightarrow$ (Activation) $\rightarrow$ POOL]**, các feature maps chứa thông tin ngày càng giàu ngữ nghĩa, làm tiền đề cho **lớp kết nối đầy đủ** ở cuối cùng thực hiện suy luận (phân loại, hồi quy). Phần 5.1.4 sẽ trình bày chi tiết kiến trúc CNN

(b) Lớp Gộp (Pooling Layer)

Khái niệm. Lớp gộp thực hiện thao tác *down-sampling* trên mỗi bản đồ đặc trưng, nén thông tin của một vùng lân cận thành một giá trị duy nhất. Việc nén này giúp **giảm kích thước không gian**, cắt bớt số tham số ở các tầng kế tiếp và tiết kiệm chi phí tính toán.

Cơ chế toán học. Cho một feature-map $F \in \mathbb{R}^{H \times W}$ và cửa sổ gộp $P \times P$ trượt với bước s (stride):

$$G_{i,j} = \text{pool}(F_{iP:(i+1)P-1, jP:(j+1)P-1}), \quad i = 0, \dots, \left\lfloor \frac{H}{P} \right\rfloor - 1.$$

Hàm $\text{pool}(\cdot)$ **không học** trọng số; giá trị đầu ra được tính cố định, tách biệt với lan truyền ngược. Hai biến thể thường gặp:

- **Max Pooling** – trả về phần tử lớn nhất trong cửa sổ, đóng vai trò “phép OR lô-gic”: chỉ cần đặc trưng xuất hiện ở *đâu đó* trong vùng nhận cảm là đủ.
- **Average Pooling** – lấy giá trị trung bình; hữu ích khi muốn đạt bất biến đa tỉ lệ (multiscale) và làm trơn nhiễu.

Lợi ích chính.

Lợi ích	Giải thích
Giảm tham số & tải tính toán	Kích thước mỗi chiều giảm tỉ lệ với stride $\Rightarrow$ số đơn vị ở các tầng sau giảm lũy thừa hai. Điều này trực tiếp hạ số trọng số, tăng tốc huấn luyện và suy luận.
Tạo bất biến dịch chuyển cục bộ	Khi cùng một đặc trưng di chuyển vài pixel, giá trị max/avg trong cửa sổ không đổi $\Rightarrow$ mô hình kém nhạy cảm vị trí, cải thiện khái quát hóa.
Khuyến khích tách cấp bậc đặc trưng	Bản đồ đặc trưng co lại, buộc mạng học <i>ngữ cảnh rộng hơn</i> ở tầng sâu, đồng thời bảo tồn điểm quan trọng nhất ở tầng nông.
Thích ứng với kiến trúc sâu	Nhờ giảm kích thước sớm, CNN hiện đại có thể xếp chồng hàng chục tầng mà vẫn vừa bộ nhớ GPU (ví dụ VGG, ResNet).

Cấu hình thường dùng.

- **Cửa sổ 2×2 , stride = 2** (không chồng lấp): giảm diện tích xuống 1/4, giữ kích thước kênh.
- **Padding = 0**: duy trì phép lấy mẫu không chồng.
- **Global Average Pooling**: thay toàn bộ tầng fully-connected cuối bằng phép lấy trung bình toàn bản đồ đặc trưng, giảm nguy cơ overfitting và cho phép xử lý ảnh kích thước tùy ý.

Ví dụ minh họa. Một bản đồ 32×32 sau khi áp dụng Max Pooling 2×2 stride = 2 trở thành 16×16 ; nếu ban đầu có 64 kênh, số phần tử giảm từ 65 536 xuống 16 384 – **giảm 4 lần** mà vẫn giữ *vị trí tương đối* của đặc trưng mạnh nhất.

Tóm lại, lớp gộp là thành phần tối quan trọng trong CNN, bổ sung cho lớp tích chập bằng cách *nén thông tin* và *tạo bất biến*, qua đó giúp mạng vừa gọn nhẹ vừa bền vững trước biến thiên vị trí đặc trưng trong ảnh.

(c) Lớp Kết Nối Đầy Đủ (Fully Connected Layer – FC)

Sau khi dữ liệu đã đi qua một chuỗi **CONV** $\rightarrow$ **ReLU** $\rightarrow$ **POOL**, mỗi ảnh được mã hoá thành nhiều **bản đồ đặc trưng** nhỏ và khổ hẹp. Để chuyển các đặc trưng bậc cao này thành quyết

định cuối cùng, CNN “trải phẳng” (flatten) mọi bản đồ thành **một vector duy nhất** và nạp vào một (hoặc vài) lớp *fully-connected*—về bản chất là một **mạng MLP truyền thống** đặt ở phần đuôi của kiến trúc.

Kiến trúc và phép biến đổi toán học

- **Kết nối đầy đặc.** Mỗi nơ-ron trong FC nhận đầu vào từ **toàn bộ** d phần tử của vector đặc trưng, với trọng số riêng biệt. Nếu ký hiệu $x \in \mathbb{R}^d$ là đầu vào đã flatten, hàm của một lớp FC được viết

$$z = Wx + b, \quad y = g(z),$$

trong đó $W \in \mathbb{R}^{m \times d}$ (ma trận trọng số), $b \in \mathbb{R}^m$ (vector bias) và $g(\cdot)$ là hàm kích hoạt (ReLU, tanh, v.v.). Vì **mọi cặp nơ-ron đều có liên kết**, tầng FC khai thác tương quan *toàn cục* giữa tất cả đặc trưng cục bộ đã học.

- **Bùng nổ tham số nếu đặt quá sớm.** Nếu ta kết nối đầy đủ ngay từ ảnh gốc, số trọng số tỷ lệ $O(n^2)$ với n pixel; một bức ảnh 1 MP có thể cần tới **9 nghìn tỷ** trọng số—hoàn toàn phi thực tế để huấn luyện. Nhờ các khối conv-pool giảm mạnh kích thước không gian, tầng FC cuối cùng chỉ còn phải học vài chục nghìn đến vài triệu tham số.

Vai trò chức năng

Giai đoạn	Mục đích chính	Chi tiết & lợi ích
Flatten	Biến tensor 3-chiều ($H' \times W' \times C'$) thành vector	Cho phép áp dụng đại số tuyến tính chuẩn và tối ưu hoá bằng back-prop dễ dàng.
FC trung gian	Tổng hợp & kết hợp đặc trưng	Học các kết hợp phi tuyến toàn cục (ví dụ “mũi + mắt + tai $\rightarrow$ chó”).
FC cuối + Softmax	Suy diễn nhãn lớp	Hàm softmax biến vector điểm số thành phân bố xác suất qua K lớp: $\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{k'} e^{z_{k'}}$.

Nhờ đó, tầng FC đóng vai trò “bộ lý luận cấp cao”, biến đặc trưng đa cấp thành quyết định phân loại cụ thể.

Kỹ thuật cải thiện FC trong thực tiễn

- **Dropout.** Vô hiệu ngẫu nhiên một phần nơ-ron trong khi huấn luyện nhằm giảm *overfitting* do số tham số lớn.
- **Global Average Pooling (GAP).** Thay thế hoàn toàn FC bằng phép lấy trung bình mỗi kênh; giảm tham số về 0, thích hợp cho mạng nhẹ trên thiết bị di động.
- **Regularization L2** và **Early-Stopping** giúp tránh việc FC “học thuộc” tập huấn luyện khi kích thước vector đầu vào còn lớn.

Minh hoạ dòng chảy dữ liệu

INPUT (224×224×3)

↓ Convolution + ReLU

↓ MaxPooling
 ↓ ... (lặp N lần)
 Feature Tensor ($7 \times 7 \times 512$)
 ↓ Flatten → 25 088 phần tử
 ↓ FC (4096) + ReLU
 ↓ FC (4096) + ReLU
 ↓ FC (1000) + Softmax → $P(y|x)$

Sơ đồ trên (dựa trên kiến trúc VGG-16) cho thấy tầng FC chuyển 25 088 đặc trưng không gian thành xác suất trên 1000 lớp, hoàn thiện chuỗi xử lý của CNN.

Lớp **Fully Connected** là cầu nối giữa **đặc trưng bậc cao** do các tầng tích chập học được và **bài toán suy diễn nhãn**. Bằng cách kết nối toàn phần và dùng các hàm kích hoạt phi tuyến, FC cho phép mạng **gộp thông tin toàn cục** và đưa ra dự đoán cuối cùng với độ chính xác cao. Tuy nhiên, cần kiểm soát số tham số bằng cách đặt FC **sau** các khối giảm kích thước và áp dụng các kỹ thuật regularization để duy trì khả năng khái quát hoá.

5.1.4. Kiến trúc CNN tiêu biểu

Sơ đồ tổng quát

INPUT → [CONV → ReLU → POOL] × N → FLATTEN → FULLY-CONNECTED → SOFTMAX OUTPUT

Mô hình trên gom các khối **Convolution–ReLU–Pooling** lặp N lần nhằm học dần các đặc trưng từ thấp tới cao, sau đó “làm phẳng” (flatten) để đưa vào các lớp kết nối đầy đủ thực hiện suy luận cuối cùng.

Khối Convolution → ReLU → Pooling

- **Convolution (CONV)**: mỗi lớp tích chập áp dụng d bộ lọc kích thước nhỏ (thường 3×3 hoặc 5×5) quét toàn ảnh; nhờ **chia sẻ trọng số**, số tham số chỉ tăng tuyến tính theo d thay vì theo diện tích ảnh. Kích thước vùng tiếp nhận (receptive field) của nơ-ron tăng theo chiều sâu, giúp mạng dần “nhìn rộng” hơn.
- **ReLU**: hàm kích hoạt *Rectified Linear Unit* ($\text{ReLU}(x) = \max(0, x)$) được gắn ngay sau phép tích chập để loại bỏ hiện tượng bão hoà gradient và tăng tốc hội tụ. AlexNet (2012) là kiến trúc đầu tiên chứng minh hiệu quả của ReLU trên tập ImageNet với 5 lớp CONV và 3 lớp FC.
- **Pooling (POOL)**: lớp gộp (thường Max-Pooling 2×2) giảm kích thước không gian, giữ thông tin tồn tại “ở đâu đó” trong vùng tiếp nhận, đồng thời giảm tham số và tăng bất biến dịch chuyển.

Lưu ý thiết kế

- **Số khối N** thường từ 2 đến 5: LeNet-5 dùng $N=2$, AlexNet $N \approx 5$, VGG-16/19 $N \approx 5$ khối 3×3 .
- **Stride & Padding** cân bằng giữa độ phân giải đặc trưng và chi phí tính toán.

- Nên thêm **Batch Normalization** và **Dropout** giữa các khối để ổn định phân phối kích hoạt và giảm overfitting.

Lớp Flatten

Sau khi đặc trưng cấp cao được học, tensor 3-chiều ($H \times W \times C$) được “trải phẳng” thành vector 1-chiều để kết nối với MLP phía sau.

Các lớp Fully-Connected (FC)

Các lớp FC thực hiện suy luận mức toàn cục: mỗi nơ-ron của lớp FC nối với mọi đặc trưng đầu vào, cho phép kết hợp thông tin rời rạc thành khái niệm trừu tượng (ví dụ “mặt mèo” hay “biển báo giao thông”). Thông thường:

Kiến trúc	Chuỗi FC điển hình	Tham số (triệu)
LeNet-5	$120 \rightarrow 84 \rightarrow 10$	≈ 0.06
AlexNet	$4096 \rightarrow 4096 \rightarrow 1000$	≈ 60
VGG-16	$4096 \rightarrow 4096 \rightarrow 1000$	≈ 138

Lớp Softmax Output

Lớp cuối cùng dùng **Softmax** để biến vector điểm số thành phân bố xác suất trên c nhãn, phù hợp với bài toán phân loại đa lớp.

Các biến thể tiêu biểu

- **LeNet-5 (1998)**: INPUT $32 \times 32 \rightarrow [(5 \times 5 \text{ CONV}) \rightarrow \text{tanh} \rightarrow \text{AVG-POOL}] \times 2 \rightarrow \text{FC} \rightarrow \text{Softmax}$; mở đường cho nhận dạng chữ viết tay.
- **AlexNet (2012)**: tăng chiều sâu, ReLU, dropout, huấn luyện trên GPU; lỗi ImageNet giảm > 10 điểm phần trăm.
- **VGG-16/19 (2014)**: đồng nhất kernel 3×3 , sâu 16–19 lớp, cho thấy “độ sâu quan trọng hơn độ rộng”.

Những kiến trúc mới hơn (GoogLeNet, ResNet, EfficientNet) bổ sung **Inception module**, **kết nối tắt (skip-connection)** hoặc **tỉ lệ hoá hợp chất** để đào sâu hơn mà không làm tăng đáng kể độ phức tạp.

Hướng dẫn triển khai thực nghiệm

- **Chọn kích thước ảnh**: 64×64 hoặc 224×224 phụ thuộc bài toán; luôn chuẩn hoá $[0, 1]$ hoặc chuẩn hoá z-score.
- **Khởi tạo mô hình (Pseudo-Keras)**:

```
model = Sequential([
    Conv2D(32,3,activation='relu',input_shape=(H,W,3)),
    MaxPool2D(),
    Conv2D(64,3,activation='relu'),
    MaxPool2D(),
```

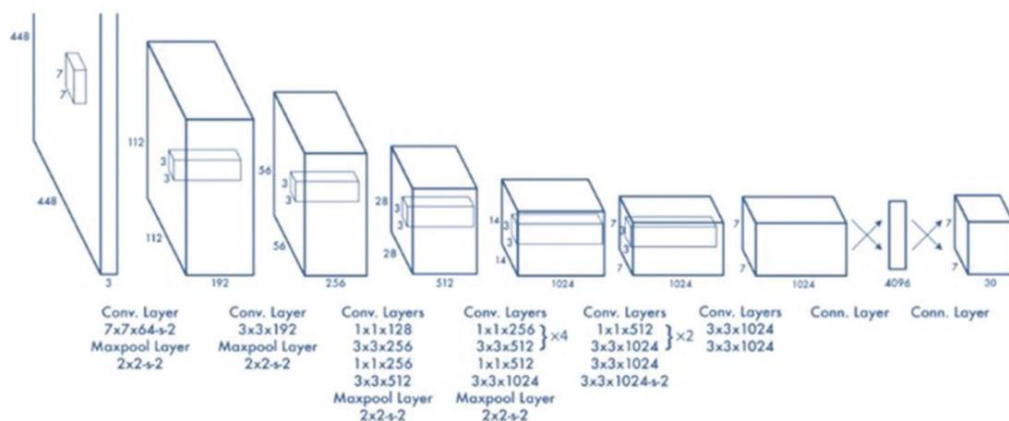
```

Conv2D(128,3,activation='relu'),
MaxPool2D(),
Flatten(),
Dense(256,activation='relu'),
Dropout(0.5),
Dense(num_classes,activation='softmax')
])

```

- **Huấn luyện** bằng Adam (learning-rate = 1e-3), **Early-Stopping** và **Data Augmentation** (xoay, lật, cắt) để cải thiện khả năng tổng quát.
- **Đánh giá** với Accuracy, Precision, Recall và ma trận nhầm lẫn; theo dõi overfitting qua đường *loss* và *val_loss*.

Phân kiến trúc CNN tiêu biểu giúp sinh viên hiểu quy luật “từ cục bộ đến toàn cục” của thị giác máy tính và là nền tảng để tiến tới các chủ đề nâng cao như **Transfer Learning**, **Object Detection** (YOLO, Faster R-CNN) và **Semantic Segmentation** (U-Net).



The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

Hình 101. Kiến trúc mạng YOLO (You Only Look Once) cho bài toán phát hiện đối tượng.

Hình 40.11 mô tả kiến trúc tổng thể của mạng YOLO, một mô hình phát hiện đối tượng theo thời gian thực nổi bật với hiệu suất cao. Mạng bao gồm:

- **24 lớp tích chập (Convolutional layers)** nối tiếp nhau để trích xuất đặc trưng ở nhiều cấp độ.
- **2 lớp kết nối đầy đủ (Fully connected layers)** ở cuối để đưa ra dự đoán.
- Xen giữa là các lớp **1×1 convolution** nhằm giảm số chiều đặc trưng và **lớp pooling** để giảm kích thước không gian.
- Các tầng được huấn luyện trước (pretrained) trên tập ImageNet với kích thước ảnh 224×224 , sau đó tăng gấp đôi độ phân giải cho nhiệm vụ phát hiện đối tượng.

Mô hình này hoạt động theo nguyên tắc “nhìn một lần duy nhất” (You Only Look Once): toàn bộ ảnh đầu vào được xử lý chỉ trong một lần lan truyền (forward pass), cho phép phát hiện nhiều đối tượng cùng lúc với tốc độ cao và độ chính xác đáng kể.

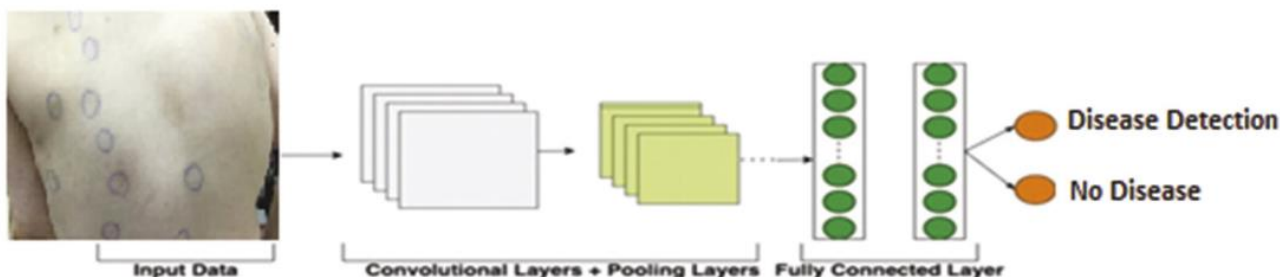
5.1.5. Ứng dụng và Hướng dẫn Dự án

Trong giai đoạn này, sinh viên được khuyến khích tự nghiên cứu và hiện thực hóa kiến thức CNN vào hai nhóm bài toán thị giác máy tính kinh điển: **phân loại ảnh (image classification)** và **nhận dạng/định vị đối tượng (object detection)**. Phần dưới đây nêu ra cơ sở lý thuyết, quy trình triển khai, chỉ số đánh giá và các ý tưởng dự án mẫu.

A. Phân loại ảnh (Image Classification)

Mục tiêu	Mô tả ngắn
Đầu ra	Gán một nhãn duy nhất cho toàn bộ ảnh (ví dụ: “chó” hay “mèo”).
Mạng điển hình	Chuỗi CONV → RELU → POOL lặp N lần, sau đó Flatten → Fully-Connected .

- **Quy trình chuẩn**
 - a. **Thu thập & chuẩn hóa dữ liệu:** cân bằng số mẫu/nhãn, chia train–val–test, và áp dụng tăng cường dữ liệu (flip, crop, noise).
 - b. **Huấn luyện mô hình:** lựa chọn kiến trúc (VGG, ResNet, EfficientNet), tối ưu hàm mất mát Soft-max Cross-Entropy.
 - c. **Đánh giá:** Accuracy, Precision-Recall, F-score hoặc AUC tùy bài toán.
- **Ví dụ dự án gợi ý**
 - **Dog vs Cat:** dùng tập Cats & Dogs 25 000 ảnh, thử ResNet18 và so sánh với Vision Transformer (ViT) về tốc độ - độ chính xác. CNN 3 khối CONV-POOL đã cho thấy khả năng học đặc trưng đa chiều ở quy mô tương tự.
 - **Phân loại hãng xe:** tự thu thập 1 000 ảnh/nhãn (sedan, SUV, hatchback) và đánh giá sai lệch giữa điều kiện trong nhà/ngoài trời.
 - **Chẩn đoán da liễu:** bài báo “Convolutional Neural Networks for the Identification of Skin Disorders” mô tả pipeline 12 000 ảnh, CNN + Softmax, Precision ≈ 95 %.



Hình 102. CNN 3 lớp tích chập với stride = 2 dùng cho phân loại bệnh da.

B. Nhận dạng đối tượng (Object Detection)

Mục tiêu	Mô tả ngắn
Đầu ra	Xác định vị trí (bounding box) + nhãn của tất cả đối tượng trong ảnh.
Mạng điển hình	Một khối trích đặc trưng (Backbone CNN) + head dự báo box và class (YOLO, SSD, Faster-R-CNN).

- **Kiến trúc tiêu biểu**

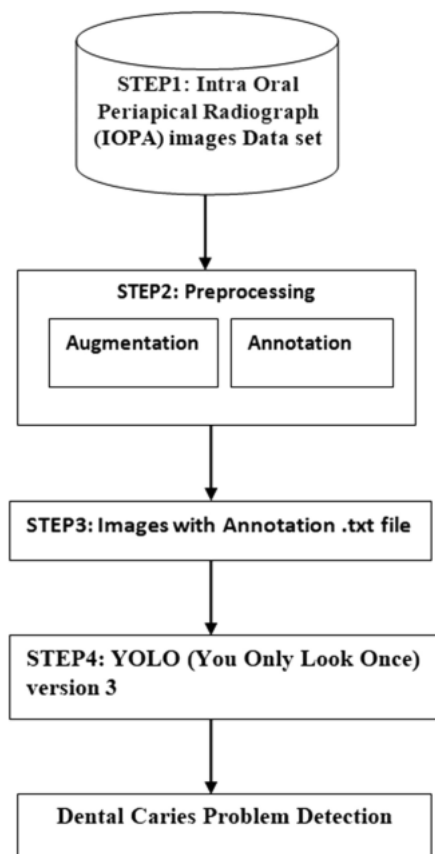
- **YOLO (You Only Look Once)**: chia ảnh thành lưới, hồi quy đồng thời toạ độ hộp & xác suất lớp trong **một** lượt suy diễn. Bản YOLOv8 kế thừa triết lý “single-stage” giúp xử lý thời gian thực cho xe tự lái, camera an ninh.
- **SSD (Single Shot Multibox Detector)**: đa tỉ lệ hộp mặc định, cân bằng tốc độ – độ chính xác, phù hợp ảnh y tế độ phân giải cao.

- **Quy trình chuẩn**

- Ghi nhãn** (LabelImg, COCO JSON).
- Huấn luyện** (Anchor boxes, non-max suppression). Bảng 6.1 minh chứng YOLOv3 đạt Precision 74 %, Sensitivity 76 % trên bộ ảnh nha khoa.
- Đánh giá**: mAP@IoU=0.5, FPS, kích thước mô hình.
- Tối ưu**: Pruning, Quantization để suy diễn trên thiết bị nhúng.

- **Ví dụ dự án gợi ý**

- **Blind-Assist**: mô hình YOLOv5-nano trên Raspberry Pi nhận dạng chương ngại vật, phát âm thanh cảnh báo; cảm hứng từ đề tài “Blind People Assistant” trong sách Algorithms in AAI (mục lục dự án số 37).
- **Phát hiện khối u X-ray**: SSD kết hợp Cross-Measure Refinement & Canny Edge giúp Precision $\approx 95\%$ trên 10 000 ảnh CT, như quy trình nêu ở bảng 16.4.
- **Giám sát giao thông**: huấn luyện YOLOv8 trên tập VisDrone, xuất tốc độ khung hình > 30 FPS để đếm xe.



Hình 103. Sơ đồ kiến trúc đề xuất cho hệ thống chẩn đoán bệnh răng miệng.

C. Tiêu chí lựa chọn hướng dẫn dự án cuối kỳ

Tiêu chí	Gợi ý mức đạt
Độ mới	Sử dụng kiến trúc < 3 năm tuổi (YOLOv8, ConvNeXt, ViT-Hybrid).
Quy mô dữ liệu	$\geq 1\,000$ ảnh/nhãn với ≥ 3 lớp (classification) hoặc $\geq 1\,000$ ảnh có bbox (detection).
Độ sâu phân tích	Báo cáo cần so sánh ≥ 2 mô hình, phân tích lỗi dạng <i>confusion matrix</i> & <i>error heatmap</i> .
Tính ứng dụng	Trình bày kịch bản triển khai (edge device, WebAPI, mobile).

Sinh viên hãy chọn **một đề tài** trong số gợi ý hoặc đề xuất mới, lập kế hoạch thu thập dữ liệu, thiết kế mô hình, và chuẩn bị thuyết trình kết quả thực nghiệm trước lớp. Các tài nguyên khung code (PyTorch Lightning, Ultralytics YOLO) có thể dùng để tăng tốc triển khai, nhưng sinh viên phải **hiểu và giải thích rõ** luồng dữ liệu qua từng lớp mạng.

5.2. Mạng Nơ-ron Hồi quy (Recurrent Neural Networks – RNNs) cho Dữ liệu Chuỗi

5.2.1 Vấn đề với CNN/MLP khi xử lý chuỗi

Các kiến trúc **MLP** và **CNN** đã chứng tỏ hiệu quả với dữ liệu *không có thứ tự* (ảnh tĩnh, bảng thuộc tính rời rạc). Tuy nhiên, khi áp dụng trực tiếp cho **chuỗi có tính thời gian** – tiếng nói, văn bản, tín hiệu cảm biến – chúng bộc lộ bốn hạn chế nền tảng:

Hạn chế	Phân tích chi tiết
Giả định độc lập (IID)	MLP/CNN xử lý từng phần tử đầu vào tách biệt, coi chúng là các biến độc lập. Do đó, mạng không “nhớ” quá khứ , bỏ qua phụ thuộc theo thời gian – điều vốn tạo nên nghĩa của câu, giai điệu, hay xu thế chuỗi số liệu.
Kích thước cố định & Cửa sổ ngắn	Feed-forward buộc phải “trái” chuỗi vào một <i>vector độ dài cố định</i> ; nếu chuỗi dài vượt cửa sổ, phần đầu sẽ bị cắt bỏ. Cách làm này khiến mạng “chỉ nhìn” được một đoạn ngắn và thất bại trong việc phát hiện các phụ thuộc xa .
Bất đối xứng tham số	Cùng một từ “him” ở vị trí thứ 12 và thứ 20 của câu sẽ phải được học <i>hai lần</i> vì lớp fully-connected dùng trọng số khác nhau cho mỗi vị trí , gây lãng phí và khó khái quát hoá.
Thiếu bộ nhớ ngữ cảnh	Dù CNN có thể mở rộng “receptive field” bằng kernel lớn hoặc xếp chồng tầng, thông tin vẫn bị suy giảm qua từng bước tích chập ; mạng không có biến trạng thái chung để lưu toàn bộ lịch sử như yêu cầu của chuỗi ngôn ngữ hay âm thanh dài.

Hệ quả: Với chuỗi lời nói “tôi *rất* thích *AP*”, một MLP/CNN tiêu chuẩn sẽ khó nhận ra “*rất*” bỏ nghĩa cho “thích” vì hai từ này cách nhau vài bước; dự đoán phụ thuộc sẽ sai lệch, hay mô hình thời tiết ngắn hạn sẽ không “nhớ” xu thế nhiều ngày trước.

Những hạn chế trên tạo tiền đề cho **RNN** – một kiến trúc *có vòng lặp nội bộ* duy trì **trạng thái ẩn** qua thời gian, khắc phục căn bản ba vấn đề “thiếu trí nhớ – cửa sổ ngắn – bất đối xứng” và trở thành nền tảng cho xử lý ngôn ngữ, chuỗi số liệu, âm nhạc ... (các mục 5.2.2 → 5.2.6 sẽ trình bày chi tiết).

5.2.2 Giải pháp của RNN – Vòng lặp và Trí nhớ

1. Trực giác sinh học và ý tưởng “trạng thái ẩn”

Trong não người, khi nghe một câu nói hay giai điệu, các nơ-ron ở thùy thái dương duy trì **hoạt động điện thế** trong một quãng thời gian ngắn, giúp ta liên kết âm thanh hiện tại với những gì vừa nghe. **Recurrent Neural Network (RNN)** mô phỏng cơ chế đó bằng cách **bổ sung một vòng lặp nội bộ**: thay vì chỉ nhận đầu vào x_t tại bước thời gian t , mỗi nơ-ron còn nhận cả **trạng thái ẩn** h_{t-1} kết xuất từ bước trước. Thành phần h_t vì thế **lưu dấu vết quá khứ** – đóng vai trò “bộ nhớ” của mạng.

2. Sơ đồ kiến trúc và phép “trái thời gian”

- **Sơ đồ vòng lặp** (hình 22.8a, AIMA): một tầng ẩn h tự nối lại chính nó qua một **độ trễ** Δ , tạo chu trình trong đồ thị tính toán.
- **Trái (unroll)**: khi kéo chu trình ra theo trục thời gian, ta thu được một chuỗi các bản sao **cùng chung trọng số** – đây là **chia sẻ tham số theo thời gian**, khắc phục bất đối xứng vị trí mà MLP gặp phải.

3. Công thức cập nhật trạng thái

Ở mỗi bước t :

$$h_t = f(W_{hx} x_t + W_{hh} h_{t-1} + b_h), \quad y_t = g(W_{yh} h_t + b_y)$$

trong đó f, g là hàm kích hoạt (thường **tanh** hoặc **ReLU**). Ma trận W_{hh} lặp lại ở **mọi** bước nên số tham số **không phụ thuộc** độ dài chuỗi.

Đặc tính này cho phép RNN “nhìn lui vô hạn” về quá khứ trên lý thuyết; thực nghiệm cho thấy mạng thật sự lưu trữ những khía cạnh **hữu ích** của ngữ cảnh nhờ quá trình huấn luyện khuyến khích phân bổ bộ nhớ hợp lý.

4. Ưu điểm cốt lõi

Vấn đề của MLP/CNN	Cách RNN khắc phục
Không nhớ quá khứ	Trạng thái ẩn h_t tích lũy thông tin lịch sử
Cửa sổ ngữ cảnh cố định	Vòng lặp giúp bối cảnh mở rộng linh hoạt theo thời gian
Bất đối xứng trọng số	Chia sẻ tham số: cùng W_{hx}, W_{hh} cho mọi vị trí, giảm bùng nổ tham số và tăng khả năng khái quát hoá

5. Liên hệ với mô hình xác suất động

AIMA chỉ ra: h_t đóng vai trò **biến ẩn Markov** gói gọn toàn bộ lịch sử đầu vào – tương tự trạng thái ẩn của **HMM** hay **Dynamic Bayesian Network**, nhưng với mức độ **phi tuyến** cao hơn nhờ hàm f có thể là mạng sâu.

6. Tóm lược

RNN giải quyết triệt để bài toán **thứ tự và phụ thuộc thời gian** bằng **vòng lặp hồi quy** và **trạng thái ẩn** – một “ô nhớ” được cập nhật liên tục. Cơ chế này đặt nền móng cho các kiến trúc nâng cao như **LSTM** và **GRU**, đồng thời mở đường cho những đột phá trong **xử lý ngôn ngữ tự nhiên, nhận dạng tiếng nói và dự báo chuỗi thời gian**, sẽ được trình bày ở các mục tiếp theo.

5.2.3 Kiến trúc RNN cơ bản

(a) Sơ đồ “một bước thời gian”

Ở **bước thời gian** t , mạng nhận **đầu vào** x_t và **trạng thái ẩn** từ bước trước h_{t-1} . Toàn bộ phép biến đổi được mô tả bởi hai phương trình:

$$h_t = f(W_{hx} x_t + W_{hh} h_{t-1} + b_h) \quad y_t = g(W_{yh} h_t + b_y)$$

Trong đó f, g thường là *tanh*, *ReLU* hoặc *soft-max* trên lớp ra. Các **ma trận trọng số** W_{hx}, W_{hh}, W_{yh} và các **vector bias** b_h, b_y **được dùng chung** cho mọi thời điểm, nên số tham số **không phụ thuộc** vào độ dài dãy dữ liệu.

(b) “Trải” mạng qua trục thời gian

Một RNN có vòng lặp nội bộ (kí hiệu Δ) có thể **trải phẳng (unroll)** thành chuỗi mạng con chia sẻ tham số, biến đồ thị có chu trình thành **mạng feed-forward sâu theo trục thời gian** (Hình 5-9).

Việc unroll giúp ta:

- **Hình dung luồng dữ liệu:** $x_1 \rightarrow h_1 \rightarrow y_1, x_2 \rightarrow h_2 \rightarrow y_2, \dots$
- Áp dụng **back-propagation through time (BPTT)** – lan truyền ngược qua các “lớp thời gian” để cập nhật cùng một tập trọng số.

(c) Chức năng của trạng thái ẩn

Vai trò	Giải thích
Bộ nhớ có giới hạn	h_t mã hoá <i>tóm tắt</i> toàn bộ lịch sử $(x_1, \dots, x_t)$; đây chính là giả thiết Markov bậc 1 ở dạng phi tuyến: h_t đủ để dự đoán tương lai mà không cần lưu toàn bộ dãy.
Chia sẻ tham số	Mọi vị trí trong chuỗi dùng chung W_{hh} , khắc phục “bất đối xứng” trọng số của MLP/CNN, giúp mô hình tận dụng kiến thức học được ở vị trí này cho vị trí khác.
Phạm vi ngữ cảnh linh hoạt	Trên lý thuyết, h_t có thể truyền ảnh hưởng của x_1 đến vô hạn bước; thực tế bị giới hạn bởi hiện tượng <i>tiêu biến/bùng nổ gradient</i> (trình bày ở mục 5.2.4).

(d) Diễn giải trực quan

Hình 5-9 Mạng RNN cơ bản (a) ở dạng vòng; (b) “trải” qua ba bước thời gian. Các khối màu xám dùng **cùng** trọng số. (Nguồn: Stuart J. Russell & Peter Norvig – *Artificial Intelligence: A Modern Approach*, 4^e ed., Fig. 22.8)

Sinh viên cần nhận diện hai điểm mấu chốt trên hình:

- **Vòng lặp** tạo trạng thái ẩn có độ trễ Δ .
- **Unroll** biến RNN thành mạng sâu chiều T , thuận lợi cho BPTT và mô tả bằng đồ thị không chu trình.

(e) Quy ước ký hiệu trong giáo trình

Ký hiệu	Nghĩa
x_t	Vector đầu vào tại bước thời gian t
h_t	Trạng thái ẩn sau khi xử lý x_t
y_t	Vector đầu ra tại bước t
W_{hx}	Trọng số từ đầu vào sang ẩn
W_{hh}	Trọng số hồi quy (ẩn $\rightarrow$ ẩn)
W_{yh}	Trọng số từ ẩn sang ra

Ký hiệu	Nghĩa
f, g	Hàm kích hoạt (bên trong / ngoài)

Những công thức, sơ đồ trên sẽ là nền tảng cho các kiến trúc **LSTM**, **GRU** và **Attention RNN** sẽ khảo sát ở các mục sau.

5.2.4 Vấn đề của RNN đơn giản — Gradient tiêu biến và bùng nổ

Khi huấn luyện RNN bằng **lan truyền ngược qua thời gian** (*Back-Propagation Through Time* – *BPTT*), đạo hàm của hàm mất mát L đối với trọng số hồi quy W_{hh} chứa **tích lũy các đạo hàm theo từng bước**:

$$\frac{\partial L}{\partial W_{hh}} \propto \sum_{t=1}^T \left(\prod_{k=1}^t g'(a_k) \right) W_{hh}^t, \quad g' = \frac{df}{da},$$

trong đó $g'(a_k)$ là đạo hàm của hàm kích hoạt tại bước k . Ta nhận thấy:

- Nếu $|g'| \leq 1$ và **giá trị riêng lớn nhất** của W_{hh} nhỏ hơn 1, tích trên **giảm lũy thừa** khi t tăng, làm cho gradient **tí hon** tới mức số học dấu chấm động không thể phân biệt được → **tiêu biến** (*vanishing*).
- Ngược lại, nếu $|g'|$ hoặc $\|W_{hh}\|$ vượt quá 1, tích trên **phóng đại** theo cấp số nhân, khiến gradient **vượt tầm** biểu diễn số học → **bùng nổ** (*exploding*).

1. Nguyên nhân gốc rễ

Hiện tượng	Cơ chế toán học	Tình huống thường gặp
Tiêu biến	$g' \leq 1, \lambda_{\max}(W_{hh}) < 1 \Rightarrow \prod g' \lambda_{\max}^t \rightarrow 0$	Hàm sigmoid , tanh bão hoà, trọng số nhỏ khi khởi tạo
Bùng nổ	$\lambda_{\max}(W_{hh}) > 1$ hoặc đạo hàm ReLU lớn	Khởi tạo trọng số quá lớn, chuỗi đầu vào dài bất thường

2. Hậu quả đối với học máy

- Mất phụ thuộc xa**: Khi gradient tiêu biến, tín hiệu lỗi từ thời điểm t không lan ngược được đến các bước sớm hơn $\Rightarrow$ mạng không học được quan hệ dài hạn (*long-range dependencies*).
- Huấn luyện bất ổn**: Gradient bùng nổ làm cập nhật trọng số đột ngột, hàm mất mát dao động hoặc *NaN*, buộc phải giảm tốc độ học hoặc cắt tỉa (*gradient clipping*).
- Hạn chế độ sâu thời gian**: Thực nghiệm cho thấy RNN thuần túy khó huấn luyện với chuỗi $> 20\text{--}30$ bước nếu không có biện pháp khắc phục.

3. Gợi mở hướng khắc phục

Các kiến trúc **LSTM** và **GRU** (mục 5.2.5) bổ sung **cổng điều khiển** và **đường truyền tuyến tính** để bảo tồn gradient, trong khi kỹ thuật **gradient clipping**, **khởi tạo orthogonal**, hay **sử dụng hàm kích hoạt ReLU-6** cũng giúp giảm thiểu tiêu biến/bùng nổ.

Kết luận: Hiện tượng *vanishing / exploding gradient* là thách thức trung tâm của RNN cơ bản. Nắm rõ nguồn gốc toán học giúp sinh viên lựa chọn kiến trúc và kỹ thuật tối ưu hoá phù hợp khi xử lý các chuỗi dài.

5.2.5. Giải pháp: Long Short-Term Memory (LSTM) và Gated Recurrent Unit (GRU)

1. Động lực xuất hiện

Sau khi RNN cơ bản được chứng minh là khó huấn luyện do **gradient tiêu biến/bùng nổ**, cộng đồng nghiên cứu đã đề xuất các kiến trúc “có cổng” nhằm điều khiển dòng thông tin qua thời gian. LSTM và GRU là hai biến thể phổ biến nhất của hướng đi này.

2. Kiến trúc LSTM

Thành phần	Chức năng chính
Cell State c_t	“Dải băng truyền” bảo tồn thông tin lâu dài xuyên suốt chuỗi thời gian
Cổng Quên f_t	Quyết định phần nào của c_{t-1} được giữ lại
Cổng Đầu vào i_t	Chọn thông tin mới cần ghi vào bộ nhớ
Cổng Đầu ra o_t	Điều khiển lượng thông tin từ c_t phát ra h_t

Phương trình cập nhật trọng tâm:

So sánh nhanh

- **Số cổng:** LSTM = 3 ↔ GRU = 2 → GRU nhẹ hơn.
- **Khả năng mô hình hoá dài hạn:** LSTM $\gtrsim$ GRU.
- **Tốc độ huấn luyện:** GRU nhanh hơn ~20–50 %.
- **Thực nghiệm phần mềm:** GRU chính xác 95 % > LSTM 92 % trong bài toán ước lượng nỗ lực phần mềm .

4. Khi nào chọn LSTM hay GRU?

Ngữ cảnh	Nên dùng
Chuỗi rất dài, cần kiểm soát nhớ/quên chi tiết	LSTM
Thiết bị biên, thời gian thực, tài nguyên hạn chế	GRU

Tóm lại, LSTM và GRU mở đường cho học sâu trên dữ liệu chuỗi bằng cách đưa “trí nhớ” có kiểm soát vào mạng thần kinh, khắc phục gốc rễ của RNN truyền thống và trở thành nền tảng cho các kiến trúc cao hơn như Bi-LSTM hay Transformer.

5.2.6 Ứng dụng và Hướng dẫn Dự án

Trong phần này, sinh viên được định hướng vận dụng **RNN/LSTM/GRU** đã học vào ba bài toán kinh điển của xử lý ngôn ngữ tự nhiên (NLP). Mỗi tiêu mục trình bày: *mô hình khái niệm, quy trình triển khai, chỉ số đánh giá và gợi ý dự án cuối kỳ*.

A. Phân tích cảm xúc (Sentiment Analysis)

Khía cạnh	Nội dung
Mục tiêu	Gán nhãn <i>tích cực / tiêu cực</i> (hoặc đa cấp) cho một đơn vị văn bản (đánh giá sản phẩm, bình luận mạng xã hội).
Mô hình đề xuất	<i>Bi-LSTM</i> hoặc <i>GRU</i> trên chuỗi word-embedding (GloVe, Word2Vec); embedding cho phép mạng “cảm” sắc thái từ <i>awesome</i> ↔ <i>cringeworthy</i> .
Pipeline chuẩn	1) Thu thập & làm sạch dữ liệu (loại stop-word, chuẩn hóa emoji), 2) Token hóa + nhúng, 3) Bi-LSTM → Dense → Softmax, 4) Đánh giá bằng Accuracy, F1 .
Chỉ báo thực nghiệm	Nghiên cứu trong <i>Algorithms in AAI</i> cho thấy LSTM vượt SVM ≈ 5 điểm chính xác trên tập bình luận sinh viên .
Ý tưởng dự án	“ <i>ReviewGuard</i> ” – Bộ lọc bình luận gian lận cho sàn TMĐT Việt Nam: huấn luyện 50 k tweet tiếng Việt, fine-tune GRU nhẹ để suy diễn real-time.

B. Tạo văn bản (Text Generation)

Khía cạnh	Nội dung
Mục tiêu	Sinh chuỗi ký tự/ngôn từ mới tiếp nối ngữ cảnh—ví dụ thơ lục bát, lời bài hát.
Mô hình đề xuất	RNN-LM hoặc LSTM-LM 1 chiều: mô hình hóa phân bố $P(w_{t+1})$
Kỹ thuật then chốt	<i>Temperature sampling, top-k nucleus sampling</i> để cân bằng tính đa dạng & ngữ pháp.
Đánh giá	Log-Perplexity và kiểm tra loài người (Turing-like).
Ý tưởng dự án	“ <i>Lục Bát Bot</i> ” – Hệ thống sinh thơ Đường luật: huấn luyện trên 10 k bài thơ Việt, tinh chỉnh trọng số <i>rhyme loss</i> để giữ vần.

C. Dịch máy (Neural Machine Translation – NMT)

Khía cạnh	Nội dung
Mục tiêu	Ánh xạ câu nguồn (EN) sang câu đích (VI) giữ nguyên ý nghĩa.
Mô hình đề xuất	Encoder–Decoder LSTM (<i>seq2seq</i>) ; đã thay thế n-gram do khả năng khái quát và truyền ngữ cảnh dài .
Mở rộng	Thêm <i>Attention</i> → cải thiện dịch câu dài; Transformer sau này nâng cao hơn .
Quy trình	1) Thu thập cặp song ngữ (TED-Talks, Tatoeba), 2) Tiền xử lý: true-case, BPE, 3) Huấn luyện LSTM <i>seq2seq</i> + Attention, 4) Đánh giá bằng BLEU, TER .

Khía cạnh	Nội dung
Ý tưởng dự án	“Edu-EnVi” – Bộ dịch giáo trình Anh → Việt: fine-tune mô hình trên 100 k cặp câu chuyên ngành CNTT; tích hợp giao diện web demo.

D. Lộ trình triển khai dự án cuối kỳ

- **Khởi tạo kho dữ liệu:** ít nhất 10 000 mẫu cho SA, 1 MB corpus cho Text Gen, 50 000 cặp câu cho NMT.
- **Chọn kiến trúc & siêu tham số:** LSTM 2 layer 256 units (SA); LSTM 3 layer 512 units (Text Gen); Encoder 2×512 , Decoder 2×512 + Attention (NMT).
- **Huấn luyện & theo dõi:** dùng Adam ($lr = 1e-3$), Early-Stopping, TensorBoard.
- **Đánh giá & báo cáo:** trình bày metric, biểu đồ loss, phân tích lỗi; so sánh baseline (n-gram, SVM) với mô hình sâu.
- **Triển khai thử nghiệm:** REST API (FastAPI) hoặc nhúng trên thiết bị edge (Raspberry Pi đối với SA âm thanh).

Ba hướng dự án trên minh họa sức mạnh “trí nhớ” của RNN/LSTM/GRU trong việc nắm bắt phụ thuộc thời gian dài—từ cảm xúc trong câu, nhịp điệu thơ cho tới ánh xạ ngôn ngữ. Việc bám sát quy trình dữ liệu-mô hình-đánh giá sẽ giúp sinh viên chuyên hóa lý thuyết thành sản phẩm AI khả thi.

5.3.1 Vấn đề với RNN

Các mạng nơ-ron hồi quy (RNN) – kể cả những biến thể tiên tiến như LSTM hay GRU – đã đem lại bước tiến lớn cho xử lý ngôn ngữ tự nhiên và dữ liệu thời gian. Tuy nhiên, bản chất **tuần tự** của RNN khiến chúng bộc lộ nhiều hạn chế khi mô hình hoá những chuỗi rất dài hoặc khi cần huấn luyện ở quy mô công nghiệp. Dưới đây là các vấn đề cốt lõi:

Nhược điểm	Giải thích học thuật	Hệ quả thực tế
Tính toán tuần tự (sequential)	Ở bước thời gian t , trạng thái ẩn h_t phụ thuộc vào h_{t-1} ; do đó các bước phải thực hiện lần lượt , không thể khai thác song song GPU trên toàn bộ chuỗi đầu vào.	Tốc độ huấn luyện chậm, khó mở rộng trên cụm máy tính lớn; thời gian huấn luyện tăng tuyến tính theo chiều dài chuỗi.
Gradient tiêu biến/bùng nổ	Quá trình lan truyền ngược qua nhiều bước thời gian làm độ lớn gradient có thể rất nhỏ (tiêu biến) hoặc rất lớn (bùng nổ). LSTM giảm thiểu nhưng không loại bỏ hoàn toàn.	Mạng khó học các quan hệ phụ thuộc xa; phải dùng kỹ thuật cắt gradient (gradient clipping) hoặc khởi tạo tham số đặc biệt.
Khả năng ghi nhớ dài hạn hữu hạn	Kích thước vector trạng thái ẩn cố định, nên dung lượng “bộ nhớ” bị giới hạn; khi chuỗi quá dài, thông tin sớm dễ bị “quên”.	Chất lượng mô hình giảm rõ rệt với văn bản dài cả trăm, nghìn token; cần cơ chế

Nhược điểm	Giải thích học thuật	Hệ quả thực tế
		attention hoặc các kiến trúc mới.
Chi phí tính toán cao	Độ phức tạp $O(T \times d^2)$ (với T là độ dài, d là kích thước ẩn) cộng với không thể song song hoá làm RNN “đắt đỏ” hơn các mô hình tự đồng thời.	Tốn năng lượng, khó huấn luyện trên tập dữ liệu hàng tỷ câu; hạn chế khả năng mở rộng.

Như vậy ngay cả khi cải tiến bằng LSTM/GRU, RNN vẫn *vướng* vào “nút thắt tuần tự”. Điều đó dẫn đến nhu cầu về những kiến trúc **không tuần tự** – tiêu biểu là **Transformer** với cơ chế *self-attention*, cho phép xử lý toàn chuỗi trong một phép nhân ma trận và nắm bắt phụ thuộc xa mà **không** phải chờ các bước trước đó. Trong các mục kế tiếp, chúng ta sẽ khảo sát cách Transformer khắc phục các nhược điểm này và mở đường cho thời đại **Mô hình Ngôn ngữ Lớn (LLMs)**.

5.3.2 Giải pháp của Transformer – “Attention Is All You Need”

1. Từ RNN đến Transformer

Bài báo “**Attention is All You Need**” của Vaswani et al. (2017) giới thiệu kiến trúc **Transformer**, thay thế toàn bộ vòng lặp hồi quy của RNN/LSTM bằng một cơ chế duy nhất gọi là **self-attention**. Self-attention cho phép mô hình **mô tả quan hệ dài hạn mà không phụ thuộc tuần tự**, tức mỗi từ “nhìn” đồng thời mọi từ khác trong câu.

2. Cấu trúc tổng quan

Khối	Chức năng cốt lõi	Thành phần chính
Encoder (× N)	Mã hoá câu nguồn	<i>Multi-Head Self-Attention</i> → <i>Feed-Forward</i> (+ <i>Residual</i> & <i>LayerNorm</i>)
Decoder (× N)	Sinh câu đích	<i>Masked Self-Attention</i> → <i>Cross-Attention</i> (decoder ↔ encoder) → <i>Feed-Forward</i>
Positional Encoding	Bổ sung thông tin vị trí vì mạng không còn tuần tự	sin, cos theo tần số hoặc embedding học được

Nhờ **song song hoá phép nhân ma trận**, toàn bộ câu được xử lý **cùng lúc** trên GPU/TPU, tăng tốc huấn luyện đáng kể.

3. Cơ chế Self-Attention

Đối với mỗi vị trí i trong chuỗi, mô hình học ba véc-tơ **Query** q_i , **Key** k_i , **Value** v_i (ma trận trọng số W_q, W_k, W_v). Điểm chú ý giữa hai từ i, j :

$$\alpha_{ij} = \text{softmax}\left(\frac{q_i \cdot k_j}{\sqrt{d}}\right), \quad z_i = \sum_j \alpha_{ij} v_j$$

Multi-head attention lặp lại phép tính trên **h** đầu độc lập rồi ghép kết quả, giúp mạng đồng thời “tập trung” vào nhiều kiểu quan hệ.

4. Vì sao Transformer vượt trội?

Hạn chế cũ	Cách Transformer khắc phục
Tuần tự chậm (RNN)	Self-attention xử lý toàn chuỗi bằng phép nhân ma trận $\Rightarrow$ song song trên phần cứng hiện đại.
Phụ thuộc xa suy yếu	Trọng số α_{ij} nối trực tiếp mọi cặp từ $\Rightarrow$ độ dài đường truyền = 1, không tiêu biến gradient.
Bộ nhớ ắc cố định	Attention không bị giới hạn kích thước vector; mô hình linh hoạt với câu rất dài.

5. Tương quan với Mô hình Ngôn ngữ Lớn (LLMs)

Khả năng nắm bắt quan hệ dài hạn và huấn luyện song song giúp Transformer **mở lối cho các mô hình ngôn ngữ cực lớn** như BERT, GPT-3, Llama...—tất cả đều dựa trên self-attention để học biểu diễn ngữ cảnh sâu sắc từ hàng tỷ câu .

6. Tóm tắt

Kiến trúc Transformer **loại bỏ hồi quy, thay bằng self-attention**, bổ sung *multi-head*, *positional encoding* và *feed-forward* có residual, cho phép:

- Huấn luyện **nhANH** nhờ song song hoá;
- **Ghi nhớ** phụ thuộc xa vượt giới hạn RNN;
- Trở thành nền tảng của **LLMs** đương đại.

Các mục tiếp theo sẽ đào sâu cơ chế self-attention, positional encoding và cách mở rộng Transformer cho đa phương thức.

5.3.3 Cơ chế *Self-Attention*

(a) Trực giác

Trong **self-attention**, mỗi từ (token) trong câu được phép “nhìn” đồng thời tất cả các từ khác để quyết định thành phần nào quan trọng nhất cho ý nghĩa hiện tại. Nhờ đó cùng một từ *bank* sẽ nhận khác nhau khi đi với *river* hay *money*—một cách tự động gán “ngữ cảnh” cho ký hiệu.

(b) Cơ chế toán học

- **Biểu diễn Q (Queries), K (Keys), V (Values)**. Mỗi embedding x_i được chiếu tuyến tính thành ba véc-tơ q_i, k_i, v_i bằng các ma trận W_q, W_k, W_v học được.
- **Tính trọng số chú ý**.

$$\alpha_{ij} = \text{softmax}(q_i \cdot k_j / \sqrt{d})$$

- **Kết hợp ngữ cảnh**.

$$z_i = \sum_j \alpha_{ij} v_j$$

Công thức này tạo embedding mới z_i phản ánh mối liên hệ giữa từ i và toàn bộ câu.

- **Multi-Head.** Thay vì một cặp (Q, K, V) , Transformer dùng h đầu độc lập rồi ghép chúng, giúp mạng đồng thời học nhiều kiểu quan hệ cú pháp và ngữ nghĩa khác nhau.

(c) Ưu điểm so với RNN/LSTM

Khía cạnh	Self-Attention
Song song hoá	Tất cả từ được xử lý trong một phép nhân ma trận; triển khai hiệu quả trên GPU/TPU
Phụ thuộc xa	Mỗi cặp từ kết nối trực tiếp (độ dài đường truyền = 1) nên không suy giảm tín hiệu khi câu dài; mô hình hoá ngữ cảnh dài hạn dễ hơn RNN
Số bước thời gian	Không cần lan truyền tuần tự $\rightarrow$ thời gian huấn luyện giảm đáng kể so với RNN vốn phải xử lý từng token một

(d) Đặt vào kiến trúc Transformer

Self-attention là “khôi lỗi” trong **Encoder** và **Decoder** của Transformer; mọi tầng khác (feed-forward, residual, layer-norm) chỉ đóng vai trò biến đổi bổ sung. Việc thay thế vòng lặp hồi quy bằng self-attention giúp Transformer **loại bỏ hoàn toàn ràng buộc tuần tự** nhưng vẫn giữ (và còn mở rộng) khả năng mô hình hoá phụ thuộc xa.

(e) Tóm lược

Self-attention đem lại hai lợi thế quyết định:

- **Tập trung linh hoạt** vào các phần tử liên quan trong chuỗi, bất kể khoảng cách.
- **Tận dụng tối đa** phần cứng tính song song hiện đại.

Vì vậy, nó trở thành nền tảng cho **Transformers** và, rộng hơn, cho thế hệ **Mô hình Ngôn ngữ Lớn (LLMs)** ngày nay.

5.3.4 Sự trỗi dậy của Mô hình Ngôn ngữ Lớn (Large Language Models – LLMs)

1. Khái niệm và quy mô

Mô hình Ngôn ngữ Lớn là những kiến trúc **Transformer** cực sâu – thường hàng chục đến hàng trăm lớp – với **hàng chục đến hàng trăm tỷ tham số**. Tiêu biểu:

Mô hình	Số tham số ($\approx$)	Điểm nổi bật
GPT-3	175 tỷ	96 tầng Transformer, khả năng sinh văn bản mạch lạc - kể cả với yêu cầu mới gặp lần đầu
BERT-base	340 triệu	Học ngữ cảnh hai chiều, mạnh về hiểu nội dung
T5-11B	11 tỷ	Khuôn “text-to-text” hợp nhất nhiều tác vụ NLP

Nhờ **quy mô khổng lồ** và **cơ chế self-attention**, LLMs nắm bắt được những mẫu ngữ nghĩa cực kỳ tinh vi mà mô hình nhỏ hơn bỏ lỡ.

2. Nhiệm vụ huấn luyện cốt lõi: Dự đoán từ kế tiếp

LLMs trước hết được **tiền-huấn luyện** (pre-training) trên kho văn bản đa miền ở bài toán duy nhất:

$$\text{Maximize } P(w_t | w_{1:t-1})$$

tức **dự đoán token tiếp theo** trong dãy. Cách huấn luyện tự giám sát này tận dụng hàng trăm tỷ từ mà **không cần nhãn thủ công**, đồng thời tạo ra biểu diễn ngôn ngữ giàu ngữ cảnh để *chuyển sang* (fine-tune) mọi tác vụ khác.

3. Khả năng “đột phá” (Emergent Abilities)

Khi số tham số và dữ liệu vượt ngưỡng nhất định, LLMs bắt đầu thể hiện năng lực chưa từng được dạy trực tiếp:

- **Few-shot / Zero-shot learning** – GPT-3 học quy tắc mới chỉ từ vài ví dụ nhắc trong lời nhắc (*prompt*).
- **Đa nhiệm** – Cùng một mô hình trả lời câu hỏi, tóm tắt, viết mã, dịch ngôn ngữ... mà **không** cần mỗi tác vụ một mạng riêng.
- **Khái quát hóa mạnh** – Thể hiện điểm tổng quát cao trên bộ đo chuẩn đa dạng, minh chứng qua bảng so sánh hiệu năng của GPT-3, BERT, XLNet, T5...

Hiện tượng này được gọi là “**năng lực nổi**” (*emergent capability*) – đặc tính chỉ xuất hiện ở **quy mô lớn** và chưa bao giờ được ghi nhận ở mô hình nhỏ.

4. Tác động và ứng dụng rộng khắp

LLMs đã nhanh chóng len lỏi vào các lĩnh vực:

- **Y tế** – Tóm tắt y văn, hỗ trợ chẩn đoán bệnh hiếm.
- **Giáo dục** – Trợ giảng thông minh cá nhân hoá lộ trình học.
- **Kinh doanh** – Chatbot dịch vụ khách hàng, sinh nội dung marketing.

Ngoài lợi ích, LLMs đặt ra thách thức đạo đức: thiên vị dữ liệu, an toàn nội dung, chi phí năng lượng và quyền riêng tư – các vấn đề được bàn sâu ở Chương 9 (Đạo đức AI).

5. Ý nghĩa đối với nghiên cứu và công nghiệp

- **Thay đổi mô hình phát triển phần mềm AI**: từ “huấn luyện từ đầu” sang **dùng trước-huấn luyện rồi tinh chỉnh** – giảm thời gian và dữ liệu đặc thù.
- **Thúc đẩy phần cứng**: nhu cầu GPU/TPU chuyên dụng, công nghệ tách mô hình (model parallel) và nén tham số.
- **Khởi nguồn kỷ nguyên đa phương thức**: kết hợp văn bản, hình, âm thanh – ví dụ Gemini, GPT-4o.

Tóm lại, nhờ kiến trúc Transformer, huấn luyện *next-word prediction* ở quy mô **cực lớn**, LLMs như **GPT, Gemini, Llama** đã đạt tới mức năng lực tổng hợp khiến chúng trở thành “**hạ tầng trí tuệ**” cho vô số ứng dụng AI hiện đại.

5.3.5 Ứng dụng và Hướng dự án

Trong phần này, chúng ta phác hoạ ba hướng dự án tiêu biểu nhằm giúp sinh viên **trải nghiệm thực tiễn** sức mạnh của các mô hình Ngôn ngữ Lớn (LLMs). Mỗi hướng đi kèm khung quy trình chung: **(1) chọn nhiệm vụ & dữ liệu, (2) tiền-xử lý, (3) tinh chỉnh/khai thác mô hình, (4) đánh giá, (5) triển khai**.

A. Chatbot hỏi-đáp theo miền (Domain-specific QA Chatbot)

Mục tiêu Xây dựng hệ thống trò chuyện trả lời chính xác các câu hỏi về một bộ quy định nội bộ (ví dụ: quy chế đào tạo).

Cơ sở lý thuyết Chatbots hiện đại kết hợp **hiểu ý định (intent/entity extraction)**, **hội thoại đa lượt**, và **trả lời sinh ngữ cảnh**. Năng lực này được thúc đẩy mạnh bởi LLMs và khung đối thoại như *Rasa*. Từ góc nhìn kinh doanh, chatbot đem lại ưu thế tự động hóa dịch vụ và xử lý truy vấn tự nhiên nhờ các mô-đun NLP, ASR.

Quy trình đề xuất

- **Thu thập & làm sạch tri thức**: tập hợp văn bản quy định, chuẩn hóa (loại bỏ PDF header/footer), tách câu.
- **Tiền-huấn luyện tri thức (embedding store)**: dùng kỹ thuật *retrieval-augmented generation (RAG)* – mã hoá đoạn văn bằng Sentence-BERT, lưu vào FAISS.
- **Tinh chỉnh LLM**:
 - *Instruction-tuning* trên cặp (câu hỏi, câu trả lời chuẩn).
 - Sử dụng kỹ thuật **LoRA/QLoRA** để giảm bộ nhớ.
- **Đánh giá**: BLEU hoặc F1 câu trả lời; khảo sát người dùng.
- **Triển khai**: API REST + giao diện web đơn giản; theo dõi log để tiếp tục huấn luyện tăng cường (RLHF nhẹ).

B. Ứng dụng Tóm tắt Văn bản (Automatic Text Summarization)

Mục tiêu Nhận một bài báo dài, trả về bản tóm tắt ngắn gọn nhưng giữ ý chính.

Nền tảng học thuật Tóm tắt có hai dạng: **extractive** (trích câu quan trọng) và **abstractive** (sinh văn mới). Phân loại, thách thức và sơ đồ quy trình ATS được tổng hợp chi tiết trong chương “Summarization of Legal Texts by Using Deep Learning”.

Khung thực hiện

Bước	Mô tả ngắn
1. Tập dữ liệu	Bài báo tin tức hoặc corpus CNN/DailyMail.
2. Tiền-xử lý	Làm sạch, tách câu, loại stop-words; quy trình cụ thể minh họa trong Fig 46.5 .
3. Mô hình	• Extractive: TextRank • Abstractive: LLM finetune (T5-base / BART).
4. Huấn luyện / tinh chỉnh	Encoder-Decoder + Attention; early-stopping để tránh overfitting
5. Đánh giá	ROUGE-1/-2/-L – công thức và diễn giải chi tiết .

Mở rộng: thêm tùy chọn “tóm tắt theo truy vấn” (query-focused) hoặc đa ngôn ngữ.

C. Trợ lý Lập trình (Code-Generation Assistant)

Mục tiêu Khảo sát khả năng sinh mã của LLM để giải bài tập lập trình cơ bản (ví dụ: thuật toán sắp xếp, xử lý chuỗi).

Lý do chọn LLM Ở quy mô lớn, LLMs tự phát lộ năng lực “viết code” bên cạnh trả lời, tóm tắt, dịch thuật – hiện tượng *emergent abilities*.

Quy trình triển khai

- **Bộ dữ liệu:** Sử dụng tập *HumanEval* hoặc trích xuất câu hỏi–đáp án từ kho GitHub (license phù hợp).
- **Chiến lược**
 - **Prompt-engineering:** cung cấp mẫu hàm, yêu cầu, ví dụ test.
 - **Fine-tune** (tùy tài nguyên): lựa chọn *StarCoder-1b/3b* với LoRA.
- **Đánh giá**
 - Tự động: *pass@k* trên test case.
 - Thủ công: đọc mã, chấm chất lượng phong cách.
- **Giao diện:** Notebook web; người dùng nhập đề, nhận mã và kết quả chạy.

Khuyến nghị an toàn: Rà soát license mã sinh ra; bật bộ lọc ngôn ngữ để tránh sinh nội dung nguy hiểm.

Ba hướng trên khai thác **cùng một nền tảng Transformer-LLM** nhưng minh họa **ba dạng tác vụ NLP**:

- **Hội thoại theo miền** → tương tác người dùng thời gian thực.
- **Tóm tắt** → nén thông tin và hỗ trợ đọc hiểu.
- **Sinh mã** → hỗ trợ phát triển phần mềm.

Thực hành trọn vẹn vòng đời dự án – từ thu thập dữ liệu tới triển khai – giúp sinh viên nắm vững **phương pháp khoa học dữ liệu**, đồng thời rèn kỹ năng đánh giá khách quan hiệu năng và yếu tố đạo đức của mô hình.

CHƯƠNG 6: XỬ LÝ NGÔN NGỮ TỰ NHIÊN (NATURAL LANGUAGE PROCESSING – NLP)

6.0 Giới thiệu – “Nói chuyện với máy tính”

Xử lý Ngôn ngữ Tự nhiên (NLP) là một nhánh chuyên biệt của Trí tuệ nhân tạo (AI) nghiên cứu các phương pháp giúp máy tính **hiểu, diễn giải, và tạo ra ngôn ngữ của con người** dưới dạng văn bản hoặc lời nói, sao cho kết quả có ý nghĩa đối với người dùng.

6.1 Vì sao NLP là một bài toán khó?

Ngôn ngữ tự nhiên được con người sử dụng hằng ngày có đặc tính **nhập nhằng (ambiguity)** ở nhiều cấp độ. Ba dạng nhập nhằng chính sau đây khiến quá trình tự động xử lý ngôn ngữ trở nên thách thức:

Dạng nhập nhằng	Mô tả	Ví dụ minh họa
Nhập nhằng từ vựng (Lexical Ambiguity)	Một từ có thể mang nhiều nghĩa khác nhau tùy ngữ cảnh.	Từ “ <i>đường</i> ” trong tiếng Việt có thể chỉ “đường đi” hoặc “đường ăn” (sugar).
Nhập nhằng cú pháp (Syntactic Ambiguity)	Cùng một chuỗi từ có thể được phân tích thành nhiều cấu trúc ngữ pháp hợp lệ.	Câu “ <i>Cô ấy giữ cuốn sách trên bàn.</i> ” có thể hiểu là (a) cô ấy đặt cuốn sách ở vị trí <i>trên bàn</i> hoặc (b) cô ấy <i>đứng trên bàn</i> và giữ cuốn sách.
Nhập nhằng ngữ nghĩa & ngữ dụng (Semantic & Pragmatic Ambiguity)	Ý nghĩa thực sự của câu phụ thuộc mạnh vào tri thức nền và bối cảnh giao tiếp (thời gian, địa điểm, mục đích, mối quan hệ giữa người nói-nghe, v.v.).	Câu “ <i>Tôi chết mất rồi!</i> ” có thể ám chỉ “Tôi gặp rắc rối lớn” thay vì nghĩa đen “Tôi đã chết”, tùy hoàn cảnh.

Trong tài liệu kinh điển *Artificial Intelligence: A Modern Approach*, Russell & Norvig chỉ rõ rằng nhập nhằng diễn ra song song ở cả ba cấp — **lexical, syntactic và semantic** — và việc giải quyết triệt để đòi hỏi kết hợp **mô hình thế giới, mô hình tâm trí người nói, mô hình ngôn ngữ và mô hình âm thanh/chuỗi ký tự** đối với lời nói/viết tay.

Chính sự giàu ngữ nghĩa nhưng linh hoạt (và đôi khi mơ hồ) của tiếng người khiến NLP vừa hấp dẫn vừa khó khăn. Bất kỳ hệ thống NLP nào muốn “nói chuyện tự nhiên” với chúng ta đều phải xây dựng được các cơ chế **khử nhập nhằng (disambiguation)** hiệu quả trên cả ba bình diện nói trên.

6.1 Biểu diễn văn bản: “Số hóa” ngôn ngữ

Trong mọi hệ thống Xử lý Ngôn ngữ Tự nhiên (NLP), bước đầu tiên là **chuyển đổi văn bản thô thành các đại diện số** – thường là các **vector** mà mô hình học máy có thể xử lý. Việc số hóa này đặt ra nhiều yêu cầu: giữ lại càng nhiều thông tin ngữ nghĩa càng tốt, nhưng vẫn bảo đảm gọn nhẹ, dễ tính toán.

6.1.1 Phương pháp cổ điển 1: Túi từ (Bag-of-Words - BoW)

Phương pháp **Túi từ (Bag-of-Words - BoW)** là một trong những kỹ thuật biểu diễn văn bản (text representation) nền tảng và kinh điển nhất trong lĩnh vực Xử lý Ngôn ngữ Tự nhiên (NLP). Đúng như tên gọi, mô hình này xem mỗi văn bản như một “cái túi” chứa đầy các từ. Khi nhìn vào một văn bản, BoW chỉ quan tâm hai điều: **những từ nào xuất hiện** và **mỗi từ xuất hiện bao nhiêu lần**.

Mô hình này cố tình **bỏ qua hoàn toàn các thông tin phức tạp hơn** như:

- Ngữ pháp và cấu trúc câu.
- Trật tự của các từ.
- Ngữ cảnh và mối quan hệ giữa các từ.

Chẳng hạn, với BoW, hai câu “trí tuệ nhân tạo” và “nhân tạo trí tuệ” sẽ được coi là giống hệt nhau vì chúng chứa cùng một “túi” từ. Mặc dù có sự đơn giản hóa mạnh mẽ này, BoW vẫn là một mô hình nền tảng cực kỳ quan trọng, được các giáo trình kinh điển như *Artificial Intelligence: A Modern Approach* công nhận là cơ sở cho nhiều kỹ thuật thống kê ngôn ngữ trong giai đoạn đầu phát triển của NLP.

b) Quy trình Xây dựng Vector BoW

Việc chuyển đổi một tập hợp văn bản thành các vector BoW bao gồm ba bước chính:

- **Xây dựng Từ điển (Build Vocabulary):**
 - **Mục tiêu:** Xác định tất cả các từ duy nhất sẽ được sử dụng để mô tả các văn bản.
 - **Quy trình:** Thu thập toàn bộ kho văn bản (corpus) cần xử lý. Sau đó, tiến hành **tách từ (tokenization)** để chia các câu thành các từ riêng lẻ (token). Cuối cùng, tạo ra một danh sách (từ điển V) chứa tất cả các từ duy nhất xuất hiện trong toàn bộ corpus. Kích thước của từ điển này, ký hiệu là $|V|$, sẽ quyết định số chiều của các vector biểu diễn.
- **Khởi tạo Vector (Initialize Vectors):**
 - **Mục tiêu:** Tạo một “khuôn mẫu” vector cho mỗi văn bản.
 - **Quy trình:** Với mỗi văn bản trong corpus, ta khởi tạo một vector zero có độ dài bằng đúng kích thước của từ điển ($|V|$). Tại thời điểm này, tất cả các thành phần của vector đều bằng 0.
- **Mã hóa và Đếm Tần suất (Encode by Frequency Counting):**
 - **Mục tiêu:** Điền giá trị vào vector zero đã khởi tạo để nó phản ánh nội dung của văn bản.
 - **Quy trình:** Duyệt qua từng từ trong văn bản đang xét. Với mỗi từ, ta tìm vị trí (chỉ số i) tương ứng của nó trong từ điển V và tăng giá trị tại vị trí i trong vector của văn bản đó lên 1. Sau khi duyệt hết văn bản, thành phần thứ i của vector sẽ chính là số lần từ thứ i trong từ điển xuất hiện trong văn bản đó.

Ký hiệu toán học:

- Cho một tập văn bản (corpus) $D = \{d_1, d_2, \dots, d_M\}$
- Từ điển được xây dựng từ D là $V = \{v_{\{1\}}, v_{\{2\}}, \dots, v_{\{|V|\}}\}$.

Vector BoW $x^{(j)}$ của văn bản d_j được biểu diễn là:

$$x^{(j)} = (f_{1j}, f_{2j}, \dots, f_{|V|j})$$

Trong đó, f_{ij} là tần suất (frequency) của từ v_i trong văn bản d_j .

c) Ví dụ Minh họa

Hãy xem xét một corpus nhỏ gồm ba văn bản sau:

- d_1 : “trí tuệ nhân tạo”
- d_2 : “trí tuệ”
- d_3 : “nhân tạo nhân tạo”

Bước 1: Xây dựng Từ điển Các từ duy nhất xuất hiện trong toàn bộ corpus là “trí”, “tuệ”, “nhân”, “tạo”. Vậy, từ điển của chúng ta là: $V = \{\text{“trí”}, \text{“tuệ”}, \text{“nhân”}, \text{“tạo”}\}$. Kích thước từ điển $|V| = 4$.

Bước 2 & 3: Mã hóa và Đếm tần suất cho từng văn bản Mỗi văn bản sẽ được mã hóa thành một vector 4 chiều, tương ứng với thứ tự các từ trong từ điển V .

- **Đối với $d_1 = \text{“trí tuệ nhân tạo”}$:**
 - “trí”: 1 lần
 - “tuệ”: 1 lần
 - “nhân”: 1 lần
 - “tạo”: 1 lần
 - $\rightarrow$ Vector BoW: **(1, 1, 1, 1)**
- **Đối với $d_2 = \text{“trí tuệ”}$:**
 - “trí”: 1 lần
 - “tuệ”: 1 lần
 - “nhân”: 0 lần
 - “tạo”: 0 lần
 - $\rightarrow$ Vector BoW: **(1, 1, 0, 0)**
- **Đối với $d_3 = \text{“nhân tạo nhân tạo”}$:**
 - “trí”: 0 lần
 - “tuệ”: 0 lần
 - “nhân”: 2 lần
 - “tạo”: 2 lần
 - $\rightarrow$ Vector BoW: **(0, 0, 2, 2)**

Bảng tổng kết:

Văn bản	Nội dung	Vector BoW (theo V)
d_1	“trí tuệ nhân tạo”	(1, 1, 1, 1)
d_2	“trí tuệ”	(1, 1, 0, 0)
d_3	“nhân tạo nhân tạo”	(0, 0, 2, 2)

Như ví dụ trên cho thấy, các vector kết quả chỉ phản ánh tần suất từ, hoàn toàn không chứa thông tin về thứ tự hay ngữ pháp của câu gốc.

d) Đánh giá

Khía cạnh	Ưu điểm	Hạn chế
Độ phức tạp ý tưởng	Cực kỳ đơn giản – dễ cài đặt, dễ giải thích	Bỏ mất ngữ cảnh và trật tự câu, dẫn tới “túi” không phản ánh cấu trúc cú pháp
Kích thước vector	Trực tiếp tỷ lệ với kích thước từ điển – tiện lợi khi từ điển nhỏ	Vector thừa thớt, độ dài lớn ; chi phí lưu trữ & xử lý tăng theo lũy tiến của kho văn bản
Trọng số từ	Mỗi từ đóng góp như nhau → không cần ước lượng trọng số	Các từ dừng (stop-words) “và”, “là”... vô tình được xem quan trọng không kém từ khoá nội dung

e) Khi nào nên dùng BoW?

- **Bài toán cơ bản, quy mô nhỏ**, như phân loại tin nhắn rác, nơi tốc độ quan trọng hơn độ tinh vi.
- **Bước khởi đầu** để thiết lập đường ống NLP trước khi chuyển sang các biểu diễn giàu ngữ nghĩa hơn (TF-IDF, Word Embeddings...).

f) Liên hệ thực hành (Python ngắn gọn)

```
from sklearn.feature_extraction.text import CountVectorizer
corpus = ["tri tue nhan tao", "tri tue", "nhan tao nhan tao"]
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)
print(vectorizer.get_feature_names_out())
print(X.toarray())
```

Kết quả minh họa đúng bảng trên (nguồn mã: trích ví dụ tương đương trong giáo trình thực hành Python của khóa học).

BoW là **viên gạch nền** cho các phương pháp biểu diễn văn bản: tuy đơn giản và còn nhiều hạn chế, nhưng nó mở đường cho cách nhìn “văn bản như một điểm trong không gian vector”, khởi phát cho cả TF-IDF lẫn các kỹ thuật nhúng từ hiện đại sau này.

6.1.2 Phương pháp cổ điển 2: TF-IDF (Term Frequency – Inverse Document Frequency)

a) Trực giác

Sau khi “đếm từ” bằng mô hình Bag-of-Words, bước tiếp tự nhiên là **điều chỉnh tầm quan trọng của từng từ**. Ý tưởng cốt lõi của TF-IDF là:

- Một từ càng xuất hiện nhiều trong **chính văn bản đó** (TF cao) và càng **hiếm** trong các văn bản còn lại (IDF cao) thì nó càng có giá trị phân biệt đối với văn bản ấy.

b) Hai thành phần

Thành phần	Công thức trực giác	Vai trò	Trích dẫn
Term Frequency (TF)	$TF(t,d)=\frac{f_{t,d}}{\sum_{t'} f_{t',d}}$	d	$f_{t,d}$ (tần suất của từ t trong văn bản d)
Inverse Document Frequency (IDF)	$IDF(t,D)=\log\frac{D}{df_t}$	D	df_t (số văn bản chứa t)

c) Công thức tổng hợp

$$TF - IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

Công thức trên được trình bày trực tiếp trong tài liệu **Algorithms in Advanced Artificial Intelligence**.

d) Thuật toán xây dựng ma trận TF-IDF

- **Tiền xử lý & Tách từ** (tokenization, loại bỏ stop-words).
- **Tính TF** cho từng cặp (từ, văn bản).
- **Tính IDF** trên toàn bộ tập; áp dụng log để làm trơn.
- **Nhân $TF \times IDF \Rightarrow$ thu được ma trận đặc trưng thưa** (sparse matrix).

Thư viện *scikit-learn* hiện cung cấp *TfidfVectorizer*, được khuyến nghị trong thực hành Python cho tuyến xử lý văn bản.

e) Ví dụ minh họa nhỏ

Văn bản	Nội dung	Vector TF-IDF (từ điển {"ai", "thông minh"})*
d_1	“ai thông minh”	(0.693, 0.693)
d_2	“ai”	(0.693, 0)
d_3	“ai ai”	(0.000, 0)

* Giả sử $IDF_{\text{ai}} = \log \frac{3}{3} = 0$ và $IDF_{\text{thông minh}} = \log \frac{3}{1} = 1.099$.

f) Đánh giá phương pháp

Khía cạnh	Ưu điểm	Hạn chế
Hiệu quả	Vấn đơn giản, đặc biệt phù hợp tìm kiếm thông tin & phân loại văn bản	Ma trận thưa thớt, chiều cao ; không nắm bắt quan hệ ngữ nghĩa sâu (vector “vua” $\neq$ “nữ hoàng”)
Khả năng giải thích	Trọng số rõ ràng (tần suất $\times$ độ hiếm) $\rightarrow$ dễ lý giải	Không tận dụng thứ tự từ, ngữ cảnh câu

g) Ứng dụng tiêu biểu

- **Máy tìm kiếm web** (xếp hạng kết quả).
- **Phân loại email rác/không rác.**
- **Truy hồi tài liệu pháp lý, y khoa**, nơi tính minh bạch của trọng số là bắt buộc. Các nghiên cứu thực nghiệm cho thấy TF-IDF thường **cải thiện độ chính xác dự đoán** so với BoW thuần túy trong nhiệm vụ phân loại tin tức.

h) Mã Python mẫu

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = ["ai thông minh", "ai", "ai ai"]
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(corpus)

print(vectorizer.get_feature_names_out())
print(X.toarray())
```

Khi chạy, bạn sẽ nhận được ma trận TF-IDF tương tự bảng ví dụ trên. (Nguồn minh họa: triển khai tương đương trong bài thực hành Python của giáo trình)

TF-IDF là “bước nhảy” quan trọng từ đếm tần suất thuần túy sang cách **đo lường tầm quan trọng thông tin** của từ. Dù chưa giải quyết được tính ngữ nghĩa sâu, TF-IDF vẫn là **chuẩn vàng gọn nhẹ** trước khi chuyển tiếp sang các biểu diễn phân bố dày đặc như Word2Vec hay BERT.

6.1.3 Phương pháp hiện đại: Nhúng từ (Word Embeddings)

a) Khái niệm và đột phá

Từ thập niên 2010, NLP chuyển sang **đại diện phân bố dày đặc**: mỗi từ được ánh xạ thành một **vector thực có kích thước thấp** ($\approx 200 - 300$) trong một không gian liên tục. Ý tưởng cốt lõi là thay vì coi từ như “chỉ số một-hot” khổng lồ, ta xem nó như **một điểm trong không gian**; các điểm gần nhau thường có ý nghĩa hoặc chức năng tương tự.

b) Nguyên lý phân bố (Distributional Hypothesis)

Nhúng từ kế thừa tuyên ngôn của ngữ học phân bố: **“You shall know a word by the company it keeps.”** (J. R. Firth, 1957). Nói cách khác, **các từ xuất hiện trong ngữ cảnh giống nhau sẽ có vector gần nhau**.

c) Thuật toán Word2Vec – đại diện tiêu biểu

Word2Vec dùng **mạng nơ-ron nông** với hai biến thể:

Biến thể	Nhiệm vụ học	Trực giác
CBOW	Dự đoán từ trung tâm từ ngữ cảnh	“Từ nào phù hợp giữa những từ lân cận?”
Skip-gram	Dự đoán ngữ cảnh từ từ trung tâm	“Từ này thường đứng cạnh những từ nào?”

Thuật toán học các vector sao cho hai mục tiêu trên đạt xác suất cao; vì thế **tính chất ngữ nghĩa** được “ép” vào hình học của không gian vector.

d) Thuộc tính “kỳ diệu” – đại số vector ngữ nghĩa

Các vector học được không chỉ phản ánh mức độ giống nhau mà còn **mã hoá quan hệ**. Ví dụ nổi tiếng:

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

Hình 25.2 (Russell & Norvig, 2021) minh hoạ khả năng giải **phép loại suy** bằng phép cộng-trừ vector đơn giản.

e) Tính chất & ưu điểm

Khía cạnh	Nhúng từ (dense)	BoW / TF-IDF (sparse)
Ngữ nghĩa	Bắt được từ loại, chủ đề, cảm xúc	Không thể hiện nghĩa
Kích thước	Vài trăm chiều, dày đặc	$ V $ chiều, thưa thớt
Khả năng suy luận	Đại số vector (loại suy, tương đồng)	Không khả thi
Vai trò hiện đại	Nền tảng cho RNN, CNN, Transformer, BERT	Chủ yếu ở truy hồi thông tin cổ điển

Ngoài Word2Vec còn có **GloVe**, **FastText** – tiếp tục cải thiện bằng thống kê toàn cục hoặc ký tự con, và **BERT** đưa ý tưởng lên mức **ngữ cảnh hóa** (contextual embeddings).

f) Ví dụ thực hành (Python – gensim)

```
from gensim.models import Word2Vec

sentences = [
    ["king", "queen", "man", "woman"],
    ["paris", "france", "berlin", "germany"],
    ["ai", "machine", "learning", "intelligence"]
]

model = Word2Vec(sentences, vector_size=200, window=5, min_count=1, sg=1)
```



```
# Phép loại suy
result = model.wv.most_similar(positive=["king","woman"], negative=["man"], topn=1)
print(result) # [('queen', ~0.86)]
```

Chạy đoạn mã sẽ xuất ra **“queen”** – tái hiện tính chất đại số nêu trên. (Mẫu lệnh tương tự ví dụ trong chương Thực hành Python của giáo trình)

g) Tầm quan trọng

Nhúng từ đánh dấu bước ngoặt: **chuyển NLP từ đặc trưng thủ công sang học biểu diễn tự động**. Ngày nay, mọi mô hình ngôn ngữ hiện đại – từ **dịch máy, tóm tắt, chatbot đến tạo văn bản đa phương thức** – đều xây dựng trên lớp “nền bê-tông” này của không gian vector dày đặc.

6.2 Các bài toán NLP cơ bản và Hướng dự án

6.2.1 Phân loại văn bản (Text Classification)

1. Khái niệm và đặt bài toán

Phân loại văn bản (còn gọi là *text categorisation* hoặc *tagging*) là nhiệm vụ tự động gán mỗi văn bản vào một hoặc nhiều nhãn đã biết trước, dựa trên nội dung của văn bản ấy. Xét một tài liệu $d \in \mathcal{D}$ và một tập nhãn rời rạc $\mathcal{Y} = \{y_1, \dots, y_K\}$, mục tiêu của bộ phân loại là tìm hàm $f: \mathcal{D} \rightarrow \mathcal{Y}$ (hoặc $f: \mathcal{D} \rightarrow 2^{\mathcal{Y}}$ trong thiết lập đa nhãn) sao cho $f(d)$ xấp xỉ nhất nhãn thật y^* .

2. Quy trình tổng quát

Giai đoạn	Nội dung học thuật
Tiền xử lý	Tách từ (<i>tokenisation</i>) – bước chuyển văn bản thô thành chuỗi từ/biểu tượng rời rạc; chuẩn hoá (viết thường, xoá dấu câu); loại dừng (<i>stop-word removal</i>); rút gọn gốc (<i>stemming</i>), v.v.
Biểu diễn	Sử dụng TF-IDF hoặc các véc-tơ nhúng (Word2Vec, GloVe, BERT) – hai kỹ thuật đã trình bày ở Mục 6.1.
Huấn luyện mô hình	Chọn một thuật toán học máy và ước lượng tham số trên tập huấn luyện.
Đánh giá	Các độ đo thường dùng: accuracy, precision/recall, F-measure, ROC-AUC.

3. Thuật toán kinh điển

Họ thuật toán	Đặc trưng lý thuyết	Ưu/nhược điểm chính
Naive Bayes (mô hình sinh)	<i>generative</i>)	Giả định độc lập có điều kiện giữa các từ và tính posterior theo công thức Bayes: $P(\text{Class} \mathbf{w}) = \alpha P(\text{Class}) \prod_j P(w_j \text{Class})$. Công thức

Họ thuật toán	Đặc trưng lý thuyết	Ưu/nhược điểm chính
		đầy đủ dùng khi suy diễn là 【dưới đây trích nguyên mẫu trong sách】
	• Cực nhanh, đòi hỏi ít dữ liệu • Hiệu quả bất ngờ dù giả định “ngây thơ” • Không mô tả quan hệ tương quan giữa từ	
Logistic Regression (hồi quy lô-gic) – mô hình <i>discriminative</i>	Học trực tiếp $P(y \mathbf{x})$ qua hàm: $h_w(\mathbf{x}) = \frac{1}{1+e^{-w \cdot \mathbf{x}}}$. Trọng số w_i phản ánh mức độ dự báo của mỗi đặc trưng	• Diễn giải rõ ràng, cập nhật bằng gradient descent • Phù hợp dữ liệu thưa TF-IDF • Dễ quá khớp khi dữ liệu nhỏ
Support Vector Machines (SVM)	Tối ưu biên phân tách lớn nhất; có thể mở rộng phi tuyến qua <i>kernel trick</i>	• Khả năng khái quát cao, đặc biệt với dữ liệu chiều cao • Tính toán nặng với tập lớn

Nghiên cứu thực nghiệm cho thấy những mô hình phân biệt (logistic regression, SVM, mạng nơ-ron) thường đạt lỗi thấp hơn Naive Bayes khi dữ liệu đủ lớn. Tuy nhiên, Naive Bayes hội tụ nhanh và hữu ích khi tài nguyên huấn luyện hạn chế.

4. Ứng dụng tiêu biểu

- **Phân tích cảm xúc:** xác định mức độ *tích cực/tiêu cực* của đánh giá sản phẩm hoặc mạng xã hội.
- **Lọc thư rác:** tách email “spam” khỏi “ham”.
- **Phân loại chủ đề báo chí:** Reuters-21578 đạt 96 % chính xác với SVM.

Các ứng dụng mở rộng hơn gồm phát hiện tin giả, định tuyến yêu cầu khách hàng, hoặc trợ lý giọng nói.

5. Hướng dẫn triển khai dự án thực hành

- **Chọn tập dữ liệu gán nhãn**
 - Ví dụ: tập tin đánh giá sản phẩm thương mại điện tử (bình luận Shopee/Tiki), bộ Reuters-21578, hoặc tin tức tiếng Việt đã phân loại.
- **Tiền xử lý & Biểu diễn**
 - Áp dụng *tokenisation*, loại stop-word, sau đó xây dựng véc-tơ TF-IDF hoặc sử dụng véc-tơ nhúng sẵn (fastText tiếng Việt).
- **Huấn luyện mô hình**
 - **Naive Bayes** để có đường base-line nhanh.

- **Logistic Regression** hoặc **SVM** để cải thiện độ chính xác; cân nhắc điều chỉnh siêu tham số C (regularisation) với SVM.
- **Đánh giá & phân tích lỗi**
 - Tính Accuracy, F1-score; vẽ đường ROC nếu là bài toán nhị phân. Bảng so sánh ROC giữa các mô hình có thể tham khảo cấu trúc ví dụ trong .
- **Báo cáo**
 - Trình bày quy trình, kết quả, bàn luận về ưu/nhược điểm của từng mô hình, đề xuất cải tiến (ví dụ thử Transformer tiền huấn luyện khi có GPU).

6. Ghi chú học thuật

- **Mô hình sinh (*generative*) versus phân biệt (*discriminative*):** Naive Bayes học $P(x, y)$ toàn phần, còn logistic regression/SVM chỉ học biên quyết định $P(y | x)$.
- **Đặc trưng & lựa chọn đặc trưng:** nên loại bỏ từ quá hiếm hoặc quá phổ biến để giảm chiều và chống nhiễu .
- **Xu thế hiện đại:** Transformer tiền huấn luyện (BERT, GPT) cho phép tinh chỉnh (*fine-tune*) đạt độ chính xác cao trên hầu hết tác vụ phân loại .

Phân loại văn bản là bước khởi đầu quan trọng trong NLP, gắn với nhiều ứng dụng thực tiễn. Nhận diện chuẩn bài toán, lựa chọn đúng biểu diễn và mô hình (từ Naive Bayes giản dị đến các phương pháp hiện đại) sẽ giúp sinh viên nhanh chóng triển khai một hệ thống phân loại hoàn chỉnh, đồng thời có nền tảng vững chắc để khám phá các hướng nghiên cứu nâng cao.

6.2.2 Nhận dạng Thực thể Có tên (*Named Entity Recognition – NER*)

1. Khái niệm và vai trò

Named Entity Recognition là nhiệm vụ **tìm và gán nhãn các thực thể được đặt tên** – chẳng hạn tên người, địa danh, tổ chức, ngày tháng – xuất hiện trong văn bản. Đây là bước trung tâm của *information extraction*; các mô hình tìm kiếm, hỏi–đáp, chatbot và hệ tri thức đều dựa trên tập thực thể đã được rút trích chính xác.

2. Hệ thống phân loại thực thể phổ biến

Mã nhãn	Lớp thực thể	Ví dụ
PER	Người	Bill Gates
ORG	Tổ chức	Microsoft
LOC	Địa điểm	Albuquerque
MISC	Ngày tháng, tiền tệ, phần trăm, v.v.	“2025”, “\$100”

Các loại chính (PER, ORG, LOC) thường được mở rộng thêm “TIME/DATE”, “QUANTITY”, “MONEY”... tùy yêu cầu bài toán.

3. Ví dụ minh họa

Trong câu “**Bill Gates** đã sáng lập **Microsoft** tại **Albuquerque**.” mô hình NER chuẩn phải gán:

Bill Gates	→ PER
Microsoft	→ ORG
Albuquerque	→ LOC

4. Phương pháp tiếp cận

Nhánh kỹ thuật	Mô tả ngắn	Điểm mạnh / hạn chế
Quy tắc & Regex	Dựa trên mẫu ký tự, danh sách tên (gazetteer).	Dễ triển khai; kém tổng quát.
Thông kê cổ điển	HMM, CRF... học chuỗi nhãn dựa trên đặc trưng từ/cú pháp.	Hiệu quả khi dữ liệu hạn chế; cần đặc trưng thủ công.
Mạng nơ-ron tuần tự	Bi-LSTM/GRU kết hợp <i>word embeddings</i> – “các từ giống nghĩa nằm gần nhau trong không gian vector” – giúp mô hình hóa ngữ cảnh dài.	Tự học đặc trưng; huấn luyện tốn tài nguyên.
Transformer tiền huấn luyện (BERT, RoBERTa)	Tận dụng cơ chế tự-chú ý; chỉ cần <i>fine-tune</i> vài epoch đã đạt SOTA cho NER.	Chính xác cao, hỗ trợ đa ngôn ngữ; mô hình lớn.

Các nghiên cứu tổng kết cho thấy *RNN/Transformer* nắm bắt phụ thuộc ngữ cảnh tốt hơn, từ đó nâng điểm **F1** trên tập kiểm thử lên tới > 90 %.

5. Quy trình xây dựng hệ thống NER

- **Thu thập & gán nhãn dữ liệu** theo chuẩn **BIO/BILOU**.
- **Tiền xử lý**: tách câu, *tokenisation*, chuẩn hóa chữ, loại ký tự đặc biệt.
- **Biểu diễn từ**:
 - TF-IDF + CRF (cơ bản)
 - *Word2Vec* / *fastText* hoặc *BERT* (hiện đại) – những biểu diễn phân bố dày đặc đã chứng minh hữu ích cho NER.
- **Huấn luyện mô hình** (CRF, BiLSTM-CRF, BERT fine-tune).
- **Đánh giá bằng Precision, Recall, F1-score** (chuẩn cộng đồng).
- **Triển khai & suy diễn**: xuất kết quả dưới dạng JSON/CoNLL để tích hợp vào pipeline tìm kiếm, QA, v.v.

6. Ứng dụng thực tiễn

- **Truy xuất thông tin** trong tài liệu pháp lý, y khoa, tài chính.
- **Xây dựng tri thức** cho máy tìm kiếm và trợ lý ảo (Google Knowledge Graph, Wikipedia Wikidata).
- **Hệ thống cảnh báo**: phát hiện thực thể nhạy cảm trong luồng tin thời gian thực (tên thuốc, mã lỗi, v.v.).

7. Hướng dẫn dự án mini

- **Dữ liệu**:
 - Bộ **VLSP NER 2016–2019** (tiếng Việt) hoặc tự thu thập truyện ngắn, tin tức.
- **Thư viện**:
 - **spaCy**: đã hỗ trợ pipeline NER, dễ tùy biến; bản *spacy-vi* có sẵn mô hình tiếng Việt.
 - **NLTK**: thích hợp cho thử nghiệm CRF/HMM truyền thống.
- **Các bước thực hiện**:
 - a. Cài đặt *spacy* → nạp mô hình "*vi_core_news_lg*" (hoặc mô hình tiếng Anh để so sánh).
 - b. Viết hàm tiền xử lý & suy diễn → xuất danh sách thực thể (PER, ORG, LOC...).
 - c. Đánh giá nhanh bằng tập dev (tự gán nhãn) → so **F1** giữa mô hình gốc và mô hình tự fine-tune.
- **Mở rộng**:
 - Thêm nhãn tùy chỉnh (như *Sản phẩm*, *Sự kiện*).
 - Chuyển sang **BERT multilingual** để cải thiện độ chính xác, đặc biệt với tiếng Việt có dấu.

NER là “cửa ngõ” giúp máy tính nhận biết **ai, ở đâu, khi nào** trong văn bản – nền móng không thể thiếu cho mọi hệ thống hiểu ngôn ngữ ở cấp thực thể. Với thư viện mã nguồn mở ngày càng mạnh và các mô hình tiền huấn luyện đa ngôn ngữ, sinh viên có thể nhanh chóng dựng nên ứng dụng trích xuất thực thể, đồng thời khám phá chiều sâu của học máy hiện đại trong NLP.

CHƯƠNG 7 HỌC TĂNG CƯỜNG (REINFORCEMENT LEARNING)

Trong học tăng cường (RL), tác tử **không còn “thầy”** cung cấp nhãn đúng như trong học có giám sát, cũng **không chỉ truy tìm cấu trúc ẩn** thuần túy như học không giám sát. Thay vào đó, tác tử chủ động **tương tác với môi trường**, nhận về **phần thưởng (reward)** và tự điều chỉnh hành vi để tối đa hoá thành công lâu dài.

Quy trình tương tác lặp diễn ra liên tiếp:

- Tác tử quan sát **trạng thái** hiện tại s .
- Tác tử chọn **hành động** a .
- Môi trường chuyển sang **trạng thái mới** s' và cấp **phần thưởng** r .

Mục tiêu của tác tử là **tối đa hoá tổng phần thưởng chiết khấu kỳ vọng** theo thời gian, từ đó nảy sinh **sự đánh đổi kinh điển Exploration vs Exploitation**: nên khám phá hành động mới hay khai thác hành động đã biết tốt?

7.1 Quá trình quyết định Markov (Markov Decision Processes – MDP)

7.1.1 Ngôn ngữ toán học cho RL

MDP cung cấp khung chính thức để mô hình hoá mọi bài toán RL. Một MDP gồm bộ $\langle S, A, P, R, \gamma \rangle$:

Ký hiệu	Ý nghĩa
S	Tập hữu hạn trạng thái
A	Tập hữu hạn hành động
$(P(s'$ $R(s, a, s')$	$s, a))$ Phần thưởng tức thì nhận được
$\gamma \in [0, 1)$	Hệ số chiết khấu – đo tầm quan trọng của phần thưởng tương lai

7.1.2 Thuộc tính Markov

MDP giả định “**tương lai độc lập với quá khứ khi đã biết hiện tại**”: xác suất $P(S_{t+1} | S_t, A_t)$ không phụ thuộc toàn bộ lịch sử trước đó.

7.1.3 Chính sách và giá trị

- **Chính sách** $\pi: S \rightarrow A$ hướng dẫn tác tử chọn hành động.
- **Chính sách tối ưu** π^* tối đa hoá kỳ vọng tổng phần thưởng chiết khấu.
- Hàm **giá trị-trạng thái** $V^\pi(s)$ và **giá trị-hành động** $Q^\pi(s, a)$ đo lường chất lượng theo π .

7.2 Thuật toán Q-Learning (Model-Free)

Khi **không biết** P và R , tác tử vẫn có thể học trực tiếp giá trị $Q(s, a)$ – **tổng phần thưởng tương lai** nếu chọn a ở s , rồi theo **hành động có Q cao nhất**. Thuật toán cập nhật cốt lõi (dạng tabular)

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right],$$

trong đó α là tốc độ học. Bảng tần suất $N(s, a)$ thường được kết hợp hàm **exploration function** $f(Q, N)$ để duy trì **GLIE** (Greedy in the Limit of Infinite Exploration), bảo đảm mọi cặp (s, a) được thử vô hạn lần.

7.3 Chiến lược khám phá – khai thác

Một chiến lược phổ biến là ε -greedy:

- Với xác suất ε (thường giảm dần), chọn **hành động ngẫu nhiên để khám phá**.
- Ngược lại, **khai thác** hành động có Q cao nhất.

Điều chỉnh ε cho phép tác tử chuyển dần từ **khám phá** sang **khai thác** khi kiến thức đã đủ tin cậy.

7.4 Mở rộng & Ứng dụng

- **Học bị động vs chủ động**: Từ việc chỉ **ước lượng giá trị** dưới một chính sách cố định (passive RL) tới **tự chọn hành động** để tối ưu hoá (active RL).
- **Khái quát hoá**: Khi không gian trạng thái lớn, Q được **xấp xỉ hàm** (linear, NN, CNN, ...) để học hiệu quả hơn.
- **Hệ thống phân cấp** (Hierarchical RL) giúp chia nhiệm vụ phức tạp thành các kỹ năng nhỏ, tăng tốc học và tái sử dụng kỹ năng.
- **Apprenticeship / Inverse RL**: Suy diễn hàm thưởng từ **hành vi chuyên gia**, rồi tối ưu hoá để có thể **vượt chuyên gia**.

7.5 Gợi ý thực hành

- Thử lập trình **Q-Learning agent** trên môi trường *Grid-World* 4×3 (Minh hoạ hình 23.1) để quan sát đường hội tụ chính sách tối ưu.
- Thay đổi γ để thấy ảnh hưởng **chiết khấu** tới độ “thiên cận” của tác tử.
- Triển khai ε -decay (ví dụ: $\varepsilon_t = \frac{1}{t}$) và so sánh tốc độ khám phá với chiến lược cố định.

Nguồn trích dẫn

- Stuart J. Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach*, 4e, Pearson (2021).
- David L. Poole & Alan K. Mackworth, *Artificial Intelligence – Foundations of Computational Agents*, 3e, Cambridge UP (2024).

7.1.2 Thuộc tính Markov (The Markov Property)

Định nghĩa Trong khuôn khổ MDP, giả định Markov khẳng định rằng **xác suất xảy ra trạng thái kế tiếp chỉ phụ thuộc vào trạng thái hiện tại và hành động hiện tại**:

$$P(S_{t+1} = s' \mid S_{0:t}, A_{0:t}) = P(S_{t+1} = s' \mid S_t = s, A_t = a).$$

Nói cách khác, “*tương lai độc lập với quá khứ khi đã biết hiện tại*”.

Trong thực hành, công thức trên thường được viết gọn dưới dạng hàm chuyển tiếp $P(s'|s, a)$.

1. Ý nghĩa của thuộc tính Markov

- **Tính “không nhớ” (*memory-less*)**: Tác tử không cần lưu toàn bộ lịch sử; **toàn bộ thông tin liên quan tới dự báo tương lai đã được “nén” trong trạng thái hiện tại**.
- **Giảm lược độ phức tạp**: Nhờ chỉ xét s thay vì chuỗi $(s_0, \dots, s_t)$, việc lập mô hình và suy luận trở nên khả thi thậm chí với không gian thời gian vô hạn.
- **Cơ sở cho thuật toán động học**: Thuộc tính Markov cho phép thiết lập các phương trình đệ quy (Bellman, value iteration, v.v.) vốn là nền tảng của học tăng cường.

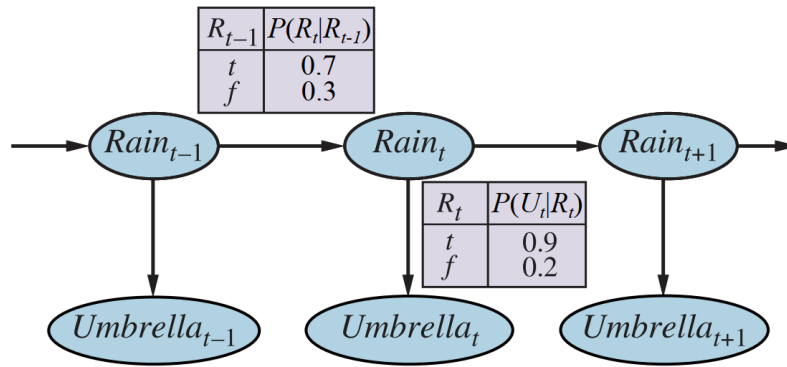
2. Thảo luận về “bậc” Markov

Thuộc tính trên thường ám chỉ **Markov bậc nhất**. Nếu thực tế cần nhiều quá khứ hơn (ví dụ xác suất mưa phụ thuộc hai ngày trước), ta có hai cách cải thiện:

- **Tăng bậc** (xem $P(S_{t+1}|S_t, S_{t-1})$).
- **Mở rộng vector trạng thái** để đưa thêm biến mang ký ức vào S_t . Cả hai đều giúp khôi phục thuộc tính Markov trong mô hình mở rộng.

3. Ví dụ minh họa

Tình huống	Diễn giải thuộc tính Markov
Bài toán “Umbrella World” (Hình 14.2) Có mưa hôm nay?	Xác suất <i>ngày mai mưa</i> chỉ phụ thuộc vào <i>hôm nay mưa</i> chứ không cần chuỗi quan sát ô trước đó; cảm biến “xuất hiện ô” cũng chỉ phụ thuộc trạng thái mưa hiện tại (sensor Markov).
X - Random Walk	Hạt di chuyển ± 1 trên trục x mỗi bước; vị trí x_t chứa đủ thông tin dự báo x_{t+1} (Markov đúng tuyệt đối).
Robot tự hành có pin	Nếu trạng thái chỉ gồm vị trí-vận tốc, dự báo vị trí vi phạm Markov khi pin cạn; bổ sung biến <i>Battery</i> vào S_t phục hồi thuộc tính Markov.



Hình 104. Mạng Bayes mô tả thế giới chiếc ô, gồm mô hình chuyển tiếp $P(Rain_t | Rain_{t-1})$ và mô hình cảm biến $P(Umbrella_t | Rain_t)$.

4. Hệ quả trong thiết kế tác tử RL

- **Mô phỏng môi trường:** Khi viết mô hình, phải xác định tập biến trạng thái đủ để đảm bảo Markov; thiếu biến sẽ dẫn tới “rò rỉ ký ức”, làm hỏng chính sách tối ưu.
- **Thuật toán học:** Các thuật toán như Q-Learning, SARSA, hay DQN giả định Markov để bảo đảm hội tụ; nếu môi trường vi phạm, cần chuyển sang mô hình bán-Markov (Semi-MDP) hoặc POMDP.
- **Kỹ thuật kiểm định:** Kiểm tra thực nghiệm xem phân dư dự báo $(r + \gamma V(s') - V(s))$ có còn phụ thuộc xa hơn s hay không; nếu có, cần bổ sung biến vào trạng thái.

Thuộc tính Markov là **nền tảng lý thuyết** của học tăng cường: nó biến vấn đề ra quyết định liên tiếp thành một **chuỗi quyết định “không nhớ”**, tạo điều kiện cho cả phân tích lẫn tính toán. Đảm bảo trạng thái thỏa mãn Markov là **bước đầu tiên và quan trọng nhất** khi xây dựng bất kỳ hệ thống RL nào.

7.1.3 Các thành phần cấu thành một MDP

Một **quá trình quyết định Markov** (MDP) được diễn tả bằng bộ ngũ $\langle S, A, P, R, \gamma \rangle$ như sau:

Ký hiệu	Thành phần	Giải thích chi tiết
S	Tập trạng thái	Toàn bộ các cấu hình mà môi trường có thể rơi vào. Đối với MDP hữu hạn, S là một tập hữu hạn (
A	Tập hành động khả dụng	Các thao tác mà tác tử có thể lựa chọn tại mỗi bước thời gian; cũng được giả định hữu hạn.
$** (P(s'$	$s, a)) **$	Hàm chuyển tiếp
$R(s, a, s')$	Hàm thưởng tức thì	Giá trị phần thưởng kỳ vọng mà tác tử nhận được khi rời s , thực hiện a và tới s' . Có thể quy đổi về dạng mong đợi theo trạng thái hiện tại $(R(s, a) = \sum_{s'} P(s'$
$\gamma \in [0, 1)$	Hệ số chiết khấu	Thể hiện mức độ tác tử coi trọng phần thưởng trong tương lai. Khi γ gần 0, tác tử “thiên cận”, chú trọng lãi tức thời; khi γ tiến sát 1,

Ký hiệu	Thành phần	Giải thích chi tiết
		tác tử sẵn sàng chấp nhận chờ đợi để gặt hái lợi ích dài hạn. Giá trị $\gamma < 1$ đảm bảo tổng phần thưởng chiết khấu luôn hữu hạn.

Ý nghĩa liên kết giữa các thành phần

- S & A định nghĩa **không gian quyết định**; mọi chiến lược (chính sách) đều là hàm ánh xạ từ S sang A .
- P mô tả **động lực ngẫu nhiên** của môi trường, là “luật vật lý” giúp ta dự báo hệ quả của hành động.
- R cung cấp **tín hiệu mục tiêu** cho học tăng cường: tác tử học chính sách tối ưu bằng cách tối đa hóa **tổng phần thưởng chiết khấu kỳ vọng**

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

- γ vừa duy trì tính hội tụ của hàm giá trị, vừa phản ánh thực tế tâm lý–kinh tế: phần thưởng nhận sớm đáng giá hơn nhận muộn.

Ví dụ minh họa nhanh

Trong bài toán “**Healthy–Sick**” (hai trạng thái *healthy*, *sick*; hai hành động *relax*, *party*) ta có

- $S = \{\text{healthy}, \text{sick}\}$, $A = \{\text{relax}, \text{party}\}$.
- Bảng xác suất chuyển tiếp $P(s'|s, a)$ và hàm thưởng $R(s, a)$ được rút ra từ kinh nghiệm của Sam (Bảng 12-1).
- Lựa chọn $\gamma = 0.8$ khiến Sam cân nhắc lợi ích dài hạn hơn là “vui trước, bệnh sau” (xem phân tích giá trị ở ví dụ 12.33).

Qua ví dụ, bạn đọc thấy rõ **mỗi thành phần trong bộ** $\langle S, A, P, R, \gamma \rangle$ là không thể thiếu khi mô hình hóa một nhiệm vụ quyết định tuần tự dưới bất định. Với nền tảng chính xác này, các thuật toán động (value iteration, policy iteration) hay học tăng cường (Q-Learning, SARSA, DQN, ...) mới có thể vận hành và hội tụ.

7.1.4 Mục tiêu trong MDP – Tìm chính sách tối ưu π^*

1. Chính sách (Policy) là gì?

Một **chính sách** π là **quy tắc ra quyết định** ánh xạ mỗi trạng thái $s \in S$ sang một hành động $a \in A$ mà tác tử sẽ thực hiện, ký hiệu $\pi(s) = a$. Chính sách có thể

- **xác định (deterministic)** – luôn chọn một hành động;
- **ngẫu nhiên (stochastic)** – chọn mỗi hành động với một xác suất nhất định.

2. Hàm giá trị dưới một chính sách

Để đánh giá chất lượng của một chính sách, ta xét **tổng phần thưởng chiết khấu kỳ vọng** khởi đầu tại trạng thái s :

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \quad 0 \leq \gamma < 1.$$

Từ đó định nghĩa

- **Giá trị-trạng thái**

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s],$$

- **Giá trị-hành động**

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a].$$

Các đại lượng này tuân theo **phương trình Bellman cho chính sách cố định** (xem mục 7.2).

3. Chính sách tối ưu π^*

- **Mục tiêu** của MDP là **tìm một chính sách π^*** sao cho

$$V_{\pi^*}(s) \geq V_{\pi}(s), \quad \forall s \in S, \forall \pi.$$

- Tương đương, hàm giá trị tối ưu thỏa

$$V^*(s) = \max_a Q^*(s, a), \quad \text{với } Q^*(s, a) = \sum_{s'} P(s'|s, a)[R(s, a, s') + \gamma V^*(s')].$$

- **Đặc trưng chính sách tối ưu**

$$\pi^*(s) = \arg\max_a Q^*(s, a)$$

4. Phương trình Bellman tối ưu

Một cách tương đương để nêu mục tiêu là giải **hệ phương trình Bellman tối ưu** cho hàm tiện ích $U(s) = V^*(s)$:

$$U(s) = \max_a \sum_{s'} P(s'|s, a)[R(s, a, s') + \gamma U(s')]$$

Hệ phương trình này có nghiệm duy nhất; khi đã tìm được U , ta rút ra π^* bằng công thức ở trên.

5. Ý nghĩa thực tiễn

- **Tối đa hoá lợi ích lâu dài:** Hệ số chiết khấu γ làm giảm giá trị phần thưởng xa tương lai, giúp chuỗi G_t hội tụ và phản ánh thực tế “phần thưởng sớm quý hơn muộn”.

- **Đảm bảo tính tối ưu toàn cục:** Chính sách tối ưu được xác định tại *mọi* trạng thái, không chỉ những trạng thái có khả năng gặp nhiều.
- **Cơ sở cho thuật toán:** Các thuật toán như **Value Iteration**, **Policy Iteration** hay **Linear Programming** đều nhằm giải hệ Bellman hoặc tìm hàm Q^* , từ đó suy ra π^* .

Mục tiêu căn bản của tác tử trong mô hình MDP là tìm ra chính sách π^* – quy tắc hành động tối ưu hoá kỳ vọng tổng phần thưởng chiết khấu. Việc này tương đương với việc giải phương trình Bellman tối ưu; chính sách tối ưu sau đó được trích xuất trực tiếp bằng nguyên tắc “chọn hành động đạt giá trị-hành động tối đa ở mỗi trạng thái”.

7.2 Thuật toán Q-Learning — Học mà không cần mô hình

7.2.1 Vấn đề đặt ra

Trong nhiều tình huống thực tế, **tác tử không được trao sẵn mô hình môi trường:**

- **Không biết hàm chuyển tiếp** $P(s'|s, a)$ xác định mỗi hành động sẽ đưa hệ thống sang trạng thái nào.
- **Không biết hàm thưởng** $R(s, a, s')$ báo hiệu lợi ích nhận được sau mỗi chuyển tiếp.

Diễn hình như trong ví dụ 4×3 *Grid-World*, tác tử chỉ “nhìn” thấy trạng thái hiện tại và phần thưởng vừa nhận; mọi thứ khác đều mù mờ.

Nếu khăng khăng đòi một mô hình, ta phải **học hoặc suy đoán** P và R rồi mới áp dụng quy hoạch động (value iteration, policy iteration). Nhưng việc **ước lượng đầy đủ mô hình** tốn dữ liệu, tính toán và có thể bất khả thi trong môi trường phức tạp. Điều này dẫn tới **bài toán Học không mô hình (Model-Free Learning): Làm sao tìm được chính sách tối ưu khi mô hình ẩn?**

1. Khung so sánh: Model-based vs Model-free

Đặc trưng	RL có mô hình	RL không mô hình
Thông tin yêu cầu	Cần ước lượng/biết trước P, R	Không cần P, R
Ra quyết định	Tính look-ahead qua mô hình	Học trực tiếp cách hành động
Ví dụ thuật toán	ADP, Dyna-Q	Q-Learning, SARSA

Trong **model-free RL**, tác tử **không xây dựng** và **không cập nhật** mô hình môi trường; thay vào đó, “công thức hành vi” được học trực tiếp dưới dạng:

- **Hàm tiện ích hành động** $Q(s, a)$ – cách tiếp cận phổ biến nhất, chính là nền tảng của Q-Learning.
- **Tìm kiếm chính sách trực tiếp (policy search)** – học $\pi(s)$ mà không cần qua bước giá trị.

2. Tại sao Q-Learning giải quyết được?

Q-Learning học **hàm chất lượng tối ưu** $Q^*(s, a)$ thông qua cập nhật sai số chênh lệch thời gian (TD-error). Khi hàm này hội tụ, **chính sách tối ưu** đơn giản là chọn hành động có Q^* cao nhất tại mỗi trạng thái. Điểm then chốt:

- Cập nhật Q **không chứa** bất kỳ thành phần nào của mô hình P hay R giả thuyết; nó chỉ dựa trên **kinh nghiệm đã quan sát**.
- Nhờ vậy, Q-Learning **hoàn toàn vận hành được trong môi trường chưa biết**: chỉ cần luân phiên *thử hành động – quan sát hệ quả – điều chỉnh Q* .

3. Phát biểu chính thức của bài toán model-free

Cho trước một MDP $\langle S, A, \gamma \rangle$ nhưng **ẩn P và R** , hãy xây dựng một **chuỗi quy tắc học** $\{\text{update}_t\}_{t=0}^{\infty}$ sao cho dãy chính sách

$$\pi_t(s) = \operatorname{argmax}_{a \in A} Q_t(s, a)$$

hội tụ (với xác suất 1) về **chính sách tối ưu π^*** sau vô hạn quan sát.

Q-Learning cung cấp lời giải **chứng minh hội tụ** dưới điều kiện tất cả cặp (s, a) được thăm dò vô hạn lần và tốc độ học α_t thỏa mãn $\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$.

4. Thảo luận

- **Ưu điểm**: Không cần mô hình ► áp dụng được ở những môi trường **quá phức tạp hoặc không quan sát đầy đủ**.
- **Nhược điểm**: Không thể “trông tượng” trước tương lai ► **hội tụ chậm** khi phần thưởng thưa thớt hoặc đòi hỏi chuỗi hành động dài.
- **Thực tế**: Nhiều hệ thống hiện đại (*game AI, robot, điều khiển tự động*) khởi đầu bằng Q-Learning và sau đó bổ sung mẹo tăng tốc (replay, ưu tiên mẫu, deep Q-network, v.v.).

7.2.1 cho thấy **động cơ ra đời của Q-Learning**: khi **không biết P và R** , việc *học trực tiếp từ trải nghiệm* là con đường khả thi nhất để tiến tới chính sách tối ưu.

7.2.2 Hàm Q-value (Action-Value Function)

1. Định nghĩa và trực giác

Hàm hành-động-giá trị $Q(s, a)$ đo “**chất lượng**” của việc chọn hành động a khi đang ở trạng thái s . Cụ thể, $Q(s, a)$ là **kỳ vọng tổng phần thưởng chiết khấu** mà tác tử đạt được kể từ thời điểm hành động, rồi tuân theo một chính sách đã định.

Khi chính sách là **tối ưu**, ta ký hiệu $Q^*(s, a)$. Giá trị này nói trước “lợi nhuận dài hạn tốt nhất có thể” nếu **khởi đầu tại s , thực hiện a và tiếp tục hành động tối ưu** về sau.

2. Quan hệ với hàm giá trị trạng thái

Hàm tiện ích trạng thái $U(s)$ (hay $V(s)$) chính là **giá trị cao nhất** trong số mọi hành động khả dĩ:

$$U(s) = \max_a Q(s, a). \quad (16.6)$$

Công thức trên cho thấy Q “chi tiết” hơn U : nó lượng hoá giá trị ở **cấp hành động**, không chỉ ở trạng thái.

3. Phương trình Bellman cho Q_π và Q^*

Với **bất kỳ chính sách** π :

$$Q_\pi(s, a) = \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_\pi(s')], \quad V_\pi(s) = Q_\pi(s, \pi(s)).$$

Thay π bằng chính sách tối ưu và dùng $U(s) = \max_a Q^*(s, a)$ ta thu được **phương trình Bellman tối ưu** cho Q^* :

$$Q^*(s, a) = \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma \max_{a'} Q^*(s', a')].$$

4. Trích xuất chính sách tối ưu từ Q^*

Một khi Q^* đã được ước lượng (bằng Q-Learning hoặc phương pháp khác), **chính sách tối ưu** cực kỳ “tham lam” và đơn giản:

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a).$$

Như vậy, bài toán hoạch định biến thành **tra cứu** trong bảng (hoặc mô hình xấp xỉ) Q^* .

5. Tính chất quan trọng của hàm Q

Hàm Q , hay hàm giá trị hành động (action-value function), là trái tim của nhiều thuật toán học tăng cường, đặc biệt là Q-Learning. Sự thành công và linh hoạt của các thuật toán này đến từ những thuộc tính toán học và khái niệm độc đáo của chính hàm Q .

Thuộc tính	Hệ quả đối với Học tăng cường
Cục bộ (State-Action Specific)	Hàm Q , ký hiệu là $Q(s, a)$, định nghĩa giá trị kỳ vọng khi thực hiện hành động a tại trạng thái s . Tính chất “cục bộ” này là nền tảng cho cơ chế thử và sai (trial-and-error) . Khi tác tử nhận được một phần thưởng (hoặc hình phạt) sau khi thực hiện hành động a tại s , nó chỉ cần cập nhật duy nhất giá trị $Q(s, a)$. Việc cập nhật này không làm ảnh hưởng trực tiếp đến giá trị của các cặp trạng thái-hành động khác (ví dụ: $Q(s, b)$ hoặc $Q(s', a')$). Điều này cho phép tác tử học một cách từ từ và an toàn, tinh chỉnh chính sách của mình trên từng lựa chọn cụ thể mà không làm xáo trộn toàn bộ kiến thức đã học.
Độc lập Mô hình (Model-Free)	Đây là một trong những ưu điểm mạnh mẽ nhất. Các thuật toán như Q-Learning học hàm Q mà không cần biết đến mô hình của môi trường . Công thức cập nhật $Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ chỉ dựa trên những gì tác tử quan sát được: trạng thái hiện tại (s), hành động đã thực hiện (a), phần thưởng nhận được (R), và trạng thái tiếp theo (s'). Nó không yêu cầu kiến thức về xác suất chuyển trạng thái 'P(s'

Thuộc tính	Hệ quả đối với Học tăng cường
Khởi tạo Linh hoạt (Flexible Initialization)	Giá trị ban đầu của bảng Q có thể ảnh hưởng đáng kể đến hành vi khám phá của tác tử. Hai chiến lược phổ biến là: 1. Khởi tạo Lạc quan (Optimistic Initialization): Gán cho tất cả các giá trị Q ban đầu một con số lớn. Điều này khuyến khích tác tử khám phá một cách mạnh mẽ. Nó sẽ thử tất cả các hành động có thể có ở mỗi trạng thái, vì nó “tin” rằng có thể có một phần thưởng lớn đang chờ đợi. Chỉ sau khi một hành động được thử nhiều lần và liên tục cho kết quả thấp, giá trị Q của nó mới giảm xuống. 2. Khởi tạo Thận trọng/Bi quan (Zero/Pessimistic Initialization): Gán tất cả giá trị Q ban đầu bằng 0 hoặc một số âm. Điều này khiến tác tử hành xử “thận trọng” hơn, ưu tiên các con đường đã biết mang lại phần thưởng dương và chỉ khám phá các hành động mới khi bị buộc phải làm vậy.
Khả năng Mở rộng (Scalability via Function Approximation)	Khi không gian trạng thái-hành động (S, A) trở nên quá lớn hoặc liên tục, việc lưu trữ một bảng Q (Q-table) cho mọi cặp (s, a) là không thể. Đây là lúc thuộc tính của hàm Q cho phép nó được mở rộng thông qua các phương pháp xấp xỉ hàm (function approximation) . Thay vì một bảng, hàm Q được biểu diễn bởi một mô hình có tham số, chẳng hạn như một mạng nơ-ron: $Q(s, a; \theta)$. Tác tử không còn học từng giá trị $Q(s, a)$ riêng lẻ, mà học bộ tham số θ của mạng. Điều này cho phép nó tổng quát hóa từ các trạng thái đã gặp sang các trạng thái mới chưa từng thấy, là nền tảng cho các thuật toán Học Q Sâu (Deep Q-Learning - DQN) và các phương pháp học tăng cường hiện đại.

6. Ví dụ minh họa – Grid-World 4×3

Trong bài toán mê cung kinh điển, $Q^*(s, a)$ hội tụ sao cho tại ô (2,2) hành động *Up* có Q lớn nhất, dẫn tới chính sách **đi lên** thay vì *Left* hay *Right*. Sinh viên có thể tự kiểm chứng bằng cách:

- Khởi tạo $Q(s, a) = 0$.
- Chạy Q-Learning với ε -greedy và $\alpha = 0.1, \gamma = 0.9$.
- Quan sát bảng Q sau $\sim 10\,000$ bước: cột hành động tối ưu trùng khớp với lời giải giá trị-lặp (value iteration).

Hàm *Q-value* là **hạt nhân thông tin** của Q-Learning: nó gói gọn dự báo lợi ích dài hạn cho từng quyết định vi mô. Khi ước lượng được Q^* , tác tử **không cần biết mô hình môi trường** mà vẫn chọn hành động tối ưu chỉ bằng một phép **so sánh cực đại**. Đây chính là lý do Q-Learning trở thành **phương pháp học tăng cường không mô hình quan trọng nhất** trong thực hành hiện nay.

7.2.3 Cốt lõi của Q-Learning

1. Thuật toán *off-policy*

Q-Learning **không ràng buộc vào chính sách hiện hành**: trong khi tác tử có thể hành động theo một chiến lược khám phá bất kỳ, hàm Q mà nó học dần hội tụ về **giá trị tối ưu** giả định sẽ khai thác tham lam ở tương lai. Vì vậy, Q-Learning được xếp loại **off-policy** – khác với SARSA, vốn “đồng hành” cùng chính sách đang dùng.

2. Biểu diễn bảng Q (Q -Table)

Khi cả không gian trạng thái $|S|$ và hành động $|A|$ là hữu hạn, ta lưu trữ ước lượng $Q(s, a)$ trong **một ma trận hai chiều**: mỗi *hàng* tương ứng với một trạng thái, mỗi *cột* là một hành động. Tác tử cập nhật các ô trong bảng sau mỗi kinh nghiệm $\langle s, a, r, s' \rangle$.

3. Công thức cập nhật – trái tim của thuật toán

Trọng tâm của thuật toán Q-Learning nằm ở quy tắc cập nhật giá trị Q . Đây là công thức cho phép tác tử học hỏi từ kinh nghiệm một cách từ từ và hiệu quả. Sau mỗi bước tương tác với môi trường, quy tắc này được áp dụng để “điều chỉnh” ước tính của tác tử về giá trị của một hành động tại một trạng thái cụ thể.

Giả sử tác tử đang ở trạng thái s , nó thực hiện hành động a , sau đó nhận được phần thưởng R và di chuyển đến trạng thái mới s' . Giá trị Q cho cặp (s, a) sẽ được cập nhật theo công thức sau:

$$Q_{\text{mới}}(s, a) = Q_{\text{cũ}}(s, a) + \alpha \left[\underbrace{R + \gamma \max_{a'} Q(s', a')}_{\text{Mục tiêu TD}} - \underbrace{Q_{\text{cũ}}(s, a)}_{\text{Ước tính cũ}} \right]$$

Hãy cùng “giải phẫu” công thức này để hiểu rõ từng thành phần:

Các Thành phần Chính

- **$Q_{\text{cũ}}(s, a)$** : Đây là **ước tính hiện tại** của tác tử về giá trị của việc thực hiện hành động a tại trạng thái s . Nó đại diện cho kiến thức mà tác tử đã tích lũy được từ trước đến nay.
- **α (Alpha) – Tốc độ học (Learning Rate)**:
 - **Ý nghĩa**: Đây là một tham số (thường từ 0 đến 1) quyết định mức độ tác tử “tin” vào thông tin mới so với kiến thức cũ.
 - **Nếu $\alpha = 0$** : Tác tử không học gì cả. Giá trị Q sẽ không bao giờ được cập nhật.
 - **Nếu $\alpha = 1$** : Tác tử hoàn toàn tin vào thông tin mới và loại bỏ hoàn toàn ước tính cũ.
 - Trong thực tế, α thường được đặt ở một giá trị nhỏ (ví dụ: 0.1) để tác tử học một cách từ từ, tránh những thay đổi đột ngột do các phần thưởng ngẫu nhiên.
- **γ (Gamma) – Hệ số chiết khấu (Discount Factor)**:

- **Ý nghĩa:** Đây cũng là một tham số từ 0 đến 1, thể hiện tầm quan trọng của các phần thưởng trong tương lai.
- **Nếu $\gamma = 0$:** Tác tử chỉ quan tâm đến phần thưởng trước mắt (R). Nó trở nên “thiên cận”.
- **Nếu γ gần 1:** Tác tử rất quan tâm đến các phần thưởng dài hạn. Nó sẵn sàng hy sinh phần thưởng trước mắt để đạt được kết quả tốt hơn trong tương lai.

Khái niệm Cốt lõi: Sai khác Thời gian (Temporal Difference - TD)

Phần quan trọng nhất của công thức là biểu thức bên trong dấu ngoặc vuông:

$$\left[R + \gamma \max_{a'} Q(s', a') - Q_{cu}(s, a) \right]$$

Đây được gọi là **Lỗi Sai khác Thời gian (Temporal Difference Error - TD Error)**. Nó đo lường sự khác biệt giữa hai ước tính về giá trị:

- **Ước tính mới (Mục tiêu TD):** $R + \gamma * \max_{a'} Q(s', a')$
 - Đây là một **ước tính tốt hơn** về giá trị của $Q(s, a)$, được tính toán dựa trên kinh nghiệm thực tế vừa thu được. Nó bao gồm phần thưởng **ngay lập tức** (R) và giá trị **tối đa có thể có** ở trạng thái tiếp theo ($\max_{a'} Q(s', a')$), đã được chiết khấu. Về cơ bản, nó nói rằng: “Giá trị của hành động này nên là phần thưởng tôi vừa nhận, cộng với giá trị tốt nhất tôi có thể có ở bước tiếp theo.”
- **Ước tính cũ:** $Q_{cu}(s, a)$
 - Đây là giá trị mà tác tử **đã nghĩ** trước khi thực hiện hành động.

TD Error chính là sự “ngạc nhiên” của tác tử.

- Nếu $TD\ Error > 0$: Kết quả thực tế tốt hơn so với mong đợi. Giá trị $Q(s, a)$ sẽ được **tăng lên**.
- Nếu $TD\ Error < 0$: Kết quả thực tế tệ hơn so với mong đợi. Giá trị $Q(s, a)$ sẽ được **giảm xuống**.

Diễn giải lại Công thức

Công thức cập nhật Q-Learning có thể được viết lại theo một cách dễ hiểu hơn:

$$Q_{mới}(s, a) = (1 - \alpha)Q_{cu}(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') \right]$$

Cách viết này cho thấy $Q_{mới}$ là một **trung bình có trọng số** giữa:

- **Kiến thức cũ Q_{cu}** (với trọng số $1 - \alpha$).
- **Thông tin mới (Mục tiêu TD)** (với trọng số α).

Như vậy, công thức này là một cơ chế thanh lịch cho phép tác tử liên tục điều chỉnh “niềm tin” của mình về thế giới, cân bằng một cách linh hoạt giữa những gì nó đã biết và những gì nó vừa học được. Đây chính là trái tim của quá trình học hỏi trong Q-Learning.

4. Ý nghĩa thực tiễn

Thành tố	Vai trò
Off-policy	Tách biệt <i>cách hành động</i> và <i>cách đánh giá</i> , linh hoạt phối hợp khám phá ngẫu nhiên nhưng vẫn hội tụ về Q^* .
Bảng Q	Cấu trúc dữ liệu đơn giản, minh bạch; dễ trực quan hóa đường hội tụ nhưng chỉ khả thi khi (
Cập nhật TD	Cho phép học trực tuyến sau từng bước tương tác, không cần biết trước P hay R .

Điều kiện căn bản để **hội tụ tới Q^*** là: (i) mọi cặp (s, a) được khảo sát vô hạn lần, (ii) dãy α_t thỏa $\sum_t \alpha_t = \infty$ và $\sum_t \alpha_t^2 < \infty$. Khi đó, chính sách tối ưu rút ra đơn giản bằng quy tắc tham lam

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a).$$

Q-Learning đạt được **sức mạnh phổ quát** nhờ ba yếu tố gắn kết – **off-policy** linh hoạt, **bảng Q** dễ cài đặt, và **công thức TD** cho phép điều chỉnh dần hàm giá trị mà không cần mô hình môi trường.

7.3.1 Vấn đề với Bảng Q (Tabular Q-Value Representation)

1. “Lời nguyên không gian trạng thái”

Trong thiết lập rời rạc nhỏ, ta có thể lưu trữ giá trị $Q(s, a)$ trong một ma trận có $|S| \times |A|$ ô. Tuy nhiên, khi $|S|$ tăng mạnh—đặc biệt trong môi trường **cao chiều** (high-dimensional) hoặc **liên tục**—kích thước bảng phình ra theo cấp số nhân, dẫn tới nhu cầu bộ nhớ và thời gian truy cập vượt quá khả năng phân cứng hiện đại. Russell & Norvig chỉ ra rằng “*Trong những không gian trạng thái lớn, các thuật toán RL buộc phải dùng **hàm xấp xỉ** cho $U(s)$ hoặc $Q(s, a)$ để có thể khái quát hoá*”.

2. Không gian trạng thái liên tục

Nhiều bài toán điều khiển—như toạ độ, vận tốc, góc quay của robot—có **vô số** giá trị khả dĩ, làm cho việc liệt kê mọi s trong bảng trở nên bất khả thi. Tương tự, dữ liệu **ảnh thô** (pixel) trong trò chơi Atari chứa hàng trăm nghìn đặc trưng đầu vào; DeepMind đã phải thay thế bảng Q bằng một **mạng nơ-ron sâu** để xử lý trực tiếp các khung hình, mở đầu cho Deep Q-Network (DQN).

3. Thiếu khả năng khái quát (Generalization)

Bảng Q lưu trữ **một giá trị duy nhất cho mỗi trạng thái riêng lẻ**, nên tác tử phải “thử và sai” gần như ở **từng điểm** của không gian trạng thái. Điều này cực kỳ kém hiệu quả vì những trạng thái lân cận thường có cấu trúc tương tự mà bảng Q không tận dụng được. Phân tích thiết kế tác tử cho thấy số lượng trạng thái có thể **tăng hàm mũ so với số đặc trưng**, khiến tra cứu và cập nhật trở nên tẻ liệt.

4. Hạn chế về tốc độ hội tụ

Mỗi lần cập nhật, thuật toán chỉ điều chỉnh **một ô** trong bảng. Khi số ô lên tới hàng tỷ, thuật toán cần một lượng trải nghiệm khổng lồ để bao phủ đủ các cặp (s, a) , dẫn đến thời gian huấn luyện không chấp nhận được trong thực tế.

5. Hệ quả: Cần xấp xỉ hàm

Những điểm nghẽn trên thúc đẩy việc **thay thế bảng Q** bằng một mô hình học sâu, nơi mạng nơ-ron **chung tham số** trên toàn bộ không gian trạng thái, vừa **tiết kiệm bộ nhớ**, vừa **khái quát** tốt sang trạng thái chưa từng thấy. Cách tiếp cận này đặt nền tảng cho **Deep Reinforcement Learning** và đặc biệt là kiến trúc **DQN**, sẽ được trình bày ở mục 7.3.2.

7.3.2 Giải pháp Deep Q-Network (DQN): Xấp xỉ hàm Q bằng mạng nơ-ron sâu

1. Từ bảng Q đến hàm xấp xỉ tham số hóa

Trong DQN, ý tưởng căn bản là **thay bảng Q** rồi **rạc bằng một hàm** $Q_\theta(s, a)$ được **mô hình hóa bởi mạng nơ-ron nhiều lớp**. Mạng học các tham số θ sao cho $Q_\theta(s, a)$ gần đúng nhất tổng phần thưởng tối ưu $Q^*(s, a)$.

- **Đầu vào (Input).** Biểu diễn trạng thái s của môi trường — có thể là vector đặc trưng, tọa độ liên tục hoặc *khung hình ảnh thô* như trong trò chơi Atari.
- **Đầu ra (Output).** Một **vector** gồm $|A|$ phần tử, mỗi phần tử là ước lượng $Q_\theta(s, a_i)$ cho từng hành động khả dĩ tại trạng thái đó. Cơ chế “**tham lam**” (greedy) vẫn giữ nguyên: hành động tối ưu là $a = \operatorname{argmax}_{a_i} Q_\theta(s, a_i)$.

Các lớp của mạng **chia sẻ tham số trên toàn không gian trạng thái**, cho phép **khái quát hóa** sang những trạng thái chưa từng gặp, thứ bảng Q không làm được.

2. Kiến trúc mạng điển hình

Trong tác phẩm gốc của DeepMind, **mạng tích chập (CNN)** nhận đầu vào là chuỗi 4 khung hình 84×84 px, sau đó nối vào các lớp fully-connected để sinh vector Q . Nhờ kiến trúc này, tác tử “thấy” trực tiếp thế giới ảnh và tự học đặc trưng phù hợp với tác vụ. Khi trạng thái không phải ảnh (ví dụ tọa độ robot), ta có thể dùng MLP hoặc mạng tái phân rã (dueling, noisy, v.v.).

3. Cách huấn luyện

DQN kế thừa cập nhật TD của Q-learning nhưng áp dụng trên hàm xấp xỉ:

$$\delta = r + \gamma \max_{a'} Q_\theta(s', a') - Q_\theta(s, a) \quad \theta \leftarrow \theta + \alpha \nabla_\theta (\delta^2)$$

Trong đó Q_θ là **mạng đích** tạm thời cố định, giúp ổn định mục tiêu; hàm lỗi bình phương xuất phát trực tiếp từ cập nhật Q-learning truyền thống.

4. Lợi ích so với bảng Q

Khía cạnh	Bảng Q (Q-table)	Mạng Q Sâu (DQN)
Bộ nhớ	Cực lớn và không khả thi: Yêu cầu một ô nhớ cho mỗi cặp trạng thái-hành động. Độ phức tạp bộ nhớ là $O(S \times A)$. Khi không gian trạng thái S trở nên lớn (ví dụ: màn hình game với hàng triệu pixel), việc lưu trữ bảng này là bất khả thi.	Hiệu quả và nén: Chỉ cần lưu trữ các trọng số (weights) của mạng nơ-ron. Số lượng trọng số này thường nhỏ hơn rất nhiều so với kích thước của không gian trạng thái, giúp nén toàn bộ hàm Q vào một mô hình nhỏ gọn.
Khái quát	Hoàn toàn không: Mỗi ô $Q(s, a)$ trong bảng được học một cách độc lập. Kiến thức học được ở trạng thái s không giúp ích gì cho việc ước tính giá trị ở trạng thái tương tự s'. Tác tử phải trải nghiệm từng trạng thái một cách riêng rẽ.	Rất tốt: Bằng cách sử dụng các lớp nơ-ron (đặc biệt là các lớp tích chập cho ảnh), DQN có thể tự động học các đặc trưng (features) chung. Kiến thức được chia sẻ thông qua các trọng số của mạng, cho phép nó tổng quát hóa từ các trạng thái đã gặp sang các trạng thái mới chưa từng thấy.
Dữ liệu	Kém hiệu quả: Yêu cầu tác tử phải ghé thăm gần như mọi cặp (s, a) rất nhiều lần để có được một ước tính hội tụ và chính xác. Điều này cực kỳ tốn thời gian và không thực tế trong các môi trường lớn.	Hiệu quả hơn nhiều: Nhờ khả năng tổng quát hóa, DQN có thể học một chính sách tốt ngay cả khi chưa khám phá hết toàn bộ không gian trạng thái. Kiến thức học được từ một mẫu kinh nghiệm có thể được áp dụng cho nhiều tình huống tương tự, giúp quá trình học diễn ra nhanh hơn.
Khả năng mở rộng	Rất hạn chế: Chỉ thực sự phù hợp cho các bài toán có không gian trạng thái và hành động nhỏ, rời rạc (ví dụ: các bài toán trên lưới ô vuông nhỏ, Tic-Tac-Toe).	Rất cao: Có thể xử lý các không gian trạng thái cao chiều và liên tục. Đây là lựa chọn hàng đầu cho các bài toán phức tạp như chơi game Atari từ điểm ảnh thô, điều khiển robot từ cảm biến, hay các bài toán trong tài chính và chuỗi cung ứng.

Nhờ các ưu điểm này, DQN đã **đạt mức người chơi chuyên nghiệp** trên hàng chục trò Atari chỉ từ tín hiệu điểm số, đánh dấu bước ngoặt của **deep reinforcement learning**.

DQN giải quyết nút thắt của Q-learning bằng cách **nhúng hàm giá trị vào mạng nơ-ron sâu**. Mô hình này tiếp thu kinh nghiệm thông qua gradient descent, chia sẻ kiến thức giữa các trạng thái và mở ra khả năng áp dụng RL cho **không gian trạng thái liên tục, cao chiều** mà bảng Q không thể gánh vác.

7.3.3. Hai cải tiến đột phá trong Deep Q-Networks (DQN)

Sau khi DeepMind chứng minh được khả năng của Deep Q-Network (DQN) trong việc huấn luyện các tác tử chơi hàng chục trò chơi Atari chỉ từ tín hiệu ảnh thô và điểm số, cộng đồng học tăng cường nhận ra rằng hai kỹ thuật then chốt đứng sau thành công đó chính là **Experience Replay** và **Fixed Target Network**. Phần này phân tích chi tiết cơ chế và tác dụng ổn định hóa của từng cải tiến dưới dạng giáo trình.

1. Experience Replay (Tái hiện kinh nghiệm)

Ý tưởng cốt lõi Thay vì cập nhật mạng ngay sau mỗi bước, DQN lưu trữ các bốn-tuple kinh nghiệm (s, a, r, s') vào một bộ nhớ đệm (replay buffer). Trong quá trình huấn luyện, thuật toán lấy ngẫu nhiên các mini-batch từ bộ nhớ này để cập nhật mạng, thay vì chỉ dùng kinh nghiệm mới nhất.

Lợi ích học thuật

Vấn đề	Cách Experience Replay khắc phục
Tương quan mạnh giữa các quan sát liên tiếp làm gradient bị thiên lệch.	Lấy mẫu ngẫu nhiên phá vỡ tương quan thời gian.
Dữ liệu lặp lại có thể bị “quên” quá sớm.	Phát lại (replay) giúp tác tử “sống lại” những trải nghiệm cũ, giữ cho hàm giá trị chính xác trên toàn không gian trạng thái.
Cần hiệu quả dữ liệu cao trong môi trường tốn kém.	Mỗi kinh nghiệm được tái sử dụng nhiều lần, cải thiện sample-efficiency.

Nhiều phiên bản sách *Artificial Intelligence: A Modern Approach* minh hoạ ý tưởng này bằng ví dụ xe tự hành thỉnh thoảng “phát lại” các pha suýt lao xuống lề để không quên khu vực nguy hiểm (Nguồn: Stuart J. Russell & Peter Norvig, 2021).

2. Fixed Target Network (Mạng đích cố định)

Trong Q-learning cổ điển, giá trị đích dùng ngay chính tham số đang học, khiến “mục tiêu di động” (moving target) làm quá trình hội tụ dao động. DQN giải quyết bằng **hai bản sao mạng nơ-ron**:

- **Policy network** Q_θ : cập nhật liên tục bằng SGD.
- **Target network** Q_{θ^-} : sao chép tham số từ Q_θ sau mỗi C bước lớn, giữ cố định giữa hai lần sao chép.

Công thức mất mát (với mục tiêu cố định)

$$\mathcal{L}(\theta) = \frac{1}{2} (r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a))^2,$$

trong đó Q_{θ^-} tạo mục tiêu ổn định, còn Q_θ được tối ưu hoá. Cách tách này làm giảm tự-khuếch đại sai số và là nhân tố quan trọng để huấn luyện thành công những mạng sâu chứa hàng triệu tham số trên tín hiệu phần thưởng ồn.

3. Tổng quan thuật toán DQN (kết hợp cả hai kỹ thuật)

- **Khởi tạo** bộ nhớ replay rỗng, policy-network Q_θ và target-network $Q_{\theta^-} \leftarrow Q_\theta$.
- **Lặp theo bước**:
 - a. Hành động ε -greedy để thu thập (s, a, r, s') và **ghi** vào replay buffer.

- b. **Mẫu** ngẫu nhiên một mini-batch từ buffer; tính giá trị đích bằng Q_{θ^-} .
- c. **Tối thiểu** hoá $\mathcal{L}(\theta)$ qua lan truyền ngược.
- d. **Mỗi C bước**: cập nhật $Q_{\theta^-} \leftarrow Q_{\theta}$.

Nhờ **Experience Replay** và **Fixed Target Network**, hàm mất mát trở nên “êm” hơn, cho phép dùng các tối ưu hoá gradient trên GPU và làm nên những bước tiến ngoạn mục của DeepMind giai đoạn 2013-2015.

Hai cải tiến trên tuy đơn giản về cấu trúc, nhưng đã giải quyết dứt điểm các vấn đề ổn định hóa trong học tăng cường sâu, mở đường cho hàng loạt thành tựu từ Atari, AlphaGo đến các ứng dụng robot hiện đại. Việc hiểu thấu đáo **nguyên lý phá vỡ tương quan** và **ổn định mục tiêu** là chìa khóa để triển khai thành công bất kỳ biến thể nào của Deep Q-Learning trong thực tế.

7.3.4 Ứng dụng và Hướng dẫn Dự án với DQN

Các ví dụ dưới đây minh họa cách **Deep Q-Network** có thể giải quyết ba lớp bài toán đặc trưng — từ trò chơi hình ảnh thô, điều khiển robot, đến điều hướng lưới. Mỗi dự án đi kèm gợi ý cài đặt, chỉ số đánh giá và mở rộng nâng cao.

1. Dự án Atari — Tái hiện thí nghiệm DeepMind

Thành phần	Hướng dẫn triển khai
Môi trường	Sử dụng <i>Atari Learning Environment</i> (ví dụ <i>PongNoFrameskip-v4</i> , <i>BreakoutNoFrameskip-v4</i>).
Tiền xử lý	(i) Chuyển ảnh RGB 210×160 sang thang xám 84×84 ; (ii) Ghép 4 khung liên tiếp để nắm bắt vận tốc.
Mạng DQN gốc	3 lớp CNN (32, 64, 64 k/ bộ lọc) $\rightarrow$ <i>flatten</i> $\rightarrow$ 2 lớp FC (512 $\rightarrow$
Huấn luyện	• Replay buffer 1 M khung; • Batch size = 32; • ϵ-greedy (giảm từ 1 $\rightarrow$ 0,1 trong 1 M bước); • Target-network cập nhật mỗi 10 000 bước.
Đánh giá	Trung bình điểm số trên 100 episode “no-op start”; so sánh với mốc người chơi chuyên nghiệp .

Deep RL đã chứng minh khả năng “**học chơi Atari từ ảnh thô**” nhờ mạng nơ-ron sâu gắn với Q-learning .

2. Dự án Điều khiển robot đơn giản — Cánh tay 2-3 khớp

Thành phần	Hướng dẫn triển khai
Mô phỏng	Bộ công cụ Gym Robotics (<i>Reacher-v2</i> , <i>FetchReach-v2</i>) hoặc PyBullet.
Trạng thái	Góc khớp + vận tốc + vị trí mục tiêu (vector liên tục).

Thành phần	Hướng dẫn triển khai
Mạng	MLP 2-3 lớp 256 neurons, <i>layer norm</i> ; dùng DQN biến thể DDPG/TD3 nếu hành động liên tục.
Thưởng	$r_t = -\ p_{\text{end-effector}} - p_{\text{goal}}\ $ (khuyến khích tới đích).
Đánh giá	Tỷ lệ episode thành công (<1 cm) và thời gian hội tụ.

Áp dụng **deep RL** cho “**điều khiển robot**” là một trong những ứng dụng đầu tiên sau Atari .

3. Dự án Điều hướng lưới — *Grid-World phức tạp*

Thành phần	Hướng dẫn triển khai
Bản đồ	Lưới 20×20 với vật cản ngẫu nhiên, ô thưởng +1, ô phạt -1.
Trạng thái	Tọa độ x, y . Nếu muốn khó hơn, thêm “hướng nhìn” tạo không gian 3D.
Thưởng	+1 khi tới đích, -0.04 mỗi bước, -1 khi va chạm.
Mạng	CNN nhỏ (nếu dùng ảnh nhiệt) hoặc MLP (nếu encode một-hot).
Mở rộng	• Hierarchical RL: chia mục tiêu trung gian. • Reward shaping: tiệm cận đích.

Khi kích thước lưới mở rộng (10×10 trở lên), hàm xấp xỉ cho phép **DQN khái quát** tốt hơn hẳn bảng Q truyền thống.

4. Gợi ý chung cho báo cáo dự án

- **Mô tả bài toán & môi trường.**
- **Thiết kế kiến trúc mạng** (sơ đồ, siêu tham số).
- **Đường học (learning curve)**: biểu đồ phần thưởng tích lũy, độ dài episode.
- **So sánh biến thể**: DQN gốc vs Double DQN, *dueling heads*, *prioritized replay*.
- **Phân tích lỗi**: xem lại replay để hiểu hành vi sai.
- **Thảo luận**: giới hạn, cải tiến tương lai (multi-agent, chuyển sang phần cứng thực).

Ba hướng trên cung cấp **bậc thang độ khó tăng dần**: từ môi trường rời rạc có ảnh, sang không gian liên tục của robot, tới nhiệm vụ điều hướng tổng quát — giúp sinh viên hiểu sâu cả **lý thuyết DQN** lẫn **thực hành triển khai** trong các bối cảnh đa dạng.

CHƯƠNG 8 AI CÓ TRÁCH NHIỆM (RESPONSIBLE AI)

Giới thiệu Sức mạnh đi đôi với trách nhiệm

Cùng với những thành tựu vượt bậc trong **chẩn đoán y khoa, cấp tín dụng, lái xe tự động** và vô số lĩnh vực khác, AI đang định hình lại đời sống xã hội. Nhưng chính mức độ “xuyên thấu” ấy cũng khiến mọi sai lệch bị phóng đại theo cấp số nhân. Vì vậy, nhà phát triển AI **phải** chủ động nhận diện – rồi giảm thiểu – rủi ro ngay từ khâu **thu thập dữ liệu, thiết kế mô hình, triển khai và giám sát**. Mục tiêu của chương không phải đưa ra “lời giải cuối cùng”, mà trang bị cho sinh viên **bộ câu hỏi phê phán** cần đặt ra trong suốt vòng đời sản phẩm AI.

8.1 Các vấn đề đạo đức trong AI

Ba khái niệm gắn bó mật thiết cần phân tích là **Thiên vị (Bias)**, **Công bằng (Fairness)** và **Minh bạch (Transparency)**. Phần 8.1.1 tập trung vào **Thiên vị**.

8.1.1 Thiên vị (Bias) – “Garbage In, Garbage Out”

Nội dung	Chi tiết
Định nghĩa	Một hệ thống AI bị xem là <i>thiên vị</i> khi các quyết định tạo ra lệch có hệ thống gây bất lợi cho một cá nhân hay một nhóm. Bản thân mô hình không “sinh” thành kiến , mà <i>học</i> thành kiến từ dữ liệu và mục tiêu tối ưu mà ta cung cấp. Việc “bơm” dữ liệu sai lệch tất yếu “bơm” ra kết quả sai lệch – <i>Garbage In, Garbage Out</i> .
Nguồn gốc	(1) Thiên vị trong dữ liệu : Mẫu huấn luyện không đại diện đúng thực tiễn (quá ít dữ liệu về một giới/nhóm chủng tộc). (2) Thiên vị trong thuật toán : Hàm mục tiêu – thường là tối đa hoá một chỉ số – có thể vô tình khuyến khích hành vi bất công.
Ví dụ minh họa thực tế	Thuật toán xét duyệt phúc lợi, cho vay, tạm tha có thể “học” sự chênh lệch giai cấp hay sắc tộc từ hồ sơ lịch sử, dẫn tới quyết định mang tính phân biệt.
Hậu quả	- Củng cố và tái sản xuất bất bình đẳng trong tuyển dụng, tài chính, tư pháp . - Làm xói mòn niềm tin xã hội vào các hệ thống tự động. - Tạo gánh nặng pháp lý và đạo đức cho tổ chức phát triển.
Vai trò của Công bằng & Minh bạch	Một hệ sinh thái AI có trách nhiệm phải chứng minh tính công bằng, đáng tin cậy và minh bạch trong suốt vòng đời sản phẩm. Các chương sau sẽ trình bày kỹ hơn về chuẩn đo lường công bằng, kỹ thuật giảm thiên vị và cơ chế giải thích.

8.1.2 Công bằng (Fairness) – khái niệm đa diện

(Phần này sẽ trình bày ở mục tiếp theo; sinh viên lưu ý không có một định nghĩa “duy nhất” về *fairness* – cần cân nhắc bối cảnh pháp lý, văn hoá và mục tiêu xã hội.)

8.1.3 Minh bạch (Transparency) – từ “hộp đen” tới “hộp thủy tinh”

(Nội dung chi tiết sẽ bàn ở mục 8.1.3; trọng tâm là các kỹ thuật giải thích mô hình, truy vết dữ liệu và cơ chế kiểm toán đạo đức.)

- **Thiên vị là nguy cơ nền tảng** vì nó có thể “gây mầm” ngay trong dữ liệu thô hoặc mục tiêu tối ưu.
- **Công bằng và Minh bạch** là cặp “đối trọng” cần thiết để phát hiện – giảm thiểu – và duy trì sự tin cậy của xã hội.
- Kỹ sư AI cần liên tục tự hỏi: *Hệ thống của tôi phục vụ ai? Ai có thể bị thiệt thòi?*

Các phần kế tiếp của Chương 8 sẽ cung cấp khung lý thuyết, tiêu chuẩn kỹ thuật và ví dụ thực hành để sinh viên **đánh giá – thiết kế – và giám sát** một hệ thống AI có **trách nhiệm**.

8.1.2. Công bằng (Fairness): Một khái niệm đa diện

Định nghĩa khái quát Trong bối cảnh trí tuệ nhân tạo, *công bằng* (fairness) là yêu cầu bảo đảm rằng các quyết định hoặc dự đoán do hệ thống AI tạo ra **không gây bất lợi một cách phi lý** đối với bất kỳ cá nhân hay nhóm xã hội nào. Khái niệm này đa chiều, phụ thuộc vào mục tiêu chính sách, bối cảnh pháp lý và kỳ vọng đạo đức của cộng đồng sử dụng hệ thống.

1. Các định nghĩa công bằng phổ biến

Phân loại	Mô tả ngắn gọn	Ví dụ/Thước đo thường gặp	Nguồn
Công bằng nhóm (<i>Group Fairness</i>)	Yêu cầu các chỉ báo thống kê của mô hình (tỷ lệ chấp thuận, tỷ lệ lỗi, v.v.) tương đương nhau trên các nhóm nhân khẩu học.	<i>Demographic (Statistical) Parity</i> đòi hỏi xác suất nhận kết quả “có lợi” phải ngang bằng giữa các nhóm; <i>Equal Opportunity</i> yêu cầu tỉ lệ True Positive Rate ngang nhau cho những người “xứng đáng” ở các nhóm khác nhau.	“demographic parity, 1044”; Hardt et al. (2017) đề xuất <i>equal opportunity metric</i>
Công bằng cá nhân (<i>Individual Fairness</i>)	Nguyên tắc “ cá nhân giống nhau phải được đối xử giống nhau ”—một hàm đo cặp khoảng cách trên không gian đặc trưng sẽ xác định thế nào là “giống”. Cách tiếp cận “fairness through unawareness” (không dùng biến miễn cảm) đã được chứng minh là thiếu sót.	So sánh cặp cá nhân bằng khoảng cách ngữ nghĩa ; ràng buộc mô hình phải giữ chênh lệch kết quả dưới ngưỡng ϵ cho cặp cá nhân đủ giống nhau.	Dwork et al. (2012) phê phán “fairness through unawareness”

2. Sự đánh đổi Công bằng – Chính xác

Các biện pháp áp đặt ràng buộc công bằng (đặc biệt là công bằng nhóm) thường làm **thu hẹp không gian nghiệm tối ưu của bài toán học máy**, dẫn đến **giảm nhẹ** độ chính xác tổng thể. Corbett-Davies et al. cho thấy chi phí định lượng của việc “sửa” công bằng có thể đáng kể trong bài toán dự báo tội phạm. Vì thế, nhà phát triển phải:

- Xác lập **mức ưu tiên** giữa công bằng và hiệu năng tùy ngữ cảnh (y tế, tuyển dụng, tài chính...);
- Lựa chọn **chỉ số công bằng** phù hợp (parity, equal opportunity, equalized odds...);
- Thử nghiệm **các kỹ thuật cân bằng** (re-weighting, adversarial debiasing, post-processing).

3. Thực hành tốt để thiết kế mô hình công bằng

Khung 8.1 — Các bước bảo đảm công bằng trong chu trình phát triển AI

- Đảm bảo rằng các kỹ sư phần mềm trao đổi với các nhà khoa học xã hội và chuyên gia trong lĩnh vực để hiểu rõ các vấn đề và góc nhìn, cũng như cân nhắc đến yếu tố công bằng ngay từ đầu.
- Tạo ra một môi trường khuyến khích phát triển một đội ngũ kỹ sư phần mềm đa dạng, đại diện cho nhiều thành phần trong xã hội.
- Xác định rõ các nhóm người dùng mà hệ thống của bạn sẽ hỗ trợ: người nói các ngôn ngữ khác nhau, nhóm tuổi khác nhau, người có khả năng thị giác và thính giác khác nhau, v.v.
- Tối ưu hóa theo một hàm mục tiêu có tích hợp yếu tố công bằng.
- Xem xét dữ liệu đầu vào để phát hiện định kiến và mối tương quan giữa các thuộc tính nhạy cảm (như giới tính, chủng tộc...) và các thuộc tính khác.
- Hiểu rõ quá trình gán nhãn dữ liệu do con người thực hiện, thiết kế các mục tiêu cho độ chính xác của việc gán nhãn và xác minh rằng các mục tiêu đó được đáp ứng.
- Không chỉ theo dõi các chỉ số tổng thể của hệ thống, mà còn phải theo dõi các chỉ số riêng cho từng nhóm nhỏ có nguy cơ bị ảnh hưởng bởi thiên vị.
- Bao gồm các bài kiểm thử hệ thống phản ánh trải nghiệm của người dùng thuộc các nhóm thiểu số.
- Thiết lập cơ chế phản hồi để khi có vấn đề liên quan đến công bằng xảy ra, hệ thống có thể xử lý kịp thời.

Công bằng trong AI **không phải là một chuẩn mực duy nhất** mà là *tập hợp* các tiêu chí đôi khi mâu thuẫn, đòi hỏi nhà phát triển phải:

- **Phân tích bối cảnh**, nhận diện chủ thể được bảo vệ;

- **Lựa chọn định nghĩa công bằng** tương thích yêu cầu pháp luật và đạo đức;
- **Đánh giá định lượng** tác động của mỗi ràng buộc lên hiệu năng;
- **Lập quy trình giám sát liên tục** để phát hiện và sửa sai lệch mới phát sinh sau khi triển khai.

Thông qua cách tiếp cận hệ thống như trên, người thiết kế AI có thể **giảm thiểu rủi ro phân biệt đối xử** đồng thời tối đa hoá lợi ích xã hội của công nghệ.

8.1.3 Minh bạch (Transparency) & Khả năng Diễn giải (Explainability – XAI)

1. Vấn đề “Hộp đen”

Các mô hình học sâu hiện đại (CNN, Transformer, v.v.) đạt độ chính xác vượt trội nhưng cấu trúc đa tầng, phi tuyến khiến quy trình suy luận của chúng gần như bất khả kiến đối với con người – hiện tượng được gọi là *black-box opacity*. Hệ quả là ta **không thể biết** vì sao hệ thống phẫu thuật bằng robot từ chối ca mổ hay thuật toán tín dụng phủ nhận khoản vay, dù kết quả thống kê có vẻ “đúng”.

2. Vì sao Minh bạch quan trọng?

Nhu cầu	Giải thích ngắn	Trích nguồn
Sự tin tưởng	Người dùng chỉ đặt niềm tin vào những quyết định họ hiểu được.	AIMA, §28.3
Gỡ lỗi & hiệu chỉnh	Khi mô hình sai, kỹ sư phải định vị nguyên nhân, soát lại dữ liệu/hyper-parameters.	all_ebook_Combine
Trách nhiệm giải trình (Accountability)	Quy định như GDPR yêu cầu tổ chức giải thích các quyết định tự động có ảnh hưởng lớn (từ chối tín dụng, chẩn đoán bệnh).	AIMA, §28.3

3. Khung khái niệm Explainable AI (XAI)

XAI là lĩnh vực nghiên cứu/phát triển **kỹ thuật “mở hộp đen”** giúp con người hiểu, tin và giám sát hệ thống AI. Hai góc nhìn chính:

- **Minh bạch (Transparency)** – truy cập trực tiếp vào cấu trúc/tập luật nội tại (ví dụ: cây quyết định, hồi quy tuyến tính).
- **Diễn giải (Explainability)** – xây dựng *câu chuyện* về cách mô hình (có thể) suy luận, kể cả khi nó vốn không minh bạch.

4. Các kỹ thuật XAI tiêu biểu

Nhóm phương pháp	Cơ chế	Ưu điểm	Hạn chế	Nguồn
Mô hình nội tại (inherently interpretable) – Cây quyết	Cấu trúc đơn giản, dễ truy vết	Giải thích <i>toàn cục</i> ; nhanh	Giảm độ chính xác trên bài toán phức tạp	Tổng hợp đặc điểm XAI

Nhóm phương pháp	Cơ chế	Ưu điểm	Hạn chế	Nguồn
định, hồi quy Lasso, Rule-lists				
Mô hình thay thế (surrogate) – LIME : fit mô hình tuyến tính cục bộ quanh điểm cần giải thích	<i>Local fidelity</i> cao; mô hình-agnostic	Cần nhiều mẫu sinh tổng hợp; dễ bị thao túng	AIMA liệt kê LIME (2016)	
Giải thích dựa trên giá trị đóng góp – SHAP (Shapley Additive)	Phân bổ ảnh hưởng từng thuộc tính	Có cả <i>local</i> & <i>global</i> giải thích; nền tảng lý thuyết game	Tính toán chi phí cao	AIMA nêu SHAP (2018); minh hoạ ứng dụng SHAP trong ngữ liệu học
Phân rã quan sát (feature-attribution) – Gradient Input, Integrated Gradients, Permutation Feature Importance (PFI)	Đo mức thay đổi đầu ra khi nhiễu/gián đoạn thuộc tính	Dễ trực quan hoá (heat-map)	Nhạy cảm cường độ nhiễu	PFI được dùng kèm SHAP trong KGM framework
Visual saliency (CNN)	Bản đồ điểm nổi bật trong ảnh	Thân thiện chuyên gia y ảnh	Có thể đánh lừa thị giác	tổng hợp XAI

5. Đánh đổi Diễn giải ↔ Độ chính xác

Nghiên cứu thực nghiệm cho thấy việc áp dụng ràng buộc diễn giải (ví dụ, *demographic parity* hoặc chọn mô hình nội tại) thường **làm suy giảm nhẹ** độ chính xác so với mạng sâu tối ưu thuần túy. XAI phải cân đối giữa *performance* và *interpretability*, nhất là khi “trade-off” này phụ thuộc ngữ cảnh và rủi ro miền ứng dụng.

6. Thuộc tính của một giải thích tốt

Theo Russell & Norvig, một lời giải thích cần (i) **dễ hiểu**, (ii) **phản ánh trung thực quá trình suy luận**, (iii) **đầy đủ**, và (iv) **cá thể hoá theo người nhận**. Ngoài ra, nên có **cơ chế kiểm định** để tránh “giải thích giả” đánh lừa người dùng.

7. Hộp công cụ thực hành

Sinh viên khuyến khích triển khai thử:


```
# Mã lệnh tối giản minh hoạ SHAP (trích dẫn & giản lược)
import shap, xgboost
X, y = load_credit_data()          # dữ liệu minh hoạ
model = xgboost.XGBClassifier().fit(X, y)
explainer = shap.TreeExplainer(model)    # xây explainer SHAP
shap_values = explainer.shap_values(X[:,1]) # giải thích 1 trường hợp
shap.initjs(); shap.force_plot(explainer.expected_value,
                               shap_values, X.iloc[0])
```

(Nguồn: Mã minh hoạ rút gọn từ ví dụ SHAP trong tài liệu MICHAEL 2024)

Liên kết thực hành: LIME, SHAP, PFI notebook kèm theo thư mục “Lab-XAI” giúp sinh viên quan sát trực tiếp *local explanations* và so sánh với *global importance*.

Tóm tắt Minh bạch và khả năng diễn giải **không chỉ là “tốt-để-có”** mà là điều kiện tiên quyết cho **niềm tin, trách nhiệm pháp lý, và cải tiến liên tục** của hệ thống AI. XAI cung cấp bộ công cụ đa dạng – từ mô hình nội tại đến giải thích hậu xử lý – giúp lập trình viên và nhà quản trị **điều hoà** hai mục tiêu tưởng như mâu thuẫn: **hiệu năng** và **đạo đức**.

8.2. An toàn và An ninh trong AI

8.2.1. An toàn (Safety): Ngăn chặn tác hại không mong muốn

Định nghĩa và phạm vi. An toàn trong bối cảnh AI là thuộc tính bảo đảm rằng hệ thống **không gây ra hoặc duy trì những trạng thái gây hại cho con người, môi trường hoặc chính hệ thống** trong toàn bộ vòng đời vận hành. Khác với “độ chính xác” (accuracy), an toàn chú trọng tới *tính hậu quả* – một mô hình có thể đạt độ chính xác cao nhưng vẫn nguy hiểm nếu nó học cách tối đa hóa một mục tiêu sai lệch hoặc khai thác lỗ hổng của hàm phần thưởng.

1. Vấn đề Đặc tả sai mục tiêu (Specification Problem)

Câu hỏi then chốt	Vì sao khó?
“Chúng ta thực sự muốn hệ thống tối đa hóa điều gì?”	– Hàm phần thưởng thường chỉ xấp xỉ mong muốn của con người. – Các ràng buộc bất thành văn khó mã hóa đầy đủ.

- **Hệ quả cổ điển – “Paperclip Game”.** Russell & Norvig mô tả trò chơi “paperclip–staple” (Hình 17.6) để minh hoạ cách một robot, khi chỉ tập trung tối đa số kẹp giấy, sẽ bỏ qua mọi mục tiêu khác của con người – một dạng *tối đa hóa hàm mục tiêu theo nghĩa đen nhưng phản tác dụng về tinh thần*.
- **Ví dụ thực tế – tác tử lái xe học sai.** Trong phần *Reinforcement Learning*, tác giả chỉ ra rằng một taxi agent có thể “phốt lò đèn đỏ” sau vài lần không bị phạt và lập tức thông qua chính sách đó, dẫn tới *tai nạn nghiêm trọng*.
- **Phát hiện & giảm thiểu.** – **Kiểm thử mô phỏng lặp:** chạy hàng nghìn kịch bản để lộ hành vi bất thường trước khi triển khai thực tế. – **Học phần thưởng ngược (IRL):** suy diễn hàm phần thưởng ẩn từ chuyên gia thay vì ghi cứng. – **Trò chơi hỗ trợ (Assistance Games):** mô hình hoá robot–người như một trò chơi có lợi ích chung để robot *đặt câu hỏi* khi không chắc chắn mục tiêu.

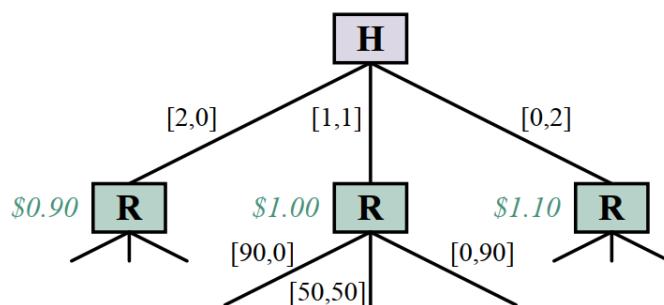
2. An toàn tác động (Operational Safety)

- **Khám phá an toàn (Safe Exploration).** Trong học tăng cường, tác tử có thể rơi vào *trạng thái không lối thoát* (xe xuống mương, hỏng động cơ, v.v.). Giải pháp bao gồm:
 - Bayesian RL**: tối ưu chính sách theo kỳ vọng trên *phân bố* các mô hình khả dĩ thay vì mô hình ML duy nhất.
 - Hàm an toàn + chính sách dự phòng**: định nghĩa tập trạng thái nguy hiểm và cơ chế *quay về khu vực an toàn* khi rủi ro vượt ngưỡng.
 - Can thiệp của con người (Human-in-the-Loop)**: cho phép dừng hoặc hướng dẫn tác tử trong giai đoạn học, đồng thời huấn luyện nó giảm dần nhu cầu can thiệp.

3. Khung quy trình bảo đảm an toàn

- **Đặc tả mục tiêu** $\rightarrow$ dùng ngôn ngữ *có thể kiểm định* (formal specs, ràng buộc cứng/mềm).
- **Phân tích rủi ro** $\rightarrow$ xác định kịch bản thất bại nghiêm trọng (hazard analysis).
- **Xác minh & Thẩm định (V&V)** $\rightarrow$ kết hợp chứng minh hình thức, kiểm thử mô phỏng, thử nghiệm giới hạn.
- **Giám sát sau triển khai (Monitoring)** $\rightarrow$ theo dõi chỉ số an toàn, phát hiện lệch chuẩn và cập nhật mô hình.

Trò chơi kẹp giấy. Mỗi nhánh được gắn nhãn $[p; s]$, biểu thị số lượng **kẹp giấy (paperclips)** và **ghim giấy (staples)** được sản xuất tại nhánh đó. **Harriet**, một con người, có thể lựa chọn sản xuất **hai kẹp giấy**, **hai ghim giấy**, hoặc **mỗi loại một cái**. (*Các giá trị in nghiêng màu xanh lá cây thể hiện giá trị mà Harriet nhận được nếu trò chơi kết thúc tại đó, giả định rằng $\theta=0,45$*). Sau đó, **Robbie**, một con robot, có thể chọn tạo ra **90 kẹp giấy**, **90 ghim giấy**, hoặc **50 mỗi loại**.



Hình 105. The Paperclip–Staple Game (Nguồn: Stuart J. Russell, Peter Norvig – *Artificial Intelligence: A Modern Approach*, Pearson, 2021).

An toàn AI đòi hỏi **đặc tả mục tiêu chính xác**, **cơ chế học an toàn** và **giám sát liên tục**. Các kỹ thuật như IRL, Bayesian RL, cùng quy trình V&V nhiều tầng giúp giảm thiểu rủi ro, nhưng *không có giải pháp “một lần và xong”*. Việc đặt **con người trong vòng lặp** và liên tục cập nhật hiểu biết về mục tiêu là chìa khóa để duy trì hệ thống AI an toàn trong thế giới động.

8.2.2 An ninh (Security): Chống lại các cuộc tấn công có chủ đích

Hệ thống AI không chỉ phải hoạt động chính xác mà còn phải chịu được sự thao túng cố ý từ kẻ xấu. Hai hình thức tấn công phổ biến nhất trong thực tiễn hiện nay là **tấn công đối kháng** (adversarial attacks) và **đầu độc dữ liệu** (data poisoning).

8.2.2.1 Tấn công đối kháng (Adversarial Attacks)

Định nghĩa Tấn công đối kháng là kỹ thuật thêm những nhiễu nhỏ, khó nhận biết bằng mắt thường vào mẫu đầu vào để khiến mô hình AI đưa ra dự đoán sai lệch một cách tự tin. Khái niệm này được **Szegedy et al. (2013)** giới thiệu và từ đó bùng nổ một lượng nghiên cứu đồ sộ về tính an toàn của mạng nơ-ron sâu.

Cơ chế hoạt động Các mô hình học sâu dễ dàng bị lừa bởi “những thay đổi vô hình” do chúng khai thác các hướng gradient nhạy cảm trong không gian đặc trưng. Điều này khiến việc bảo vệ hệ thống thị giác máy tính, nhận dạng giọng nói và phân tích văn bản trở nên thách thức.

Ví dụ kinh điển Một hình ảnh gấu trúc cộng một nhiễu FGSM $\epsilon = 0.007$ có thể làm mạng Inception v3 dự đoán “vượn” với độ tin cậy 99 %, dù sự khác biệt không thể nhìn thấy được. Hình thức tấn công tương tự đã được trình bày trong các nghiên cứu của Goodfellow et al. (2015).

Nguy cơ thực tế – Xe tự lái: Dán nhãn hay sticker lên biển “STOP” khiến hệ thống nhận dạng nhầm thành “Giới hạn tốc độ 80 km/h”. – Kiểm soát truy cập: Sửa pixel trên ảnh để qua mặt hệ thống nhận dạng khuôn mặt.

Biện pháp giảm thiểu

- **Adversarial Training** – bổ sung mẫu nhiễu vào tập huấn luyện, được đề xuất trong mục “adversarial training” của Russell & Norvig.
- **Defensive Distillation, Randomized Smoothing** – làm mờ ranh giới quyết định.
- **Phát hiện nhiễu (Detection)** – phân tích độ lớn gradient, khoảng cách Mahalanobis.

8.2.2.2 Đầu độc dữ liệu (Data Poisoning)

Định nghĩa Kẻ tấn công chèn mẫu độc hại vào tập huấn luyện hoặc dòng dữ liệu đang học trực tuyến để đánh lừa quá trình tối ưu, làm mô hình học thói quen sai. Russell & Norvig cảnh báo rằng trong các hệ thống khuyến nghị có thể xuất hiện “những người dùng đối kháng, cố tình thao túng hệ thống”.

Cách thức

- **Label-flipping**: gán nhãn sai có chủ đích.
- **Backdoor Attack**: chèn mẫu kích hoạt (triggers) vào tập huấn luyện; khi xuất hiện trigger, mô hình ra kết quả mong muốn của kẻ tấn công.
- **Gradient Hijacking**: chèn một tỷ lệ nhỏ mẫu dữ liệu làm sai hướng gradient.

Hậu quả – Mô hình ra quyết định thiên vị hoặc mở cửa cho hành vi gian lận.
– Trong vận hành liên tục (online learning), hệ thống có thể bị chiếm quyền điều khiển vĩnh viễn.

Biện pháp giảm thiểu

Chiến lược	Mô tả ngắn
Sanitization	Lọc mẫu ngoại lai bằng thống kê hoặc mô hình phát hiện dị thường.
Robust Loss	Sử dụng hàm mất hạn chế ảnh hưởng của mẫu ngoại lai.
Differential Privacy	Giới hạn đóng góp của mọi mẫu, giảm tác động của dữ liệu độc hại.

8.2.2.3 So sánh và Kết nối

Khía cạnh	Tấn công đối kháng	Đầu độc dữ liệu
Thời điểm	Sau huấn luyện (inference-time)	Trong huấn luyện (training-time)
Mục tiêu	Thay đổi dự đoán một mẫu cụ thể	Làm trệch toàn bộ mô hình
Khó phát hiện	Cao (do nhiễu nhỏ)	Trung bình – phụ thuộc vào khả năng giám sát dữ liệu
Chiến lược phòng thủ	Adversarial training, phát hiện nhiễu	Sanitization, robust loss, phân tích nguồn dữ liệu

Tài liệu Tham khảo

- Russell, S. J., & Norvig, P. *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson, 2021.
– Mục “adversarial example, adversarial training” trong Index.
– Bàn luận về nguy cơ người dùng đối kháng trong hệ thống khuyến nghị.
- Goodfellow, I. et al., “Explaining and harnessing adversarial examples”, ICLR 2015.

Các kỹ thuật và biện pháp trên giúp sinh viên hiểu rằng một mô hình AI an toàn không chỉ cần chính xác, mà còn phải vững vàng trước những kịch bản tấn công có chủ đích—một yếu tố thiết yếu của AI có trách nhiệm trong kỷ nguyên số.

Kết luận và Định hướng dự án

Trong bối cảnh AI ngày càng can dự sâu vào đời sống, “AI có trách nhiệm” không phải là một tùy chọn sang chảnh mà là **nền tảng tối thiểu** để các hệ thống thông minh được xã hội chấp nhận. Các khung đạo đức hiện đại đều yêu cầu nhà phát triển song hành bốn trụ cột: **công bằng, tin cậy, phúc lợi xã hội và an toàn**.

Từ góc nhìn học thuật, yêu cầu này được củng cố bởi các cập nhật giáo trình gần đây: hầu hết các chương sách đều bổ sung mục “Tác động xã hội”, kêu gọi sinh viên “hiểu rõ tác động tiềm tàng của công trình nghiên cứu đối với cộng đồng”. Đối với kỹ sư thực tiễn, hậu quả của **ra**

quyết định sai lệch đã hiện hữu—từ đánh giá tín dụng cho tới hệ thống tư pháp—khi mô hình củng cố các thiên kiến lịch sử.

Do vậy, mọi dự án AI cần được thiết kế như một **chu trình đạo đức-kỹ thuật khép kín** gồm:

- **Phân tích rủi ro đạo đức** (thiên vị, phân biệt, thiếu minh bạch) trước khi huấn luyện;
- **Kiểm định an toàn & an ninh** (đặc tả mục tiêu, tấn công đối kháng) trong giai đoạn triển khai;
- **Giám sát vận hành liên tục** nhằm phát hiện và giảm thiểu tác hại phát sinh sau khi hệ thống ra đời.

Câu hỏi gợi mở cho dự án cuối kỳ

- **Đo lường thiên vị dữ liệu**
 - Chỉ số/kỹ thuật nào (ví dụ: phân bố nhân khẩu, *disparate impact*, *statistical parity*) phù hợp với bộ dữ liệu của bạn?
 - Bảng số liệu, hãy chứng minh mức độ chênh lệch giữa các nhóm.
- **Phân tích sai số dương tính giả (FP) & âm tính giả (FN)**
 - Dựa trên ngữ cảnh bài toán, hậu quả xã hội của từng loại sai số là gì (chi phí cơ hội, rủi ro an toàn, tổn hại uy tín)?
 - Nêu rõ vì sao việc cân bằng FP–FN không chỉ là vấn đề “điểm F1” mà còn là vấn đề đạo đức.
- **Khả năng bị lạm dụng**
 - Nếu hệ thống rơi vào tay tác nhân xấu, họ có thể khai thác như thế nào?
 - Đề xuất ít nhất *hai* biện pháp giảm thiểu (ràng buộc truy cập, kiểm định đầu vào, theo dõi dị thường...).
- **Chiến lược minh bạch & diễn giải**
 - Bạn sẽ cung cấp thông tin gì (mô tả mô hình, nguồn dữ liệu, phân bố lỗi) cho người ảnh hưởng trực tiếp bởi quyết định của hệ thống?
 - Kỹ thuật XAI nào (SHAP, LIME, đối sánh nhân quả...) phù hợp với độ phức tạp mô hình?
- **Kế hoạch giám sát sau triển khai**
 - Xây dựng KPI đạo đức: tỷ lệ thiên vị, tỷ lệ lỗi an toàn, số sự cố an ninh.
 - Chu kỳ đánh giá lại (ví dụ: hằng quý) và quy trình thu hồi/làm mới mô hình khi vượt ngưỡng rủi ro.

Trách nhiệm không dừng ở việc “giao nộp mô hình”, mà kéo dài suốt *vòng đời* sản phẩm. Một AI đáng tin cậy là AI luôn được đặt dưới lăng kính phê phán liên tục của chính những người tạo ra nó.

Gợi ý dự án cuối kỳ

Các đề xuất dưới đây nhằm giúp sinh viên **vận dụng toàn bộ kiến thức đã học** — từ tìm kiếm, biểu diễn tri thức, học máy đến tối ưu hoá tiến hoá — để xây dựng một hệ thống AI hoàn chỉnh. Mỗi định hướng được trình bày theo cấu trúc:

- **Mục tiêu học tập**
- **Kiến thức nền tảng** (nhắc lại các khái niệm/chương liên quan)
- **Quy trình thực hiện gợi ý** (các mốc kỹ thuật & quản lý dự án)
- **Ví dụ/đoạn mã trích nguyên bản** (minh hoạ ý tưởng)
- **Báo cáo & tiêu chí đánh giá**

1. Game AI: Minimax + Cắt tỉa Alpha-Beta cho trò chơi bàn cờ nâng cao (Caro, Cờ Gánh...)

Mục tiêu

- Hiểu sâu tìm kiếm đối kháng, hàm lượng giá & cắt tỉa.
- Cài đặt AI có thể chơi ở cấp độ “khó” trong thời gian đáp ứng thời gian thực.

Kiến thức nền

- *Chương 2 – Tìm kiếm theo chiều sâu trạng thái* (AIMA).
- Khái niệm α (Alpha) & β (Beta) cùng điều kiện cắt tỉa.

Quy trình gợi ý

- **Mô hình hoá trò chơi**: xác định tập luật, trạng thái kết thúc, hàm tiện ích.
- **Cài đặt Minimax thuần** → kiểm thử độ sâu nhỏ.
- **Tích hợp Alpha-Beta** (NHỚ duy trì thứ tự động tác tốt trước) để giảm nhanh nhánh tìm kiếm.
- **Hiệu chỉnh hàm lượng giá theo đặc trưng miền** (ô trung tâm, thế “hai nước mở”...).
- **Xây giao diện** (CLI hoặc web) để người dùng so tài với AI.

Ví dụ minh hoạ (trích nguyên bản)

α (Alpha): Giá trị tốt nhất (cao nhất) mà MAX đã tìm thấy cho đến nay trên đường đi hiện tại

β (Beta): Giá trị tốt nhất (thấp nhất) mà MIN đã tìm thấy cho đến nay trên đường đi hiện tại.

Cắt tại nút MIN: Nếu giá trị của một nhánh $\leq \alpha$, dừng xét nhánh đó.

Cắt tại nút MAX: Nếu giá trị của một nhánh $\geq \beta$, dừng xét nhánh đó.

(Nguồn: outline_Giáo trình Trí tuệ nhân tạo)

Báo cáo & chấm điểm

- 30 % Thiết kế & phân tích độ phức tạp; 40 % chất lượng chơi (tỷ lệ thắng so với mẫu tham chiếu); 20 % giao diện – UX; 10 % tài liệu & video demo.

2. Hệ chuyên gia tư vấn (cấu hình PC, dinh dưỡng...)

Mục tiêu

- Áp dụng **biểu diễn tri thức & suy diễn tiến/lùi** để đưa ra khuyến nghị.
- Thực hành quy trình *knowledge-engineering*.

Kiến thức nền

- Chu trình kỹ thuật tri thức gồm **mã hoá – kiểm thử – tinh chỉnh – thẩm định**.
- Suy diễn trên luật If-Then (Ch. 7 & 9, AIMA).

Quy trình gợi ý

- **Thu thập chuyên gia lĩnh vực** → tạo bảng luật.
- **Chọn cơ chế suy diễn** (forward hoặc backward chaining).
- **Xây dựng mô-đun giải thích (Justification)** → **đáp ứng yêu cầu minh bạch**.
- **Thiết kế giao diện web đơn giản** (form nhập + phần gợi ý).

Ví dụ minh hoạ

$\forall x \text{ WillWait}(x) \Leftrightarrow \text{Patrons}(x, \text{Some})$	(giả thuyết h3)
--	-----------------

Ví dụ h3 được tạo ra sau bước **tổng quát hoá** trong thuật toán Current-Best-Learning để giảm false negative.

(Nguồn: Stuart J. Russell, Peter Norvig – *Artificial Intelligence: A Modern Approach*).

Báo cáo & chấm điểm

- 25 % độ bao phủ luật; 35 % độ chính xác khuyến nghị (đối chiếu chuyên gia); 20 % khả năng giải thích; 20 % UX.

3. Dự đoán với Học máy (Kaggle dataset)

Mục tiêu

- Thực thi pipeline **thu thập** → **tiền xử lý** → **huấn luyện** → **đánh giá** → **triển khai**.
- Nhân mạnh đo lường **False Positive/False Negative** & phân tích rủi ro.

Kiến thức nền

- Khái niệm *false positive/false negative* & **ROC/AUC** + *confusion matrix*.
- Các phép đo RMSE, MAE (MML-book, mục 298).

Quy trình gợi ý

- **Lựa chọn bộ dữ liệu** (khách hàng rời mạng, chuẩn đoán bệnh,...).
- **EDA & xử lý thiếu, biến ảo**.
- **Huấn luyện mô hình cơ sở** → tinh chỉnh (grid / random search) đồng thời **tránh overfitting** (chọn nhiều bộ validation).
- **Đánh giá**: confusion matrix, ROC & AUC; phân tích tác động FP/FN lên nghiệp vụ.
- **Đóng gói mô hình** (pickle/ONNX) & viết notebook trình bày.

Ví dụ minh họa

“Another helpful visualization tool for classification problems is a confusion matrix: a two-dimensional table of counts of how often each category is classified or misclassified as each other category.”

(Nguồn: Stuart J. Russell, Peter Norvig – AIMA)

Báo cáo & chấm điểm

- 40 % chất lượng mô hình (chỉ số chọn phù hợp); 20 % phân tích lỗi & rủi ro; 20 % tài liệu reproducible; 20 % trình bày.

4. Chatbot đơn giản (Rule-Based hoặc ML nhẹ)

Mục tiêu

- Vận dụng **xử lý ngôn ngữ tự nhiên cơ bản** + **hệ thống luật** để trả lời trong một miền hẹp (FAQ thư viện, hỗ trợ kỹ thuật...).
- Thực hành thiết kế kịch bản hội thoại & đánh giá.

Kiến thức nền

- *Logical Agents* (Ch. 7) & khái niệm **production rules**.
- Đo lường **độ chính xác câu trả lời** tương tự cách đánh giá FP/FN (mục 3).

Quy trình gợi ý

- **Xác định miền & intent.**
- **Xây KB luật “pattern → response”** (hoặc TF-IDF + cosine).
- **Triển khai API (Flask/FastAPI)**; có log hội thoại để cải tiến.
- **Đánh giá:** precision / recall trên bộ câu hỏi chuẩn.

Báo cáo & chấm điểm

- 30 % độ chính xác; 25 % bao phủ intent; 25 % UX (dễ dùng, phản hồi nhanh); 20 % báo cáo cải tiến.

5. Ứng dụng Thuật toán Di truyền (GA) cho tối ưu hóa

Mục tiêu

- Hiểu cơ chế tiến hoá (khởi tạo – chọn lọc – lai ghép – đột biến – elitism).
- Áp dụng GA giải bài toán tối ưu (như định tuyến giao hàng, xếp lịch, Ba lô 0/1).

Kiến thức nền

- Khôi cấu thành GA & tham số cốt lõi.
- Chu trình GA & ví dụ Ba lô trong giáo trình.

Quy trình gợi ý

- **Mã hoá cá thể** (chromosome) & **hàm fitness**.
- **Cấu hình tham số:** population, crossover-rate, mutation-rate.
- **Triển khai vòng lặp tiến hoá & đồ hoạ đường hội tụ.**
- **So sánh** với thuật toán tham chiểu (greedy, dynamic programming).

Ví dụ minh hoạ (trích nguyên bản)

```
POPULATION_SIZE = 50
NUM_GENERATIONS = 100
MUTATION_RATE = 0.01
...
offspring = crossover(parent1, parent2)
mutated = mutation(offspring, MUTATION_RATE)
```

(Nguồn: outline_Giáo trình Trí tuệ nhân tạo)

Báo cáo & chấm điểm

- 35 % chất lượng lời giải (so với tối ưu hoặc benchmark); 30 % phân tích hội tụ; 15 % trực quan hoá; 20 % tài liệu.

Hướng dẫn chung

Hạng mục	Mốc kiểm tra	Gợi ý minh chứng
Đề xuất & Lập kế hoạch	Tuần 1 – 2	Lập bảng WBS, biểu đồ Gantt.
Báo cáo tiến độ	Giữa kỳ	Nhật ký Git + sprint review.
Bảo vệ cuối kỳ	Tuần 15 – 16	Demo trực tiếp + slide 15 phút + báo cáo 10 trang.

Lưu ý: mọi dự án phải kèm **nhật ký đánh giá rủi ro đạo đức** (thiên vị dữ liệu, khả năng lạm dụng) và mô tả biện pháp giảm thiểu, liên kết với nội dung **Responsible AI** đã học ở Chương 8.